DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

Introduction to Generative Adversarial Networks with PyTorch

From the GAN game to image-to-image and video-to-video synthesis

26
SECTIONS

364
FIGURES

323
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Course Agenda 4
2 PyTorch Forward and Backward Propagation 47
3 PyTorch Autograd Mechanism 52
4 PyTorch Custom Loss Function 60
LECTURE 5 Introduction to GANs 65
LECTURE 6 Working of GAN Loss Function 74
LECTURE 7 Number of Pieces and Fully Connected 85
Implementing GAN Training Methodology 85
LECTURE 8 Implement Vanilla GAN on MNIST Dataset to Generate Digits 89
LECTURE 9 [Coding Exercise] GAN Evaluation Metrics Inception Score 133
LECTURE 10 [Coding Exercise] GAN Evaluation Metrics FID Score 140
LECTURE 11 Introduction to Conditional GANs 146
LECTURE 12 Implement Conditional GAN on MNIST Dataset 155
Implement Conditional GAN on MNIST Dataset 155
LECTURE 13 Working of Wasserstein Loss Function 163
LECTURE 14 Partial Derivative and Earth Mover's Distance 178
Implement Wasserstein Loss Function 178
LECTURE 15 Gradient Penalty 186
[Coding Exercise] Gradient Penalty Wasserstein GAN - GP-WGAN 186
LECTURE 16 Introduction to DC-GANs 192
LECTURE 17 Implement DC-GAN on UC Birds Dataset 209
Implement DC-GAN on UC Birds Dataset 209
LECTURE 18 Working of Multi-way Loss Function 215
LECTURE 19 Coding Challenge 228
Implement multi-way loss with Auxiliary-GAN on UC Birds Dataset 228
LECTURE 20 Introduction to Progressive GANs 233
LECTURE 21 Conjoined Triangles of Artificial Intelligence 242
Implement Progressive GANs on Celebs Dataset 242
LECTURE 22 Hints, Tips, and Tricks for GAN Training 253
LECTURE 23 Introduction to U-NET Architecture 264
LECTURE 24 Working of Pix2Pix GAN and CycleGAN 275

LECTURE 25 Upsampling Path and Downsampling Path 295
[Coding Exercise] Hands-on Pix2Pix GAN 295
LECTURE 26 Target Domain and Cyclic Loss 297
[Coding Exercise] Hands-on CycleGAN 297
LECTURE 27 Working of Vid2Vid GAN 301
LECTURE 28 Reading Assignment 311
Diving Deeper into Vid2Vid GAN using YouTube Dance Video Dataset 311
LECTURE 29 Conclusion, Next Steps, and Future Directions 321

LECTURE 1

Course Agenda

Hello and welcome to a new course on generative adversarial networks. GANs with PyTorch. This is your host Mustafa Comer Dean. My name is Mustafa and I'm a top rated machine learning engineer and I have been in the software development industry for over 10 years. I've been working with many startups on solving challenges in e-commerce.

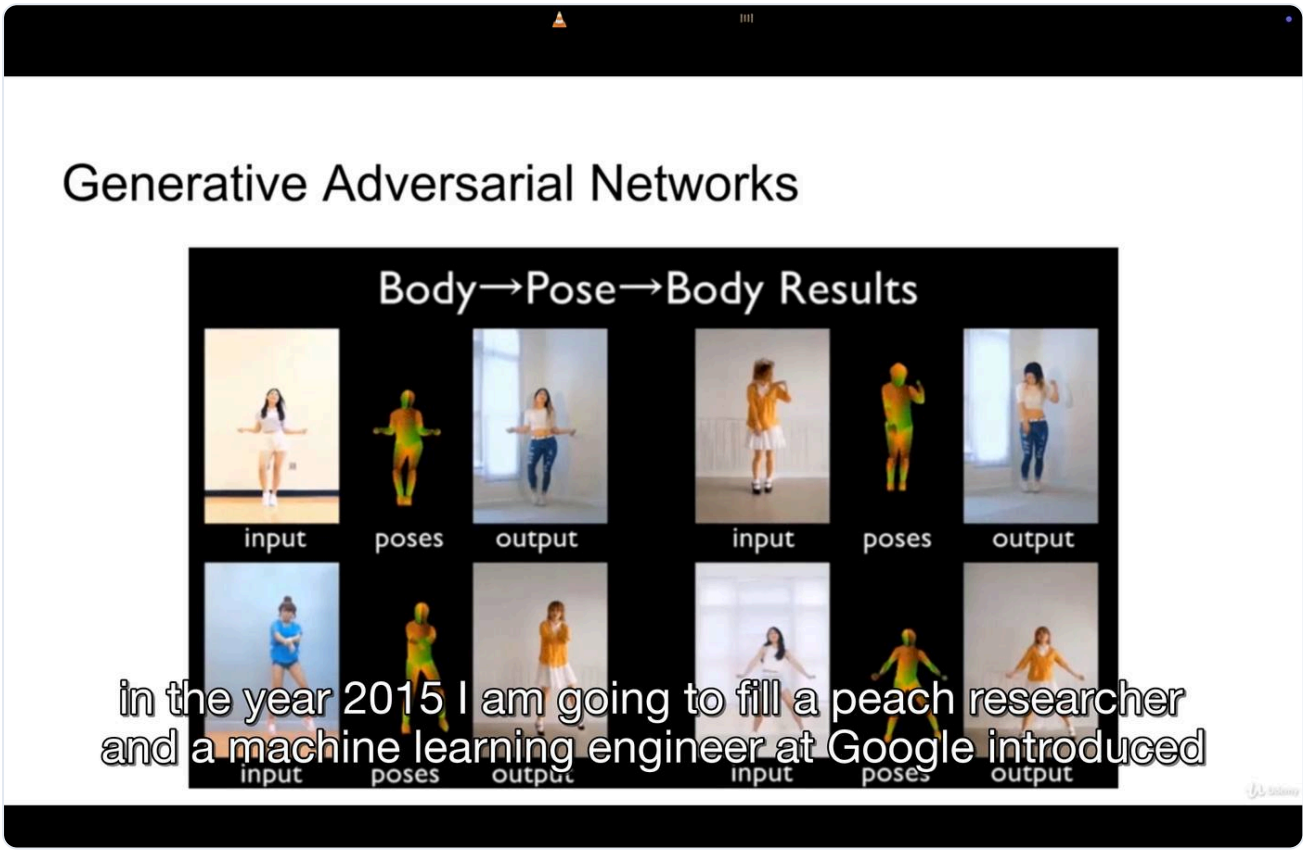


FIGURE 1.1 | Generative Adversarial Networks

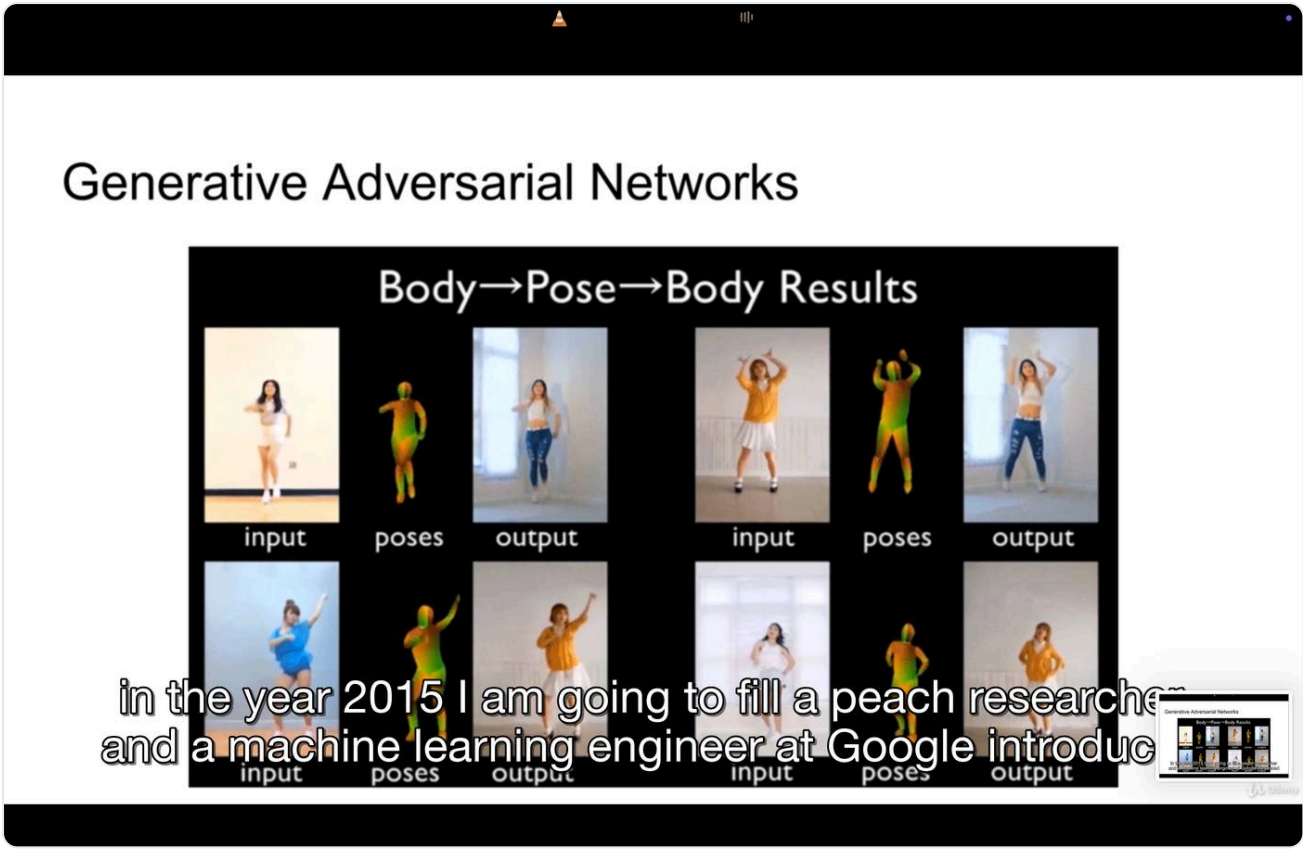


FIGURE 1.2 | Generative Adversarial Networks

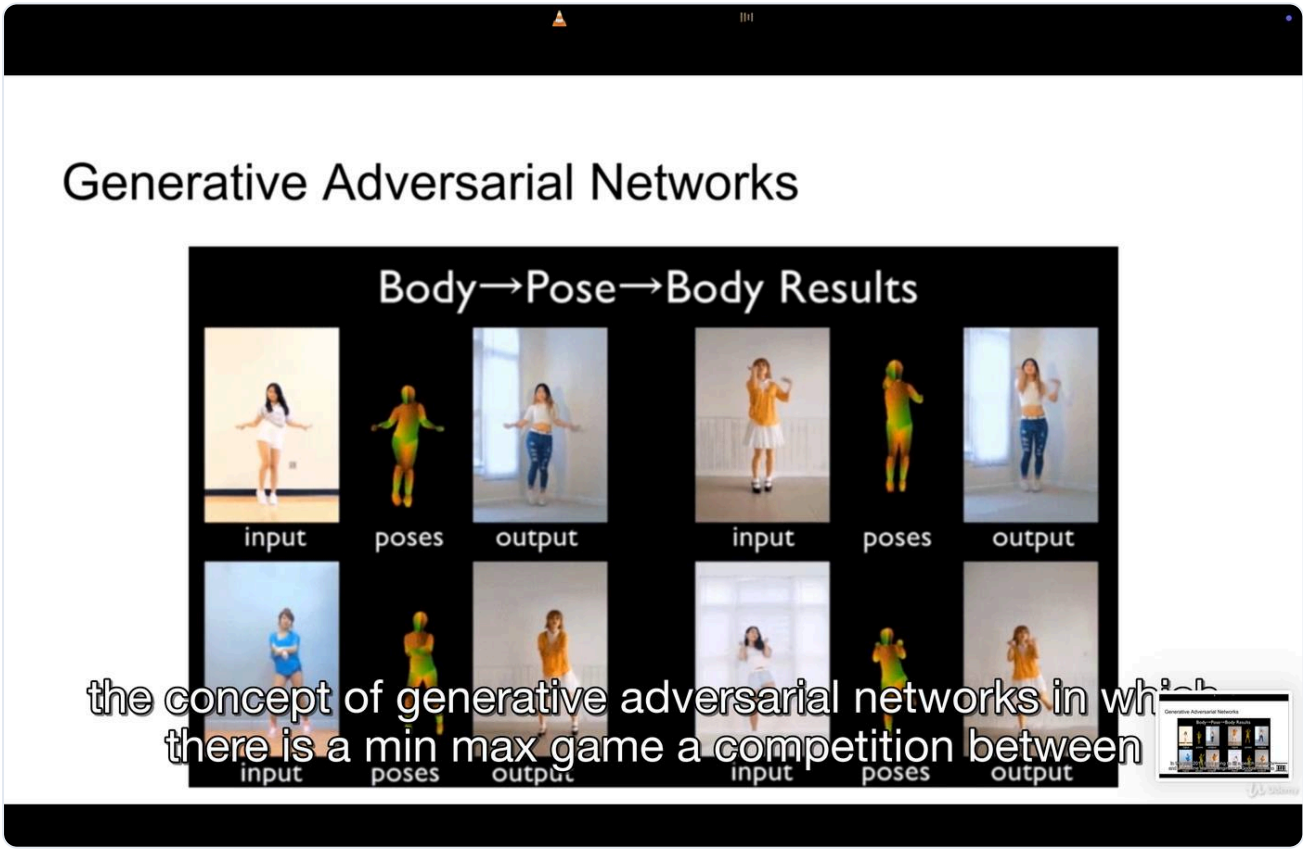


FIGURE 1.3

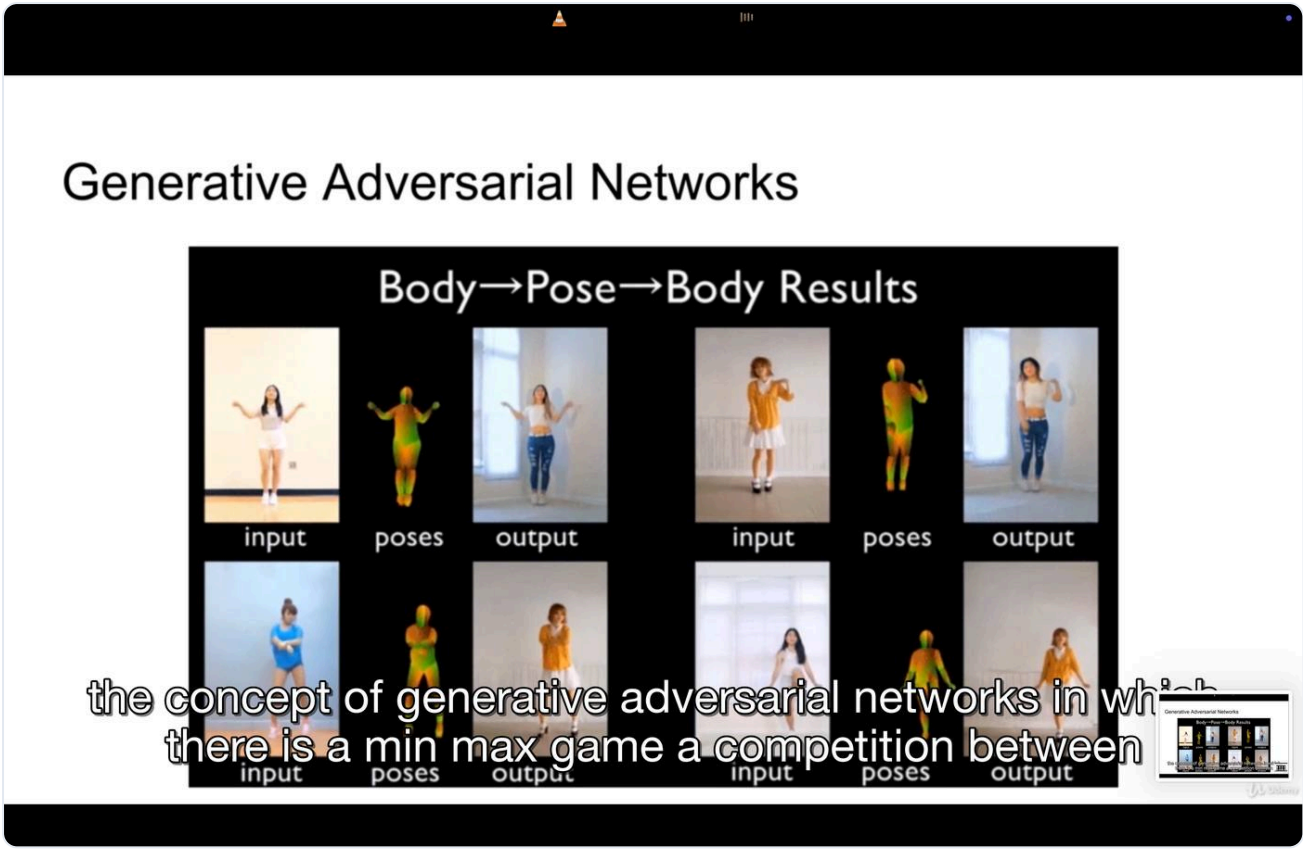


FIGURE 1.4

Computer Vision recommender systems and financial trading in the year 2015 Ian Goodfellow, a PhD researcher and a machine learning engineer at Google introduced the concept of generative adversarial networks in which there is a min-max game a competition between generator model and the discriminator model.

The generator is trying to generate images that are very close to real images and the discriminator is trying to find out whether they are real or fake. The idea may seem intuitive but it has a very dramatic effect in the area of deep learning and machine learning and it has introduced and opened the door wide to a lot of applications that go beyond imagination. For example we can see here a video two sequences generated by a generative adversarial network in which it learns the dance the poses, the motion from an input sequence and uses it to generate a completely new random sequence for a different person that may not even exist in real life.

This is only one application of generative adversarial networks. A lot of applications exist in entertainment in games in graphics augmented and virtual reality.

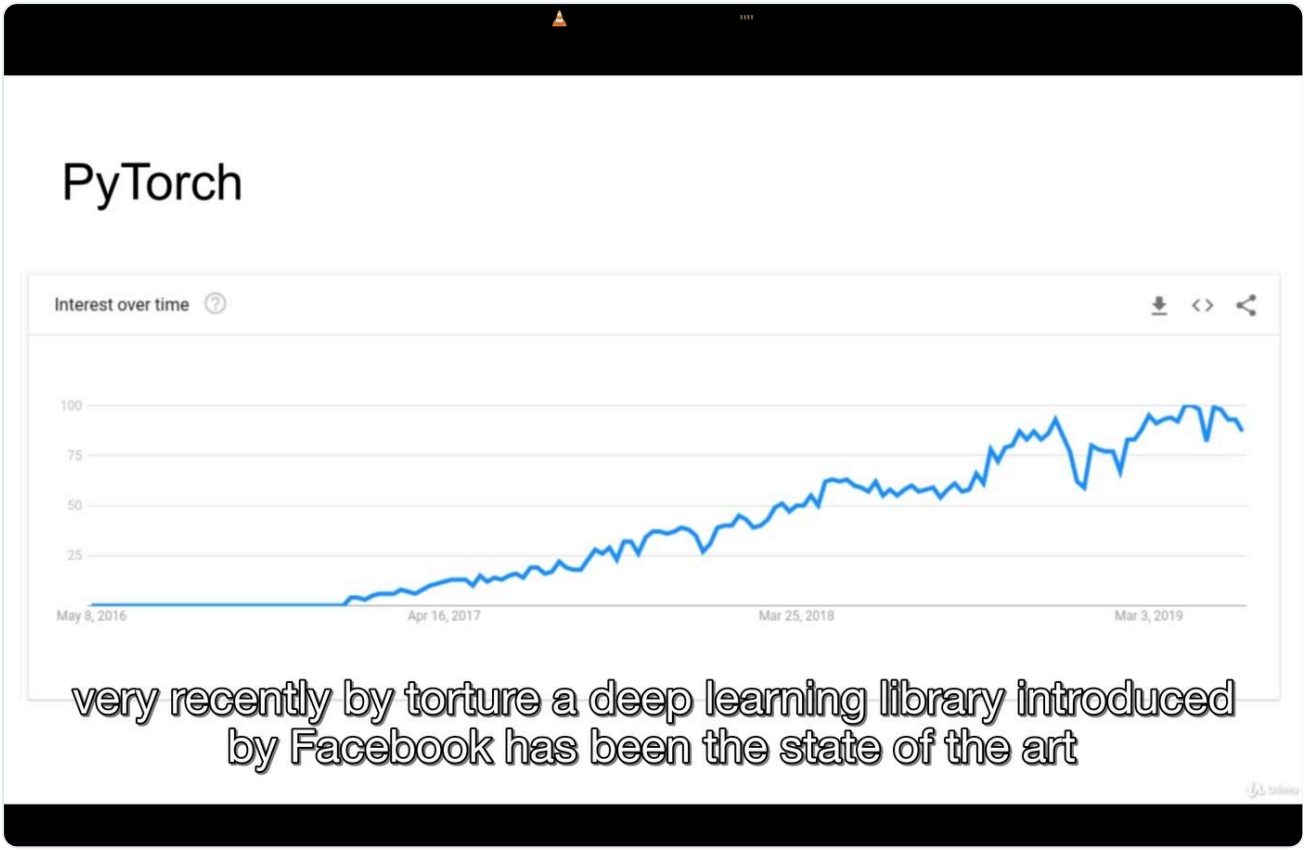


FIGURE 1.5

And even for generating synthetic data sets for medical imaging and health care very recently PyTorch a deep learning library introduced by Facebook has been the state of the art and the favorite tool for most machine learning and deep learning researchers and implementers who want to quickly prototype their ideas without worrying too much about syntax issues.

PyTorch gives you the flexibility to customize every single aspect of the neural network training starting with the Forward and backward path the gradient the propagation and how the loss function is calculated and optimized.

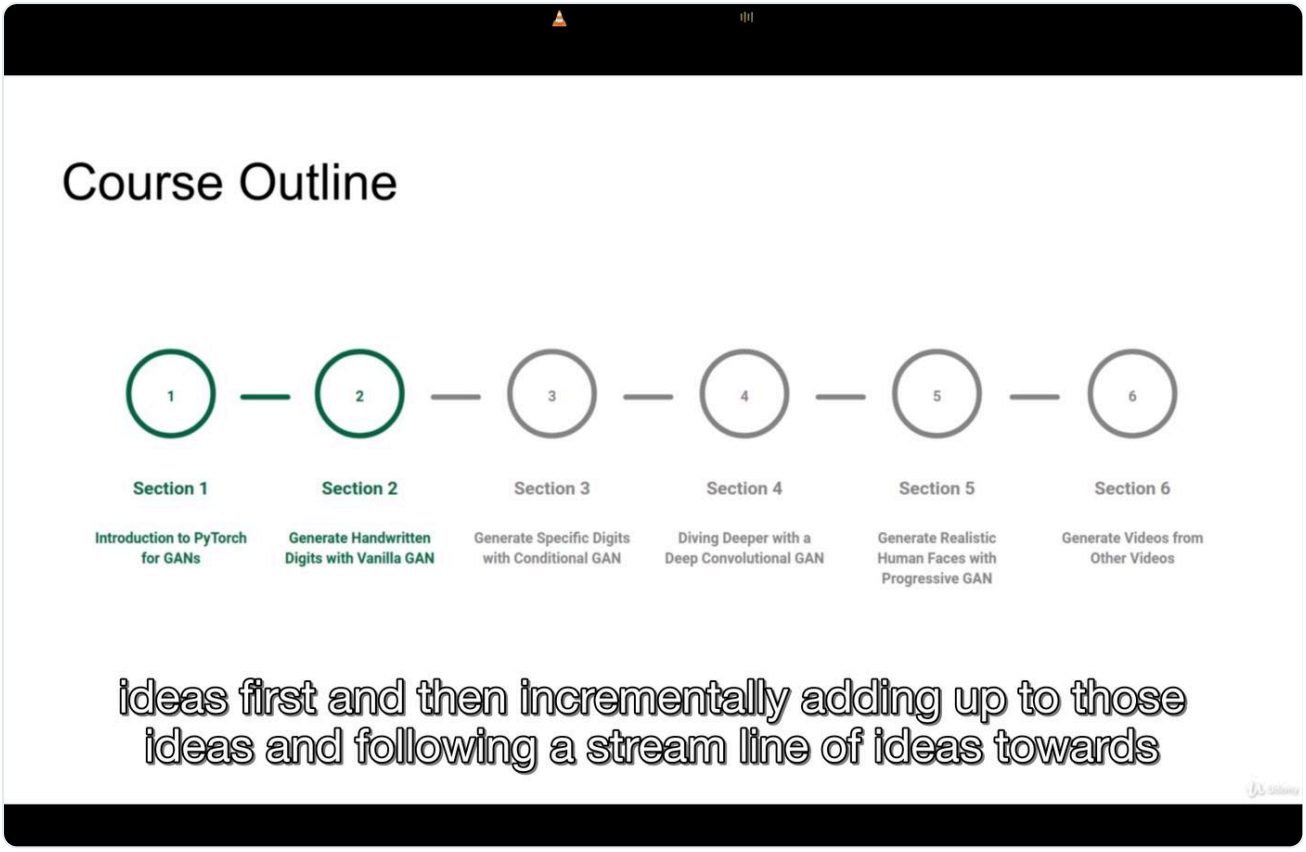


FIGURE 1.6 | Introduction to PyTorch Generate Handwritten

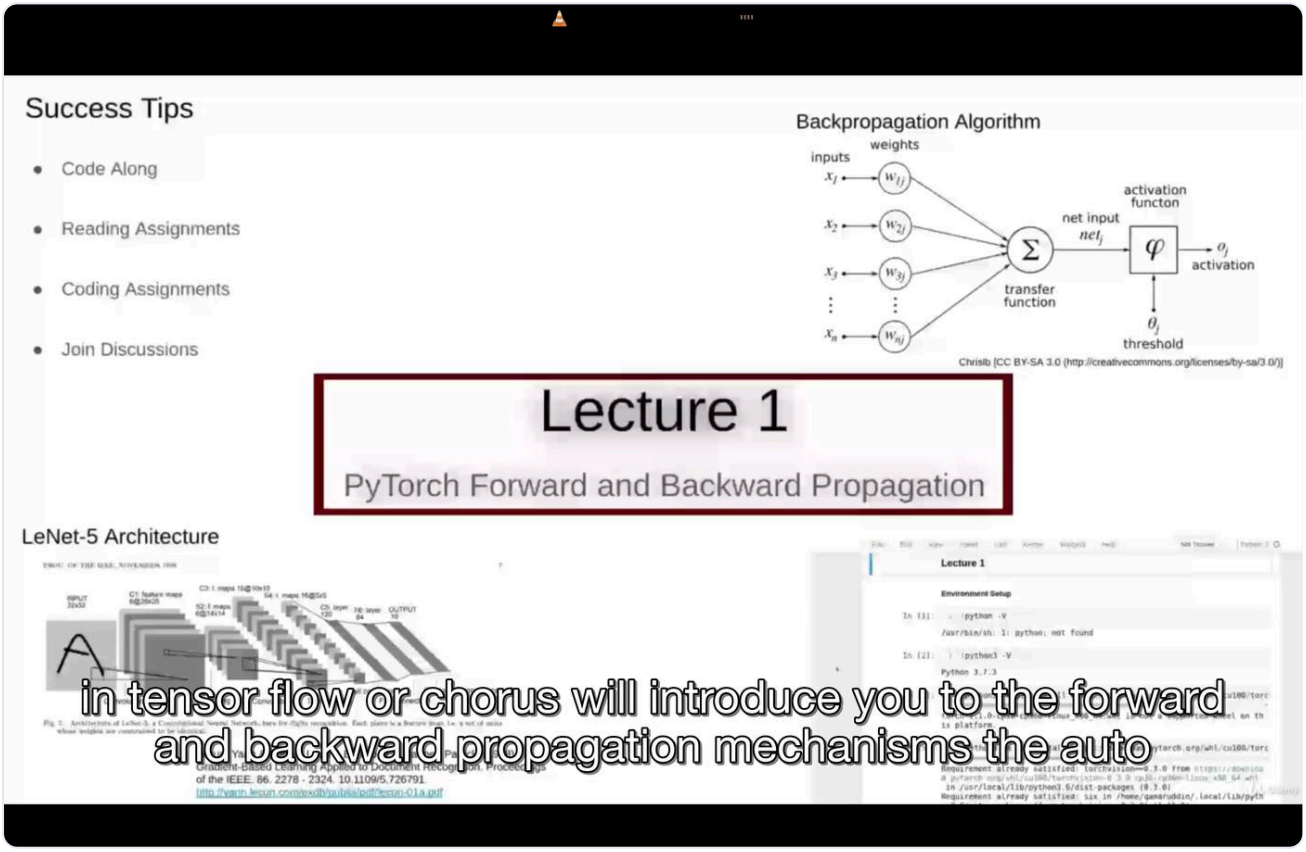


FIGURE 1.7

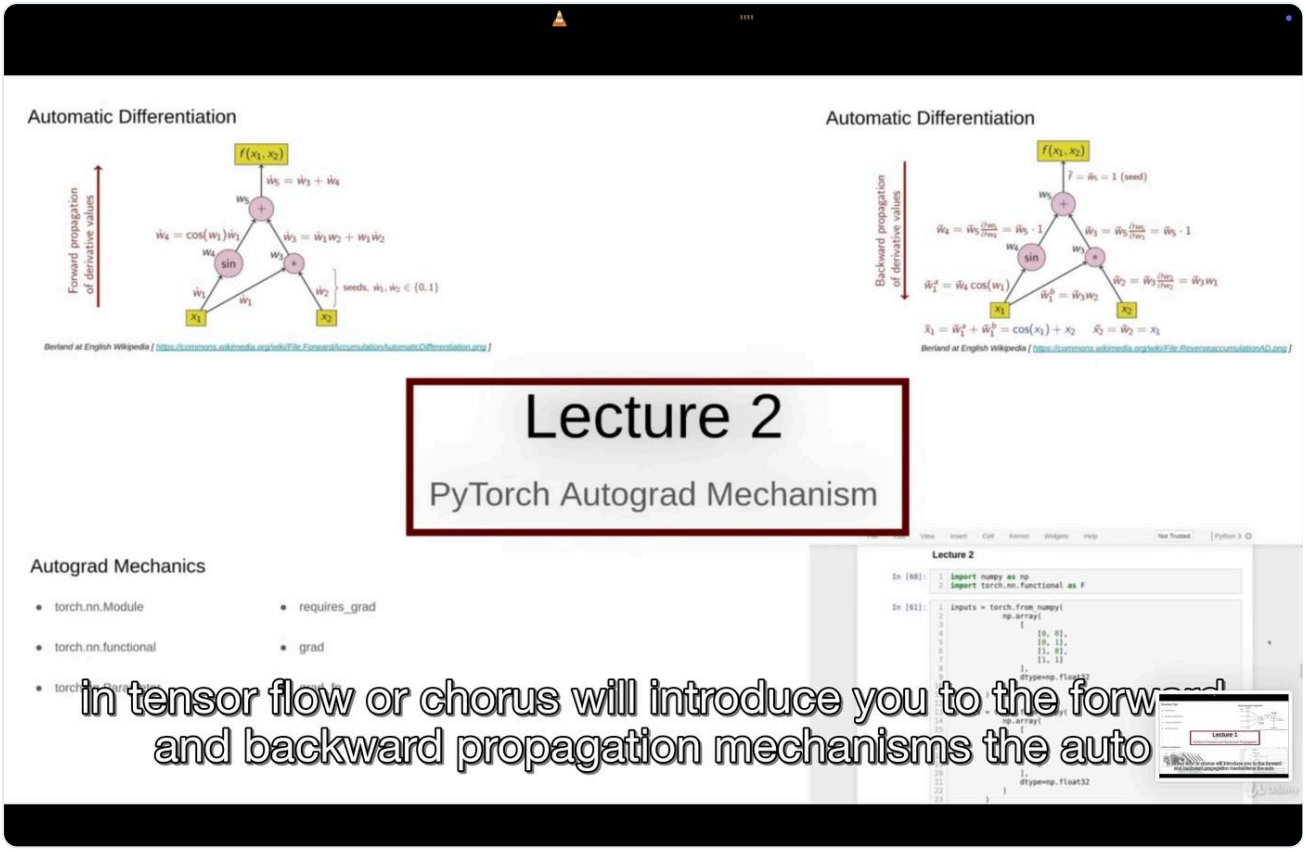


FIGURE 1.8 | Automatic Differentiation Automatic Differentiation

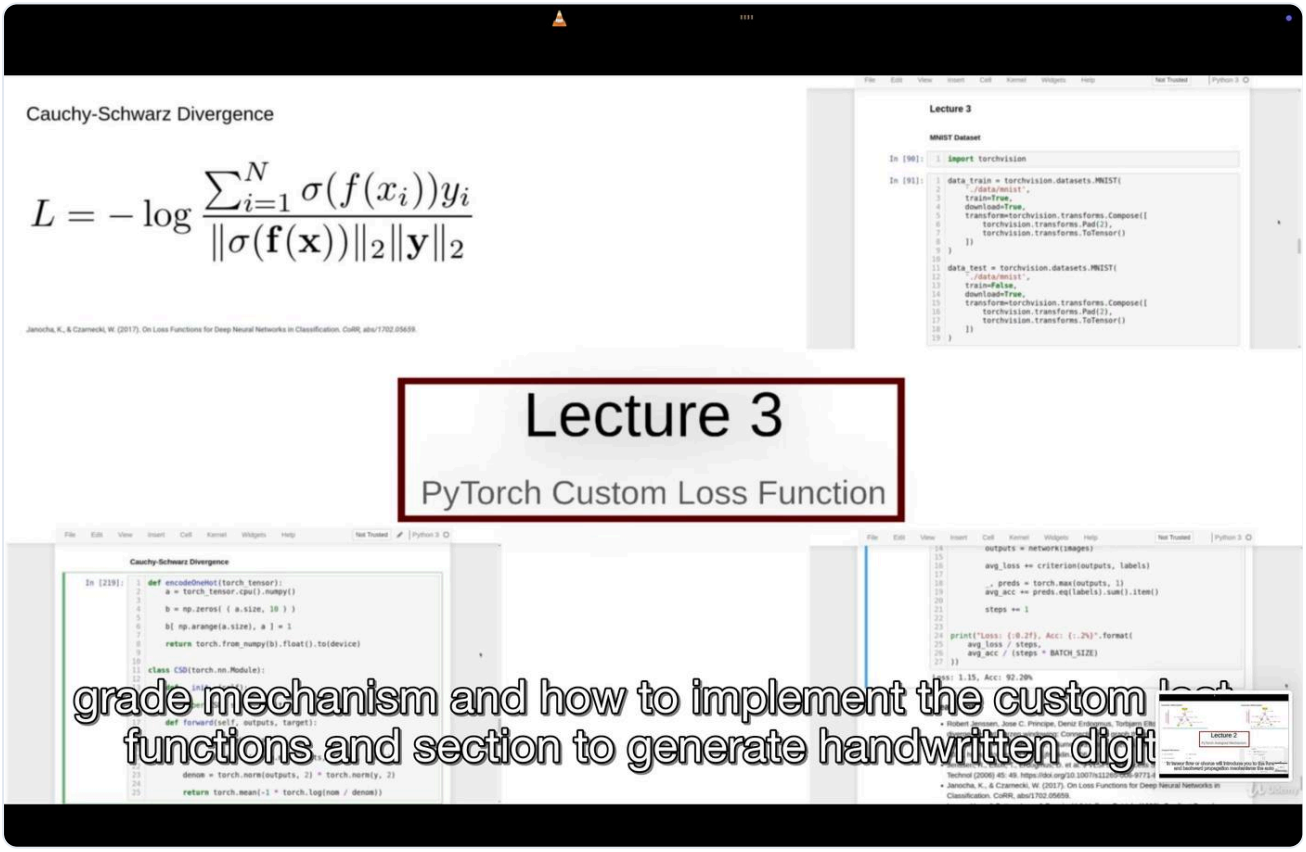


FIGURE 1.9

This is our all required features for implementing and generative adversarial networks GANs and if you come through you will find it the most widely adopted Deep Learning library tool for implementing GANs in almost all of the research papers available on GitHub and on arXiv and this course will learn the most commonly used the modules and ideas for GANs and do it have the right balance between theory and implementation will take you from zero to hero by introducing the simple ideas first and then incrementally adding up to those ideas and following a stream line of ideas towards the most advanced and complex implementations of GANs in Section 1 Introduction to PyTorch for GANs we will learn the basics of PyTorch so whether you have used it before or you are coming from a background in TensorFlow or Keras will introduce you to the forward and backward propagation mechanisms the autograd mechanism and how to implement the custom loss functions and section two generate handwritten digits with vanilla GANs.

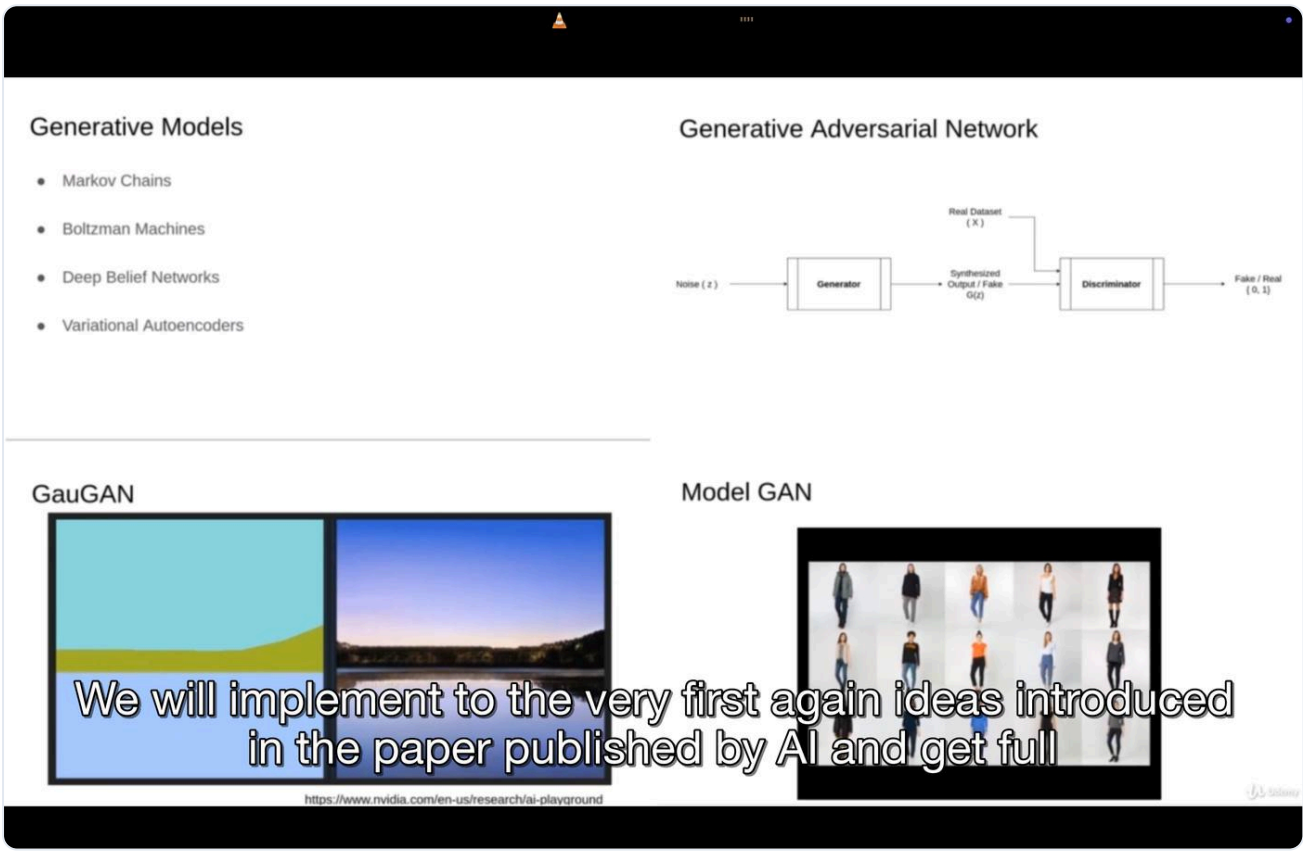


FIGURE 1.10 | Generative Models Generative Adversarial Network

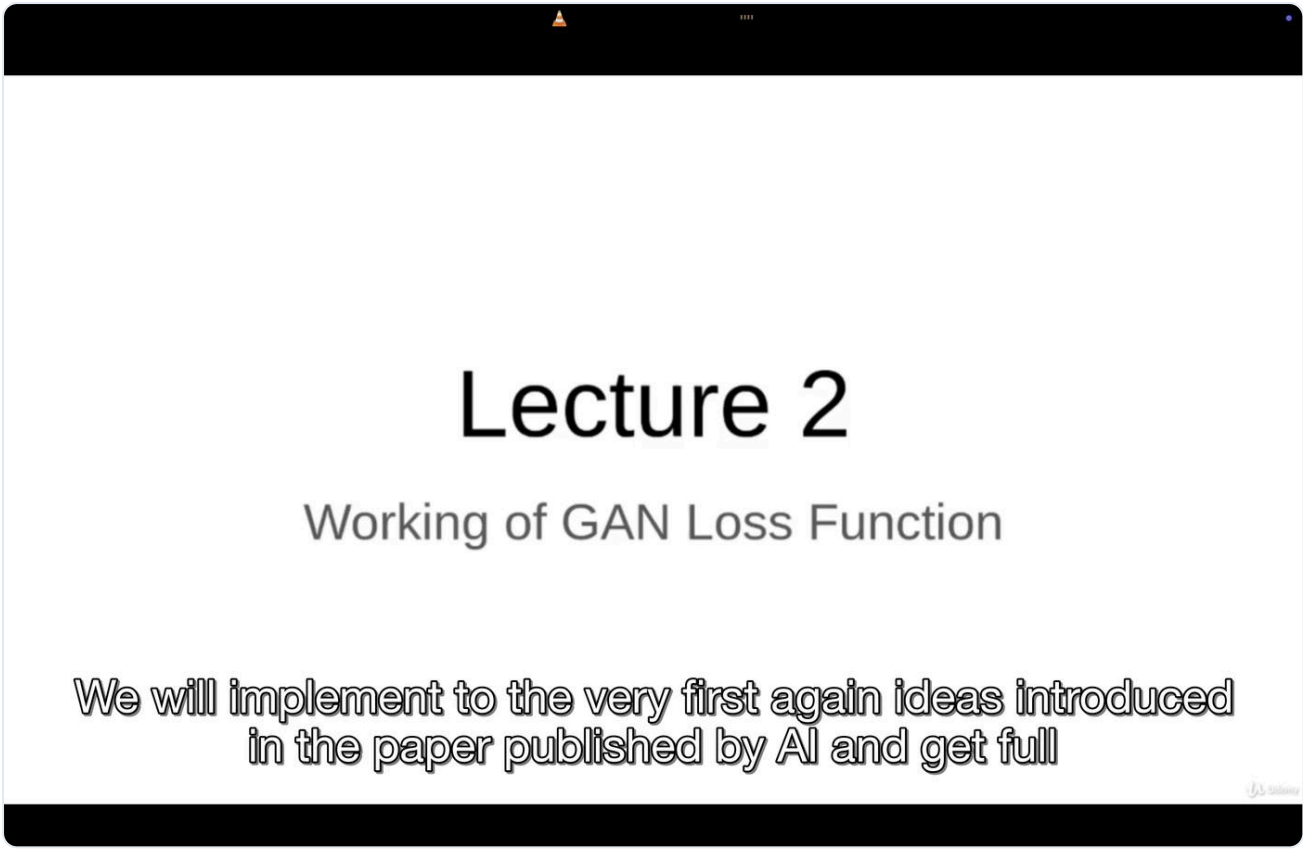


FIGURE 1.11 | Working of GAN Loss Function

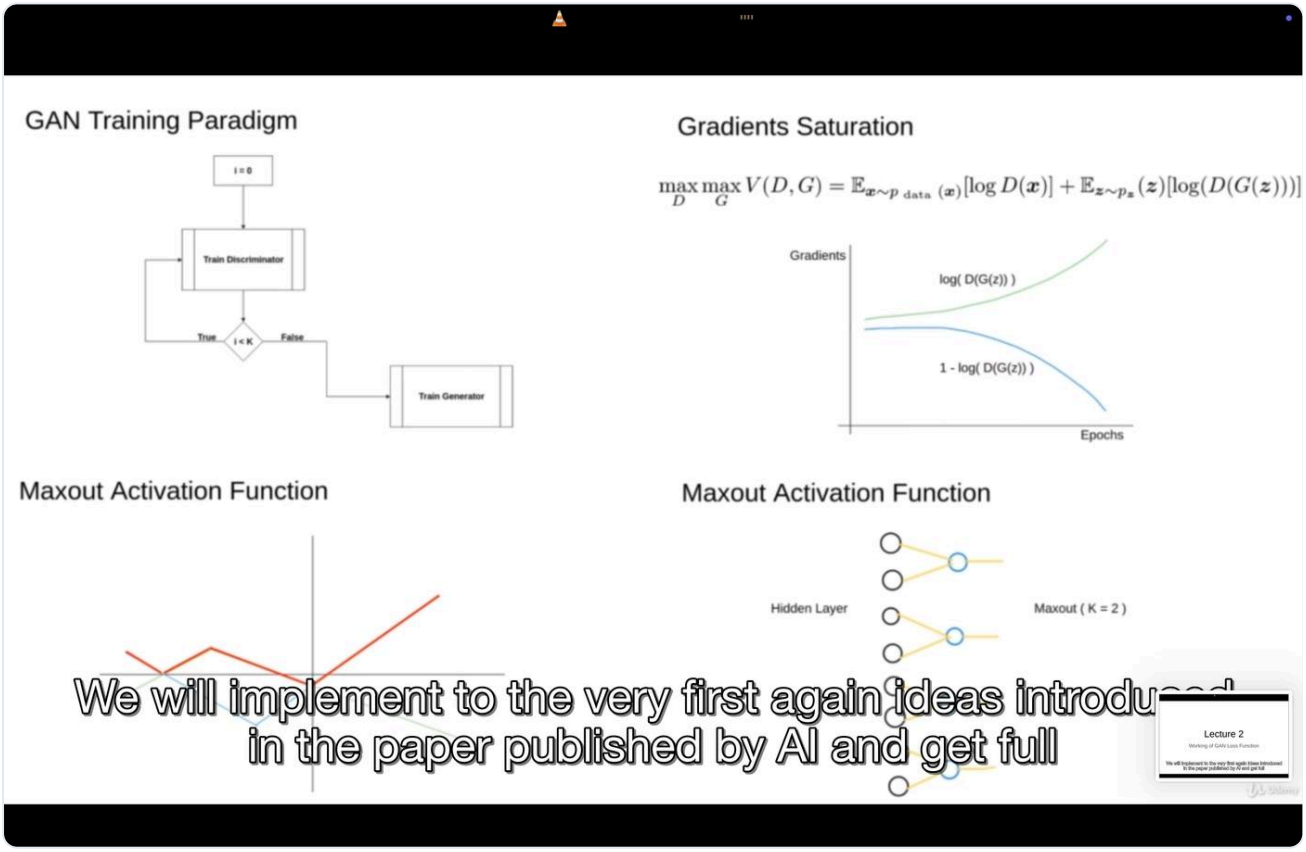


FIGURE 1.12

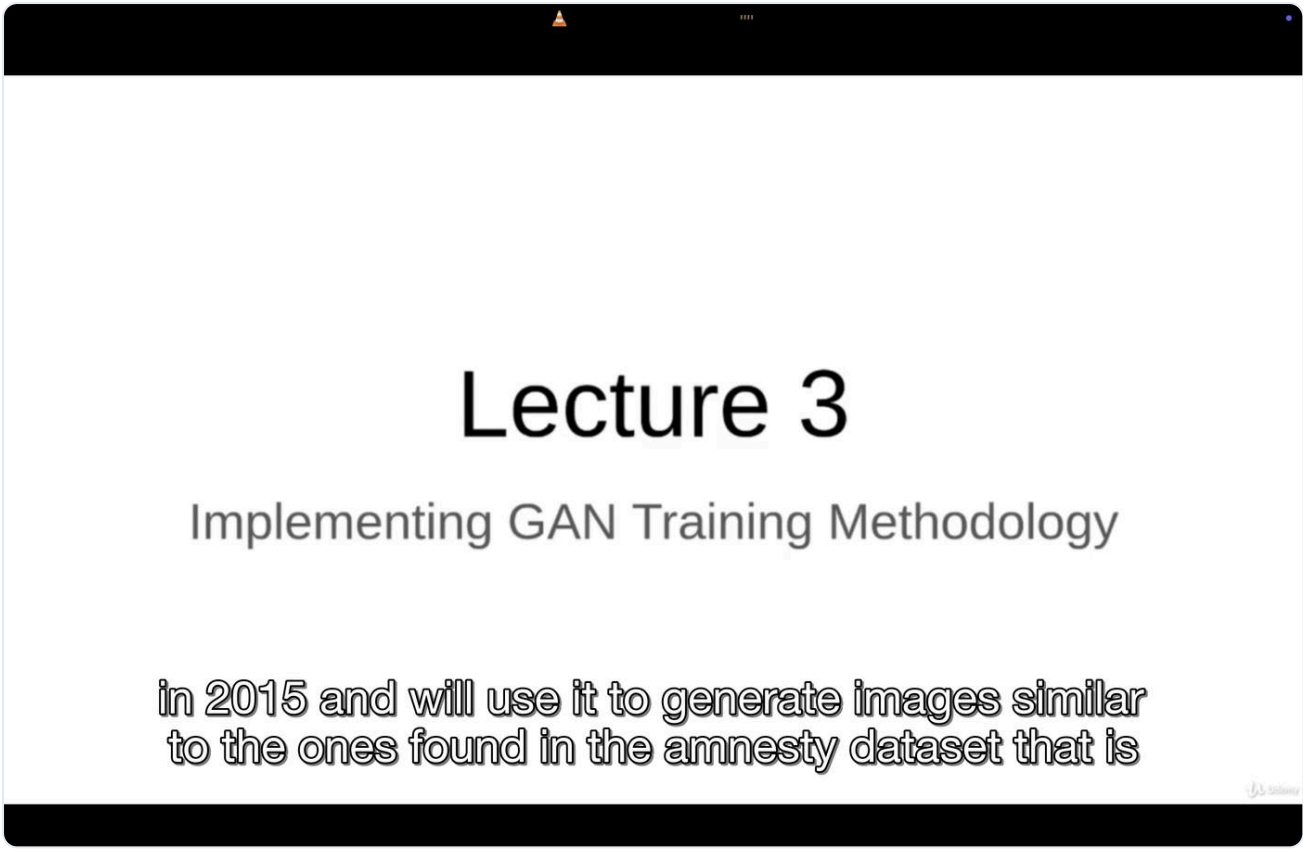


FIGURE 1.13 | Implementing GAN Training Methodology

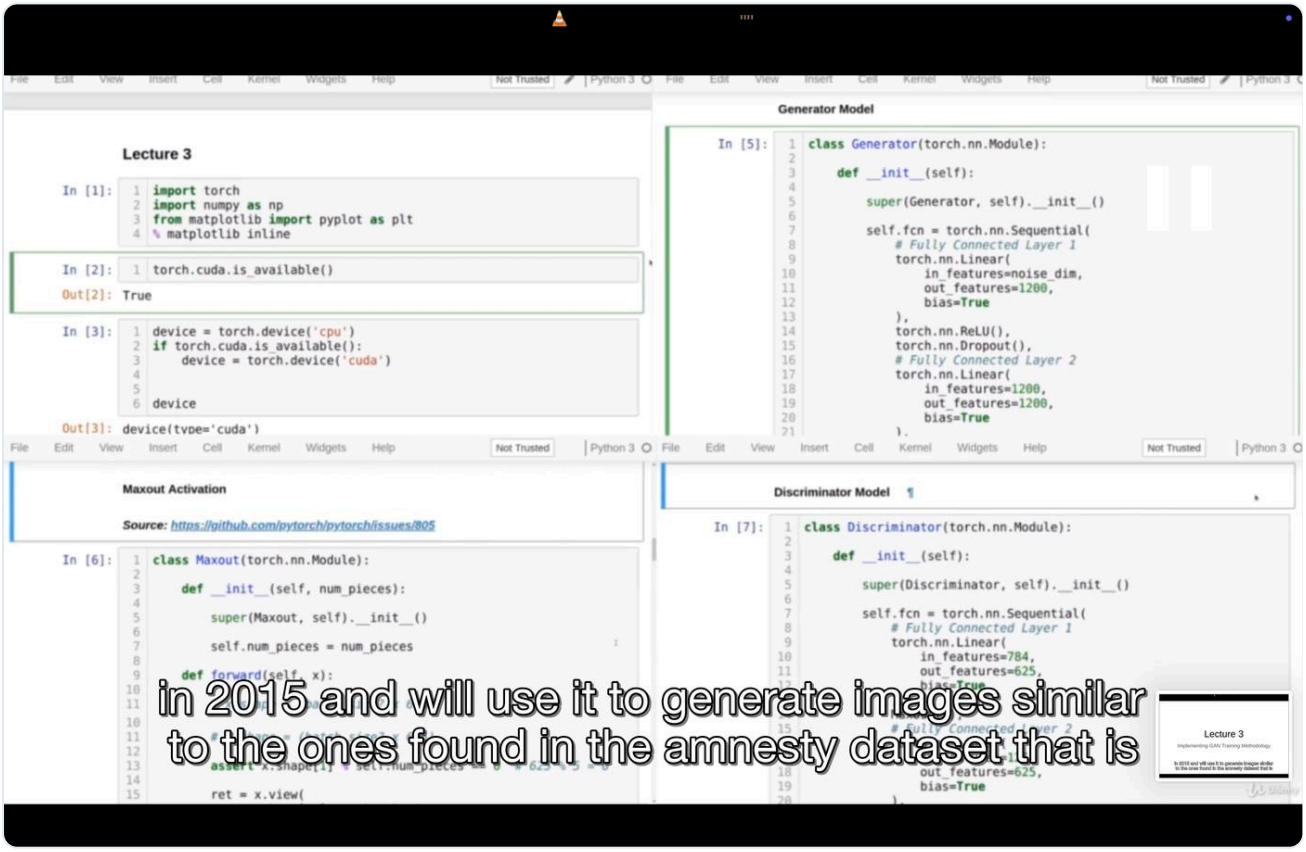


FIGURE 1.14

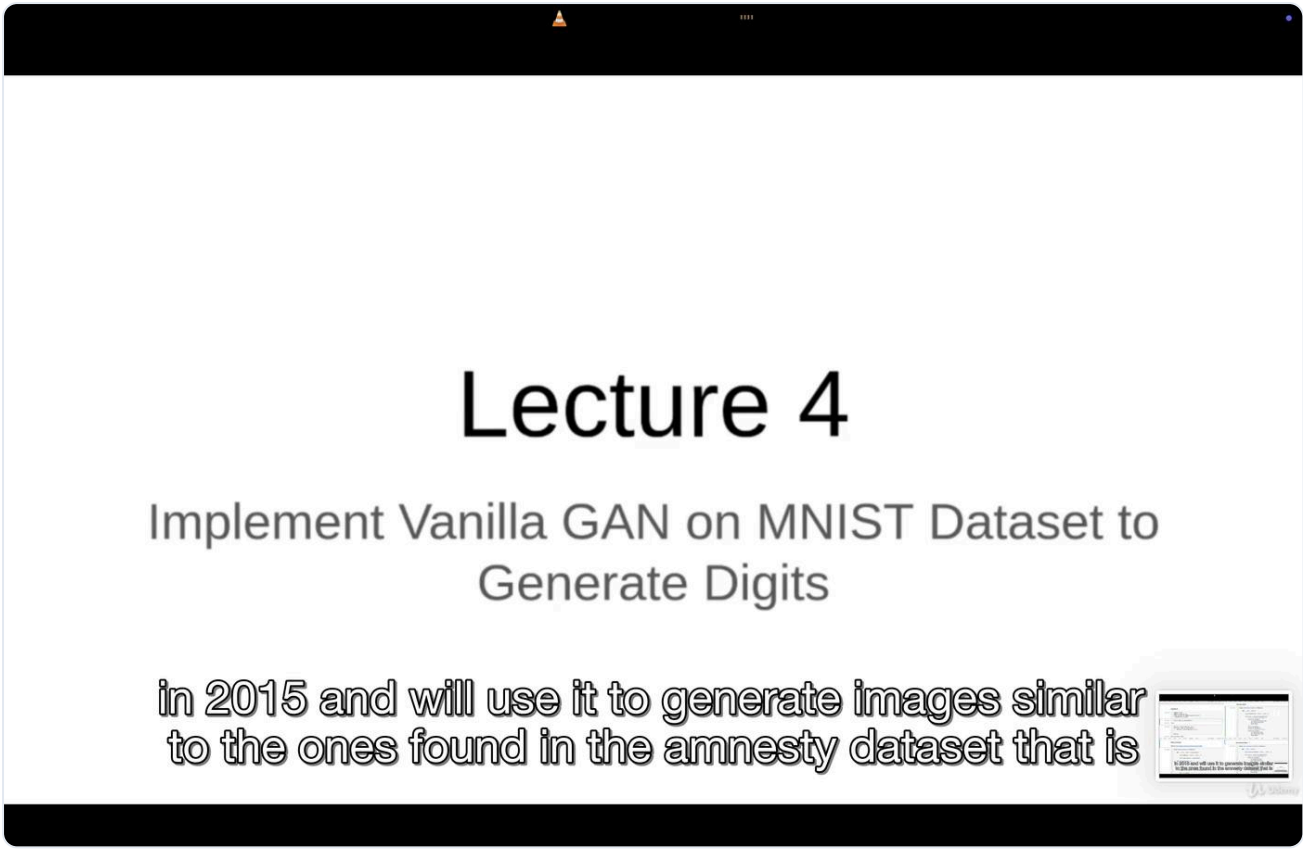


FIGURE 1.15 | Implement Vanilla GAN on MNIST Dataset to

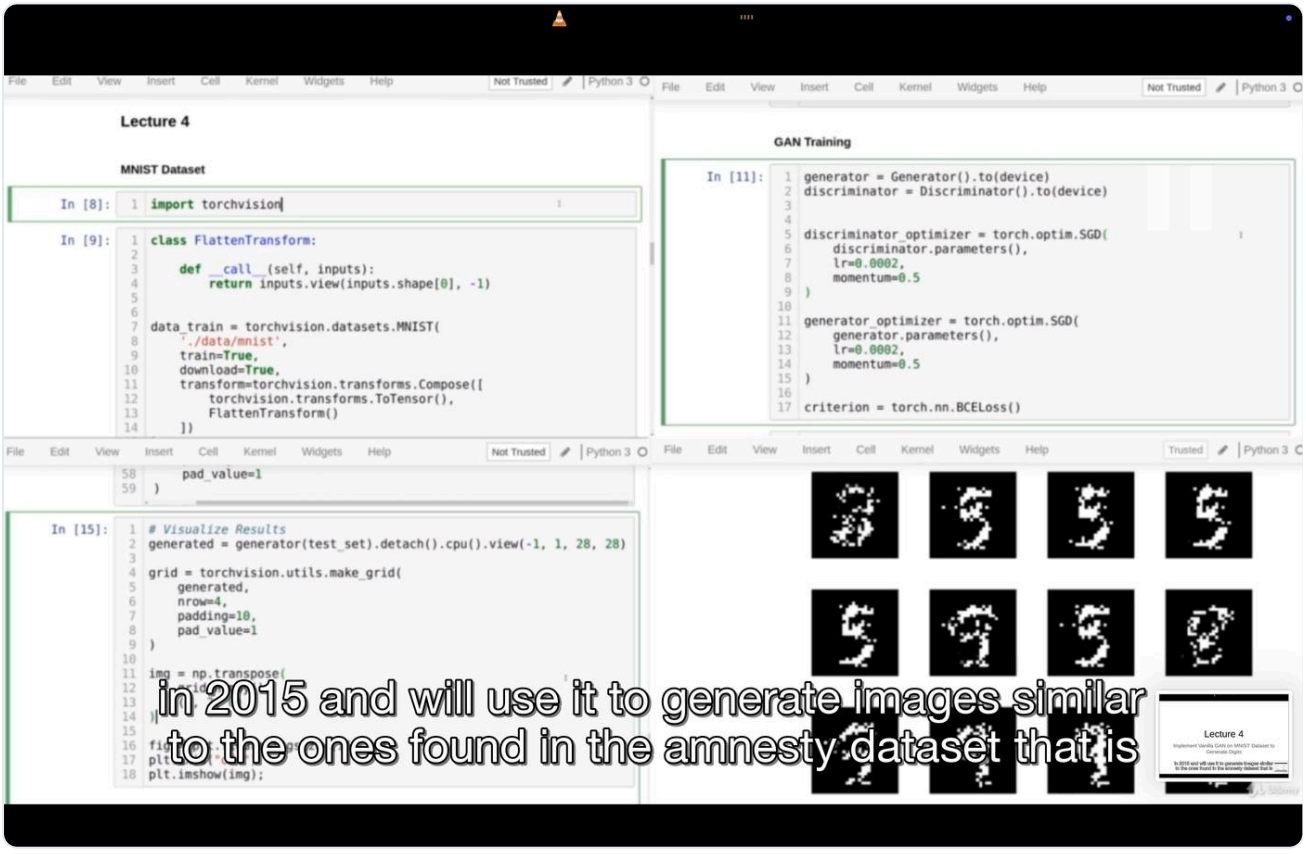


FIGURE 1.16

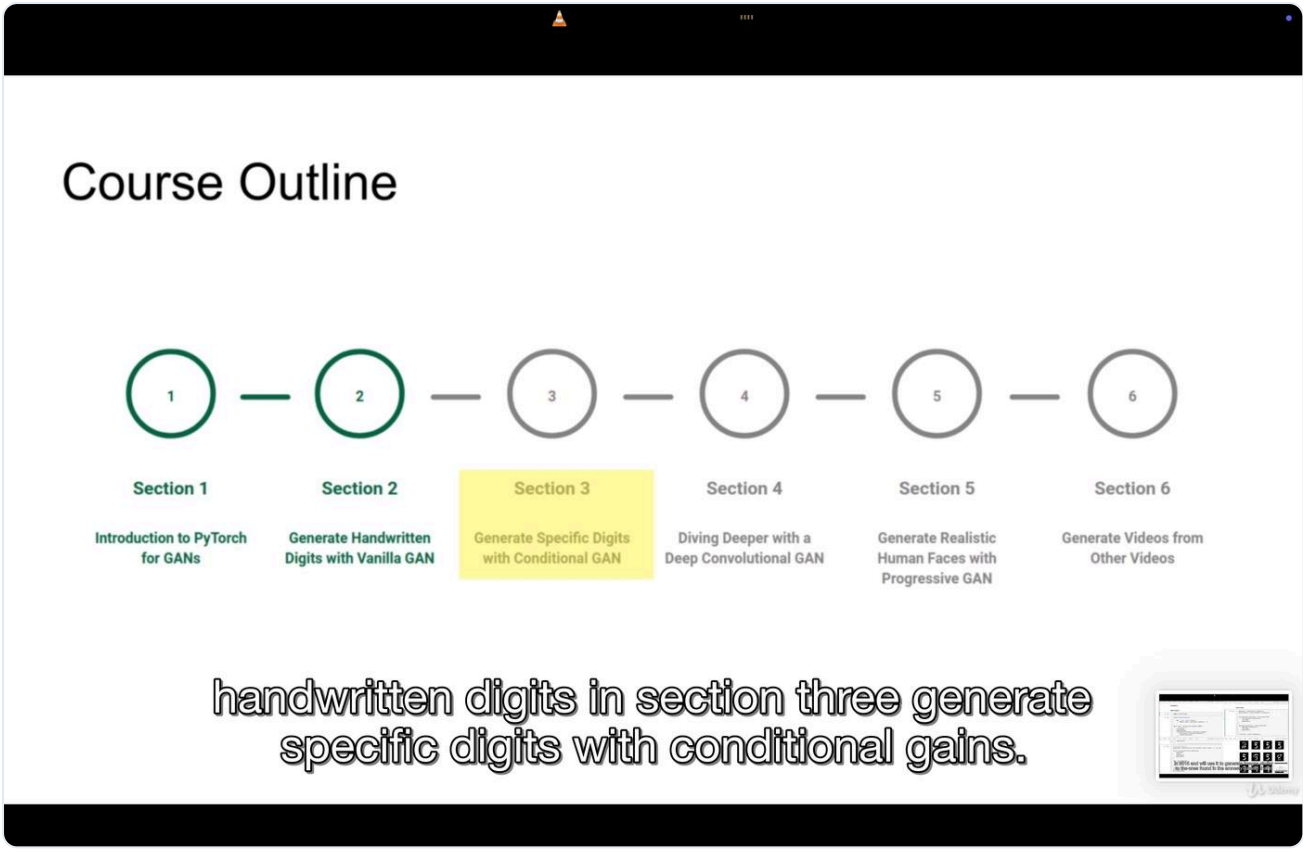


FIGURE 1.17 | Introduction to PyTorch Generate Handwritten

We will implement to the very first GAN ideas introduced in the paper published by Ian Goodfellow in 2014 and will use it to generate images similar to the ones found in the MNIST dataset that is handwritten digits in section three generate specific digits with conditional GANs.

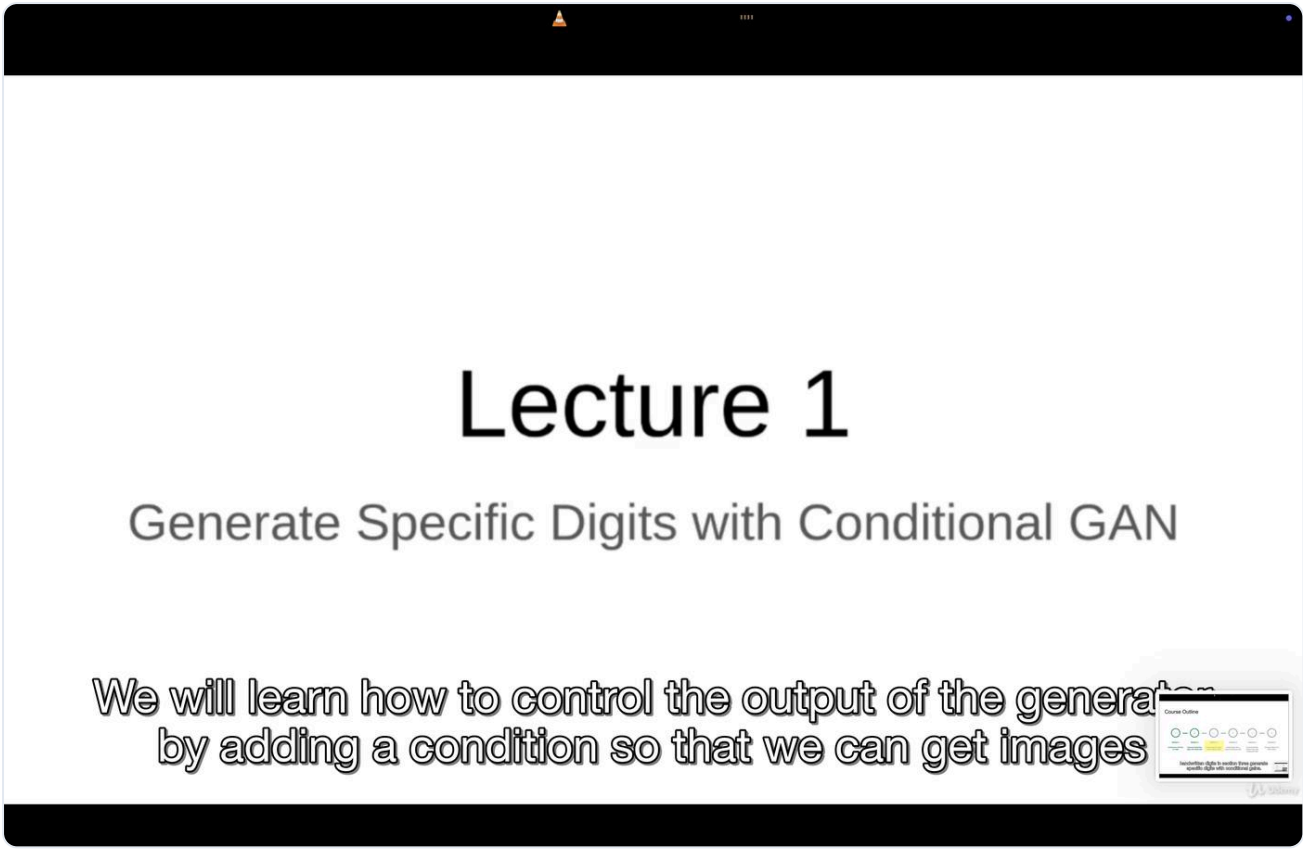


FIGURE 1.18 | Generate Specific Digits with Conditional GAN

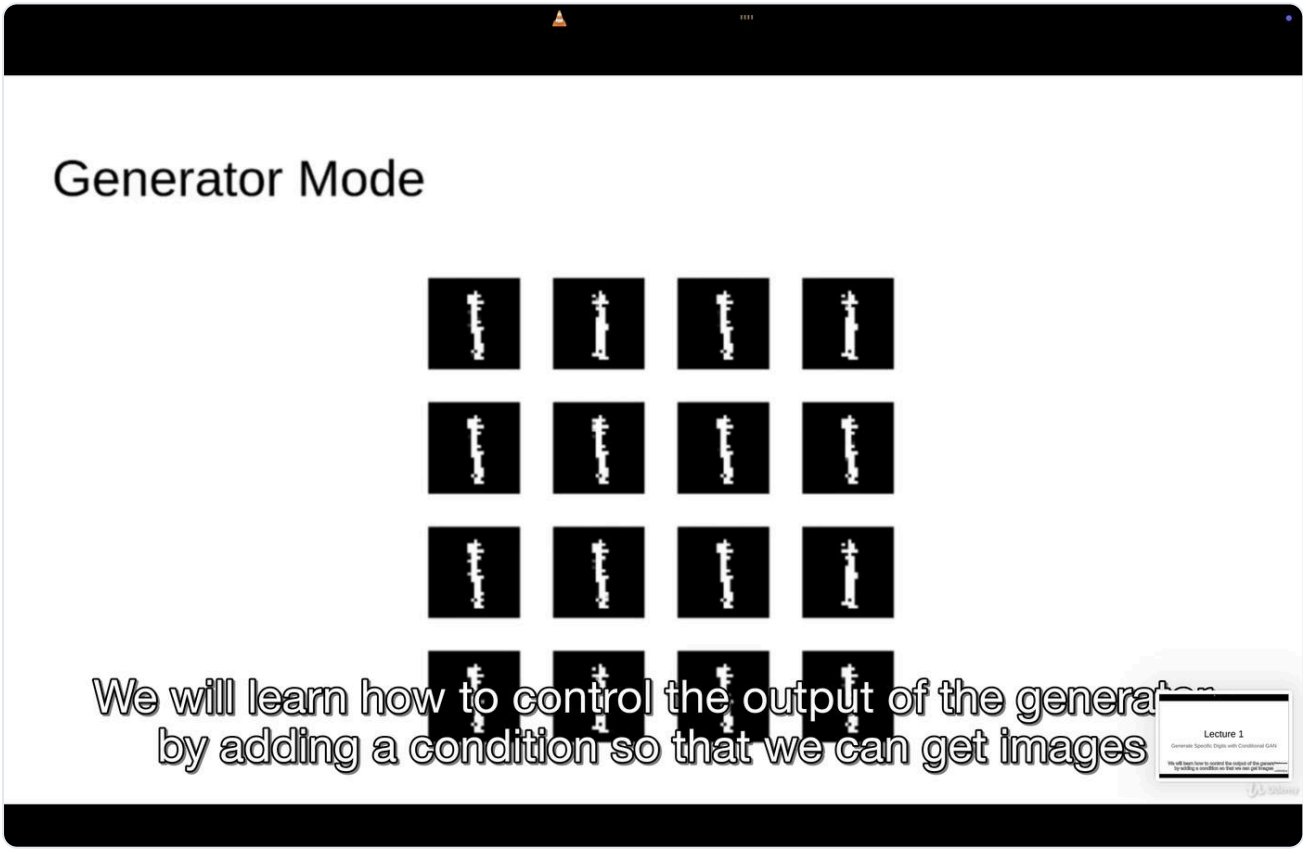


FIGURE 1.19 | Generator Mode

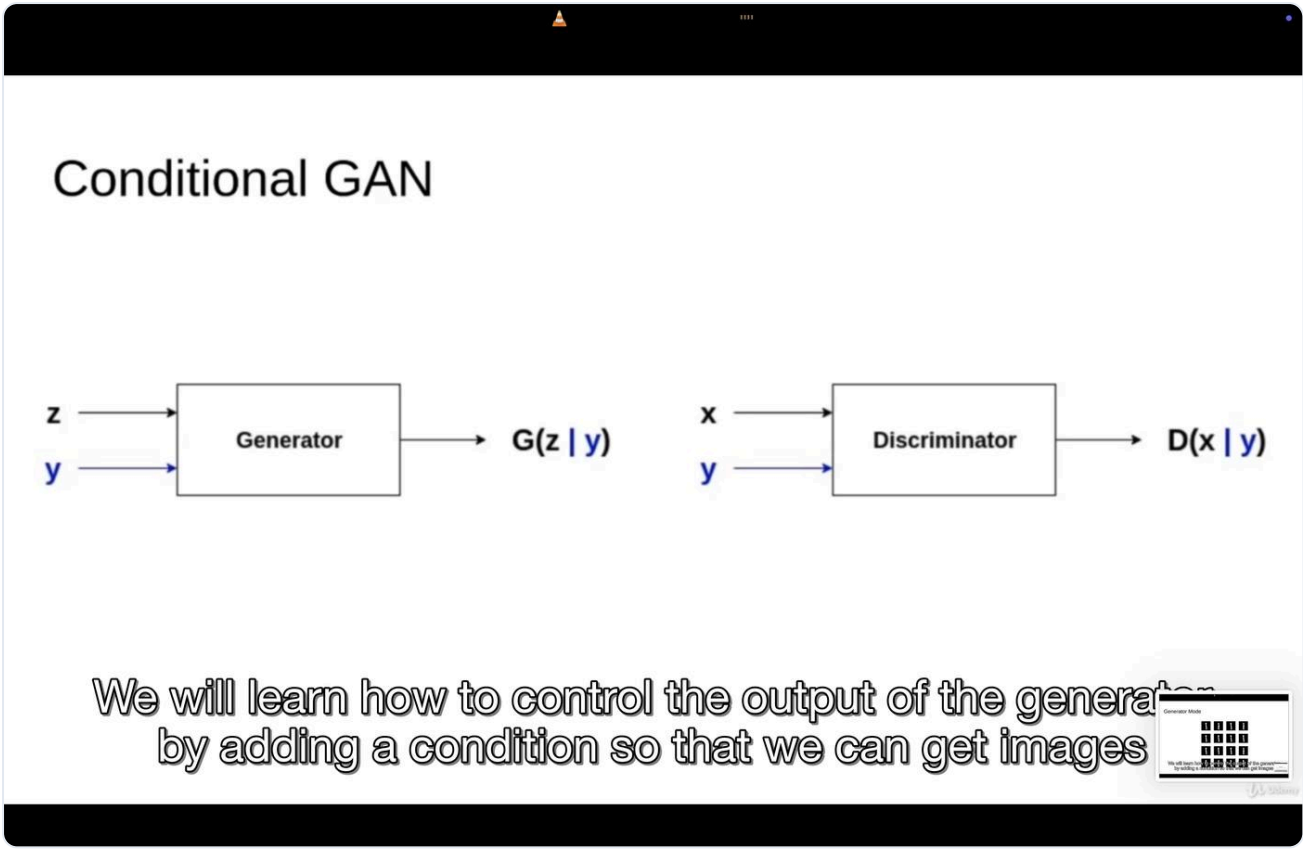


FIGURE 1.20

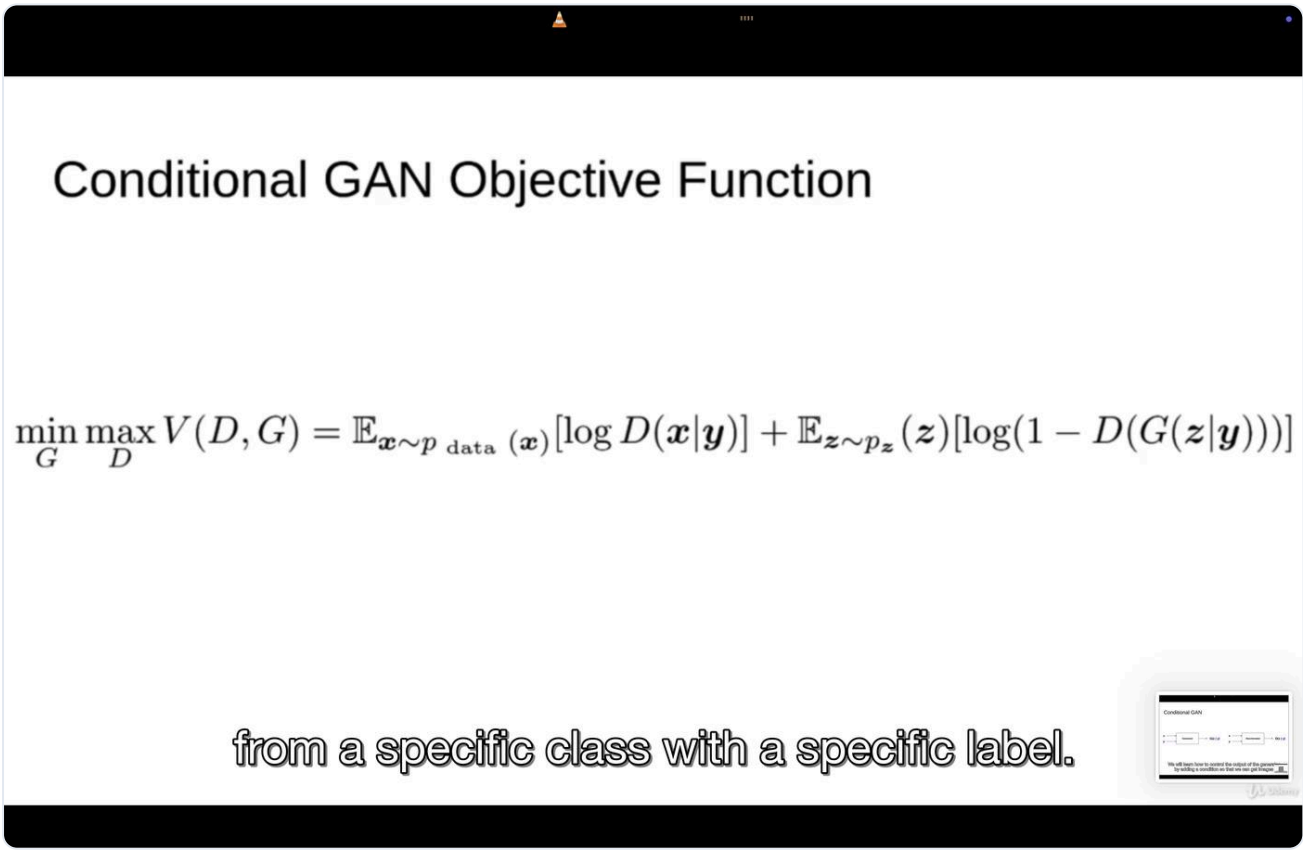


FIGURE 1.21 | Conditional GAN Objective Function

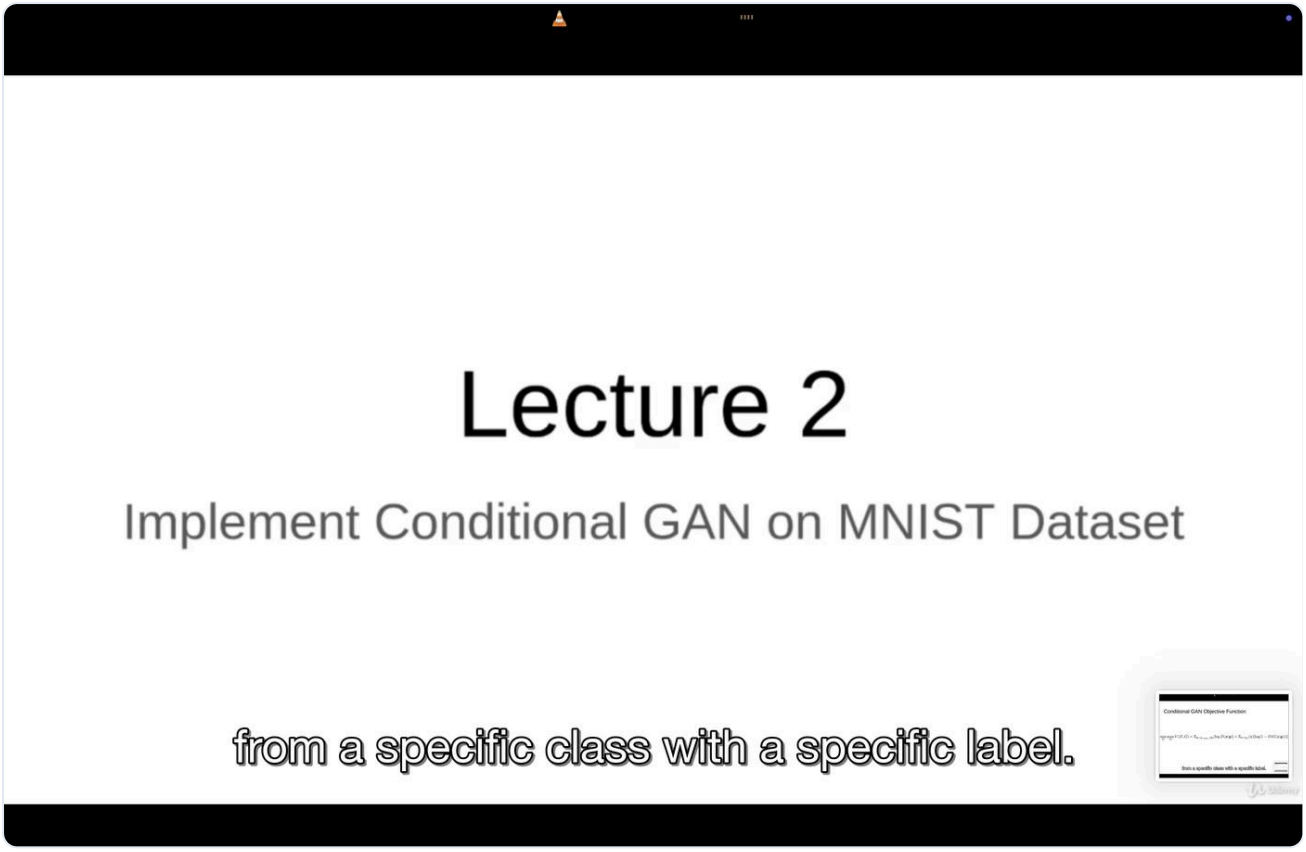


FIGURE 1.22 | Implement Conditional GAN on MNIST Dataset

We will learn how to control the output of the generator by adding a condition so that we can get images from a specific class with a specific label.

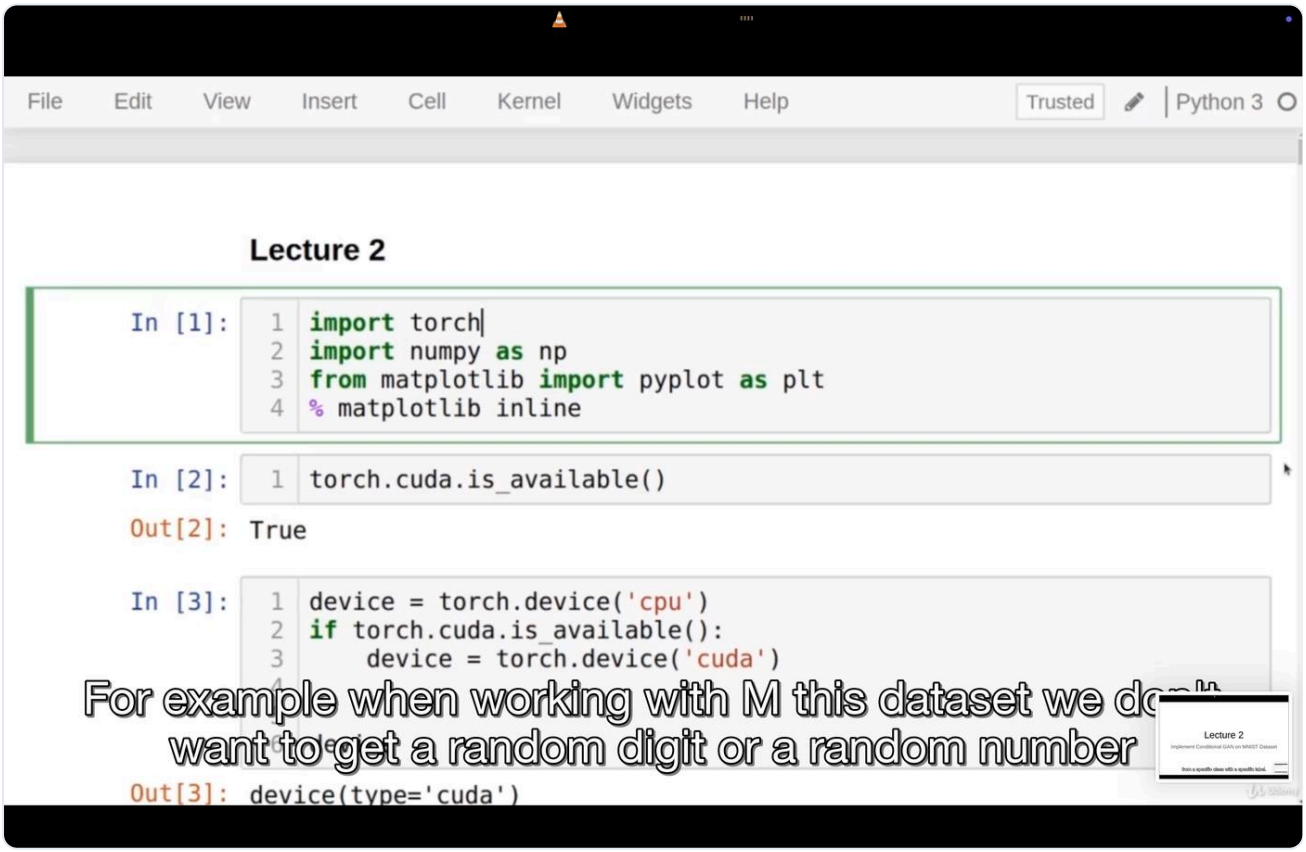


FIGURE 1.23

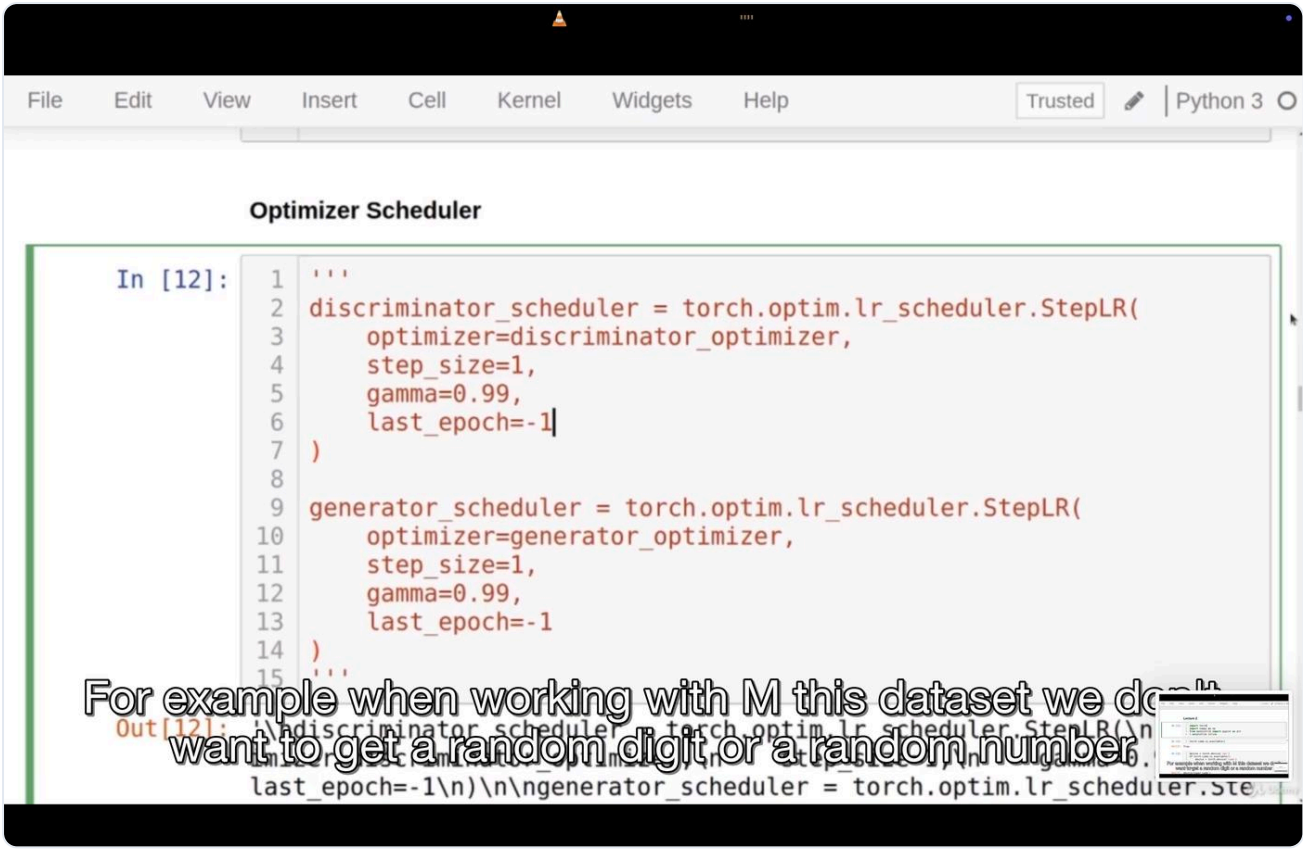


FIGURE 1.24

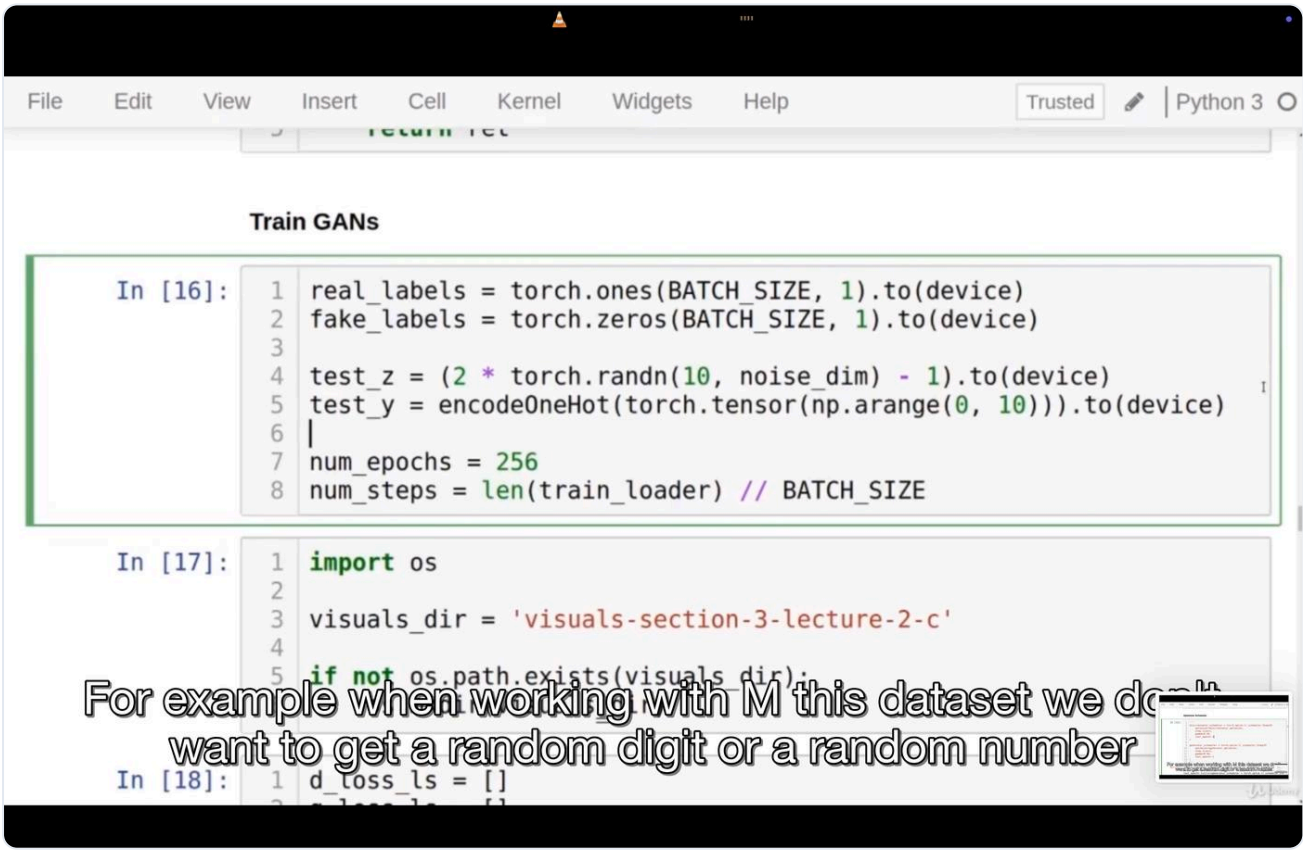


FIGURE 1.25 | Train GANs

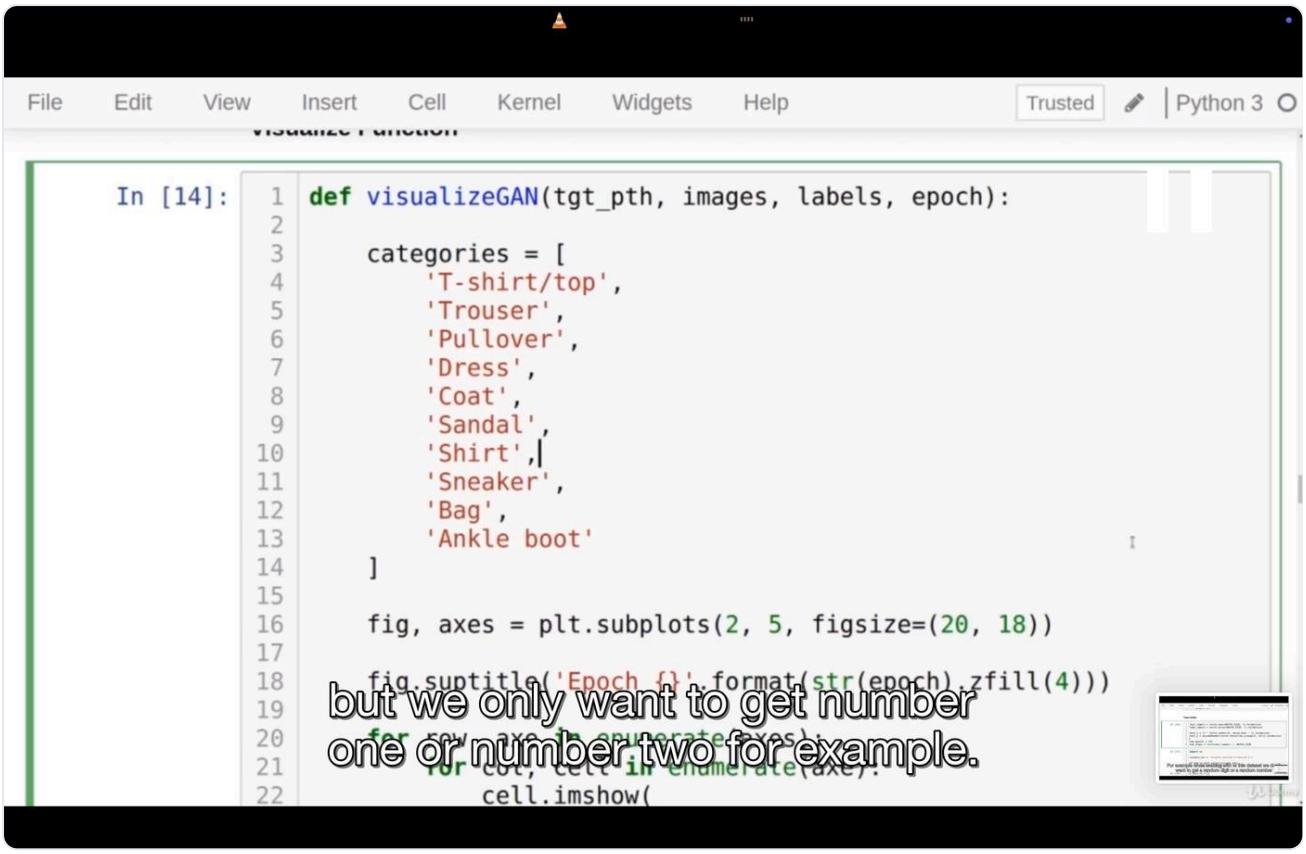


FIGURE 1.26

FIGURE 1.27

For example when working with MNIST dataset we don't want to get a random digit or a random number but we only want to get number one or number two for example.

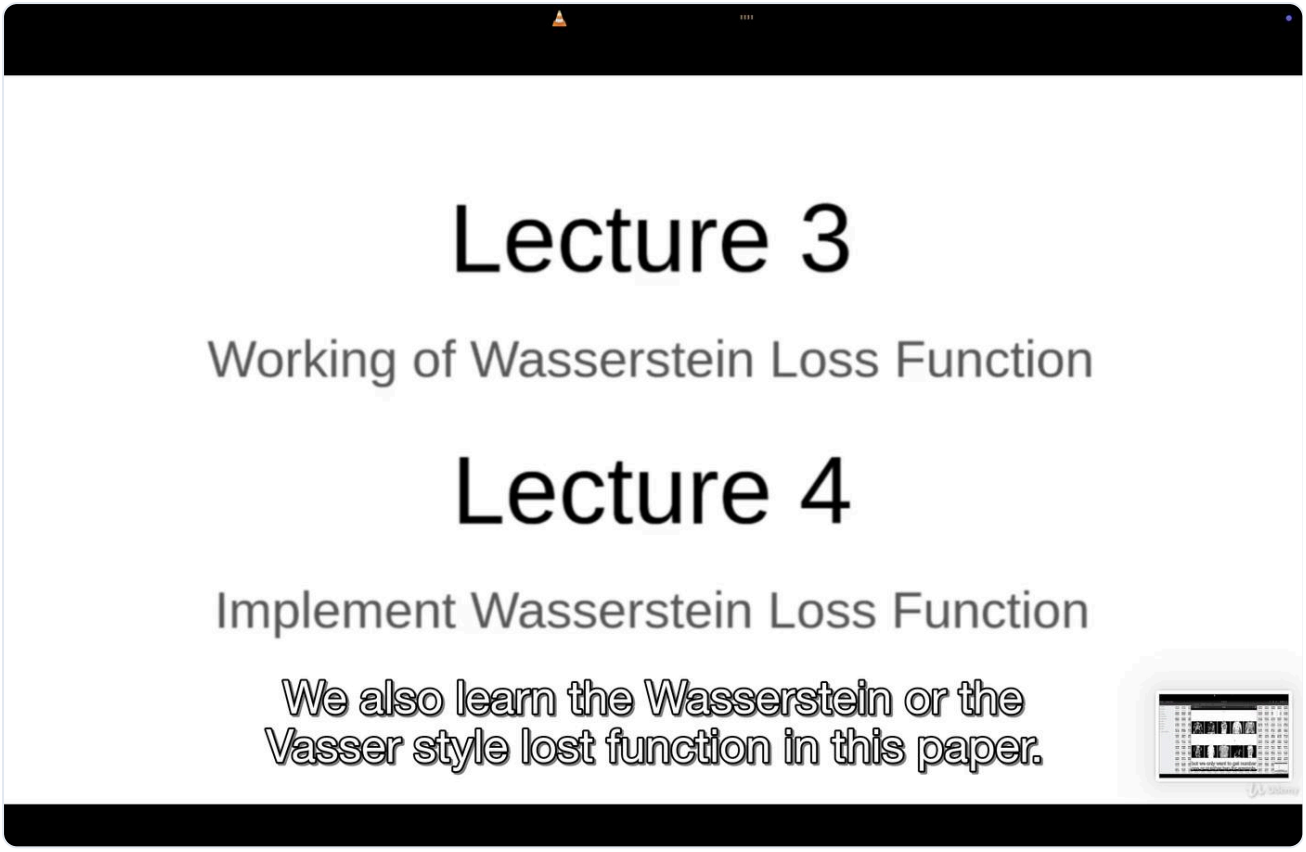


FIGURE 1.28

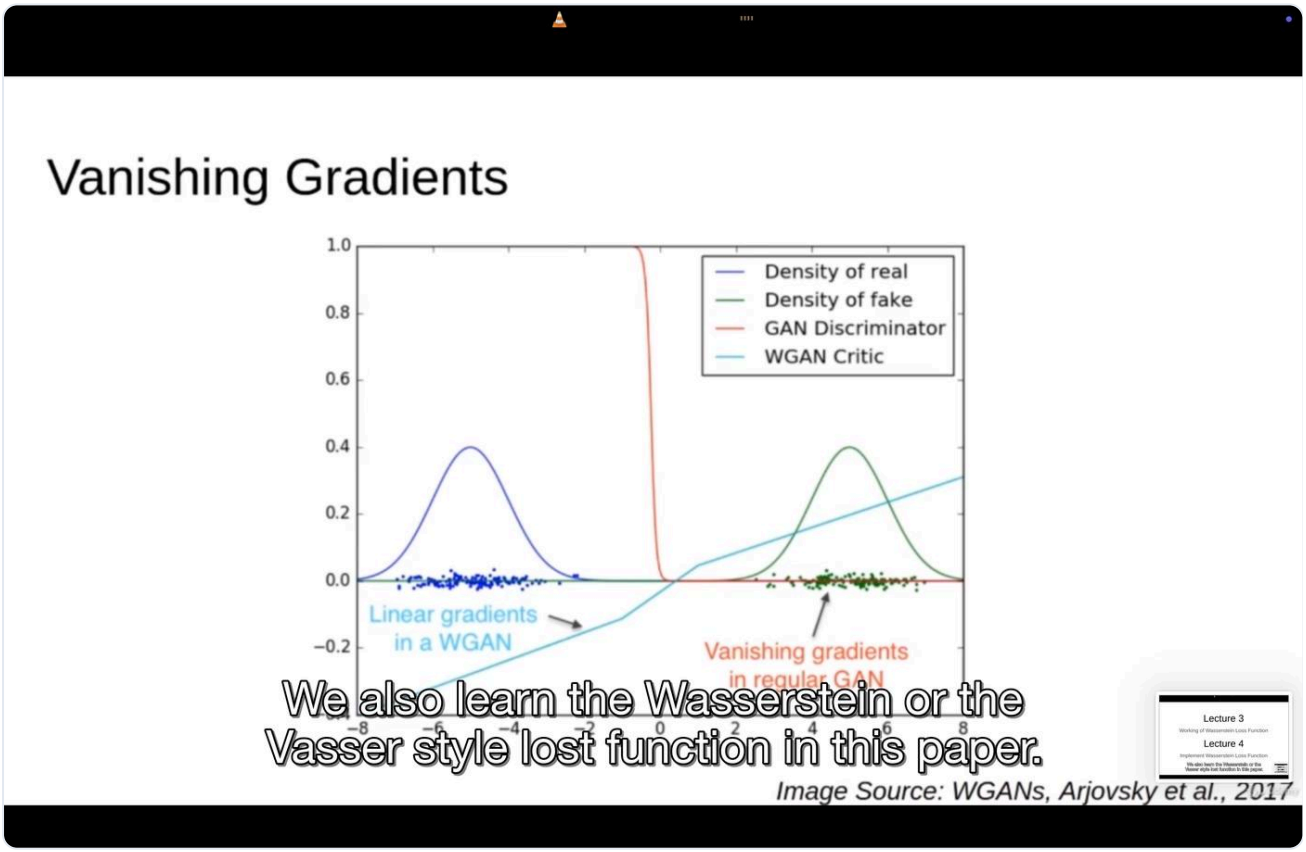


FIGURE 1.29

We also learn the Wasserstein or the Wasserstein loss function in this paper.

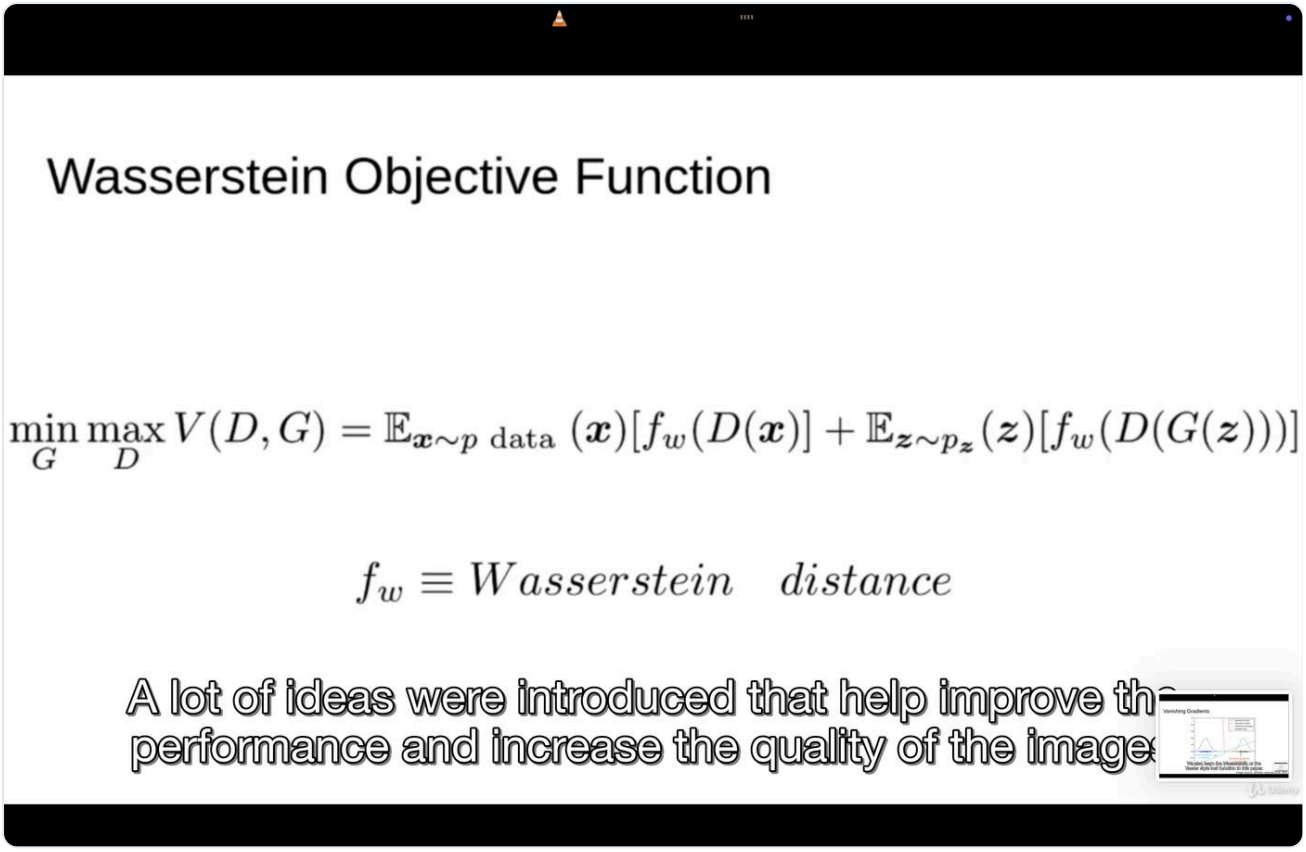


FIGURE 1.30 | Wasserstein Objective Function

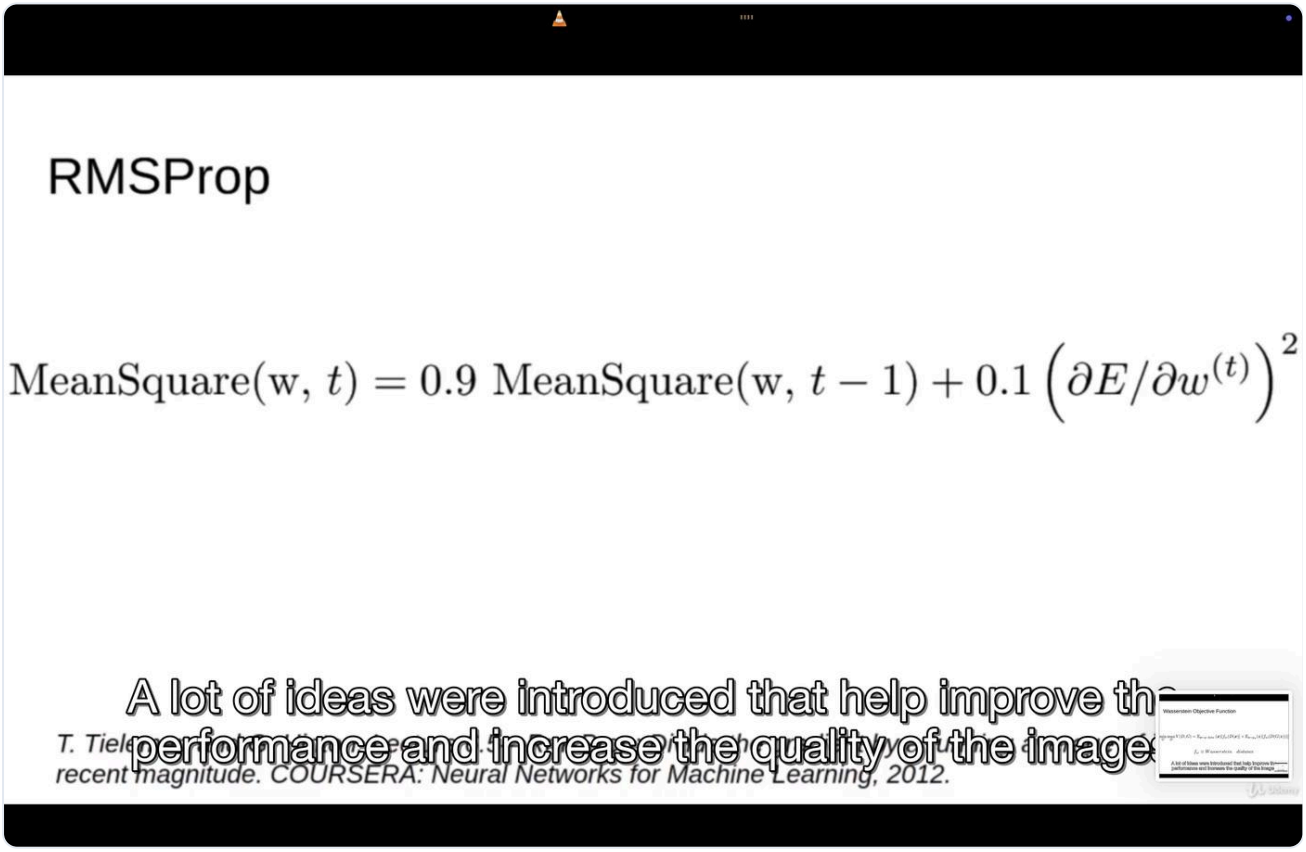


FIGURE 1.31

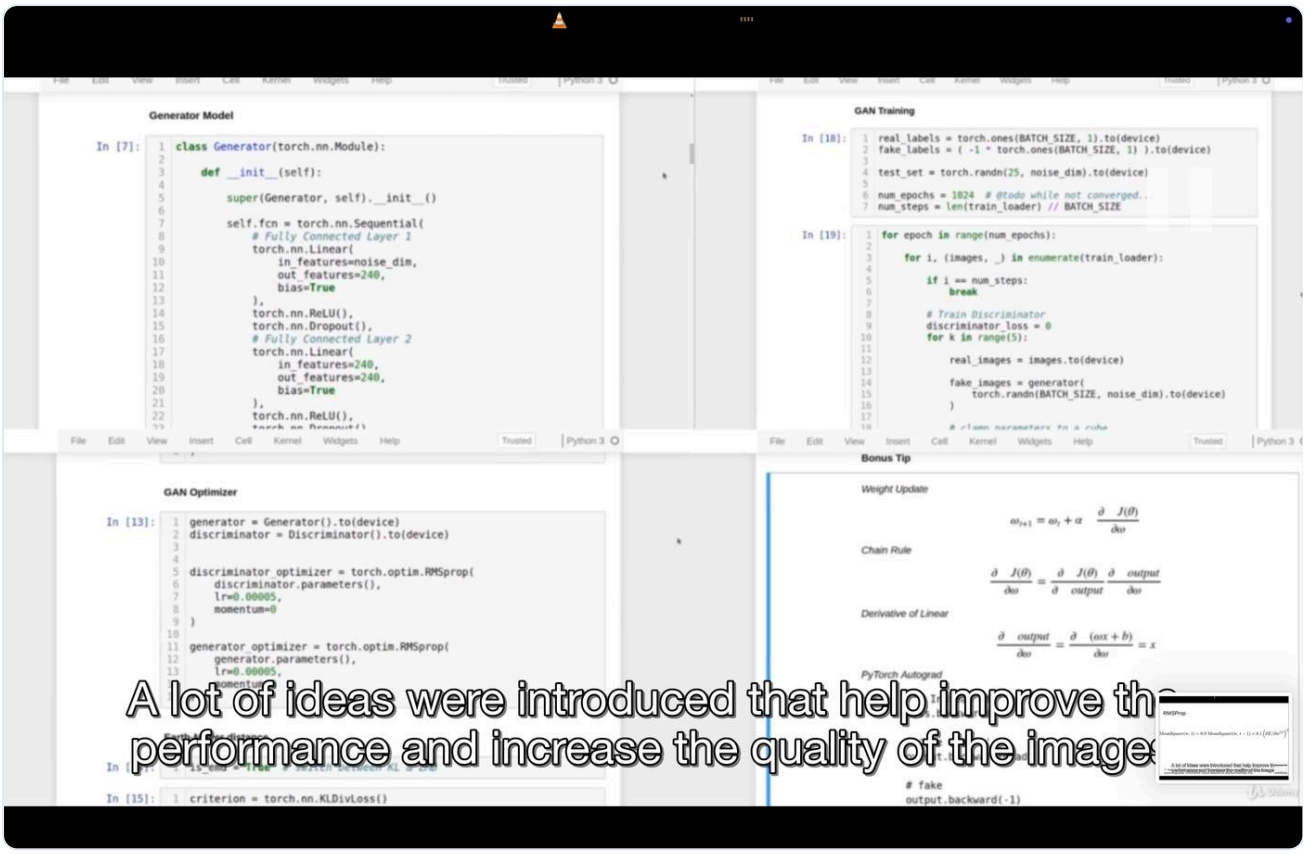


FIGURE 1.32

FIGURE 1.33

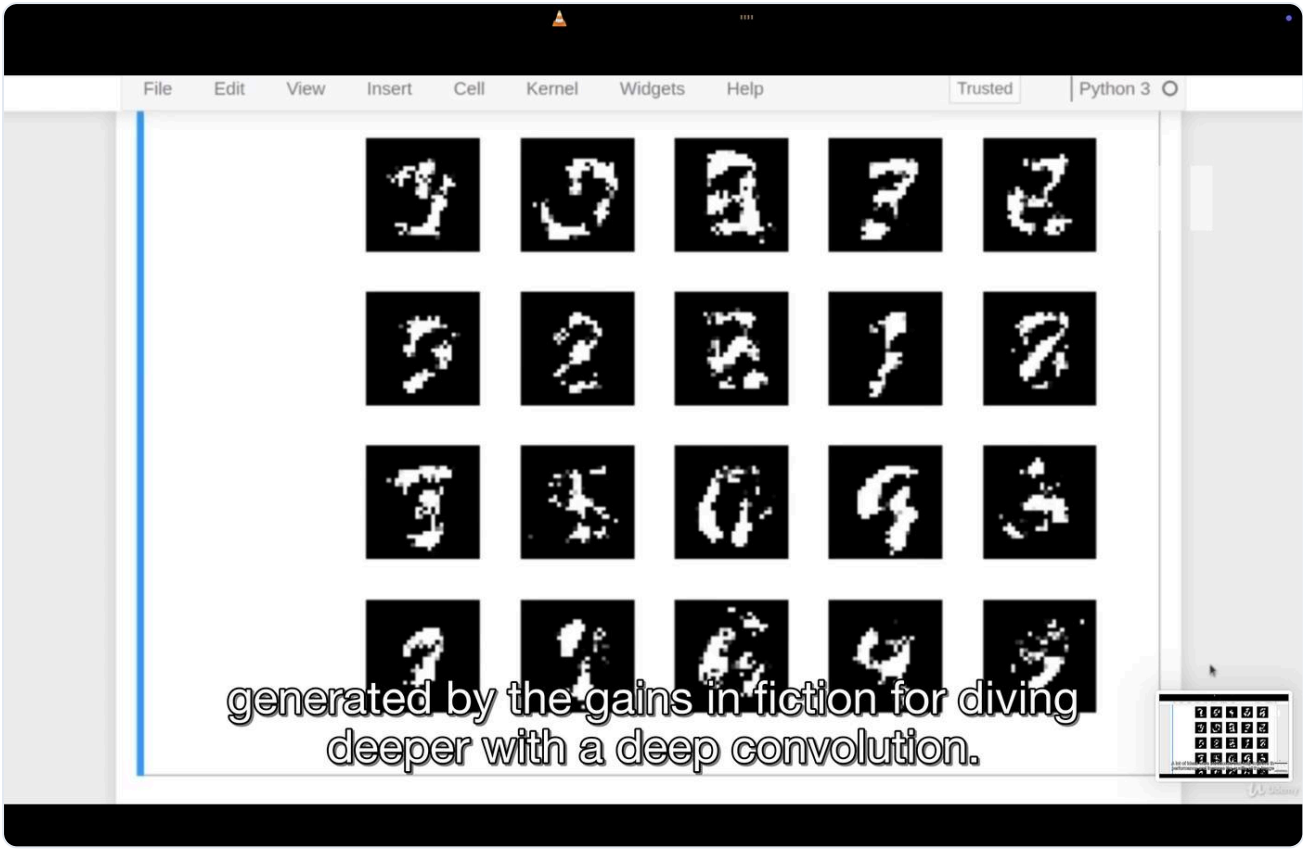


FIGURE 1.34

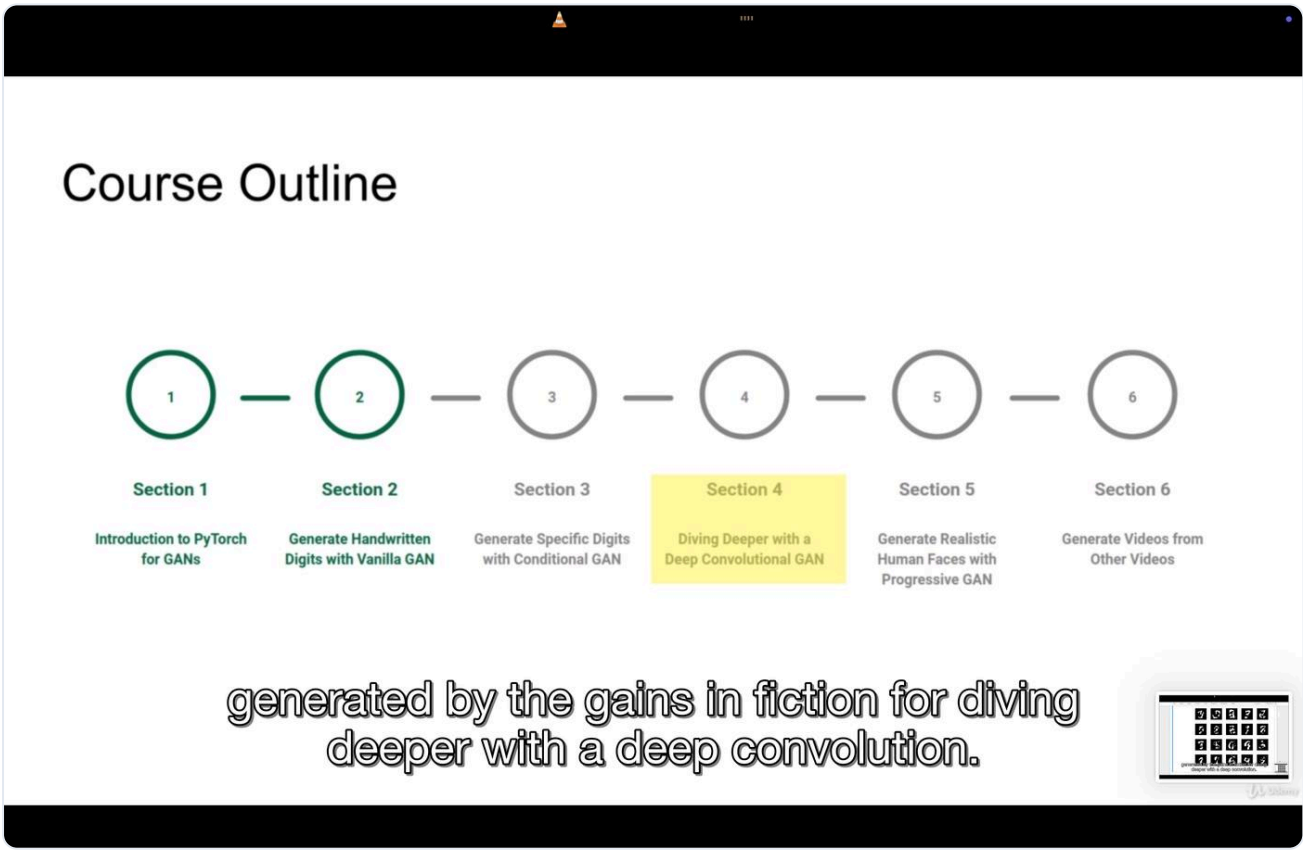


FIGURE 1.35 | Introduction to PyTorch Generate Handwritten

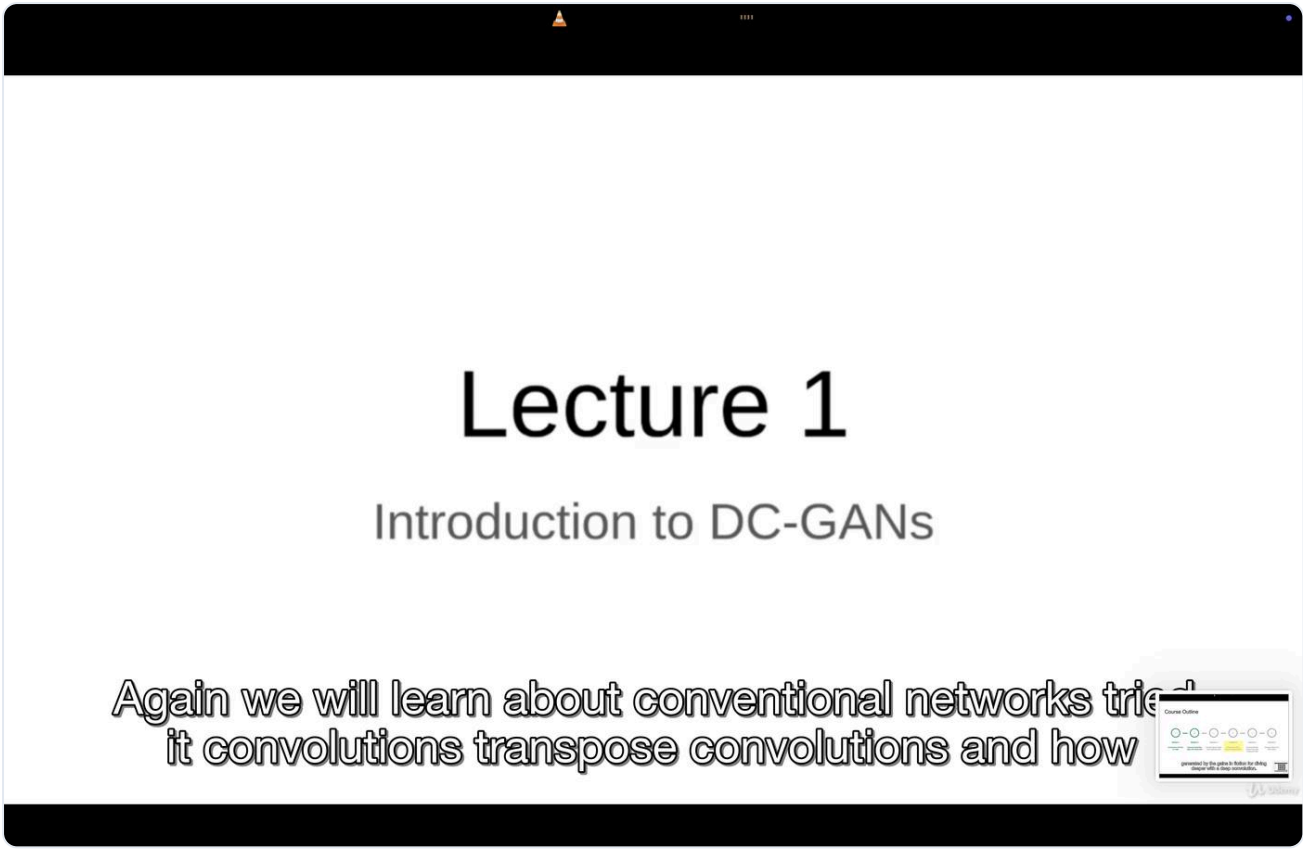


FIGURE 1.36

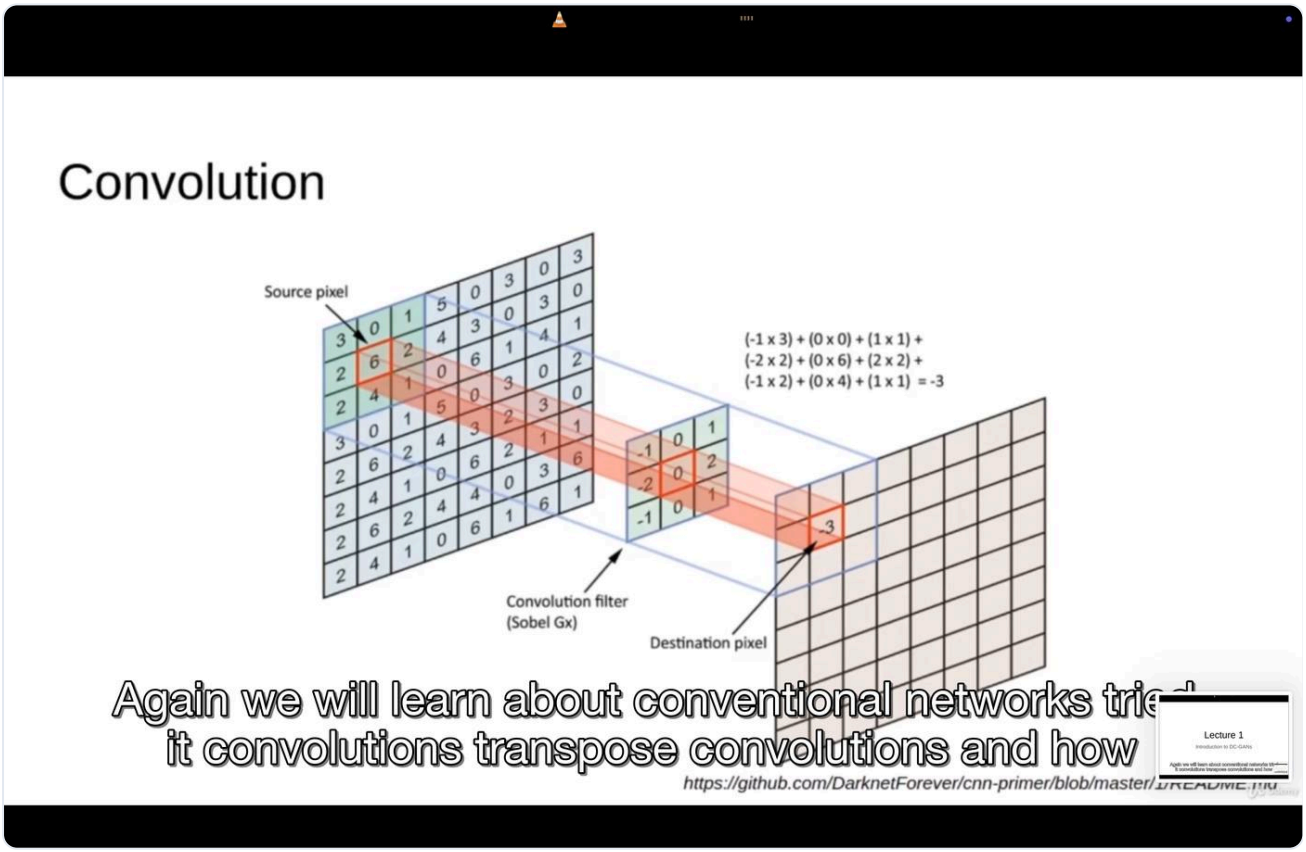


FIGURE 1.37

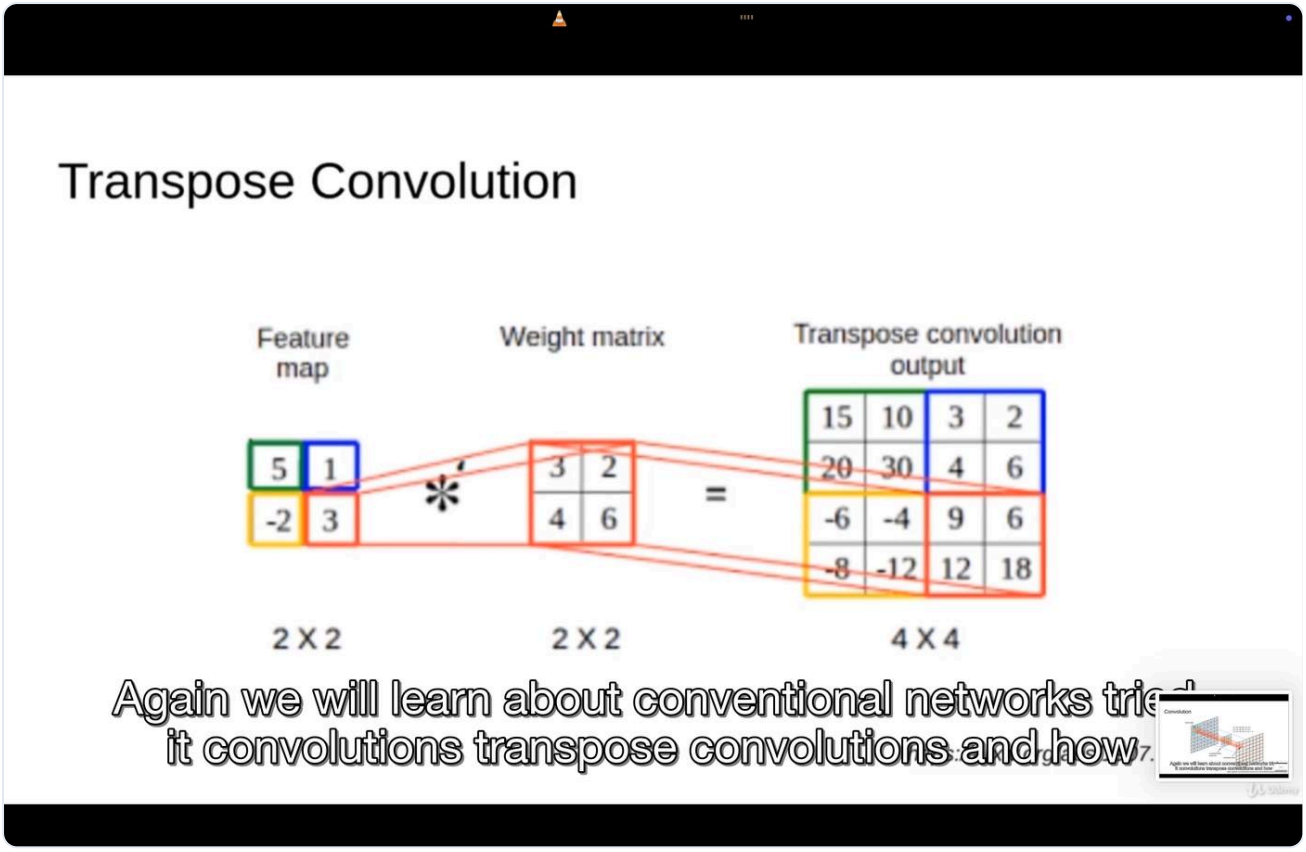


FIGURE 1.38 | Transpose Convolution

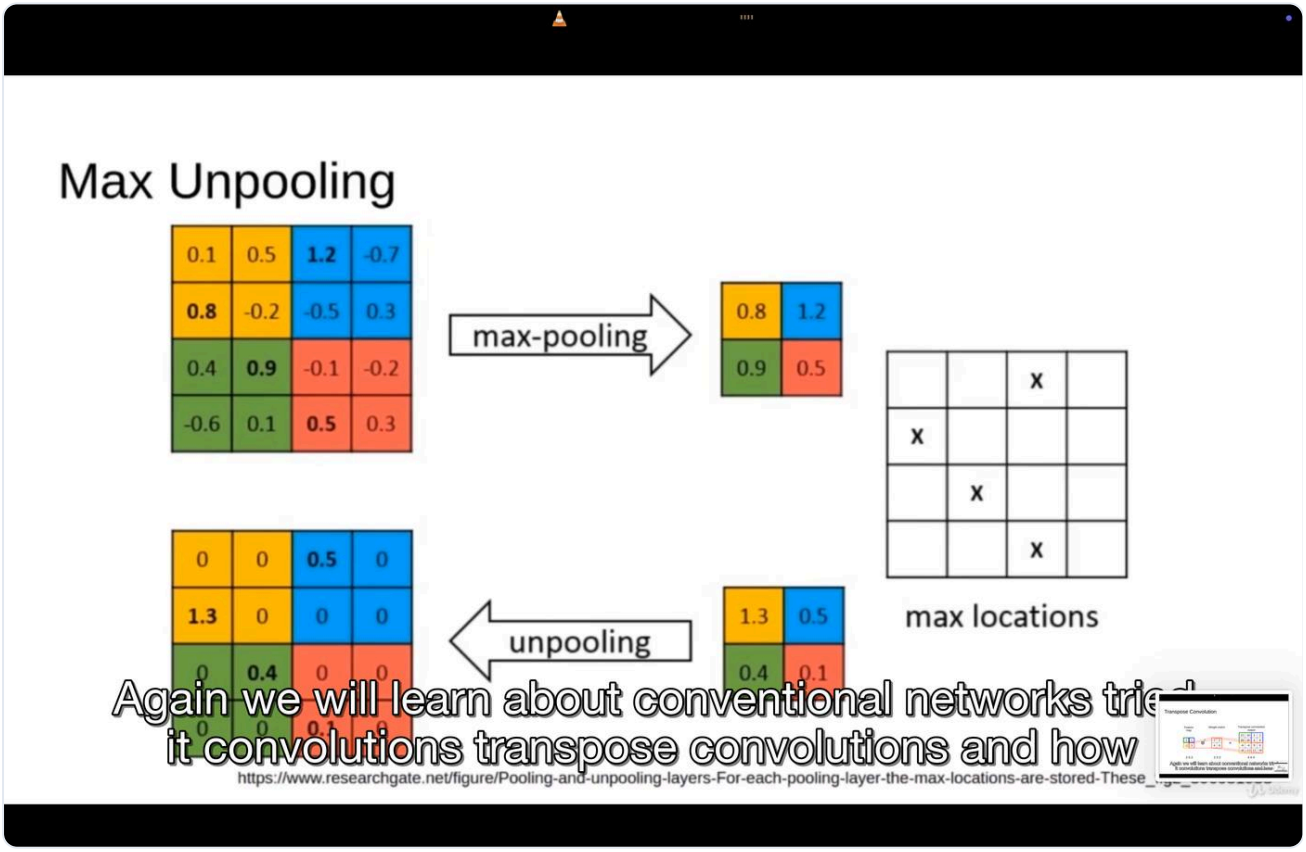


FIGURE 1.39

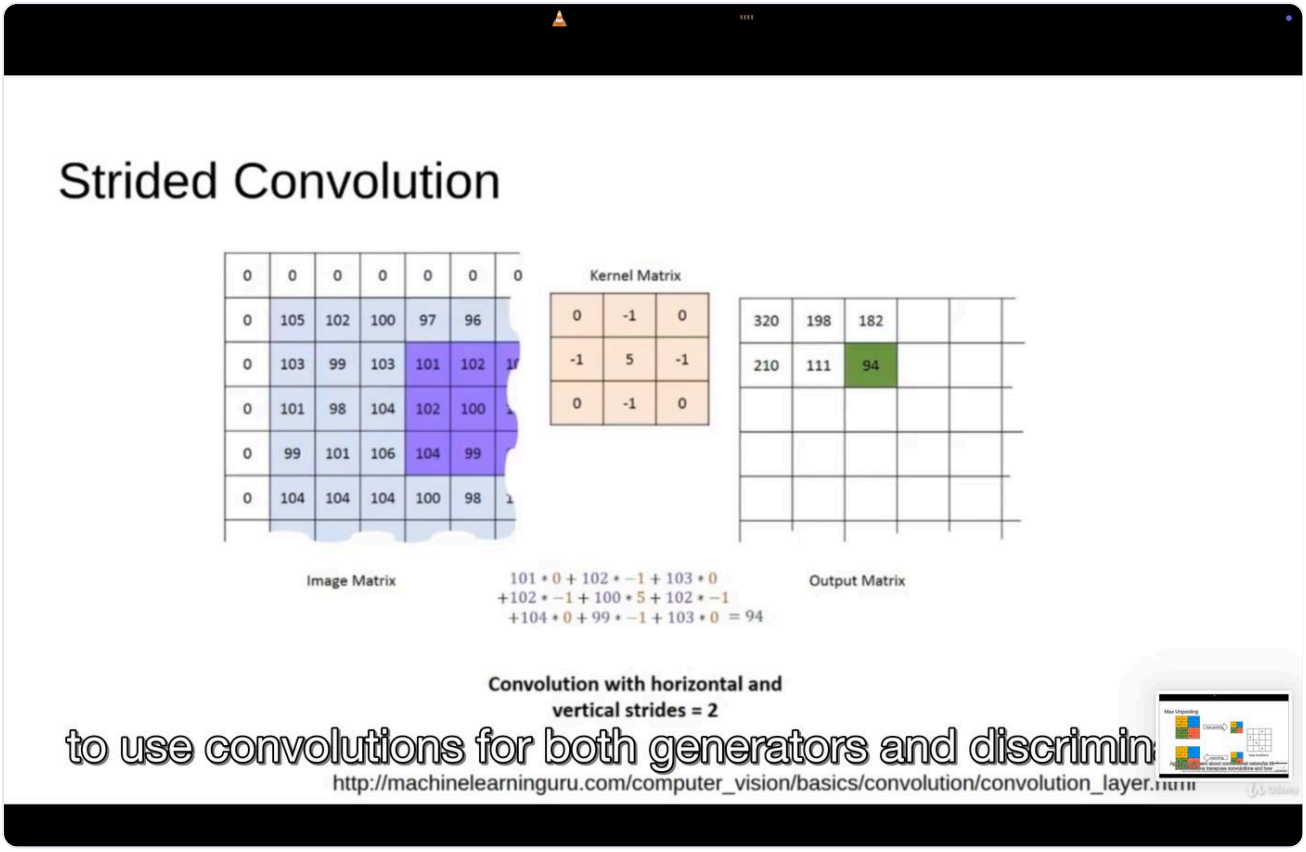


FIGURE 1.40 | Strided Convolution

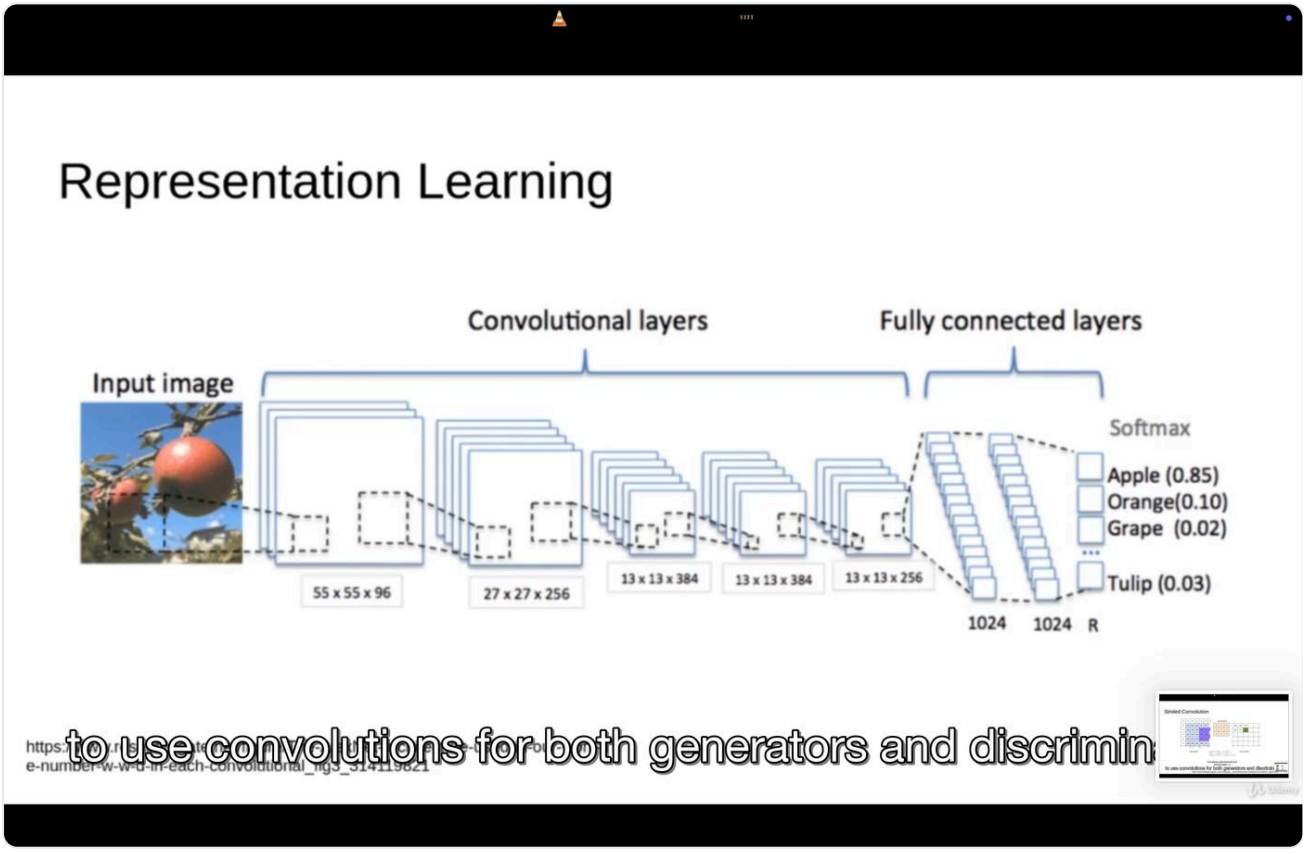


FIGURE 1.41 | Representation Learning

A lot of ideas were introduced that help improve the performance and increase the quality of the images generated by the GANs in section four diving deeper with a deep convolutional GAN. we will learn about convolutional networks, strided convolutions transpose convolutions and how to use convolutions for both generators and discriminator.

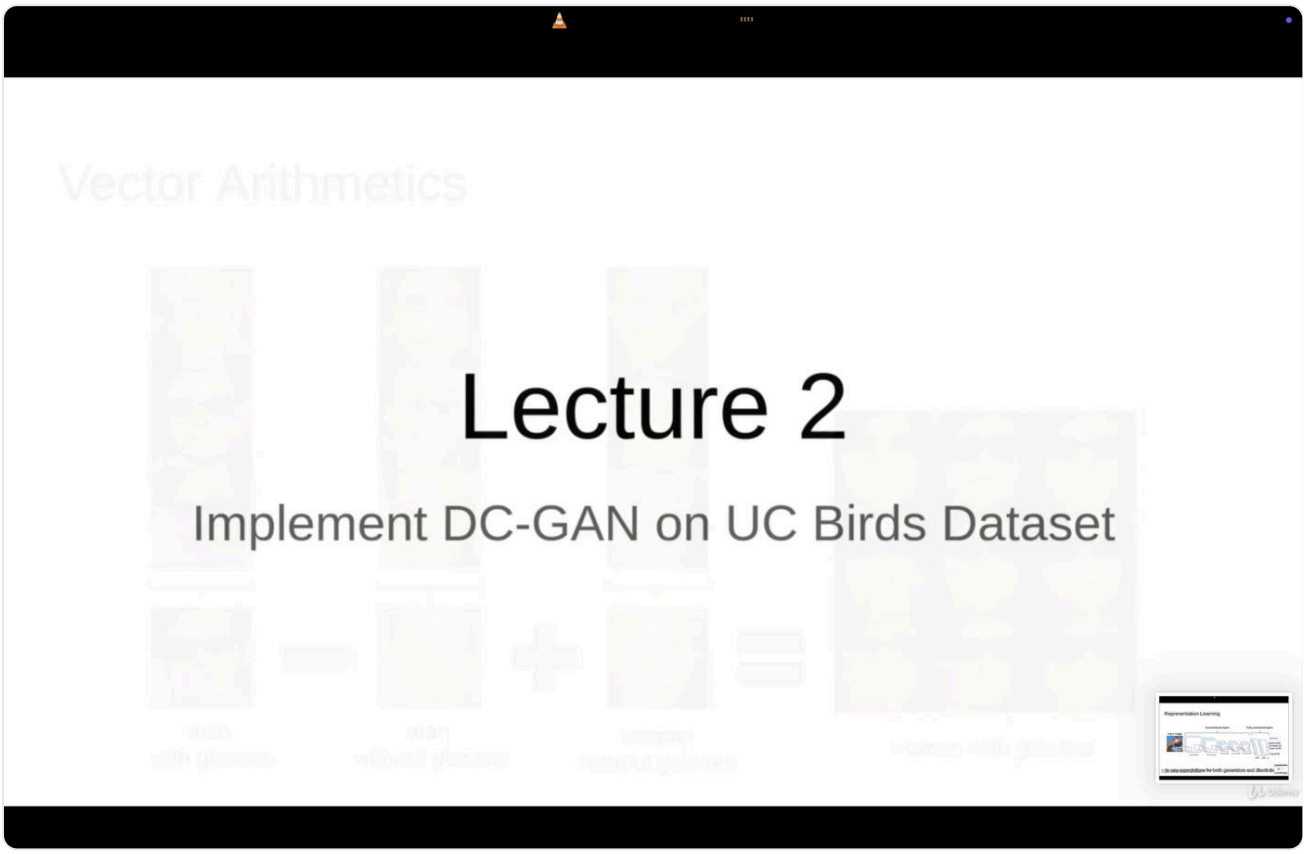


FIGURE 1.42 | Implement DC-GAN on UC Birds Dataset

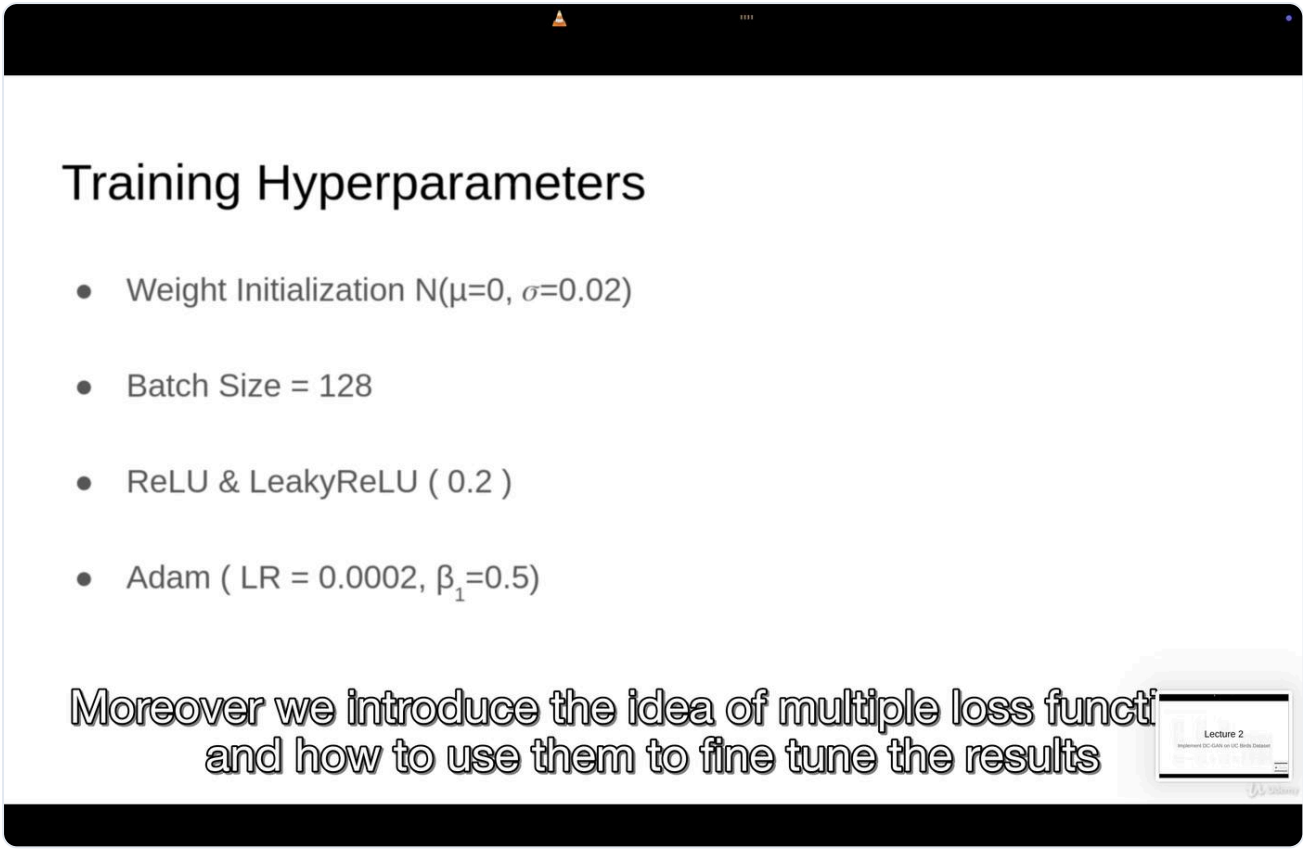


FIGURE 1.43

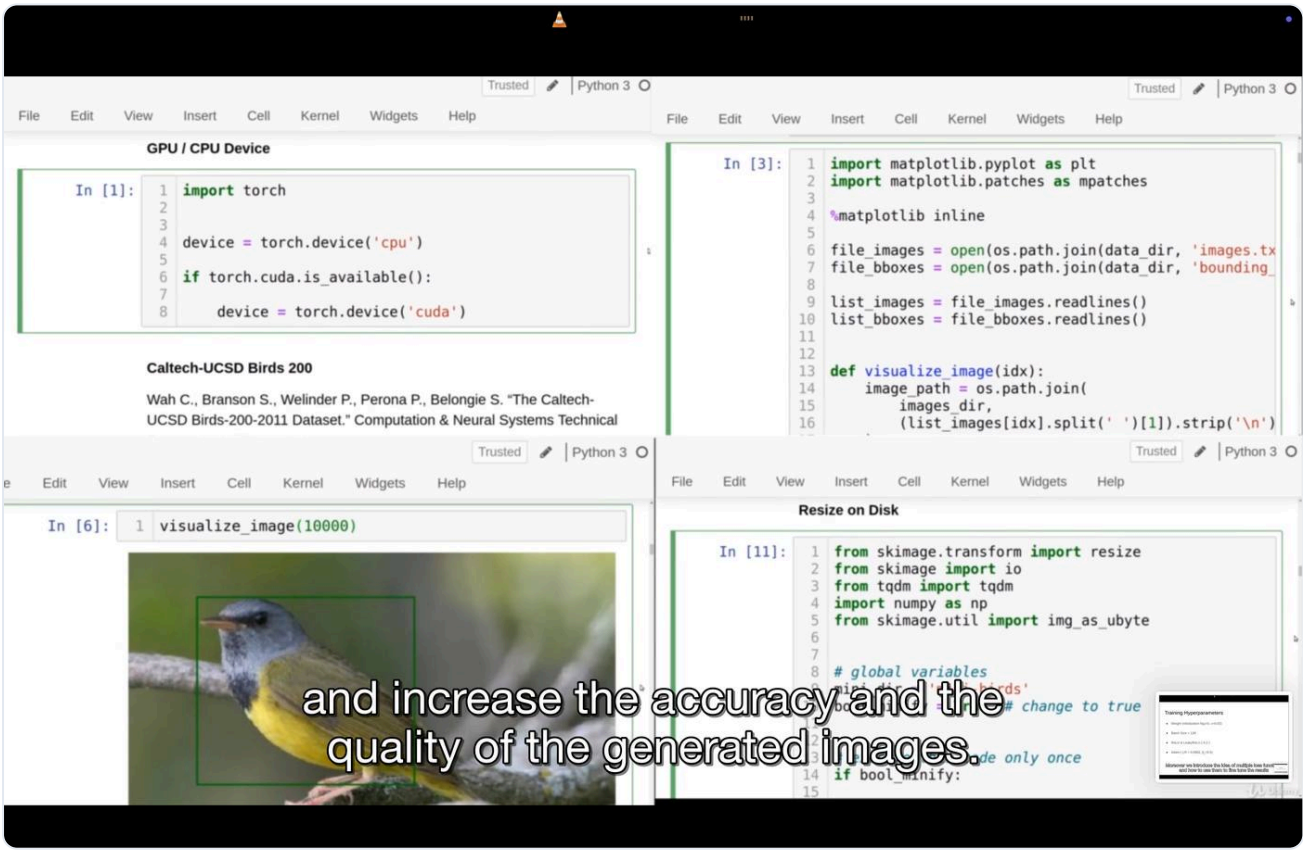


FIGURE 1.44

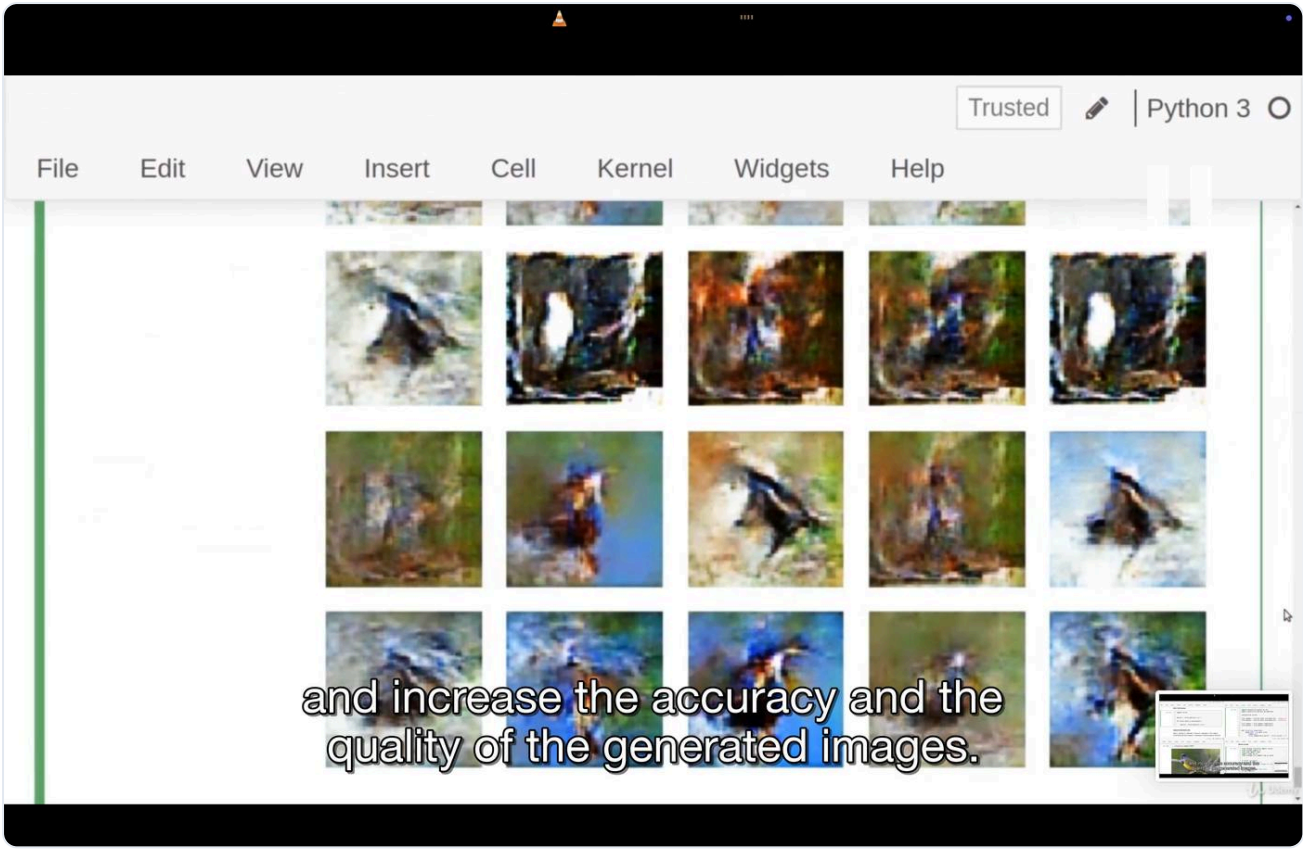


FIGURE 1.45

Moreover we introduce the idea of multiple loss functions and how to use them to fine tune the results and increase the accuracy and the quality of the generated images.

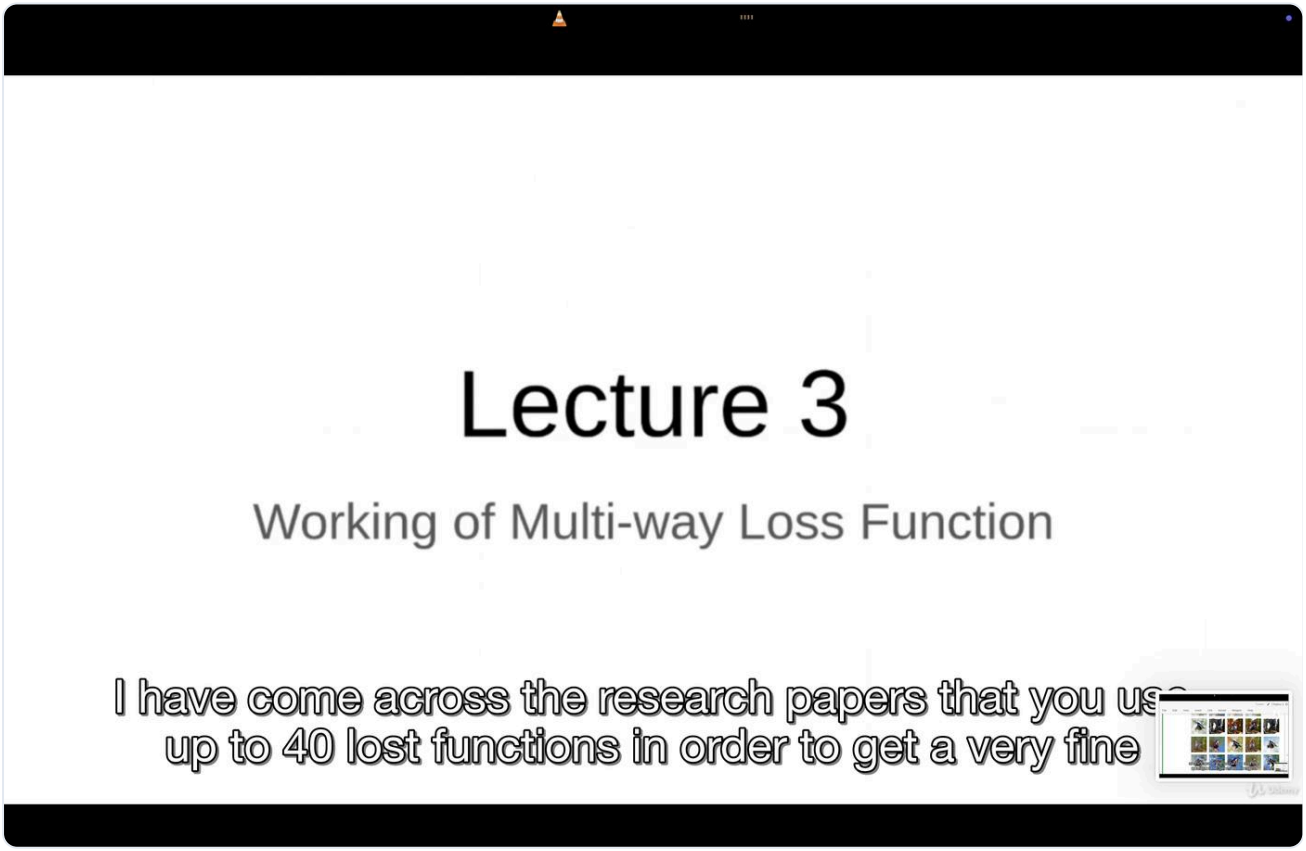


FIGURE 1.46 | Working of Multi-way Loss Function

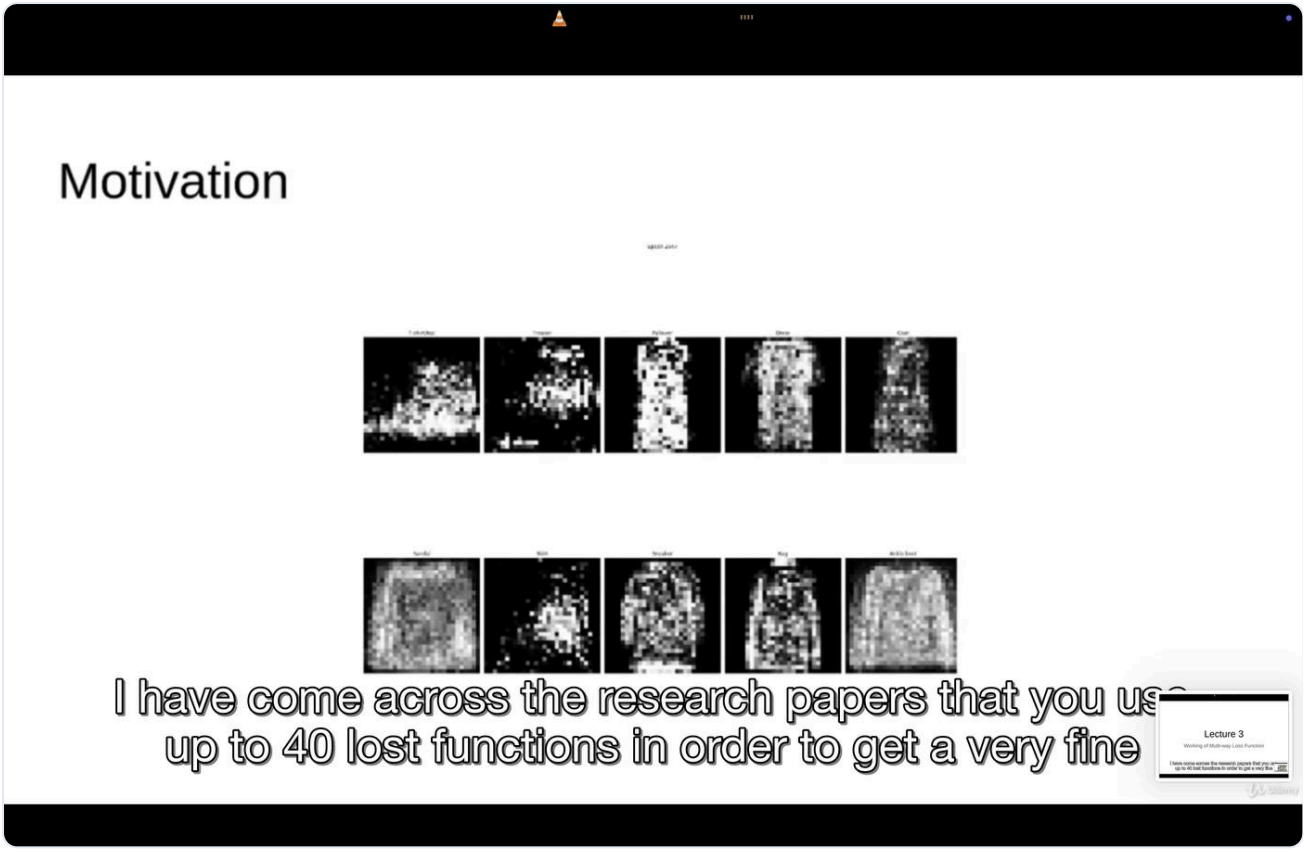


FIGURE 1.47

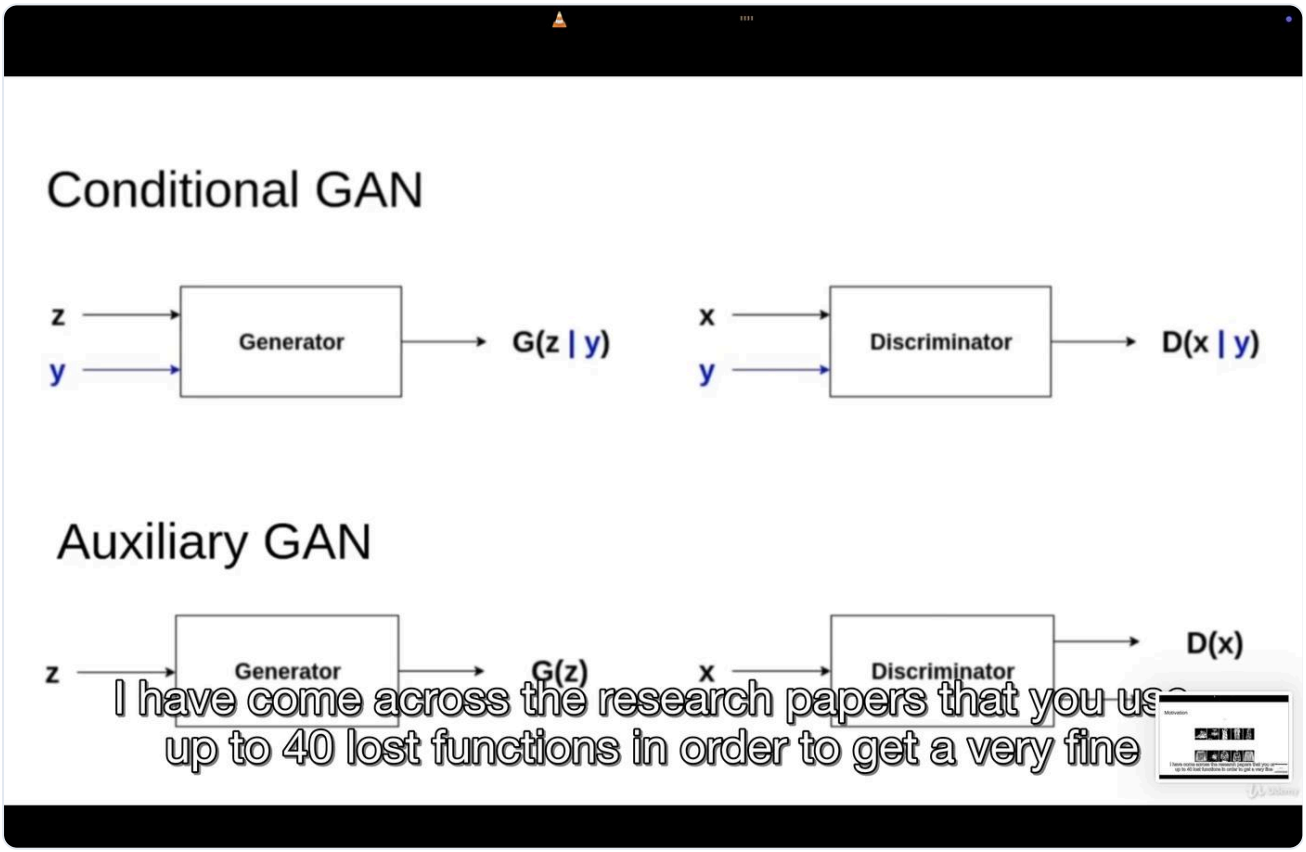


FIGURE 1.48

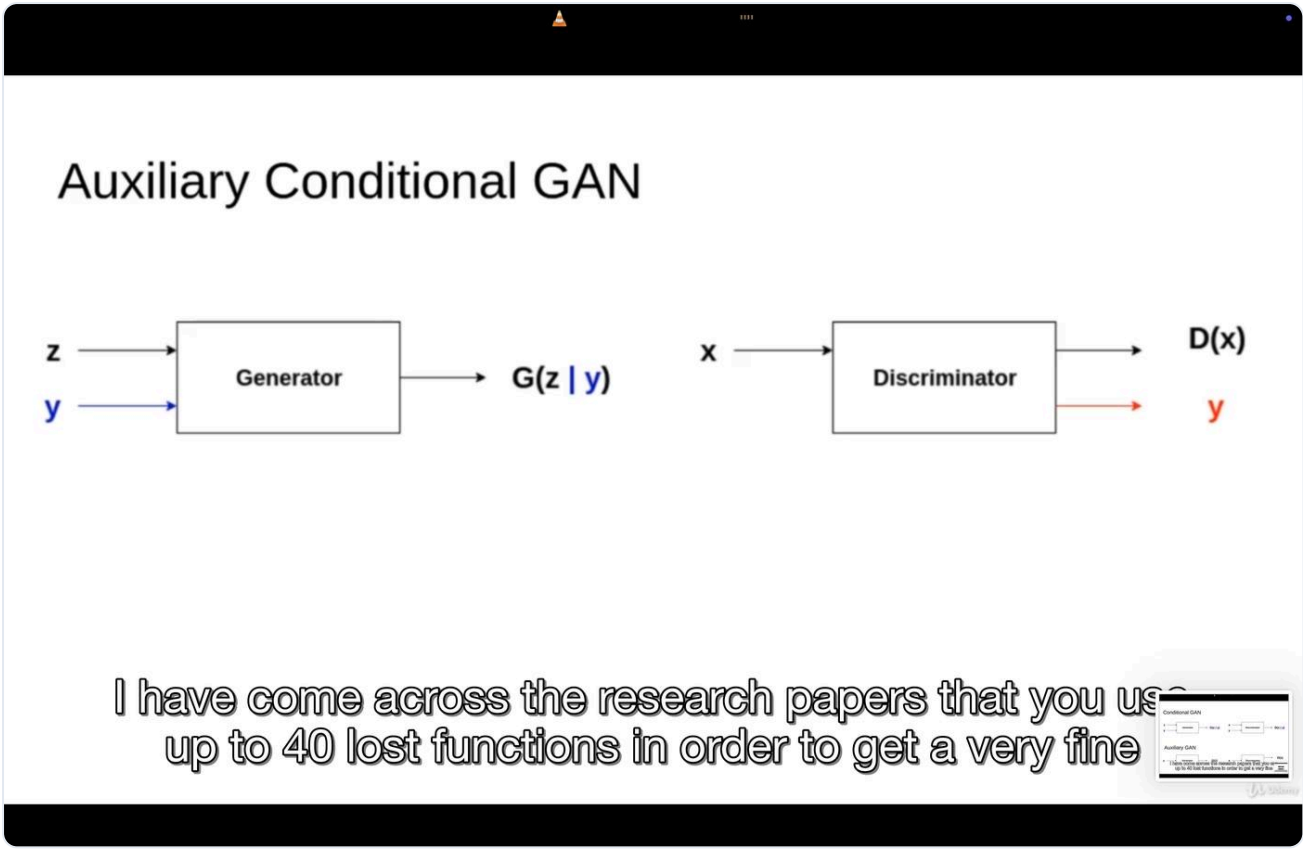


FIGURE 1.49

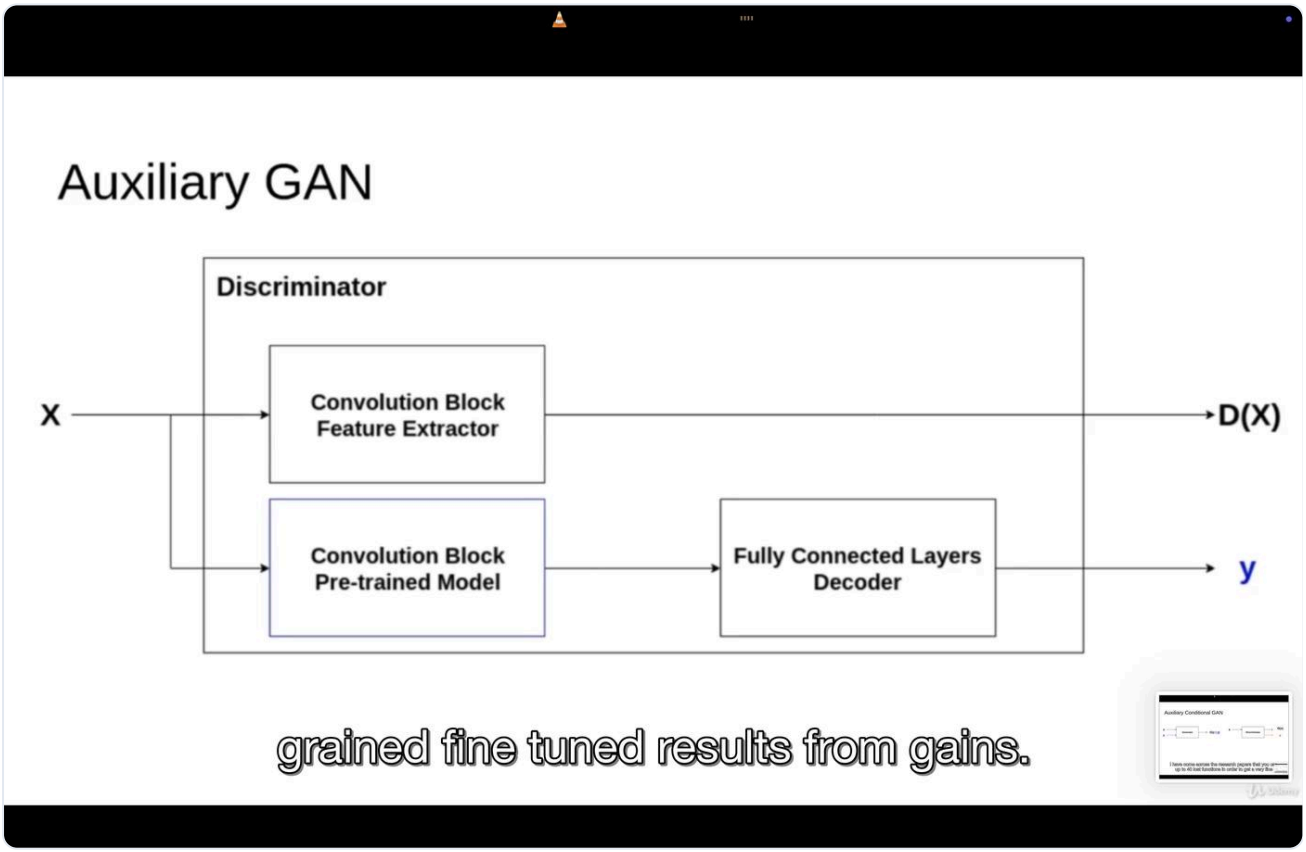


FIGURE 1.50 | Convolution Block

I have come across the research papers that you use up to 40 loss functions in order to get a very fine grained fine tuned results from GANs.

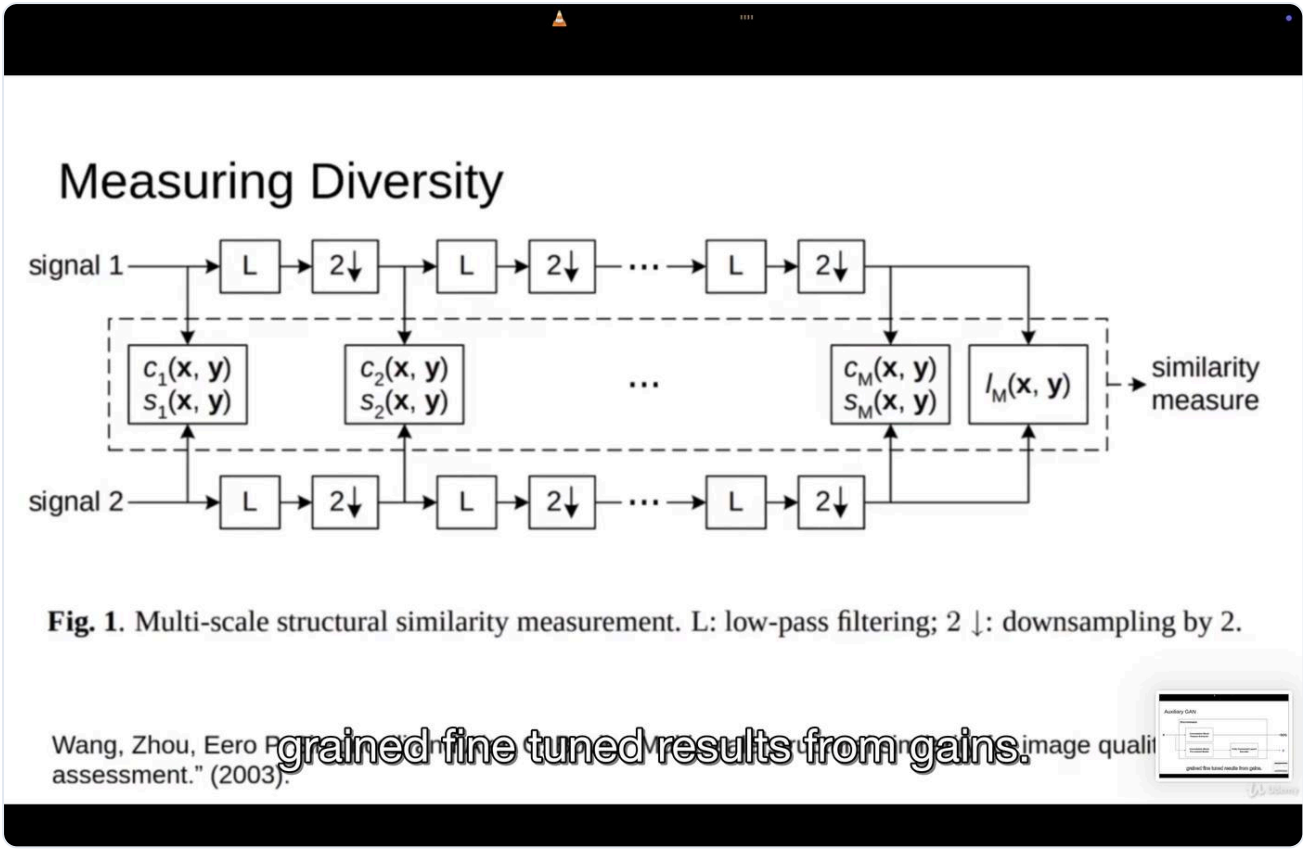


FIGURE 1.51

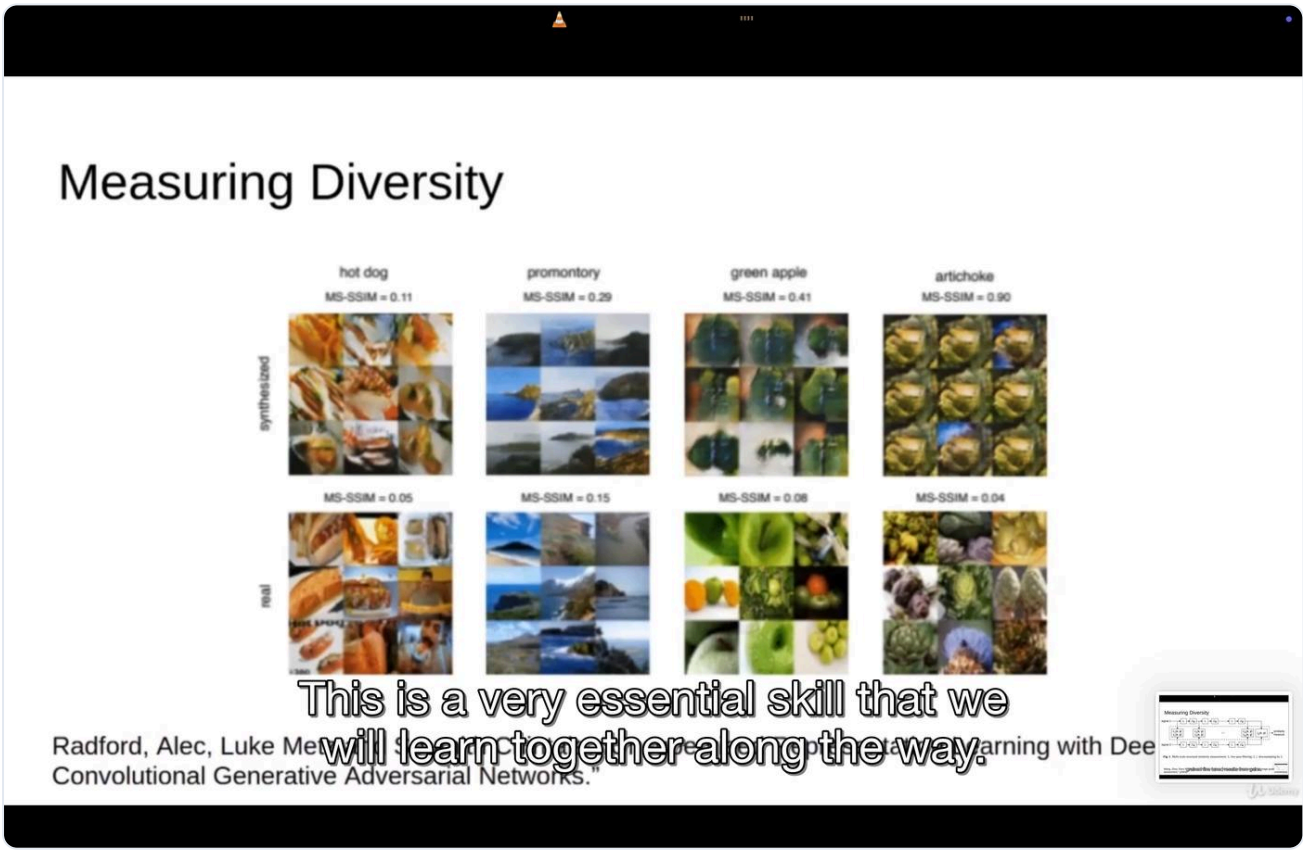


FIGURE 1.52

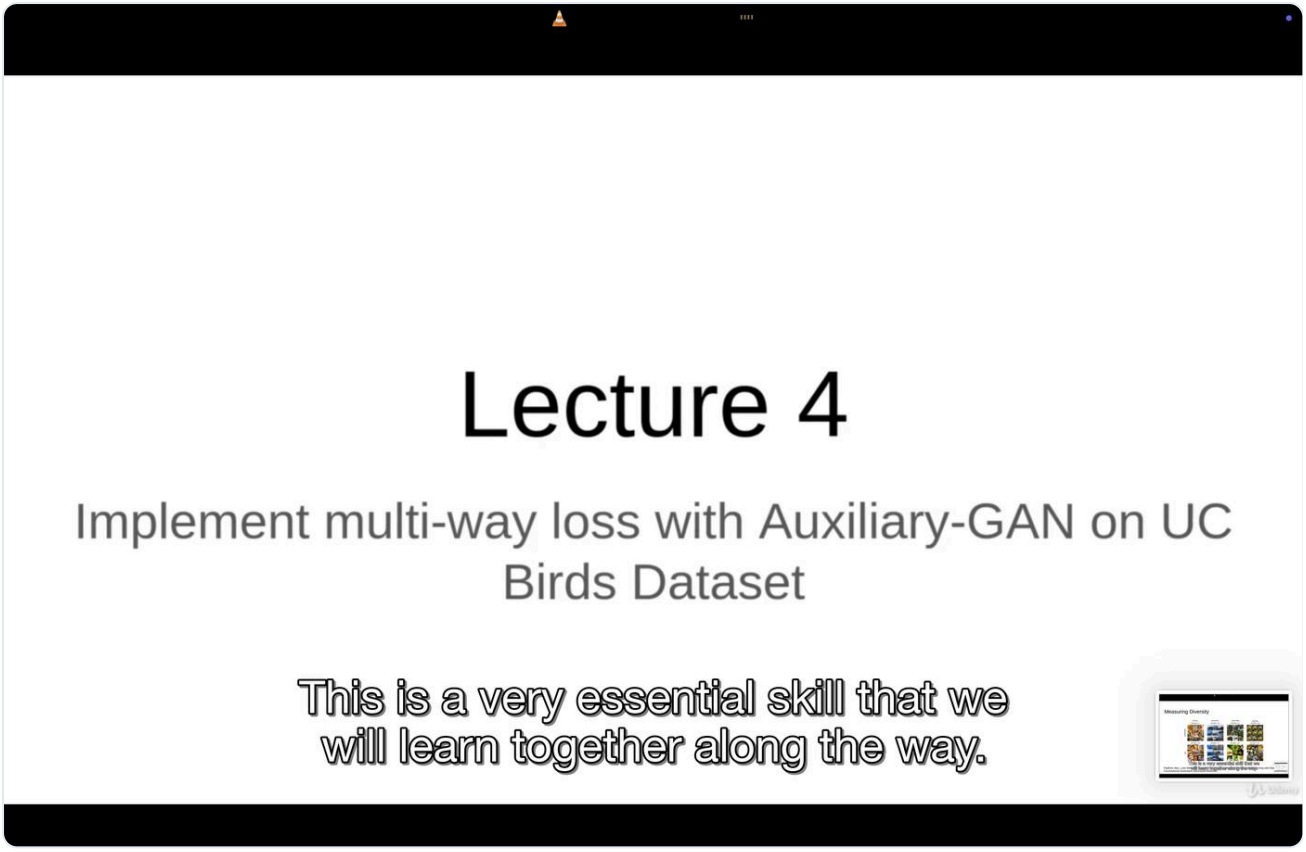


FIGURE 1.53 | Implement Multi-way Loss with Auxiliary-GAN on UC

This is a very essential skill that we will learn together along the way.

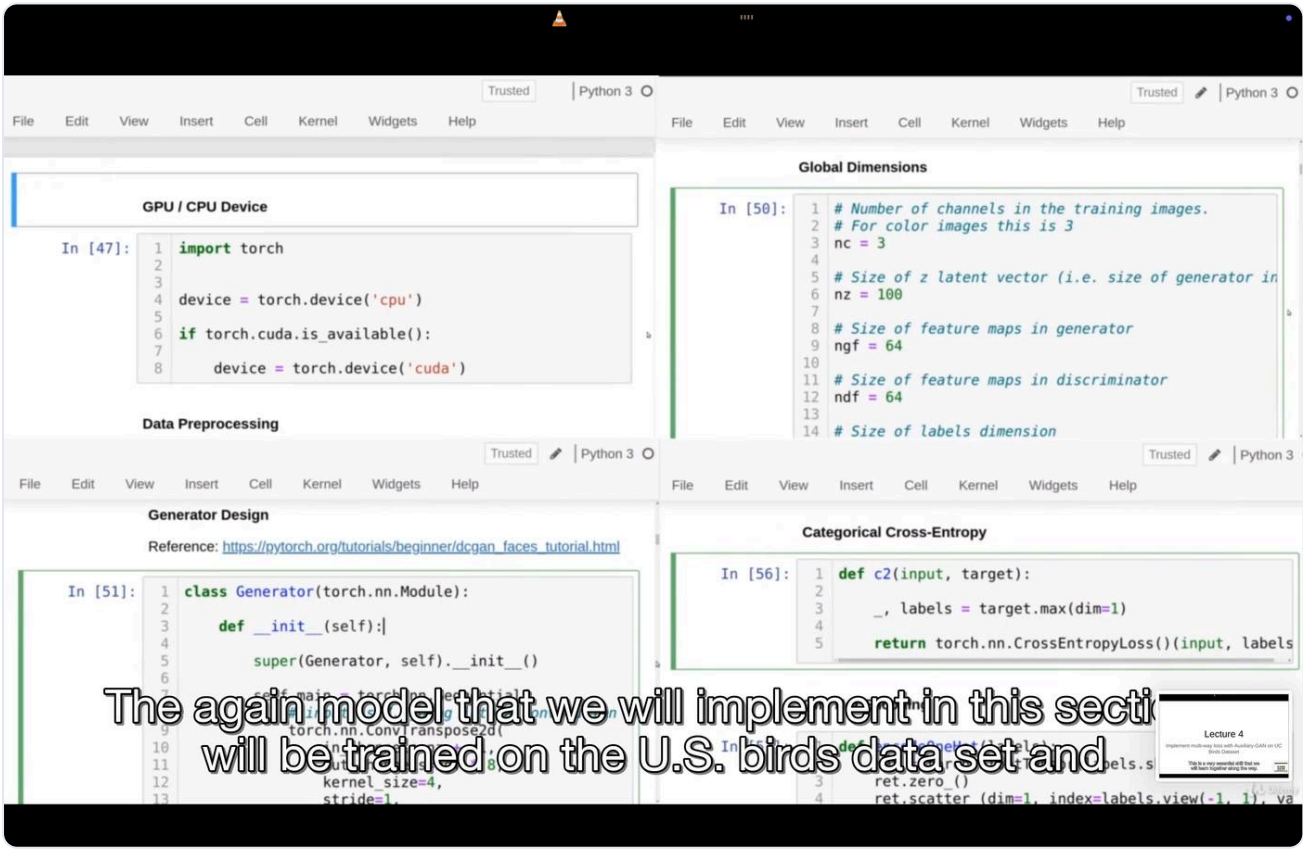


FIGURE 1.54 | I 47 Import Torch Nc 3

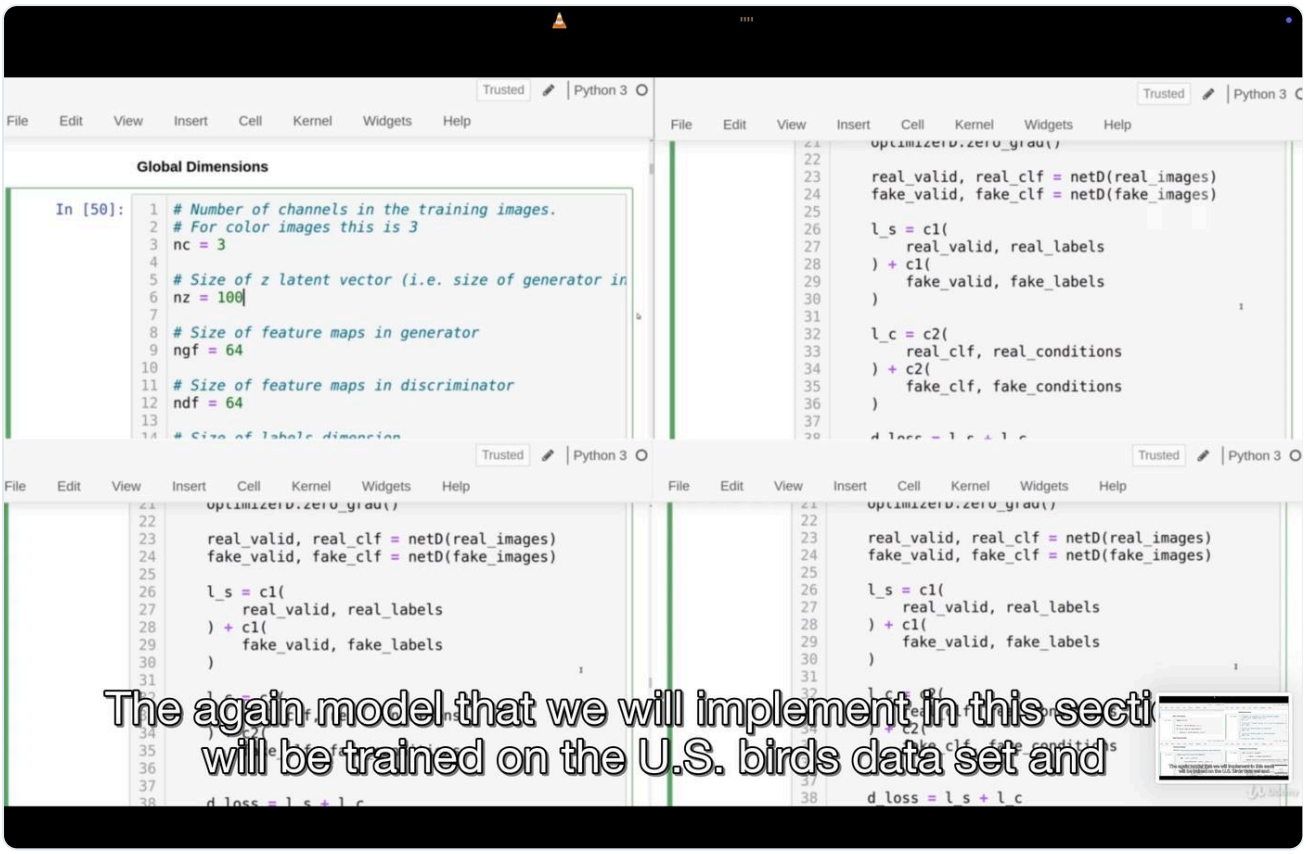


FIGURE 1.55

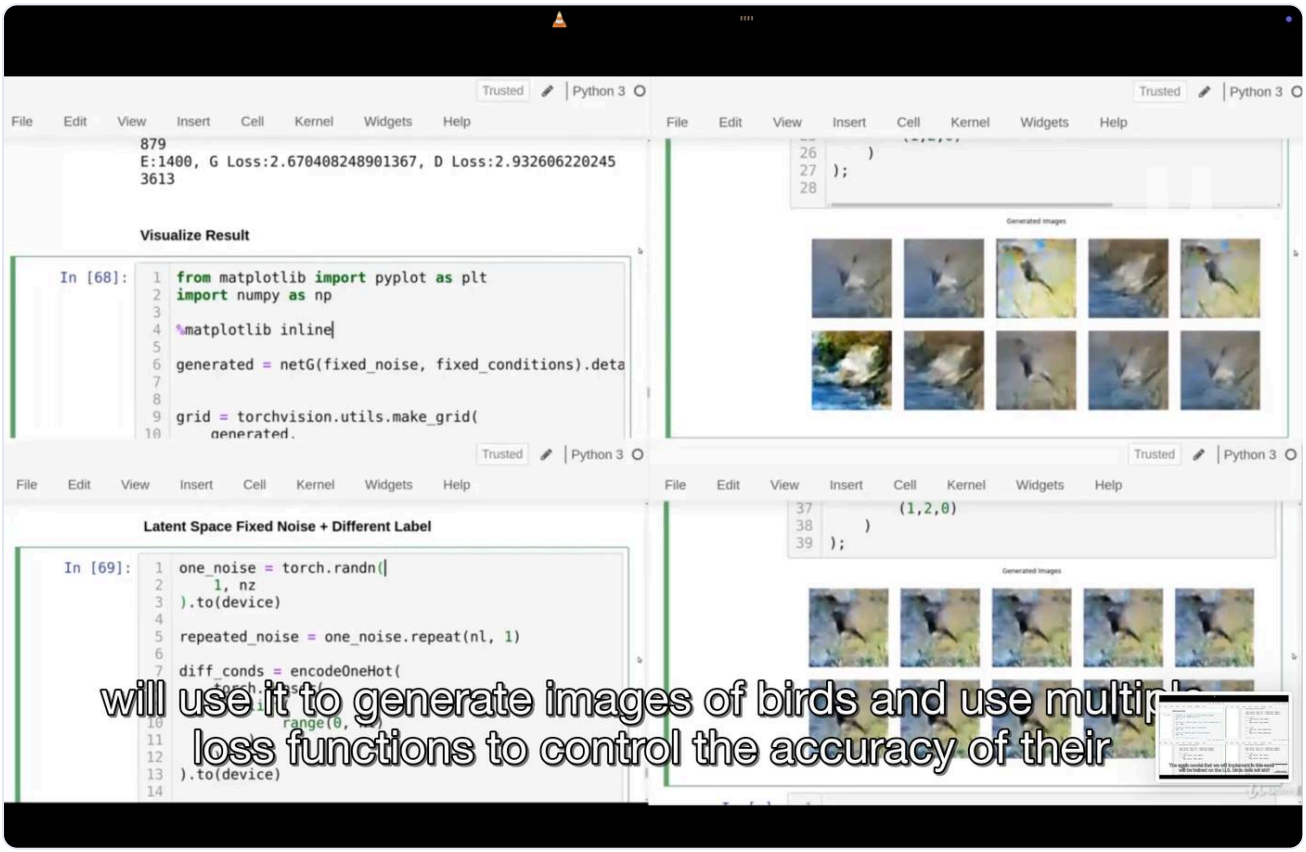


FIGURE 1.56 | Visualize Result

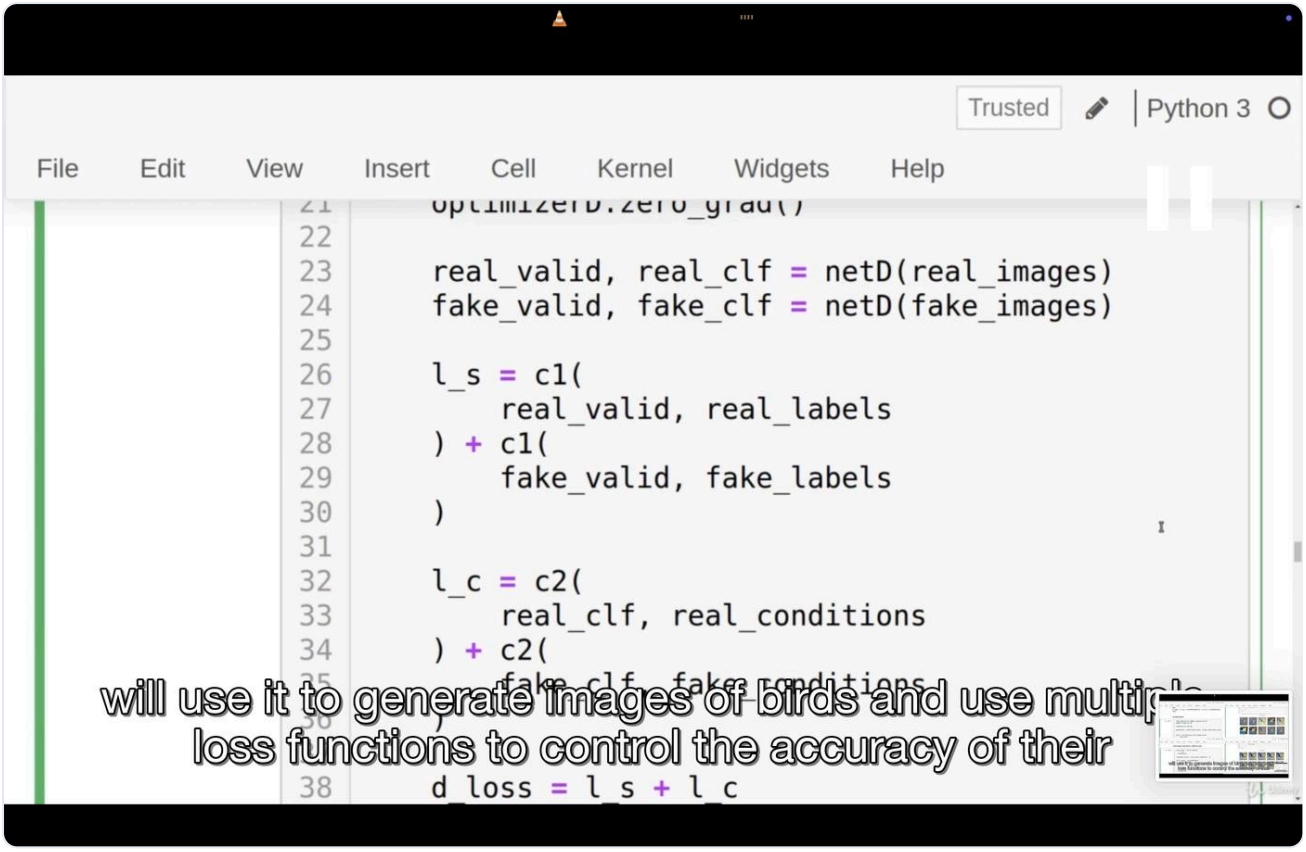


FIGURE 1.57

The GAN model that we will implement in this section will be trained on the UC Birds dataset and will use it to generate images of birds and use multiple loss functions to control the accuracy of their generator.

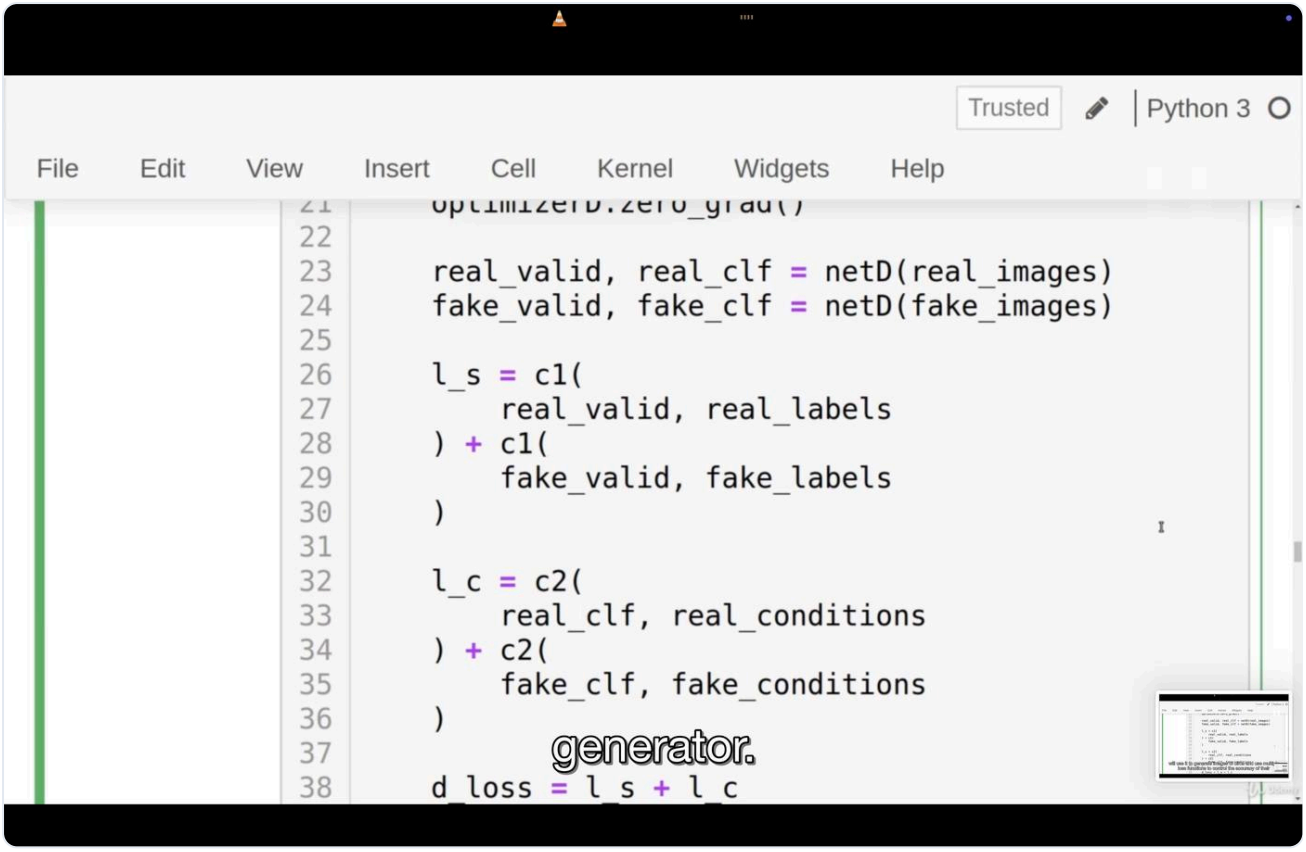


FIGURE 1.58

The first loss function will tell whether the image is real or fake.

The other loss function will tell whether the image belongs to the right class that was given as a condition to the discriminator or not.

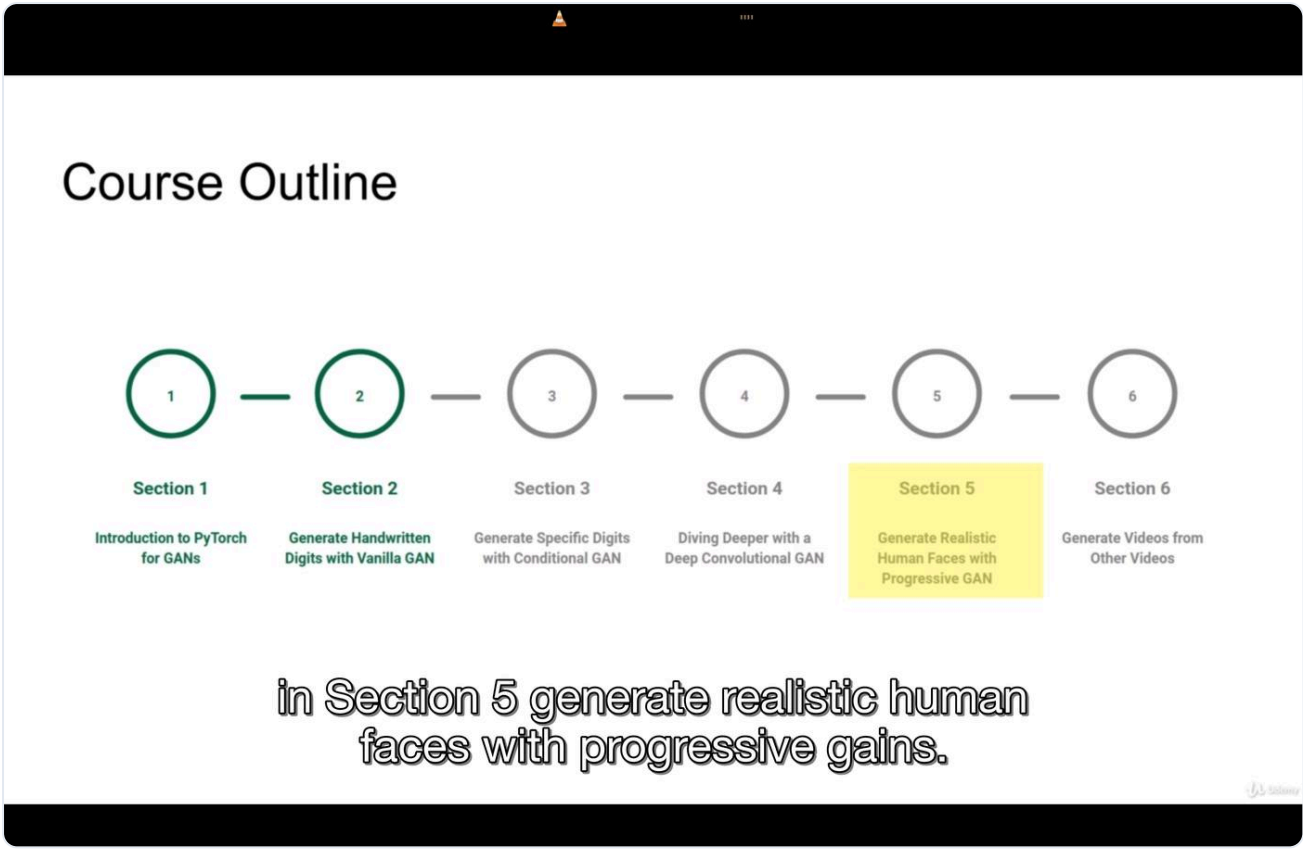


FIGURE 1.59 | Introduction to PyTorch Generate Handwritten

I will see how this improves the accuracy and the quality of the generated images by the generator model in Section 5 generate realistic human faces with progressive GANs.

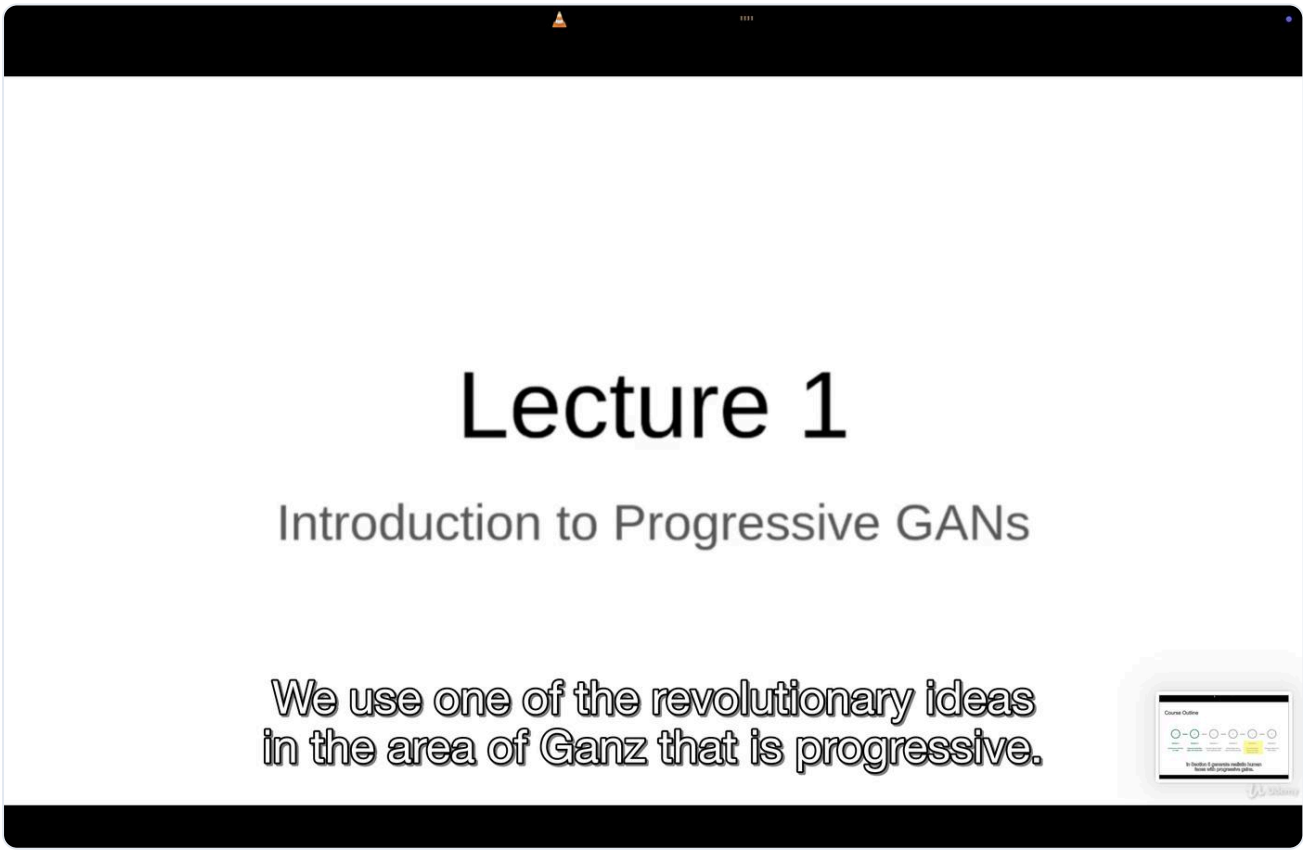


FIGURE 1.60 | Introduction to Progressive GANs

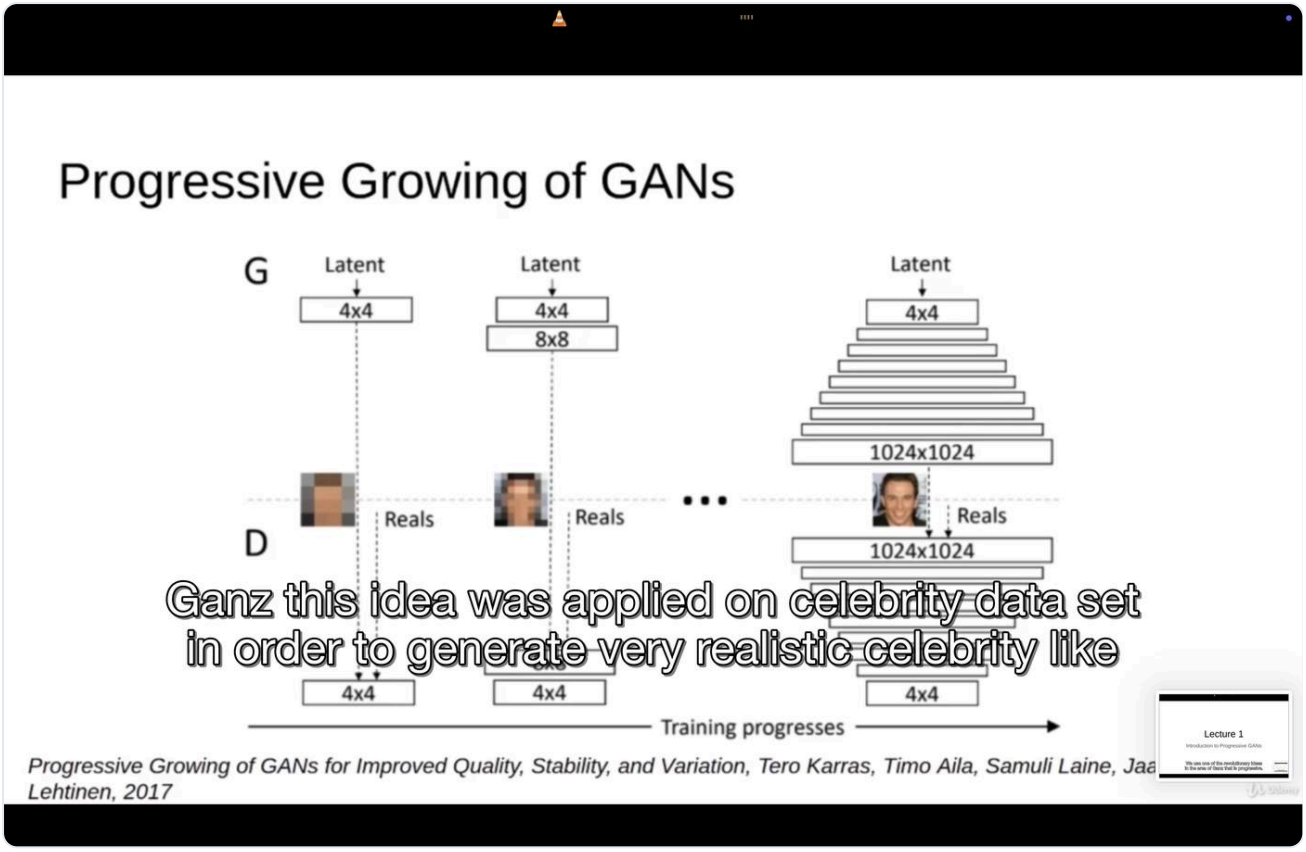


FIGURE 1.61

We use one of the revolutionary ideas in the area of GANs that is progressive.

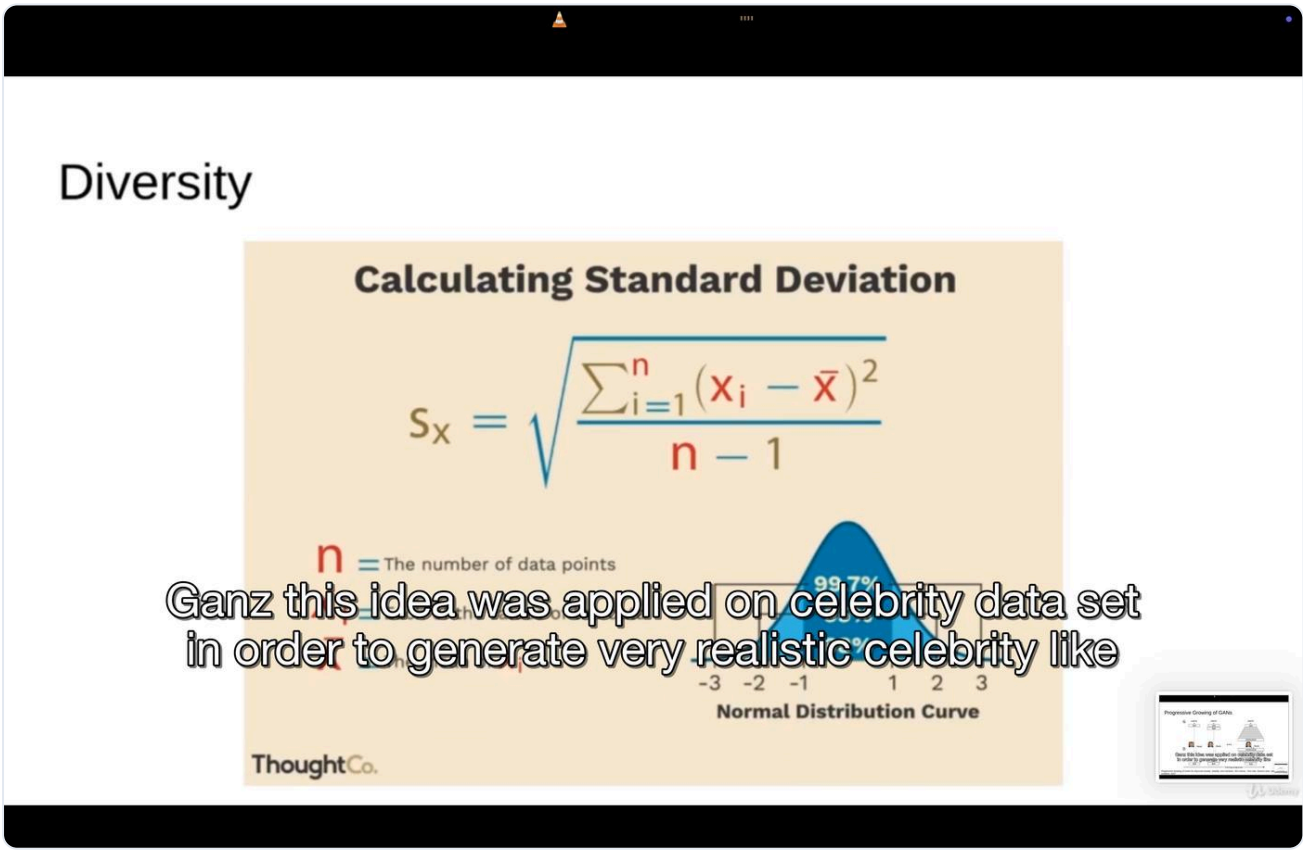


FIGURE 1.62

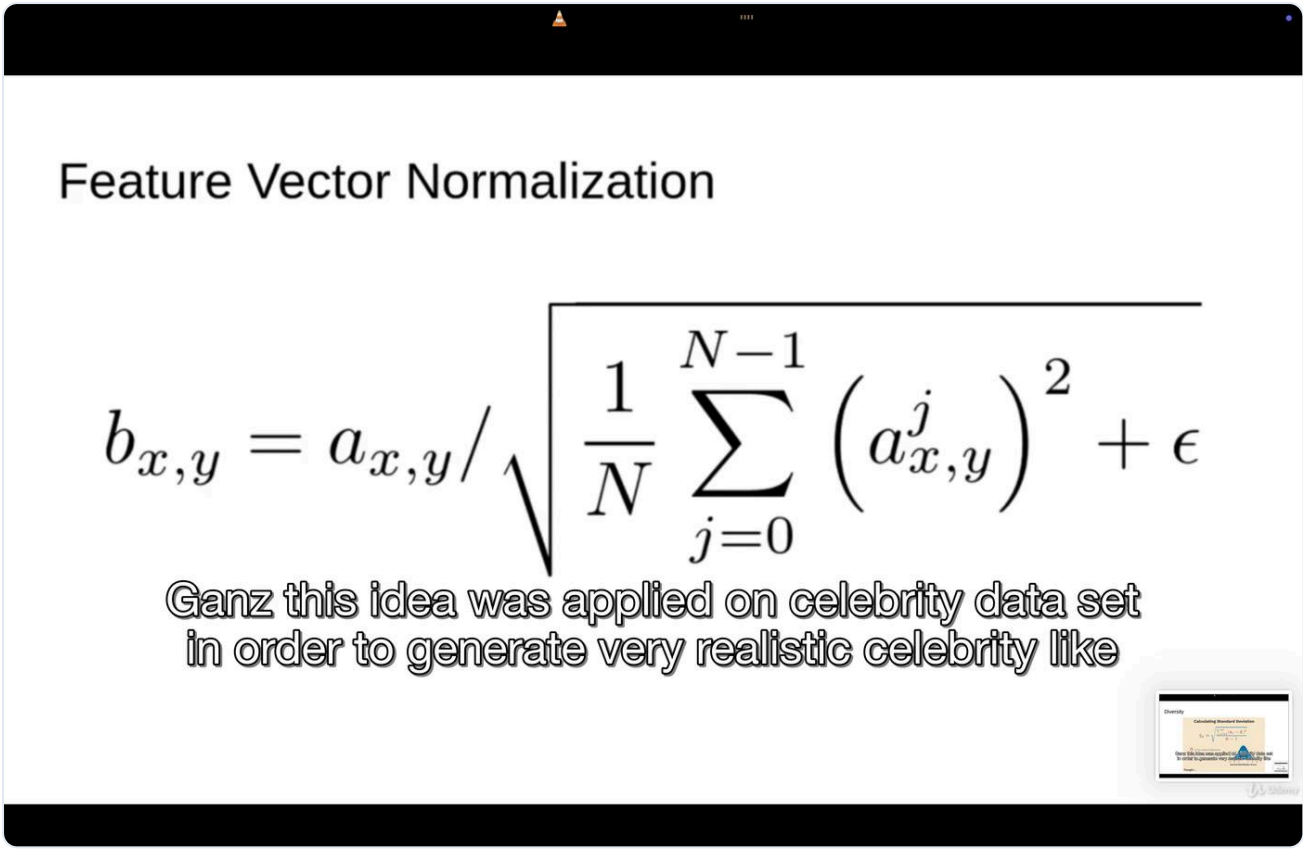


FIGURE 1.63 | Feature Vector Normalization

FIGURE 1.64

GANs this idea was applied on celebrity data set in order to generate very realistic celebrity like human faces even a human being will not be able to tell whether these images are real or fake.

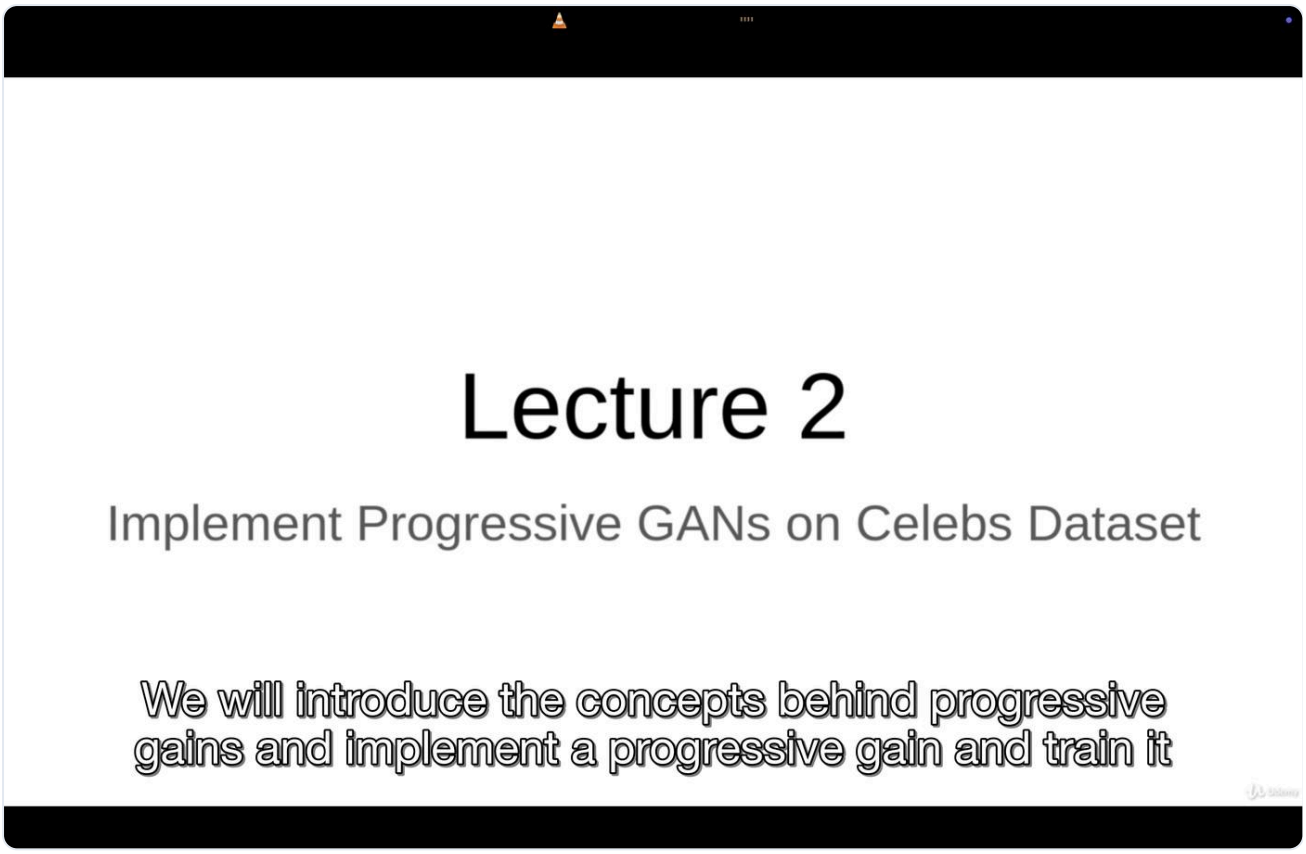


FIGURE 1.65 | Implement Progressive GANs on Celebs Dataset

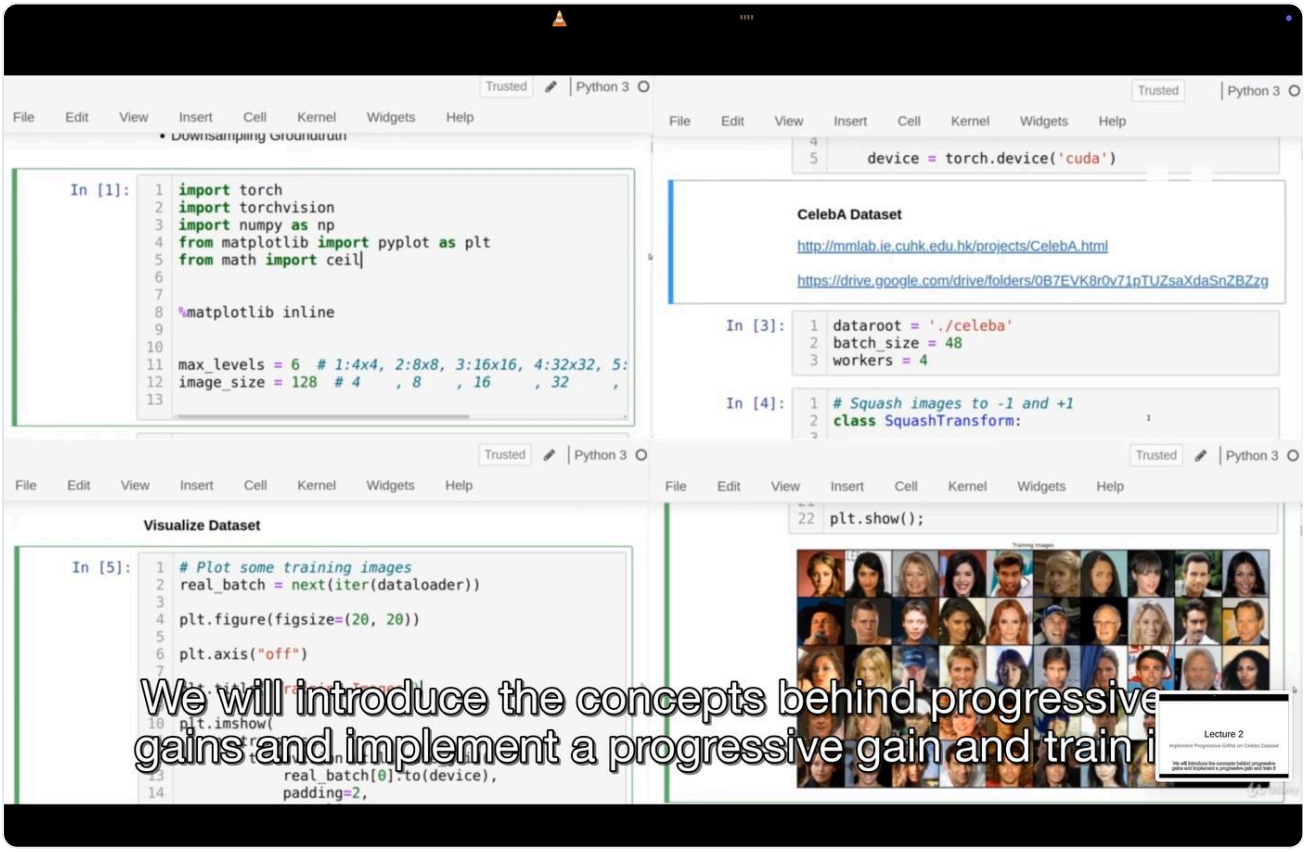


FIGURE 1.66

We will introduce the concepts behind progressive GANs and implement a progressive GAN and train it on the celebrities dataset.

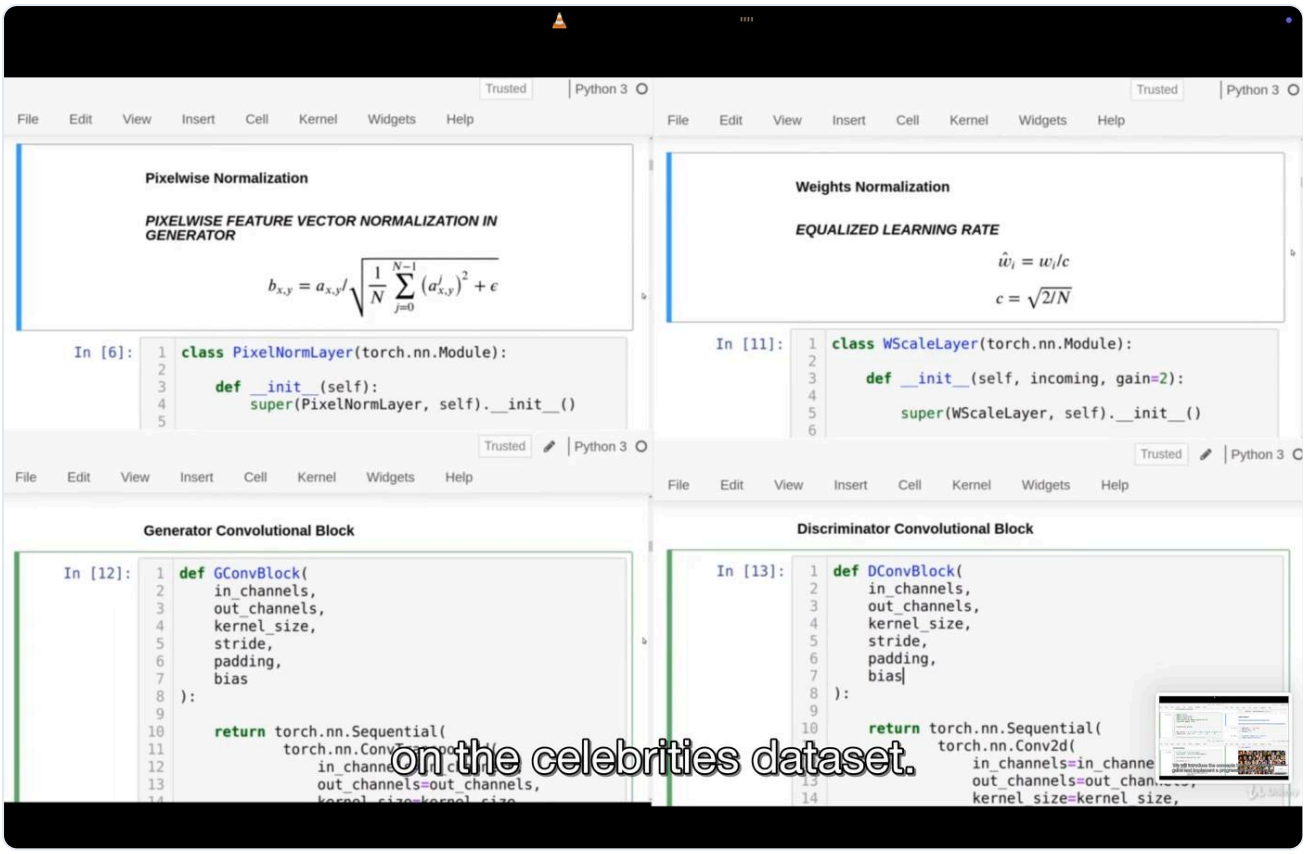


FIGURE 1.67

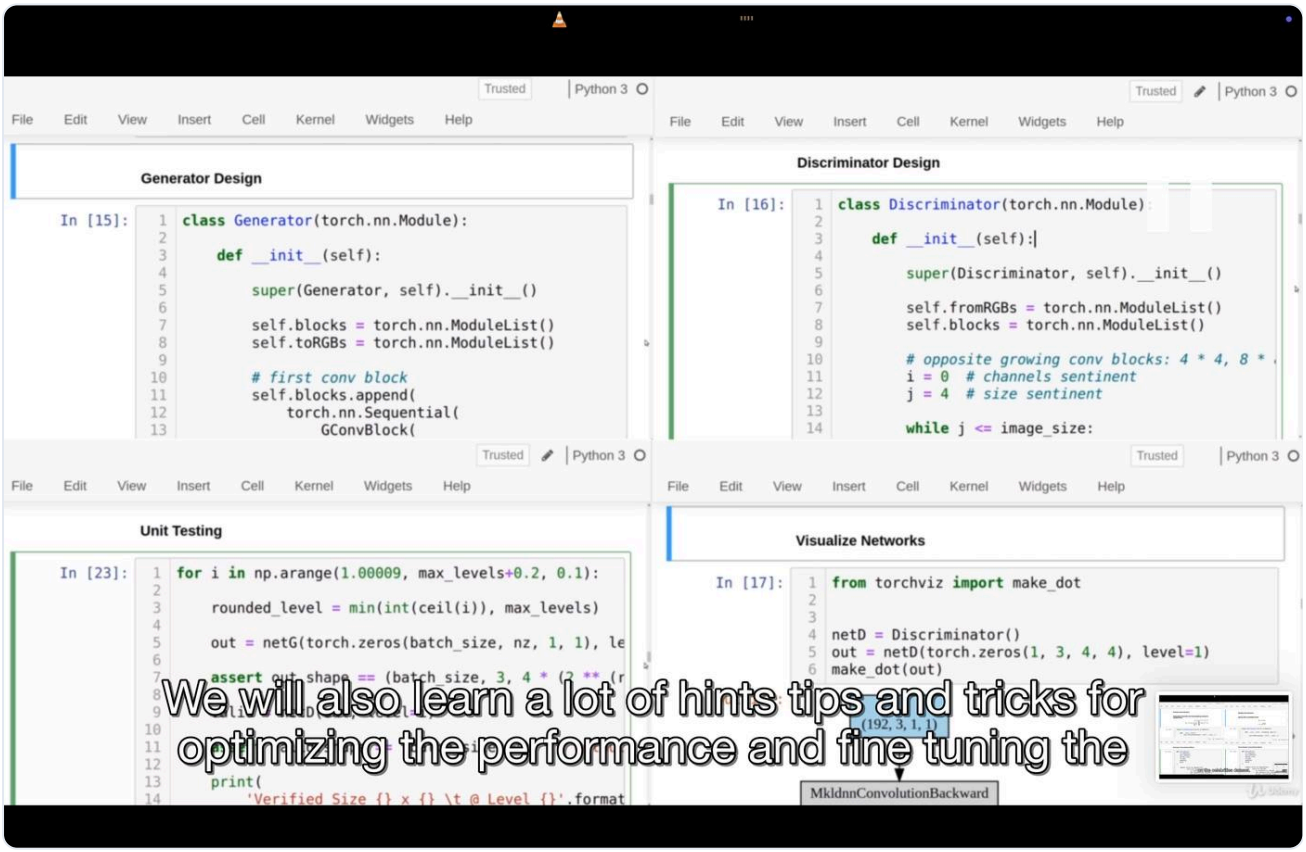


FIGURE 1.68 | Generator Design

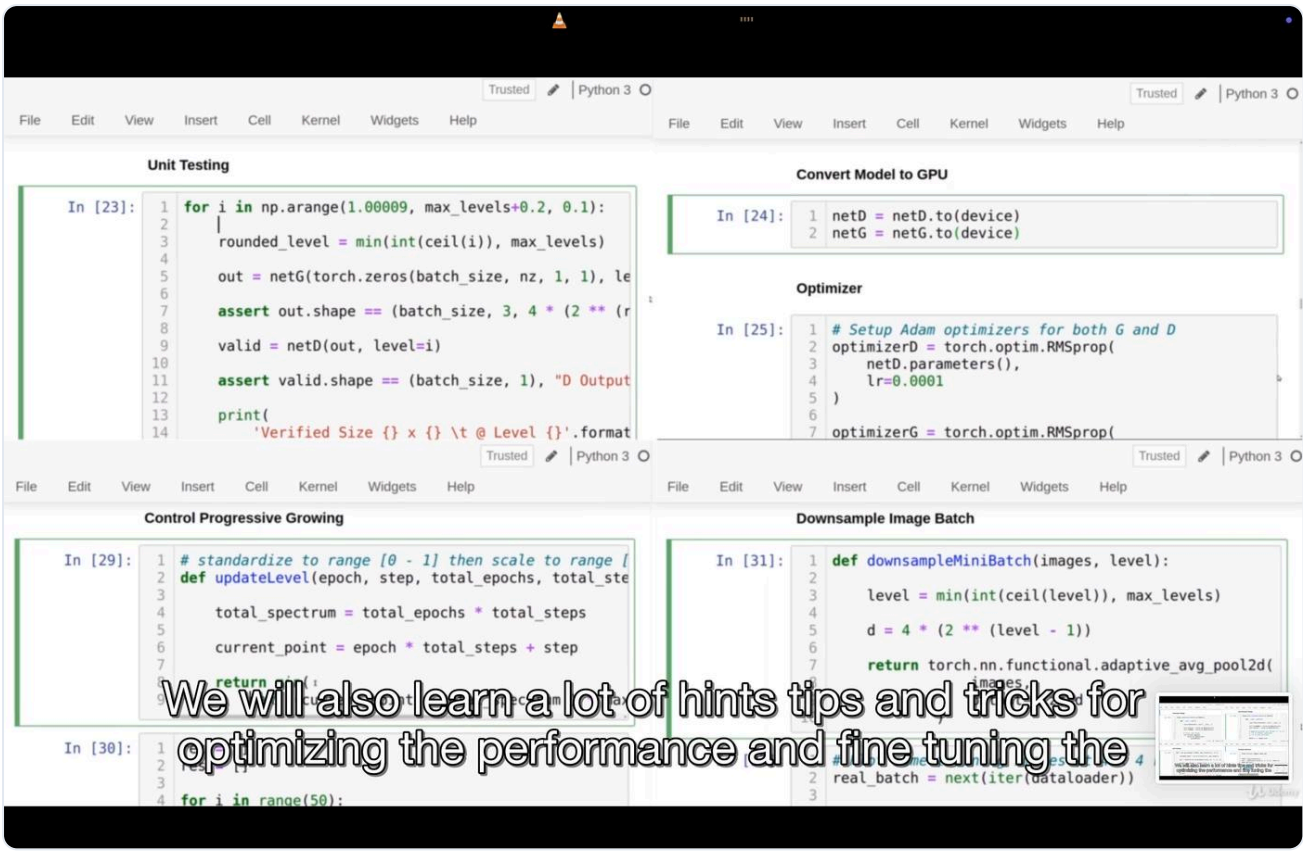


FIGURE 1.69

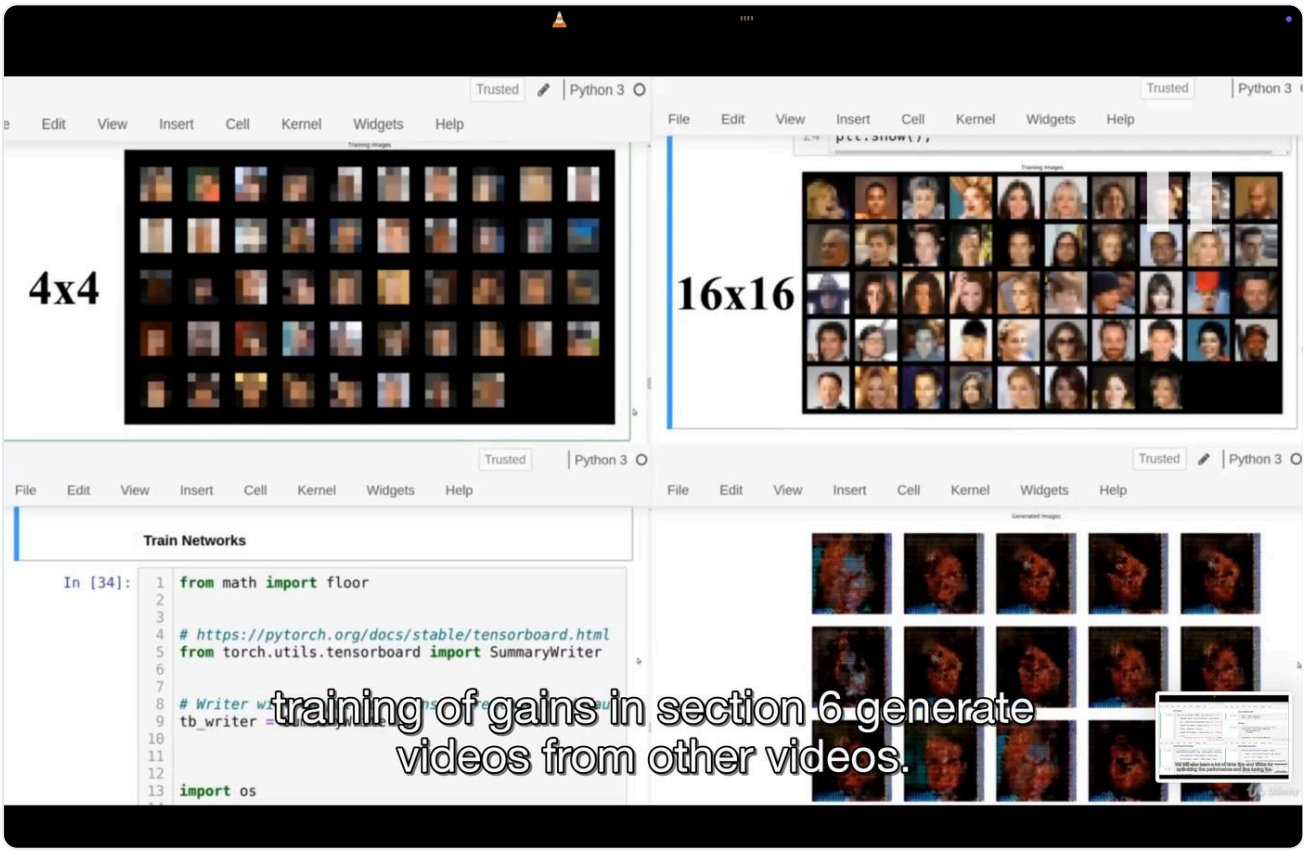


FIGURE 1.70 | Train Networks

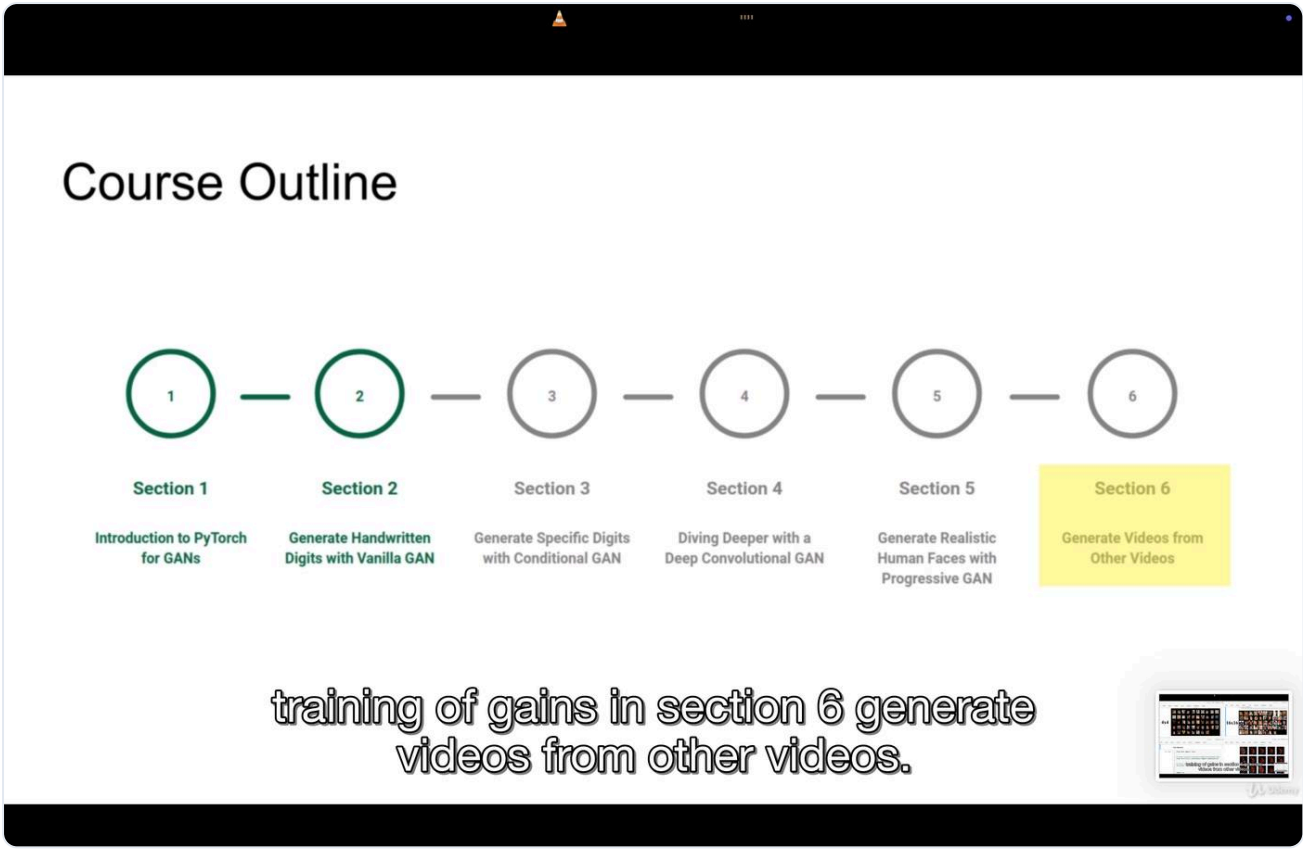


FIGURE 1.71 | Introduction to PyTorch Generate Handwritten

We will also learn a lot of hints tips and tricks for optimizing the performance and fine tuning the training of GANs in section 6 generate videos from other videos.

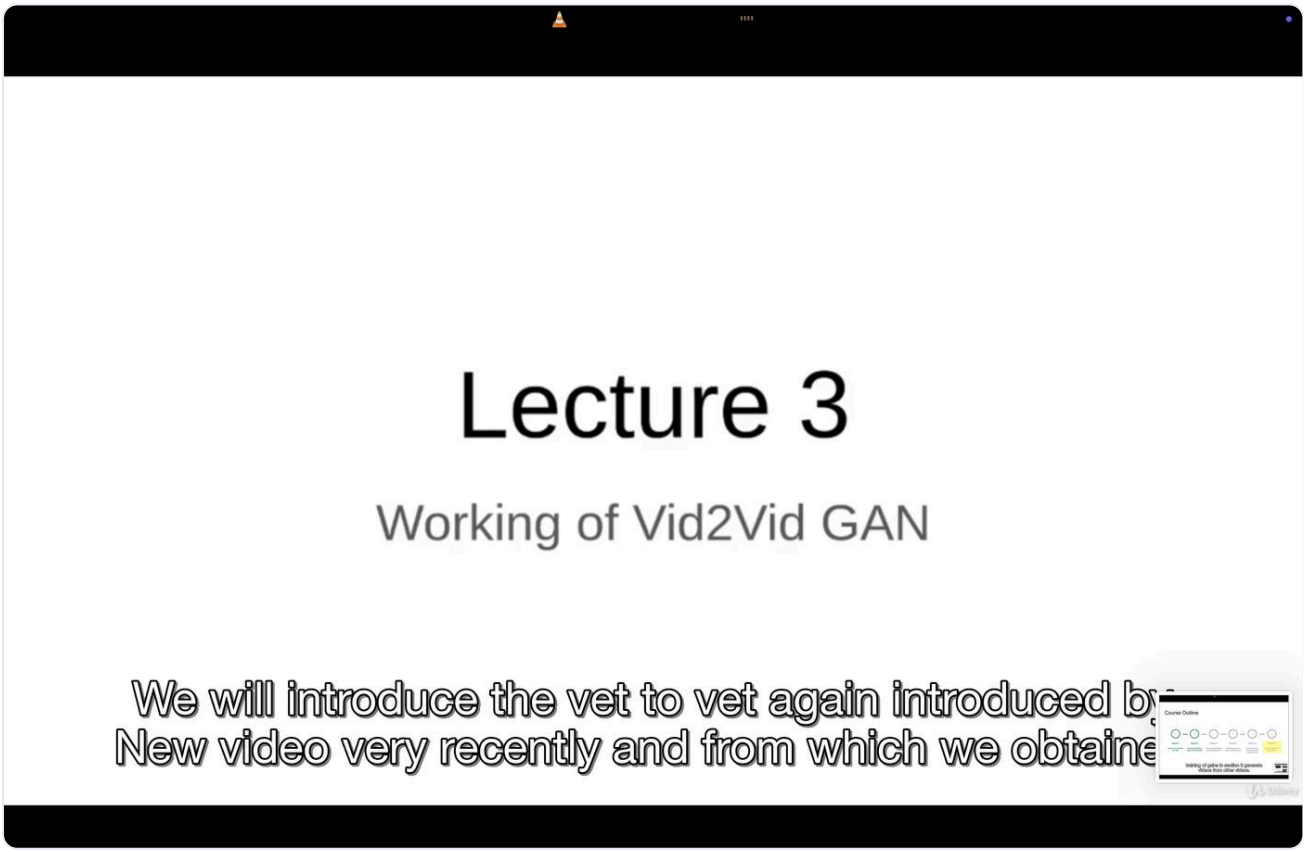


FIGURE 1.72 | Working of Vid2Vid GAN

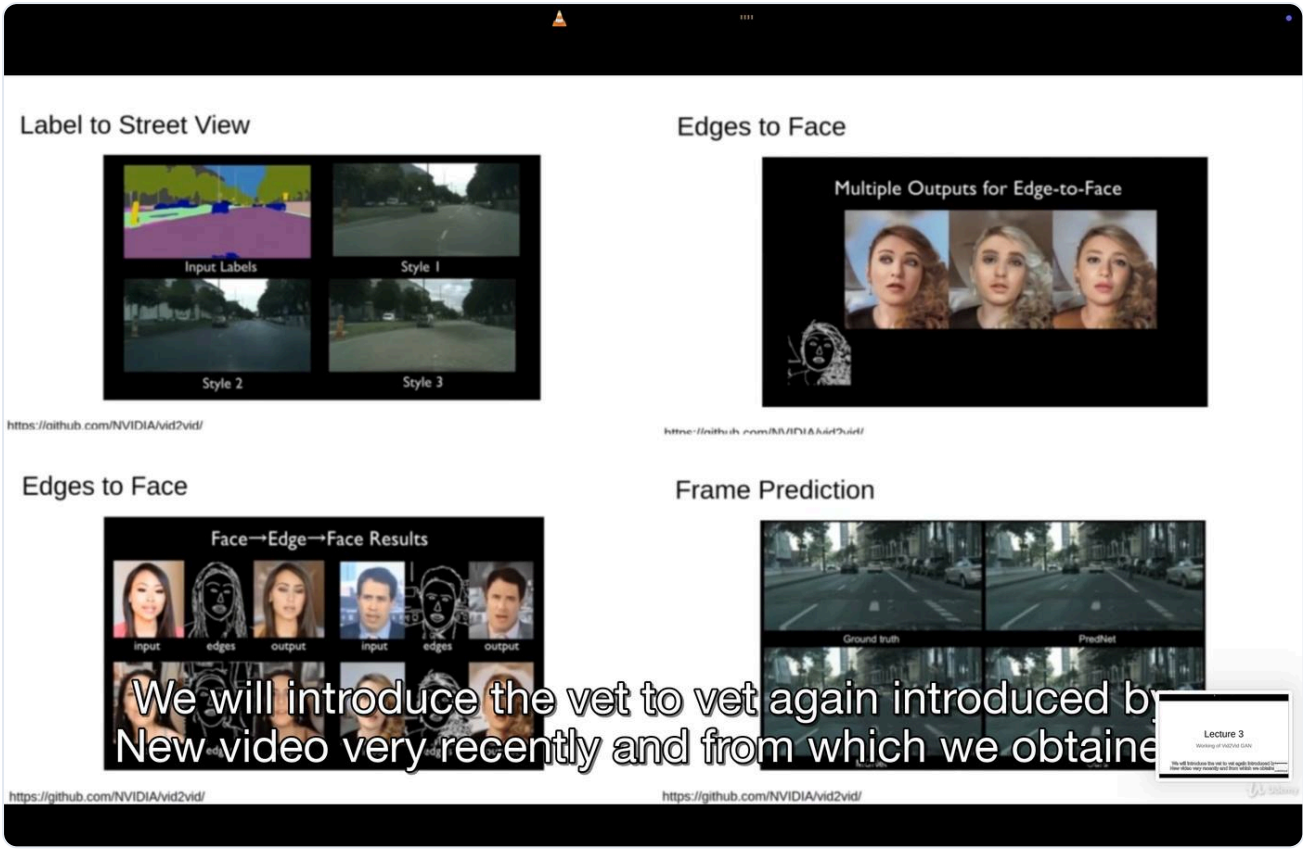


FIGURE 1.73 | Label to Street View Edges to Face

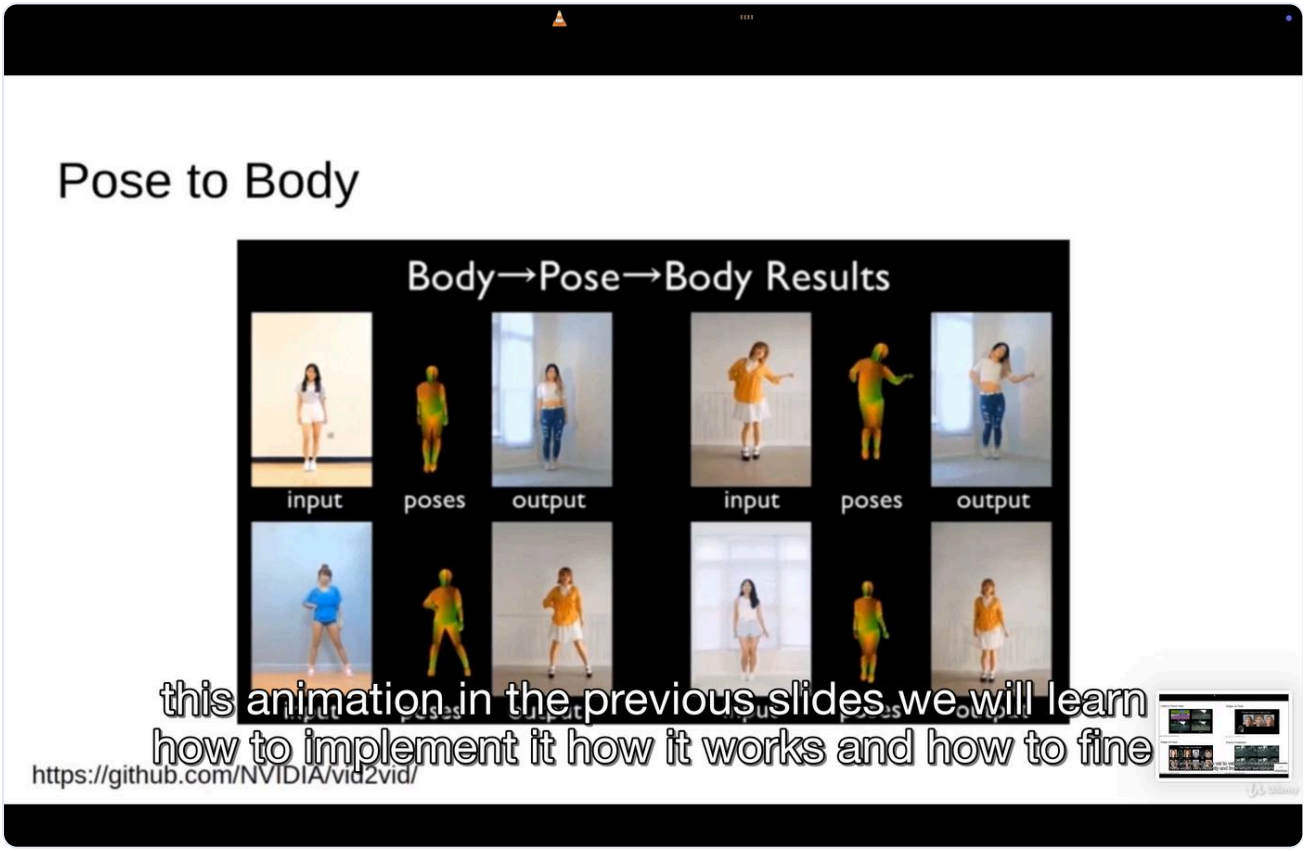


FIGURE 1.74

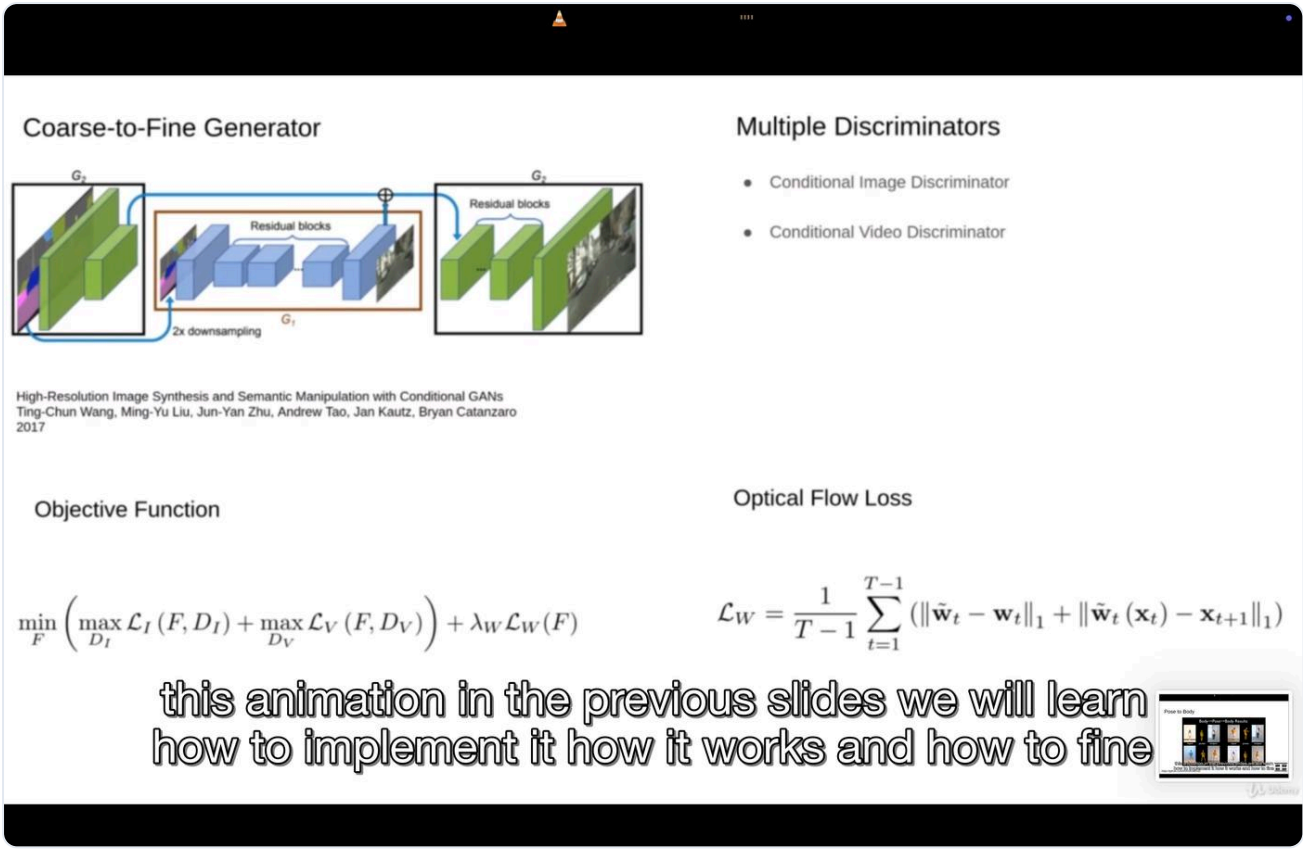


FIGURE 1.75

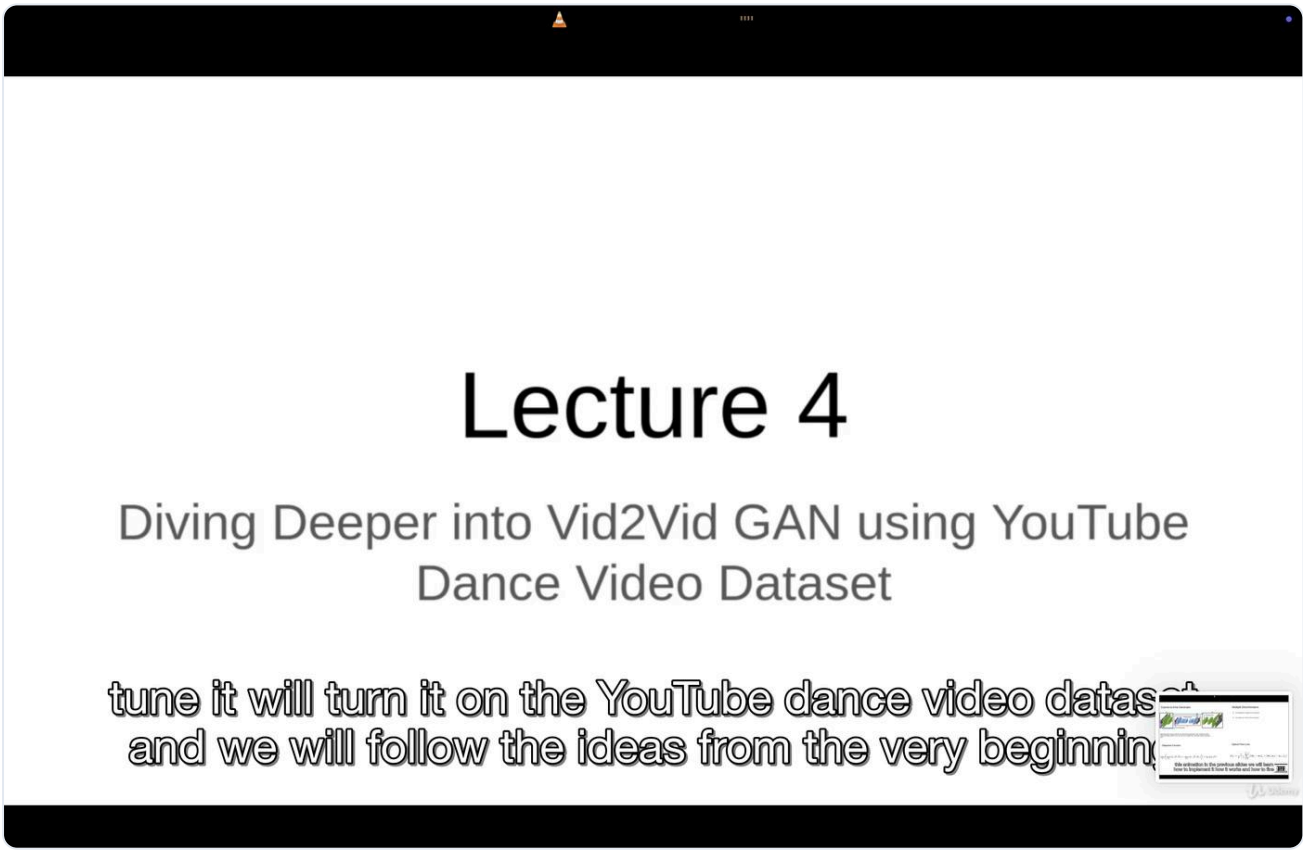


FIGURE 1.76 | Diving Deeper into Vid2Vid GAN Using YouTube

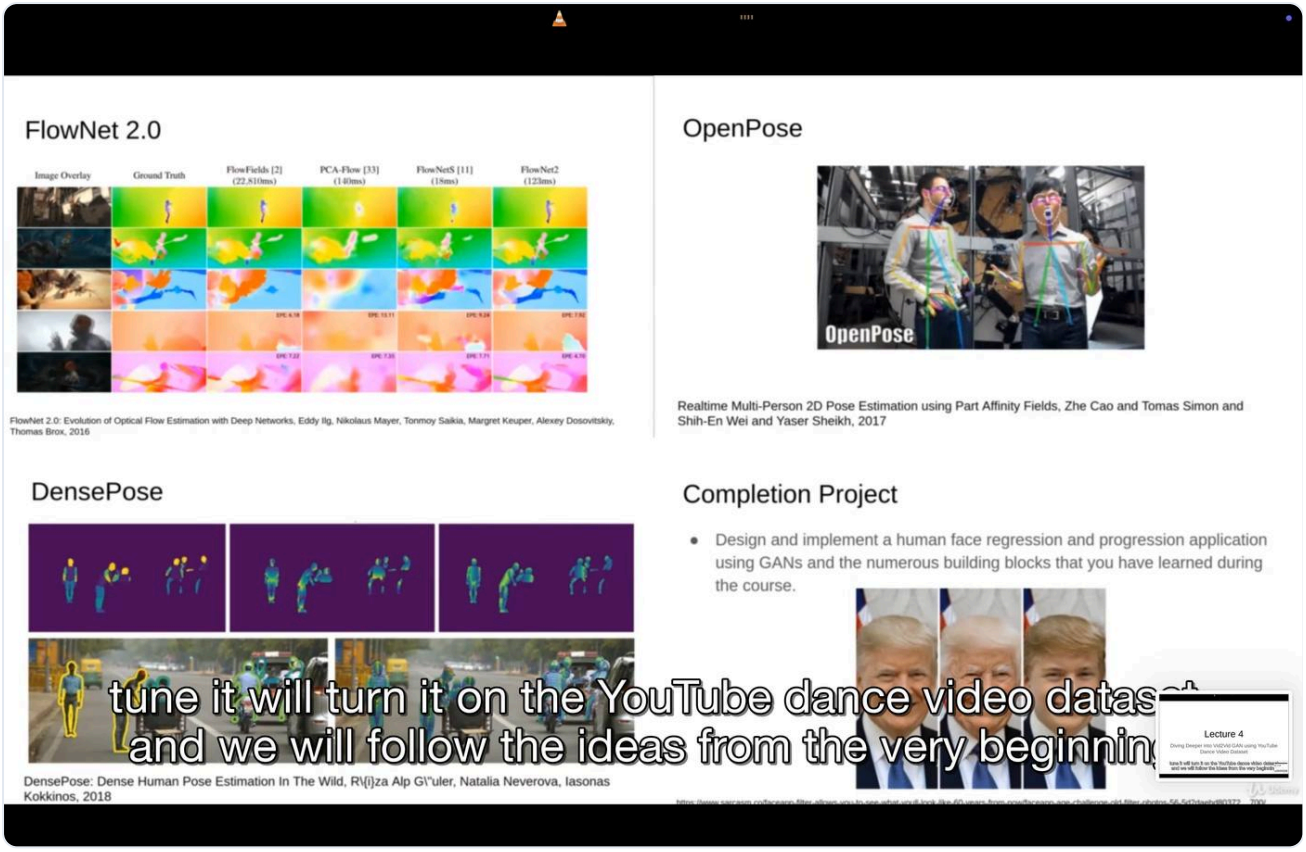


FIGURE 1.77

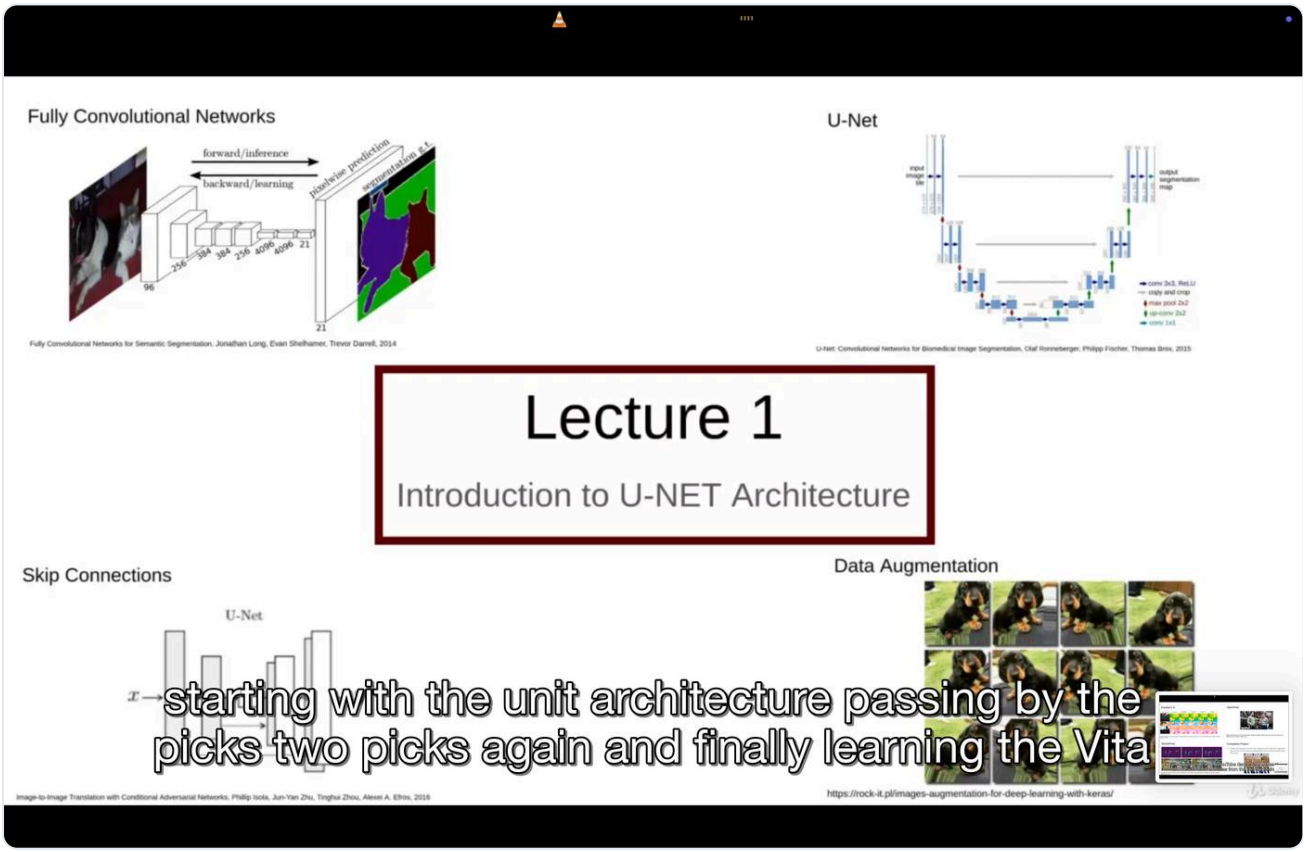


FIGURE 1.78

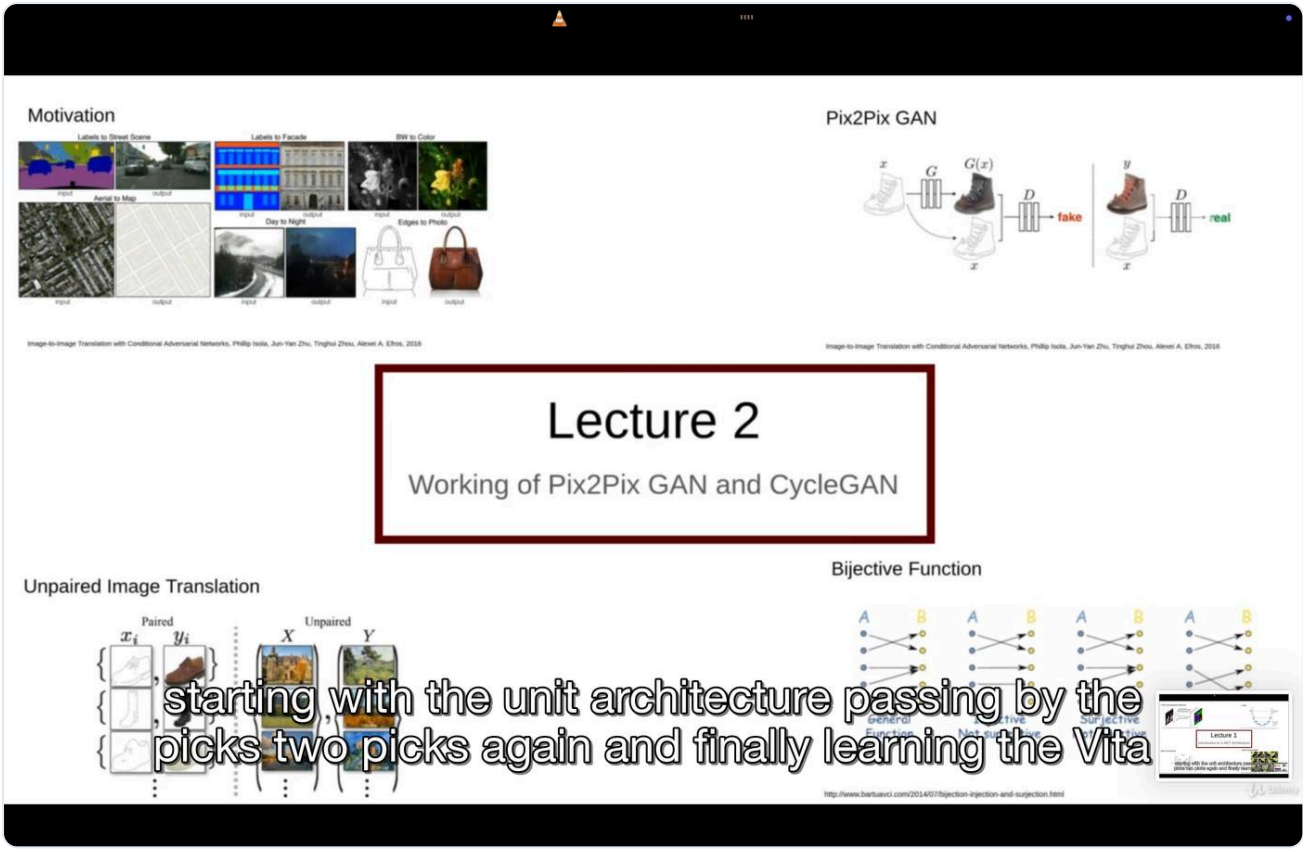


FIGURE 1.79 | Working of Pix2Pix GAN and CycleGAN

We will introduce the Vid2Vid GAN introduced by NVIDIA very recently and from which we obtained this animation in the previous slides we will learn how to implement it how it works and how to fine tune it will turn it on the YouTube dance video dataset and we will follow the ideas from the very beginning starting with the U-Net architecture passing by the Pix2Pix GAN and finally learning the Vid2Vid GAN by the end of this course you will have learned a lot of concepts and get your hands on a lot of projects that will help you in your career path and also in your future research I shall be waiting for you in the first section.

Let's get started.

2 · PyTorch Forward and Backward Propagation

See the notebook `1.Lecture-1-Forward-Backward-Propagation.ipynb` for the code.

Hello and welcome to the first section. Together we shall embark on a journey through the advanced and the complex generative adversarial networks. We will learn the basic building blocks that you can stitch together to build any kind of GAN you dream of. This section is a refresher on the PyTorch deep learning library. But you will find it amusing even if you are already familiar with PyTorch. This section concludes with a video lecture on how to build a custom loss function.

This skill is essential for developing your own GANs and in general Deep Learning Solutions and in the first lecture Pytorch forward and backward propagation, we shall review the very first basics of deep learning and implement LeNet-5 architecture let's get started in order to get the most

out of this course. I have a few hints for you. First download the course materials and the coding exercises and open them on another screen as you watch the video lectures.

Feel free to post the lectures and make changes to the code and see how it affects the output. Second there are lots of reading materials that I attach both on slides and in the course notebooks. I believe you can learn a lot by reading through them. So don't miss that experience. Third there would be coding exercises that will not be graded but it will give you experience and challenge your understanding of the content explained in this course. Finally feel free to join the Slack channel of this course to discuss the assignments share ideas ask questions and collaborate with other learners who have taking this course.

The back propagation algorithm is the learning mechanism that allows the machines to reach such astonishing levels of accuracy. It starts with input, inputs represented as a vector x that is fed into the network. The inputs are multiplied by the weights of the synapses between the neurons then the output of the product is summed and passed through a non-linear activation function that allows the neural networks to outperform a traditional machine learning techniques in that we can learn a representation for non-linear manifolds.

This is called the forward path. The results are then compared again to the ground truth responses using an objective function. The loss which is then used to update the weights in the backward path using the chain rule for calculating partial derivatives. I would highly recommend reviewing the chain rule from any reference on elementary calculus in the year 1998. LeCun and his research team published the LeNet-5 architecture for handwritten digits recognition.

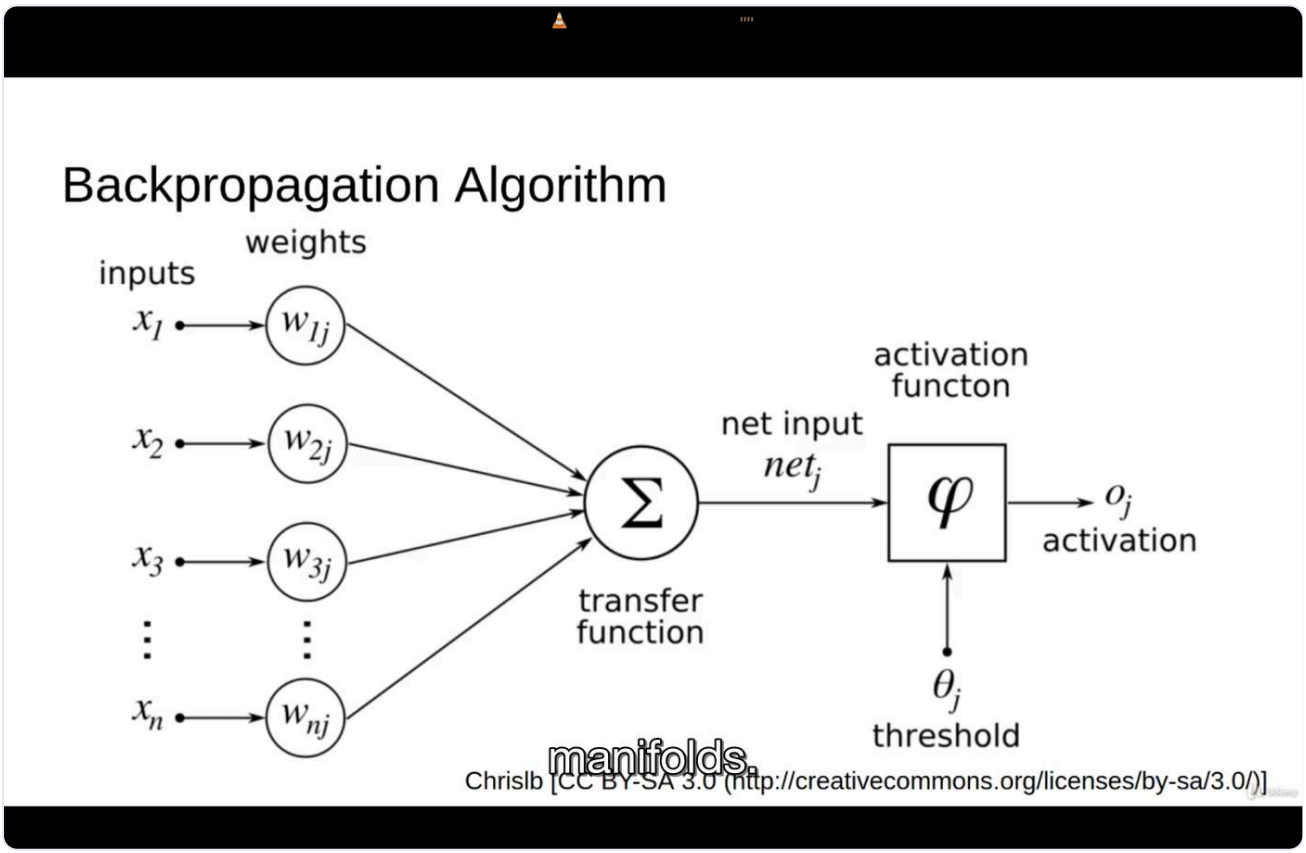


FIGURE 1.80

The paper is considered one of the pioneering papers that made it possible to reach the state of the art of performance in various deep learning applications. The input image is fed into convolutional blocks, each block consists of a convolution layer, a non-linear activation layer and a max pooling layer. Then the output is fed into a feedforward network and finally through a softmax activation to obtain probability distribution over target labels.

I have attached the reference to the paper so that you can read it and explore many concepts in the now the coding exercise. Let's start by opening a new terminal window and typing change directory C.D.. PacktPub-GANs-PyTorch-2019. And then let's launch Jupyter notebook. This should initialize the shell with a notebook inside the directory where the course material is located. We start with Section 1 we first start by installing the PyTorch library and torchvision library.

You can find the right version for your CPU or GPU and all your operating system using the PyTorch official documentation on the Getting Started page then we run a few tests to make sure that our installation was successful by importing torch and torchvision and checking for the version and checking whether CUDA is enabled or not. In my case I have got a GPU GTX 1050 and that's why I've got cuda.is_available returning true.

Don't worry if you don't have a GPU I will show you how to get that done. Using Google Colab as a nifty feature of Jupyter notebook you can run them remotely using the command on the screen Jupyter notebook that dashboard and you specify the port number and then there's dash no dash browser and then you specify the IP using dash dash IP. I use it when I am running code from my laptop onto the computer a machine that has got the GPU installed.

This way I can move and have more portability when running machine learning models. If you find any trouble when you are installing PyTorch and if you are using windows here you are a reference link for troubleshooting the Windows installation as we have explained in the previous slides we are going to implement LeNet-5 architecture which is a series of convolutional blocks to do that in Python. It is very simple. We import torch. We define a class LeNet-5 that inherits from torch.nn.Module and then inside the constructor we inherit from the superclass using super method and then inside the constructor method.

Def underscore underscore it underscore underscore we trigger the parent the constructor using the super method and then we define the network using torch.nn.Sequential. We divide the network into two modules the convolutional part and the feedforward part. I'll show you in a few steps why with it is done this way. Each convolutional block is defined using three layers: Conv2d, ReLU and MaxPool2d. It is worth mentioning that activation function ReLU.

It's not explicitly mentioned in the paper or originally published by Yann LeCun but it is a convenient to add this non-linear activation to the network to make the layers more robust and versatile itchy convolutional layer takes as a parameter the number of input channels in the first case.

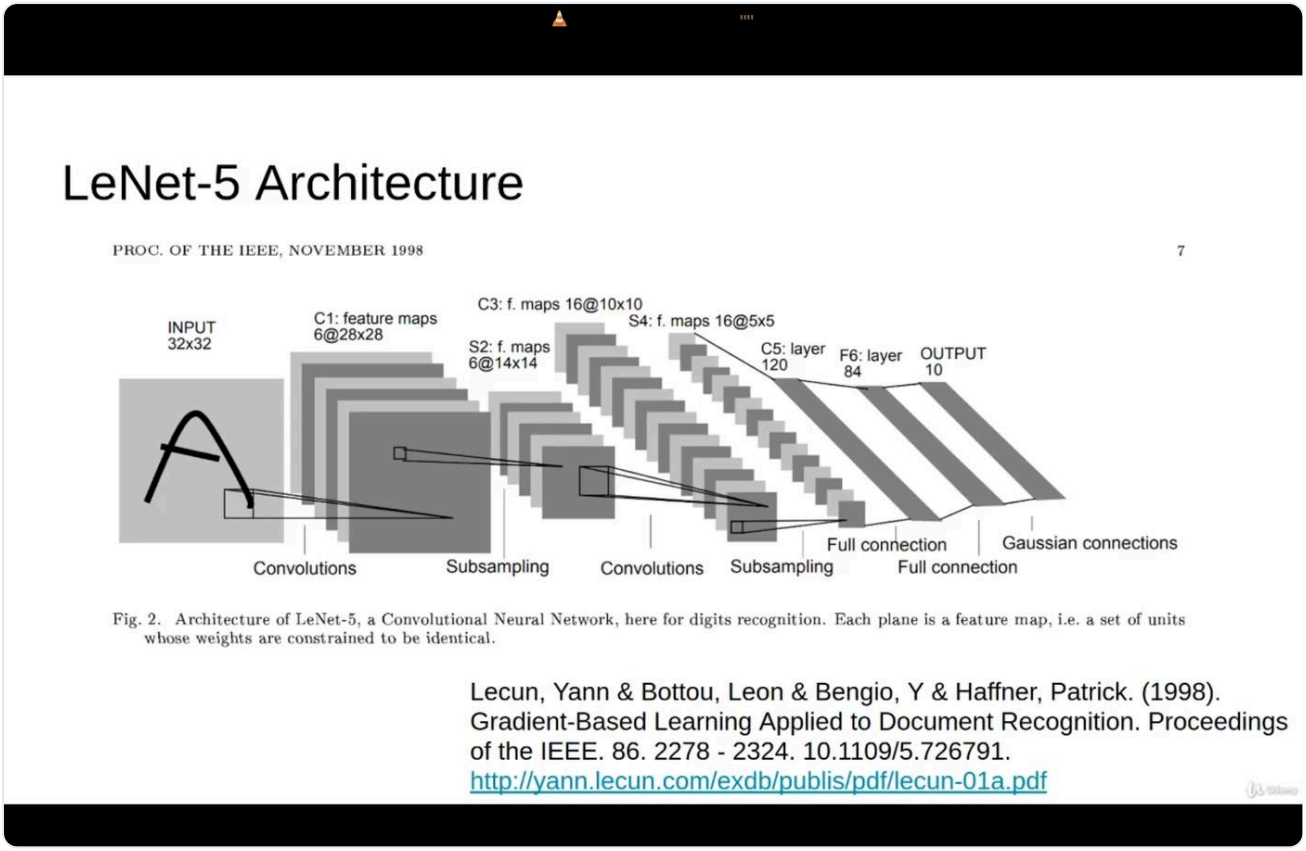


FIGURE 1.81

Since we are training a grayscale image the in-channels number is 1 and then we use 6 filters or 6 layers 6 channels out-channels and we define the kernel size to be 5 by 5 as instructed in the original paper with a stride of 1 and we instruct the bias to be true.

That means this layer will learn both the filter values for the convolutional layers and the bias added to them. Next we apply a non-linear activation, ReLU. And finally a MaxPool2d of kernel size 2 by two and a stride of 2. We do the same for the next convolutional block but with 16 filters and then we have a third convolutional block with 120 filters and the output will be passed from here onto the feed forward network the feed forward network self.fc is a torch.nn.Sequential layer with two linear layers.

The first one is 120 by 84 and the second one is eighty four by 10. Finally we use a softmax activation to get the probability distribution of the outputs. Now that we have defined the structure of the network we have to define the mechanism by which the forward path will be implemented. We defined that using a method def forward which takes the self as the parameter and then the batch which is the images that to be propagated through the network forward and backward.

First we pass the batch through the convolutional block. We get the output return ret and we change the dimensions because the output will be three dimensional because convolution and output is almost three dimensional. So we use the ret.view and we pass it the batch dimension which is batch that size of zero. That is the number of samples in this batch the learning. Usually we use 32 for the batch size and we use negative ones so that PyTorch figures out the rest of the dimensions on its own.

By here we know that the output would be two dimensional. The batch size by the output from the last convolutional layer of the convolutional block. And then we take the return rate pass it through the feed forward network and return the output because PyTorch does not implement a flatten layer just like the one we have in Keras or in TensorFlow. That's why inside the forward path we have to divide the convolutional blocks and the feedforward blocks into two different blocks and in between we do the flat in using the view method is upon us.

Hint visualization is very critical in implementing and debugging a deep learning models. So I suggest this library hidden layer and destroy it using pip install graphviz and hidden layer and then import hidden layer and the build graph and I pass the new object from LeNet-5 architecture we have just implemented and we pass a patch of 1 sample and each image is 1 channel grayscale and 32 by 32 in width and height and then when we run this code it prints a visualization of the network that we've just implemented we can see you starts with 5 by 5 convolution 2 by 2 pooling, ReLU, to another convolution of block and it goes on.

Interesting expand see on the left side the gradient path or the directed acyclic graph of the gradients which starts with constant in shape and then gathers squeeze and squeeze and concatenation finally then we go to reshape this is because we have used inside the forward path the method to view with the negative one dimension. So this when we do this we we append new nodes to the graph of the network that PyTorch applies internally to smooth in its operations as we explain in the next lecture the network goes on until we reach the softmax layer at the output.

I would suggest that you play around with hidden layer. I have attached the reference to the documentation of this a library and you can use it for visualizing gradients of visualizing GAN loss and accuracy as well.

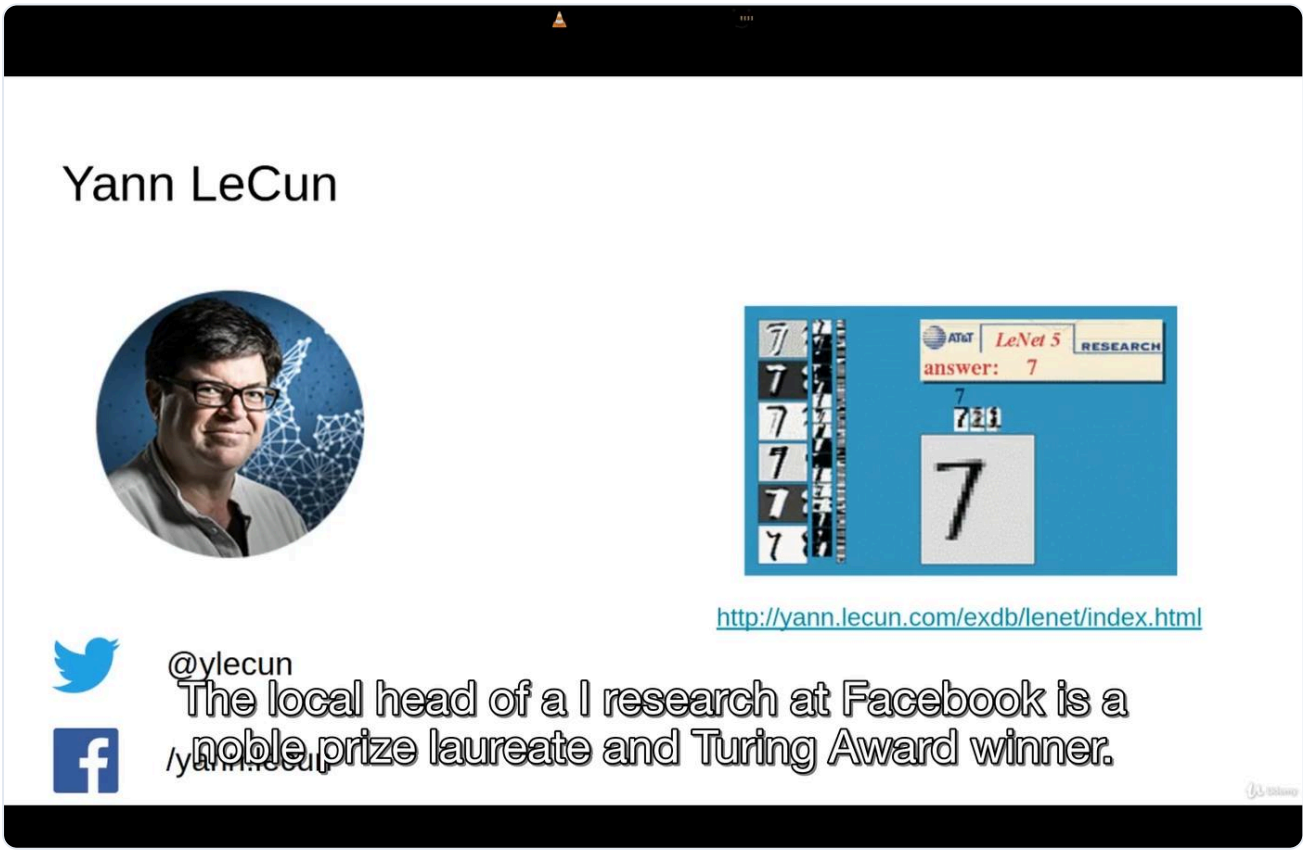


FIGURE 1.82 | Yann LeCun

Yann LeCun, head of AI research at Facebook is a noble prize laureate and Turing Award winner. I personally follow him on both Twitter and Facebook to stay up to date on advancements in the field. This animation illustrates the performance of LeNet-5 on handwritten digits.

Here you are a short link to the course of discussions. Thank you and see you in the next video lecture.

3 · PyTorch Autograd Mechanism

See the notebook `2.Lecture-2-Autograd.ipynb` for the code.

Hello and welcome. And this video lecture PyTorch autograd mechanism we talk in-depth about the way PyTorch updates its weights and calculates the gradients. This is an essential principle if you want to understand and design your own novel Deep Learning ideas.

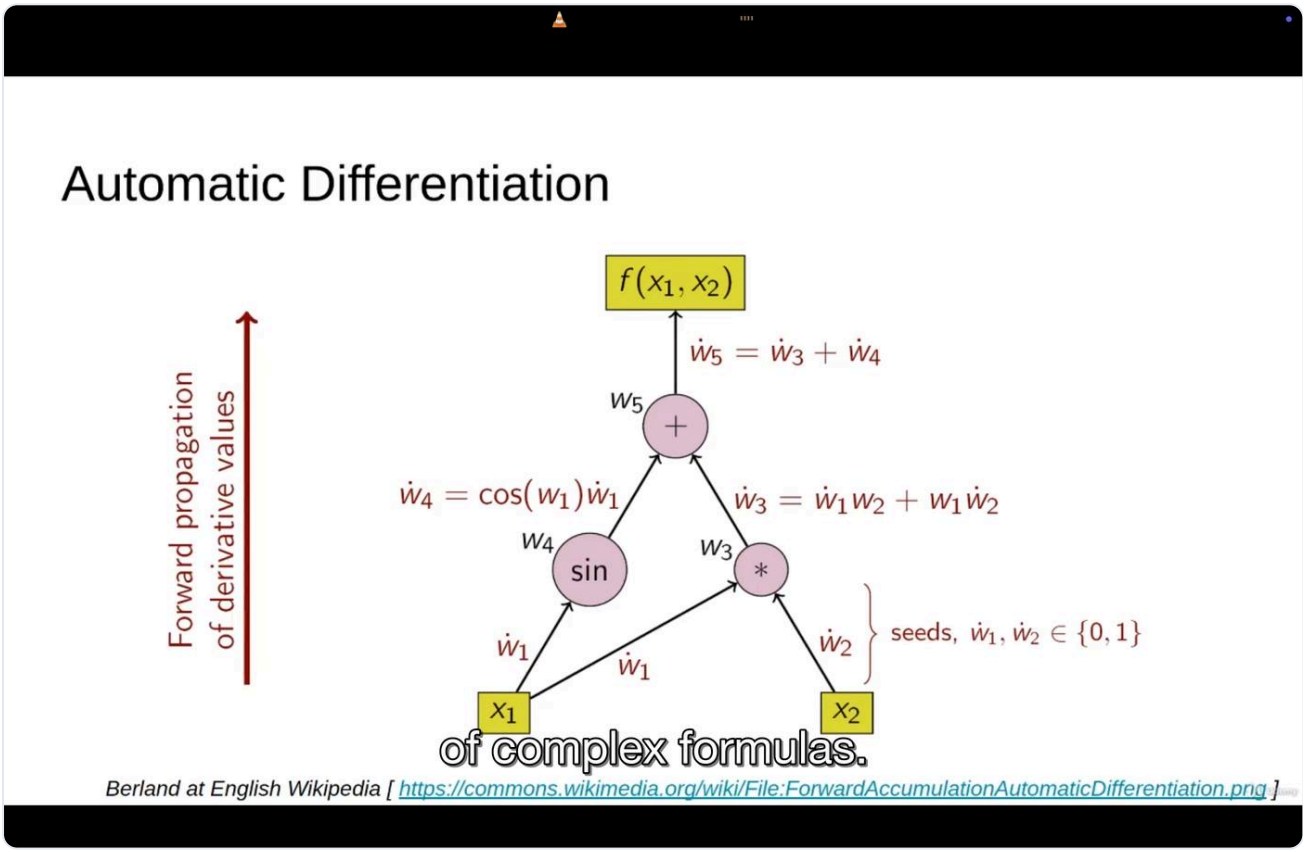


FIGURE 1.83 | Automatic Differentiation

Let's get started automatic differentiation is an algorithm that allows computers to perform derivations of complex formulas. There are two modes of operation for automatic differentiation.

The forward accumulation which is illustrated in this slide.

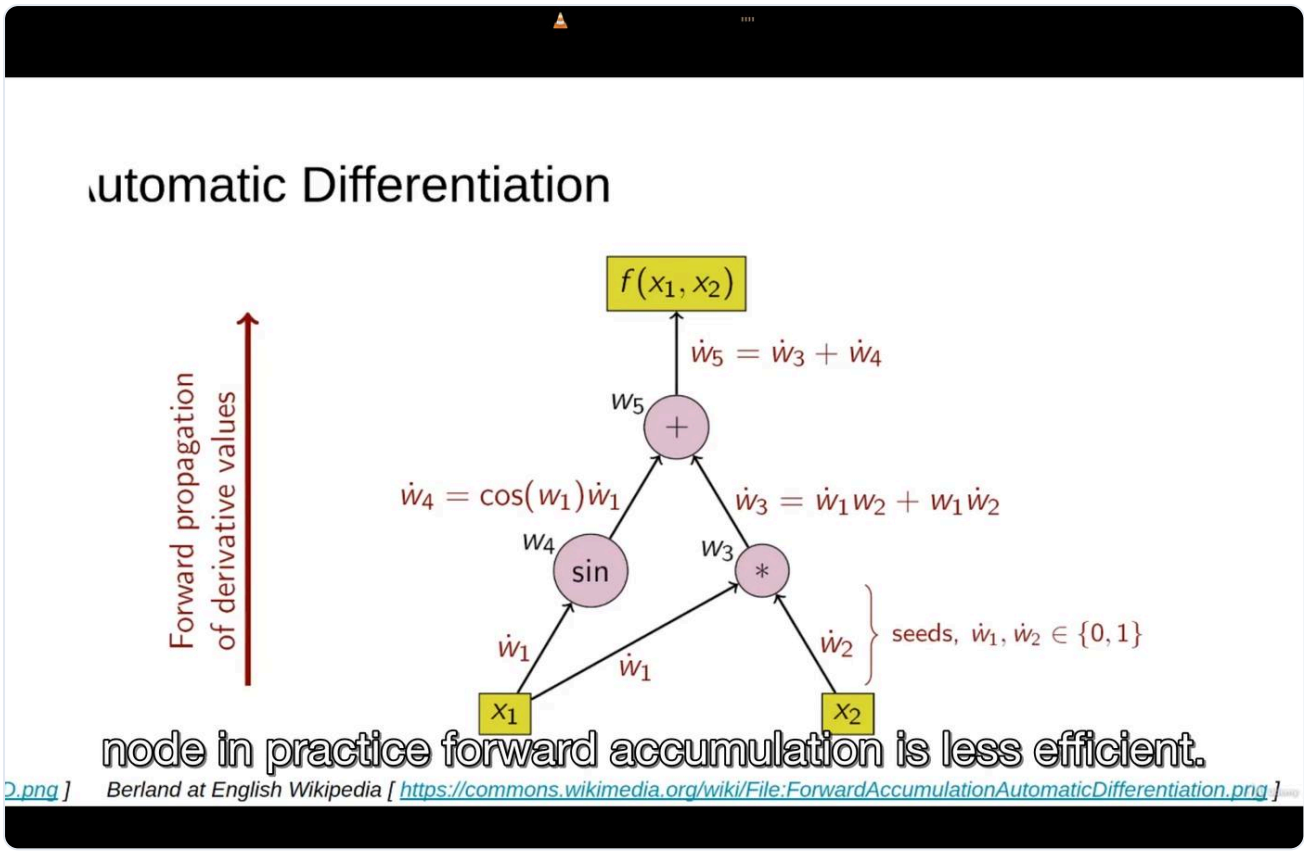


FIGURE 1.84

It starts by representing the formula as a directed acyclic graph (DAG) and traverses the graph from the bottom to the top. Each node represents an operation such as addition or multiplication. Each edge represents the dependency in the calculations the very top node which is the output is represented in terms of the derivatives of its parent nodes and the same applies for each intermediate node in practice forward accumulation is less efficient.

So the reverse accumulation is the one commonly used and implemented.

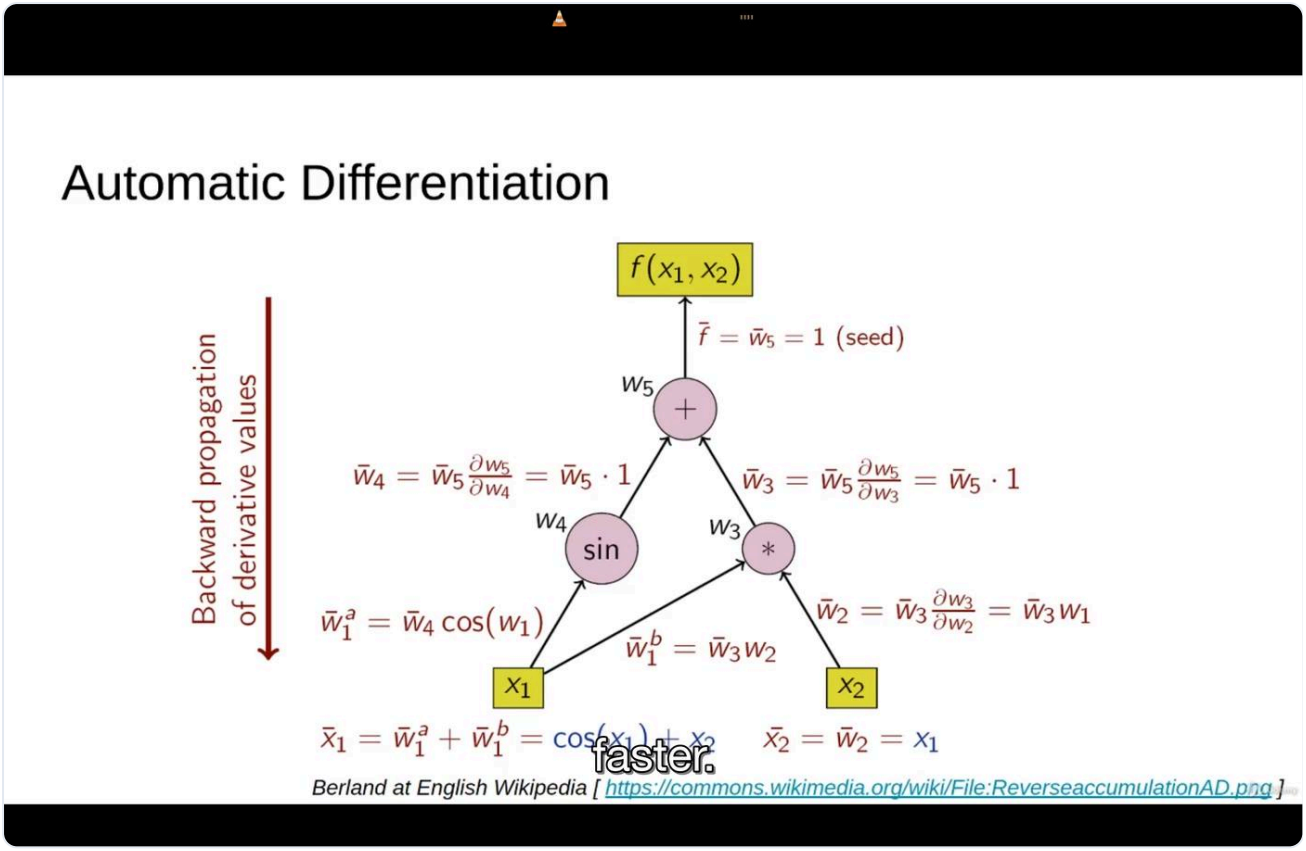


FIGURE 1.85 | Automatic Differentiation

In this case the graph is traversed from the top to the bottom. The outputs are readily available at each node without having to evaluate the lower nodes. That's why back propagation can benefit from this behavior and the calculations can be performed much faster.

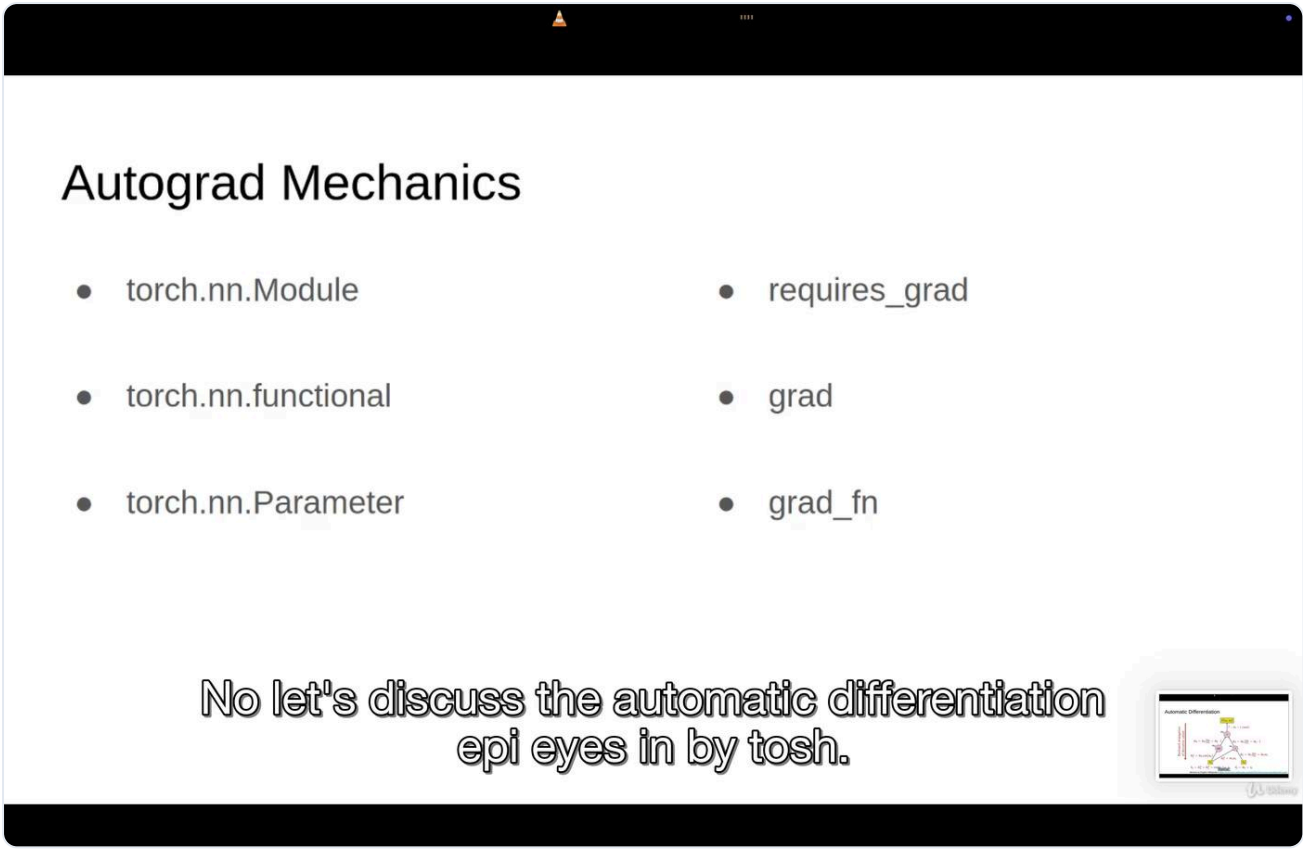


FIGURE 1.86

Now let's discuss the automatic differentiation APIs in PyTorch.

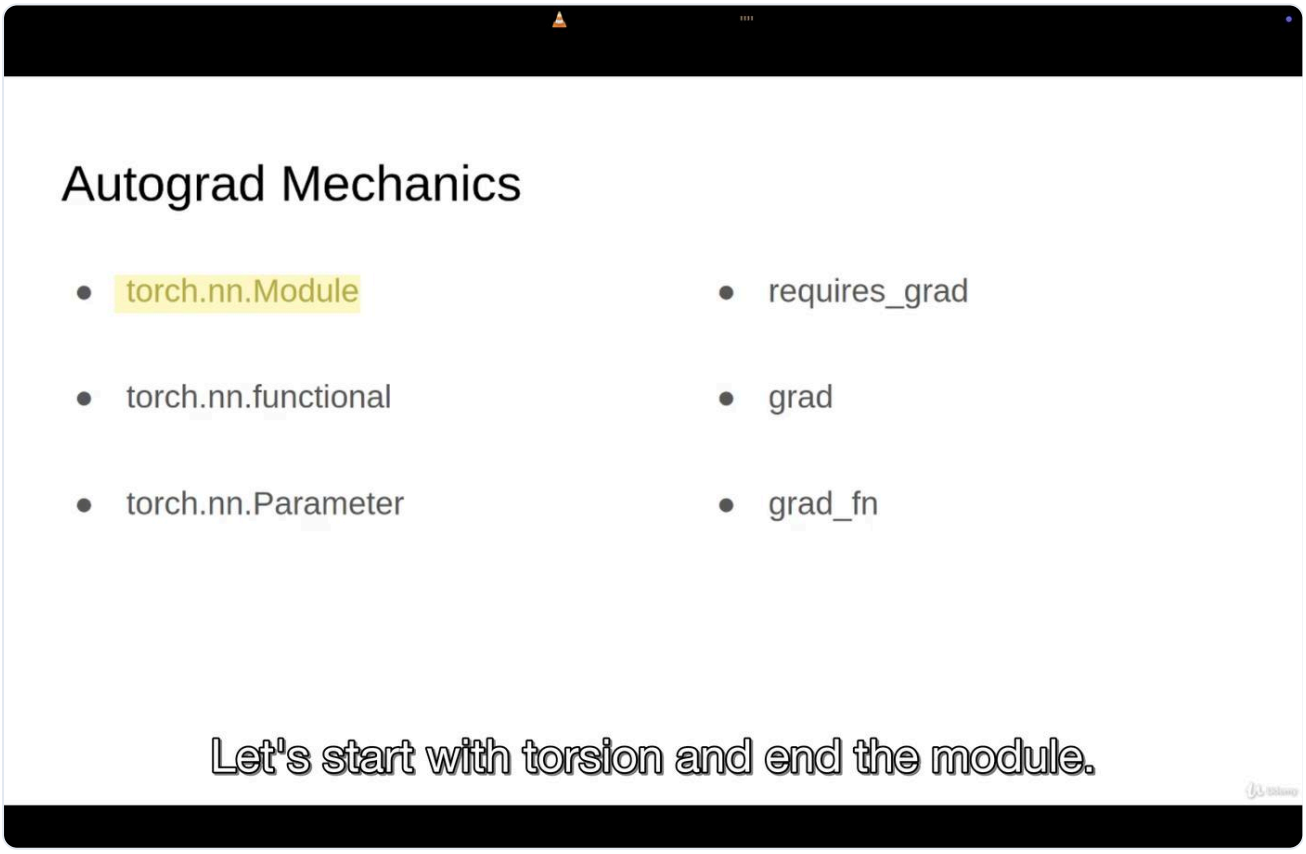


FIGURE 1.87

Let's start with torch.nn.Module. This is a wrapper for deep learning layers and functions that maintains the state.

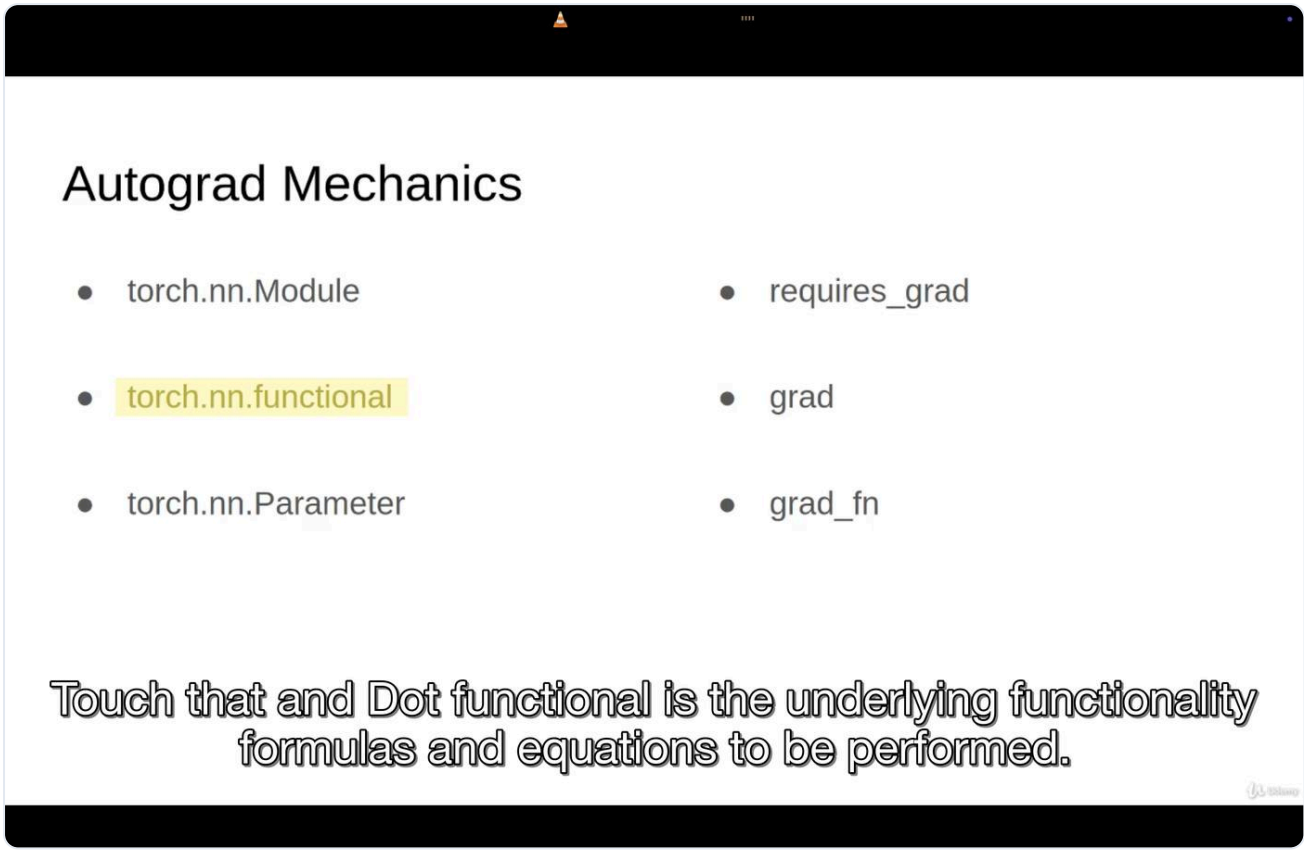


FIGURE 1.88

The state here means the gradients of the variables the gradients are automatically calculated for us next. torch.nn.functional is the underlying functionality formulas and equations to be performed. It is mostly implemented in PyTorch or C language for efficiency but it doesn't have a state that means you have to maintain the gradients manually.

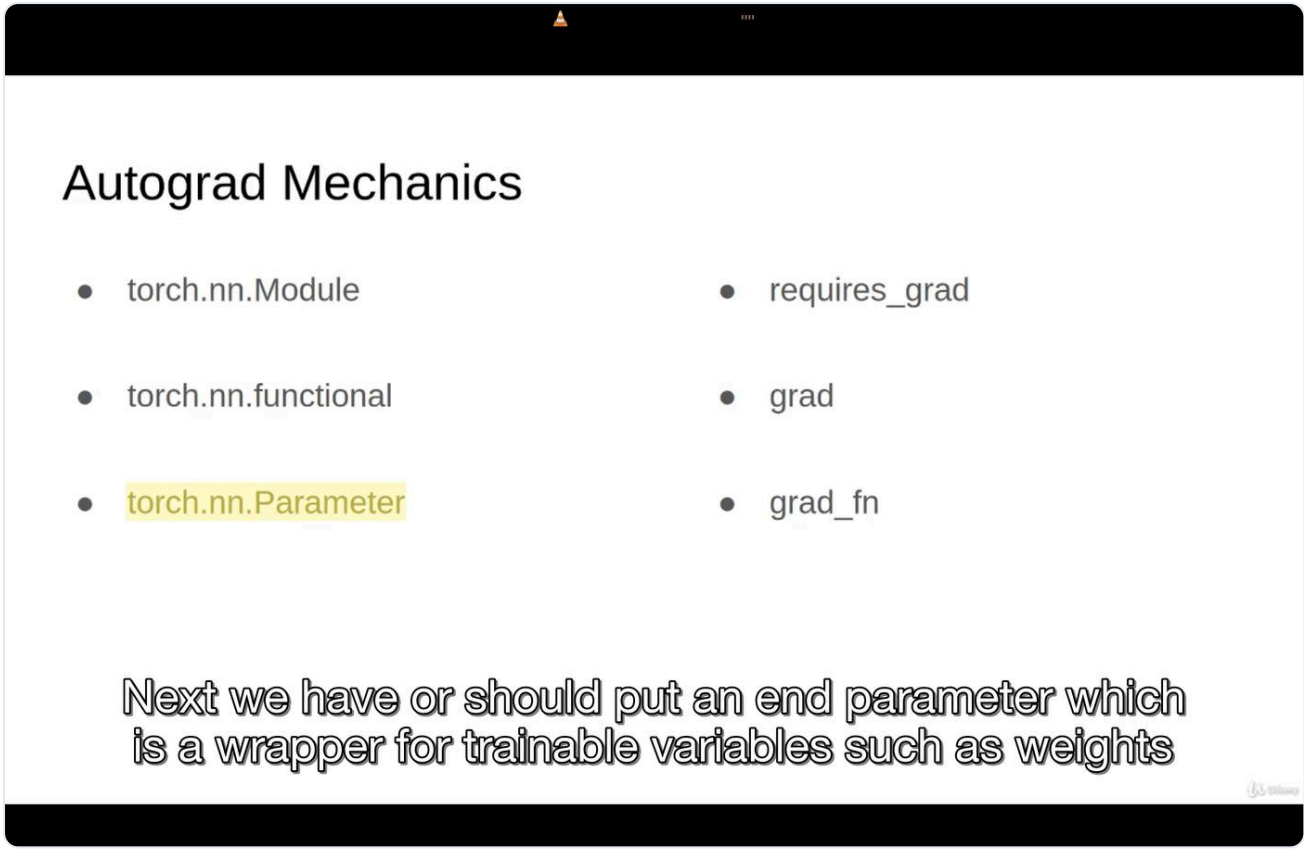


FIGURE 1.89

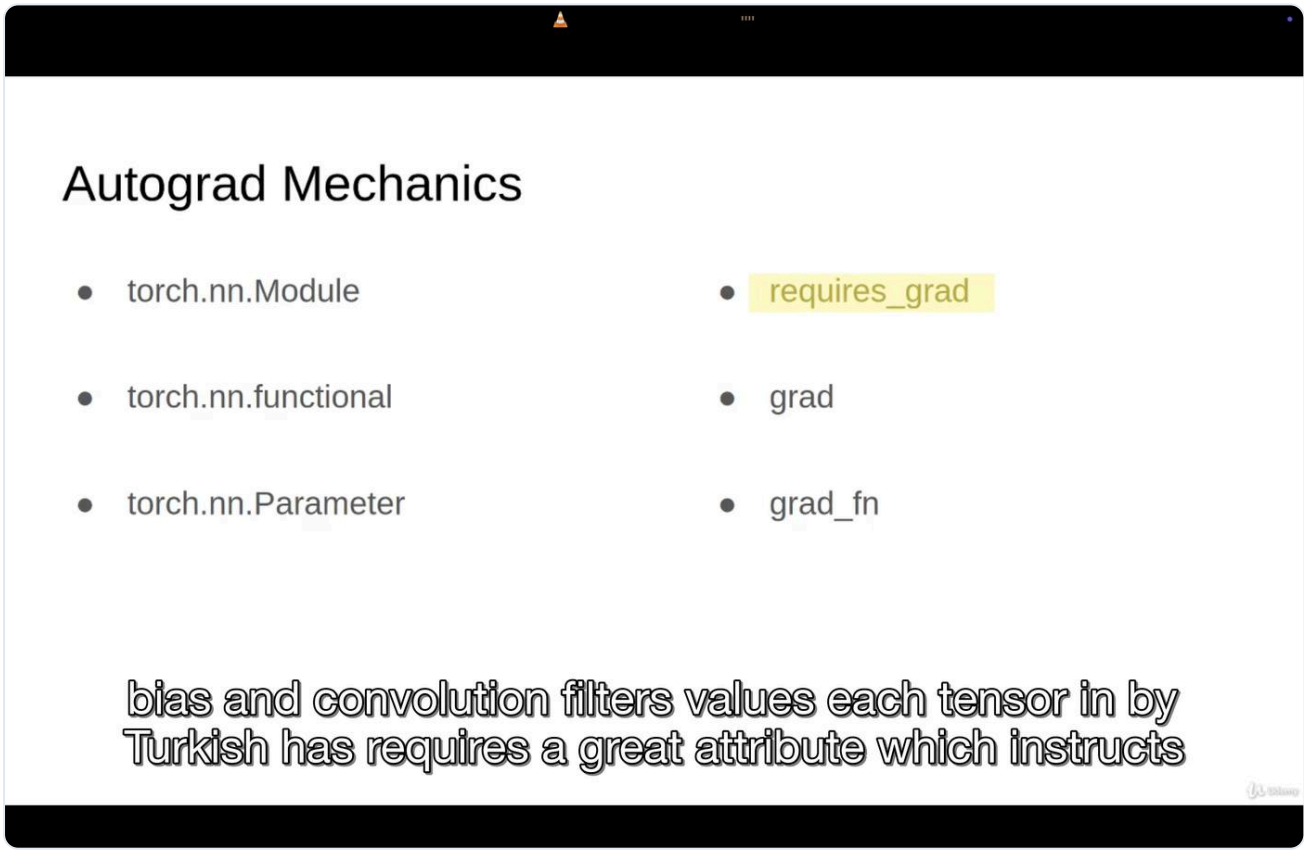


FIGURE 1.90

Next we have torch.nn.Parameter which is a wrapper for trainable variables such as weights bias and convolution filters values each tensor in PyTorch has a requires_grad attribute which instructs Python to maintain a graph to calculate gradients for these variable.

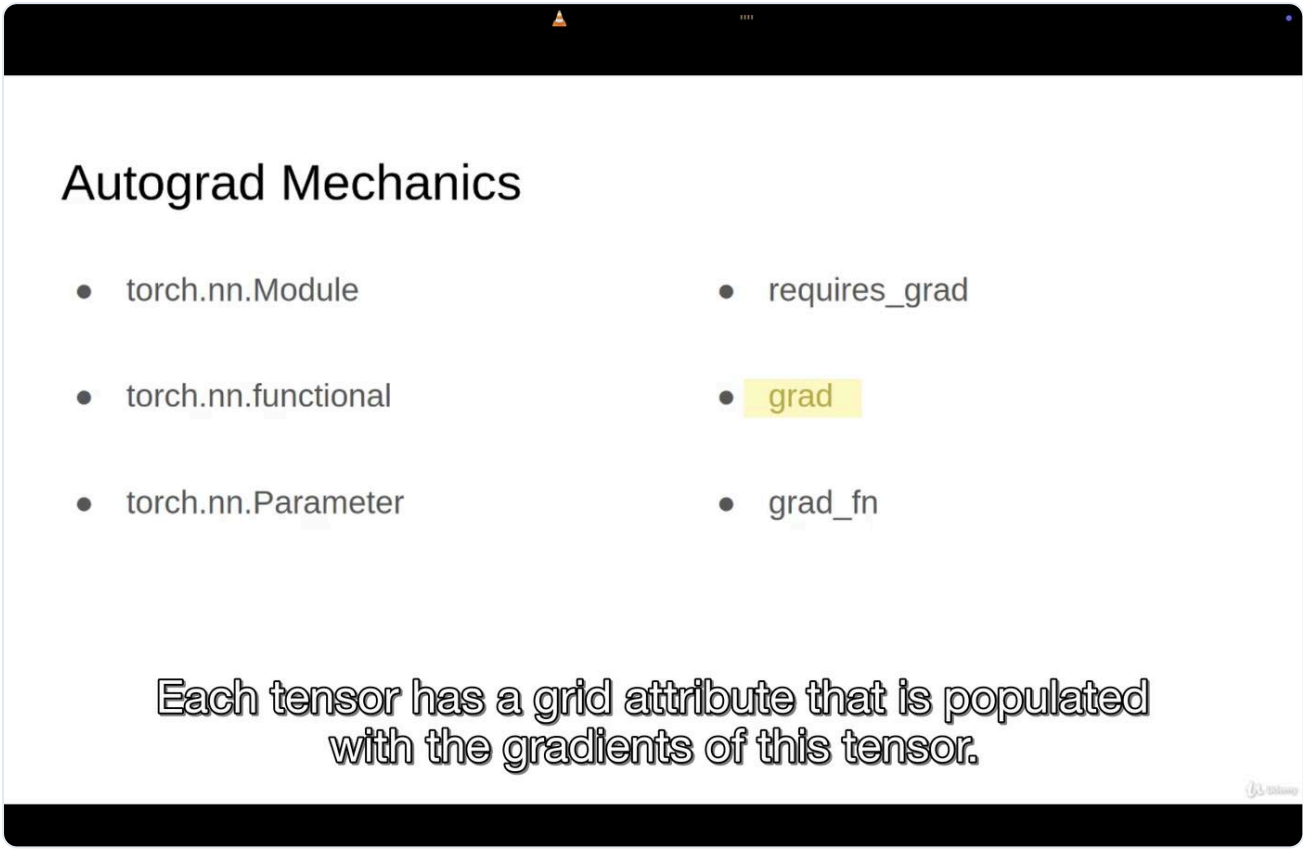


FIGURE 1.91

Each tensor has a grad attribute that is populated with the gradients of this tensor.

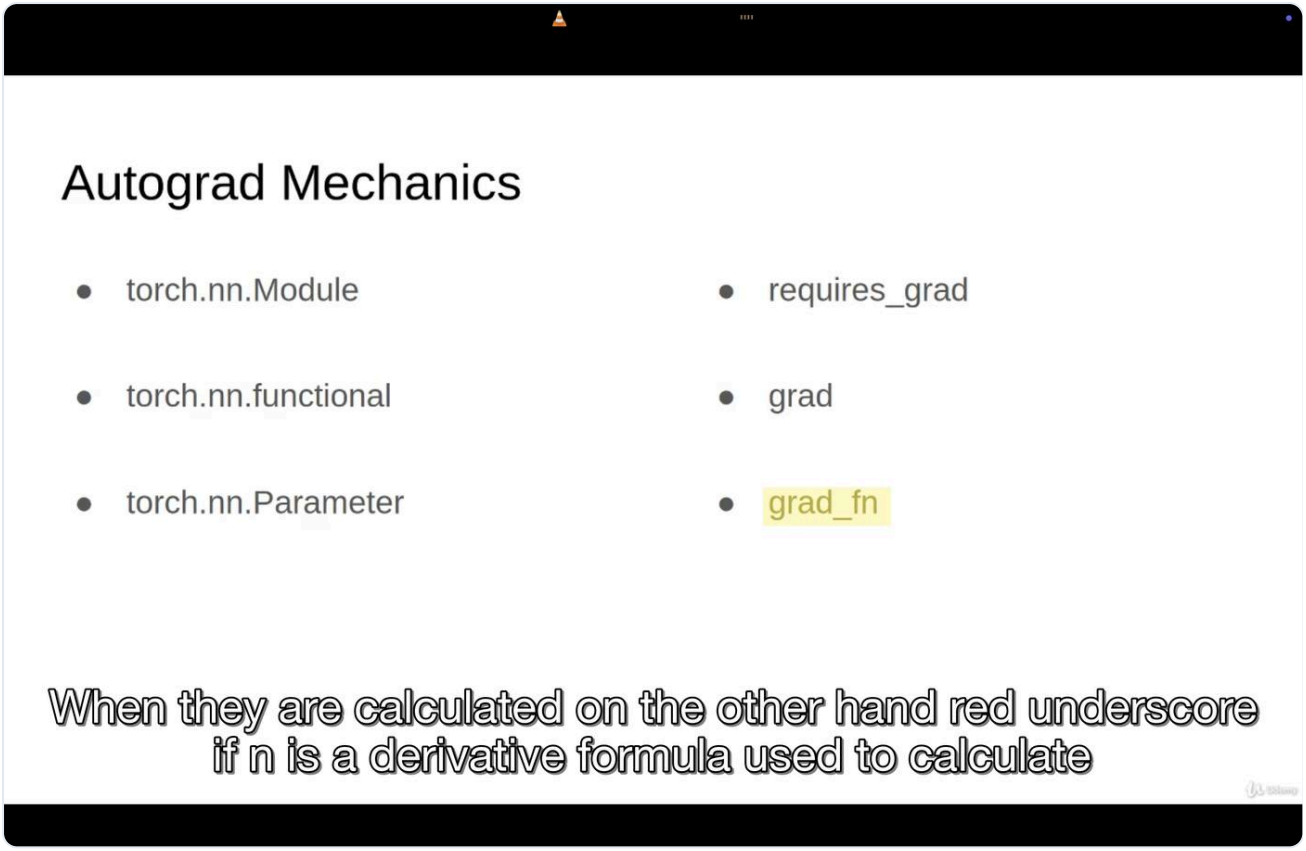


FIGURE 1.92

When they are calculated on the other hand grad_fn is a derivative formula used to calculate the gradients for this tensor. Now let's understand the autograd mechanism using a very simple example. We started by importing numpy as np and we import torch.nn.functional as F this is a convenient use by many developers to import this module.

Then we define it tensor using torch.from_numpy that will import to the tensor from a NumPy array into a torch tensor representation with defined NumPy array to have four samples. And if you focus on these samples you find out that these samples belong to the XOR problem 0 6 0 0 0 0 x or 1 is 1 1 x or 0 is 1 1 x or 1 is 0 and we define the type to be numpy.float32. So this represents a dataset set a very simple data set so that we can understand the autograd mechanism was a very simple flexible example and then we defined the outputs of the ground truth which is 0 1 1 0 using the same way we have converted it from NumPy to PyTorch tensor.

So assume that we've got only one neuron with two inputs so that neuron will have weights 1 by 2 or 2 by 1. We have done it this way so that the product is correctly calculated so we reverse it one instead of two and then we define weights. That requires_grad equals True we have set the attribute that instructs PyTorch to apply the gradient mechanism to this tensor another tensor for this one neural network is a bias we initialize it using torch.randn and here we've got only one neuron with 1 bias and we pass requires_grad as a parameter to the constructor.

So you've got two ways to instruct PyTorch to use the gradient for a tensor either by setting that attribute explicitly or by passing it as a parameter to the constructor of a tensor now we use F.linear which applies a dot product between the inputs and the weights and the bias and you have both the inputs the weights and bias and we've got the result which is four elements for

each input sample we can see here. Also that as a parameter of the tensor we've got grad_fn which instructs the kind of formula or the function used to calculate the gradient for this tensor so we do the same but this time we save the output in a variable named preds for predictions and we calculate a loss.

A very simple L1 norm which is an absolute error loss function by subtracting outputs minus predictions or preds and then we get the mean value for this loss. So if we expect that the attribute of preds as a tensor grad_fn then we see the formula or the function used for calculating the gradient. Another one which is the mean is used to calculate the gradient of the loss. Here we can see that for each tensor there is a different gradient function.

And then if we print the weights.grad we get nothing because the gradient is not populated yet. How do we populate the gradient. We use loss.backward. This will initiate the backward path or the back propagation or the reverse mode of the auto differentiation algorithm to calculate and update the gradients for each layer. Now if we print weights.grad we see that it has calculated the derivative for each of the inputs to be negative half.

Of course this calculation is not very accurate and this will not be used to get the optimal results on the X or dataset. But now you can see and visualize the mechanism of the autograd algorithm. You can try to play around with the bias and its gradients as well. Another thing that you can observe here is that we've got two values for the gradient for the weights because here we've got one neuron and two input nodes. So for connections between the two elements of the input vector and the one you we have got two synapses or two edges and each one of them has a weight.

From here you can see why you have got two values for weights. Thank you. And see you in the next lecture.

4 · PyTorch Custom Loss Function

See the notebook `3.Lecture-3-Custom-Loss-Function.ipynb` for the code.

Hello and welcome to a new lecture, PyTorch custom loss function in this section we learn an essential a skill that is going to prove beneficial in almost all aspects of deep learning the ability to design your own loss functions will empower you to build advanced networks that break the norms of the field. Let's get started in a research paper published in 2017 studying the effect of the loss function choice on the accuracy in deep learning as in the attached reference on the screen.

The authors have tested many less frequently used loss functions and compared them to the more common loss functions such as mean square error and log loss in information theory. The concepts of entropy and cross entropy describe the amount of information held in a particular bit and the probability distribution describing such information. The information theoretic learning makes use of these concepts in order to learn the weights of the model that best describe a phenomenon.

Cauchy-Schwarz loss function is an example of a distance function between two probability distributions. The first distribution is the one obtained by the softmax layer at the output layer of a deep learning model and the second distribution. It is the one related to the one-hot encoded target labels of the given samples the mathematics may seem scary at the beginning but it's very simple. The numerator is a product between the outputs and the ground truth labels and the denominator serves as a normalization to downscale the value the logarithm is used to shrink the value range further in finally a negative sign is used to inverse the values and convert the objective into a minimization optimization.

Hence it is very convenient to minimize the loss function in almost all the reference papers in the literature. So in a nutshell this function describes the distance between the probability distributions of both the model outputs and the ground truth labels and through the training process. We try to minimize such a distance. Next we implement a custom loss function for training. The LeNet-5 architecture on the MNIST dataset. We start by importing torchvision and then we initialize the dataset from torchvision.

torchvision.datasets.MNIST we specify the directory for the dataset to be downloaded slash data slash mnist. We set this first variable for the training set by specifying trained parameter to equal to and then we specify the download variable to equal to so that it can download the image dataset from the cloud. And then we apply the transformations array that will be applied for every sample of the dataset. In our case we apply only padding to increase the size from 28 by 28 to 32 by 32 by padding 2 pixels at each dimension.

And then we convert the image to a tensor that will bring the image in the values between 0 and 1. Originally it is in the range between 0 and 255 but applying to tensor will convert it to a byte tensor in the range of between 0 and 1. Feel free to apply any preprocessing function you feel it will improve the performance. You can apply and normalize. You can apply center crop. You can apply any other of the available transformations or even a lambda transform that you define on your own.

I would encourage you to play around with the transformations and observe the performance boost or degradation as a result. Next we define another variable for the testing dataset and we apply the same except here we set the parameter train equals False. Next we specify the batch size to be 32. I would encourage you to play with different values 32. It sounds like a most convenient value to be used when you are not sure what value to use or if it is not explicitly referenced in the original paper.

And then we use another utility function from PyTorch which is a data loader and we pass their variables we have created in the previous a step for the dataset. into this data loader and this data loader is the actual function that will generate the generators. The Python generators that will yield the samples in batches from the data set. This allows PyTorch to efficiently perform data loading without loading too much data into the memory with specified parameters to a data loader.

The variable for the training set the batch size and we specify whether shuffling should be true or false for the training set. I would set shuffle to be equal to True and then we set the number of workers to 4 depending on how many workers you have got on your CPU or GPU. We do the same

for the testing set except this time with set shuffle equals false as it should become clear in the next steps. Why was it shuffle equals True for training and equals false for testing.

Is that because the generator mechanism of Python it keeps looping over and over again without stopping over the data set. And here we get the concept of an epoch and the epoch. We've got the concept of a batch and step so simply as a batch is 32 samples and the step is iterating over one batch and then inside the epoch we iterate over steps so we can literally iterate over many steps in one epoch not necessarily the number of steps required to cover the dataset in its entirety.

We can go beyond the dataset size here. That's why we can shuffle the dataset and we get random sample at each step or we can set the number of steps not to cover the entire dataset. Maybe only 30 percent 70 percent or so. This is a flexibility that can prove useful for you later in your career. We don't do that for the testing because we want to test the entire testing set and see how our algorithm or model is performing on the testing set. In this case we use torch.cuda.is_available to specify the device.

Our training will be performed on don't worry if you don't have a GPU you can perform that on you but it will take longer and the code will run smoothly without errors. Since we have a defined device to be generic regardless of what happens next. Now we reach the climax of this section by implementing the custom loss function the Cauchy-Schwarz divergence. we start by defining an encode one-hot method that will convert a label from a single digit to a one-hot encoded vector of zeroes and only one in place of the target label.

For example if their target label is the digit number three then we will get an array with 0 0 0 1 and zeroes for the remaining digits and so on. So we start by converting that tensor from the GPU to the CPU using .cpu(). And then you convert it to NumPy and then we use syntactic sugar from numpy so that we can convert the digit into a numpy array of one-hot encoding by initializing an array numpy.zeros all the values would be those that would be the size in the first dimension of the batch size.

And then in the columns we've got 10 columns for 10 digits in this dataset and then we use the third line. Line number six B of numpy.arange. Edit size and comma eight that will say for every element in this batch and for every column that is set as a target 3 5 6 and so on we set the value to 1 the rest to remain zeros. Finally we convert it from number to bytes tensor using torch.from_numpy. And then we convert it to floating points that float.

And finally we converted back to the GPU or CPU using the two device we will see why this method is useful as we implement the Cauchy-Schwarz divergence loss function so over here we define a class CSD standing for Cauchy-Schwarz divergence we make it so that it inherits from torch.nn.Module in the constructor we invoke that constructor of the parent using super method and then in the forward path we pass in the outputs from the model and the ground truth target next we define a variable Y which is the one-hot encoded vector of the target labels and here if you're coming from Keras or TensorFlow background you find it a little bit weird because you have got one-hot encoding layer in Keras and TensorFlow but this is not the case in PyTorch and you have to implement that in your loss functions depending on your logic and how it is performed I know we apply the formula from the slide denominator is defined the numerator which is the sum of the dot product between the outputs and the target label y transpose so that we can

perform the dot product and the sum is performed along the vertical dimension the columns that's why we set the dim dimension equals one for the denominator it is simply the L2 norm and we can do that using torch.norm and passing the variable outputs and comma 2 for L2 norm, 1 for L1 norm and so on.

and we multiply it by the L2 norm of their target labels Now I have got both the numerator and the denominator calculated we apply the logarithm natural logarithm using torch.log numerator over denominator and we multiply the output by negative 1. And finally because we need to apply the backward using only single value we use torch.mean that will return a scalar value one single element for all this batch and propagate the loss through backwards here we need to know two observations the first observation is that if we are applying a custom loss function that is not continuous then we have to define a custom backward method and then backward method is defined the same way we've done for the forward instead of passing the outputs and the target we pass the gradient from the previous layer and we perform as a special case for calculating the derivative and then we return to the new gradient.

The other observation is that this method may not be as fast as the built in methods in PyTorch because they are inherently built using C language or a torch language but we have optimized it by making sure that all our operations are performed as multiplications or additions or divisions using the basic built-in methods efficiently implemented by PyTorch now in order to perform the model training we define an object of the LeNet-5 architecture and we convert it to the device we are using GPU and we name it network next we define the optimizer that will perform the actual update of the variables how much will it update the variables after each forward and backward pass and even we defined the network parameters at the variables to be optimized and the learning rate to be 0.05.

I would encourage you to play around with the learning rate and see whether you've have got better results using a smaller or higher learning rate. Next we define the loss function which is criterion which is an object of the class we have just implemented Cauchy-Schwarz divergence. Again try to play around with different loss functions implement your own loss functions and use built in loss functions in Python. Next we perform the training in the form of epochs inside each epoch we need to make observations of three steps.

The first step here when we convert the data set to the device we are using for the training, GPU or CPU. The second step here when we initialize the gradients to be zero for the optimizer so that we are not accumulating gradients from previous epochs. And finally the third step here. We apply the backward pass for the loss function and then we run the optimizer step so that it can update the weights. You can see here that I have trained this network for 120 epochs and the loss has been decreasing from epoch to epoch now that we have trained the network we need to evaluate the performance on the testing dataset we do that by setting network.train to False so that any operation that we are going to perform next will not affect the actual network and the weights that have been updated as extra guarantee that we are not performing any backward.

We use torch.no_grad so that no gradients would be computed and updated. And next we calculate both the loss and the accuracy. I have trained them network and I evaluated it and I've got a loss of one point fifteen and the accuracy of ninety two point twenty percent and I am very sure

that you can do better and the assignments and share your results with your colleagues and let's see who gets the highest performance on this data set.

Here you'll find reference to all the papers that I have mentioned throughout this section and over here at the very bottom of the notebook you find a reference of the Google Colab notebook file of this exercise so that you can run it on Google Colab using GPU and TPU and obtain the same results. If you don't have a GPU at home in this YouTube video I have reviewed many examples of loss functions and I hope you find it useful another influential professional in deep learning is Aurélien Géron and I would recommend you to watch his video on information theory concepts such as entropy and cross entropy in order to get a deeper understanding of the concept.

You could also follow him on Twitter for continuous updates in the field.

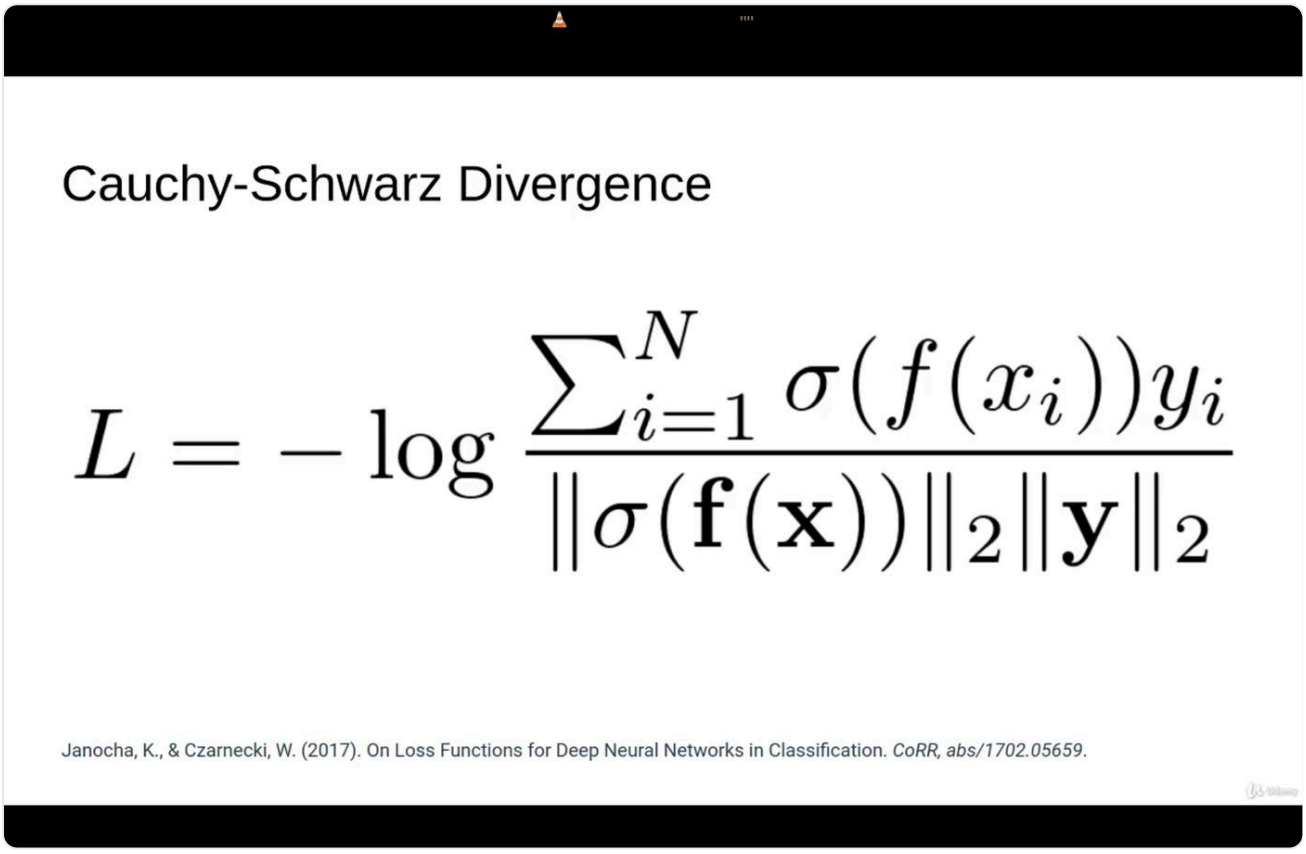


FIGURE 1.93

This is an optional assignment and I would like you to read the paper on loss functions and pick one of the discussed loss functions and implement it based on what you have learned in this section. Feel free to discuss the assignments on this lecture congratulations you have made it through Section 1. See you in Section 2 where the fun begins. Thank you.

LECTURE 5

Introduction to GANs

Hello and welcome to a new section in this section generate handwritten digits with Vanilla GANs.

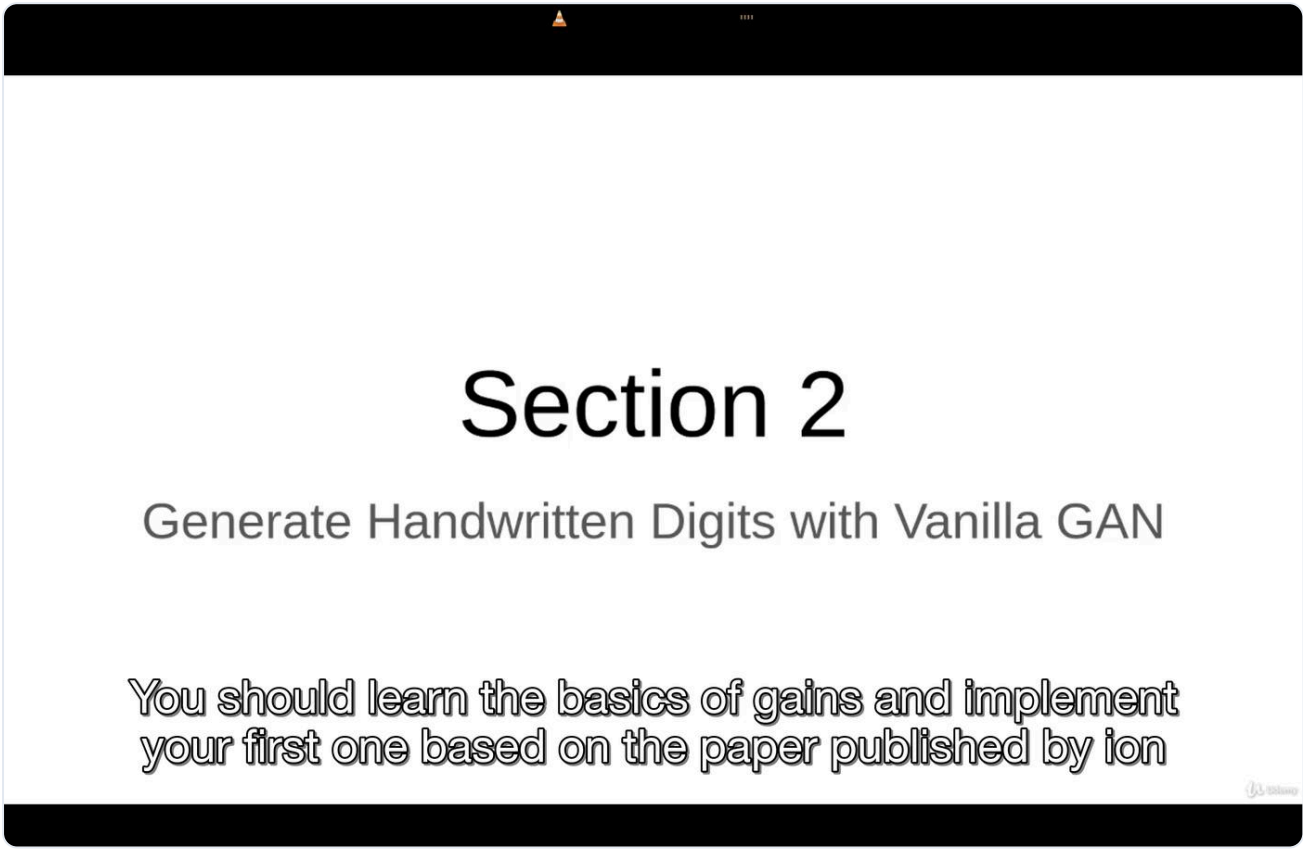


FIGURE 2.1 | Generate Handwritten Digits with Vanilla GAN

You should learn the basics of GANs and implement your first one based on the paper published by Ian Goodfellow in the first lecture. Introduction to GANs.

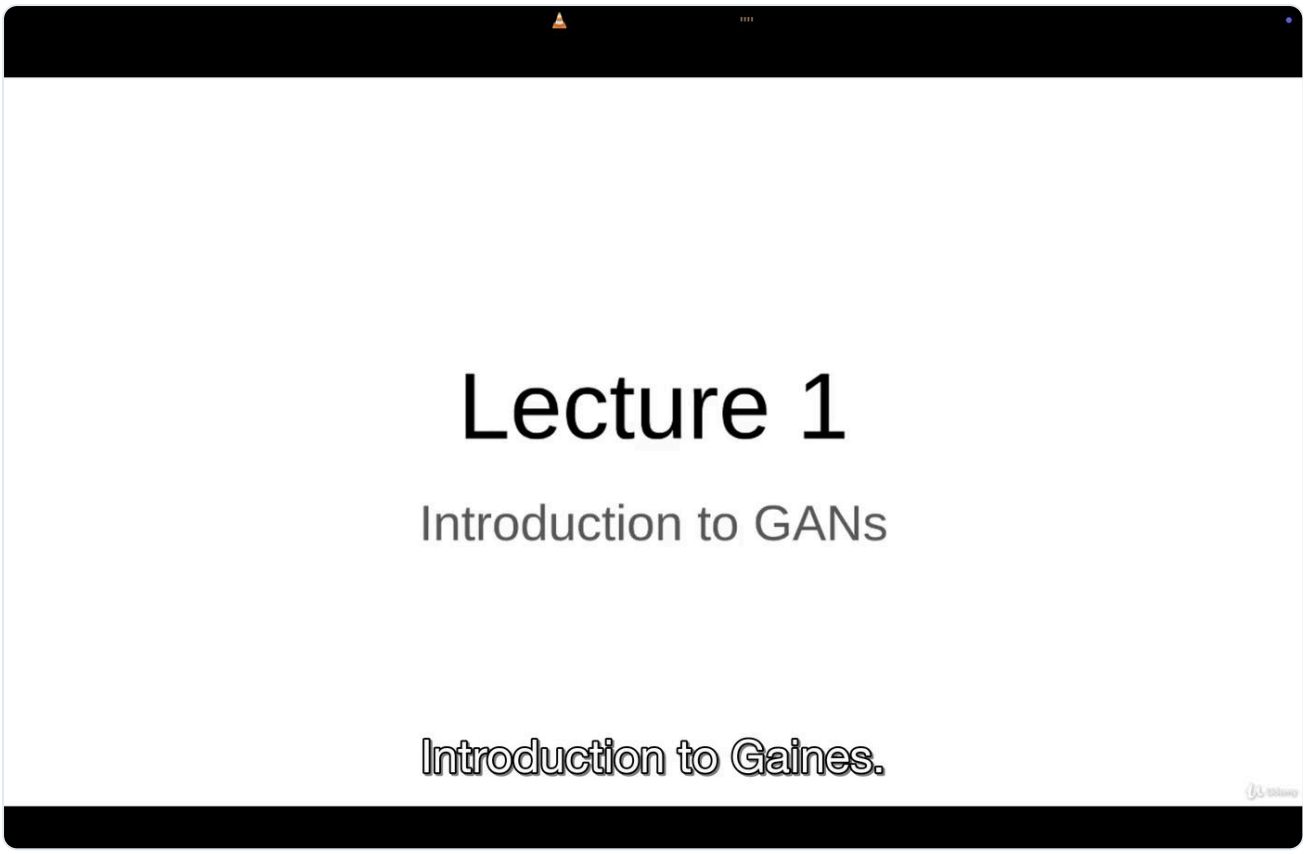


FIGURE 2.2

I shall review a quick historical background of generative models and introduce the first building blocks of GAN models. Then I shall share with you some amazing applications of GANs.

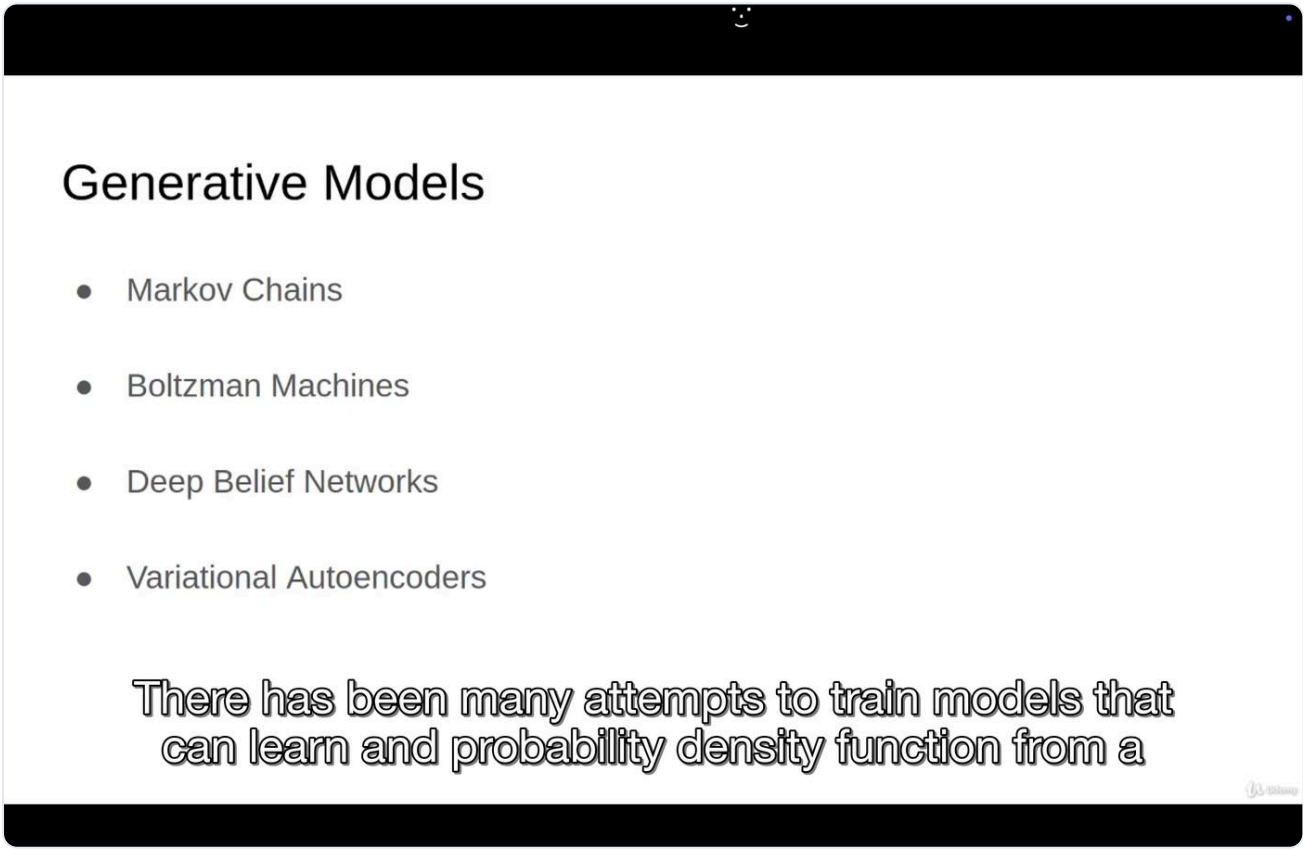


FIGURE 2.3 | Generative Models

There has been many attempts to train models that can learn a probability density function from a training set and then generate new samples that are drawn from the same distribution methods such as Markov chains Boltzmann machine deep belief nets and variational autoencoders.

These methods were lacking in one aspect or another. Some were not analytically solvable and required settling for an approximate lower bound as a work around while others were proven to overfitting variational autoencoders were ahead of them all with impressive results. And at the same time had a sound mathematical explanation generative models are designed to solve the problem of probability density approximation.

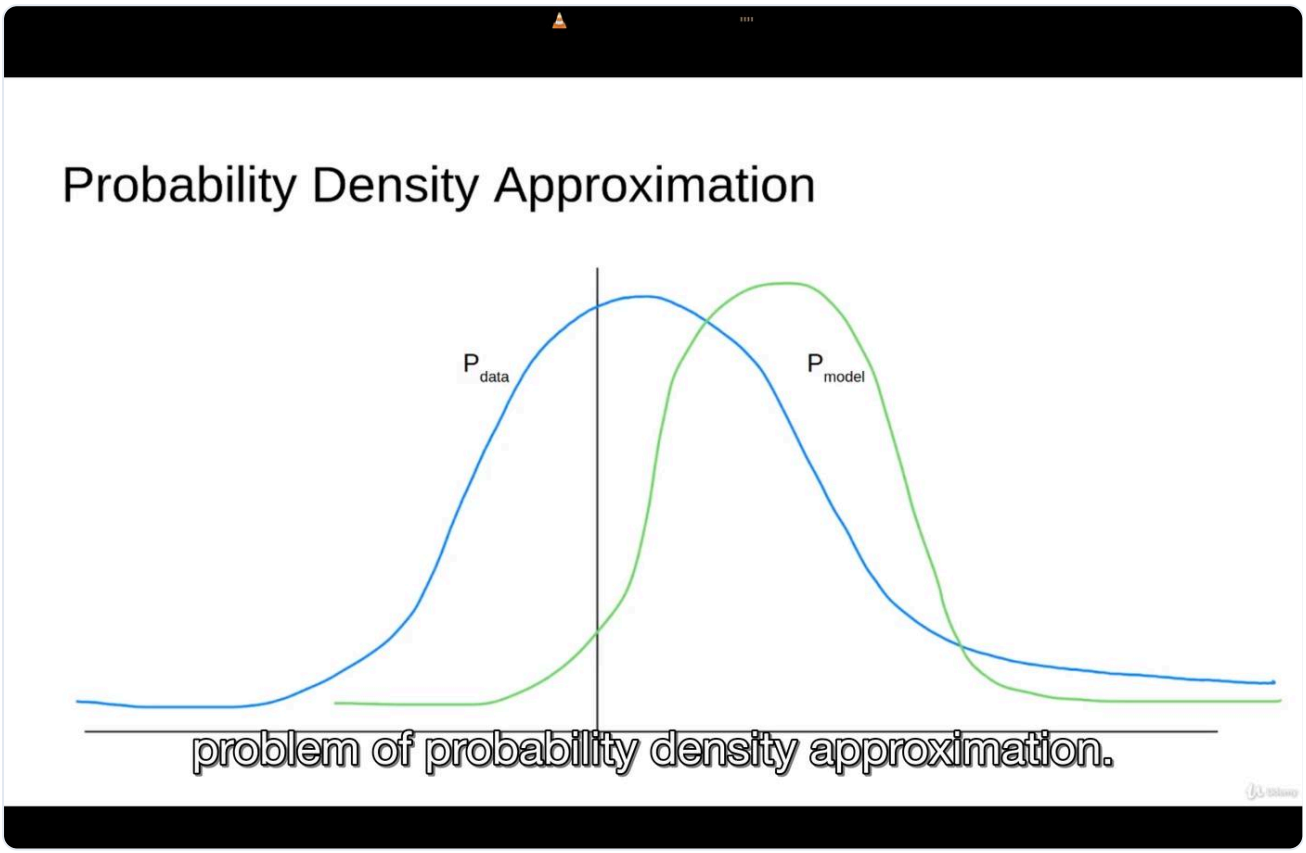


FIGURE 2.4 | Probability Density Approximation

The problem is best of visualized by this diagram.

The blue color indicates a probability density function for one dimensional data set. The green line on the other hand represents the probability density function of the samples generated from a statistical model the generative model is trained iteratively to make the density of its generated samples as close as possible to that of the original data set.

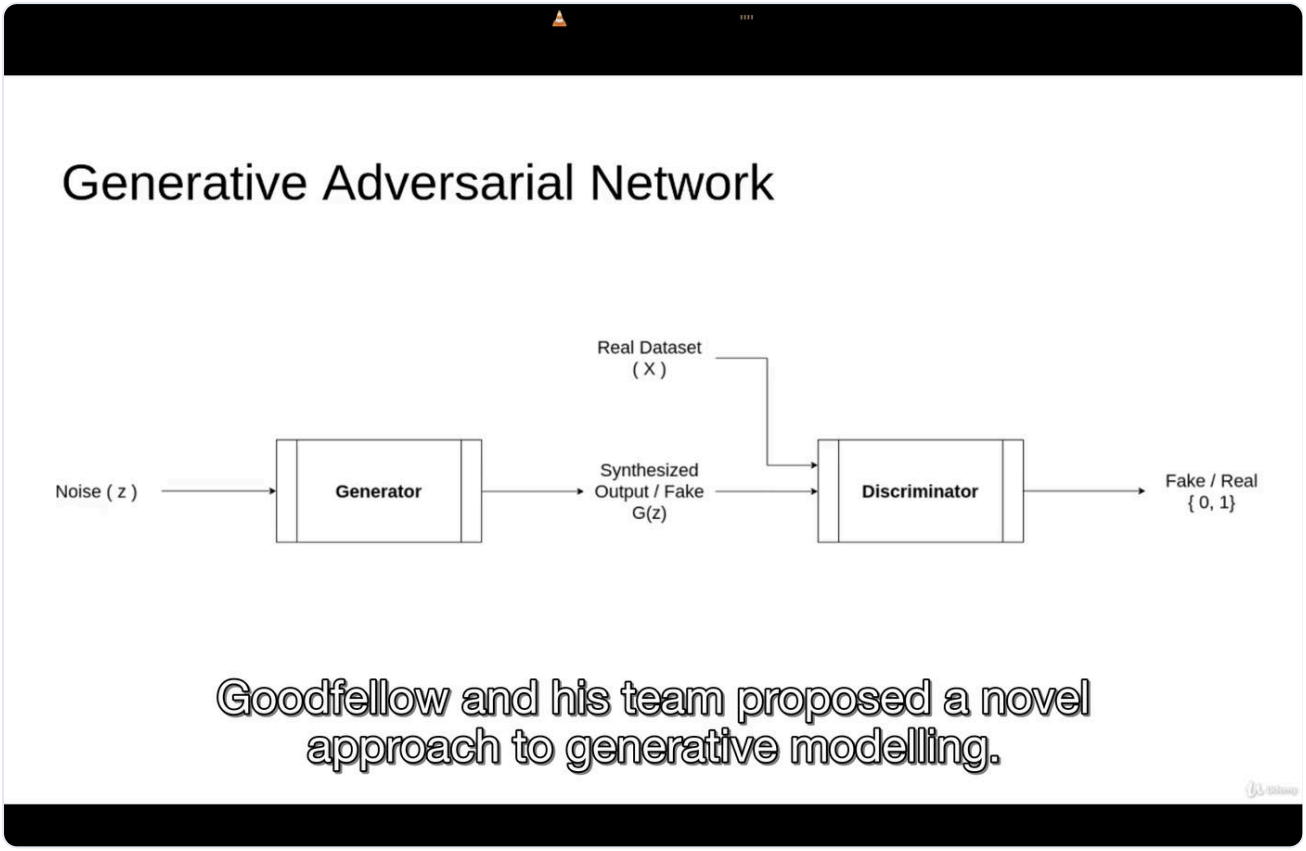


FIGURE 2.5

In simple words the model converges when the green curve overlaps the blue curve in the year 2014 and Goodfellow and his team proposed a novel approach to generative modelling.

It is by far one of the major breakthroughs in the field of deep learning. The idea is very intuitive. They train two adversarial models.

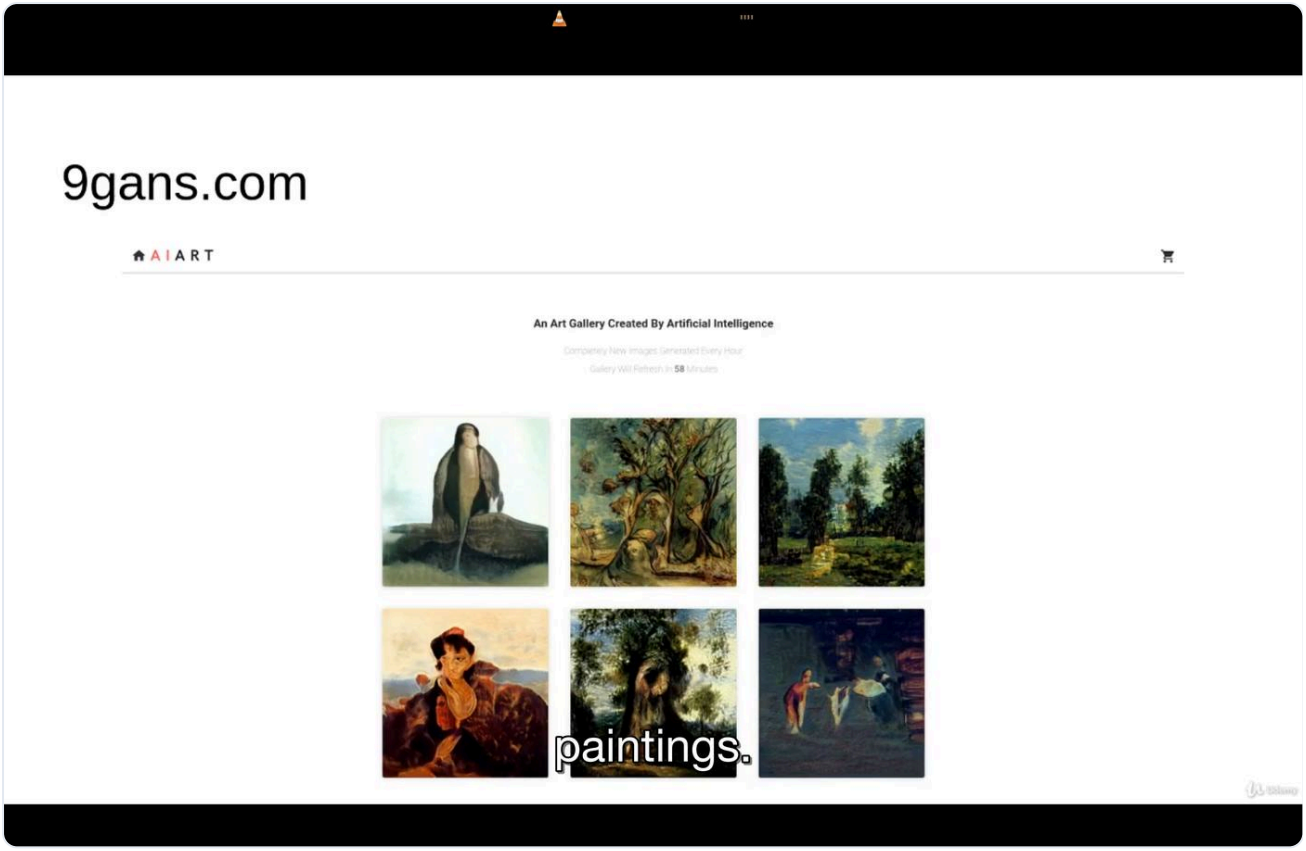


FIGURE 2.6

One is named the generator and the other is a discriminator The generator is trying to generate samples that look as real as possible. On the other hand the discriminator is trying to find out whether the samples are real or fake The generator is given noise as input drawn from a random distribution. Its job is to learn a mapping from a random distribution to the original data distribution.

The discriminator is given as input during training both real and generated samples the output of the discriminator is a zero for fake images or one for real images generative adversarial networks caused a very strong vibe on their release.

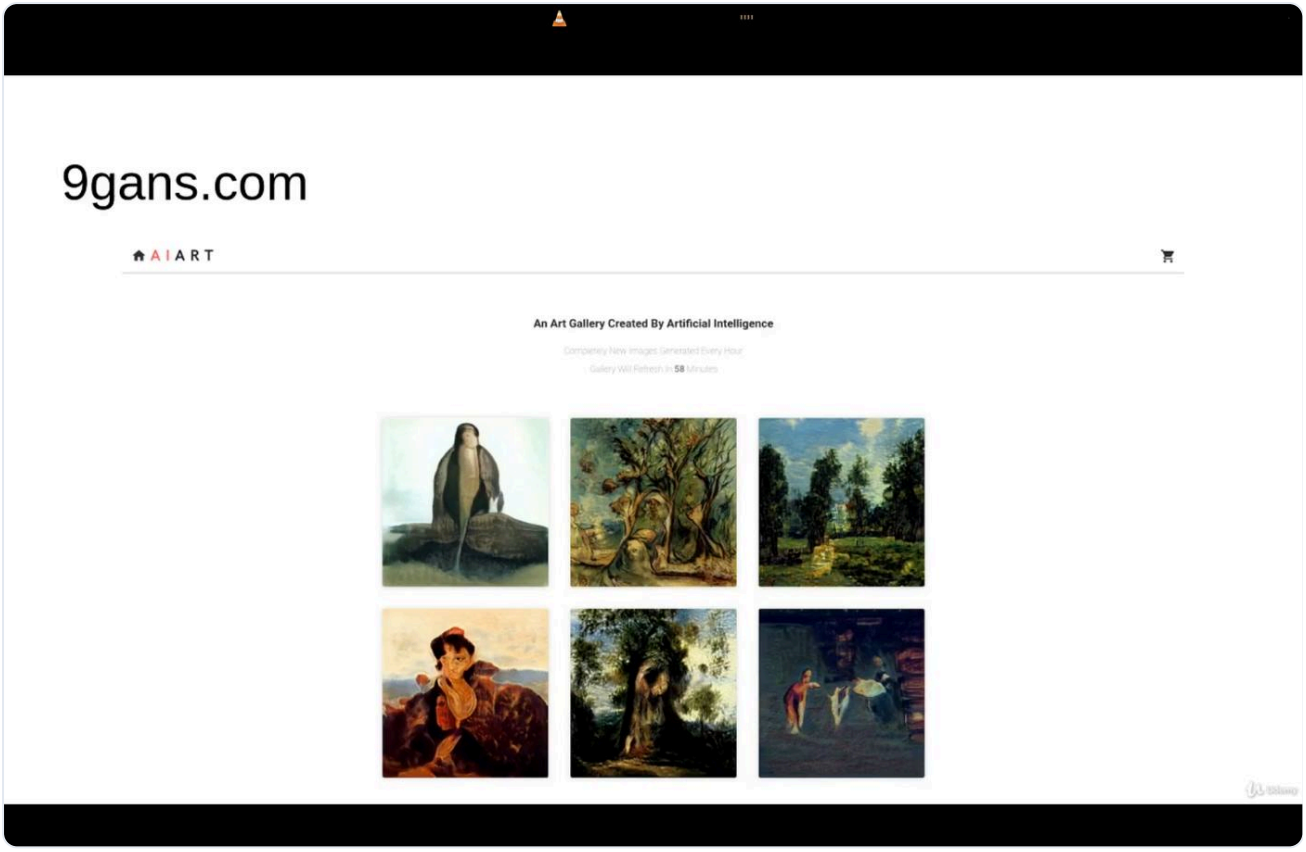


FIGURE 2.7

Next we review some of the entertaining applications of GANs can be used to generate artistic paintings. These paintings are for sale or 9 GANs that come. You can see that these paintings cannot really be distinguished from the ones painted by human artists.

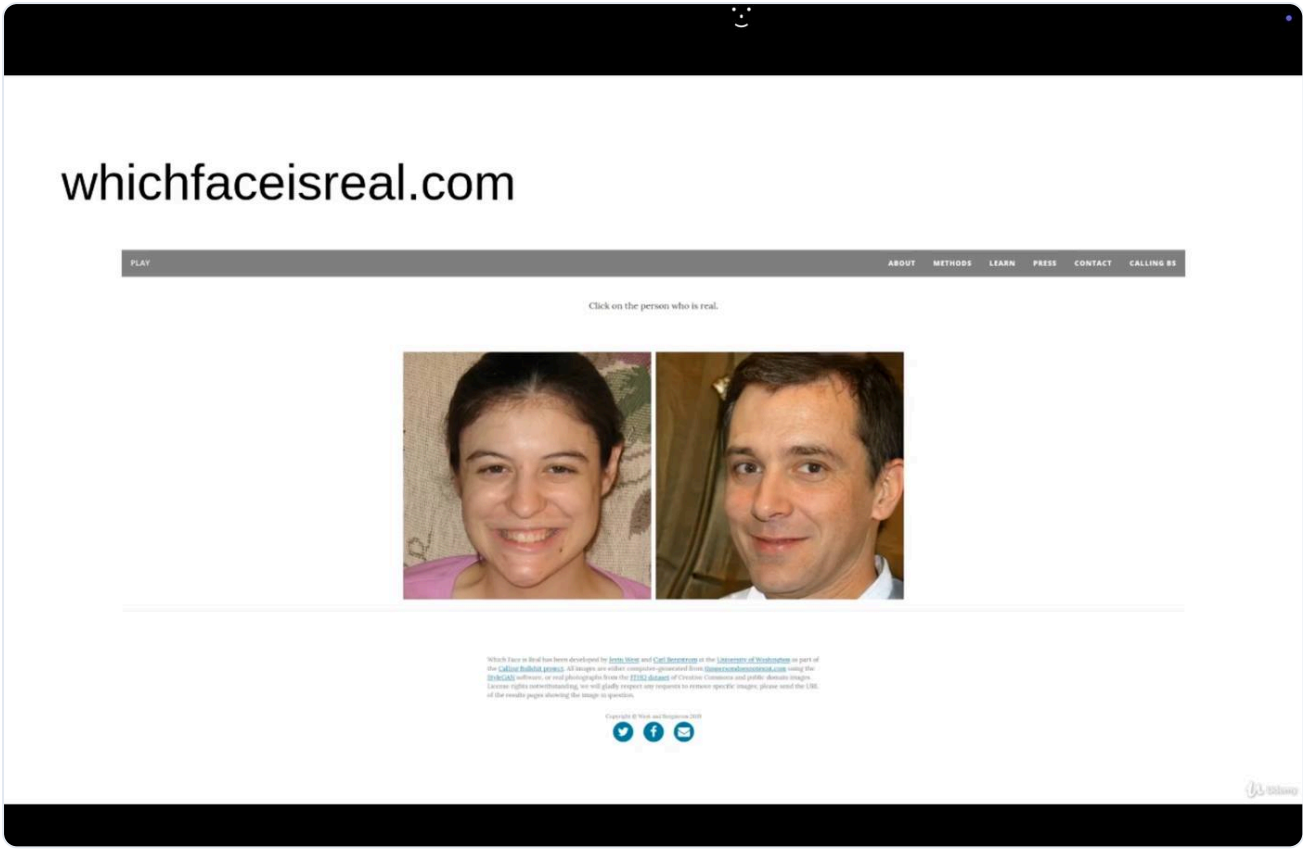


FIGURE 2.8

Which face is real that come is a game where you are given two human faces. One of them is real and the other is generated by a GAN. Then you are asked to click which one is the real one. I must tell you that I get confused and choose the wrong one sometimes.

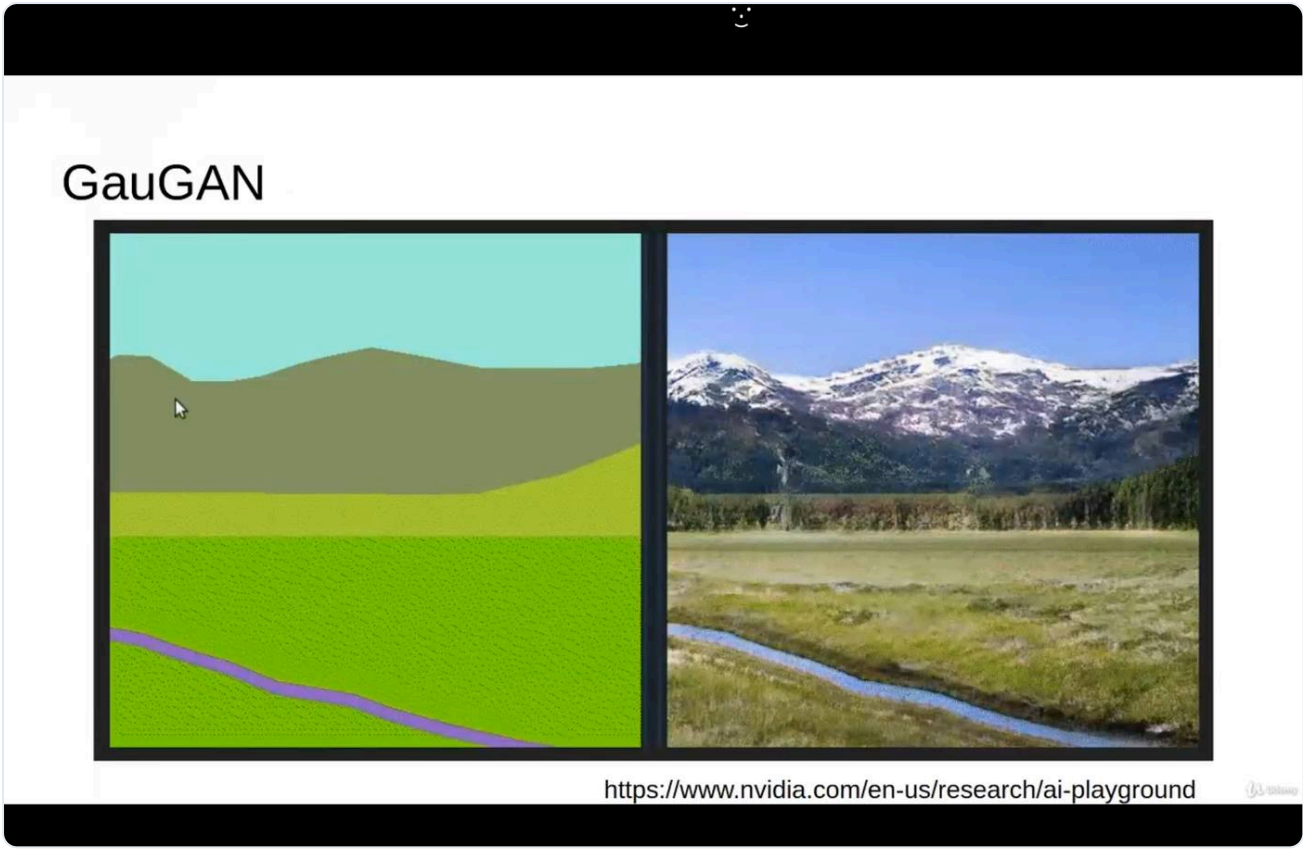


FIGURE 2.9

I think you should give it a try Nvidia has been pushing the frontiers of deep learning applications in general and especially GANs on the NVIDIA GauGAN playground you can find them where you draw regions with a brush and choose category for each region such as clouds, land, snow, forest or mountains.

Then the model will generate natural scenes based on your drawing the link is on the screen. If you'd like to give it a try retail business is one of the most potential areas for applications of GANs. There are many attempts to revolutionize the shopping experience.

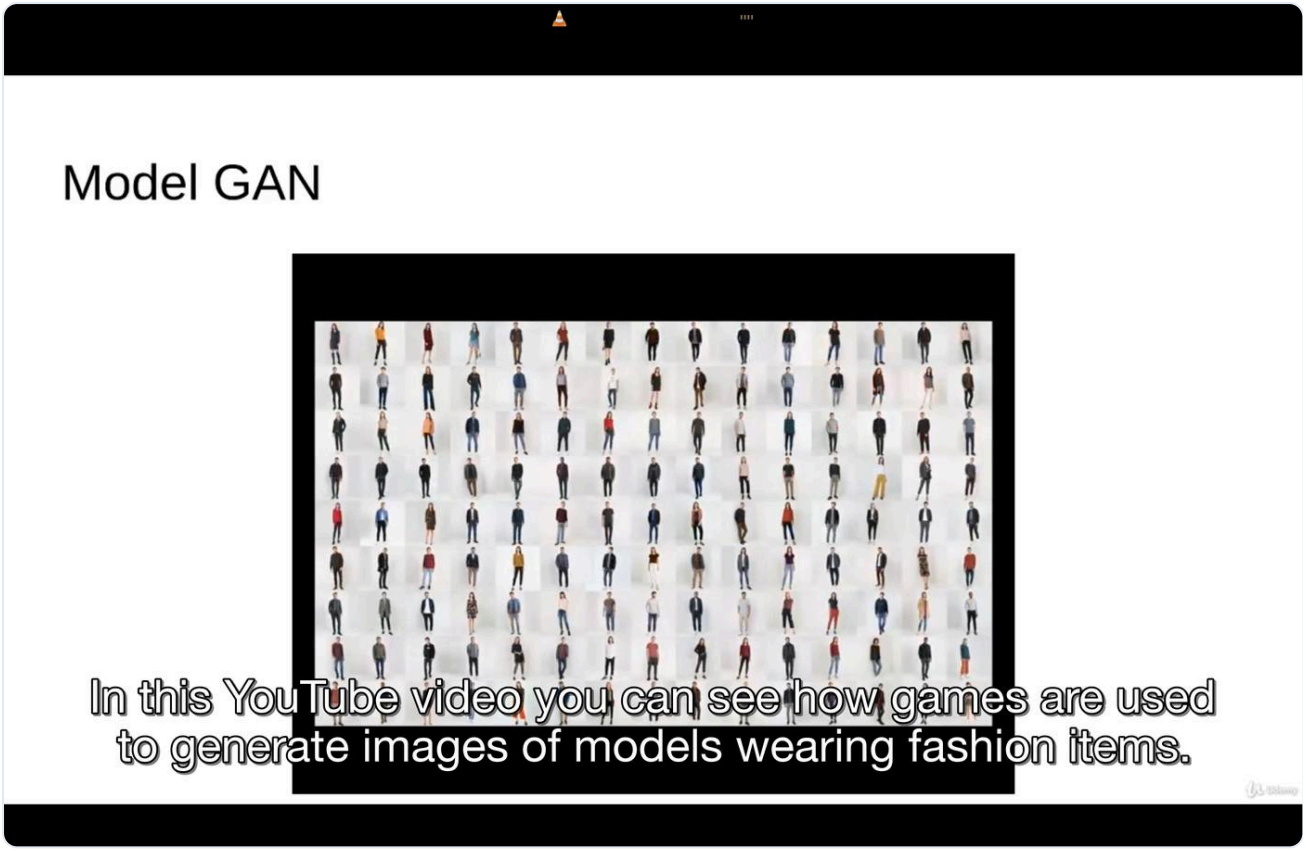


FIGURE 2.10

In this YouTube video you can see how games are used to generate images of models wearing fashion items. The poses, the motion the faces is all generated by a GAN and the next video lecture.

we shall discuss the GAN loss function in detail. Thank you and see you in the next lecture.

LECTURE 6

Working of GAN Loss Function

Hello and welcome to a new lecture.

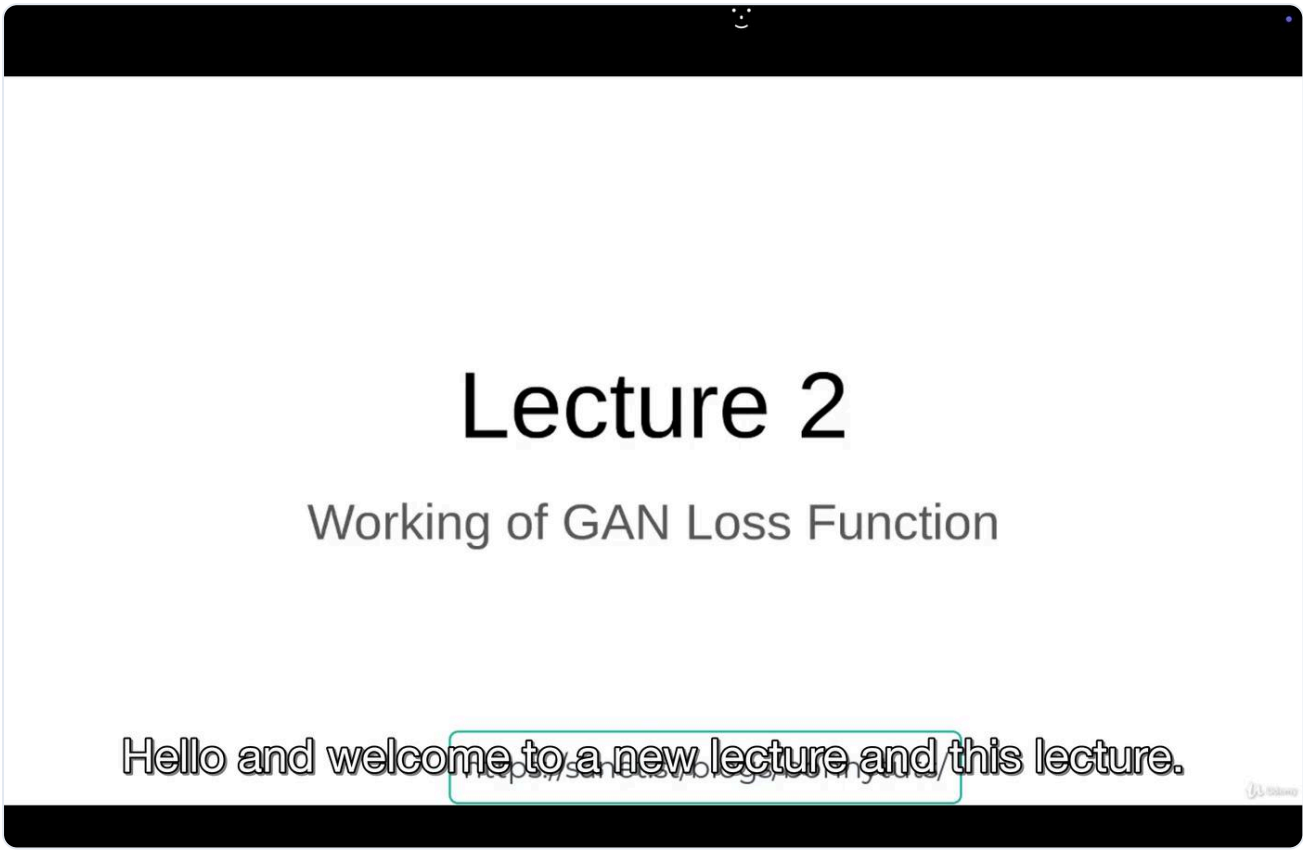


FIGURE 3.1 | Working of GAN Loss Function

In this lecture.

FIGURE 3.2 | Working of GAN Loss Function

Working of the GAN loss function.

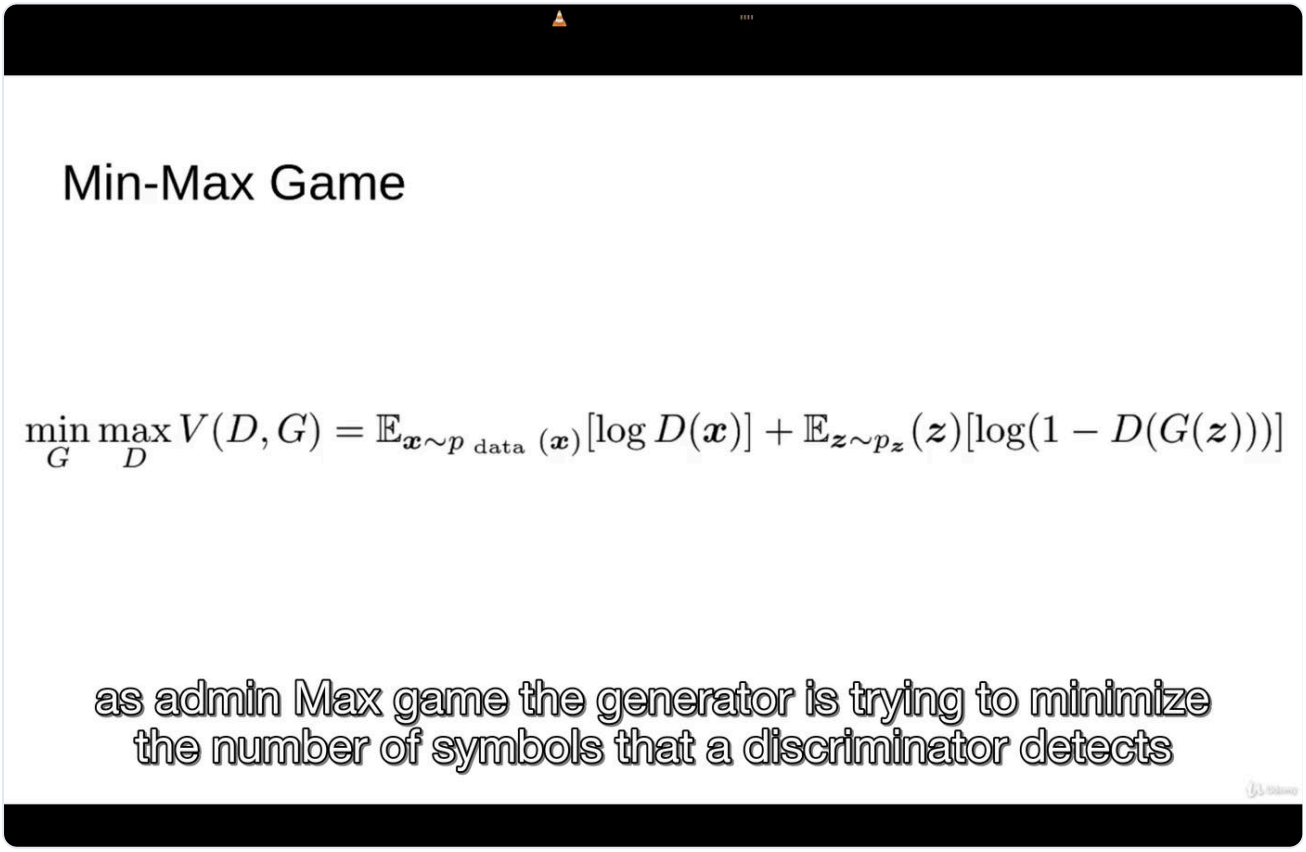


FIGURE 3.3

We shall discuss the min-max game the training pattern of GANs maxout activation functions and a quick review of GANs evaluation methods generative adversarial networks can be formulated mathematically as a min-max game the generator is trying to minimize the number of samples that a discriminator detects on the other hand the discriminator is trying to maximize the number of fake samples it detects in this equation x denotes real samples and Z denotes noise so d of X is discriminator output for real samples and D of G of Z is discriminator output for generated samples since the output for generated samples is expected to be zero we subtract it from 1 to make the formula correct.

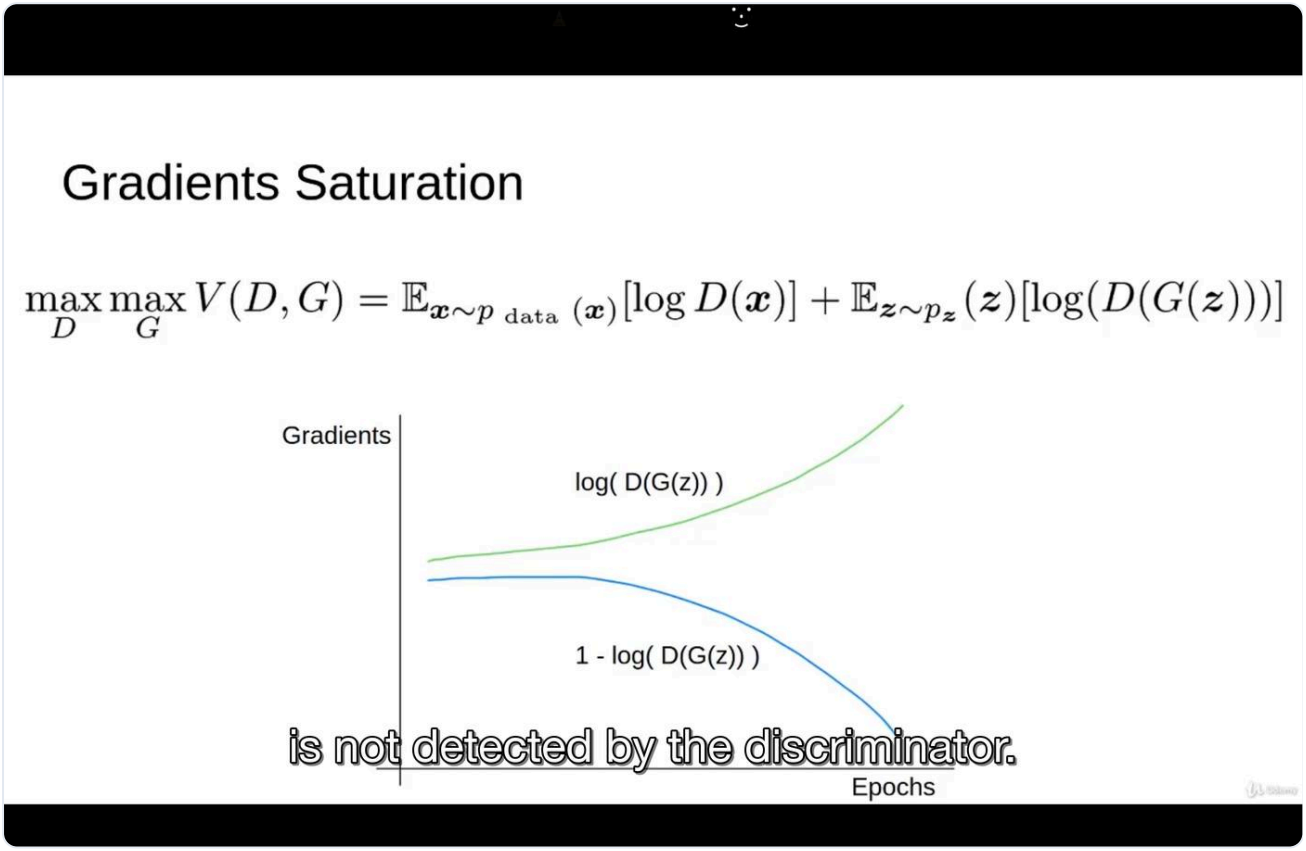


FIGURE 3.4 | Gradients Saturation

It is however found in practice that the discriminator can easily find out fake samples at the beginning of training hence the gradients are almost zero and the generator cannot update its weights to match the efficiency of the discriminator because of this gradient saturation problem is solved by changing the objective of the generator that is by making the generator maximize the probability that its output is not detected by the discriminator.

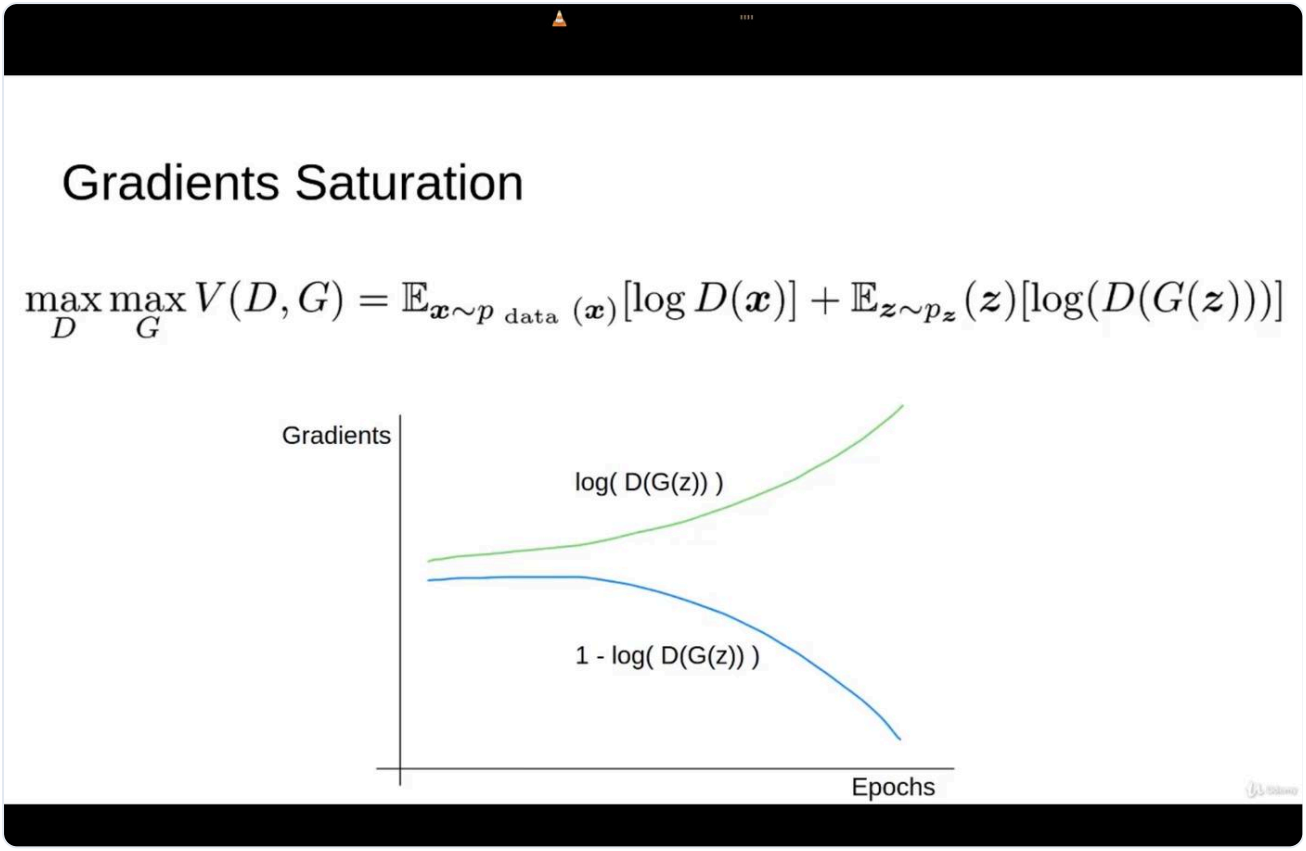


FIGURE 3.5 | Gradients Saturation

This way the generator has a better chance of learning in the graph below the blue curve represents the gradients in the case of min max formulation which is almost plateau for the first training iterations and the green curve represents the new gradient which is of higher values you may ask what if the generator is trained first then the discriminator afterwards.

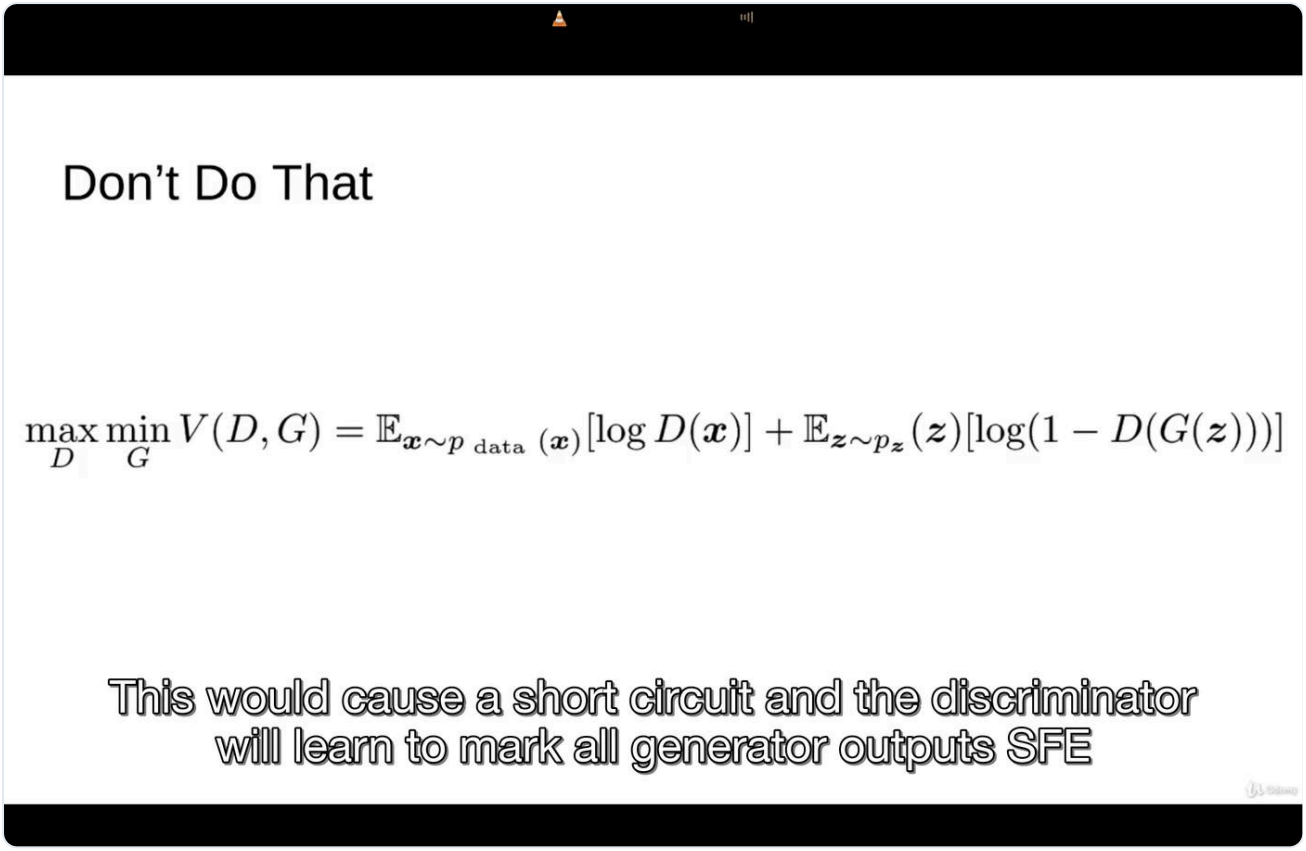


FIGURE 3.6

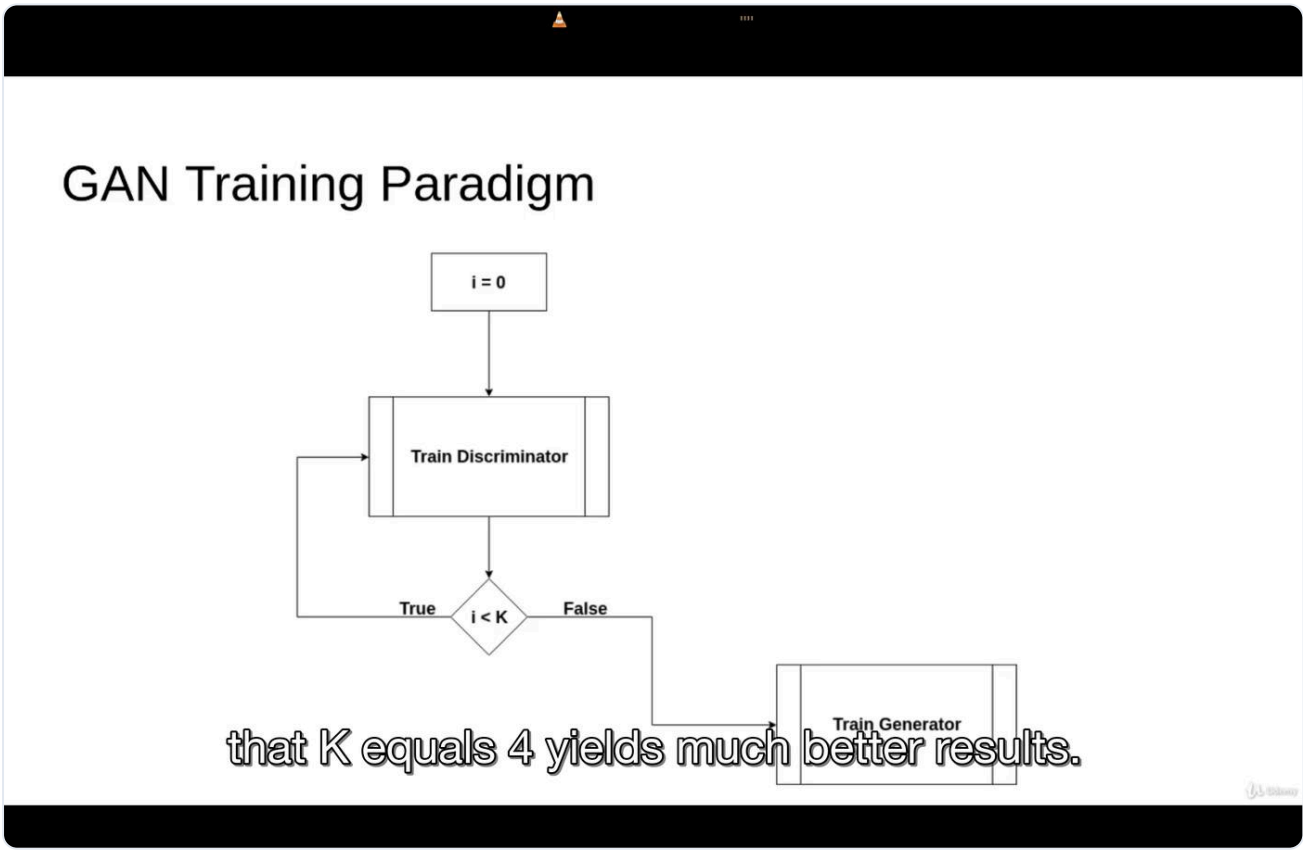


FIGURE 3.7 | K 4 Much Better Res

This would cause a short circuit and the discriminator will learn to mark all generator outputs as fake both the generator and the discriminator are trained in tandem the discriminator is trained first for K steps where key is an integer in the original paper Key was equal to 1 but it is found in practice that K equals 4 yields much better results.

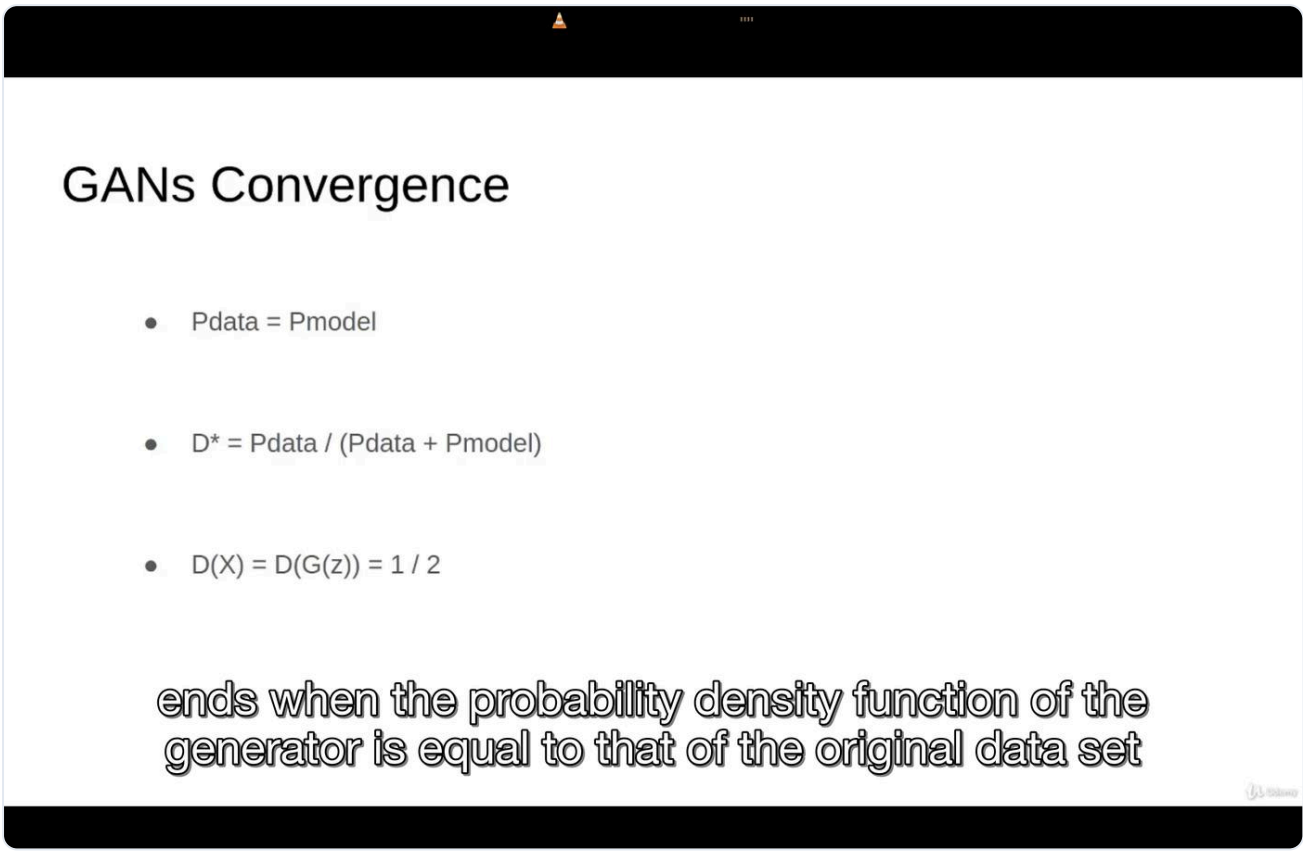


FIGURE 3.8 | GANs Convergence

That is the discriminator is trained for four steps and then the generator is trained once the training ends when the probability density function of the generator is equal to that of the original data set all the mathematical equations on the screen right now are equivalent for those who are interested in the mathematical proof of convergence I have put a reference to the paper at the end of this section in an earlier paper published by Ian Goodfellow and his team they contributed a novel activation function.

maxout activation which is a generalization of ReLU and Leaky ReLU activation.

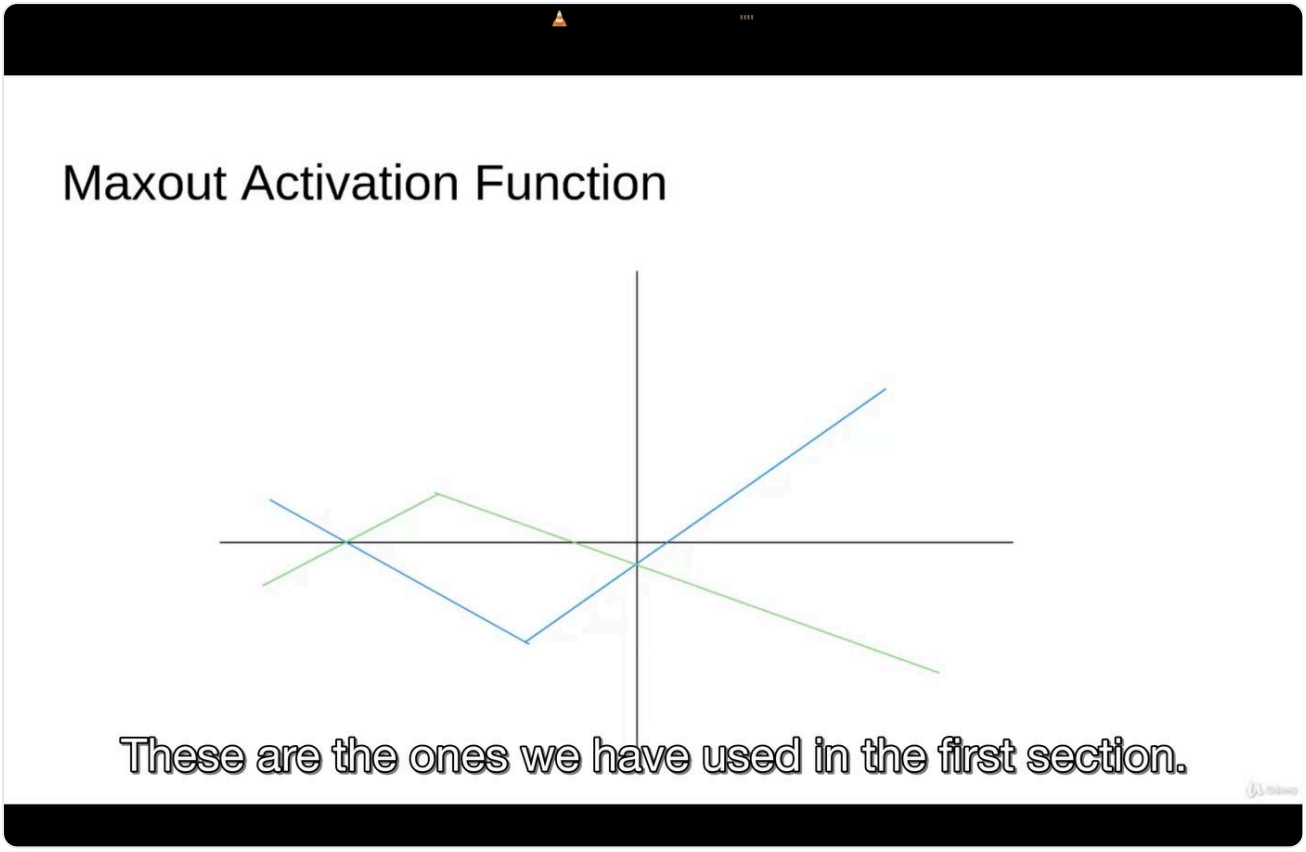


FIGURE 3.9 | Maxout Activation Function

These are the ones we have used in the first section. maxout is a more robust and avoids overfitting. It does not also suffer from vanishing gradient problem. The downside however is that the number of parameters is doubled for every single neuron which leads to a high number of parameters maxout activation is the one used by Ian Goodfellow and his team in the original paper of GANs and many other papers published afterwards.

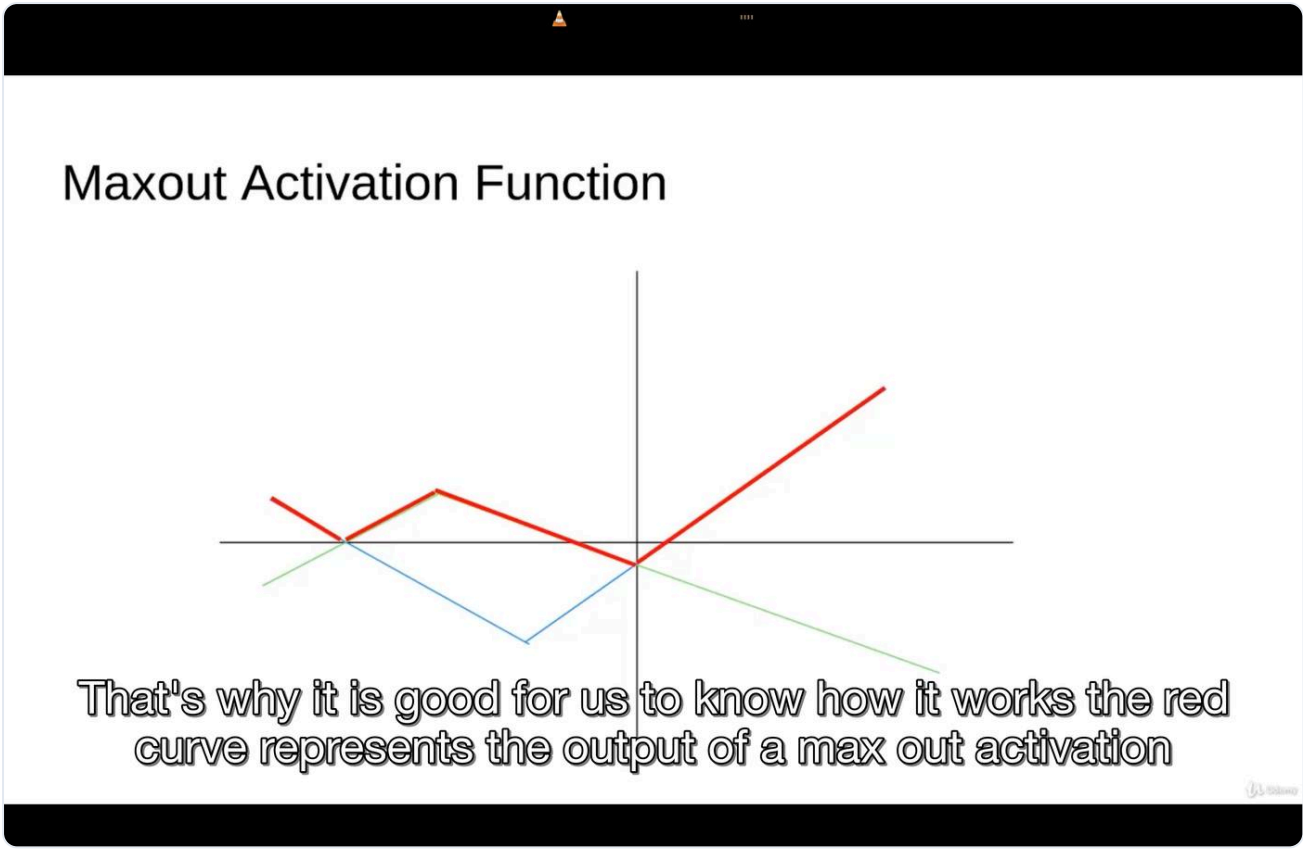


FIGURE 3.10 | Maxout Activation Function

That's why it is good for us to know how it works the red curve represents the output of a maxout activation which is given two linear functions.

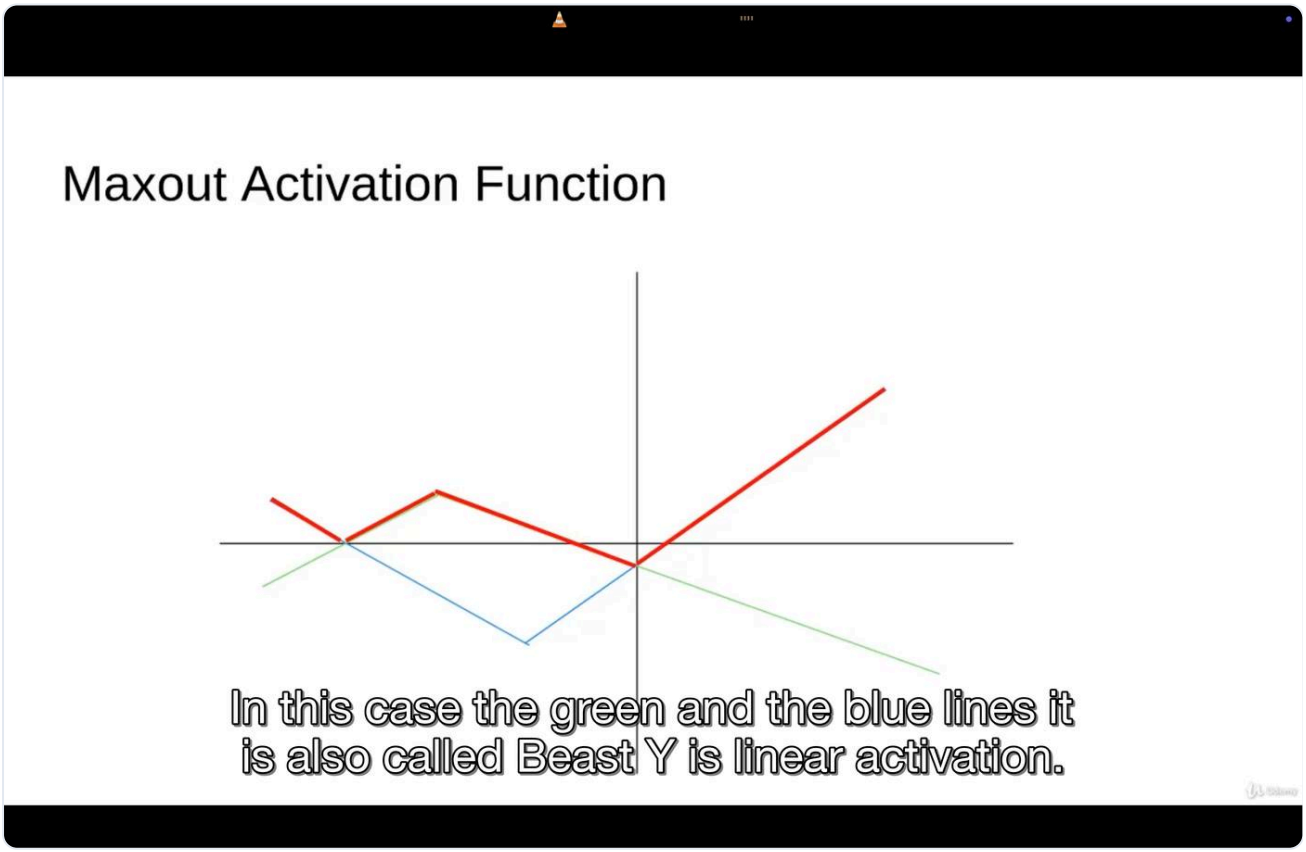


FIGURE 3.11 | Maxout Activation Function

In this case the green and the blue lines it is also called piecewise linear activation. This has the advantage overall since it has steeper slopes which allow for better gradient flow in the network.

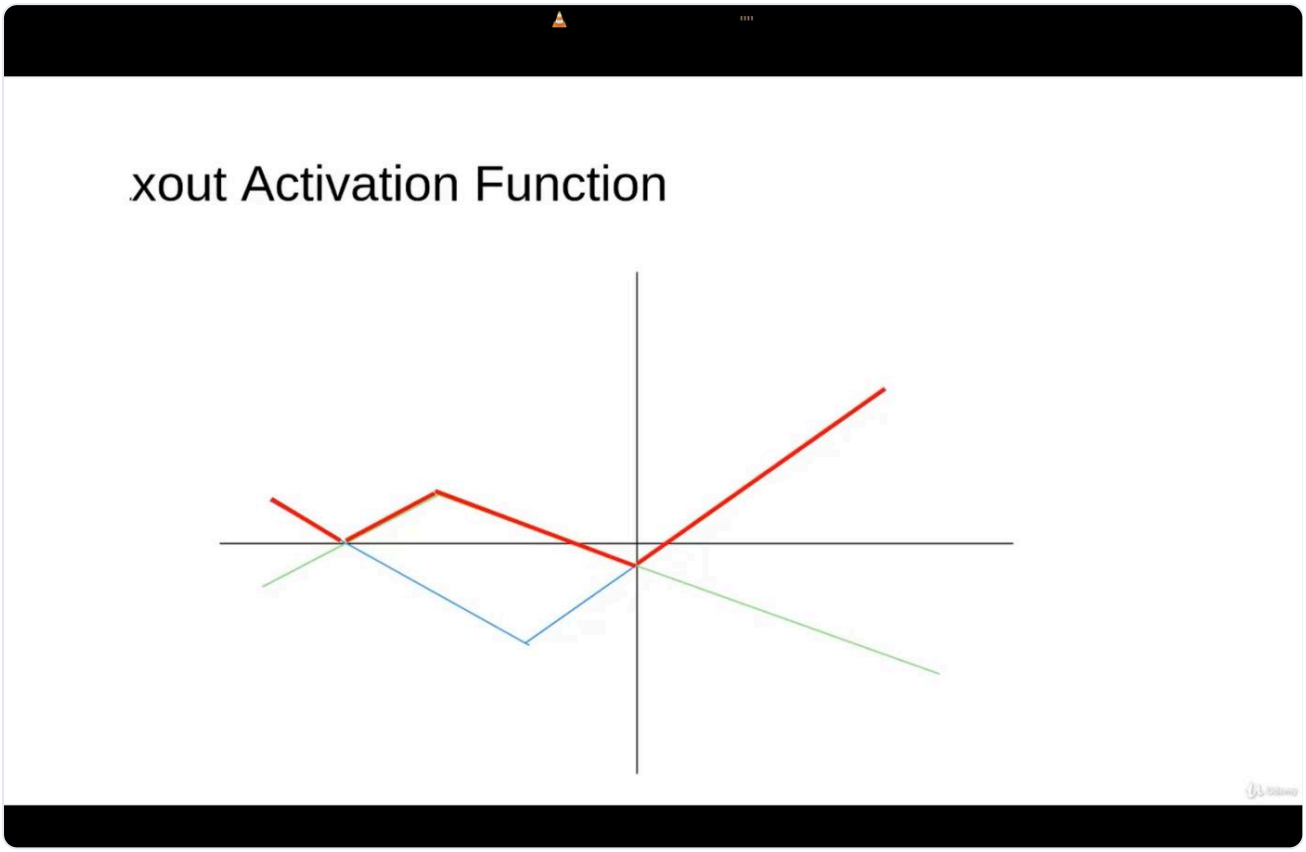


FIGURE 3.12

Let's review a quick example of maxout in this example we have a hidden layer of a network and the maxout is performed with k equals 2.

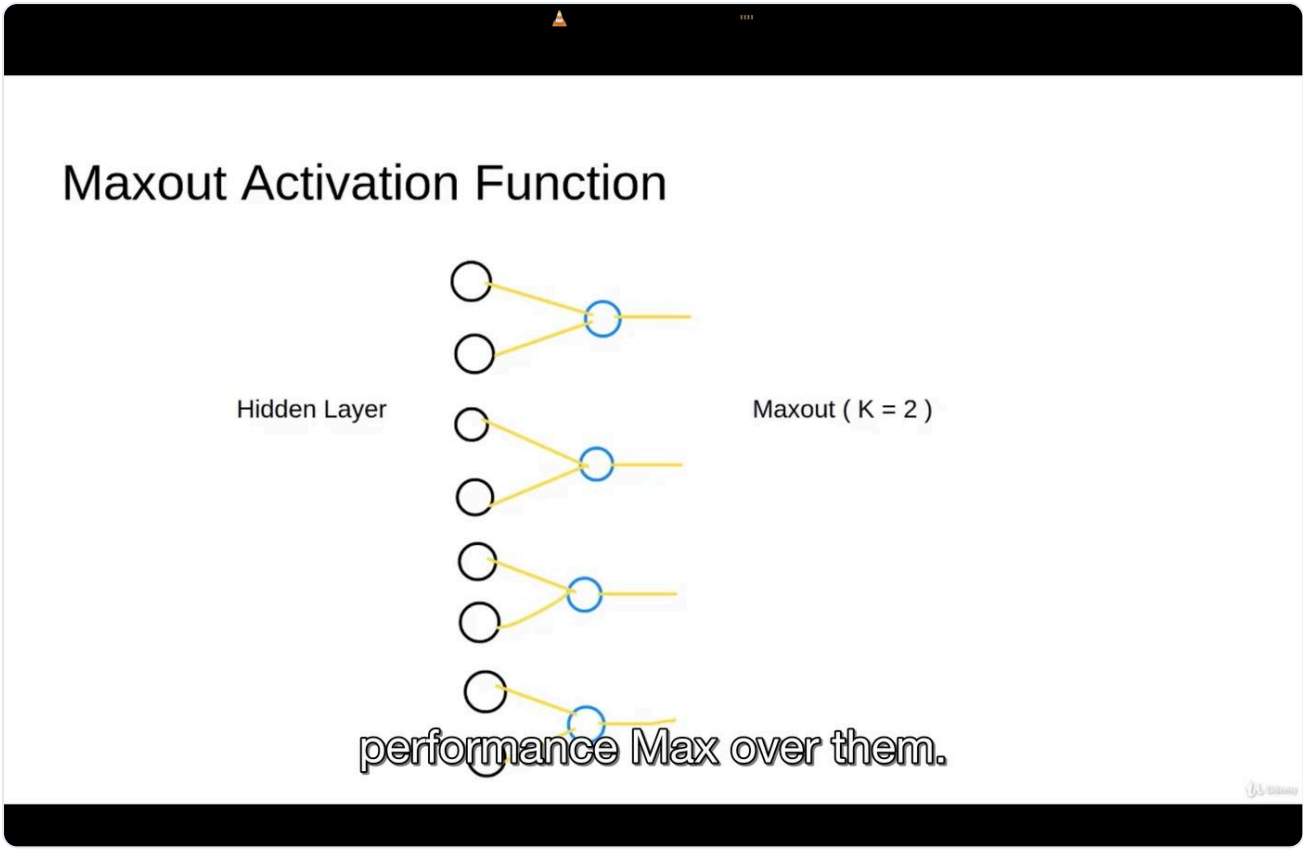


FIGURE 3.13 | Maxout Activation Function

That is the max is taking over every couple of neurons. Think of maxout as another hidden layer that is only connected to K units of previous layer and that performs max over them.

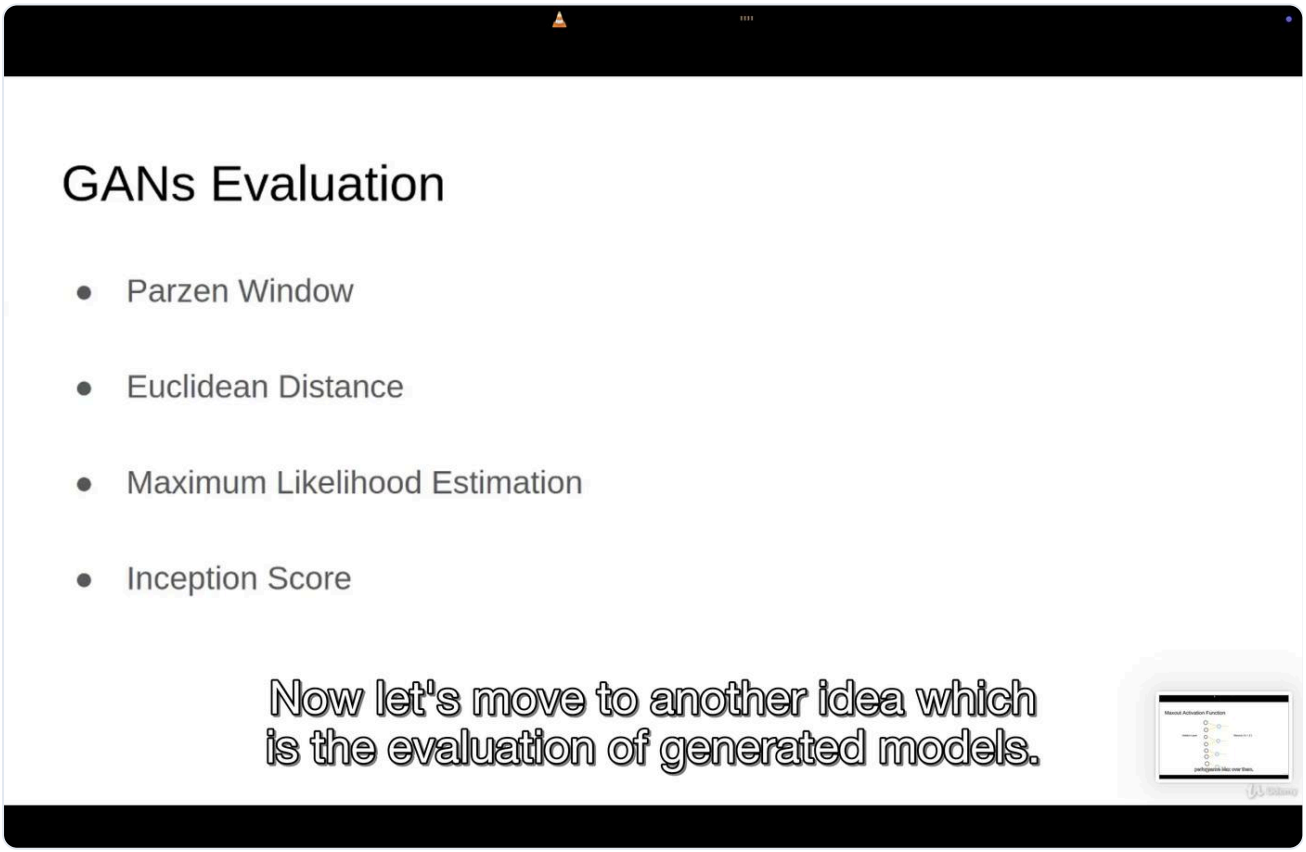


FIGURE 3.14 | GANs Evaluation

Now let's move to another idea which is the evaluation of generative models. There are a few ways to evaluate the results of GANs. Some of these methods also work for generative models in general but each has its own downside. For example Parzen window which is a kernel based density estimator fails to capture high dimensional features such as of images.

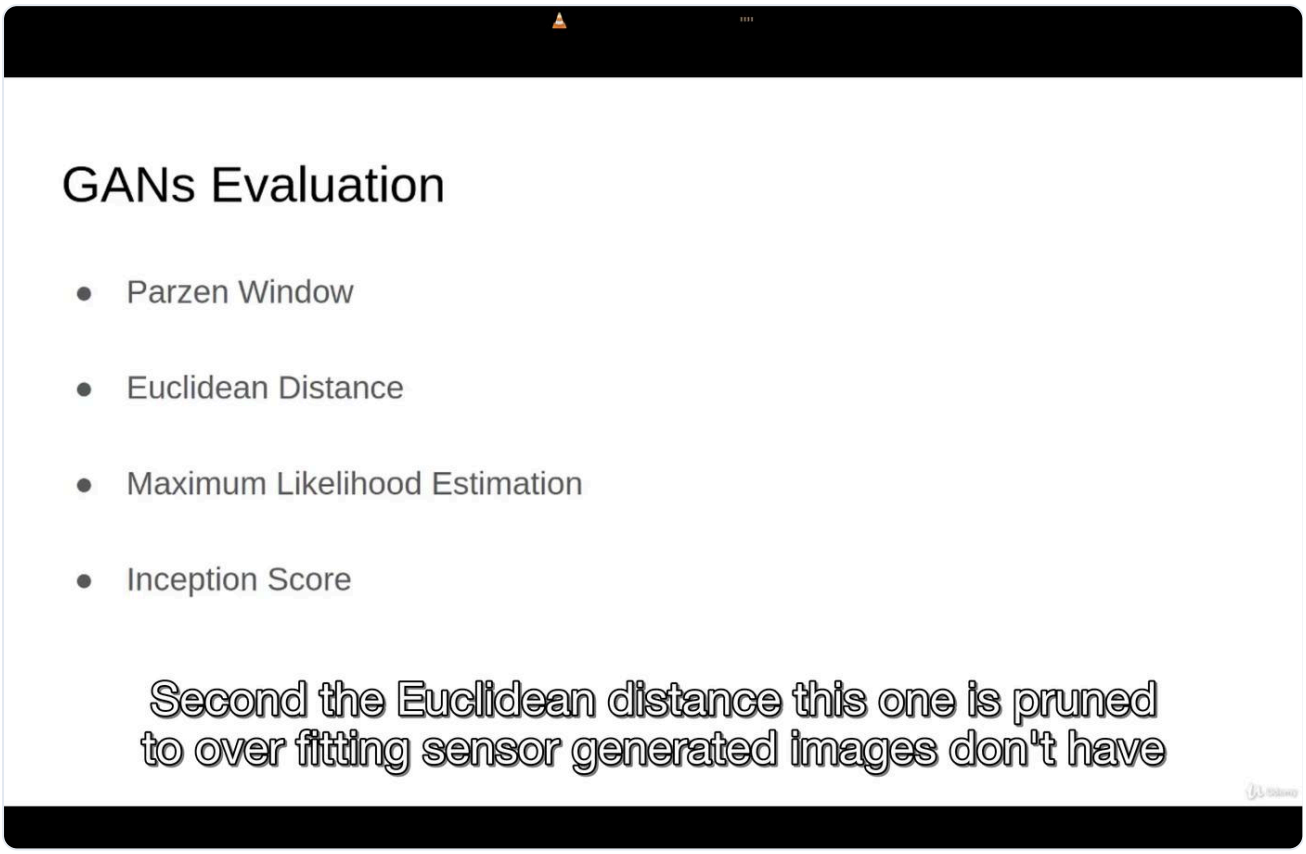


FIGURE 3.15 | GANs Evaluation

Second the Euclidean distance this one is prone to overfitting since the generated images don't have to be exactly the same like the original ones. Third we have maximum likelihood estimation which in most generative models is intractable. That's another way of saying that it cannot be solved analytically. Maximum Likelihood Estimation works by giving the wrong benchmark. The downside of maximum likelihood estimation is apparent when it gives it the wrong benchmark for a lower bound.

For example if we have one model and that one has a lower bound on its value and this lower bound is not used on the other model which we are comparing. In this case we will not be comparing an apple to an apple. The last evaluation metric on our list today is the Inception score which can be misleading as well because it is relevant for images similar to ImageNet dataset but irrelevant. Otherwise that's why the evaluation of generative models is an open area of research in the next lecture.

We shall get our hands dirty and implement our first GAN. Thank you and see you in the next lecture.

LECTURE 7

Number of Pieces and Fully Connected

See the notebook `4.Implementation-of-GAN-Training-Methodology.ipynb` for the code.

Implementing GAN Training Methodology

Hello and welcome to a new lecture. In this lecture we are going to implement the base GAN model and apply the training methodology.

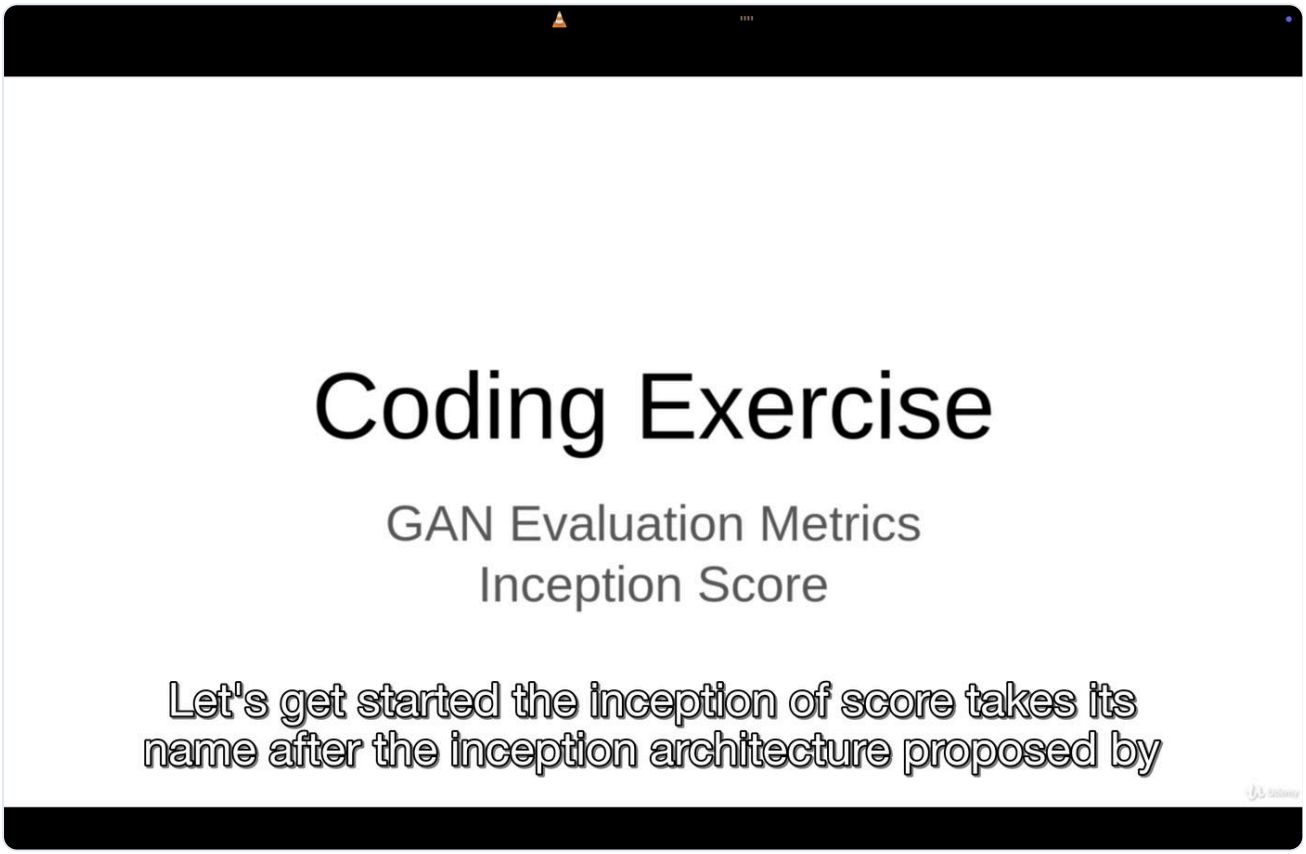


FIGURE 4.1

Let's get started we start by opening the Jupyter notebook for this section. Section 2 which is available in the course material. We start by importing the necessary libraries torch, numpy and matplotlib. And don't forget to use the directive matplotlib inline so that the figures will be rendered inside the Jupyter notebook next to it make sure that CUDA is available.

If you've got CUDA installed and you've got a GPU. And if you don't have a GPU don't worry. You can always use Google Colab which is available at the end of this section a reference to the same code on Google Colab afterwards which defines a device. If CUDA is available the device would be the GPU cuda device. Otherwise we use the CPU. This is important to ensure that the code runs whatever you have as a GPU or you don't have a GPU know we define a global variable noise_dim for noise dimension.

This defines the length of the vector of the noise that will be given to the generator so that the generator will use that noise which is sampled from the random distribution Gaussian distribution to map the random distribution to the data distribution available in the training set. In this case MNIST dataset it is convenient to define the noise dimension to be 100. You can play around with other values as well next to define the generator model similar to what we have done in Section 1.

We define a class generator which inherits from torch.nn.Module inside the constructor def underscore underscore and it underscore underscore we invoke or trigger the constructor of the superclass by using the method super and then we define the sequential model which is the actual model design we are using in this case. We have three fully connected layers. The first layer is linear which is a fully connected dense layer which has got twelve hundred or 1200 neurons and then followed by ReLU activation function then followed by a dropout which is used for regularization.

Remember it acts like bagging or an ensemble of models. This helps the model through training to choose the best one by deactivating some of the links connecting some neurons in the hidden layers. Finally the last fully connected layer is given sigmoid activation which is a squashing function which squashes the output to be either 0 or 1. Remember that when we import the MNIST dataset we converted the data to a tensor and that automatically converts the pixel intensity values from 0 to 255 down to from 0 to 1.

And this way when we apply the sigmoid activation we ensure that we always get output of the generator for the pixel values between 0 and 1 and that will act similar to the original samples available in the data set. Finally inside the forward method we defined the operations to be carried out during the forward pass. We reshape the model to get a two dimensional matrix that's of the batch size and then of the other size by using the negative one to mean whatever is the next size going to be.

And then we pass the batch through the fully connected network which we have defined above. Finally we return to the output now let's define the maxout activation function which we have discussed in the slides in the earlier lecture again we define a class maxout inheriting from torch.nn.Module in the constructor we invoke the superclass and then we set the attribute of the class self.num_pieces to be equal to the number of pieces passed as a parameter to the constructor and then inside the forward method we define the operations to be carried out through the forward pass remember forward method is given a batch and the batch in this case is two dimensional matrix.

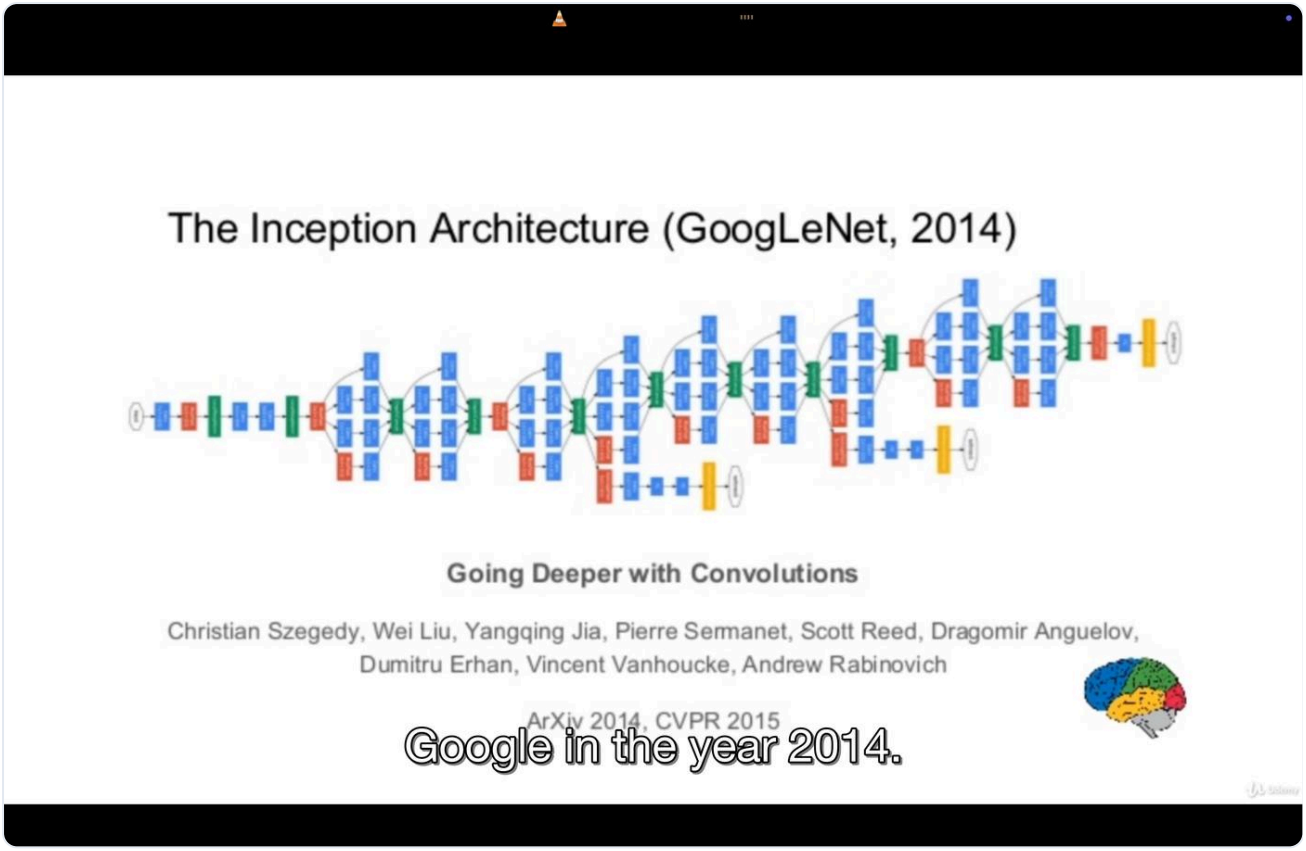


FIGURE 4.2

Since we are working with flattened images in future sections we may deal with three dimensional and four dimensional tensors but this is not the case in this example. First we start by using the method assert which throws an error. If the condition given does not hold in this case we make sure that the second dimension, that is the shape at index one of the batch, is divisible by the number of pieces which we assigned in the constructor. For example if we have got an input the second dimension is 625 and then we have got the number of pieces to be five.

We ensure that 625 modulus 5 is equal 0. Otherwise it throws an error. Next we reshape the input batch to be four dimensional. The first dimension will be the shape of the batch size. The second dimension we divide the original second dimension by the number of pieces. And here we use double slashes which means integer division. The third dimension is the number of pieces. So for example if we have got a batch of 64 samples and then we have got the second dimension of 625 then the first dimension after we reshape the input tensor will be 64.

The second dimension will be 625 divided by five which is one hundred twenty five. The third dimension will be five and then if we've got the remaining dimension we use them. This is only used to make the method more generic and applied for other high dimensional tensors. Next we use the max method available in PyTorch. We use ret.max along the third dimension dimension equal to 2 because the counting starts from zero. So too is a third dimension so that will return the maximum value along with the number of pieces that will give us a tensor of dimensions batch size multiplied by hundred twenty five.

If the original dimension was 625 which is the original dimension divided by the number of pieces and then we return to the output next we design the discriminator model similar to the generator we inherit from torch.nn.Module in the constructor we define the sequential model fully connected layers. But be aware here that we are using fully connected layer and maxout activation and then fully connected layer. So the out features are from the first fully connected layer 625 the maxout number of pieces is 5 then the input features for the next fully connected layer is 125 and then we repeat again by making the out features 625 and maxout activation and then another fully connected layer and so on.

Finally we apply a sigmoid activation function to do the squashing but this time we mean by zero is interpreted as a fake sample a generated sample and one is a real sample, a sample that's available in the training set or a generated sample that cannot be distinguished from the original dataset. Finally we defined that forward method and apply the same re shape to get 2 dimensional vector or tensor and then we apply the fully connected network on the batch and return the output now that we have defined the model and activations in the next lecture we continue the implementation and apply the training methodology and see the output.

Thank you and see you in the next lecture.

LECTURE 8

Implement Vanilla GAN on MNIST Dataset to Generate Digits

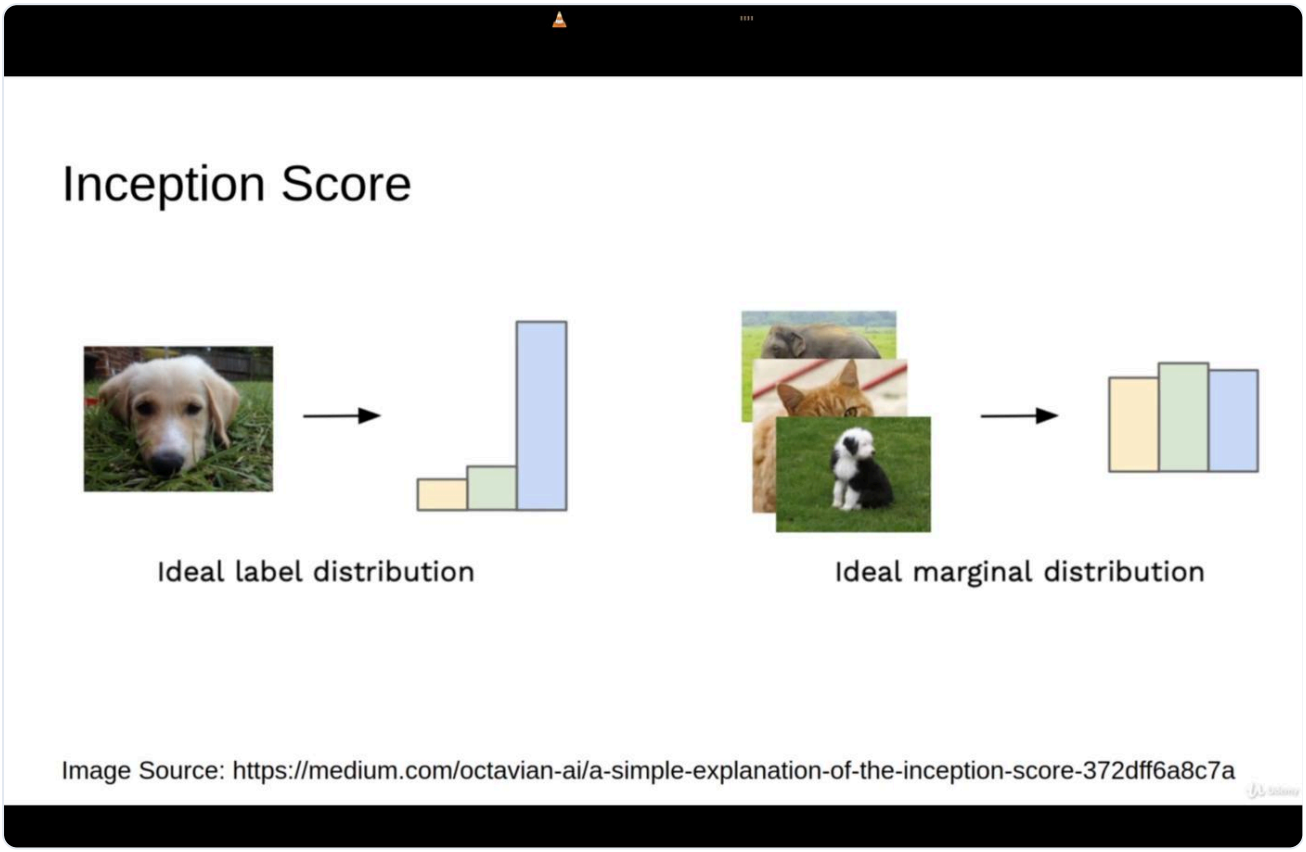


FIGURE 5.1

See the notebook `4.Implementation-of-GAN-Training-Methodology.ipynb` for the code.

Hello and welcome to a new lecture. In this lecture we're going to continue the implementation of the base GAN model and apply the training methodology. Let's get started in the previous lecture we ended by designing the model for the generator and the model for the discriminator and we applied the maxout activation function and the other details of the model designs so we continue by importing torchvision which contains the data set. Next we define the flattening transform which is a transformer to convert the two dimensional images to one dimensional flat vector and then we import to the images data set from torchvision and apply the transformers in the following order.

First we convert to tensor using torchvision.transforms.ToTensor which converts the intensity from the range 0 to 255 down to 0 to 1 and returns a byte tensor. And then we apply a flatten transform next we define a global variable the batch size I set it to 32. You can play around with the value in order to fine tune the hyperparameters and then will define a data loader which uses

the data train from the torchvision dataset we have defined above and is given the batch size and we don't forget that we set shuffle to True and we set the number of workers I set it to 4 continue to 2 or 1 as you like.

Next we do the actual GAN training we started by creating a new instance of the generator and the discriminator models and we convert them to the device we are working on. Now we get an object in a variable generator and discriminator next we define the optimizer remember that we need an optimizer for the discriminator and another one for the generator. And you can set a different learning rate for each one of them and different momentum. In this case we are using stochastic gradient descent and we pass the model parameters either the discriminator or the generator and then we set the learning rate.

In this case it was set to 0.02 and then we set the momentum to 0.5. Next we defined the loss function. We use a loss function, criterion, which is binary cross entropy loss now things get more interesting. We define the real labels. That's when we say that the sample is real to be one and we define the fake label when we say that the sample is fake to be zero. Remember that torch.ones returns a matrix of the batch size and one dimensional column and then we convert it to the device so that it is converted either to the CPU or the GPU for the fake labels we use torch.zeros returns two dimensional matrix of zeros the first dimension is the batch size and the second dimension is one column.

Next we define a variable test underscore set. This is going to be a fixed noise that we are going to test along the training epochs so that we see for the same noise how the output of the generator is different. This is kind of a consistent way to see how the network evolves with time test_set is a variable of torch.randn that's random normal. The first dimension is 16. So we are using 16 samples and then the second dimension is the noise dimension.

Finally we convert it to the device we are working on, the GPU.

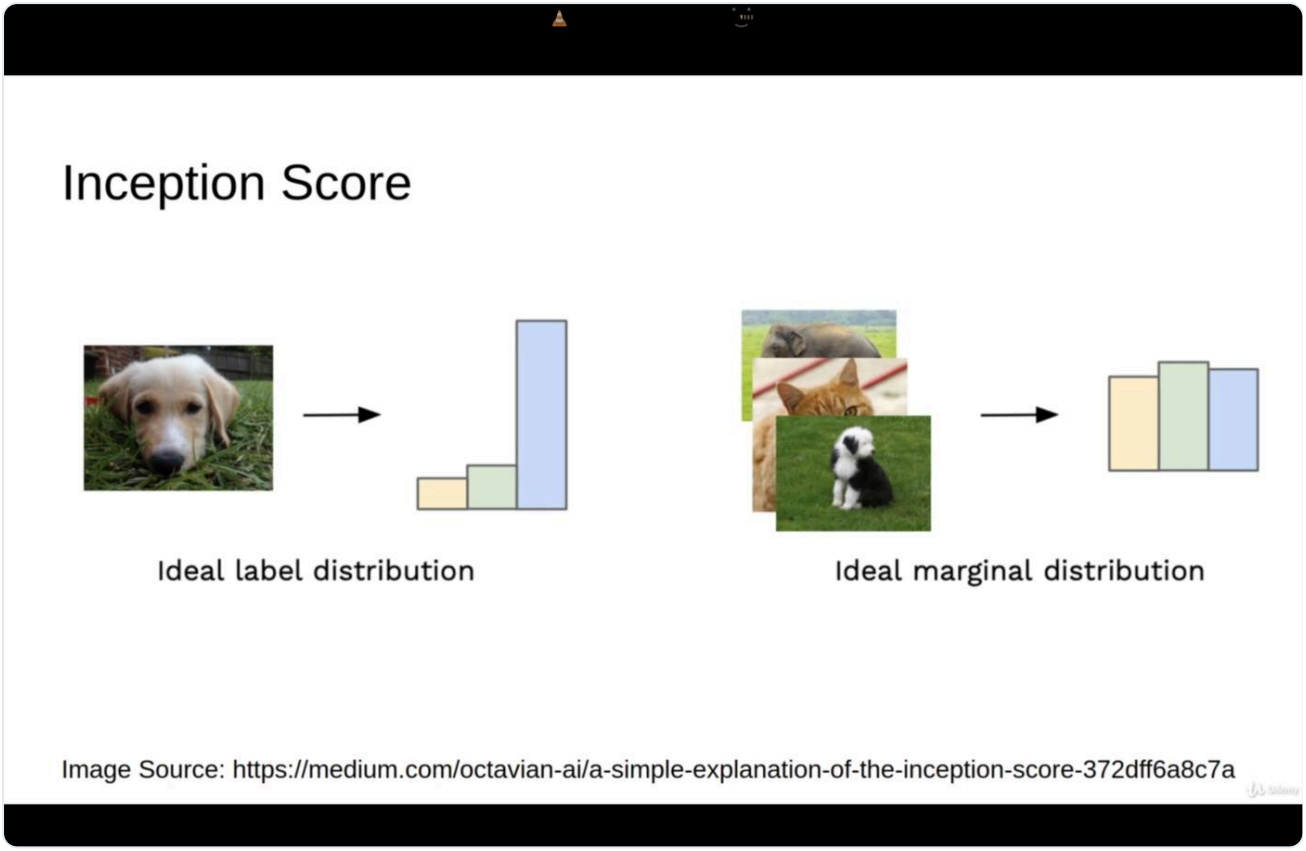


FIGURE 5.2

Next we set the number of epochs to 512 and the number of steps to be the length of the train loader divided by integer division. The batch size since we are going to be creating visuals of the output of the generator for each epoch. Then we create a directory. If it doesn't exist by importing os and then if not os.path.exists. Visuals the name of the directory then we use the method makedir so that we can create the directory.

Here is the actual training for each epoch in the range of number of epochs. That's five hundred twelve and then for each step of the train loader we enumerate the train loader. Remember that it is a generator enumerate will return the index of the iteration variable i. And then the images and the labels. Since we are not interested in the labels in this exercise we use underscore and we get the images in the variable images. If I is equal to the number of steps then we break the loop since the train loader generator will not stop automatically it will keep on going on and on fetching more data.

Actually the same data but will cycle over them since it is a generator. Next we perform the actual training of the discriminator. Remember that we can train the discriminator more than the generator so we use a variable key. In this case I set it to 8 for underscore we are not interested in the index of the iteration, and range it so that is K. Which we have discussed in the lecture slides. We perform this training. The real images are going to be converted to the device by using images to device and then the fake images will come from the generator.

At this point we have not trained the generator yet so they will be very easy for the discriminator to find out whether they are fake or real. But for the next iterations, the next epochs of the training they will be more challenging as you will see when the exercises the input to the generator is

random noise from a random normal distribution. So we use torch.randn and then we give it the dimensions of the vector the batch size for the first dimension and the noise dimension for the second dimension.

The batch size for the first dimension and the noise dimension for the second dimension and again the two device so we trigger the method zero_grad. zero_grad on the discriminator optimizer. So we give the real images to the discriminator and save the output in real_outputs and give the fake images to the discriminator and saves the output in fake_outputs. Next we apply the loss function twice one for the real outputs and the other for the fake outputs.

So criterion is given the real underscore outputs and the real underscore labels which is ones in this case and next time is given the fake_outputs and the fake labels which is zero.

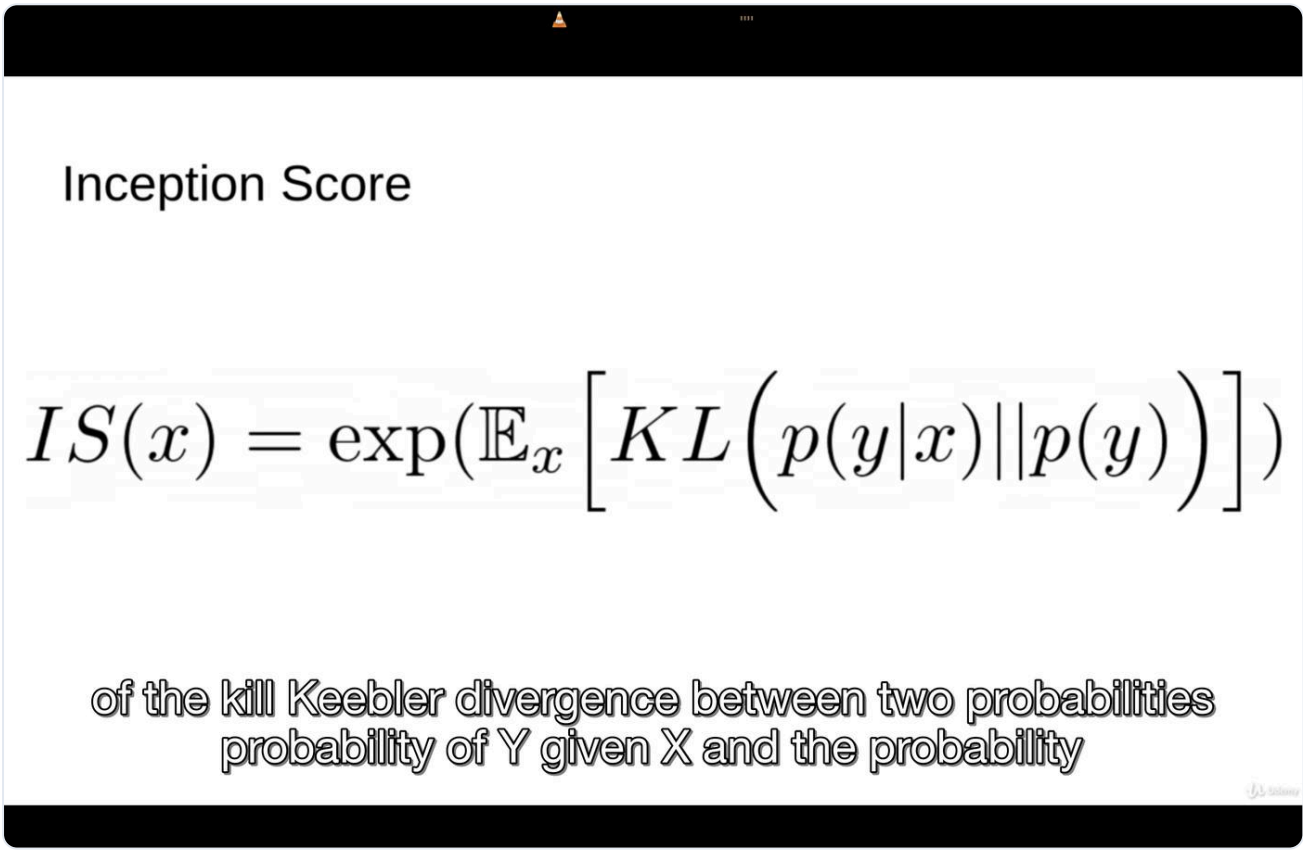


FIGURE 5.3

In this case the first one is a variable we named D underscore x so that you can relate it easily to the equation we have discussed in the slides and D of G of Z so they can relate it to the second part of the equation we are using log 1 minus the G of Z so we apply the backward on both D of X and the backward pass on D of G of Z independently that will accumulate the gradients but will not update the weights till we use discriminator_optimizer.step that will perform the actual update of the weights according to the learning rate that will train the discriminator for K steps.

Next we train the generator we use a random noise tensor of two dimensions batch size and noise dimension and we convert it to the device we are working on and save the output in a variable named Z we use the method generator.zero_grad so that the gradient will be reinitialized or

reset before we apply any training we pass the noise to the generator generator of noise and then the output of the generator would be passed to the discriminator and would be saved in a variable outputs.

So what we are doing here is that we are passing random noise through the generator and then passing the output image to the discriminator to decide whether it is real or fake and then we are using the criterion the loss function to decide whether the output is actually real labels or ones.

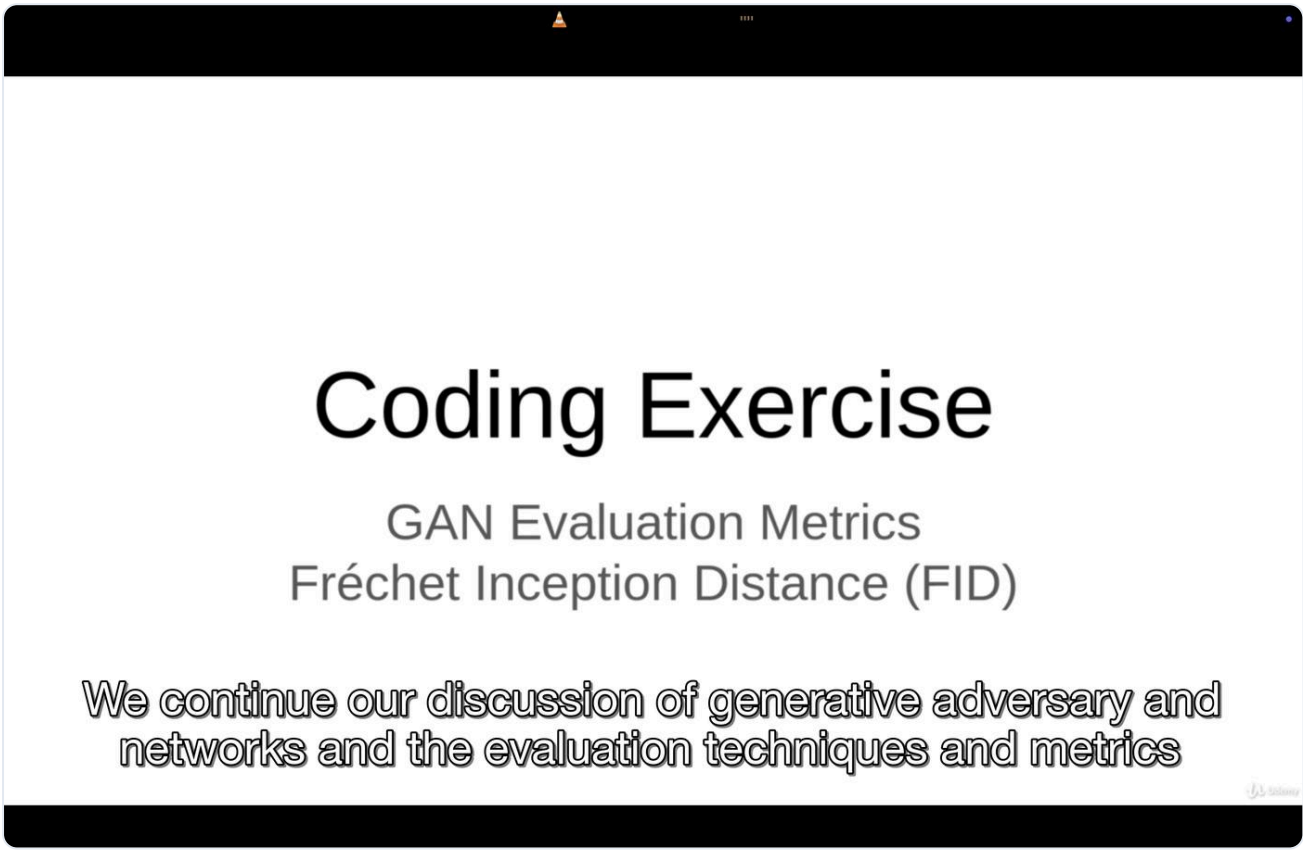


FIGURE 5.4

In this case we are trying to maximize the generator ability to generate real images and see the loss the loss would be saved in a variable loss loss will be used to apply the gradients loss.backward that will calculate the gradients according to the loss and then generator optimizer step will update the weights next every ten epochs we are going to visualize the results we use the test_set we have defined above as a fixed random noise and we detach it from the GPU and convert it to the CPU using .detach().cpu() and then we reshape it like this using the view method.

The first dimension would be the batch size which is 16 in this case the second dimension is the channel, which is only one channel. Since it is a grayscale image and then we reshape it using the view method the first dimension will be the batch size.

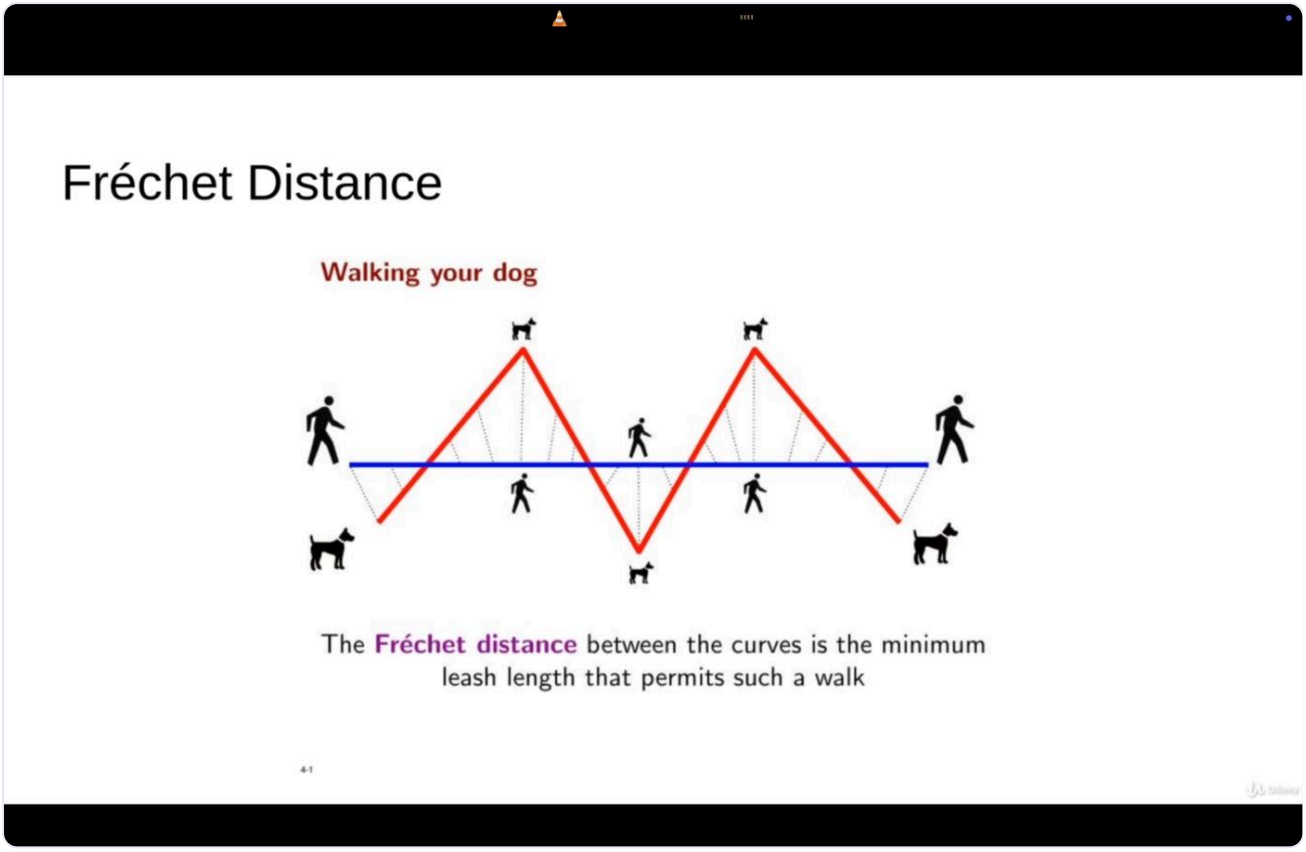


FIGURE 5.5

In this case we are using 16 samples. The second dimension is one which stands for the channel since it is a grayscale image of one channel. The third and the fourth dimensions are the width and height of the image.

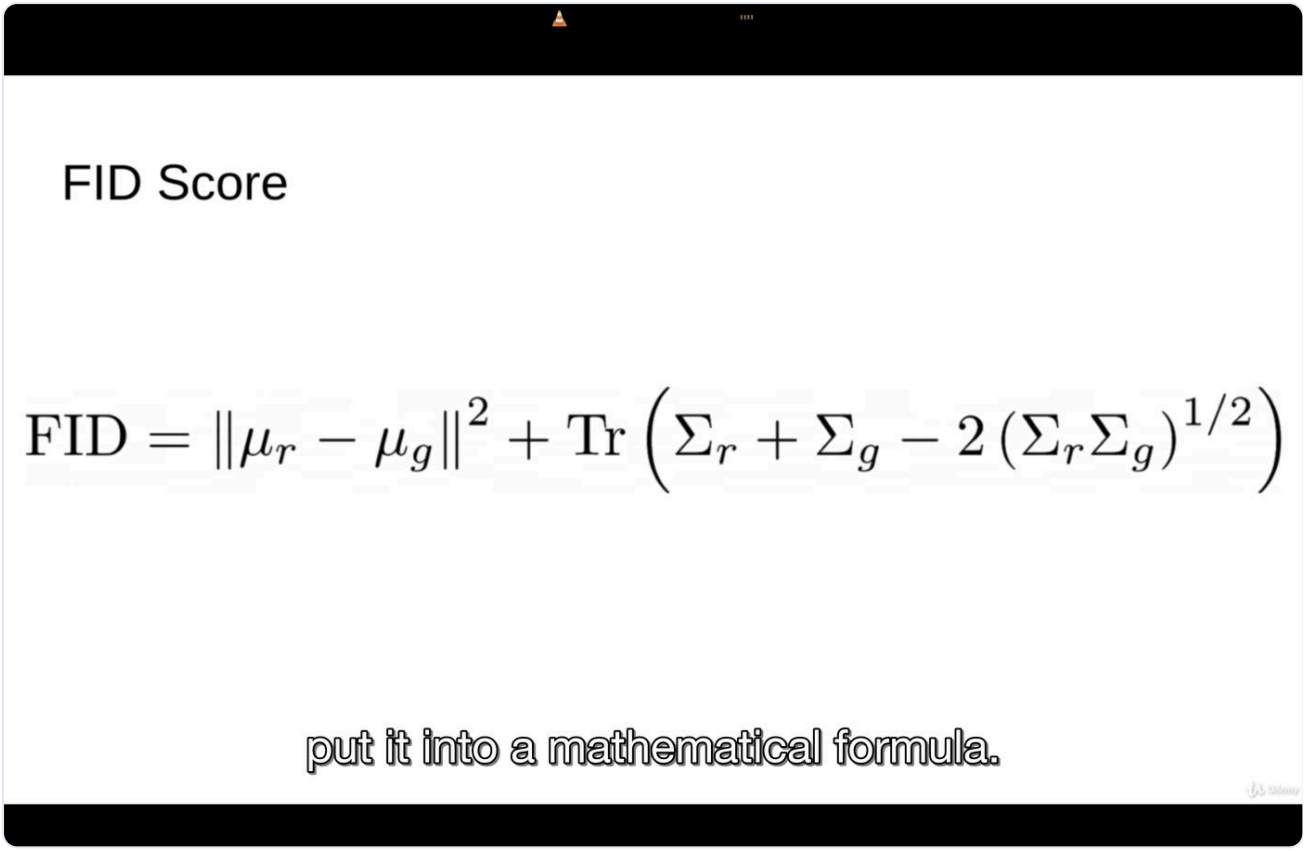


FIGURE 5.6

Twenty eight by twenty eight which is the size of any image in this dataset. We save the output which will be the generated images from the generator that has been trained in the previous steps. Next we use a utility method available in the torchvision using torchvision.utils.save_image that will create a grid of images for the generated images and then we'll save it in the file which we are passing and the second argument by using os.path.join that will create a.

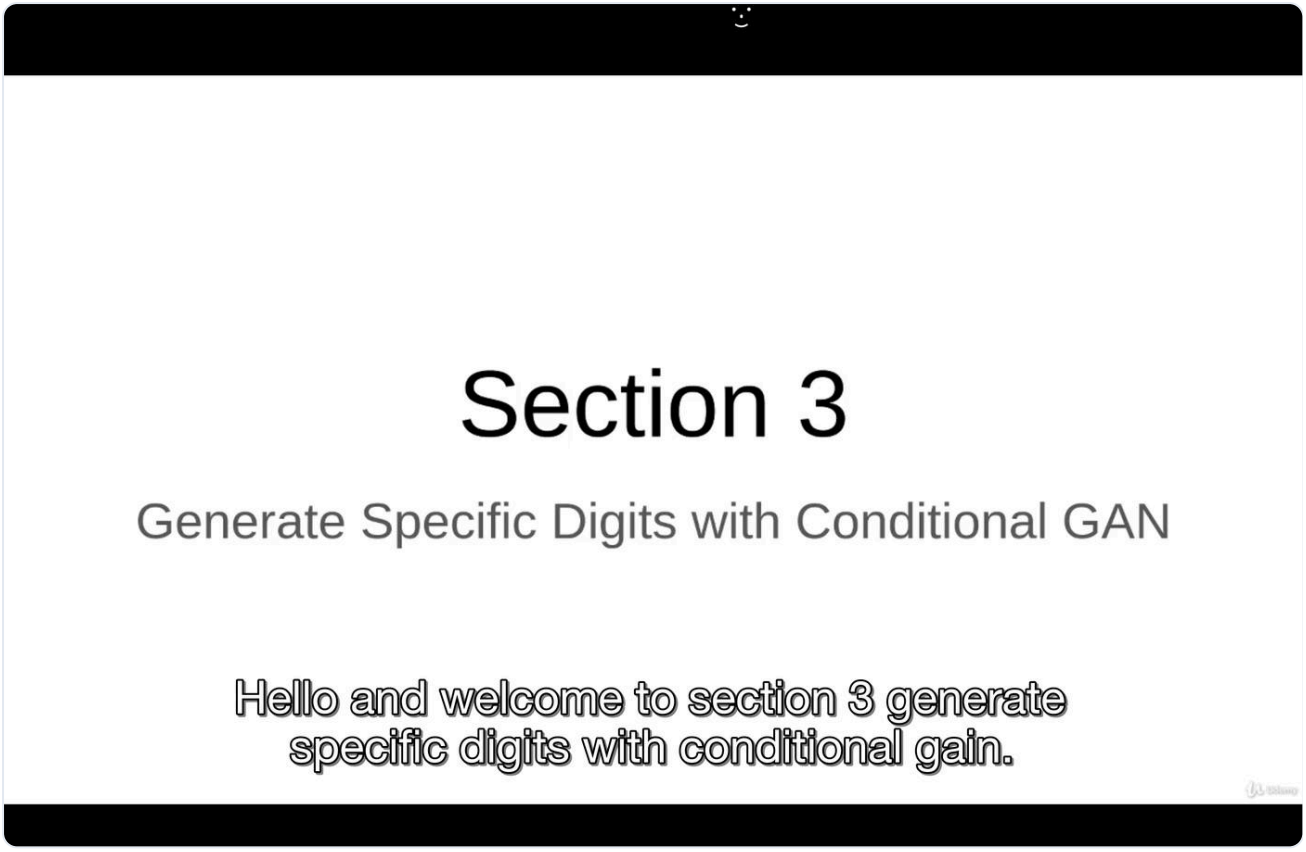


FIGURE 5.7

According to the environment we are using either Windows or Linux. The first parameter is visuals so the directory. The second parameter is the name of the file.

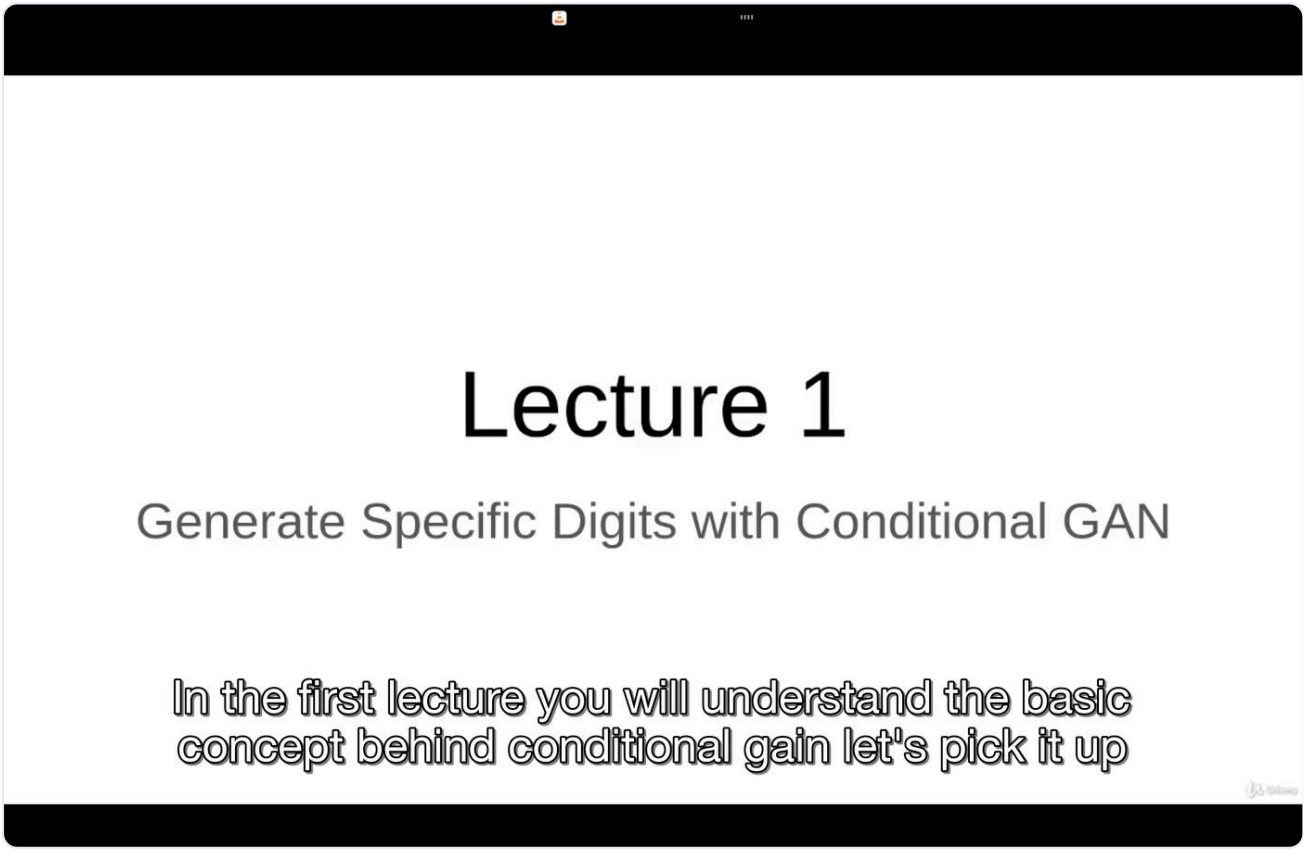


FIGURE 5.8

In this case we are using placeholder .jpg or jpeg and then we format bypassing the epoch number cast it as a string and then use zero filling of six zeros just in case you want to try 10000 or more epochs for the training. Next, nrow is the number of rows for the grid that it will create.

I'm using four rows the padding between each image is going to be 10 pixels and the padding value will be the color one or the white color next to use the generator that was trained for 512 epochs.

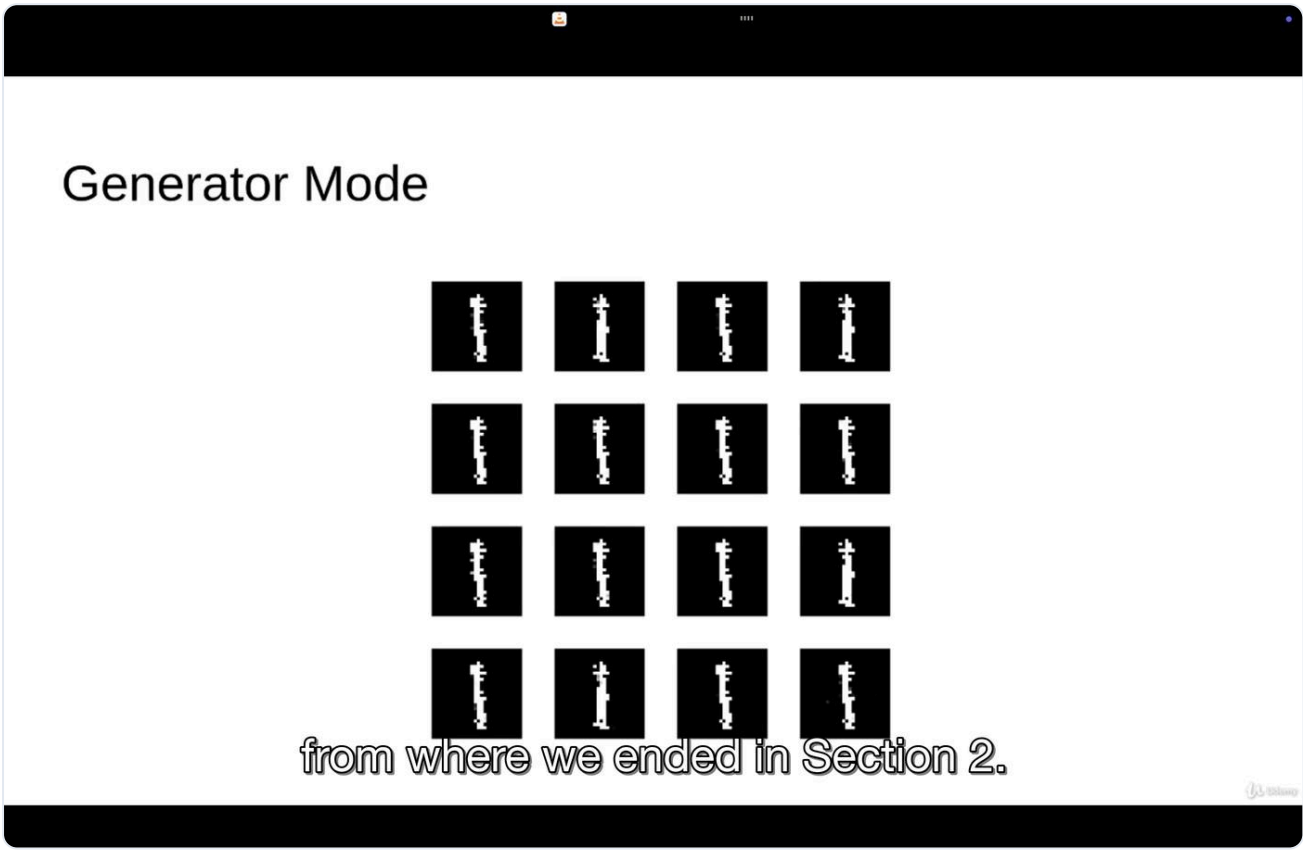


FIGURE 5.9 | Generator Mode

Remember that the visualized method above is invoked every ten epochs but since we are trying for 512 epochs the last one will not be visualized. So we are visualizing it here. We do the same generator we pass the fixed noise test_set and then detach using .detach() and then detach from the GPU using torch .cpu() and then we reshape using that view first dimension batch size negative 1.

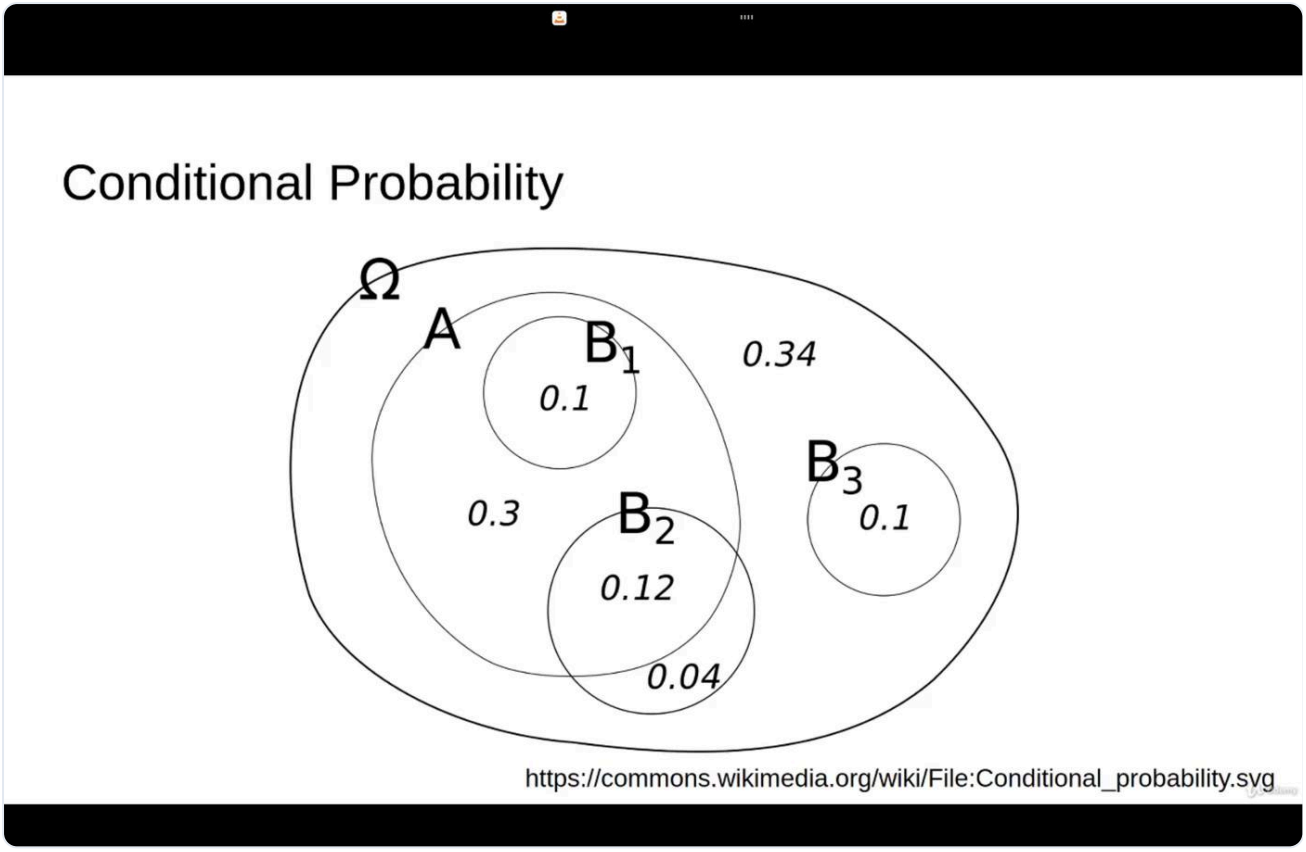


FIGURE 5.10

The second dimension 1 for grayscale and then for width and height 28 by 28. And this time I would like to introduce to another method from the torchvision API, torchvision.utils.make_grid and this will return a grid variable a tensor actually of the same specifications of the number of rows, padding and pad value for the generated images. The one that we have used above is save_image which will save the grid to the actual file on the disk.

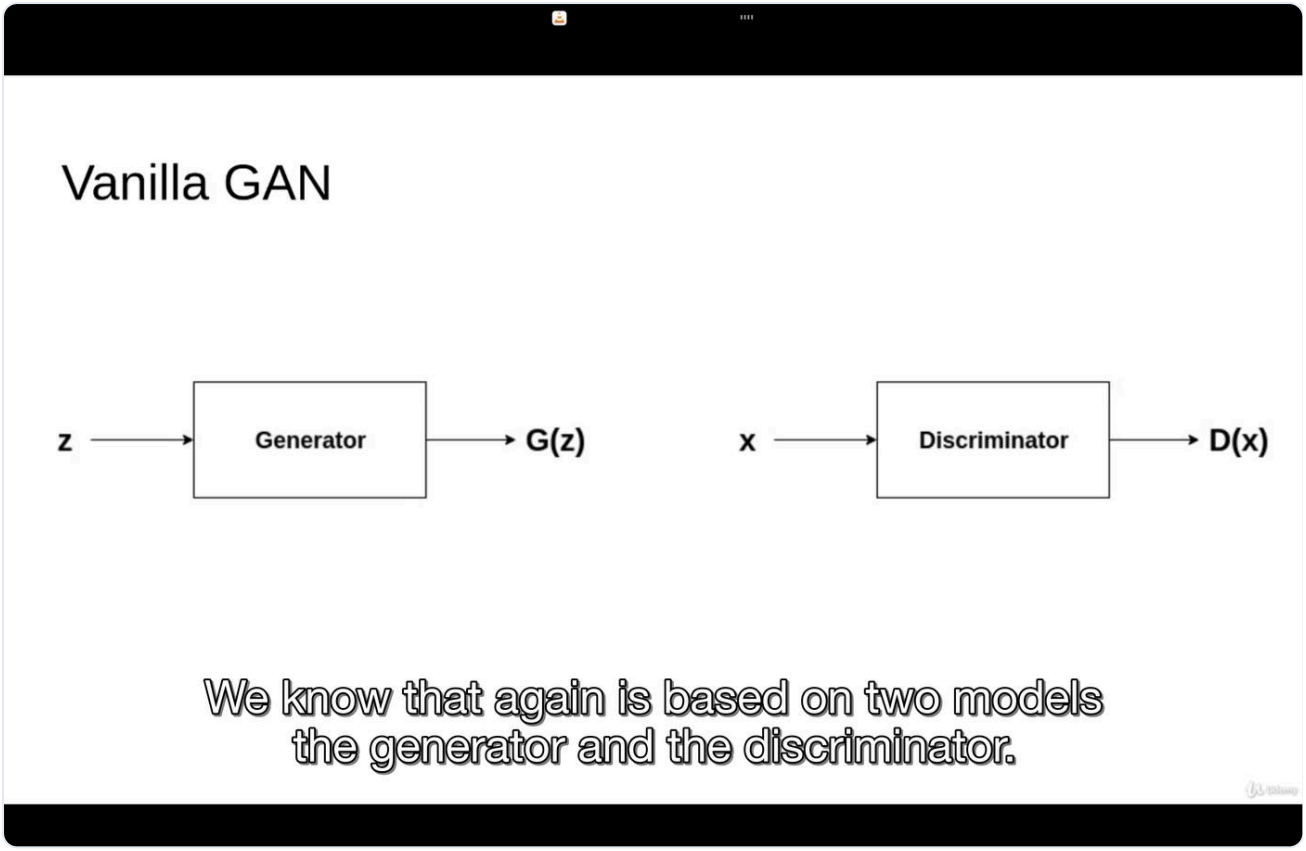


FIGURE 5.11

But this one will return a tensor. Next we convert the grid from bytes tensor to NumPy tensor using grid.numpy and then using a numpy method, the transpose to change the order of the dimensions. So remember that the grid now has the grayscale or the channel as a first dimension and then the width and height and the subsequent two dimensions.

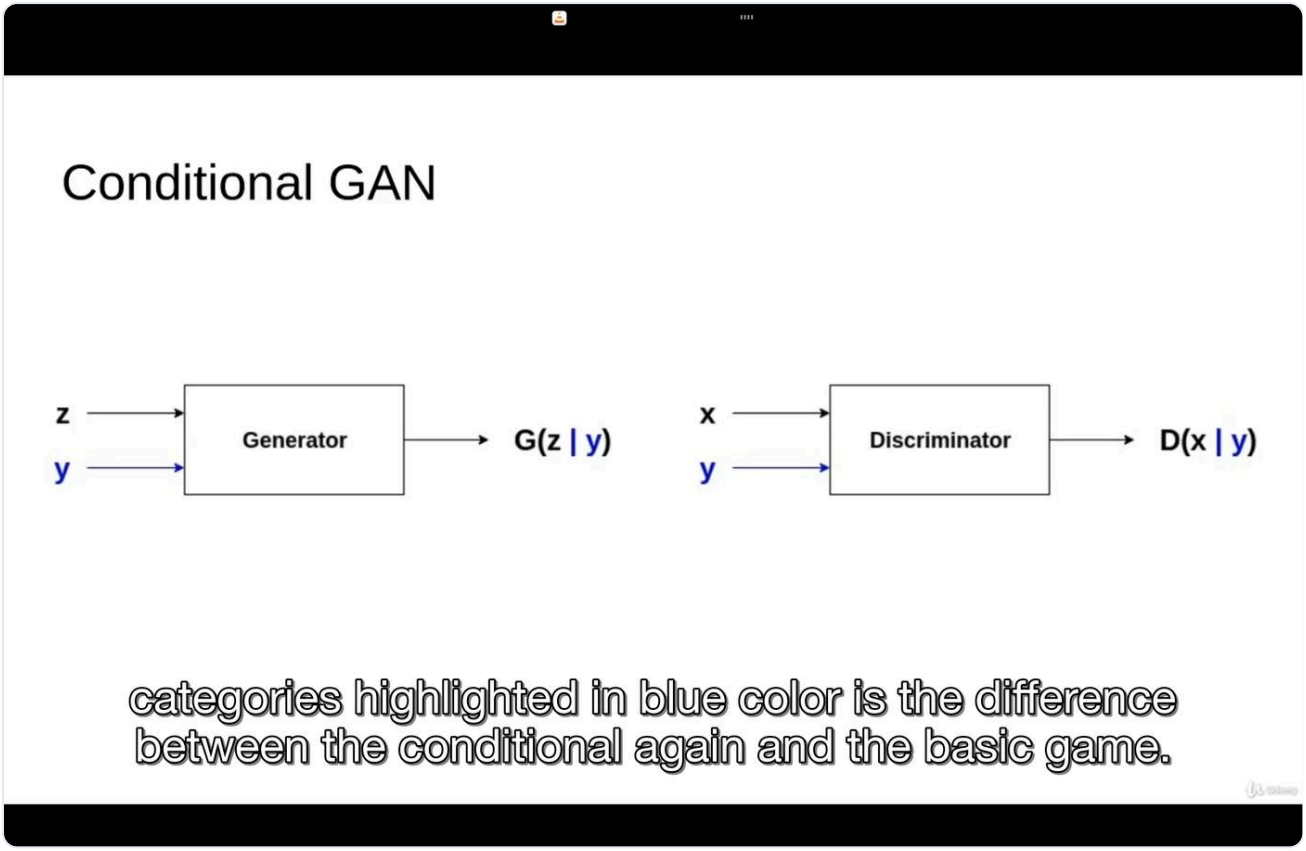


FIGURE 5.12

While we want it to be the opposite so that we can display it using matplotlib. So we use the numpy.transpose and we use the second dimension and the third dimension as the first two dimensions by indicating one comma two and then the first dimension indicated by zero to be the last dimension.

Finally we use matplotlib API is plt.figure and pass the figure size in inches 16 by 16.

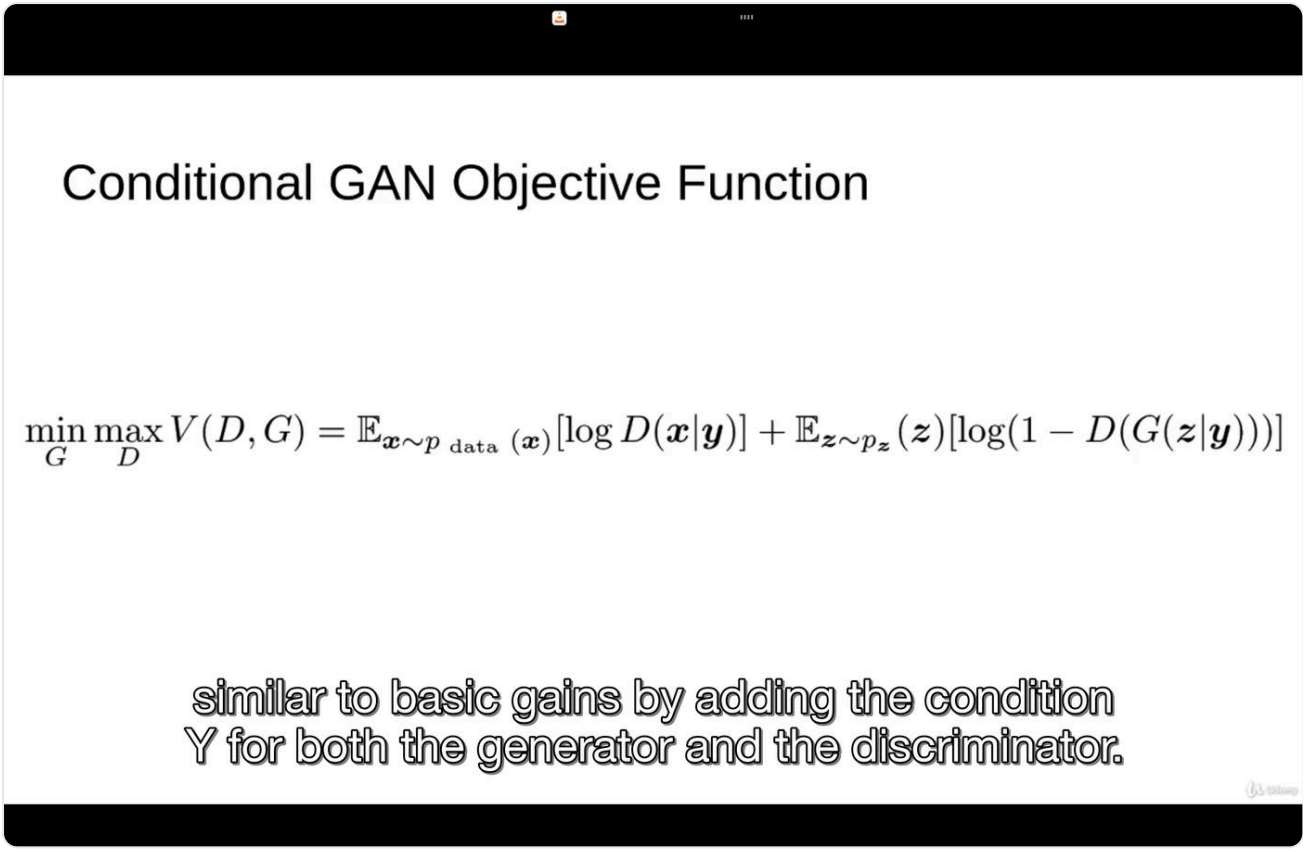


FIGURE 5.13

And we turn off the axes and use the method plt.imshow to display the image here what we have got after training the model 512 epochs all the samples are the same the generator mode is the same. It is only outputting the mode for digit number one. This is an interesting problem upon which section 2 and Section 3 are based.

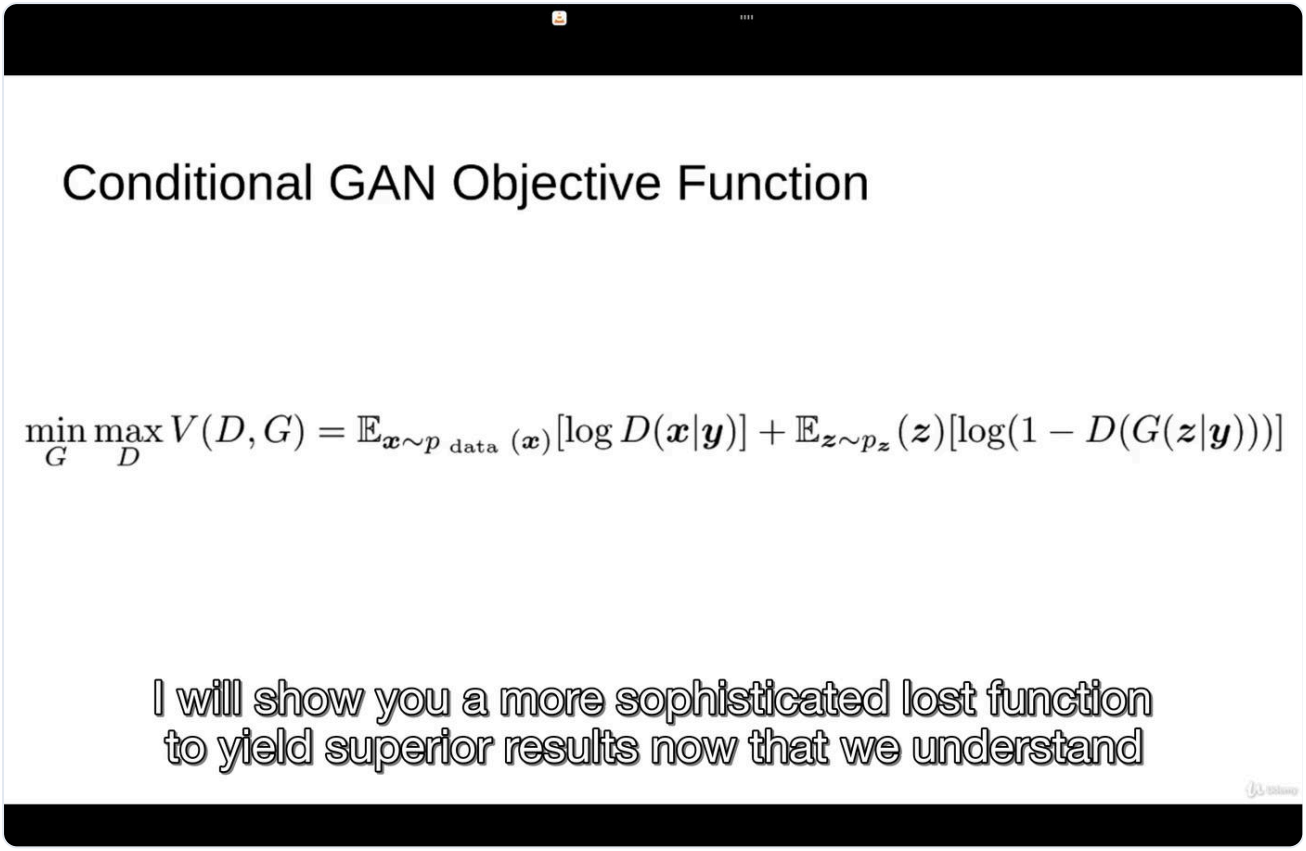


FIGURE 5.14

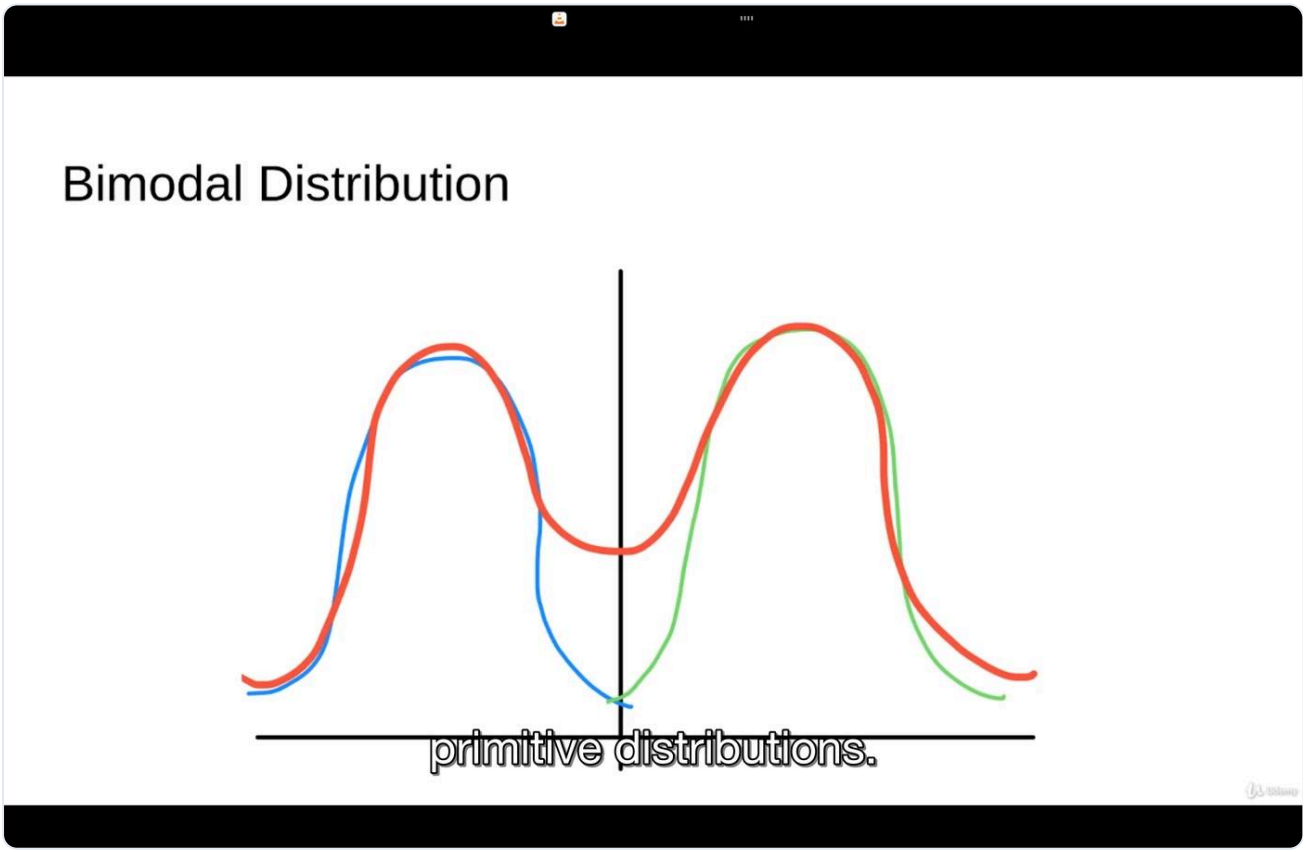


FIGURE 5.15

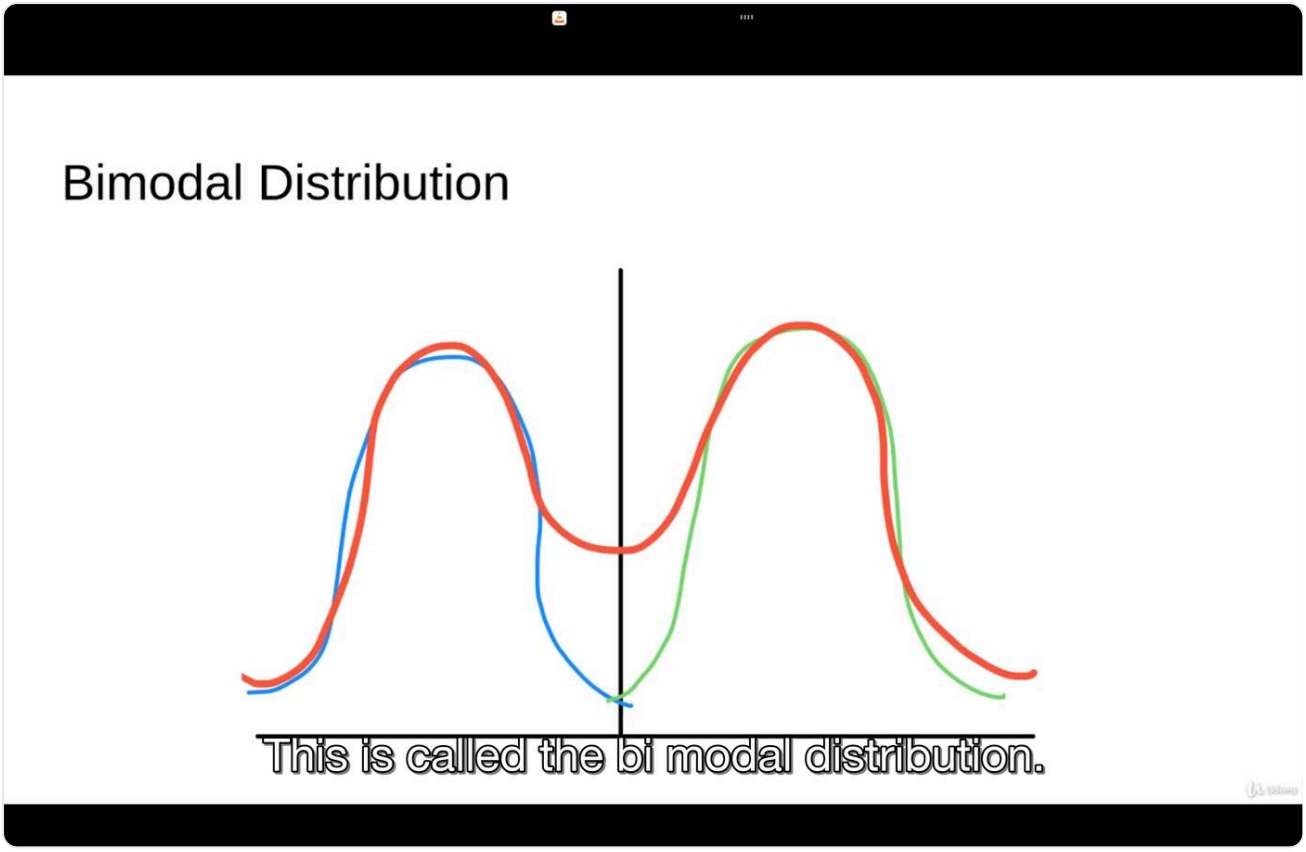


FIGURE 5.16

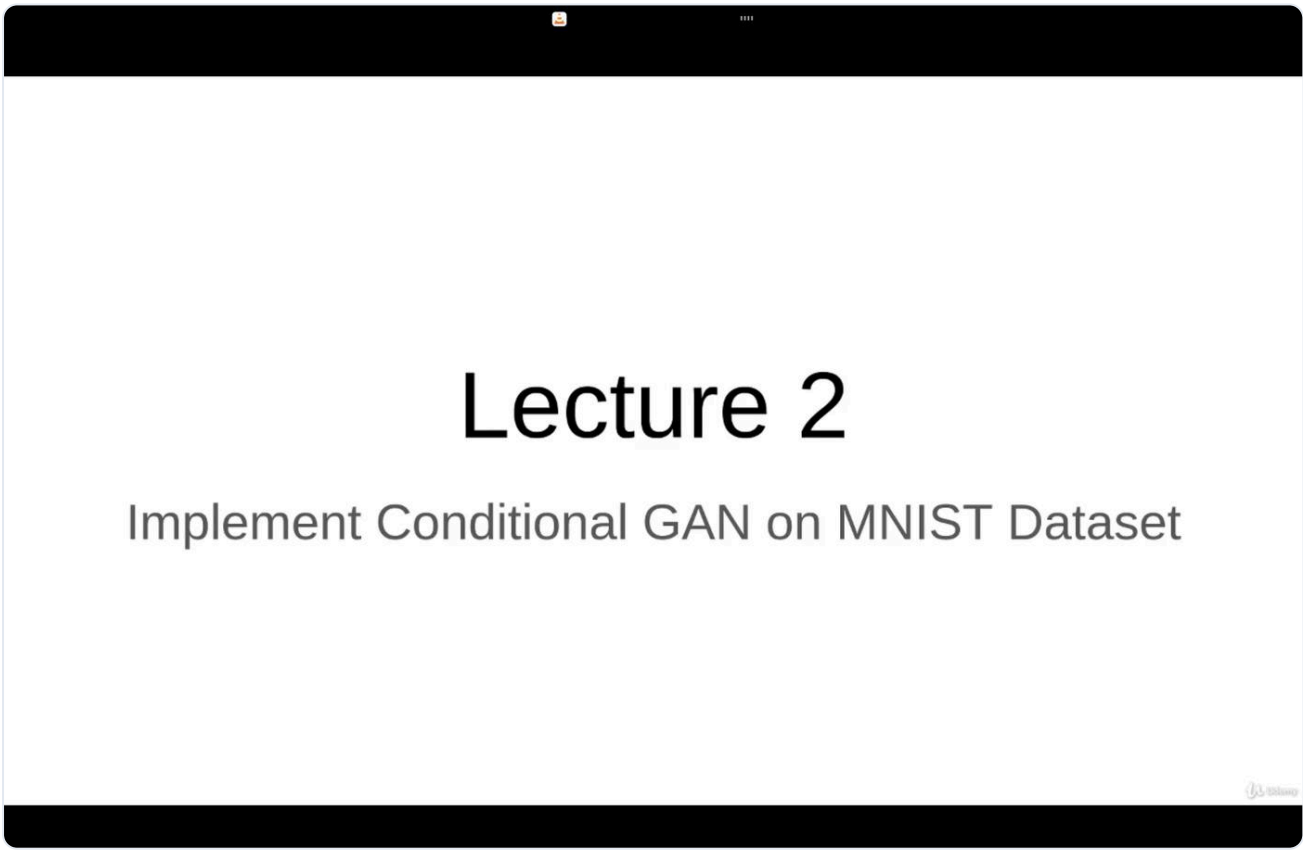


FIGURE 5.17 | Implement Conditional GAN on MNIST Dataset

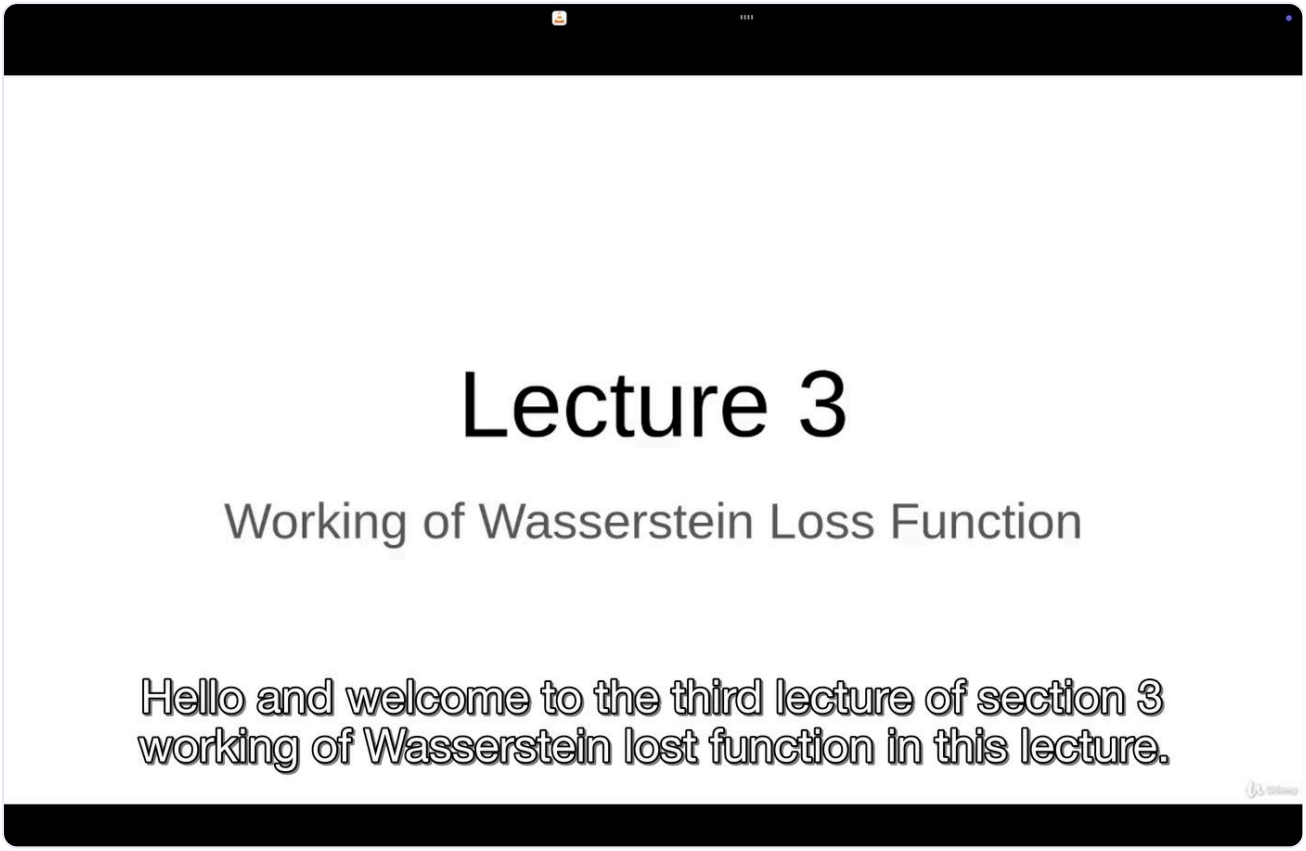


FIGURE 5.18

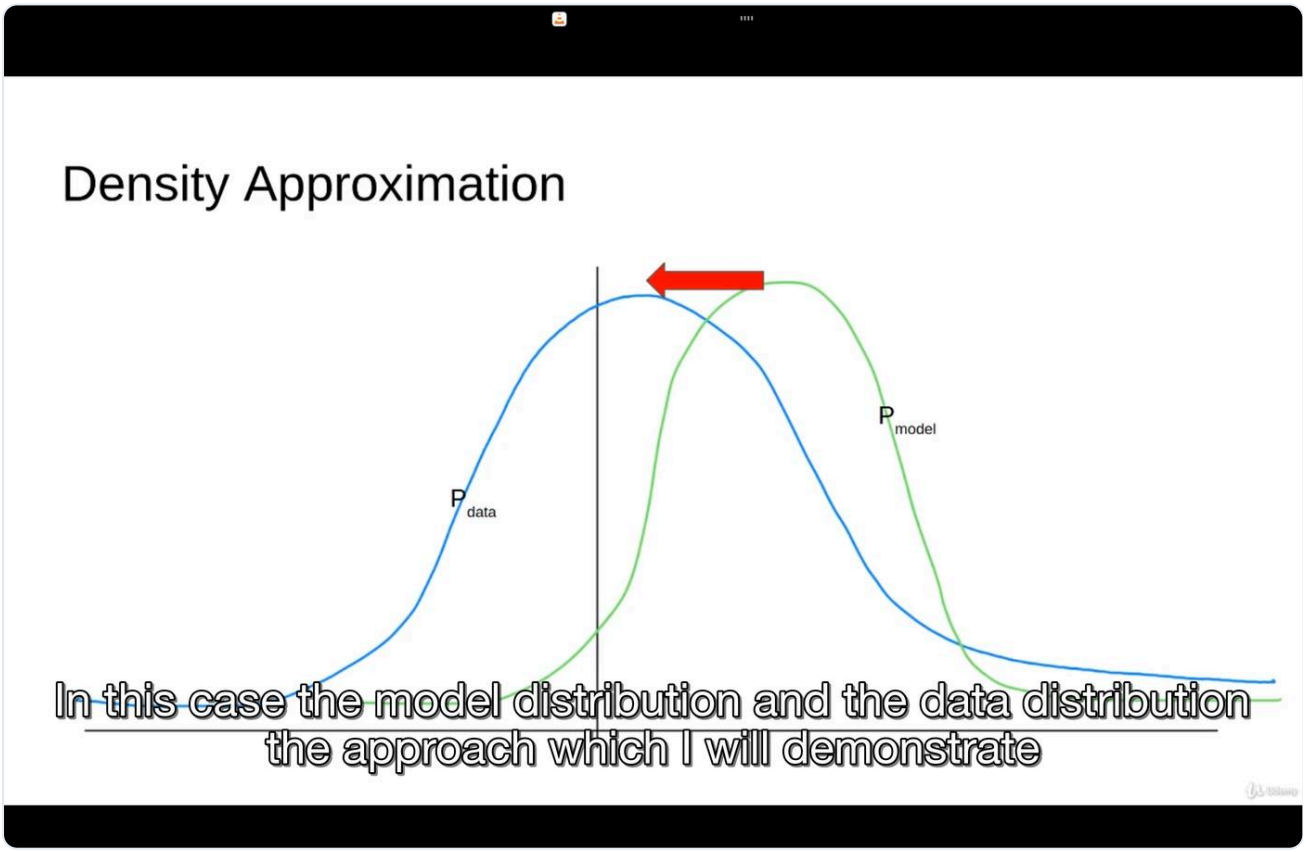


FIGURE 5.19

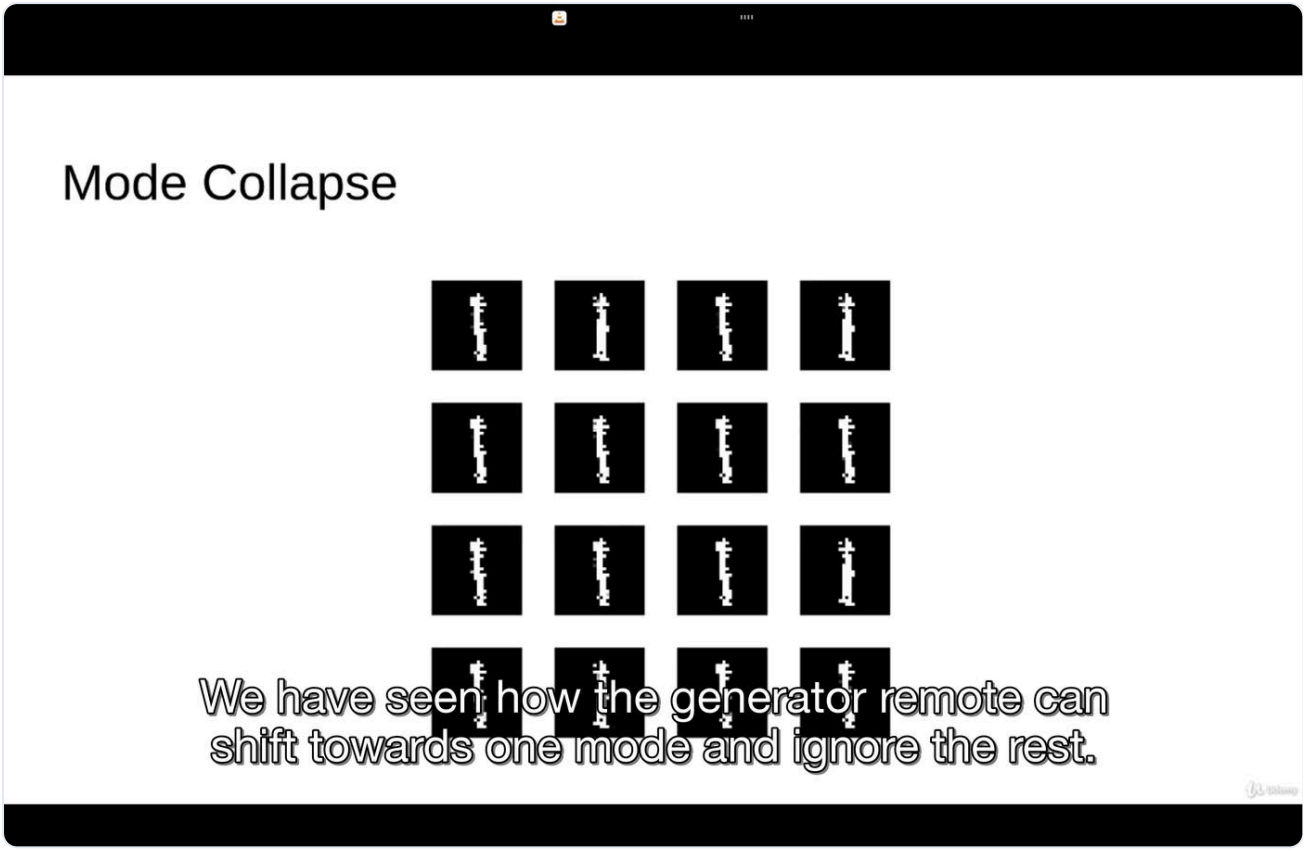


FIGURE 5.20 | Mode Collapse

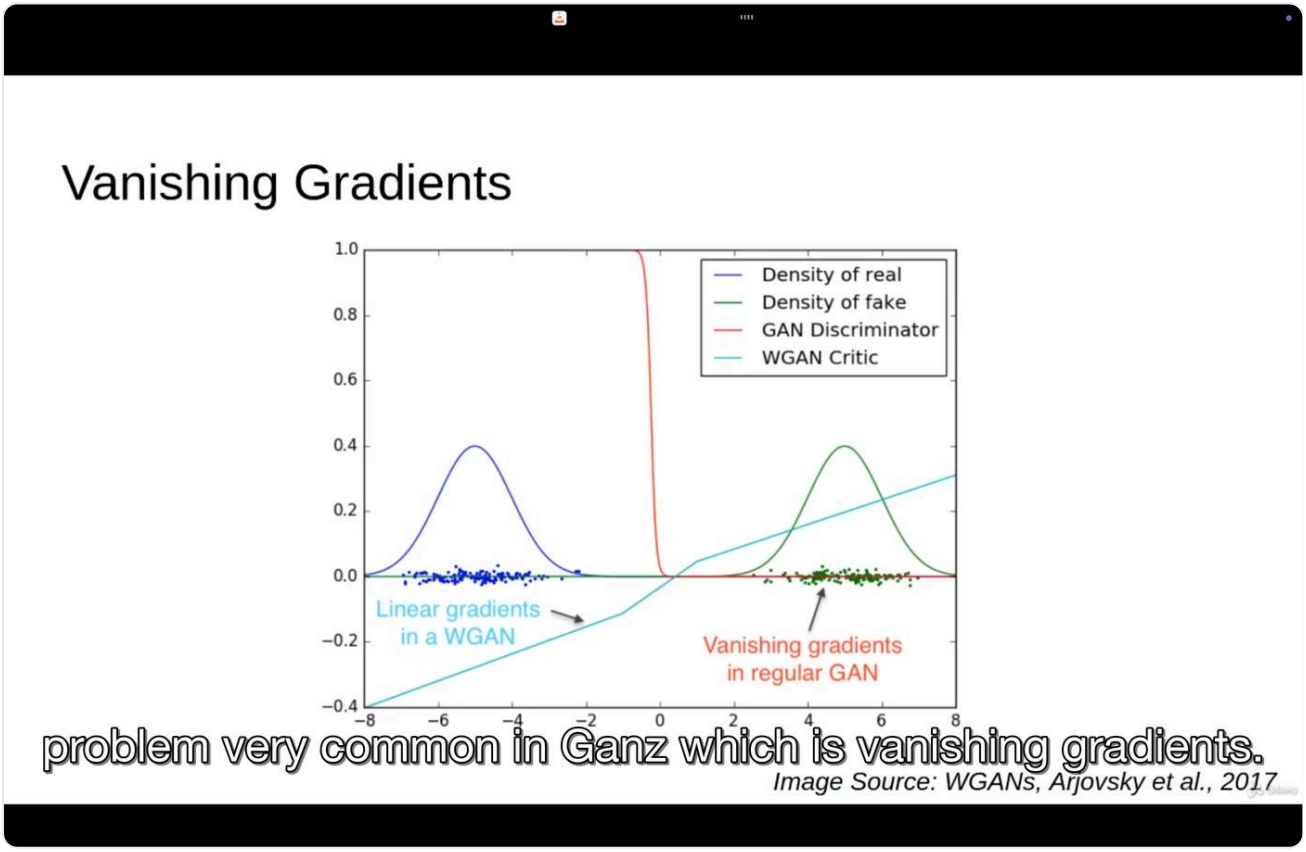
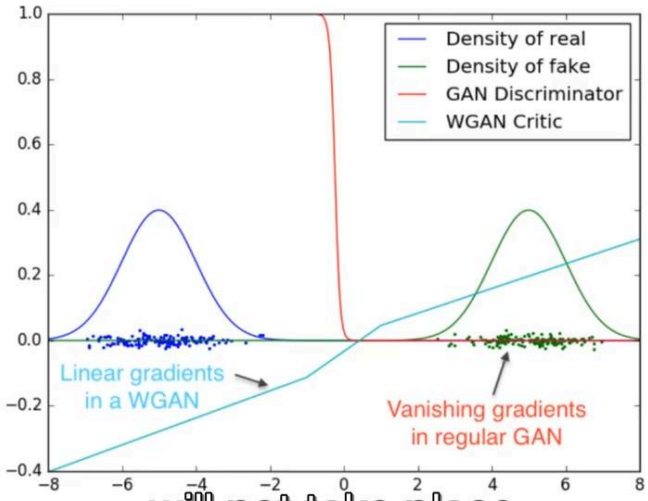


FIGURE 5.21

However I am not satisfied with this result and I will show you another notebook in a moment where I have upgraded the results.

Vanishing Gradients



will not take place.

Image Source: WGANs, Arjovsky et al., 2017

FIGURE 5.22

f-divergence

- Total Variation (TV) distance
- Kullback-Leibler (KL) divergence
- Jensen-Shannon (JS) divergence
- Earth-Mover (EM) distance = Wasserstein

two probability distributions.

FIGURE 5.23

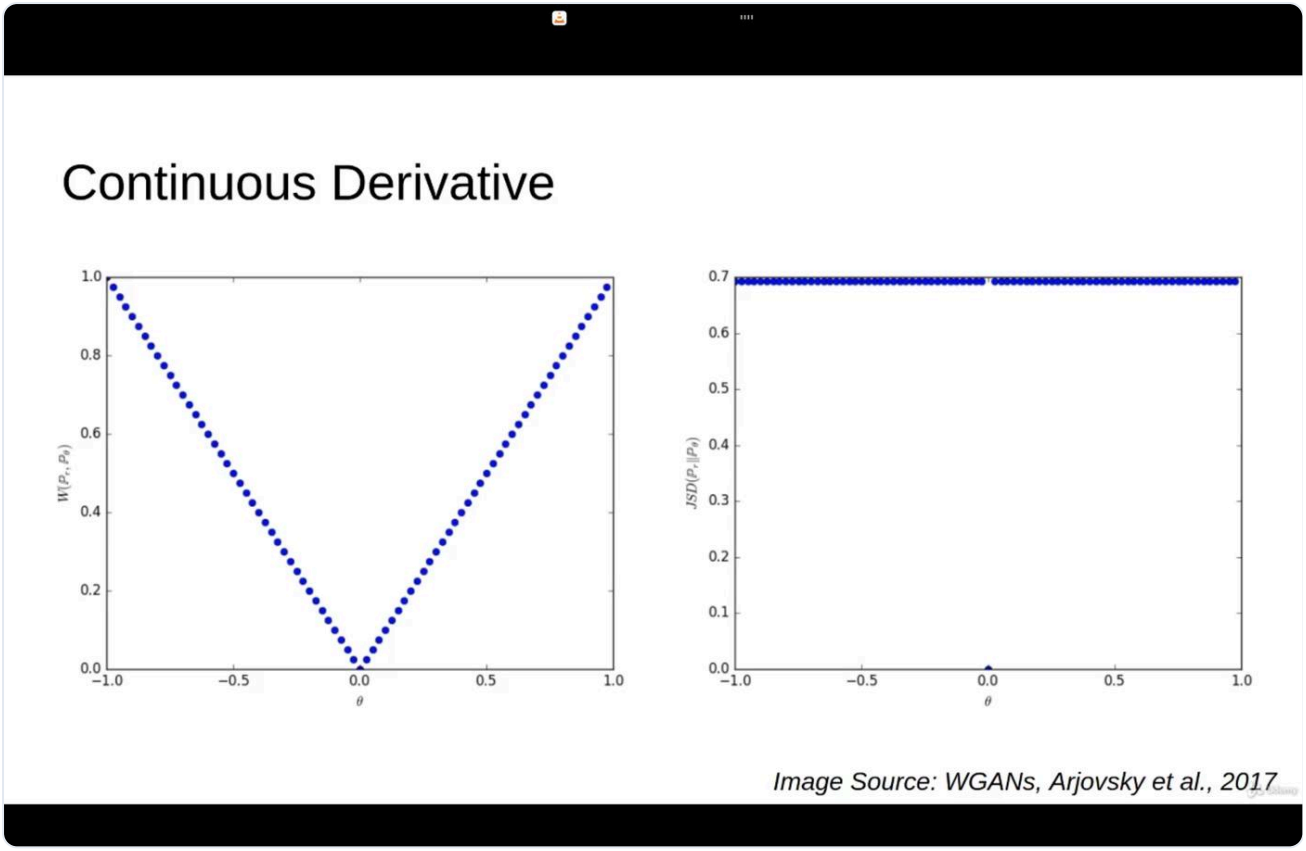


FIGURE 5.24

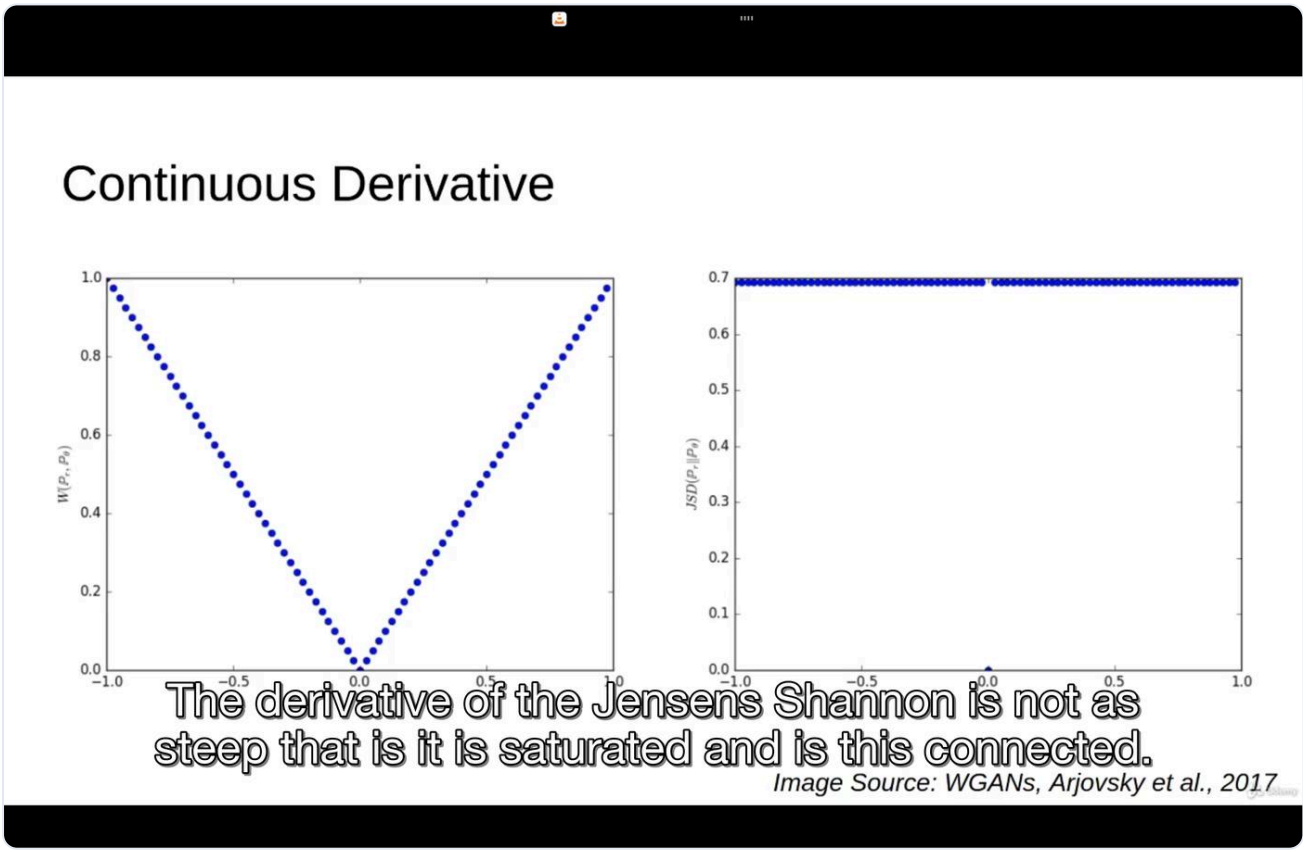


FIGURE 5.25

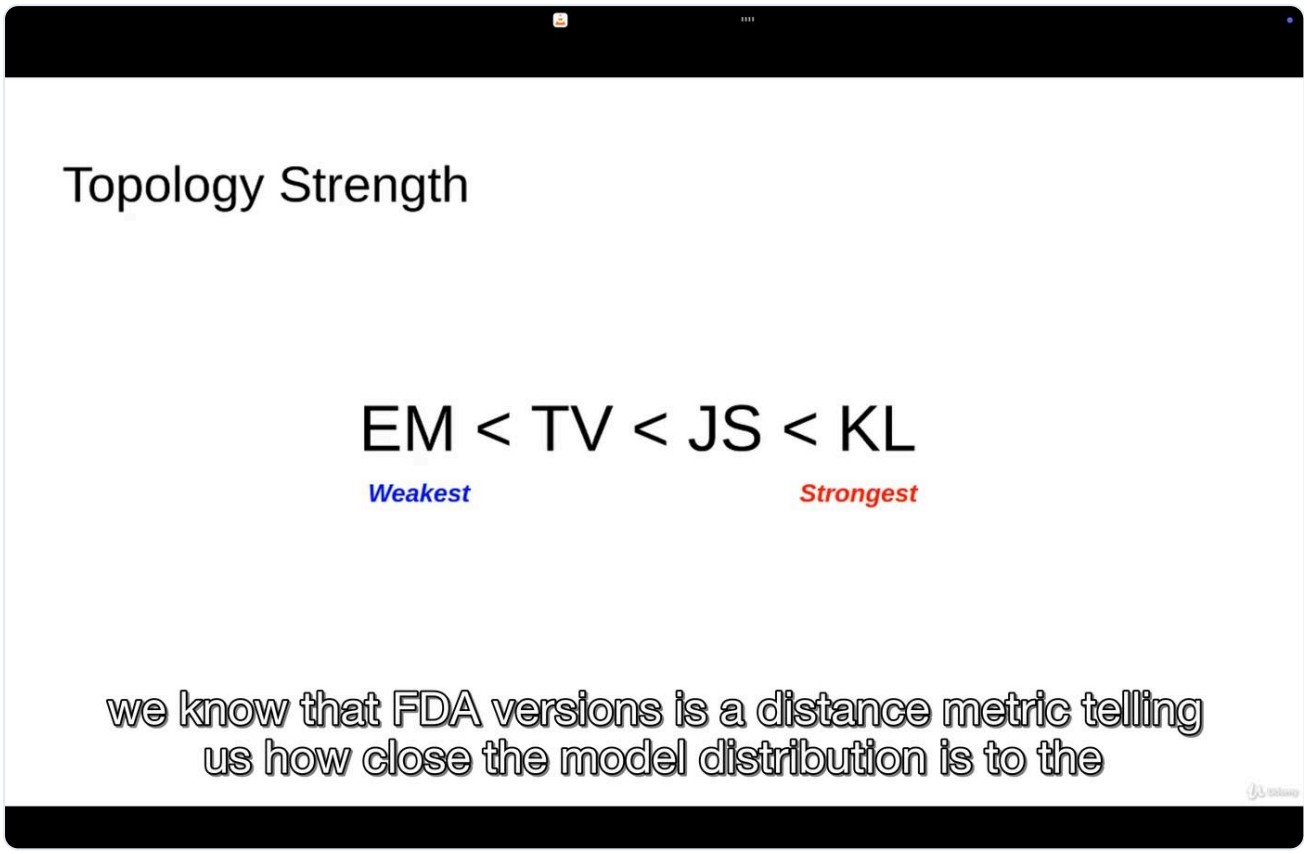


FIGURE 5.26

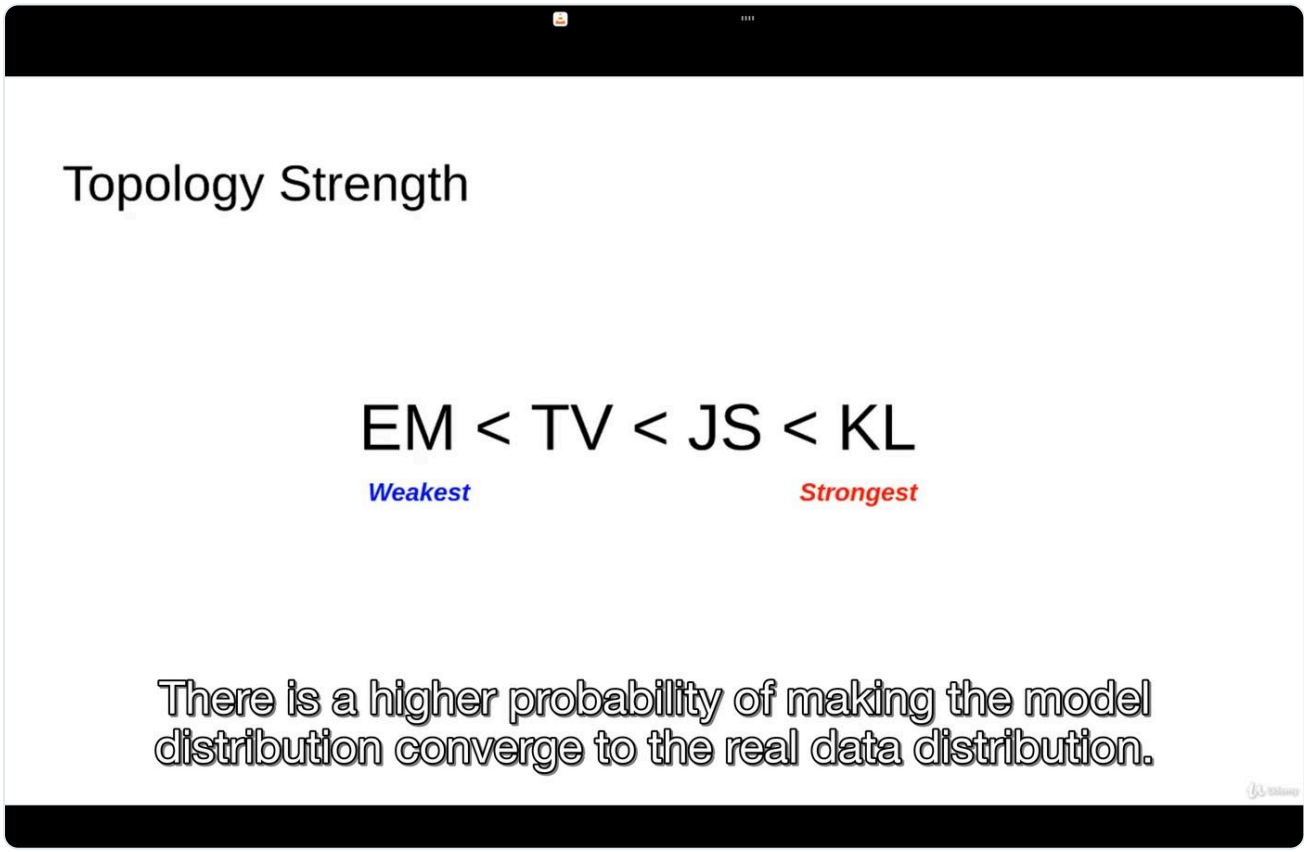


FIGURE 5.27

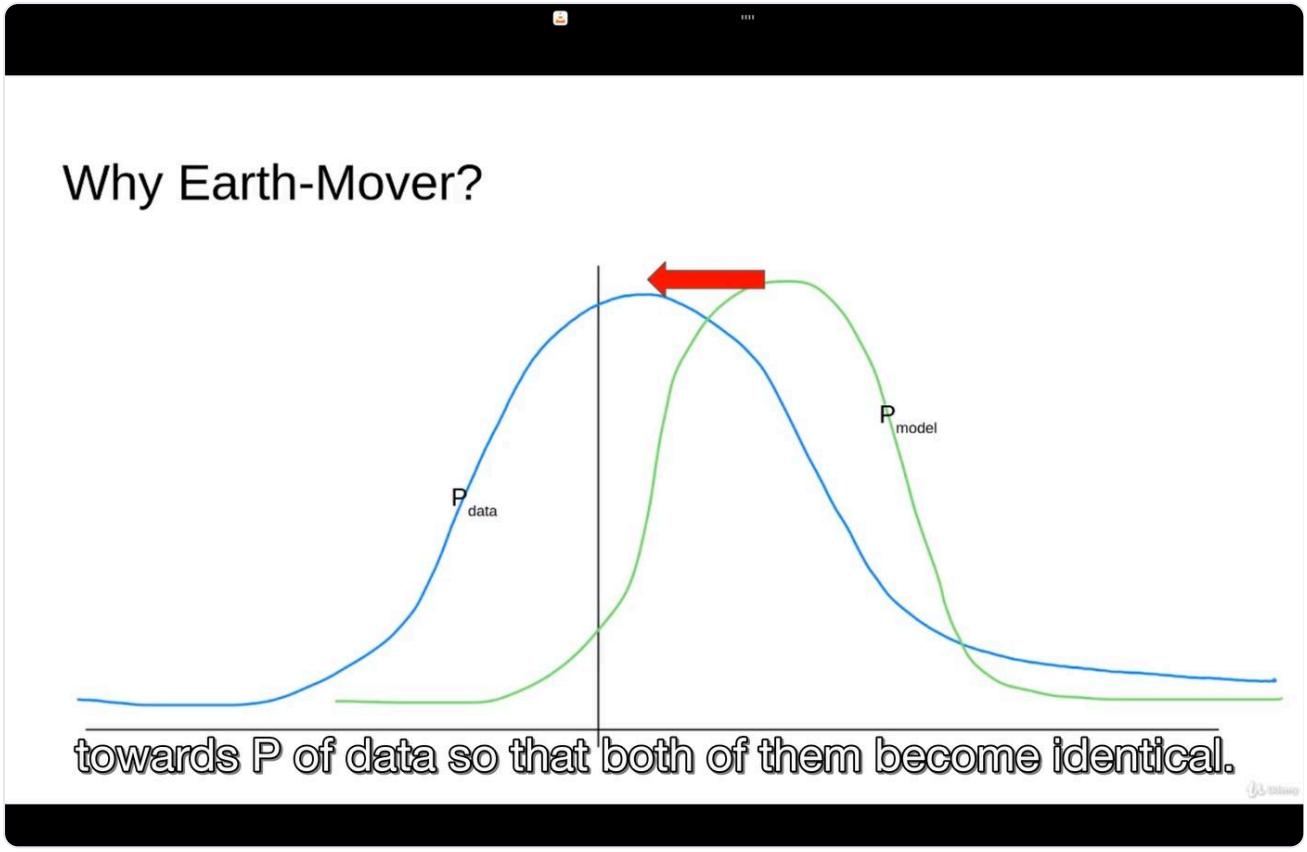


FIGURE 5.28 | P of Cata So That of Then Identical

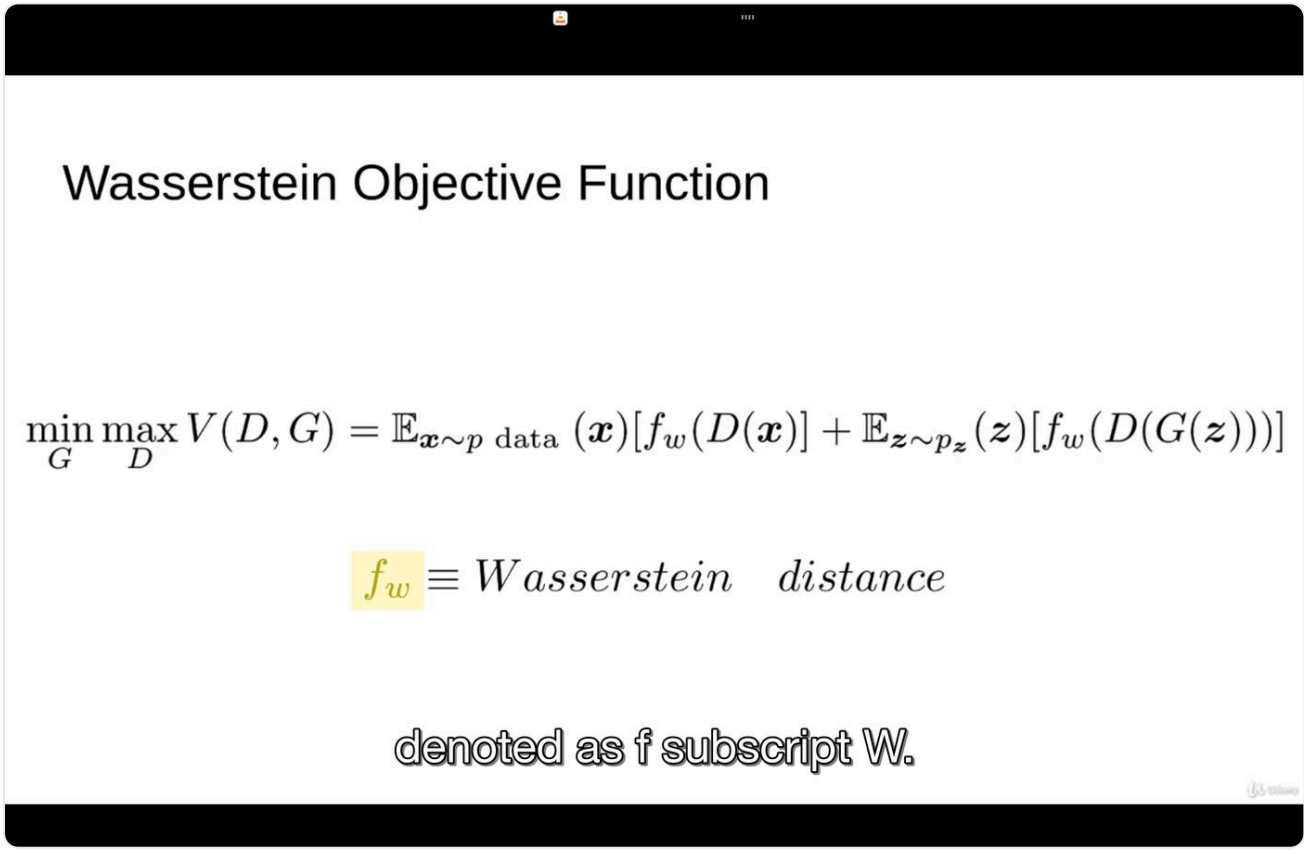


FIGURE 5.29

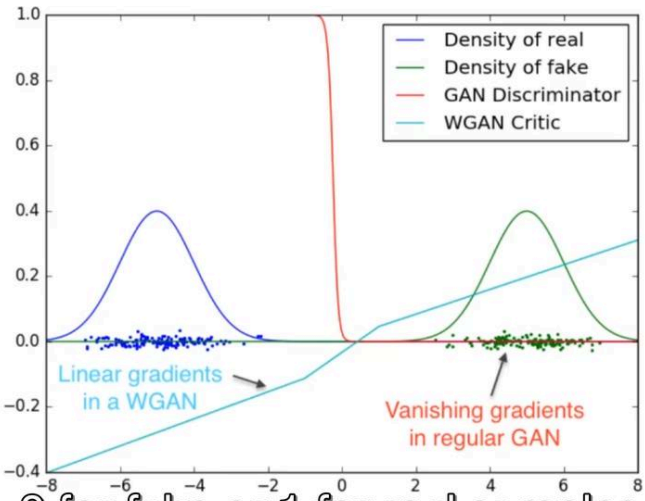
Lipschitz Inequality

$$|f(x_1) - f(x_2)| \leq |x_1 - x_2|$$

Now the question is how to calculate f subscript w
it turns out that is an intractable problem for which

FIGURE 5.30

Why are gradients not vanishing?

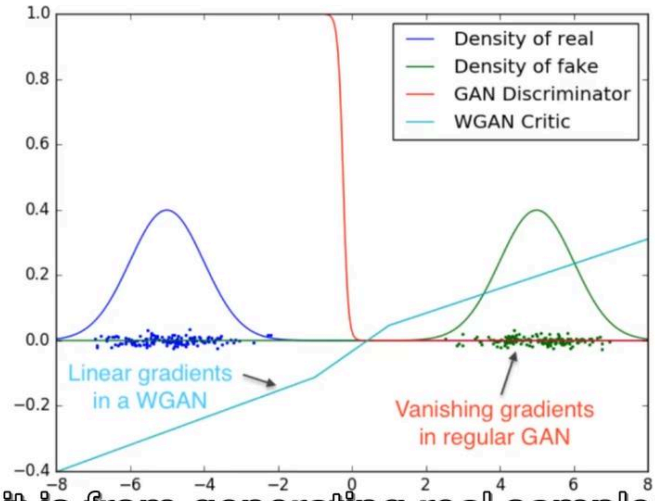


0 for fake or 1 for real samples.

Image Source: WGANs, Arjovsky et al., 2017

FIGURE 5.31 | Why Are Gradients Not Vanishing?

Why are gradients not vanishing?



it is from generating real samples.

Image Source: WGANs, Arjovsky et al., 2017

FIGURE 5.32 | Why Are Gradients Not Vanishing?

Hyperparameters

- Learning Rate 0.00005
- RMSprop Optimizer
- Batch Size 64
- Discriminator Training K=5 Steps
- Clamp Discriminator Weights [-0.01, 0.01]

for amnestied dataset.

FIGURE 5.33

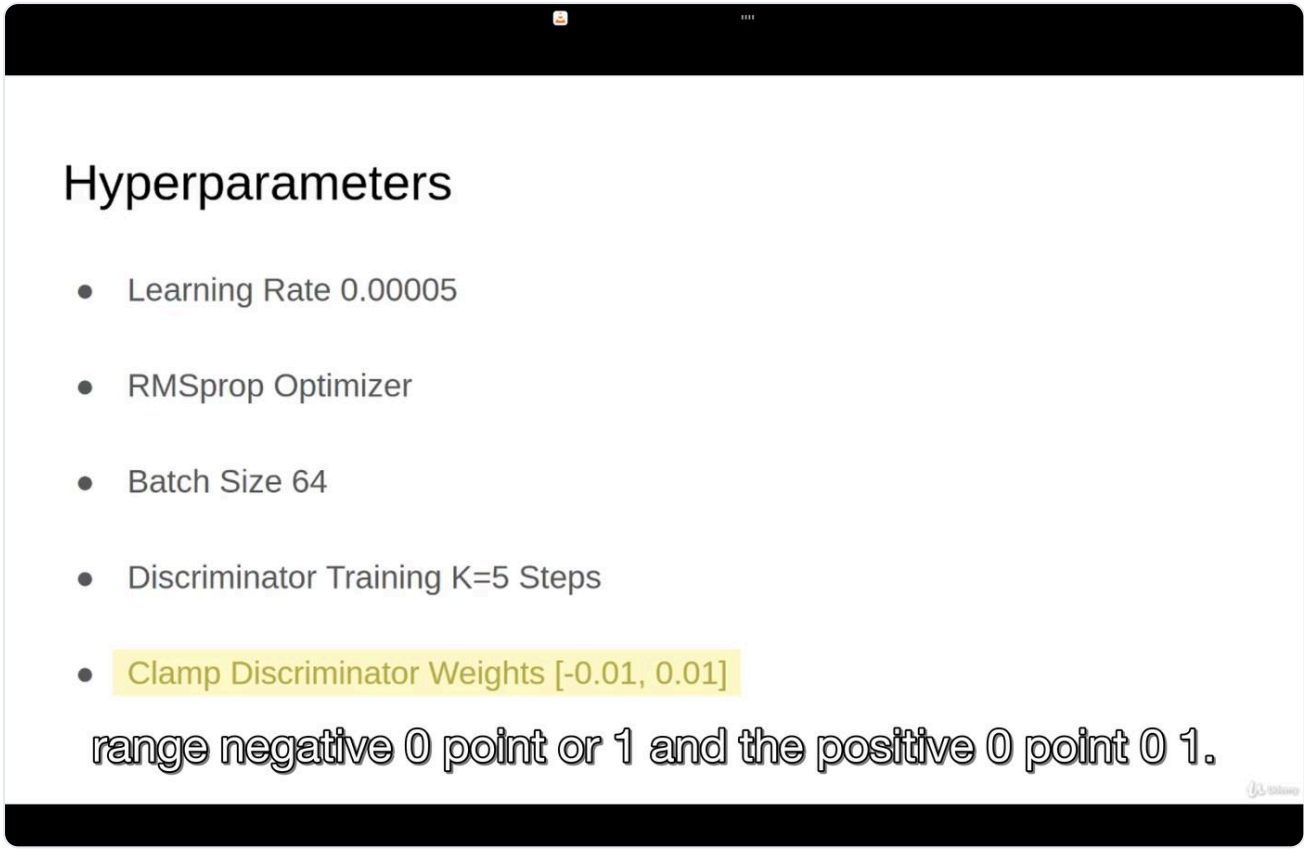


FIGURE 5.34

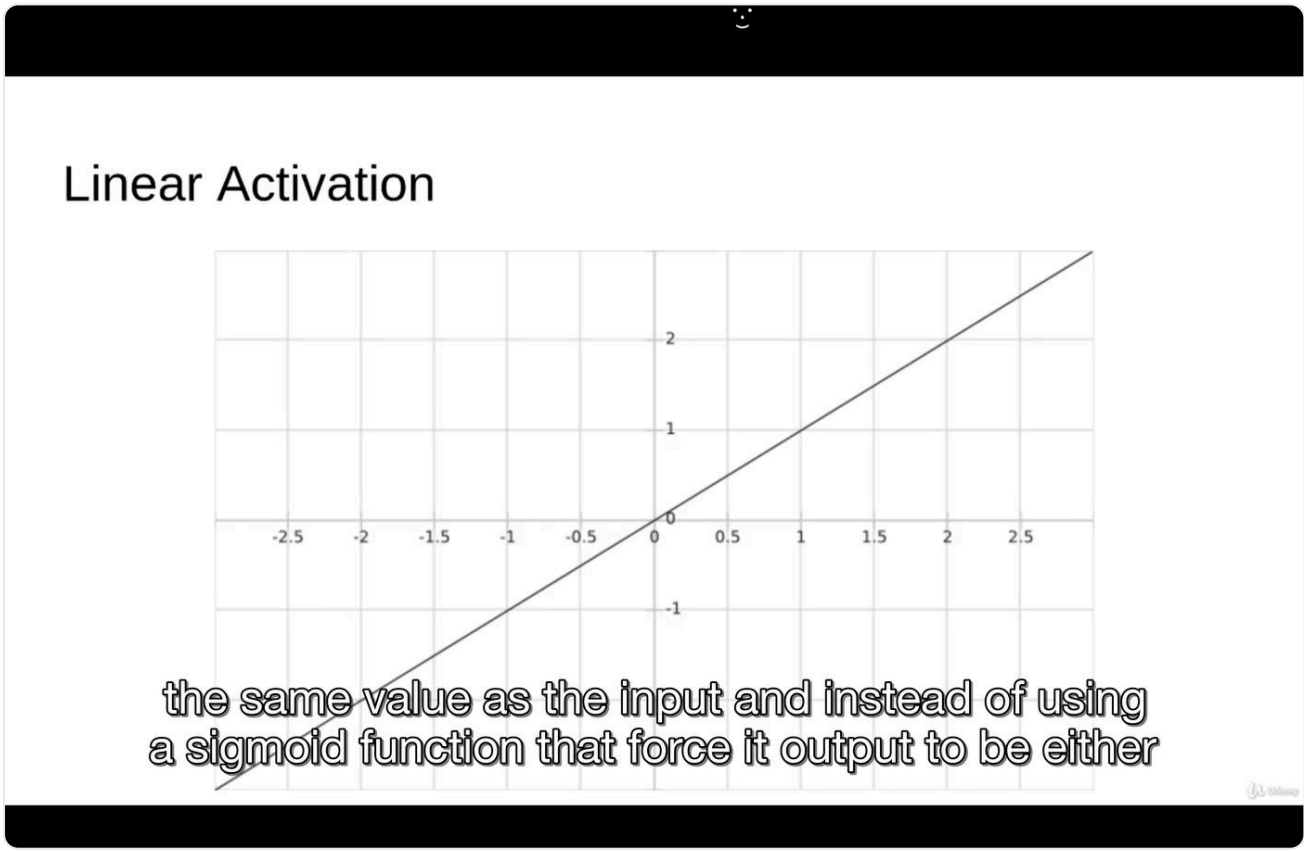


FIGURE 5.35 | Linear Activation

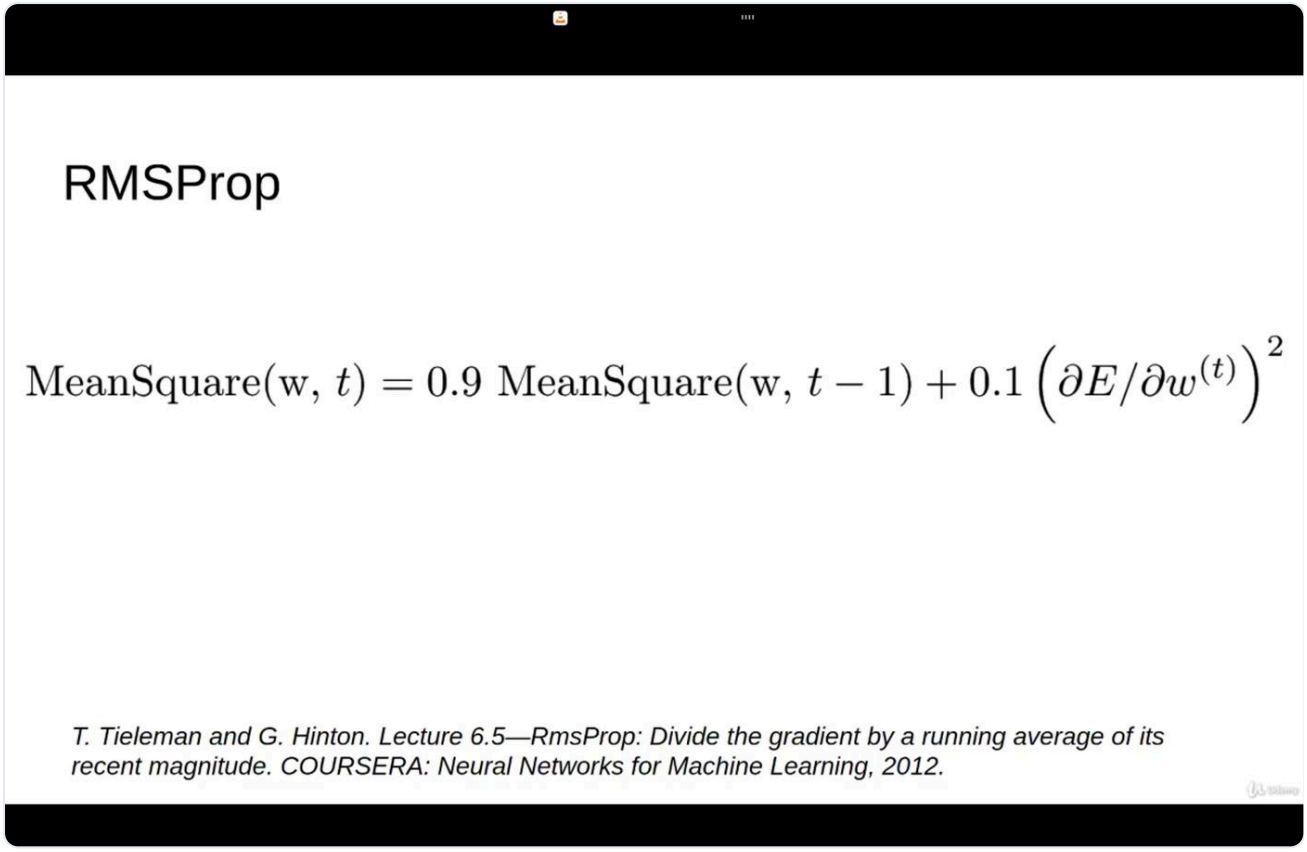


FIGURE 5.36

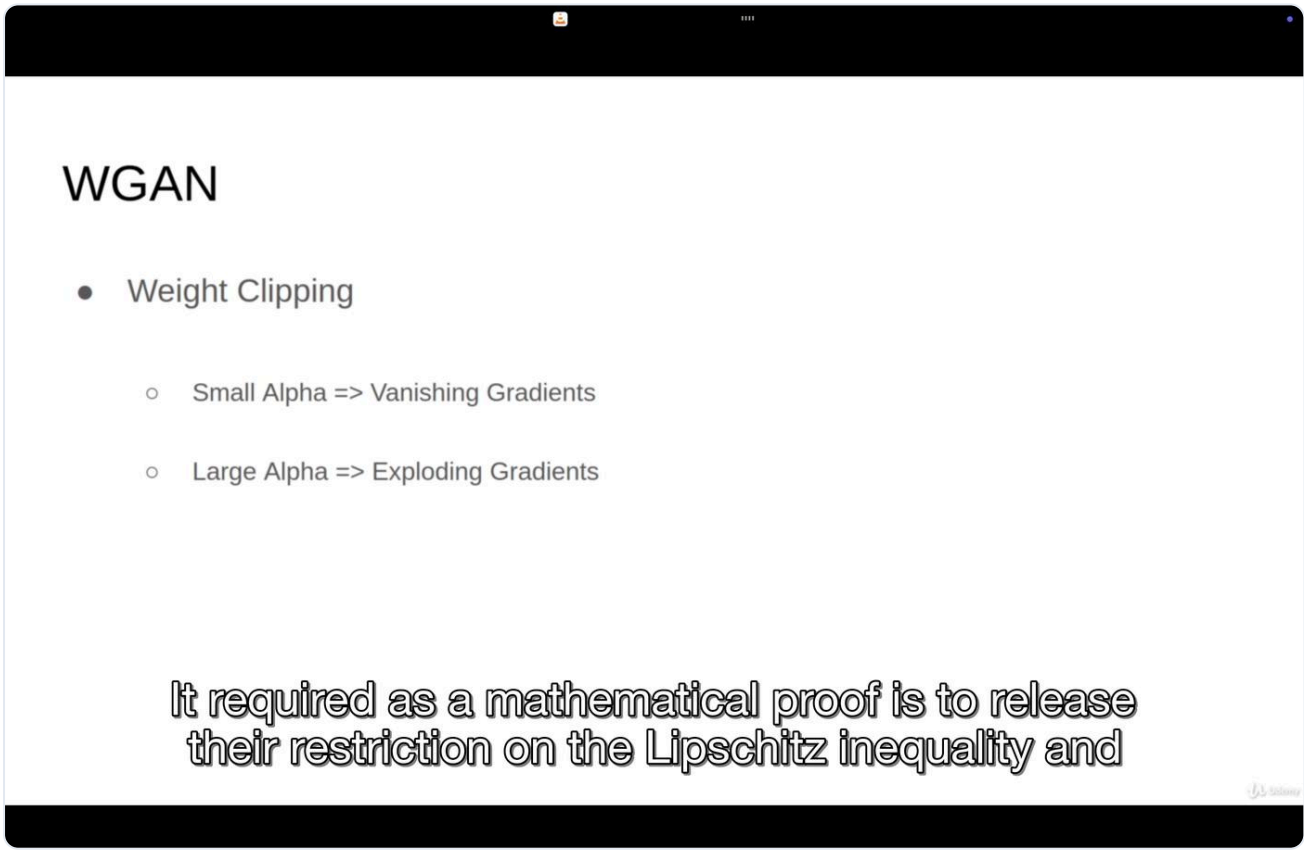


FIGURE 5.37

But the point here I would like to highlight that this is a very common problem when training GANs which is called mode collapse at the end of the notebook you find a reference to the Google Colab notebook file that includes the same exercise as this one.

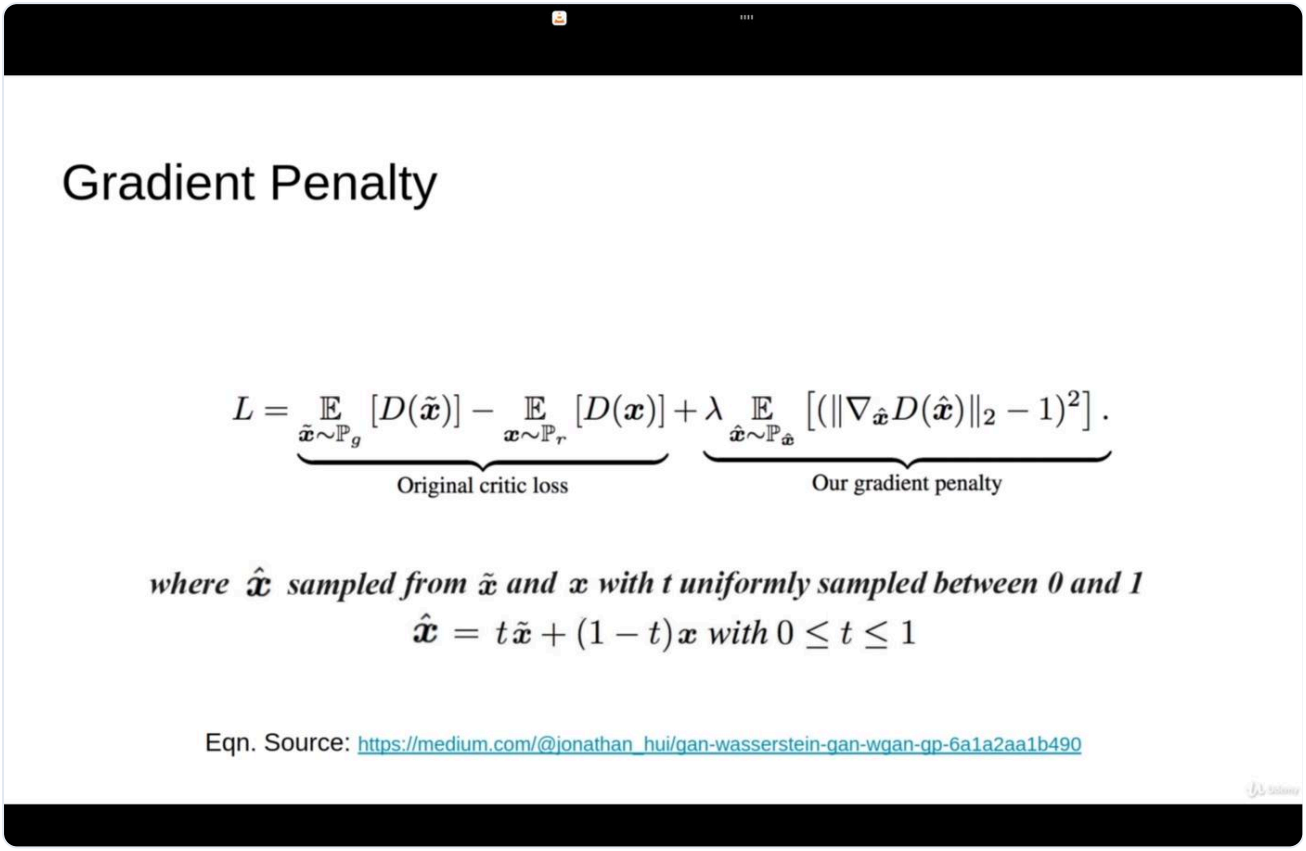


FIGURE 5.38

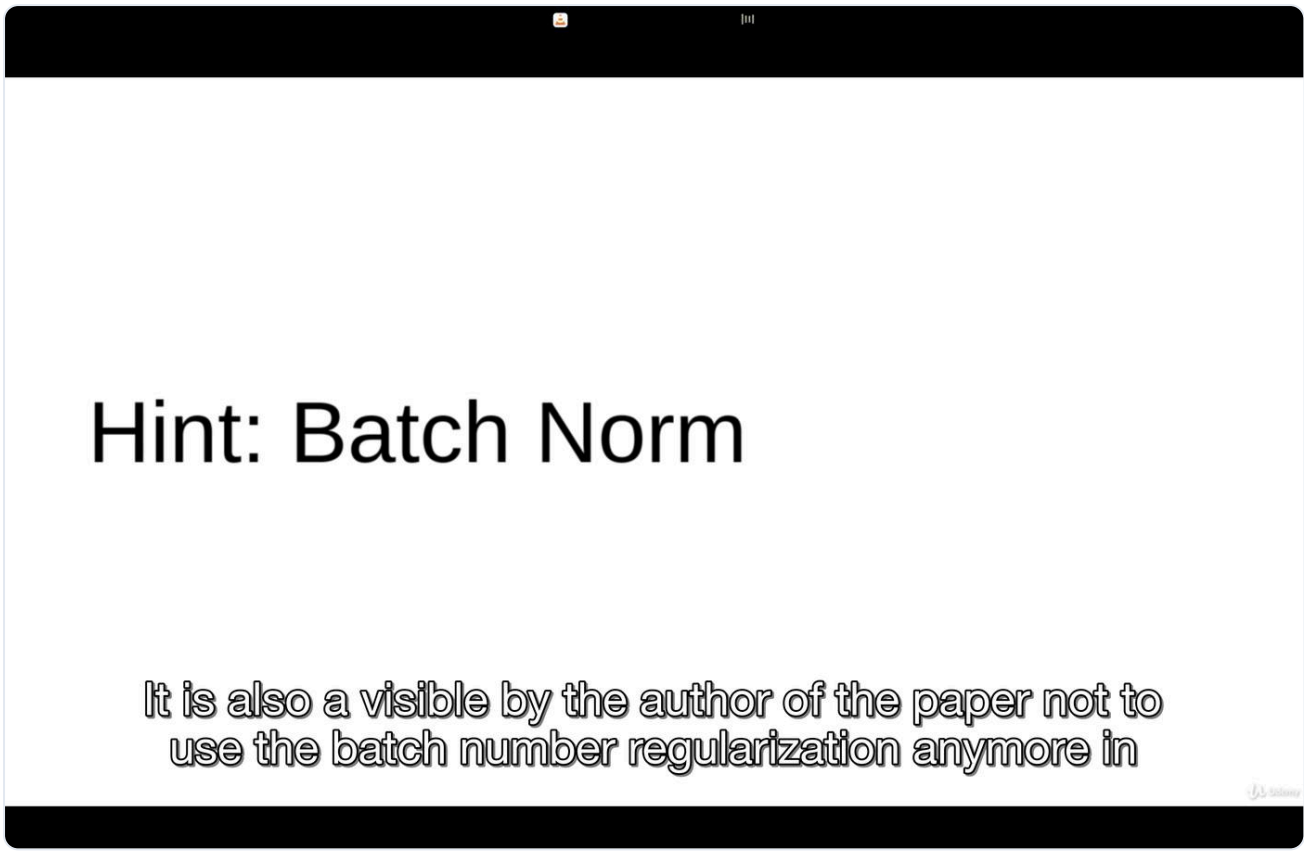


FIGURE 5.39

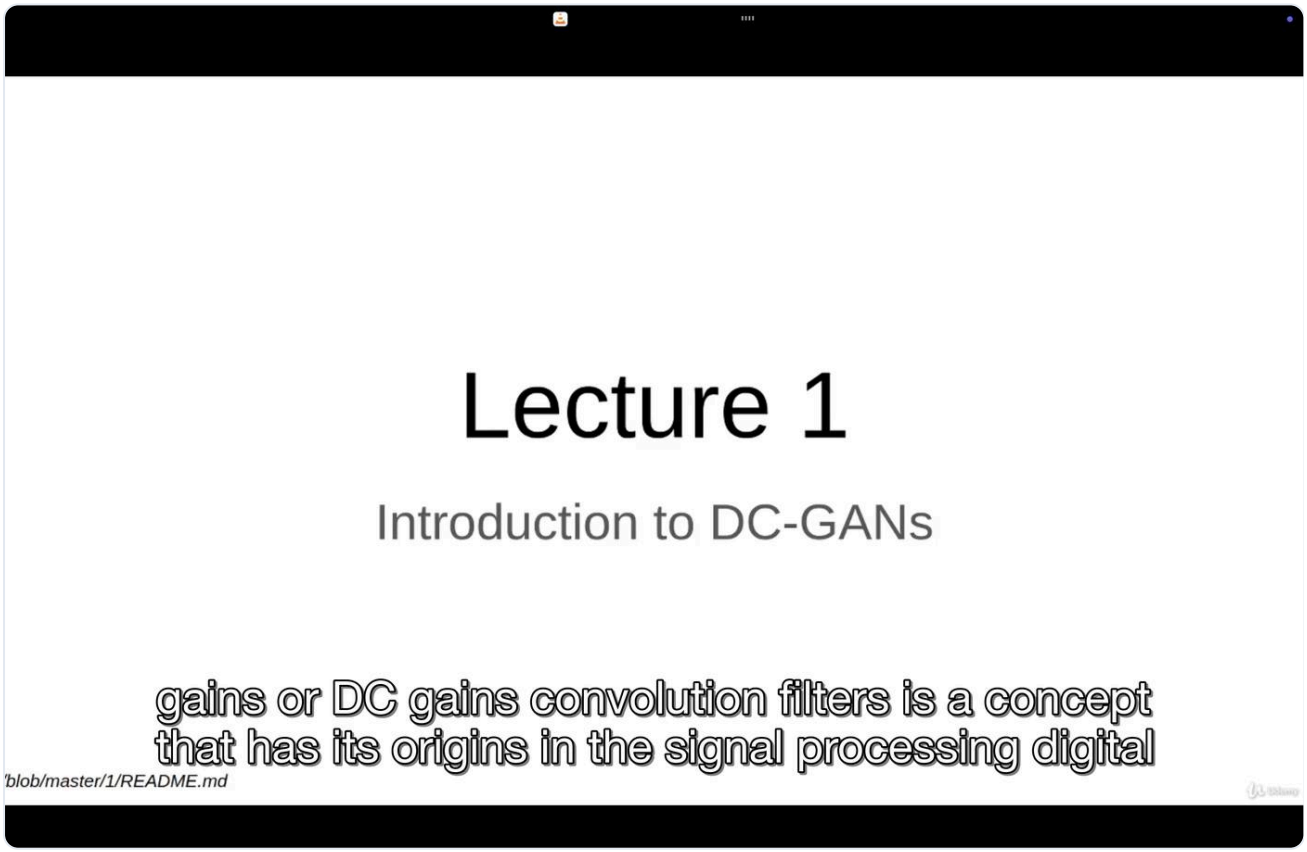


FIGURE 5.40

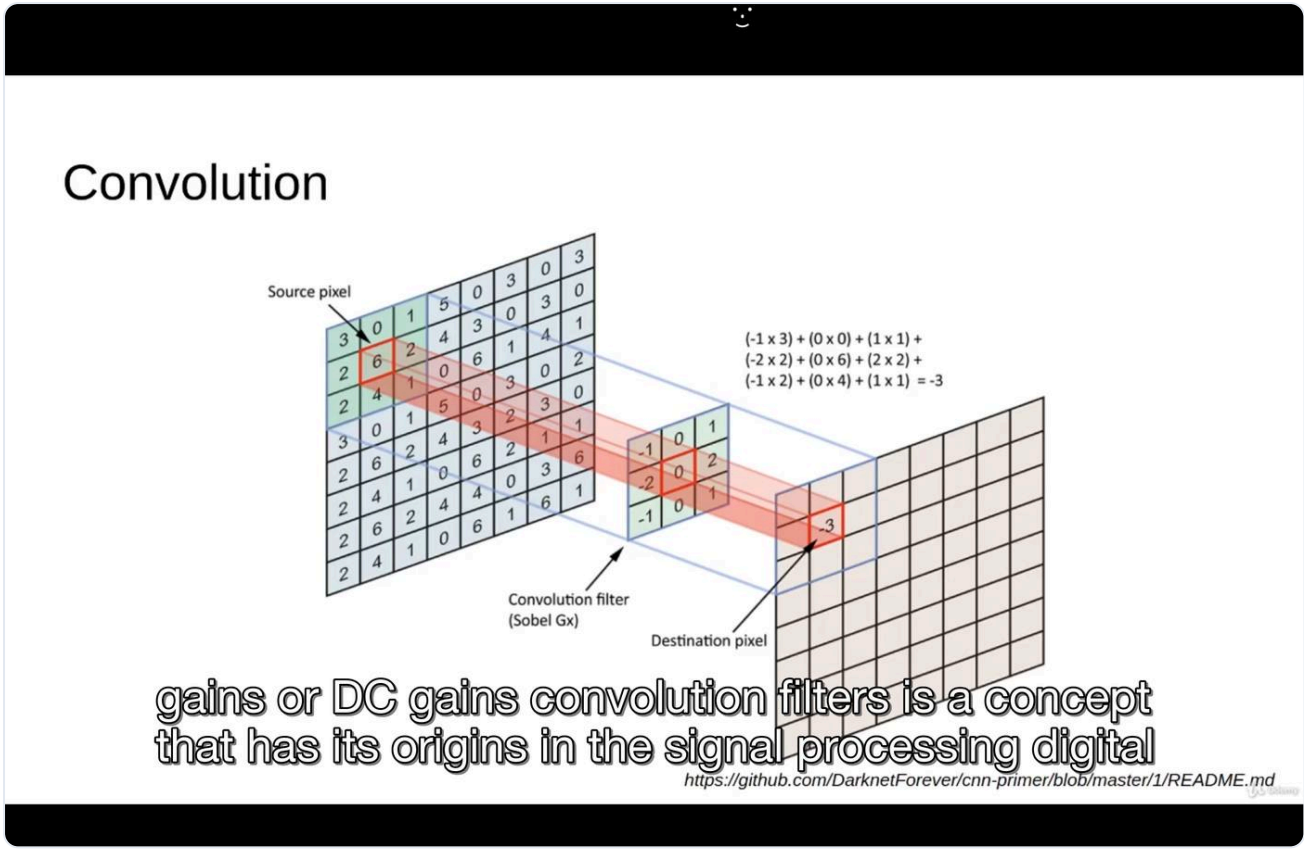


FIGURE 5.41

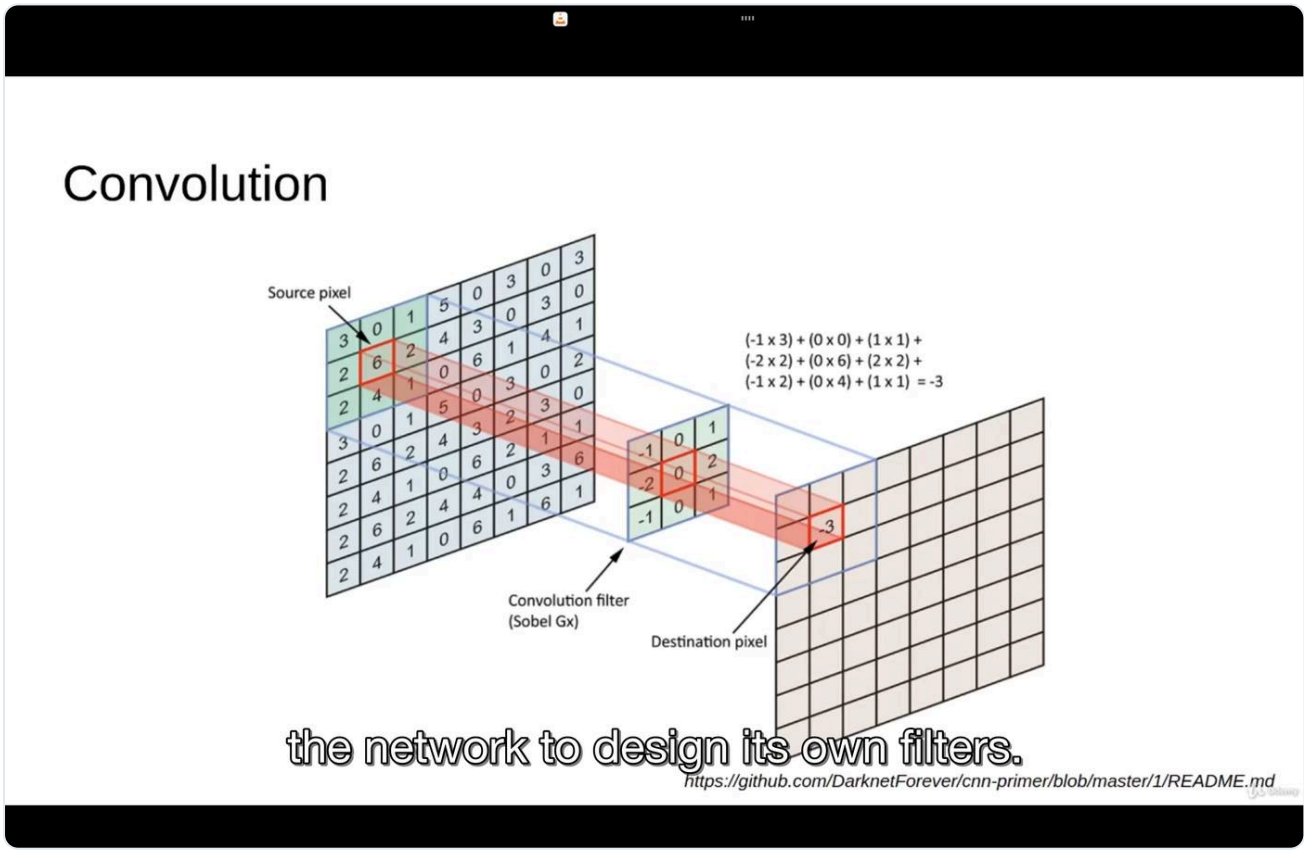


FIGURE 5.42

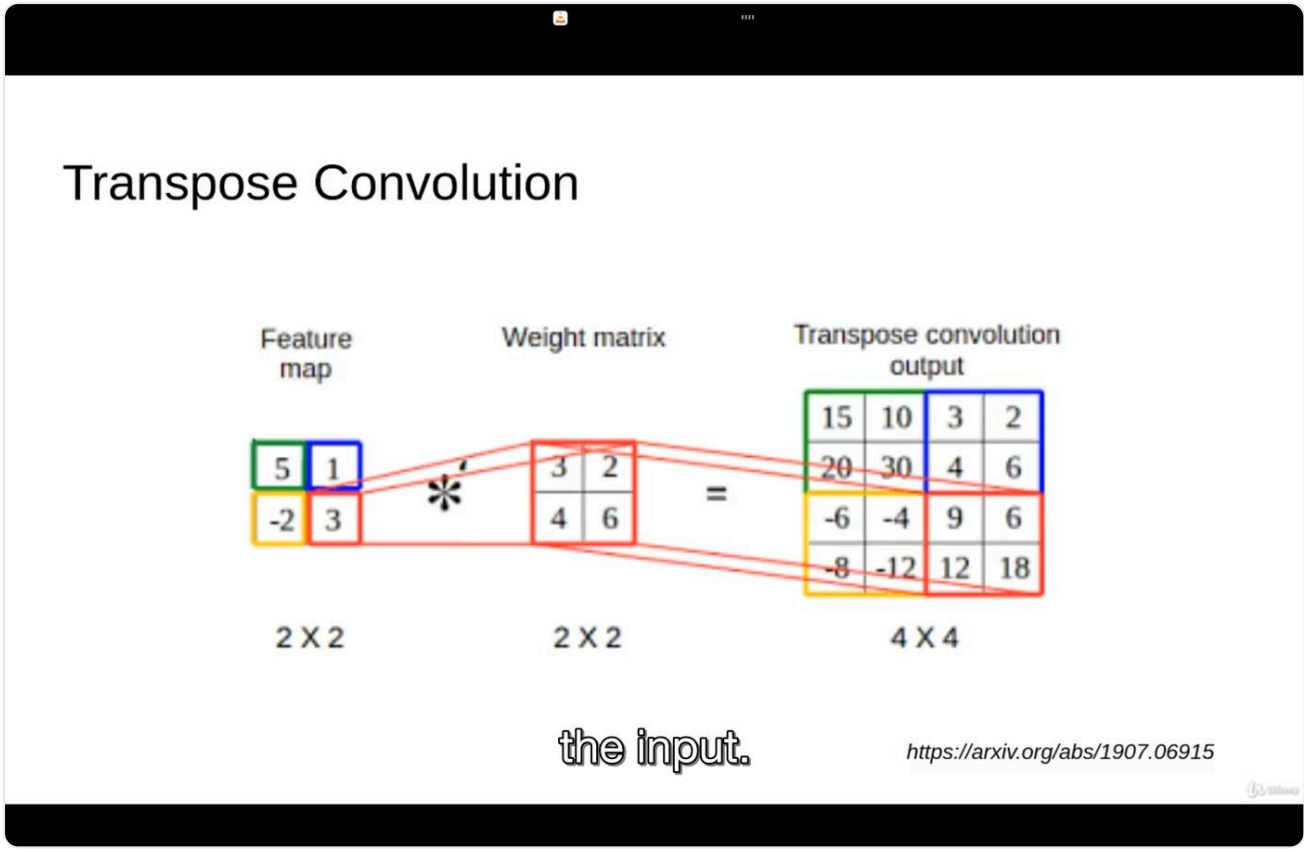


FIGURE 5.43

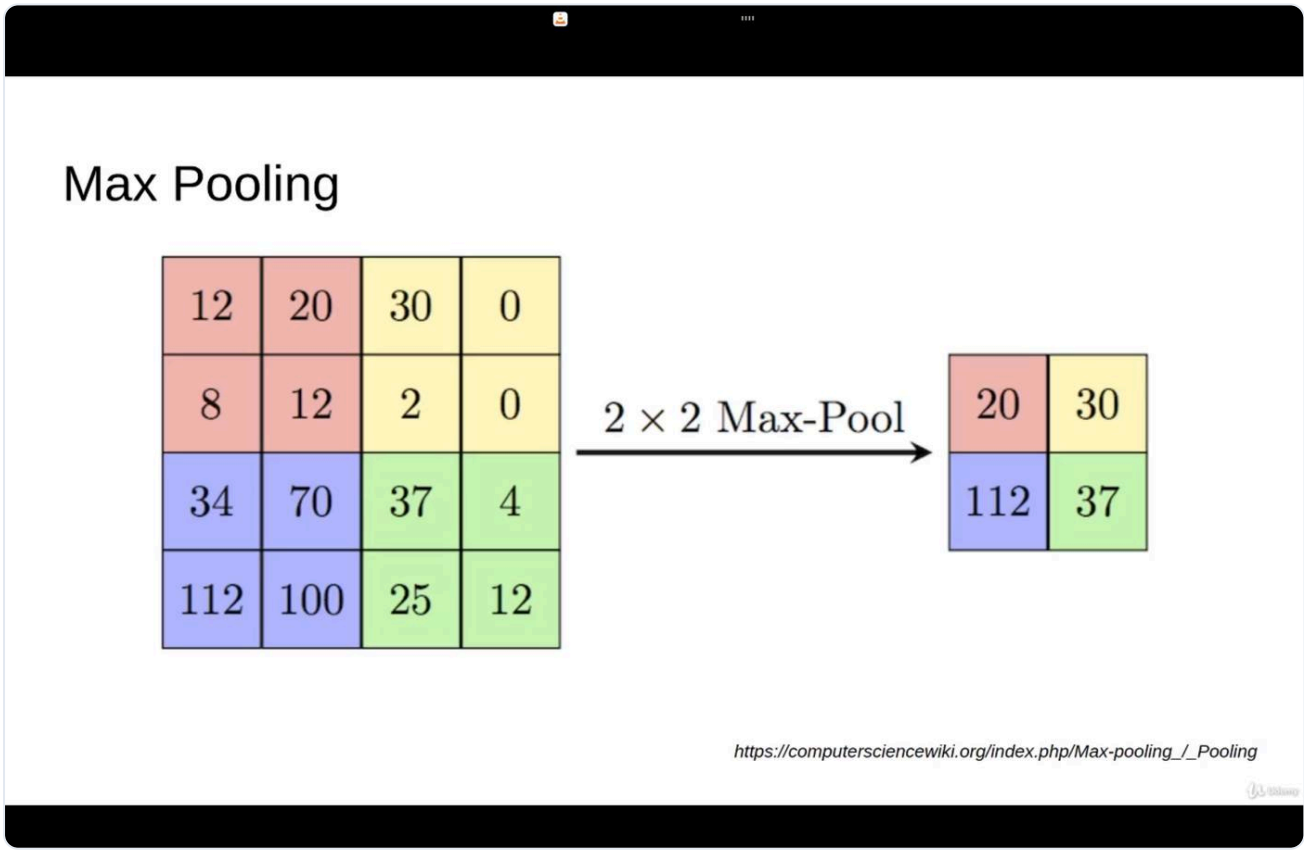


FIGURE 5.44

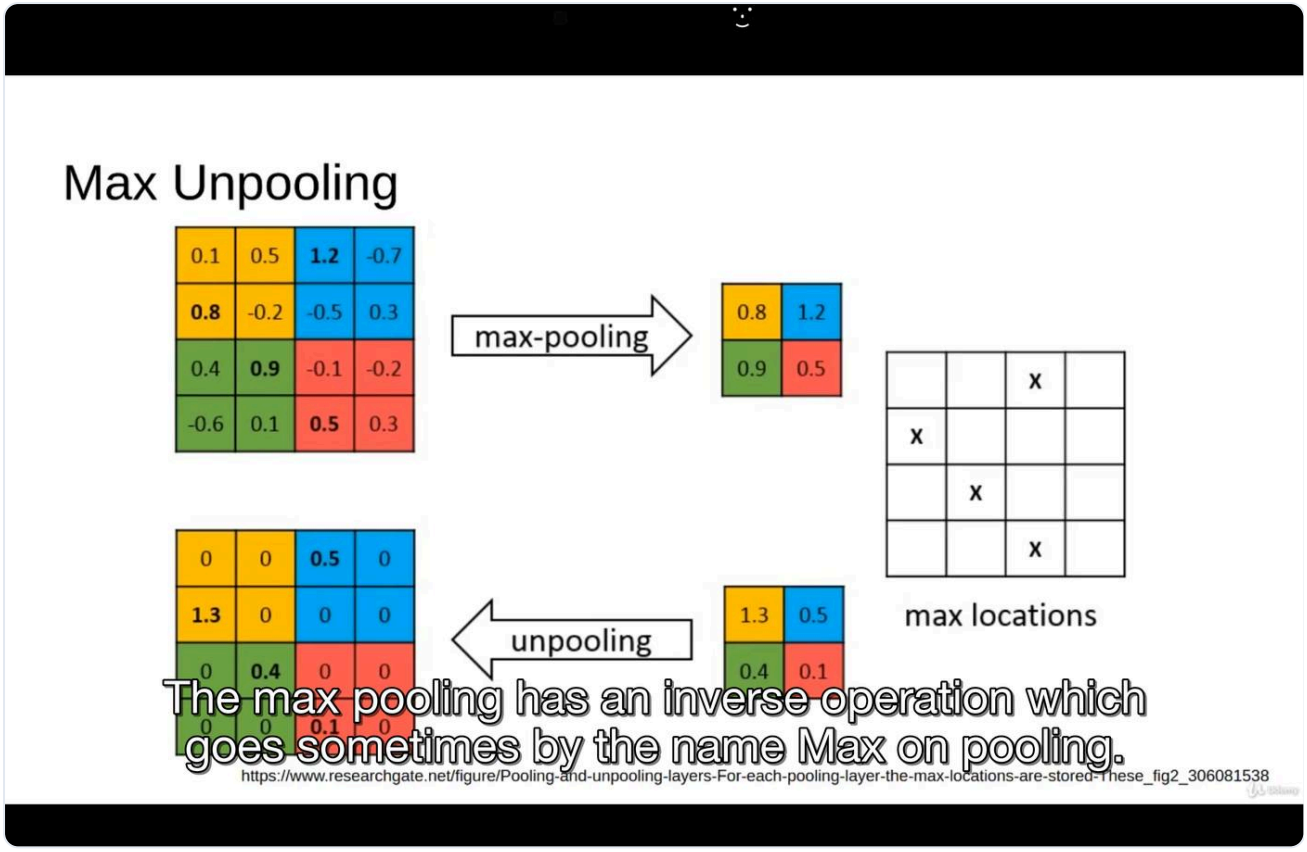


FIGURE 5.45

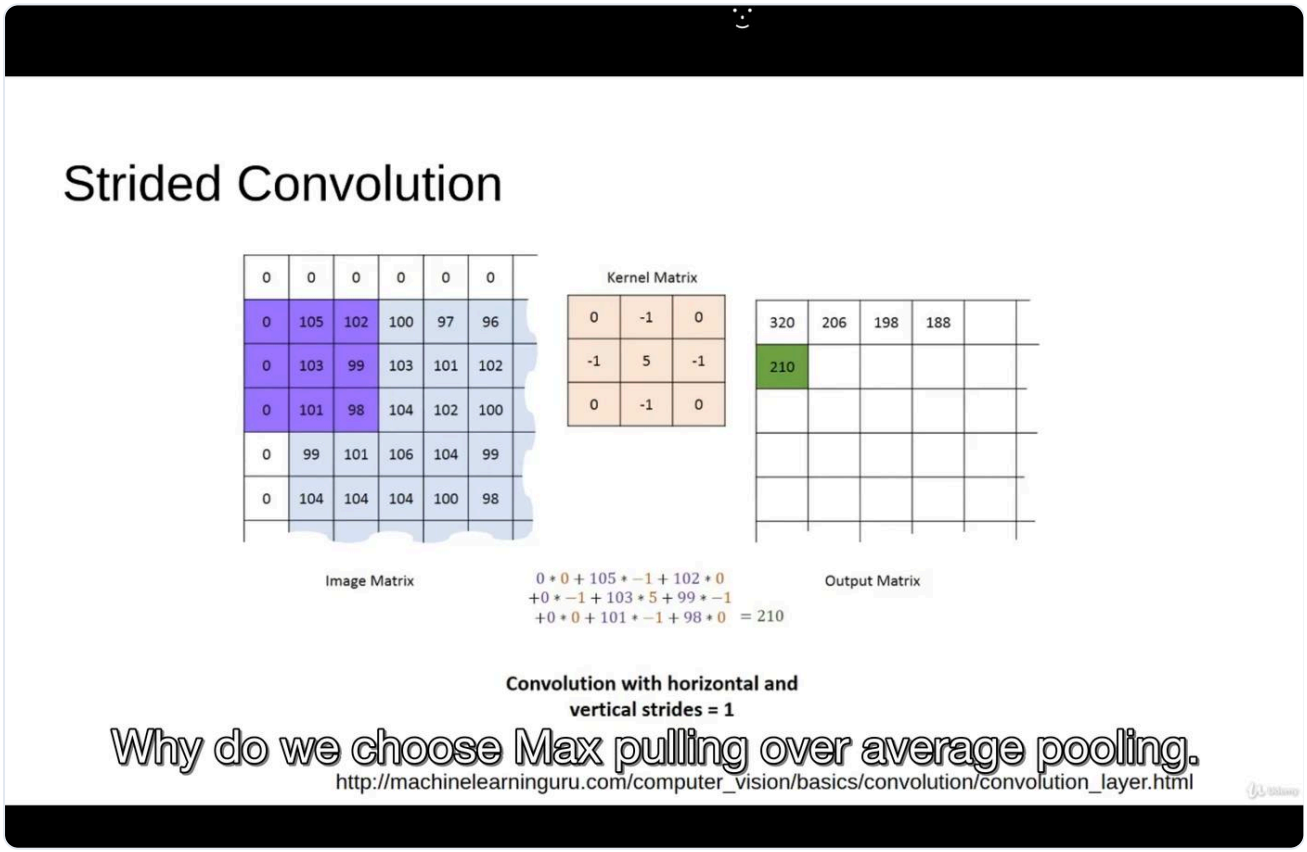


FIGURE 5.46

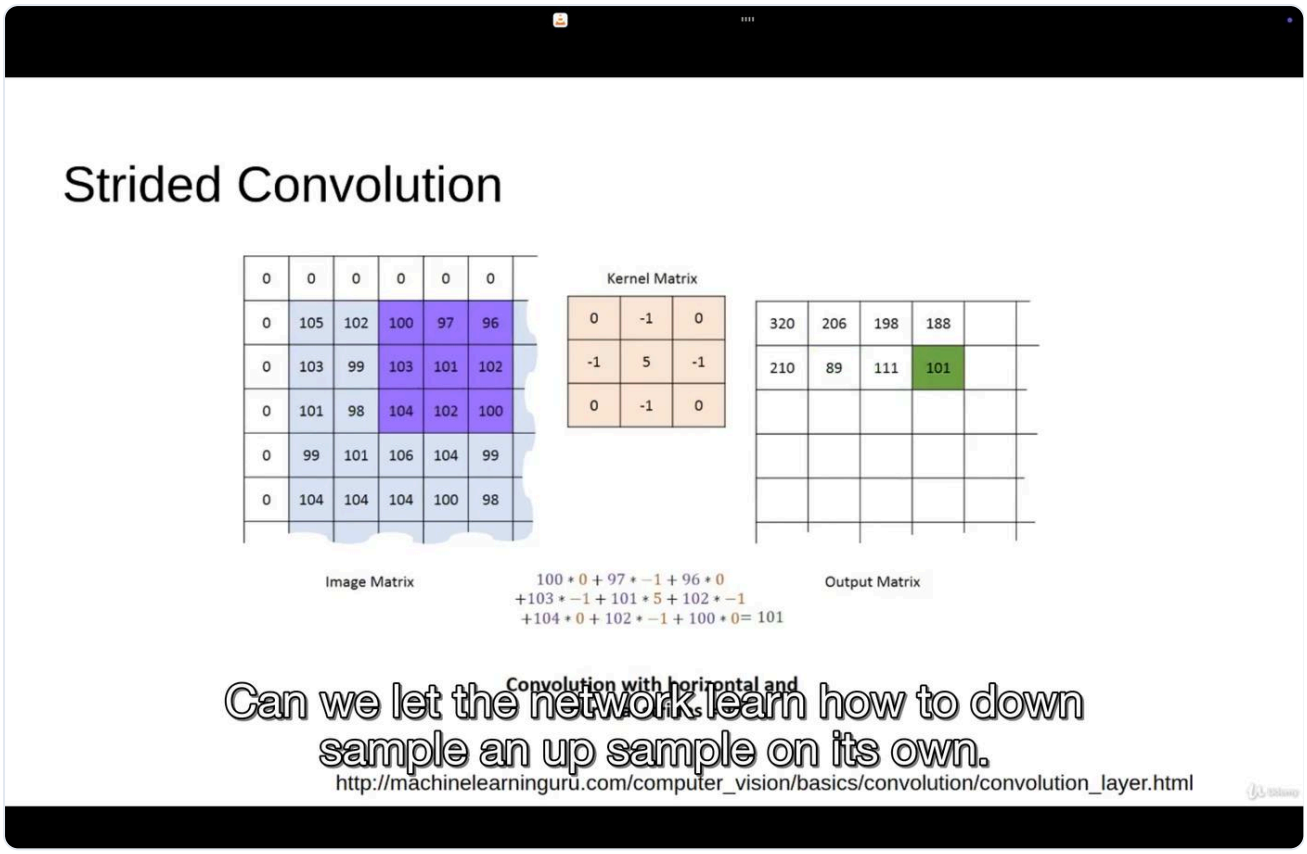


FIGURE 5.47

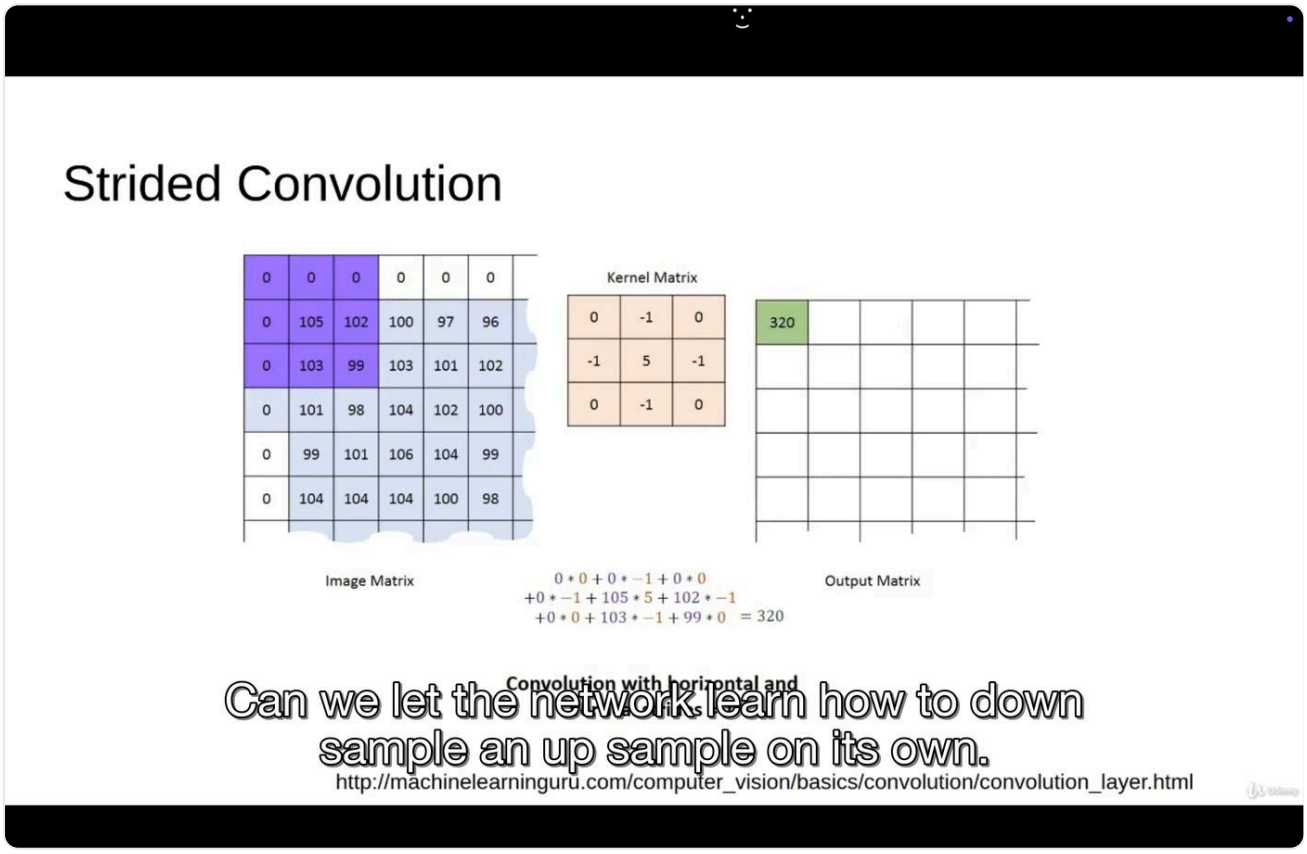


FIGURE 5.48

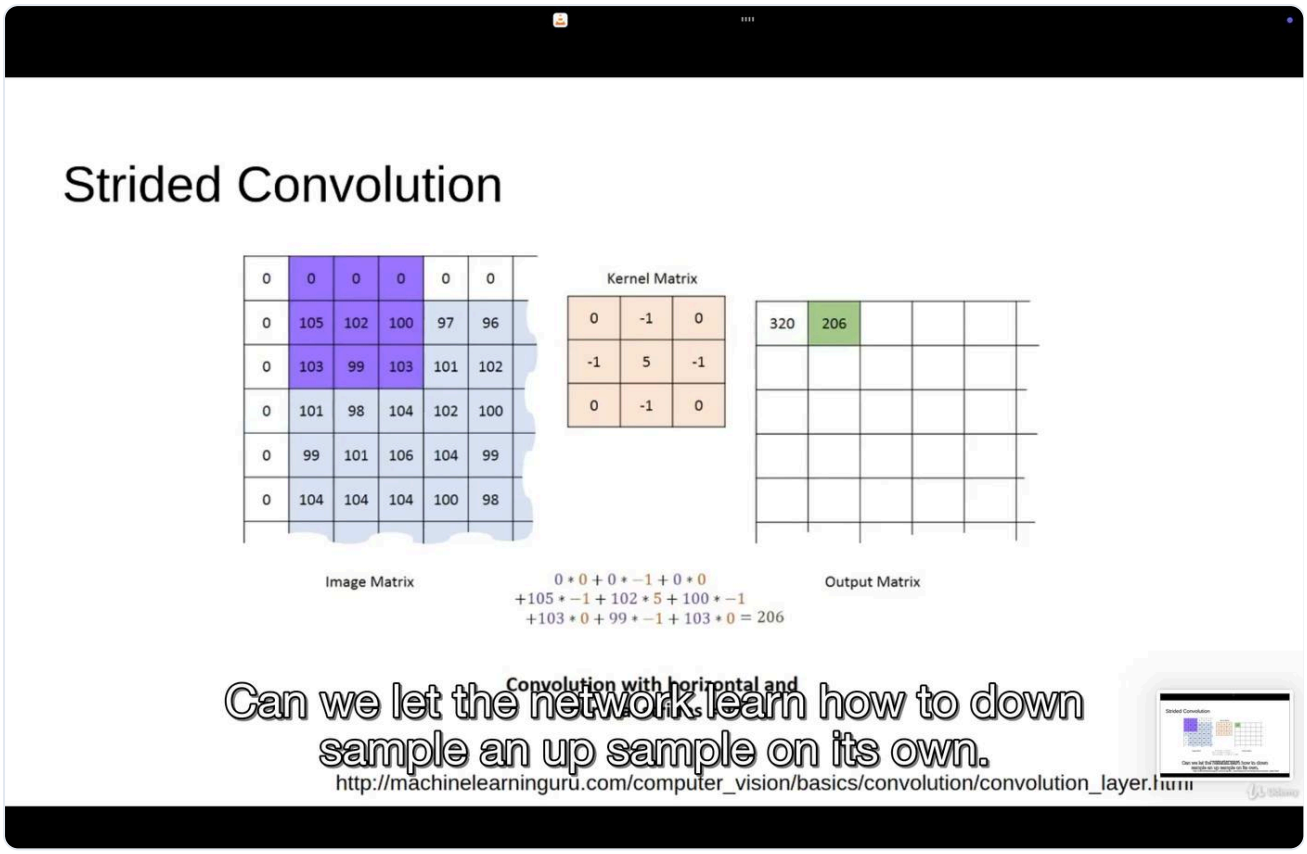


FIGURE 5.49

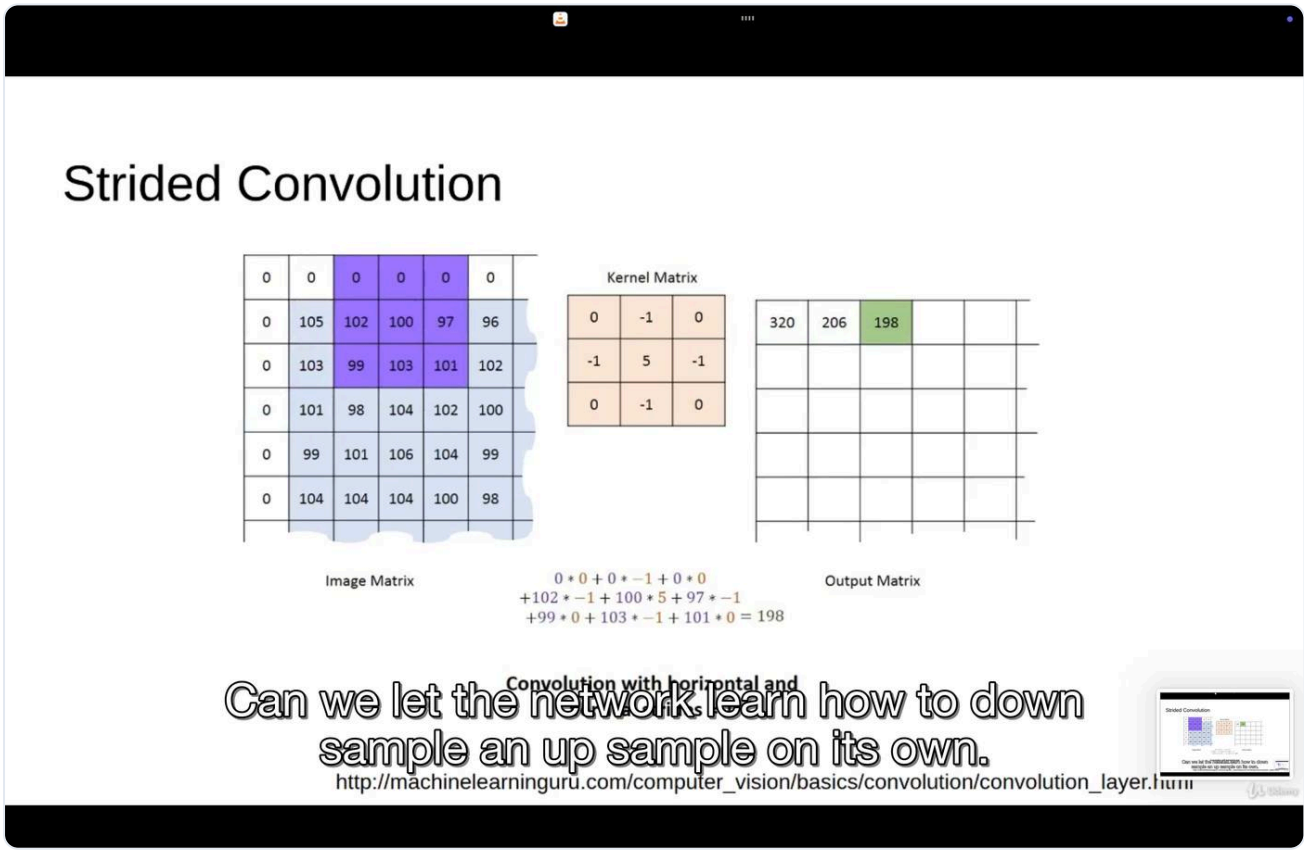


FIGURE 5.50

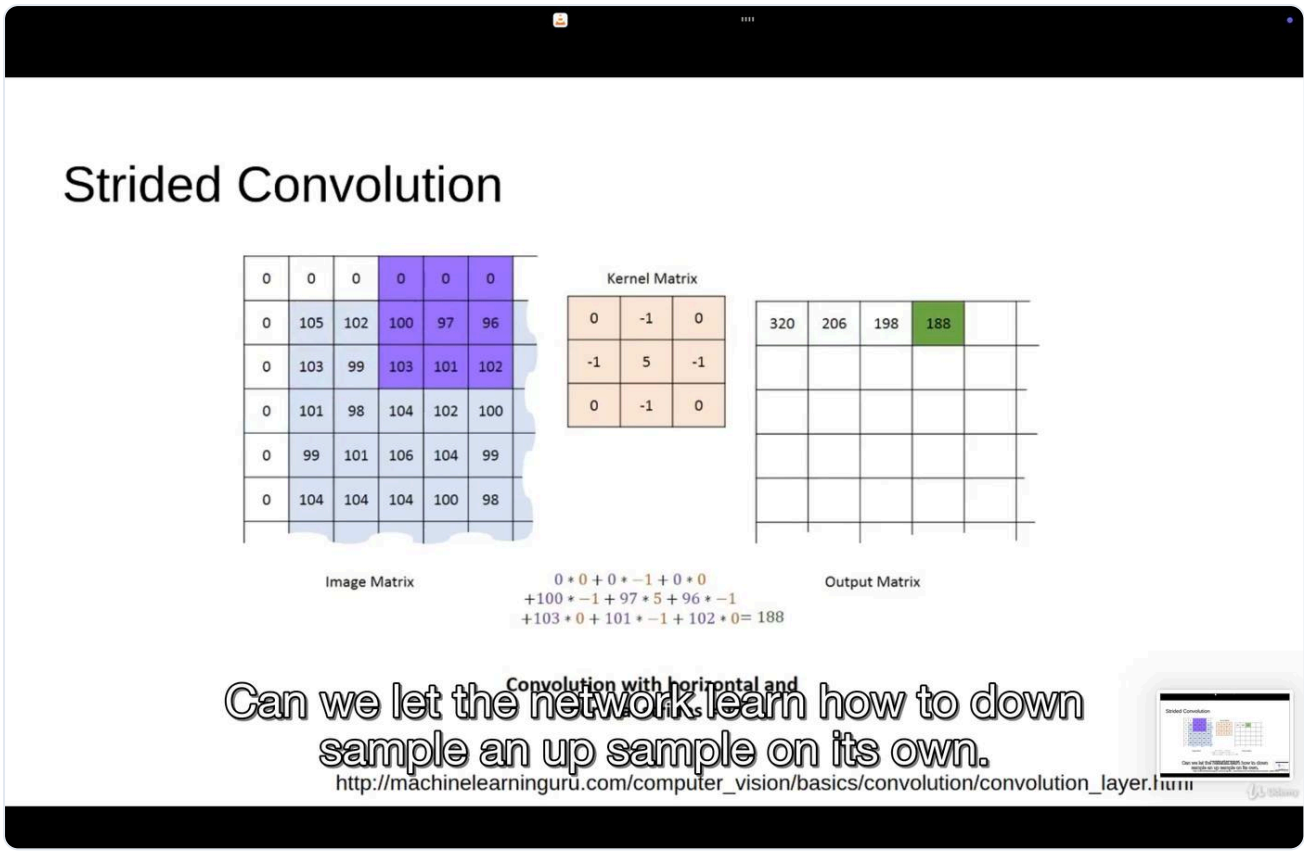


FIGURE 5.51

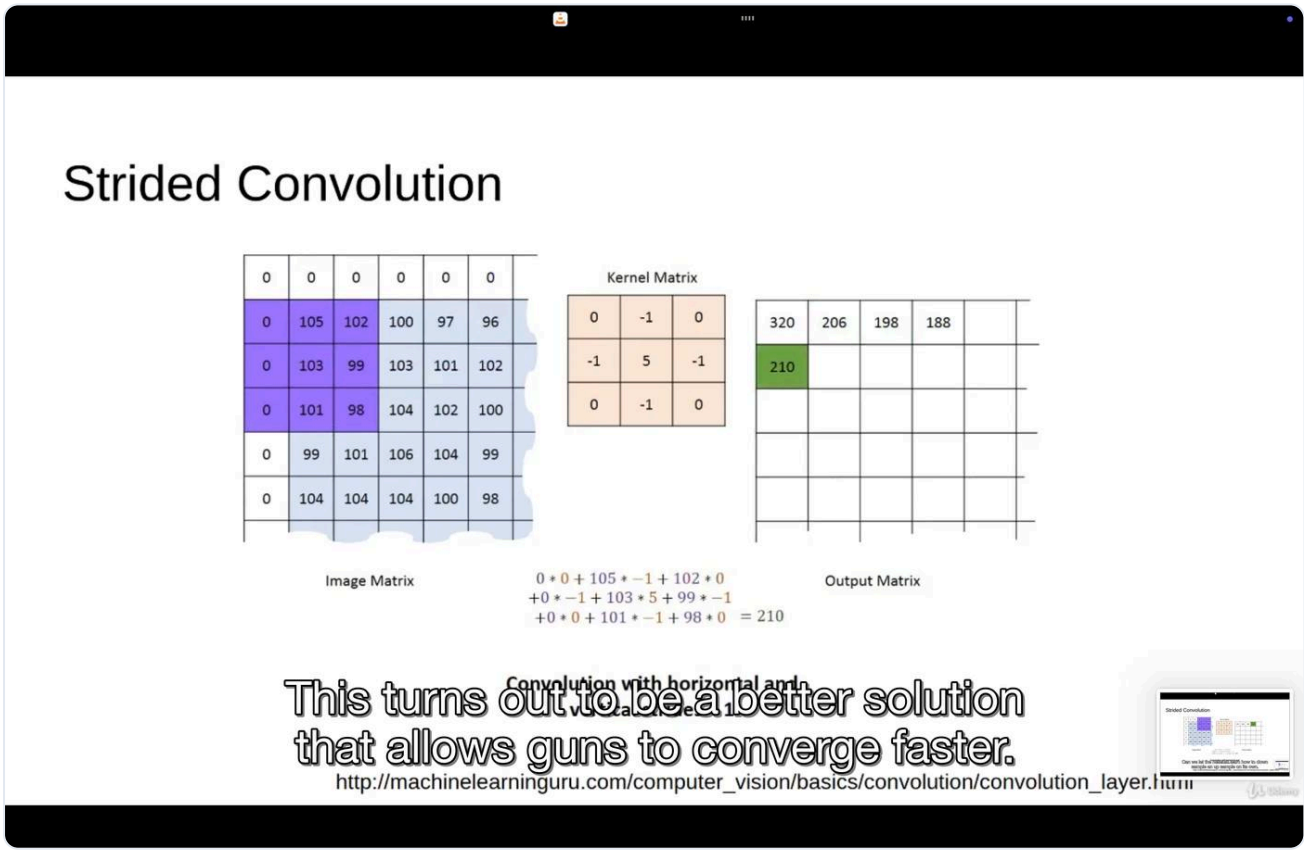


FIGURE 5.52

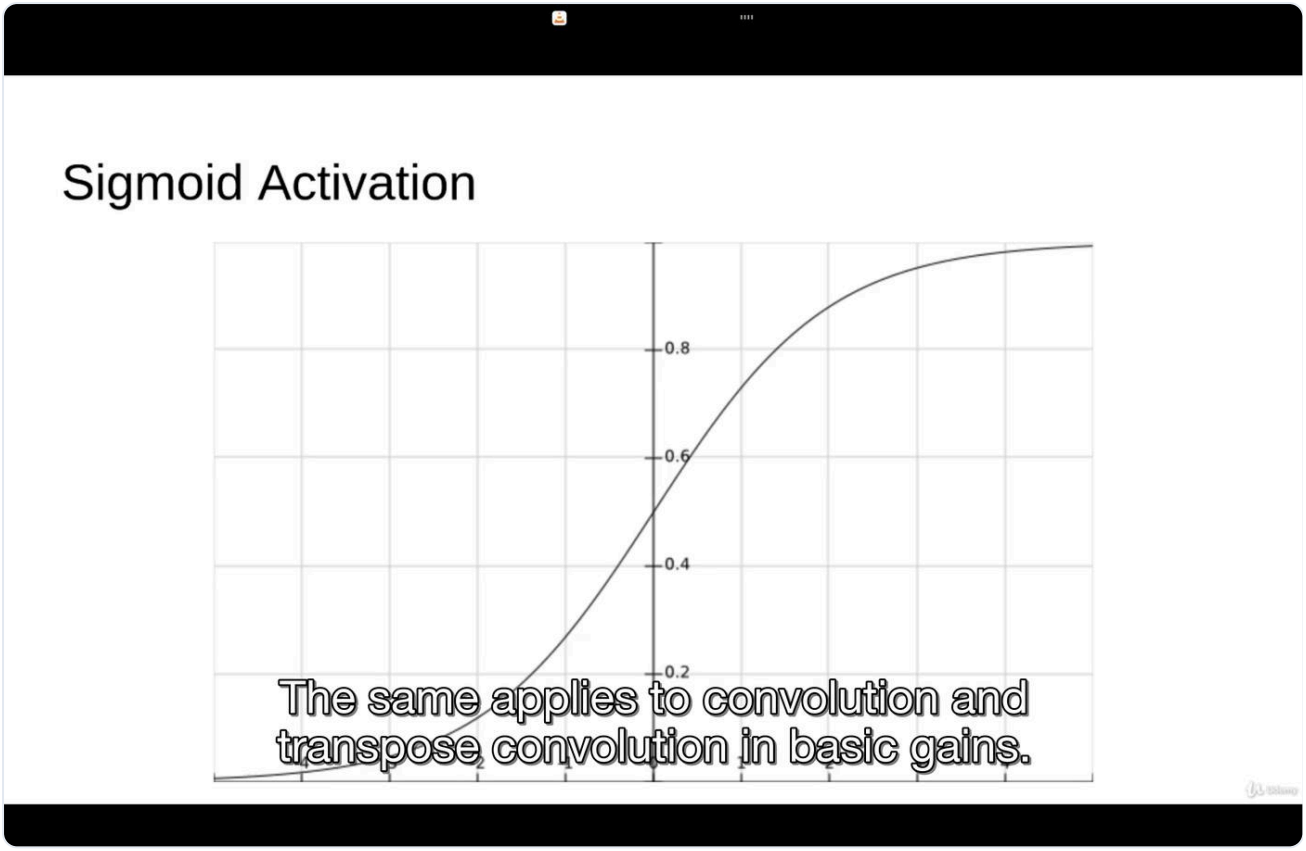


FIGURE 5.53 | Sigmoid Activation

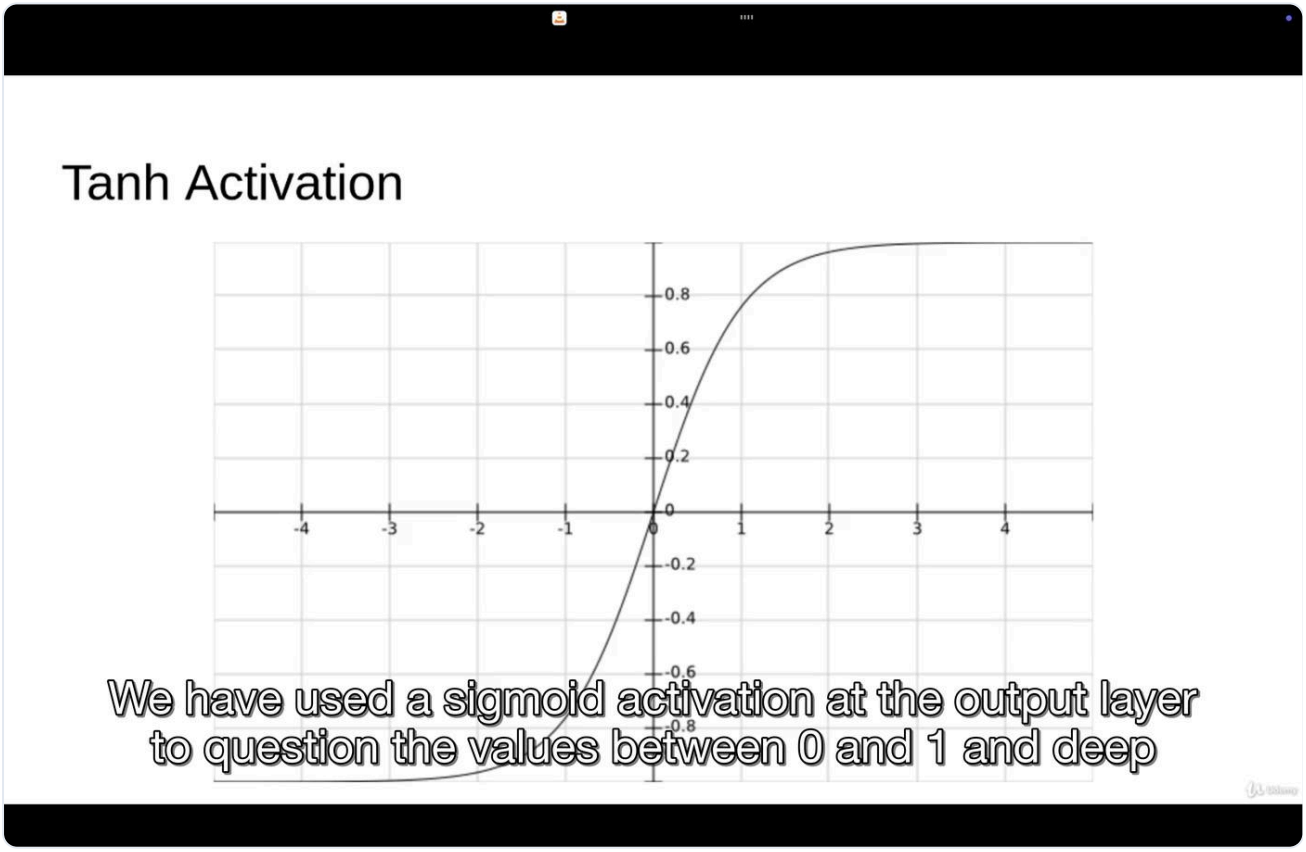


FIGURE 5.54 | Tanh Activation

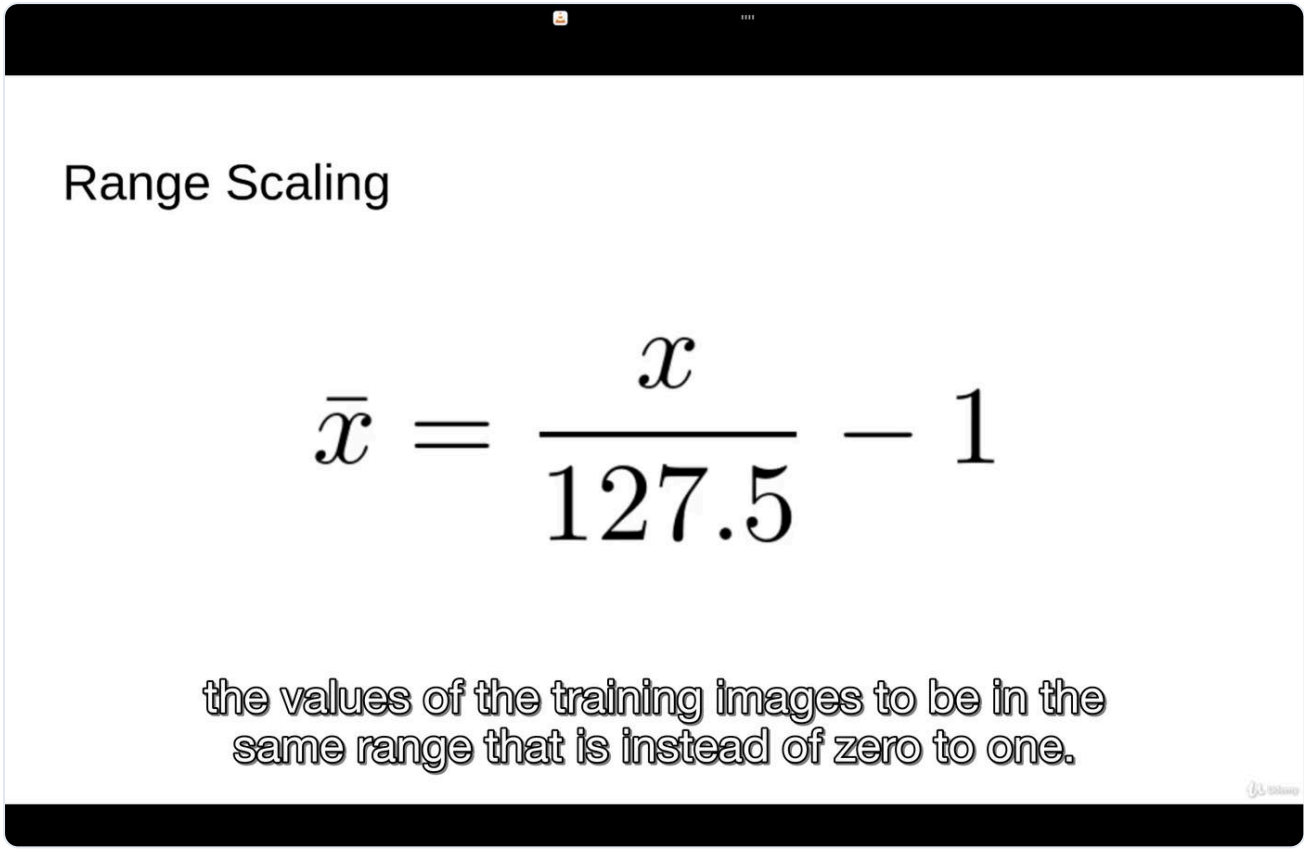


FIGURE 5.55

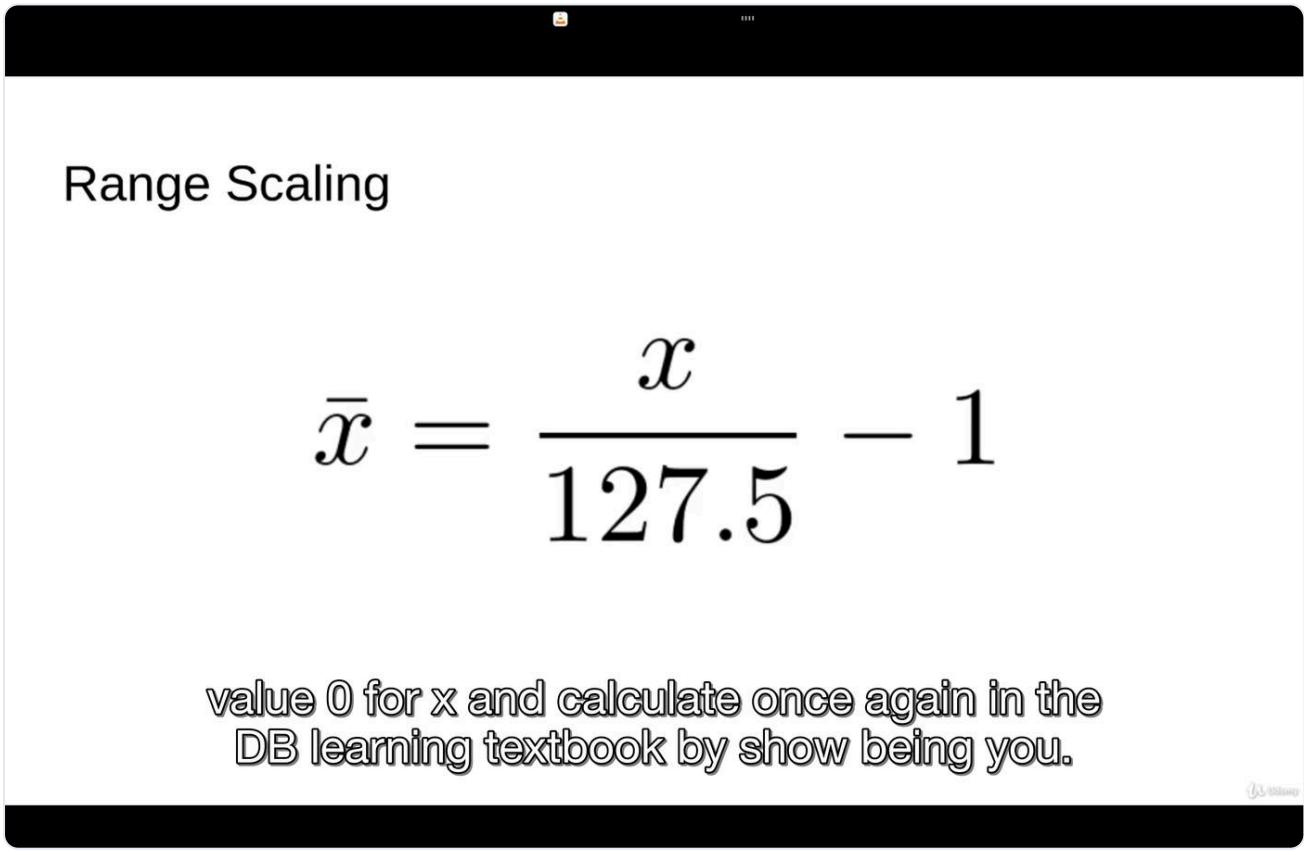


FIGURE 5.56

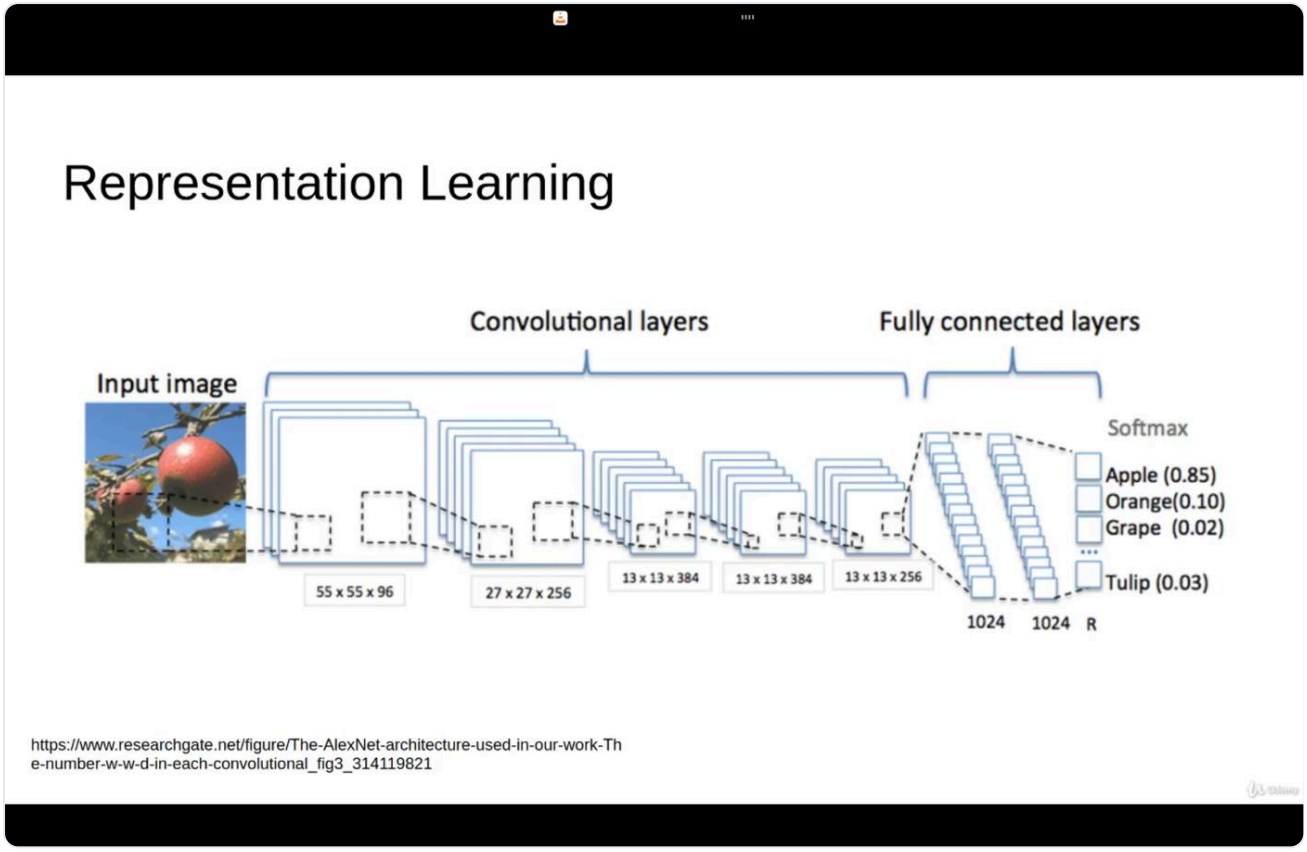


FIGURE 5.57

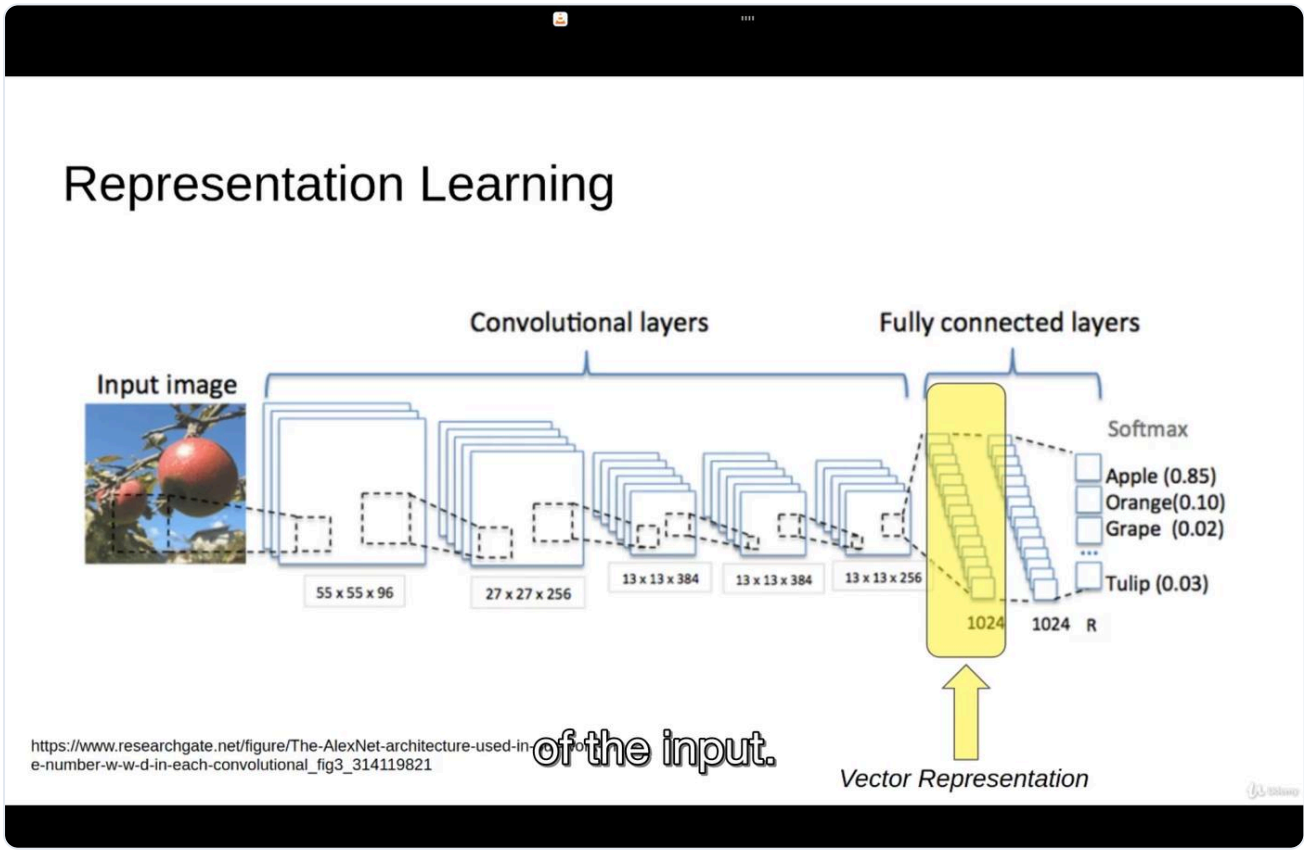


FIGURE 5.58

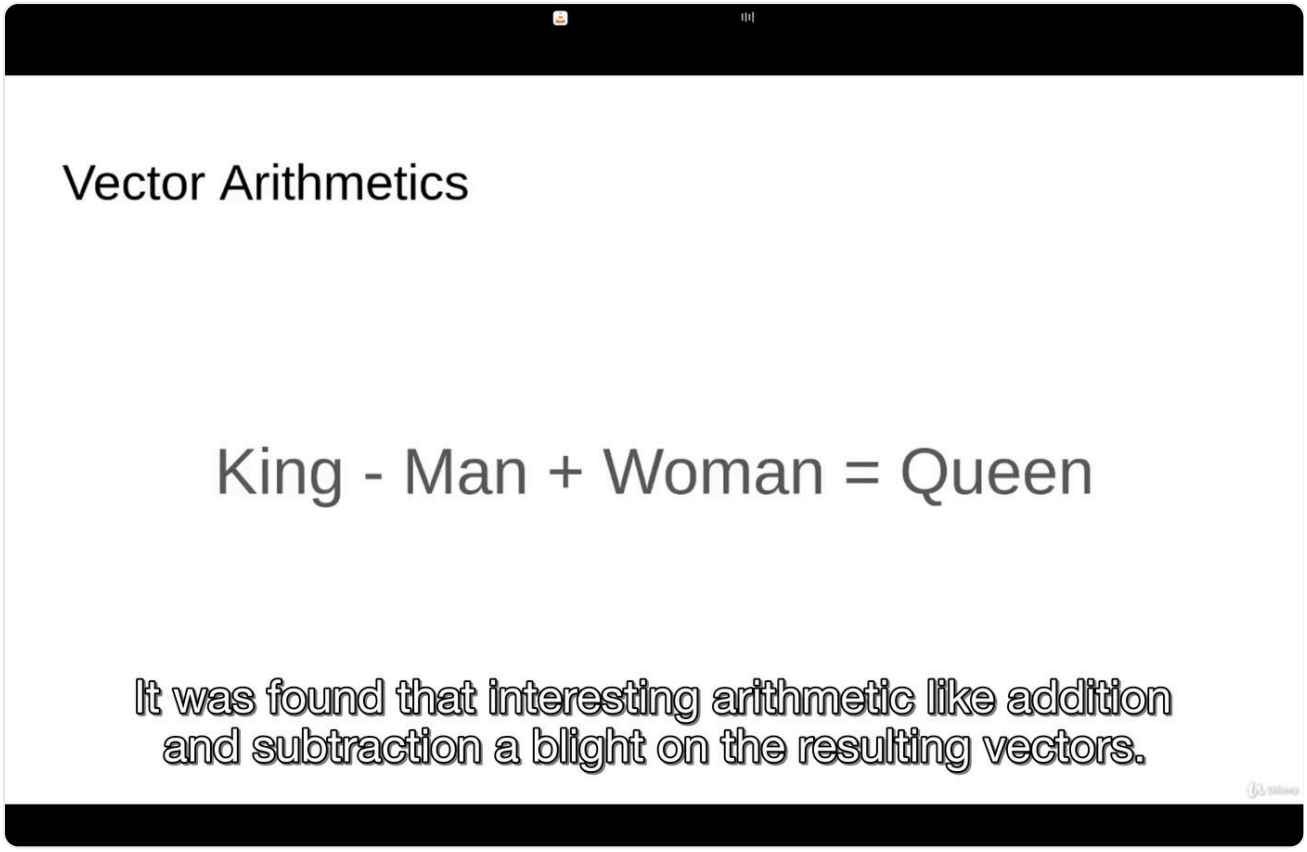


FIGURE 5.59

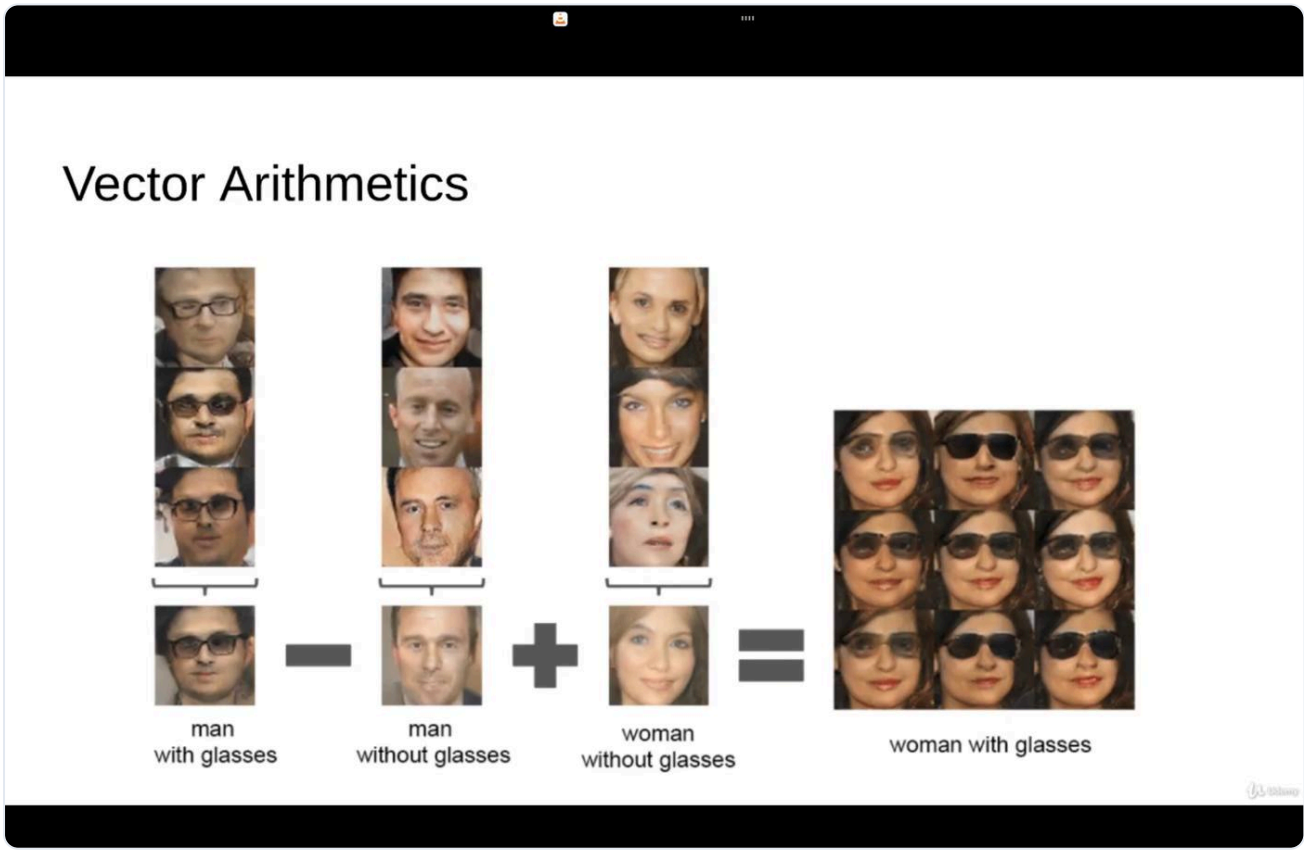


FIGURE 5.60

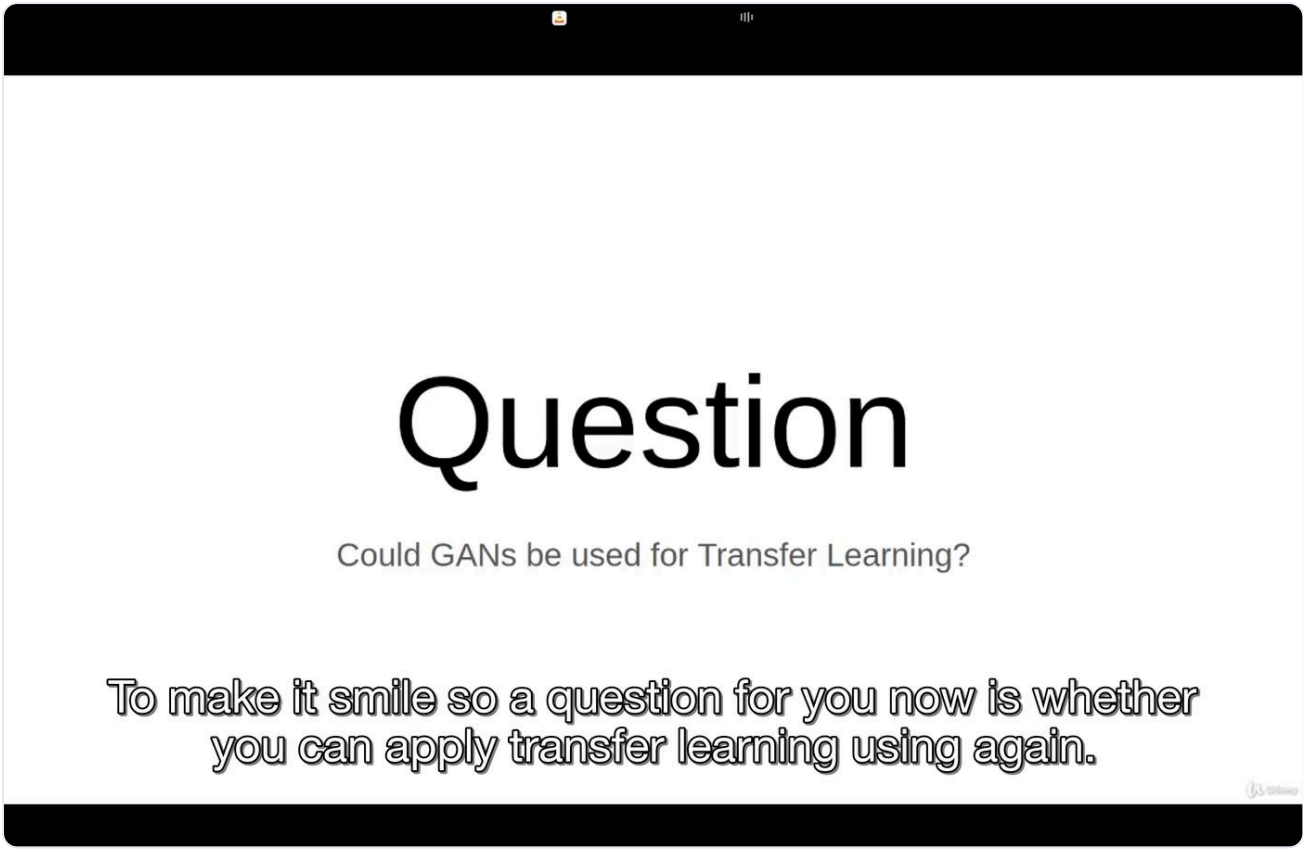


FIGURE 5.61 | Could GANs Be Used for Transfer Learning?

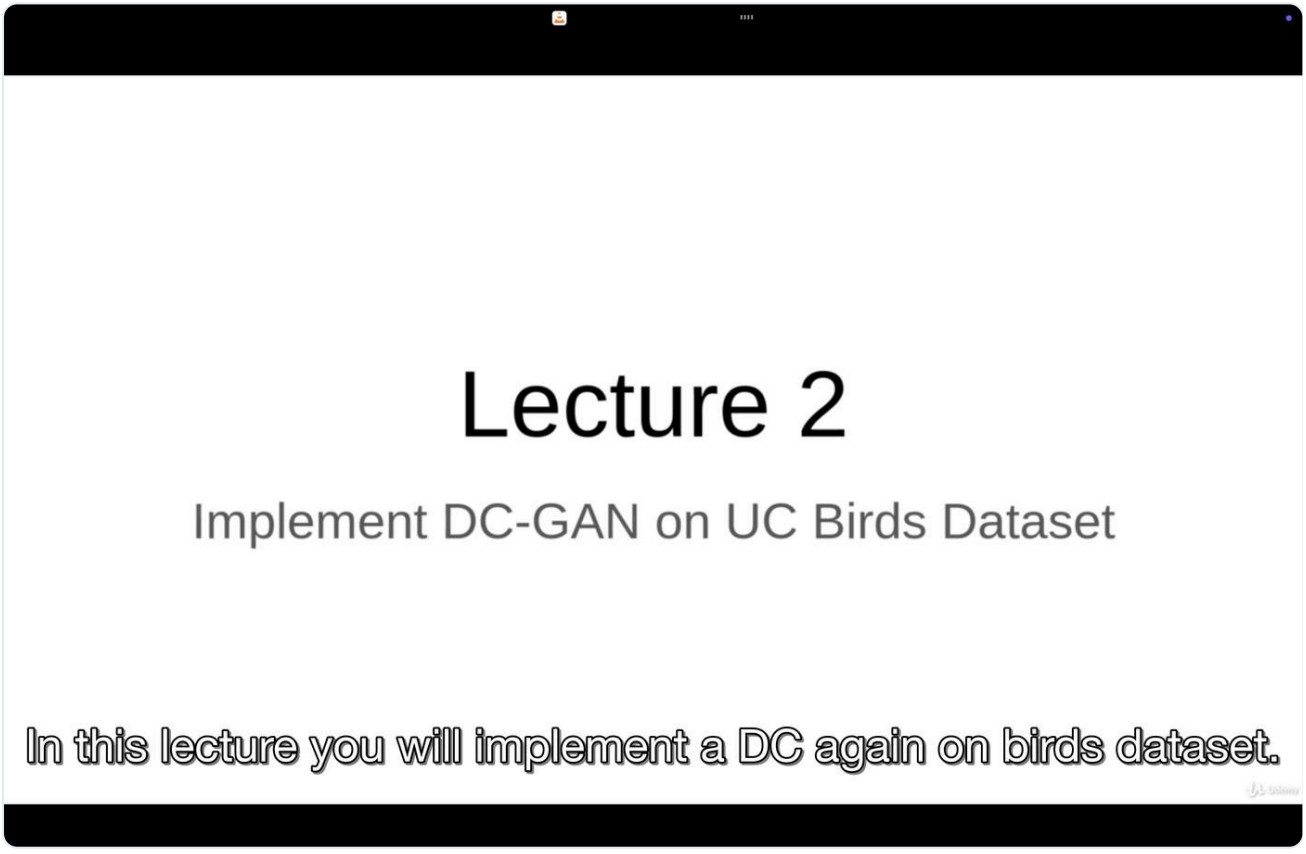


FIGURE 5.62

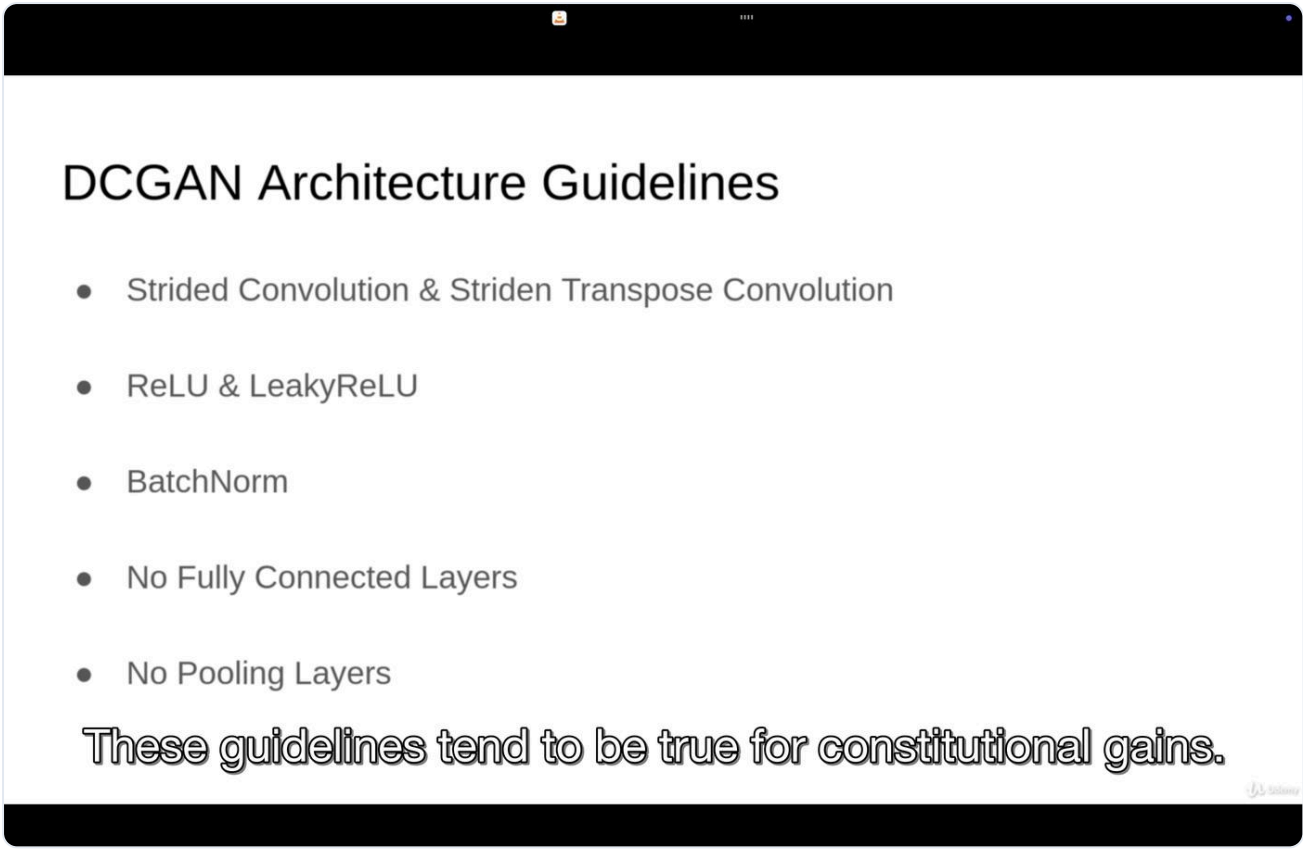


FIGURE 5.63

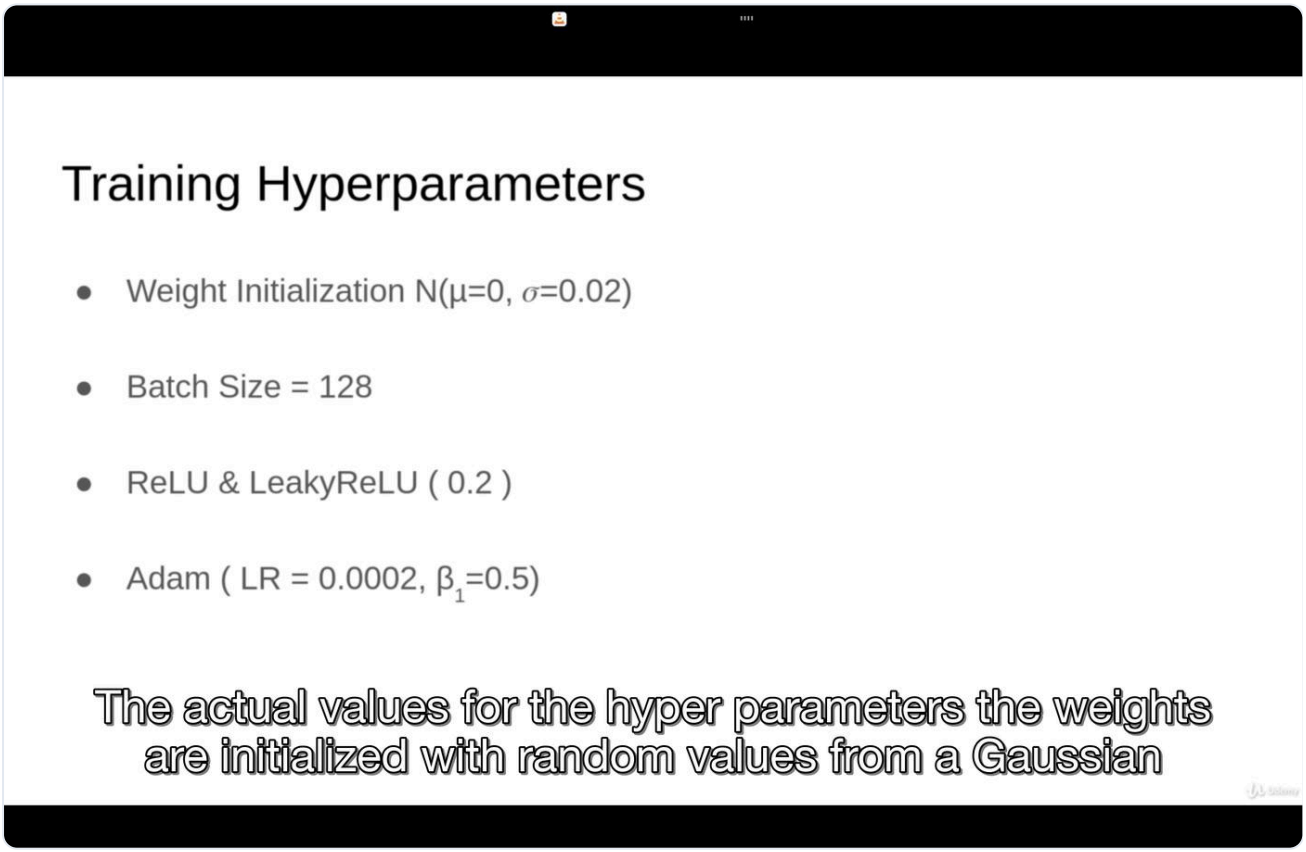


FIGURE 5.64 | Training Hyperparameters

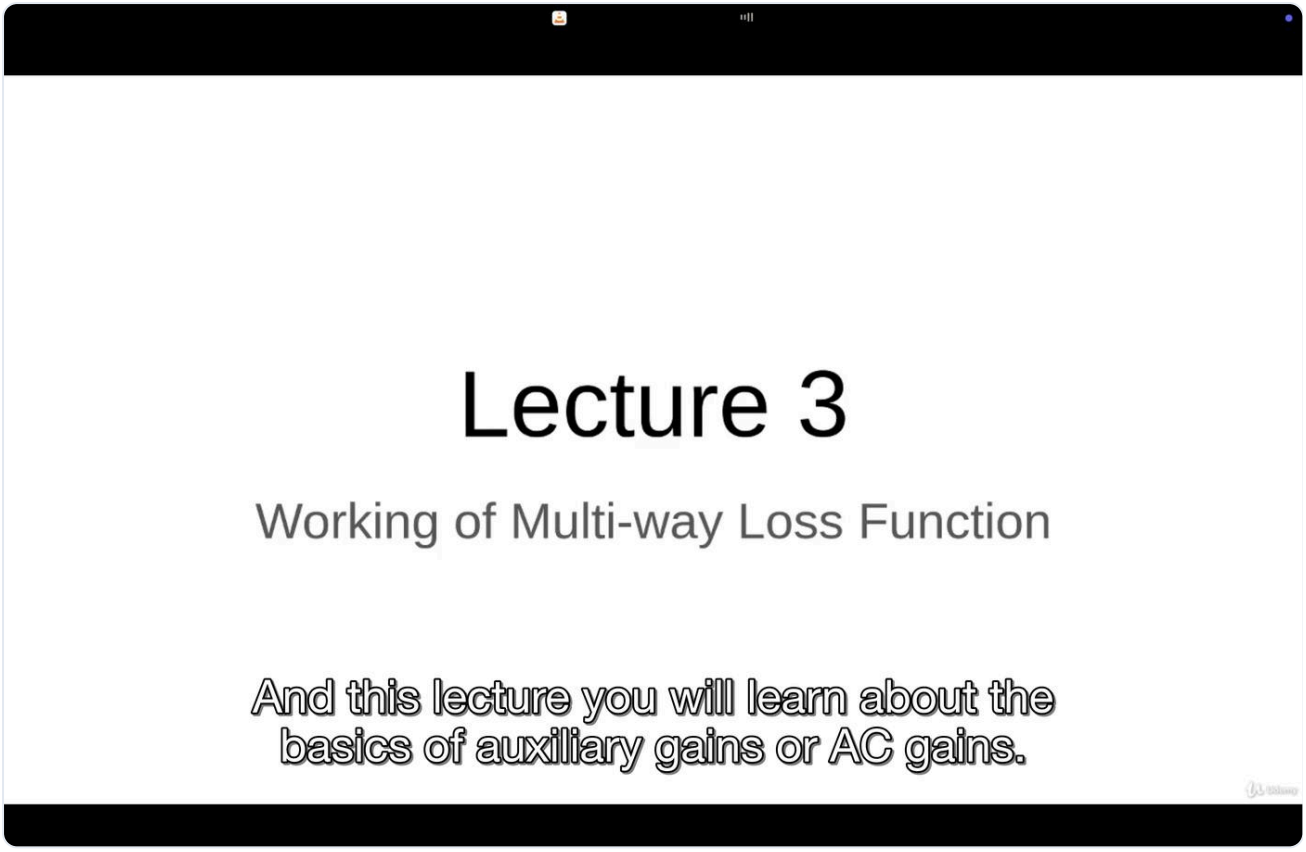


FIGURE 5.65 | Working of Multi-way Loss Function

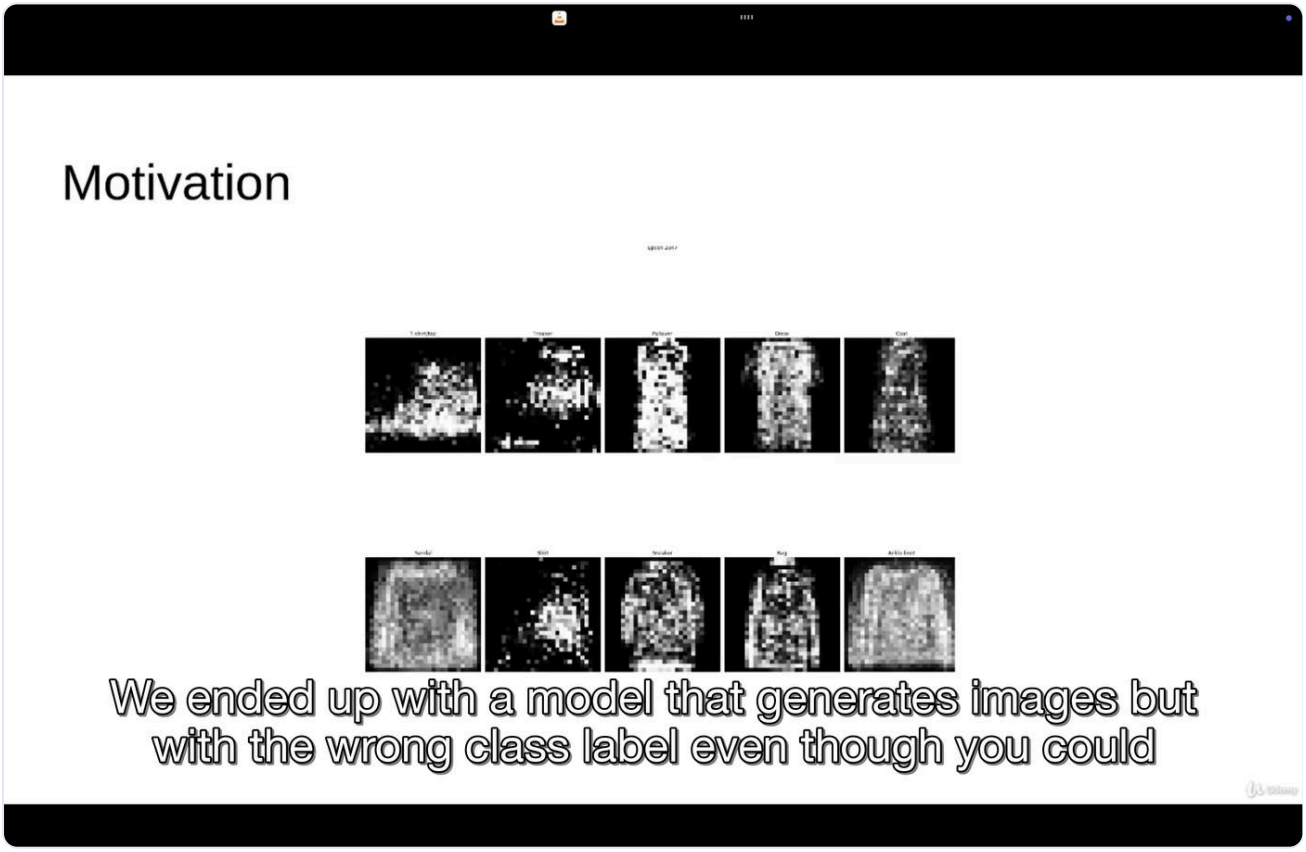


FIGURE 5.66

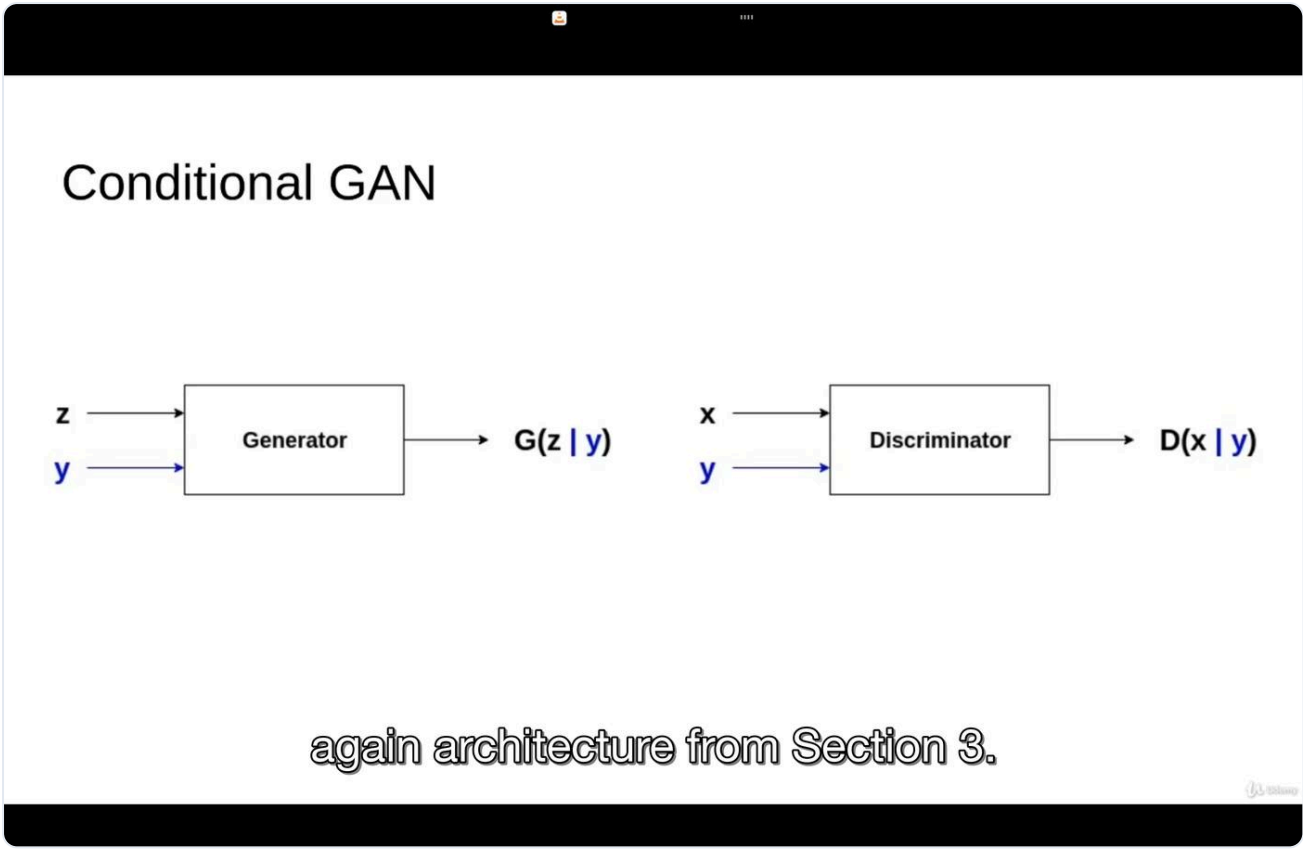


FIGURE 5.67

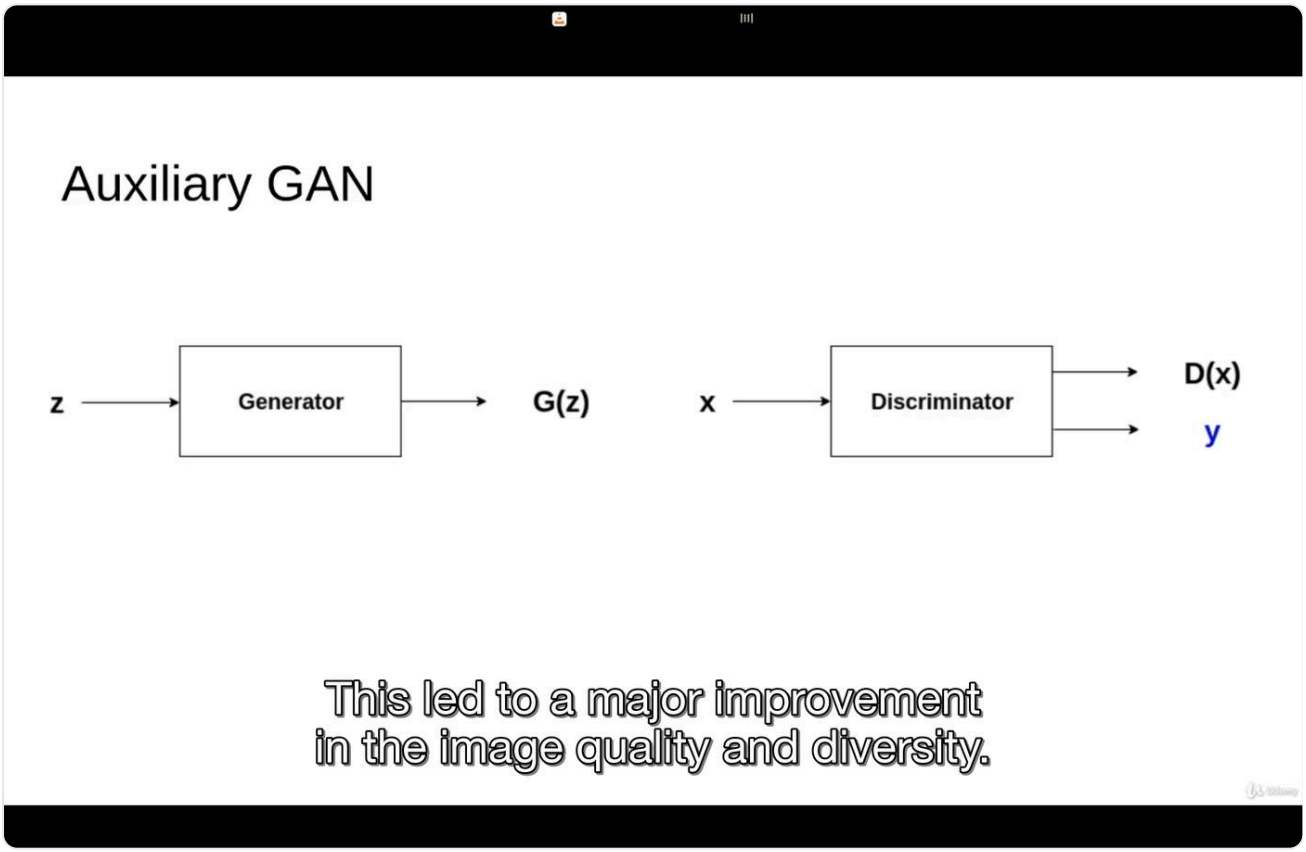


FIGURE 5.68

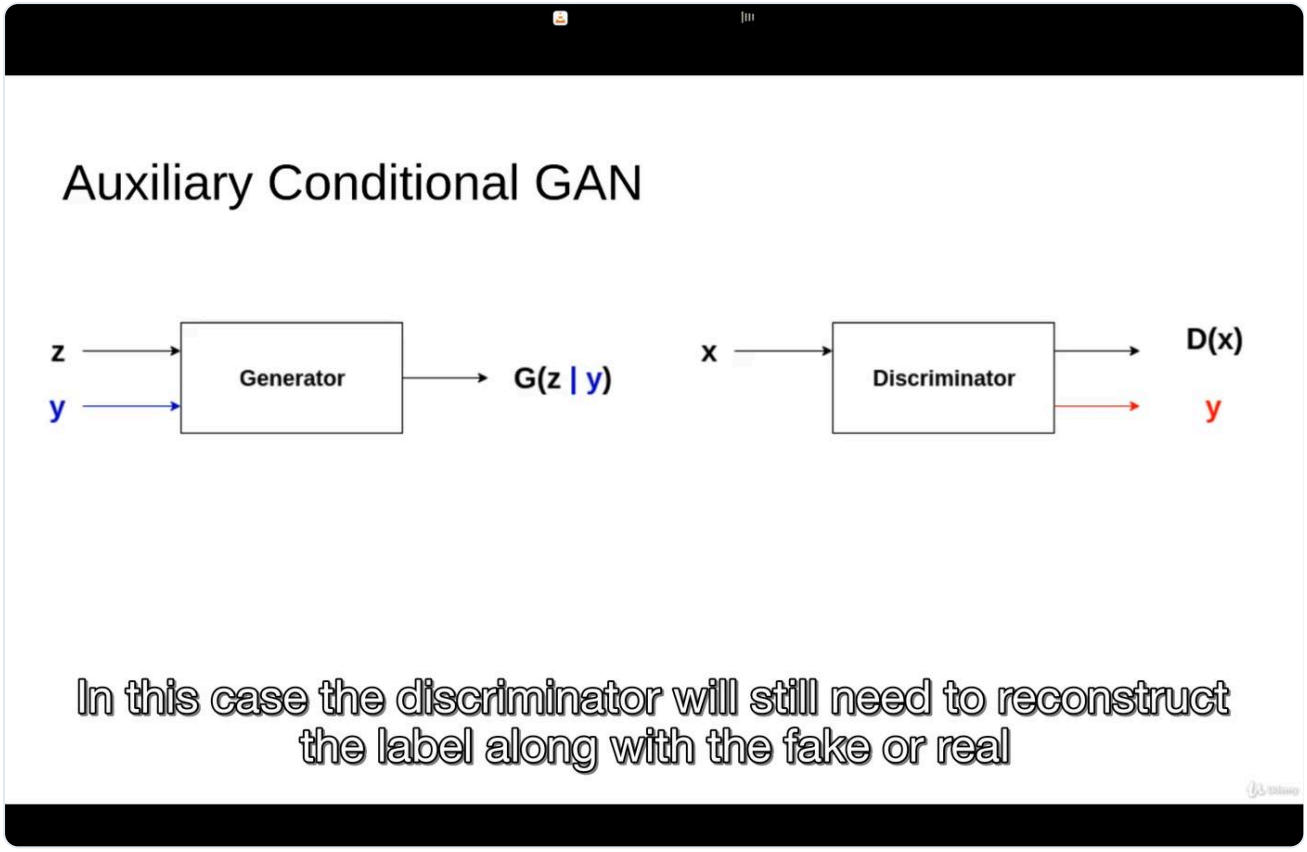


FIGURE 5.69

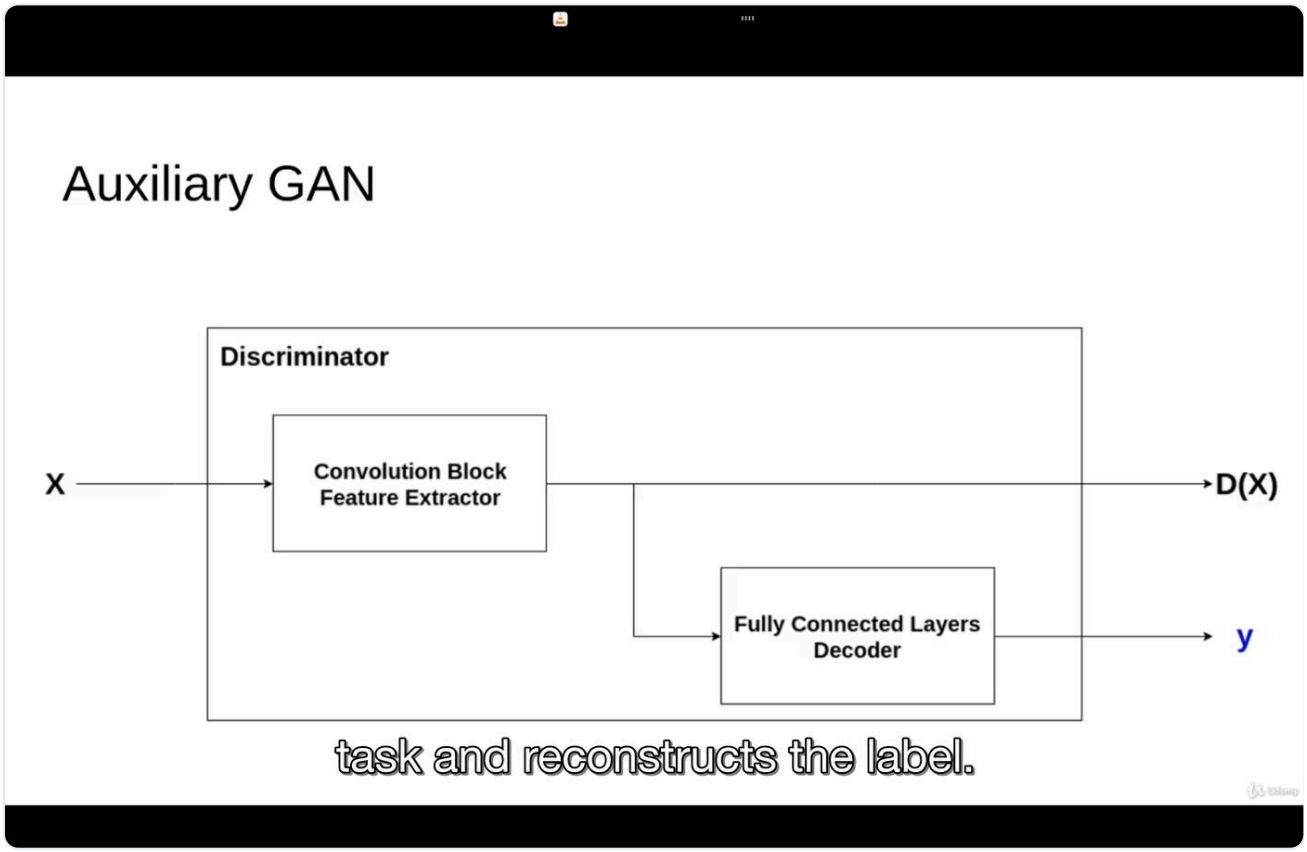


FIGURE 5.70

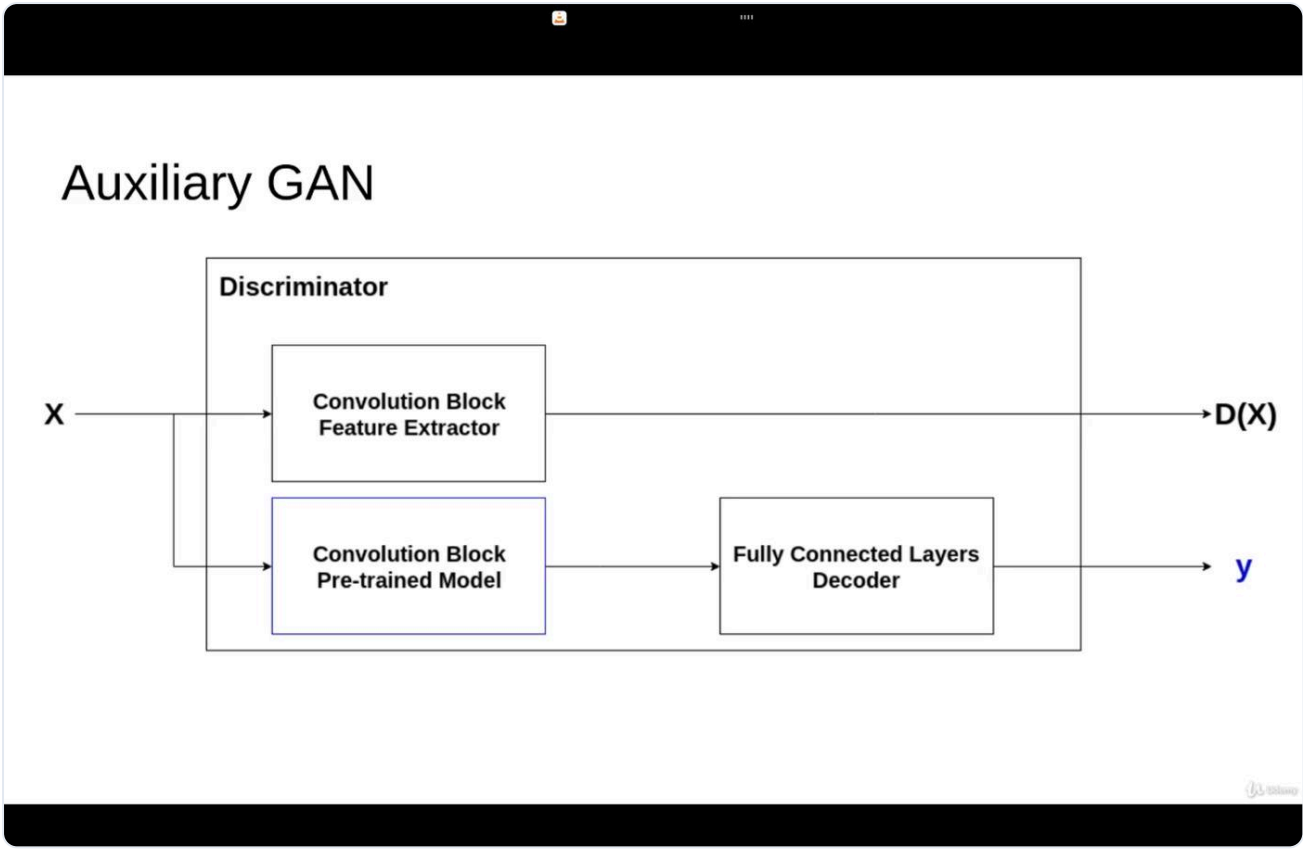


FIGURE 5.71

If you don't have a GPU you'd like to try to run the code on the GPU in the cloud in the course the material you you'll find another file by the name named section 2 Dash. Version 2 where I have updated the learning rate and the momentum as follows to be learning rate point 0 5 and the momentum point 7 for both the generator and the discriminator as you can see here.

I have trained the discriminator for K steps which is equal to 16. That is the discriminator is trained sixteen times more than the generator over here you can see the output after 512 epochs and you could recognize the digits and they are more like the endless dataset letters. So for example the first one we have got something between eight and 6. The second one is somewhere between five and the same third one fourth one are all five. Here we have got something else.

I'm not quite sure what it resembles but here we have got something that looks like three and then something that looks like it. And then for the third row we have got more lines for the fourth row. We have got 9 9 5 9 so that's much better than the previous result. Still we cannot control the output. We cannot set conditions and we cannot make sure that the conditions are obeyed. These are things that we are going to discuss in deep detail in Section 3 and section 4.

So stay tuned again here you have got Google Colab notebook link for the file which I have also uploaded to the cloud. In case you'd like to try it on Google Colab So that concludes our discussion of Section 2. I hope that you have enjoyed it but to make things more challenging. Here are some ideas that I'd like you to try so implement the same base GAN model on Fashion-MNIST dataset and try some of these ideas so implement the base GAN model we have discussed throughout this section on the Fashion-MNIST dataset and try some of these ideas try to train only on one class on three classes try to fine tune the value of the dropout layers try uniform

noise instead of random Gaussian noise and then instead of sigmoid at the generator output try to use tanh and make the required changes and also try to find out the optimal number of pieces for the maxout activation using cross validation and finally try to visualize the gradients.

This is very essential step and there will be some of the discussion in Section 3 and section 4. to inspect the gradient flow in the network. Finally I would like to recommend that you read the original papers on the generative adversarial networks by Ian Goodfellow and his team and the maxout networks in which he proposed that the maxout activation also by Ian Goodfellow and his team. There is a good open source book Deep Learning by M.I.T. press and I would like to recommend that you read chapter six section six point three point one for rectified linear units and their generalization in this case it talks about maxout activation and Chapter 20 section twenty point fourteen which talks about evaluating generative models and how this is very challenging and an open research question congratulations on finishing Section 2.

Thank you and see you in Section 3. Stay tuned.

LECTURE 9

[Coding Exercise] GAN Evaluation Metrics Inception Score

See the notebook `5.GAN-Evaluation-Metrics-Inception-Score.ipynb` for the code.

Hello and welcome to a new coding exercise. And this exercise we're going to learn a new evaluation metric for GAN models which is named the Inception score.

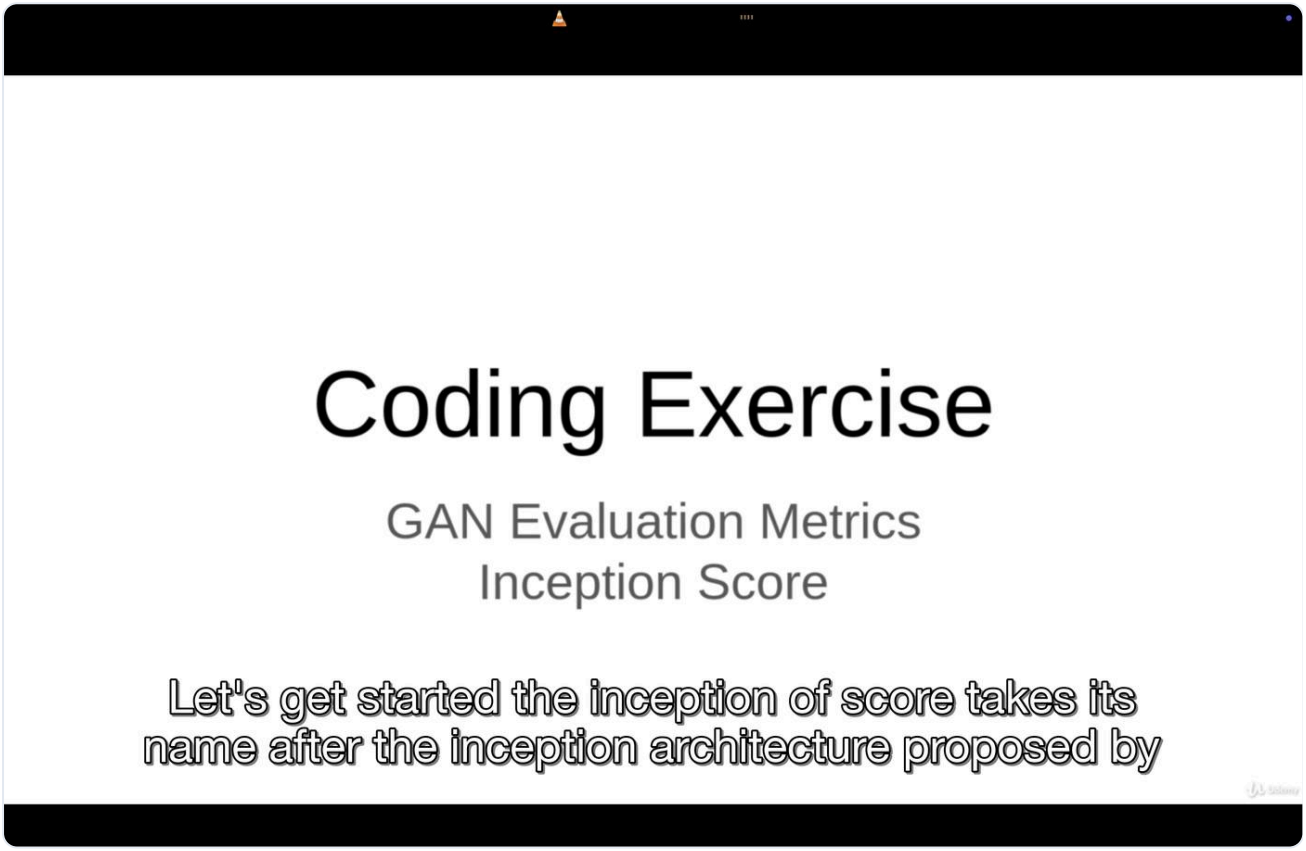


FIGURE 6.1 | Coding Exercise

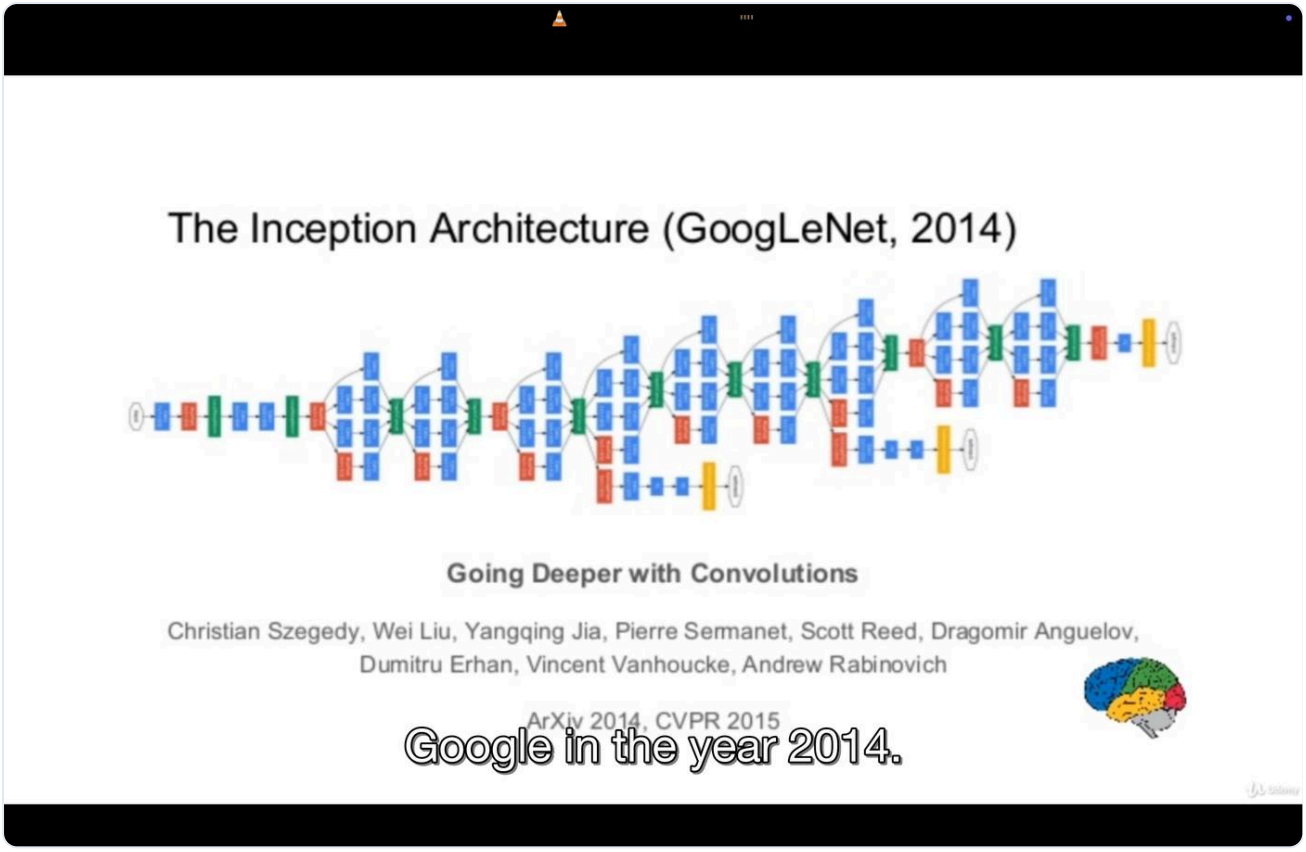


FIGURE 6.2

Let's get started the Inception score takes its name after the inception architecture proposed by Google in the year 2014. One of the pioneering architectures and convolutional neural networks that performed a high score on the ImageNet classification problem. The ImageNet has thousand classes and the network is designed so that it has thousand output neurons for each label of the classes in the data set.

The architecture of the Inception score allows for very deep networks by having auxiliary networks additional output layers that are put in the intermediate layers. This way they provide gradient flow in the shallow layers of the network meaning that because of the vanishing gradient problem inherent in very deep neural networks the designers of the inception architecture designed an auxiliary output that's a secondary output not actually essentially one for the output.

And this one is placed not at the end of the network but at intermediate places of the network. And then when the loss is calculated on the outputs of these intermediate layers there will be a gradient flow in the network that finds its way through to the very first layers of the network and hence it solves the vanishing gradient problem inherent in deep learning. The idea behind the Inception score that we are going to use a pre trained Inception network to evaluate the performance of the GAN model.

Let's see how this works. the Inception score aims at providing a metric for two values.

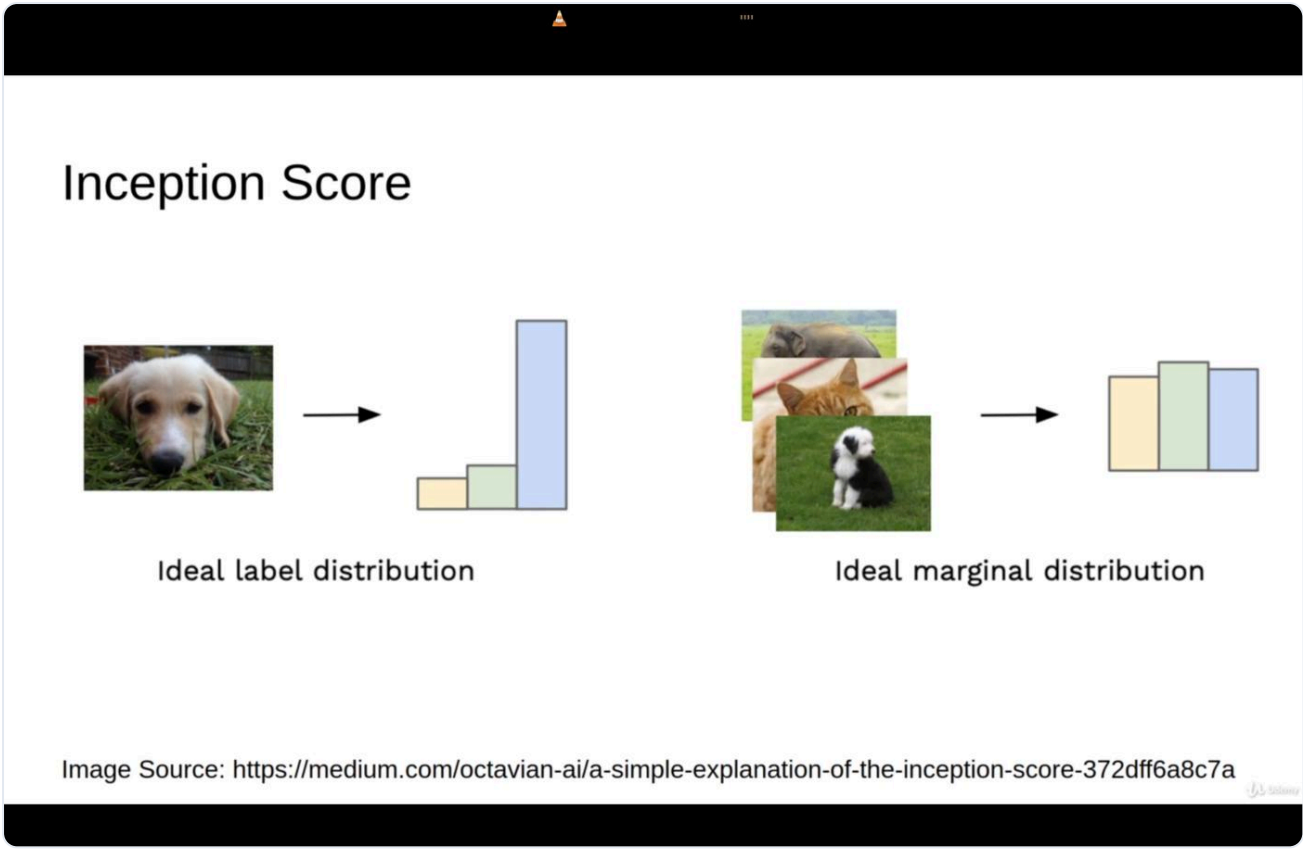


FIGURE 6.3 | Ideal Label Distribution Ideal Marginal Distribution

The first is whether the image is correctly classified by the Inception network. That means if we have a network GAN model that generates images of let's say canines we feed the image to the inception network which is pre trained on thousand classes of image it and these thousand classes they do actually include images of canines and cats and so on. So the output of the network will be a classification distribution over all of the thousand classes with a peak, a very high value a very high probability value at the label corresponding to the canine class by reporting this value what we get is that we have a measure an objective measure of the image quality that is very close very similar.

And this was proven by the authors of the Inception score paper. This is very close and very similar to the evaluation of a human being. That means if given the same image to a human and given the same image to an inception network and calculating the score the values whether the image is real or fake of a desired object a canine for example will be very close. So that helps us in the design and evaluation of generative adversarial networks because we need an objective way to assess the quality of generated images.

Remember we had to look every epoch what the images are generated or every ten epochs and then manually inspect the quality of the images. This is not feasible at larger scale and it cannot be very decisive in deciding the best model so there must be a mechanical a mathematical term that helps us better evaluate the performance of GANs on the other hand.

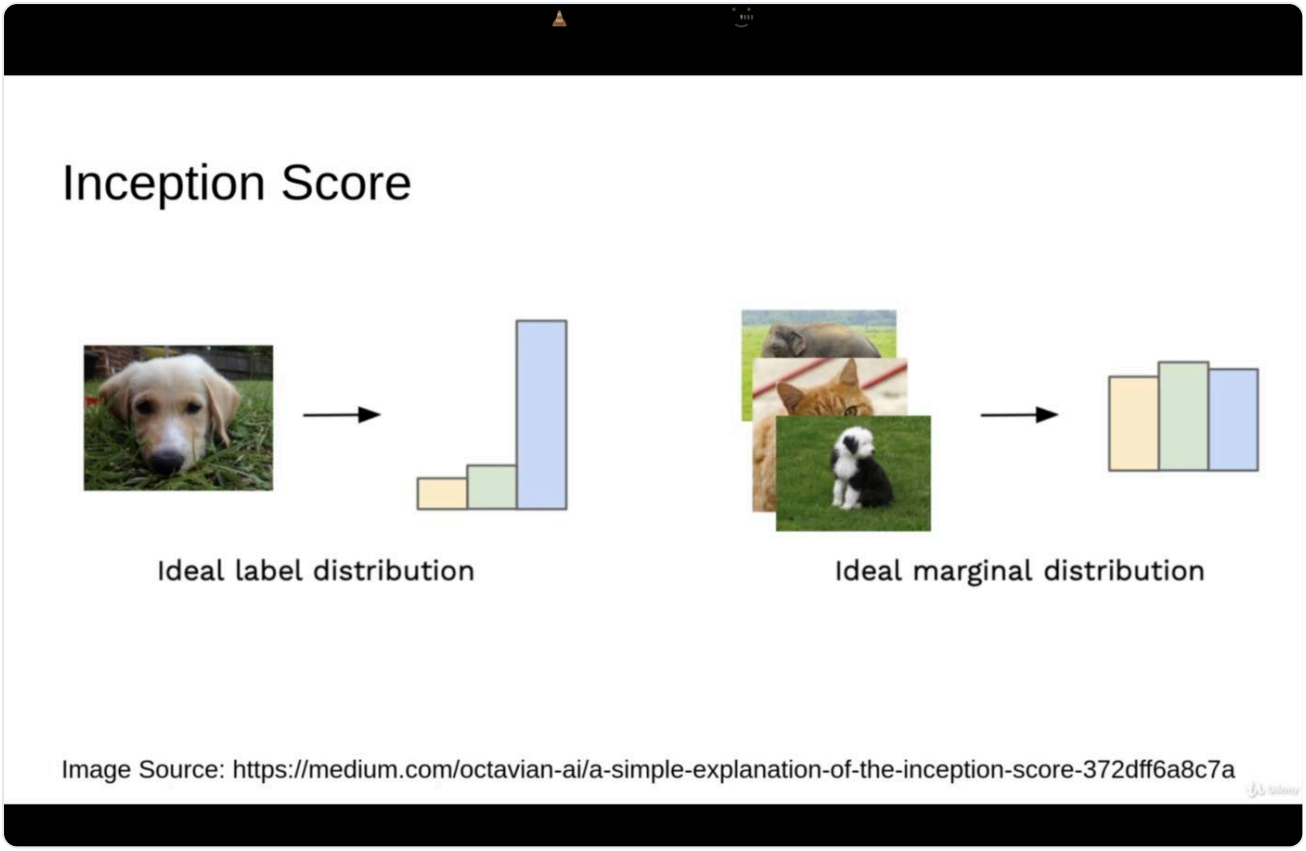


FIGURE 6.4 | Ideal Label Distribution Ideal Marginal Distribution

We wanted the model to output a variety of classes not necessarily as we done an MNIST number one or number two or number three but we want it to have a uniform distribution equal number of samples generated from each class.

For example if the image rates the number 1 2 9 or 0 to 9 then generate 10 numbers for 0 then images for one thing images for 2 and so on for each class in the dataset should have its equal proportion to the total output of the images. This is called the inter-class diversity and this is also one of the qualities that is measured by the Inception score. So as you can see here on the left side of the slide we have the kind of probability distribution that is given when we have a classifier and given an image and then we have a high probability at the label or the neuron corresponding to that class.

And this is how it looks like if we try to plot the output of the softmax layer and on the right side we have a uniform distribution. It means if we have three classes elephant cat and dog then we want each one of them to have the same number of generated images done by the GAN model. So having these two kinds of probabilities that gives us two probability distributions and if we recall we have a distance measure between probability distributions that is the Kullback-Leibler divergence which is a member of the f-divergence distance metric family in the probability theory we have already introduced one of these divergence distance metrics which is Cauchy-Schwarz distance in the first section and the following sections we will have a very deep comprehensive discussion of the Kullback-Leibler divergence but for the sake of understanding the Inception score let's put it simply this way the Kullback-Leibler divergence is a distance metric between two probability distributions the higher the difference between the two probabilities the higher the Kullback-Leibler divergence distance that more similar the two probability distri-

butions the lower the value of the Kullback-Leibler divergence distance so we can benefit from this kind of metric to actually maximize the inception score because the inception score will be at the end the mean or the average Kullback-Leibler divergence between the two distributions the distribution of the class labels given the generated images and then their average values over all the samples in the dataset or the generated dataset for all the classes so this way we have the conditional probability working on the horizontal axis and the probability of the class label on the vertical axis and then by giving them as to probability distributions to Kullback-Leibler divergence.

We make sure that we get a very high distance meaning that we always get this kind of distribution. The ideal label distribution when the image is classified correctly by the inception network and also we are getting a uniform distribution when the images are very diversified and different from each other for different labels.

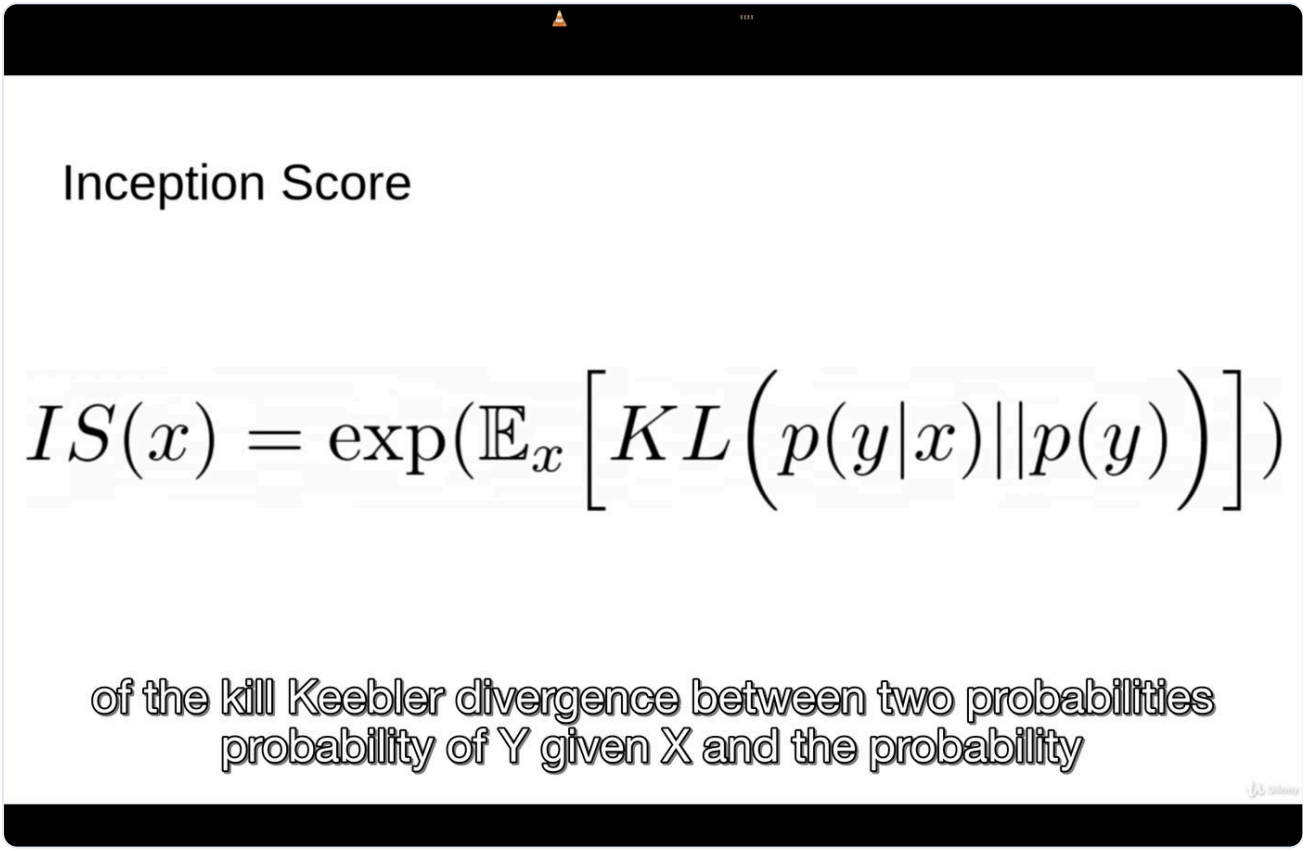


FIGURE 6.5 | Inception Score

So let's see how this is formulated in terms of mathematics so the mathematical formulation is the mean of the Kullback-Leibler divergence between two probabilities probability of Y given X and the probability of Y Y here stands for the label the class label and X stands for the generated images.

So what we are trying to calculate is actually a distance the further the distance between these two probabilities the better the Inception score the more close they are to each other the lower the Inception score and hence it is worse. Some caveats happen when using the Inception score.

For example our network our GAN model may generate many images for each class and they are all equally likely according to a uniform distribution. But for each class it will output the same image and will not output diversified images for the same class.

The inception score helps us to measure the inter-class diversity but it still fails on the intra-class diversity. This is one of the open questions. If you happen to pass by any of the research papers addressing this problem please pass it to me and share it with your colleagues. In this course on the course forum. Otherwise it may be a good motivation for you to try to come up with a formulation that helps also to measure at the same time the inter-class diversity so let's get to coding.

I will use the same coding exercise we have used in Section 2 and then I will add the Inception score function at the end of the file you will find a file with this Inception score implementation with the name of the same lecture in the course materials. So what we do now is to scroll down at the end of the notebook. So you'll find a section with the name evaluation metrics Inception score and here you find actually the source where I have got this PyTorch implementation so we started by using a new object of the class Inception v3 which is a pre trained model for the Inception network and we call it a variable net.

And then we define the inception score function or subroutine that will calculate the Inception score given some images. And please note here that the images according to the paper you pass all the images in the dataset or you pass groups of images each one has 50000 images. So the Inception score doesn't work by applying it on only a few number of images but applied on a large number of set of images or groups of images. Each one has also a large number of images and then you average the score over these groups so we initialize an array scores that will hold the values of the inception score calculated on every batch of the mini-batches available in the data set of images we can create the number of steps how many times we have to iterate over the dataset giving the batch size and then we loop over the data set and then we divide the images into batches each one will have a start index and end index so that between the start and the end index we have a batch size let's say sixteen for example and then we pass the mini batch of images to the Variable method of the PyTorch autograd so that it can be a variable with gradients we pass it to the network and then Inception network returns to values the actual output from a linear layer and the output from the auxiliary layers in the intermediate layers of the network we append that to the scores and we convert the scores to a PyTorch tensor using a concatenation by taking the list of scores together as tensors remember at this point that each image will have a tensor or a vector of thousand values because this pretrained model is trained on ImageNet thousand classes thousand labels so for each label we have one output that gives us thousand neurons, thousand outputs and over these thousand neurons we have a probability distribution just like the one we have seen in the image on the slides and then we started the actual calculation of the conditional probability and the marginal probability and best them to the Kullback-Leibler divergence we need to best the output from the inception network to the softmax because it doesn't have a softmax layer at the output by default it has a linear output and then to calculate the conditional probability we just take the output along the first dimension that is dimension indexed by number 1 this is the second dimension this will give us the softmax probability distribution over all of the classes for all the images in the dataset and we'll call that probability of Y given X and then we do the marginal probability or the probability over the same class.

But for all the images and the data is. For example if the first class is the cat we want to sum and average all of the probability outputs for all the images in the same class. And we do the same for the second class and the third class. And then you can see for the conditional probability probability Y given X we use the softmax along the dimension one. But for the marginal probability probability y we use the same value but we do the averaging with the index zero for the dimension.

That means over all the images in the dataset not all the columns. And then we calculate the Kullback-Leibler divergence according to the formula conditional probability multiplied by the log of conditional probability minus the log of the marginal probability. And then we take the average Kullback-Leibler divergence distance and return it and the final score over here. I try to test the Inception score on only one batch generated by the generator trained in the previous lectures and it needs some preprocessing because first of all it has only one grayscale channel.

We need to convert that to RGB and it also size of twenty eight by 28. We need to resample this one to the size of 299 by two hundred ninety nine so that it is compatible with the design of the inception network and then we pass the resized images and the batch size to the inception score. We return a tensor variable we use score.item to get the actual value inside the tensor. You can see here we are getting a very low value for Inception score point 0 0 0 3.

This is very low value. First of all because we are training on this data set which is a data set that the Inception network is not trained on. That means it has different labels different categories from the ones available in the ImageNet dataset. That's one of the caveats. The other thing that we are calculating in the Inception score over only one mini batch you can see how computationally demanding it get. So when we tried to calculate Inception score over all the mini-batches in the data set and we do that every epoch.

So it is a very demanding computationally expensive process but it is a must know a tool in your toolbox as a machine learning and deep learning engineer especially when working with GANs because you cannot always use the subjective way to evaluate the performance of your networks. You need to have an objective metric and then if you scroll down you find that I try the same experiment over a batch of real images of the original dataset. Again a mini batch of 16 images and we do the same and we get a little bit higher.

Inception score point 0 0 0 7 which is even though it is higher than the ones we got for the generated images but it's still very low for the same reasons we have just explained. I hope that give your idea about the complexity of evaluating GAN models and about the novelty and creativity in this area. Well you have to find new metrics to evaluate against models. It's an open area of research. I encourage you to read more about this one and here is a reference to the original paper where the Inception score was first introduced.

Thank you and see you in the next lecture.

LECTURE 10

[Coding Exercise] GAN Evaluation Metrics FID Score

See the notebook `6.GAN-Evaluation-Metrics-FID-Score.ipynb` for the code.

Hello and welcome. And this coding exercise.

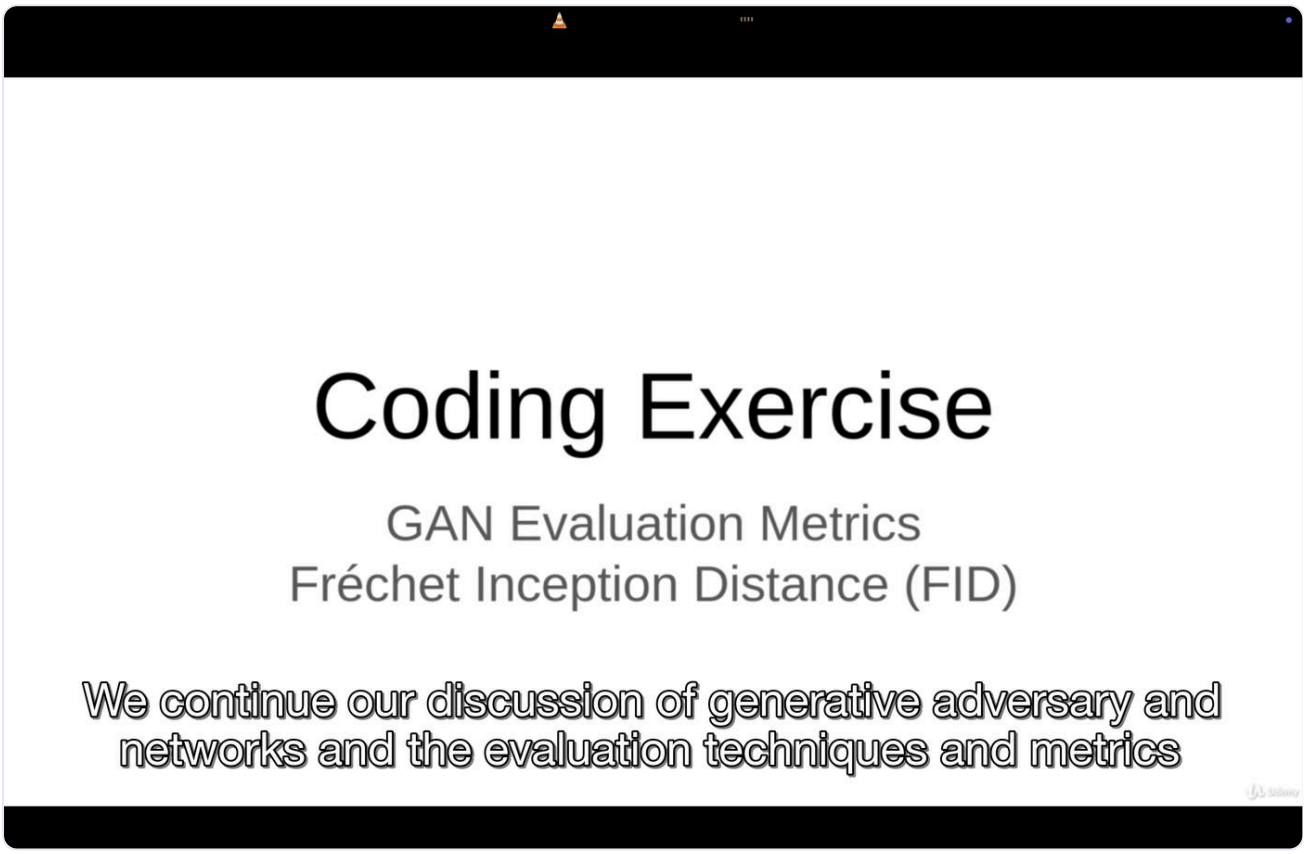


FIGURE 7.1 | Coding Exercise

We continue our discussion of generative adversarial networks and the evaluation techniques and metrics and this exercise to use the Fréchet Inception Distance or FID score. Let's get started Fréchet distance is a distance metric that is best fit for curves and polygons and it is perfect for pattern matching and shape matching it doesn't require the two curves to be identical it allows them to be within a given approximation range from each other.

As an example of this if you are going to walk your dog outside you have a leash and the dog is on the leash. It is allowed to be on the same path with you but it can also go on its own path as long as the leash allows for it the shortest leash that is going to allow you and your dog to walk on the same path on the same curve without diverging from each other is the minimum Fréchet distance. This is the intuition behind the Fréchet distance the real intuition behind using it for generative adversarial networks and their evaluation is that even though they we have studied the Inception score in the previous lecture it is found to be inferior to the Fréchet Inception Distance because it doesn't take into account the statistics of both the real image and the generated image.

When I talk about the statistics I mean the average the mean and the variance which define almost any statistical Gaussian distribution Fréchet distance or the Fréchet Inception Distance is designed to allow to study the mean and the variance of both generated images and the real images using the Inception network unlike the Inception score which uses the final output of the inception network in the Fréchet Inception Distance. We are going to use the last layer or the second to last layer before the last output in the Inception network.

That means we are using the inception network as a feature extractor. A very common concept in deep learning is called representation learning. It was found that in deep architectures such as AlexNet, VGG and ResNet and Inception network some particular layers are valuable as feature extractors because all of the convolutional layers in the network they learn the different features such as curves corners edges and so on of the input image and then they keep on accumulating this knowledge in a hierarchical topology from the lower layers up to the higher layers in the network till we arrive at a flattened vector which is a valid feature representation that maximizes the inter-class difference and minimizes the intra-class difference meaning that images for the same object will have similar features and images from different objects will be very far from each other according to this feature representation.

You can think of it as similar to PCA or principal component analysis for feature reduction. where you try to project the input features to different coordinates that try to maximize the difference between the features and allow us to have linear decision boundaries between the classes. So the idea of Fréchet Inception Distance that we are going to use the statistics of the mean and the variance of the features learned by an inception network on both the real images and generated images and then we calculate the Fréchet distance between this is that using these statistics let's put it into a mathematical formula.

Here we have the mean mu R which is the mean of the real images and mu of G, the mean of the generated images. Of course here I mean the mean of the feature representation of these images. We subtract them and then we have the square of the mu R and mu G difference and then we have a new operator TR which is the trace operator. It allows us to take the sum along the diagonal of a matrix and since this operator is applied to the variance the variance here is taken for the representation of the real images and the generated images.

So because we know that the variance matrix is the aggregate we take the sum along the diagonal. Then we take the product. Sigma are multiplied by Sigma G and then we take the square root multiply by 2 and subtract from Sigma R plus Sigma G A very simple equation.

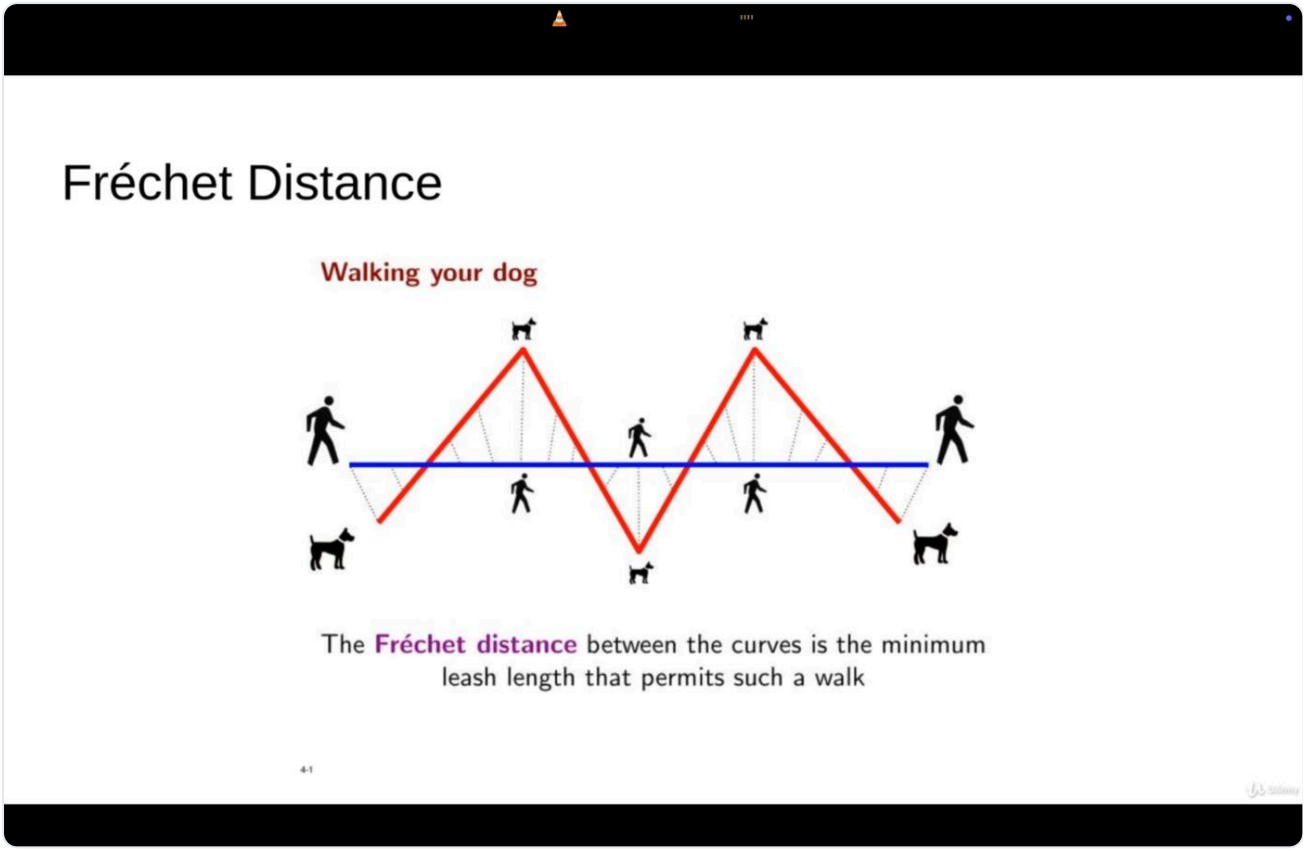


FIGURE 7.2

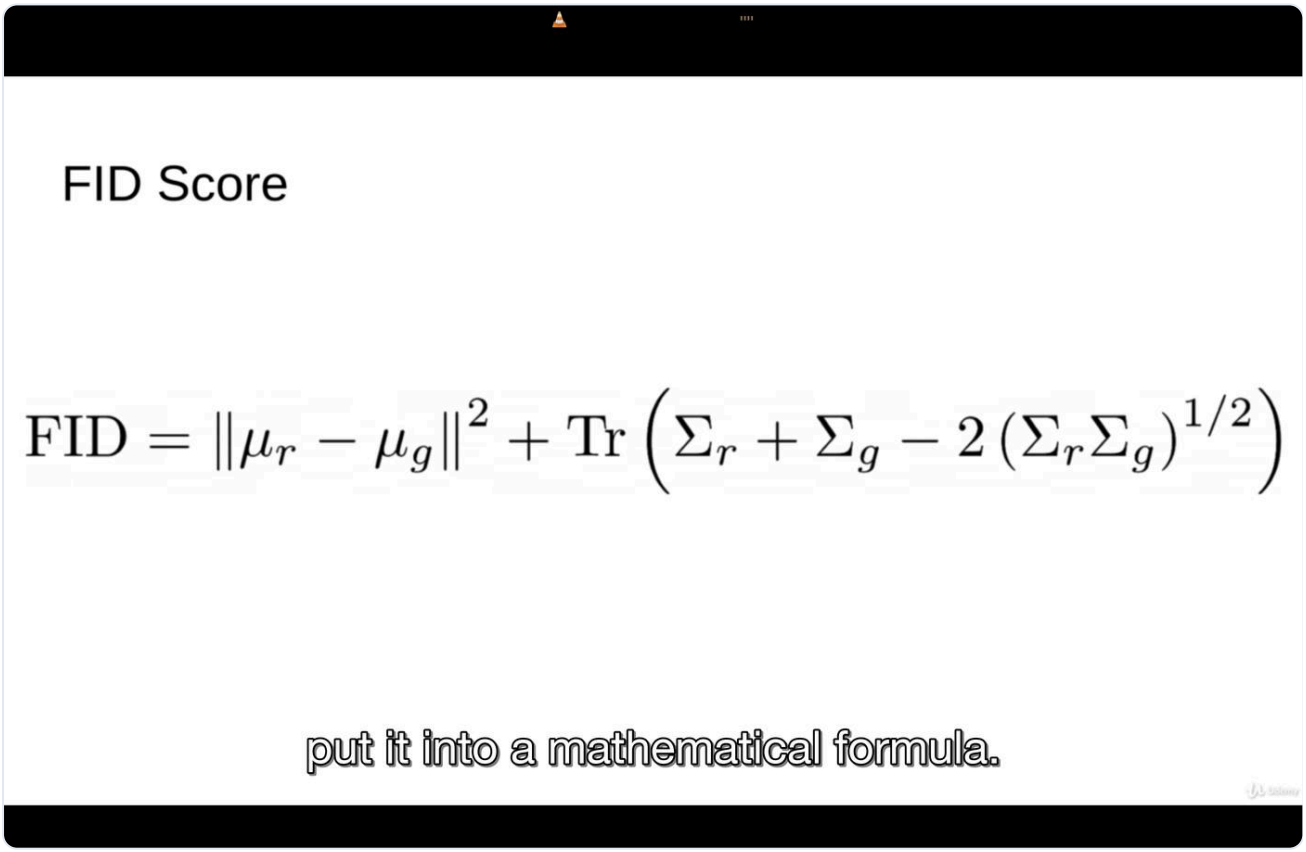


FIGURE 7.3

It gives us a scalar value at the end which is a distance the Fréchet distance between the generated images and the representation and the real images and I mean the representations using the mean and the variance of the feature representation learned by the inception network.

Let's see how this can be applied using Python. You can find the notebook associated with the section materials and it has the title evaluation metrics. FID score and of course is the source where the source code is being benchmarked on so we need a mechanism by which we take the output of the second to last layer in the inception network the code over here defines a new partial Inception network that inherits from torch.nn.Module and then it instantiates the pretrained Inception v3 network.

And it defines the layer with the title Mixed_7c which is actually the output from the last block of the inception network. Just before the final layer or the output layer and then it defines a hook because by default and I went into this error a loop that allows PyTorch internals to give you the output of this internal block directly when you pass the inputs through the forward method it is a syntactic sugar so that you can apply the output on an intermediate layer of the network then inside the forward path we pass the inputs through the inception network and then we get all the outputs we extract the output we are interested in which itself Mixed_7c output and put it in a variable activations and this output will have the shape of the number of batches multiplied by 2048 channels by eight by eight.

And then we need to reduce that to one by one so that we have only 2048 which is actually an ideal representation vector for any image that passes through the network. You find similar concept in AlexNet and VGG-Net and ResNet and so on. So in order to reduce the dimension we use average pooling 2D we pass the activations through the average pooling of kernel size one by one and then we view the output vector the output matrix or tensor using activations.view with the first parameter saved for the batch size and then 2048 which is the number of features in each sample or each image then we return the outputs and start by creating a new object of this partial Inception network and define a method.

FID score which would perform the actual calculations of the Fréchet Inception Distance so unlike the inception score we did in the previous lecture in this one we have both the real images and the generated images passed to the FID score method we accumulate the results or the representations of both the real and rated images in two lists, real_activations and generated_activations we loop over the number of steps in that dataset given for simplicity the number of steps here will be 1 because I'm passing at the later step only 1 mini batch but I encourage you to try it with the larger samples and see if it is computationally feasible because technically what I found through these two exercises is that the evaluation metrics whether it is Inception score FID score they are both computationally demanding and require a lot of computing resources and taking into consideration that you have to do these evaluations after every epoch or every 10 epochs of training the network which is itself is two networks to the generator and the discriminator.

So it is a bit of a headache to deal with every exercise you are going to do it again but it is a necessary evil that you must work with because it gives you an objective evaluation metric for GANs unlike the subjective manual inspection of every image that is generated by the network we loop

over the number of steps we define the boundaries for the mini batch at the start and the end index. We slice the arrays the real images and the generated images by these indices we pass them to Variable to make them into PyTorch tensors with gradients then we pass them through the network and will have the feature vectors we are interested in.

We append them to the real activations and the generated activations list. Now we are due to calculate the statistics to the mean and the variance over these features vectors. What we do is to stack the list into a PyTorch tensor and then detach it from the gradient because it has gradients as a variable or a PyTorch variable. So we need to detach it using the method detach and then convert it to CPU if we have been using GPU and convert it to NumPy so that we can perform the calculations we are using the NumPy API for mean and variance and trace.

Operator it becomes a straightforward from here applying the formula in the previous slides directly in terms of NumPy using numpy.mean for the average, numpy.cov for the Sigma or the variance. Note here that rowvar equals False means that the rows will be on the horizontal side and the observations will be on the vertical side. Again you wrap your head around it because if you do rowvar equals True that will have the opposite the transpose of The Matrix.

Just give it a couple of tries to make sure that you are comfortable with this kind of visualization of the matrices in your head and then we subtract mu two from mu one which is the mean from each other and we do the squaring by doing the dot product of the diff twice because it is a matrix and squaring a matrix is not simply as just the squaring every element in the matrix but you apply that product to the Matrix so we can get a squared matrix then you apply the dot product to the variance 1 and 2 and then you apply the square root using the scipy linalg sqrtm.

This way you will have the square root applied correctly to the matrix and disp equals False will allow you to return the errors if any and I have ignored it using underscore and sometimes a square root. In this case will be a complex number with imaginary part we can get rid of it if it exists and we do that using sqrt_product.real which returns the real part of the imaginary object and then we apply the trace operator on the square root and score product.

And finally we calculate that FID score using the terms we have just calculated in the previous steps so using the term as we have calculated in the previous steps we apply the numpy trace operator on the Sigma 1, the variance of the real images. Now the trace on Sigma 2, the variance of the generated images minus two multiplied by the product trace which we have just calculated and we sum the difference and return the overall FID score. Next I need to test this method on some real and it generated images from our model so we get a batch from the train loader convert it to the device, CPU or GPU and then reshape it to twenty eight by twenty eight.

Repeat the RGB channels and upsample the image 299 by 299 and do the same for the generated images by applying the generator on the fixed noise vector. Do the same steps and finally pass the real images and the generated images to. FID score method and get the score printed here we've got 140. Please note here unlike the Inception score in the previous lecture where a small value meant a bad generator model and the higher value meant a better model.

In this case though FID score a small value near zero is the best larger value is bad. So they are opposite to each other. Don't be confused by that. Also one more thing the inception for a shared Inception distance. If I did is more sensitive to variance it means I would encourage you to have a better evaluation whether the images have high variety of classes and objects in your dataset. So it is actually even as recommended on Twitter by the inventor of the generative adversarial networks.

Ian Goodfellow. He also thinks and believes that FID is superior to the Inception score which he has contributed to the authoring of the paper of it himself. Again if you are interested in reading more about this topic you will find the original paper as referenced on the screen appendix A1 is the one concerned with the Fréchet Inception Distance. FID because the rest of the paper discusses a new technique for training generative adversarial networks and FID score is only a secondary contribution of the paper so cut it short and get to appendix 1.

If you are specifically looking for the FID you can also read the rest of the paper to help you understand better the other ideas and methods existing in the area of GANs. I would like to share with you for inspiration. A new paper published using deep reinforcement learning and generative adversarial networks together to teach an unsupervised way a the computers to draw and paint from real images or portrait images. And it is done completely unsupervised.

And if you scroll down here you can choose any of the images in this dataset. I would show you how it arrives at the painting and learning how to do doodles in a nice interesting way. So this is an inspiring idea inspiring topic where it merges both reinforcement learning and generative adversarial networks to promote the unsupervised learning of such skills and it can be applied to robotics that can be applied to game rendering it can be applied to many potential ideas and the paper I believe is done by DeepMind which is part of Google Alphabet.

Thought that was inspiring thank you and see you in the next lecture.

LECTURE 11

Introduction to Conditional GANs

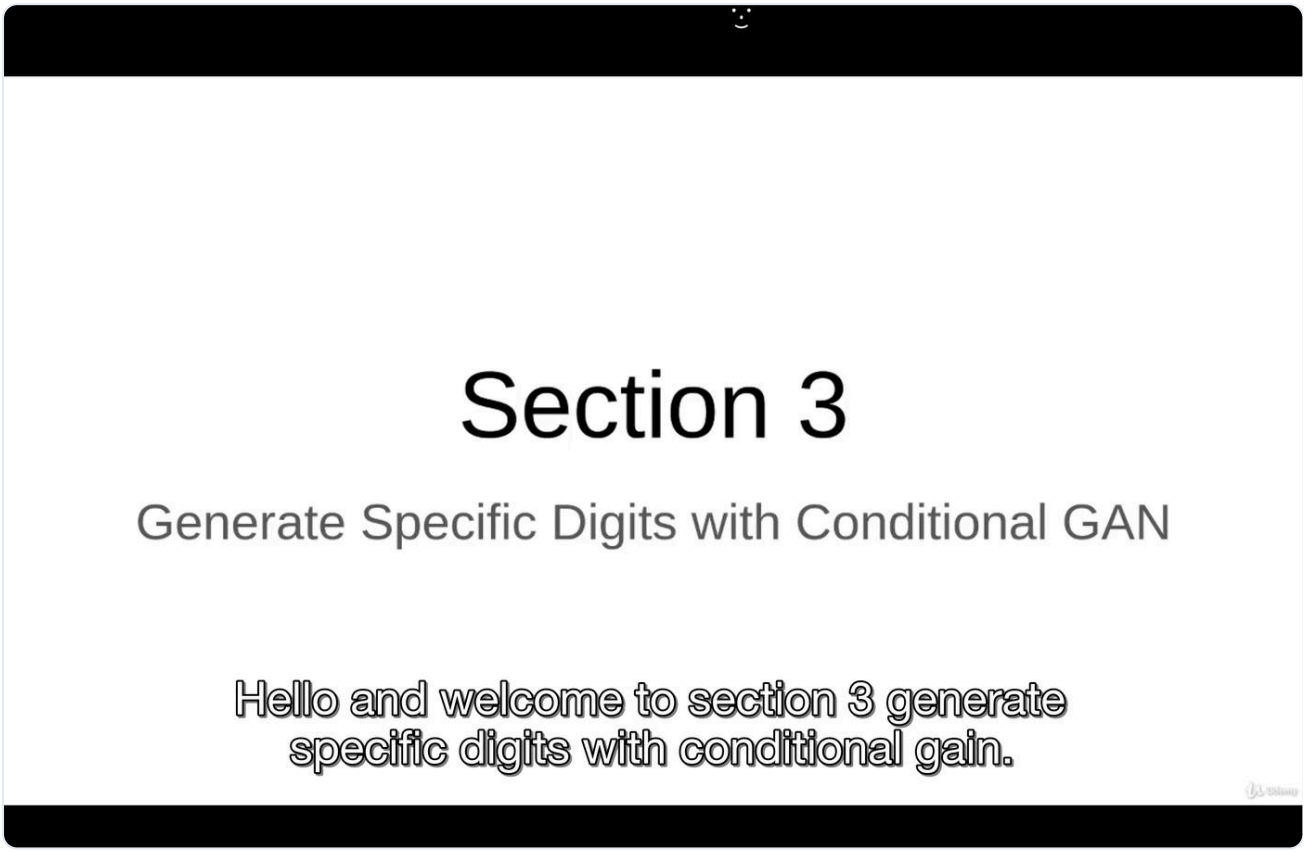


FIGURE 8.1 | Generate Specific Digits with Conditional GAN

Hello and welcome to section 3 generate specific digits with conditional GAN. In this section you will learn and implement a conditional GAN and the Wasserstein GAN. Let's get started.

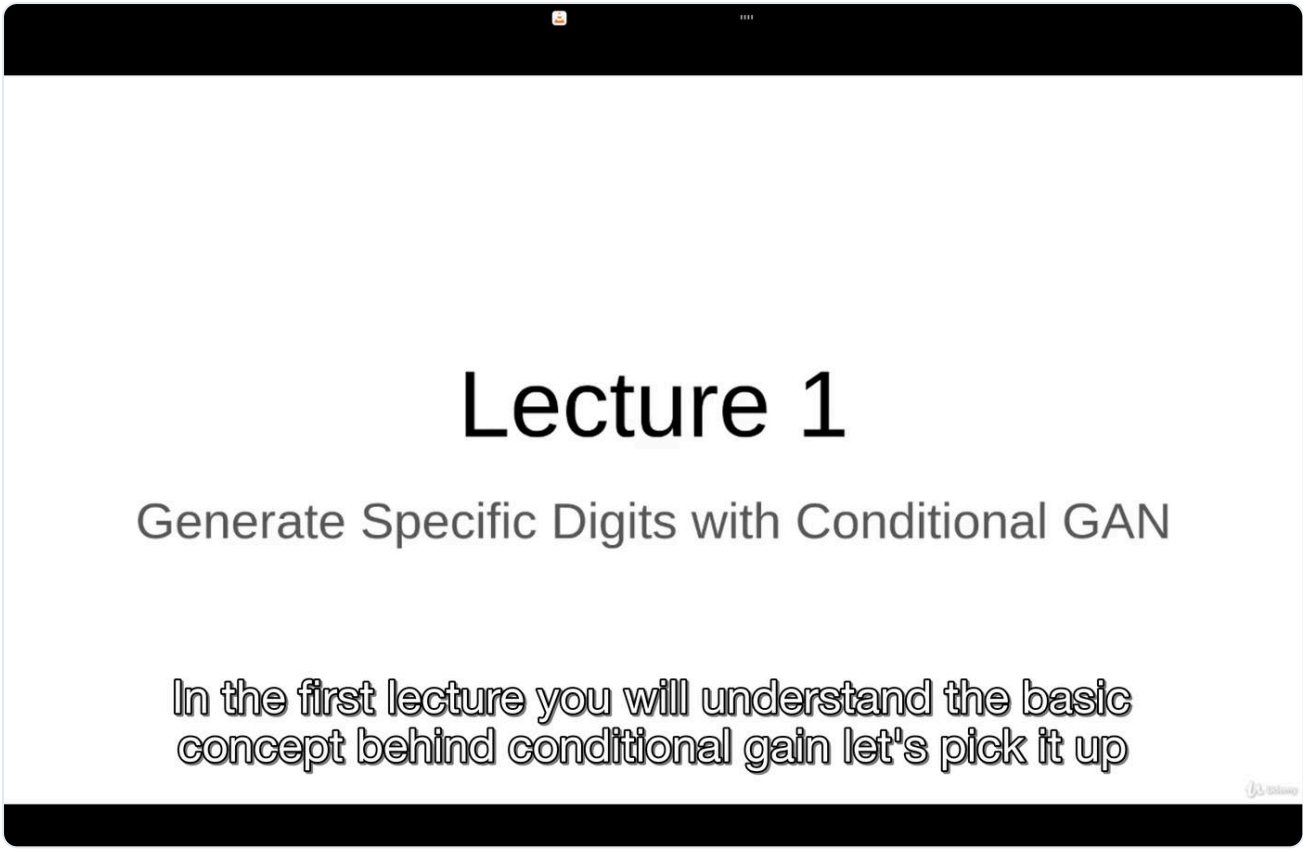


FIGURE 8.2 | Generate Specific Digits with Conditional GAN

In the first lecture you will understand the basic concept behind conditional GAN let's pick it up from where we ended in Section 2. The basic GAN model produced only images of the same digit. In this case it was number one. It turns out that this is a very common problem in GANs.

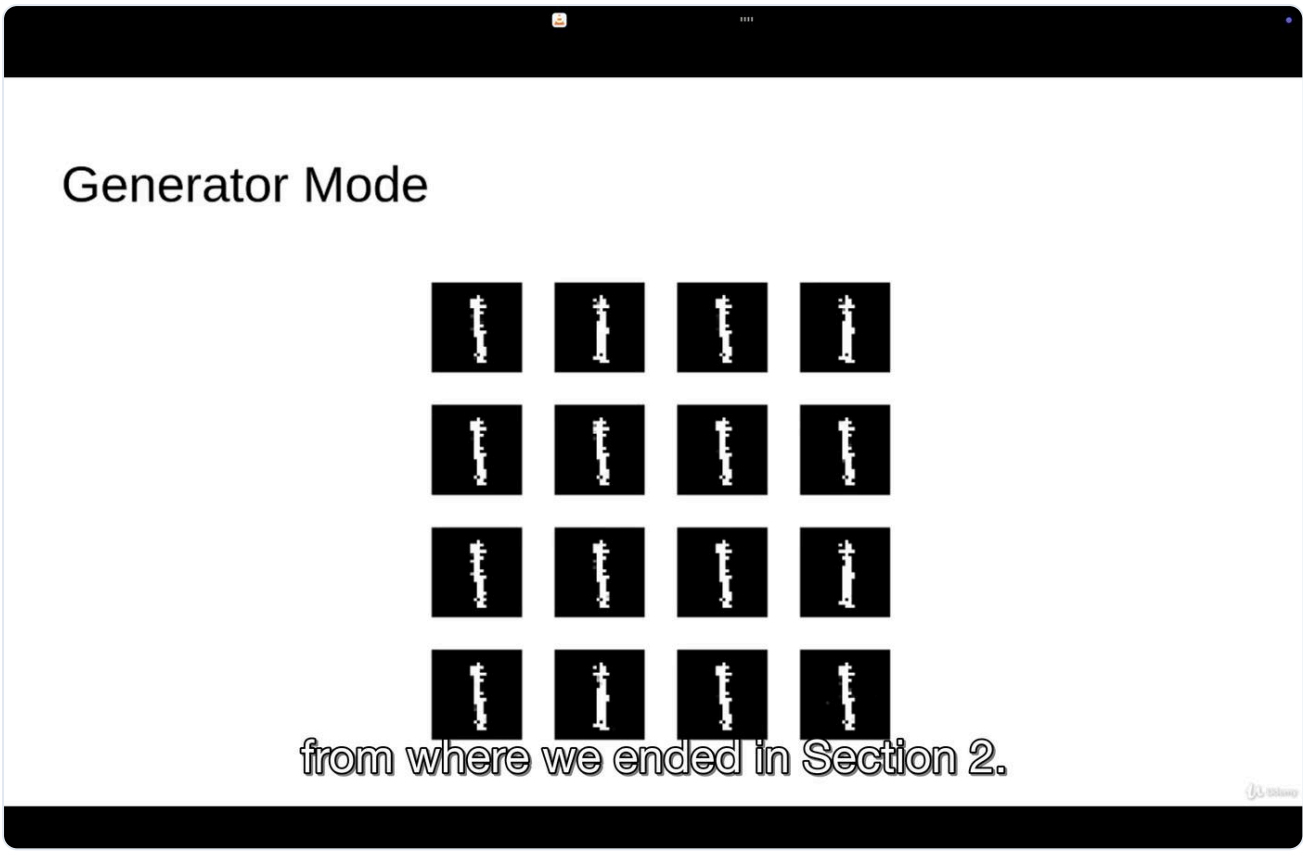


FIGURE 8.3 | Generator Mode

It goes by the name mode collapse the generator tricks the discriminator by generating the same image that was classified as real. In this way we don't obtain the desired outputs as planned however as a reminder using the base GAN model you can generate much better results by fine tuning the hyperparameters. I was interested in illustrating the problem of mode collapse and I wanted to highlight the concepts behind GANs and conditional GANs for the sake of completeness.

Let's first introduce conditional probability since it is essential for understanding conditional GANs from probability theory.

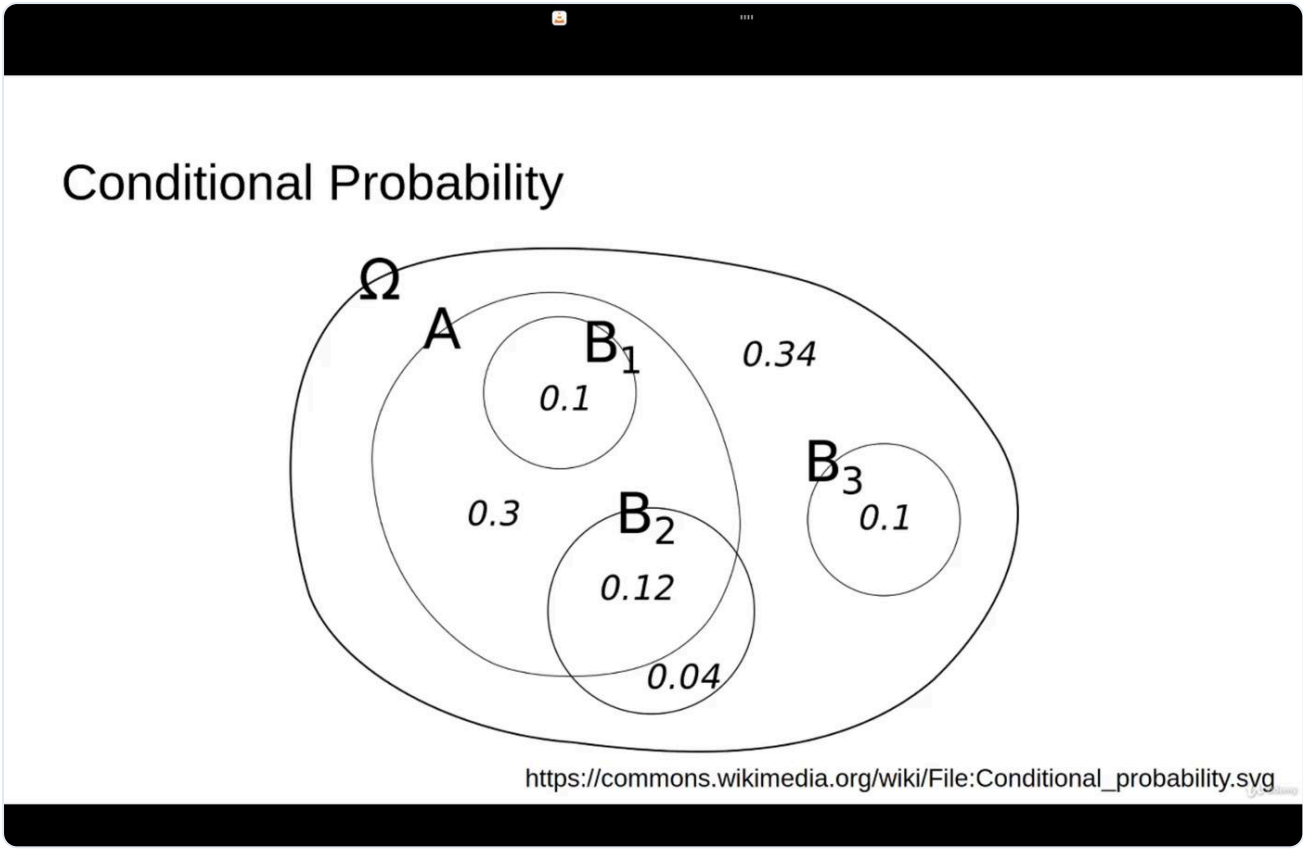


FIGURE 8.4

We know that conditional probability is the probability that an event A happens. Given the fact that event B has happened it is best to visualize that using Venn diagrams as on the screen right now. In this example the probability of the event A given B 1 is point 1 since event A is a superset of event B one the probability of the event A given B 2 is point twelve divided by point sixteen which is the intersection of both events A and B 2 and divided by the probability of B two.

In this case the point twelve plus point 0 4 which is point sixteen Lastly, the probability of event A given B 3 is zero since there is no intersection in the basic GANs which are implemented in Section 2.

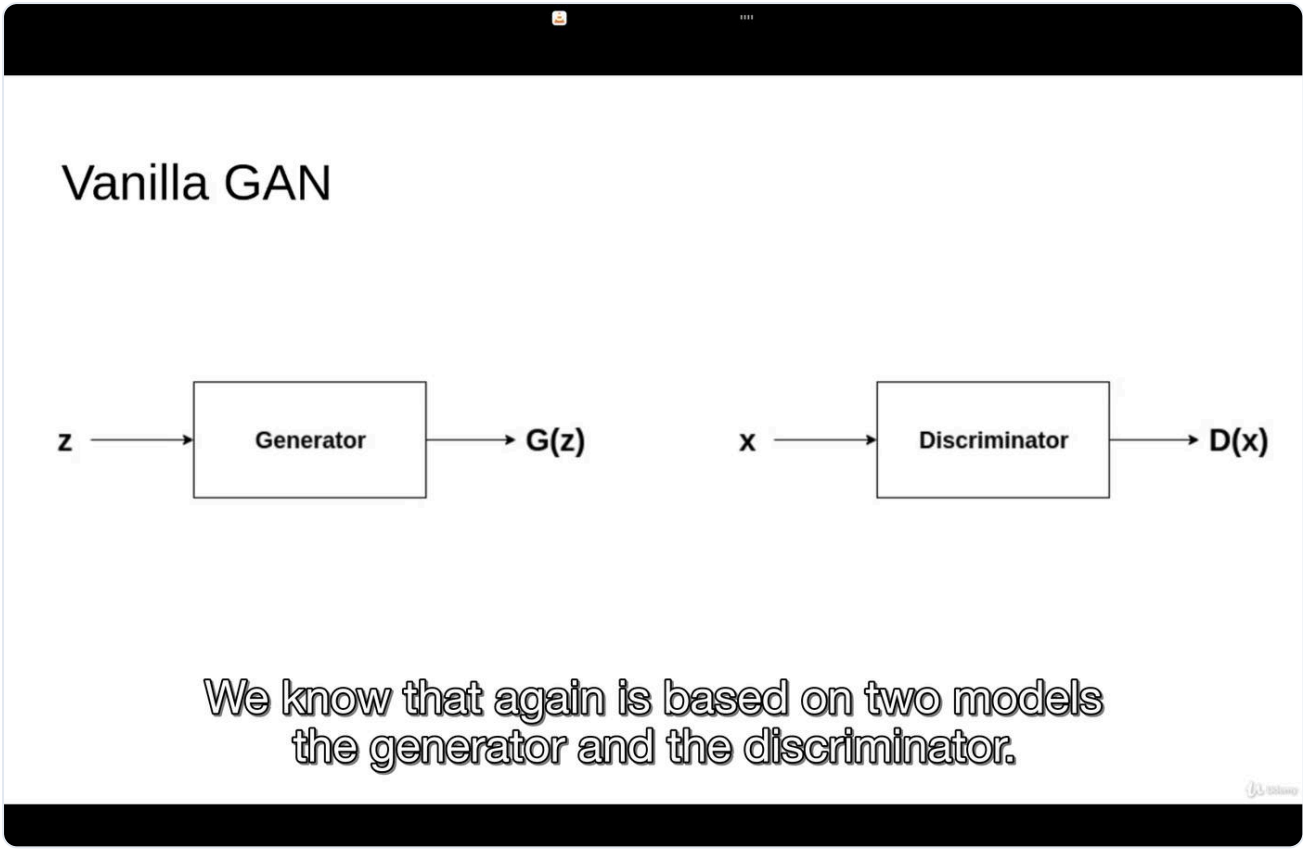


FIGURE 8.5

We know that a GAN is based on two models the generator and the discriminator. The generator is given random noise drawn from a known distribution then g the generator learns a mapping to produce samples drawn from the training set distribution on the other hand the discriminator is given a sample either from the training set denoted as X or generated by the generator and is denoted as G of Z.

Then it decides whether it is real or fake.

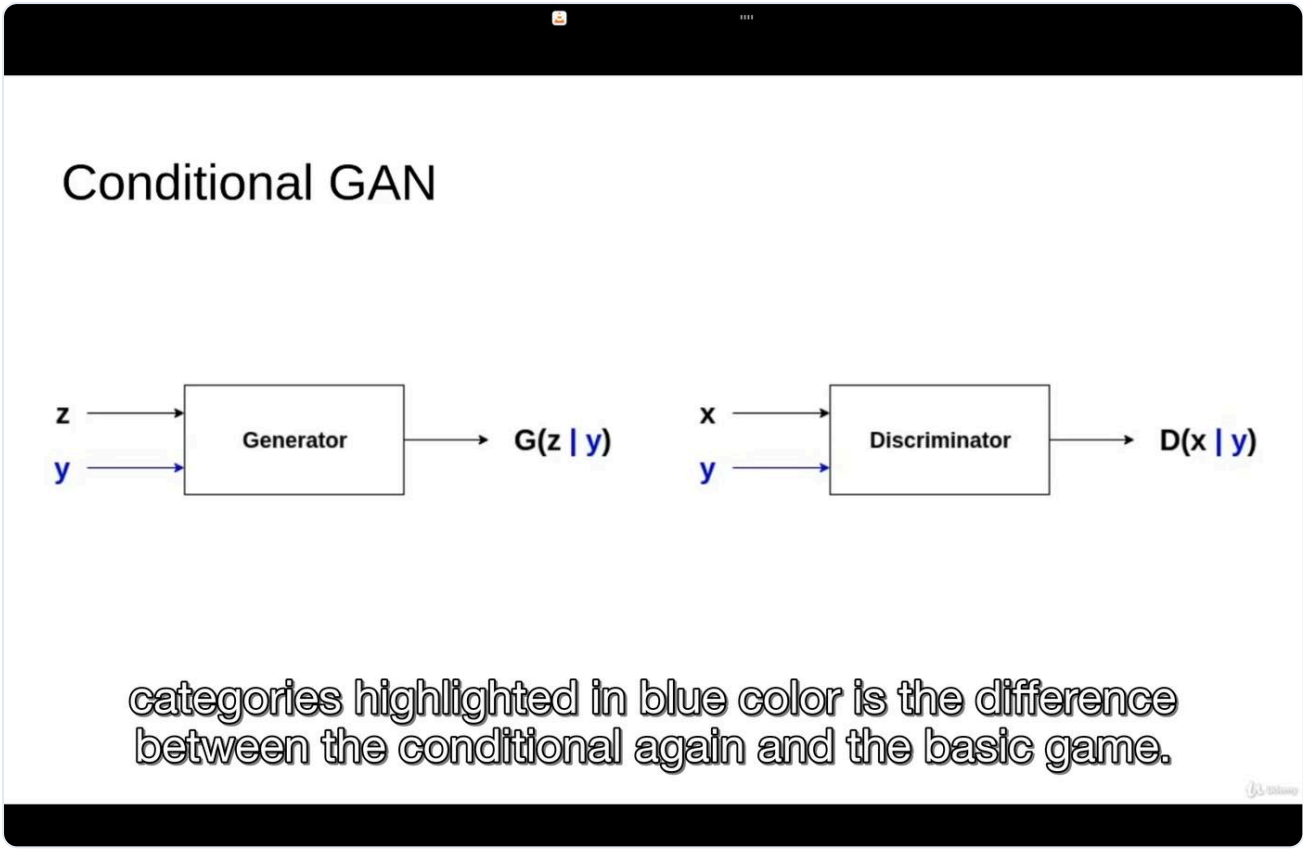


FIGURE 8.6

Now let's see how the conditional GAN tweaks this paradigm in order to produce images from different categories highlighted in blue color is the difference between the conditional GAN and the basic GAN. You introduce a condition denoted as Y the generator is given both random noise as before and a condition it is in the case of MNIST the class label. Between 0 and 9 then the noise is concatenated with the condition and is passed through the model.

The model learns the mapping as before and outputs a sample drawn from the conditional distribution X given Y.

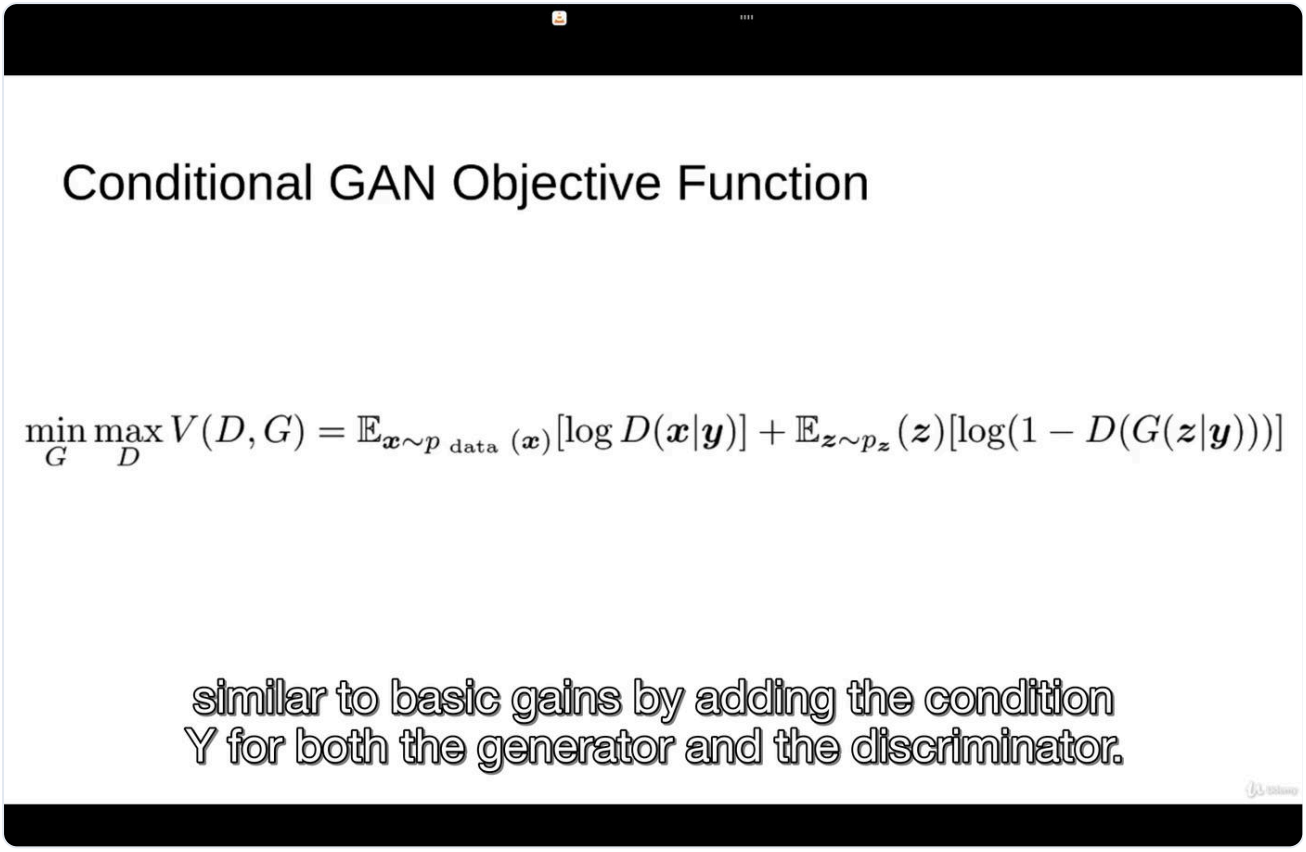


FIGURE 8.7 | Conditional GAN Objective Function

The same applies to the discriminator which is given the condition Y and outputs a decision D of X given y the idea is very intuitive and works like a charm as an exercise for you you can try passing the condition Y only to the generator and see if it still works the min-max game can now be reformulated similar to basic GANs by adding the condition Y for both the generator and the discriminator.

The discriminator loss on real images becomes the expected of log of D of X given Y and the loss on the generated images becomes the expected of log of 1 minus D of G of Z given y in the next section.

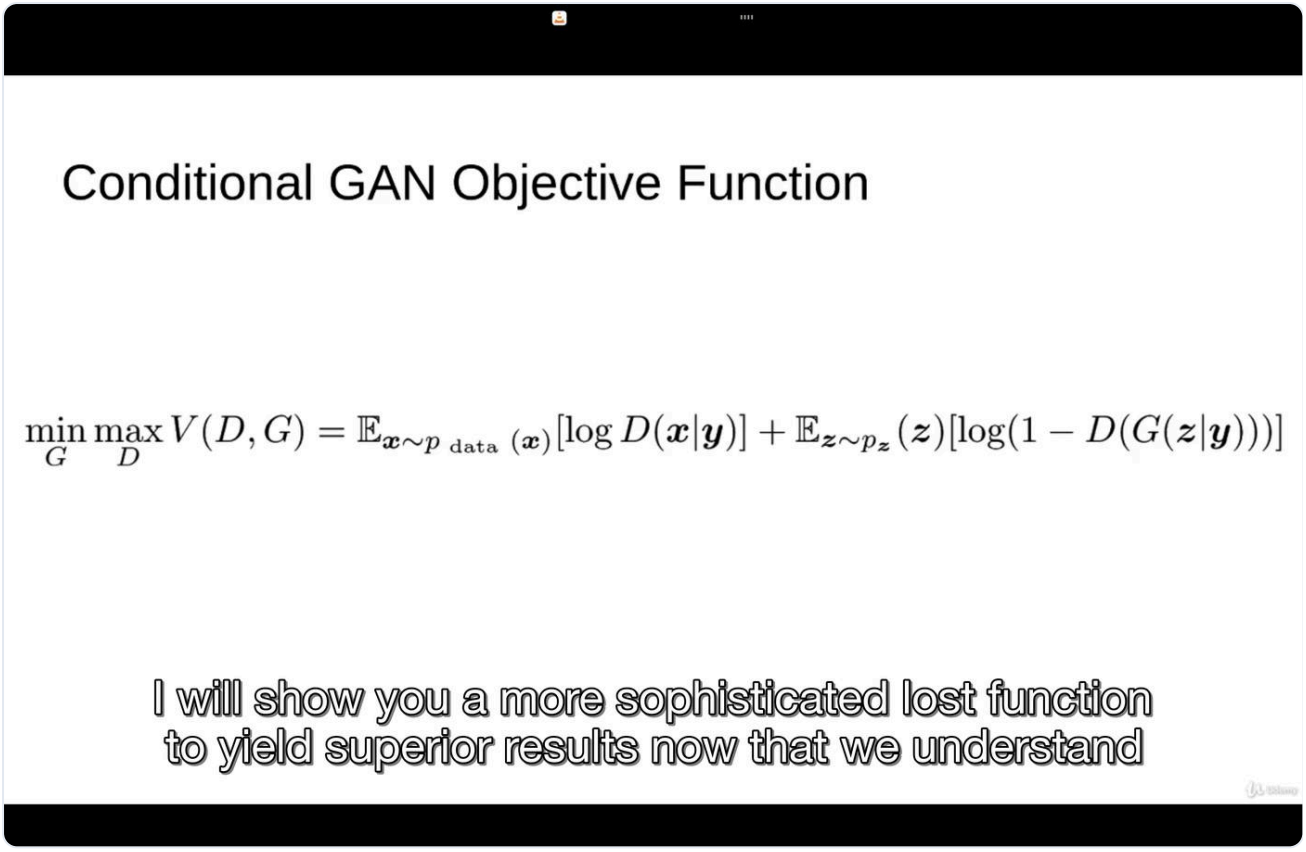


FIGURE 8.8 | Conditional GAN Objective Function

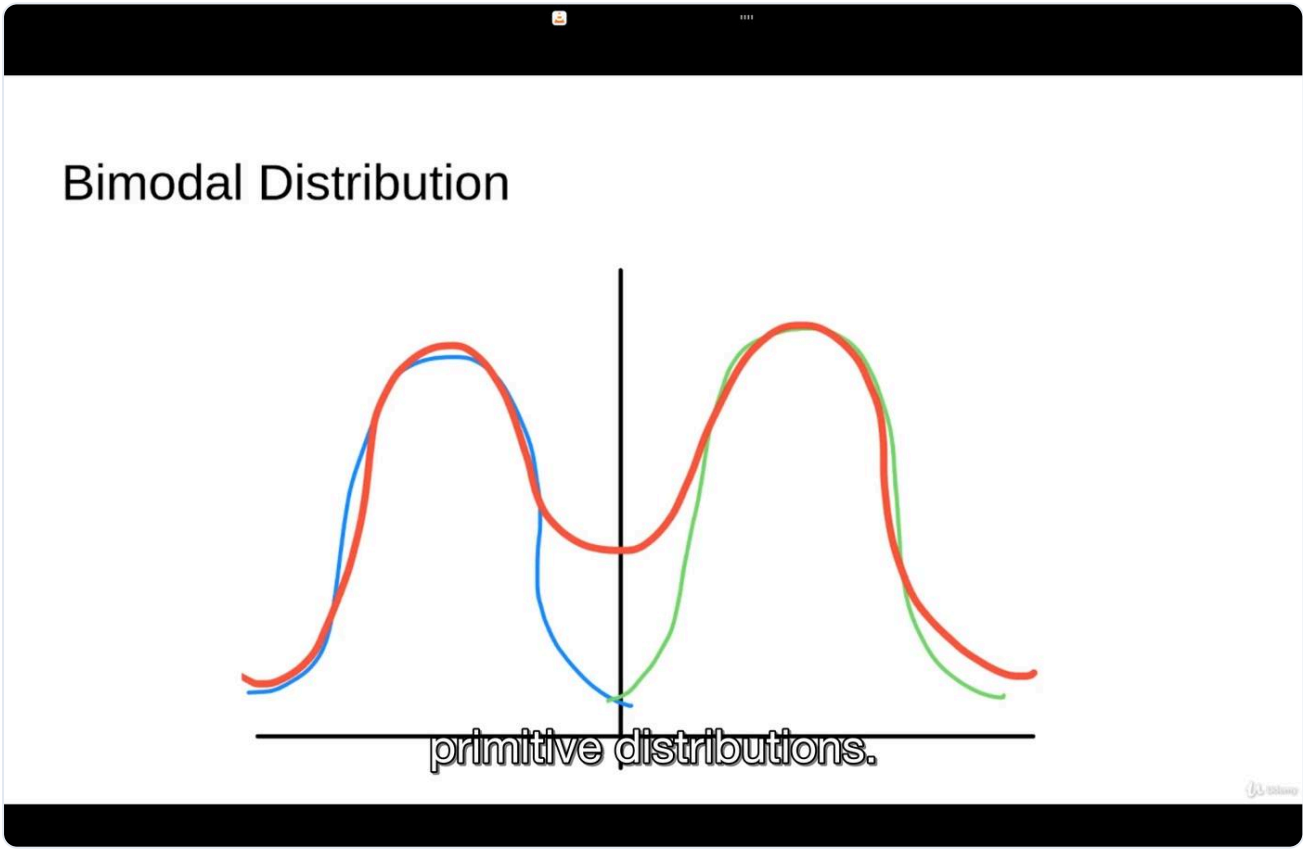


FIGURE 8.9 | Bimodal Distribution

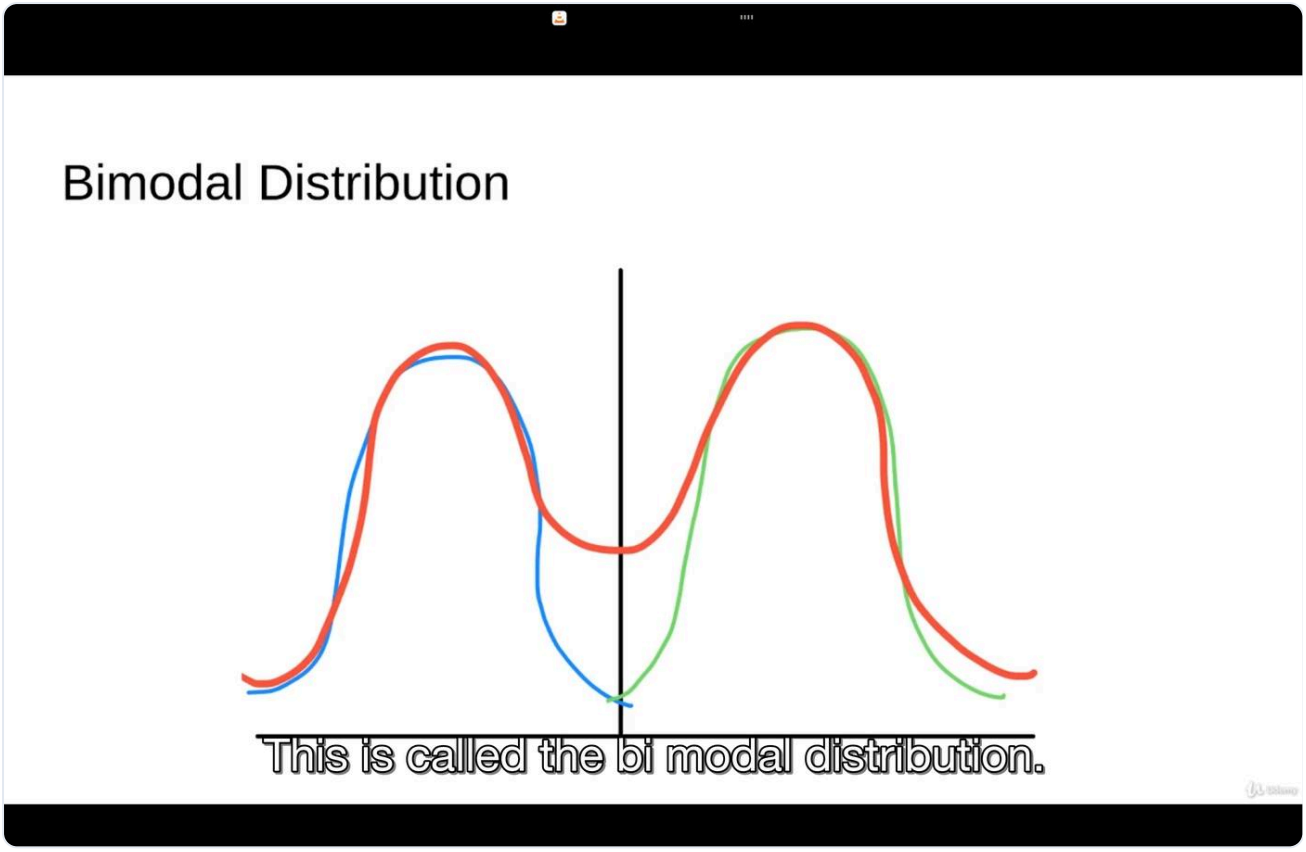


FIGURE 8.10 | Bimodal Distribution

I will show you a more sophisticated loss function to yield superior results now that we understand what a conditional GAN is. And before we head to the implementation I would like to give you another conceptual hint so that you can go smoothly through the paper in the reading assignment.

The modality of a probability distribution describes whether the distribution is a composite of other primitive distributions. In this example the red line represents a probability distribution that is composed of two Gaussian distributions with different means and variance in case of two basic underlying distributions. This is called the bimodal distribution. In the case of more underlying distributions It is called multimodal distribution. Remember that it is not limited only to normal distribution.

Now the question for you is whether MNIST and CIFAR-10 datasets have a unimodal or multimodal distribution. Think about it and share your answers together let's implement the conditional GAN in the next lecture.

LECTURE 12

Implement Conditional GAN on MNIST Dataset

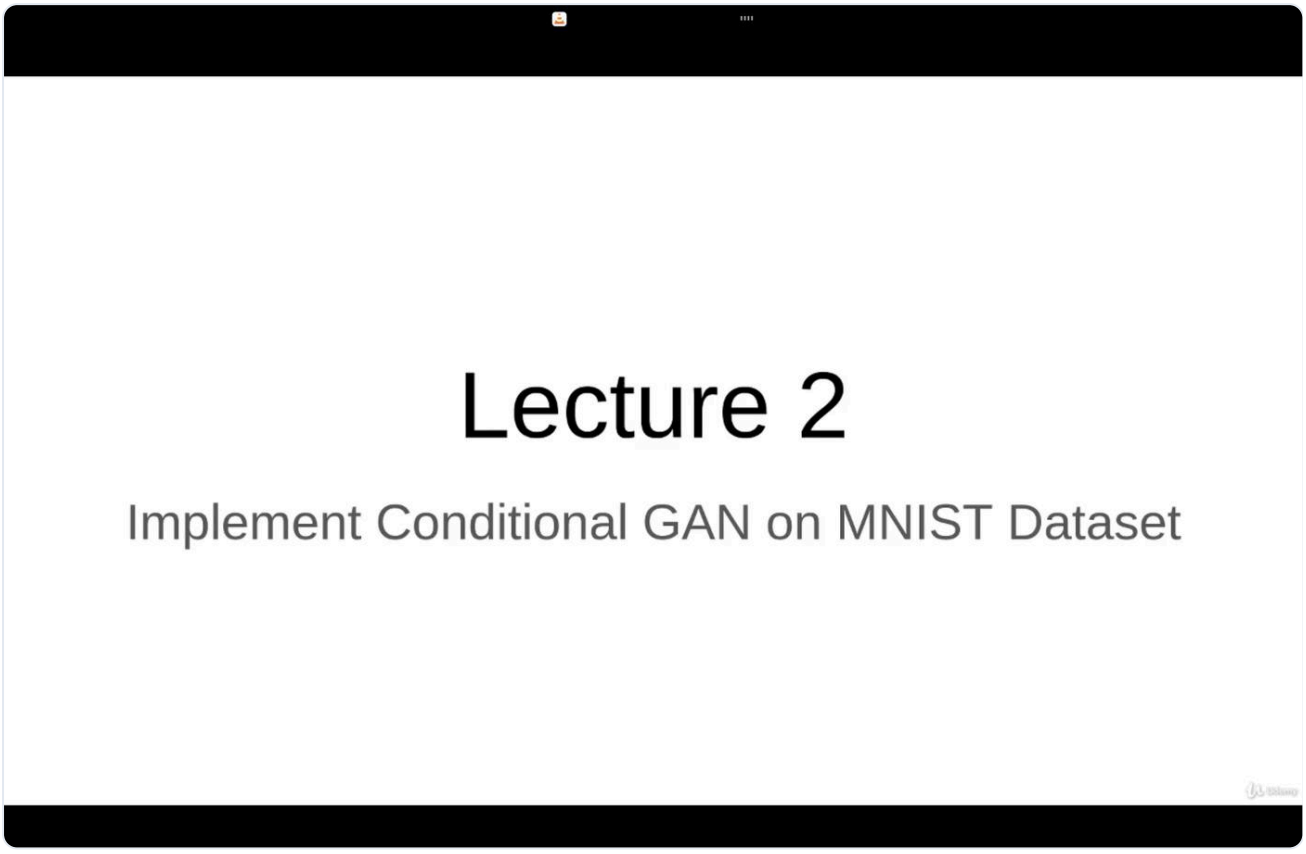


FIGURE 9.1 | Implement Conditional GAN on MNIST Dataset

See the notebook `7.Implement-Conditional-GAN-on-MNIST-dataset.ipynb` for the code.

Implement Conditional GAN on MNIST Dataset

Hello and welcome to a new lecture. In this lecture we will implement the conditional GAN on MNIST dataset. Let's get started we started by opening the Jupyter notebook file available in the course material for Section 3 lecture 2. In the beginning we import to the required libraries torch, numpy and matplotlib. And don't forget the inline directive for matplotlib so that the images will be displayed inside the Jupyter notebook.

Instead of giving an error next we use PyTorch to check for CUDA and GPU; if CUDA is available we'll use a device on cuda. Otherwise we assign the device CPU to the device variable. Next we define two global variables. The first is noise dimension, the dimension of the noise that will be given to the generator which is one dimensional array of hundred elements. The other global variable is label underscore dimension which is the number of labels available in the M this set which is 10 classes for the 10 digits from 0 to 9 next.

We designed the generator model we use the same design we have used in Section 2 with a few changes. We define a class generator that inherits from torch.nn.Module inside the constructor def **init** we trigger the parent constructor using the method super and then we define the sequential layers of the network. The first linear layer will take the input features to be of dimensions noise_dimension plus the label_dimension.

Since we concatenate together the noise vector and the label condition then we apply ReLU activation and dropout regularization we add more fully connected layers which consists of linear, ReLU and dropout and then add the final output layer we add the sigmoid activation to output between 0 and 1. Unlike Section 2 and the forward method which used to take only the self and the batch parameters we add one more parameter for the conditions. So now we have the forward self come batch command labels where labels are the conditions so that the generator will tune its output according to the label input inside the forward method.

First we reshape the batch input to be two dimensional array of batch size and the negative one for the remaining dimensions. Then we concatenate together the inputs which is the noise and the labels which is the conditions along with the first dimension by passing dim equals 1. Now our input is ready. We best the input to the network by using self.fcn and pass ret which is the inputs and we return to the output as the return of the forward method.

The output in this case will be the generated images next we design the discriminator model again. The discriminator class is defined like what we have done in Section 2 with a few changes and the first sequential linear layer of the discriminator model. We change the shape of the input to be in_features equals 784 which are the dimensions of the MNIST dataset images twenty eight by twenty eight. And then we add as a concatenated layer the label dimension which is the condition the discriminator is designed as fully connected layers each one of them has a linear layer maxout activation and drop out regularization we continue by adding a few more fully connected layers.

And finally at the final output layer we add sigmoid activation to force the output to be squashed in the range 0 and 1 0 for fake and 1 for real samples similar to the generator we add the labels as a parameter to the forward method of the discriminator and concatenated together with the input images and finally best the concatenated images and the conditions to the network and return the output which is a label is a 0 or 1 now that we have designed our models.

We fetch the dataset of MNIST digits we use torchvision data sets and apply the flatten transform which it converts the two dimensional image to one dimensional vector then we use a data loader from torch.utils with a batch size of 64 and shuffle equals True we create instance objects of the generator end the discriminator classes and convert them to the device, CPU or CUDA then we define the optimizer algorithm for the networks for both the discriminator and the generator to be stochastic gradient descent or torch.optim.SGD; we pass the parameters of the models and we set the learning rate the initial learning rate to be point or 1 and we set the momentum of 0 point 5 commented here is a dampening which is a feature in PyTorch that allows you to decay the momentum with the epochs by a factor.

This is a multiplicative factor that gets multiplied by the momentum I have commented it out since it didn't produce the required results but I would encourage you to try it out and see how the behavior is changed. It is also a good tool in your tool set so that you can know when to use it as a result of using dampening the momentum of the optimizer will be decayed or decreased as the epochs progressed towards the end of the training. This is a desired feature that is instructed in the original papers of conditional GANs and basic GANs by Ian Goodfellow.

Then we defined the loss function torch.nn.BCELoss, the binary cross entropy loss and other recommended feature is learning rate decay which I dedicate a bonus section to it at the end of this video lecture. I have also commented it out since it is implemented differently in PyTorch from what is desired. But here I give you an idea about what is possible using the APIs of PyTorch. Here we can define a scheduler for learning rate that decays the learning rate by a certain step at each epoch.

The first parameter is the optimizer. In this case the discriminator and the generator optimizers so that it can control the learning rate upon them. The step size is the second parameter and it decides how the learning rate will be changed either each epoch or each other epoch or every three epochs or every ten epochs and so on and then Gamma is the factor by which the learning rate will decay. This is a multiplicative factor. That is it gets multiplied by the original learning rate.

So for example here point ninety nine will multiply the original learning rate by 0.99 and return the output as the new learning rate. Finally the last parameter is last_epoch which I set it to negative 1. This is the default and this instructs their scheduler to use the initial learning rate at the first epoch. PyTorch APIs provide variations of learning rate decay. One of the variations is the lambda learning rate which allows you to define your own class or your own method that will tune the learning rate as the training progress in this case have defined the class.

the decay learning rate class with the constructor that stores the initial learning rate and the step size to reduce the learning rate at every two steps are three steps and so on. And then I defined the method call that gets triggered whenever the decay learning rate class is invoked and next we define an if condition. If epoch modulus self.step_size equals equal zero then the learning rate will be decayed by a multiplicative factor of point 1. This allows us to get sort of multi-step learning rate which is also actually a defined method in the PyTorch APIs.

But I wanted to show you how to apply your own customized learning rate decay method. And then we create two objects for learning rate scheduler for both the discriminator and the generator and pass the lambda learning rate to be the decay learning rate class we have just created and we set the initial learning rate similar to the one we have set on the optimizer and the step size to be a hundred. Again I have commented this part out as it didn't produce the required results but I insist on showing you how to use this kind of functionality in your model design and the exercise and in the bonus section at the end of this lecture I will show you a more mature example of the learning rate decay next we define a visualization method that will help us visualize the output of the generator at every epoch.

This is essential as we know from Section 2. There is no objective method of evaluating the output of the generator unless we visualize and inspect the images generated by the generator manually. So we define a method visualize_gan. And this method will use matplotlib subplots to present ten images for the ten digits and each cell of these subplots will have the image and the title which is torch.argmax of the labels of the index of this image at the end of the method the plot will be saved as a figure in the target directory the labels of the MNIST dataset are numerical for each label of the digits.

That's why we need to encode them as one-hot encoding a vector of zeros and a one only in the place of the target label. In Section 1 I have showed you one way to do one-hot encoding and this section I show you another way of doing one-hot encoding and use it for preprocessing the data or the target labels so defined the method encode_one_hot which takes a label as a parameter and then it creates a torch float tensor of the target shape labels, the shape at zero which is usually the batch size along the first dimension and then the second dimension will be the label dimension which is 10 elements.

Since we have 10 labels and this dataset this float tensor will be initialized to be all zeros by using the method ret.zero_ and then using another method from PyTorch which is named scatter scatter will replace only the zeros. at the indices defined by the labels. For example if the first sample has the label three then three is the index and that will be used to be replaced from 0 to 1 and that will indicate one-hot encoding for the vector 3.

We do that by passing the dimension along which these replacement will happen. dim equals one, that is the second dimension and then we pass the indices labels.view of negative one comma one. And finally the value to be replaced which is 1 so comma value equals 1. And then we return the output which would be the one-hot encoding of the batch and the target values. Now we are ready to train our GAN model. So we create the real labels as ones and the fake labels as zeros.

And then we create a fix it nice dimension test underscore Z which is the noise that we will be given to the generator and we will use a fixed noise so that we can visualize the output of the generator on a consistent sample. At this moment I would like you to pause the video and think a little about the way we initialize this test_Z and think why it is different from the one we have used in Section 2. Now that your back. I hope that you have found it.

Here you can see that we multiply the random numbers returned from torch.randn by 2 and subtract 1 from them. What effect does this have. So let's take for example randn returns the value 0 so 2 multiplied by zero. That will be zero and then subtract one that will give us the value of negative 1. On the other end imagine torch.randn returns the value 1 so 2 multiplied by 1 is 2 and subtract 1. That will give us 1 or plus 1.

So in this case we have obtained Gaussian random noise with mean zero and standard deviation negative 1 and plus 1. You can play around with these values I have set them this way so that I can show you you can control even the distribution that is going to be given to the generator. If you remember that as an exercise in Section 2 there was a hint to use a uniform distribution and see how the changes are going to be on the generator outputs. Then we create the conditions test_Y which is the conditions that will be passed to the generator and the discriminator.

Along with this batch we set the number of epochs to be 256 which is a very low number. At the end of this video lecture I show you the same example which I run for four thousand sixty nine epochs on Google Colab. Then we define the number of steps which is the length of the train loader divided by the batch size. Note here the double slash for integer division then we define the directory where the images generated will be saved for further inspection and create the directory.

If it doesn't already exist next we define four arrays to log the loss and log the learning rate then we define a for loop for the number of epochs we have defined above. Then we initialize the counters and the loss to 0 then we loop over the data set and we train the discriminator for 4 steps more than the generator we convert the images to the device and then we one-hot encode the labels and convert them to the device. We do that twice once for real images and other for fake images where we get fake conditions at random which are also one-hot encoded and then we get fake images bypassing the noise to the generator and passing the output to the device we reset the gradients of the discriminator by using discriminator_optimizer.zero_grad and then best the real images and the real conditions to the discriminator receive the output as the real_outputs and then best the fake images and the fake conditions to the discriminator and save the output as fake outputs.

Remember that we should never mix the real and the fake images when training the discriminator some papers were published that have tried to do this and it didn't work out. Then we evaluate the loss on the real outputs and the real labels and evaluate the loss on the fake outputs and the fake labels. This way you have obtained the D of X and the D of G of Z. Then we apply the backward to calculate the gradients and use the optimizer to update the weights.

Finally we increase the discriminator counter, d_counter. Plus equals 1 and then we calculate the loss as D of X plus D of G of Z .item so that we can get the floating point value inside the PyTorch tensor afterwards. We train the generator again we generate the noise to be Gaussian random noise with zero mean and plus and minus 1 deviation and convert it to the device and then we generate random numbers for one-hot encoding as labels or conditions we reset the gradients of the generator by using generator.zero_grad and best the noise and the conditions to the generator and to the discriminator and we get the outputs.

We calculate the loss using criterion of outputs comma real labels. Notice here that we flip the labels for the generator so that we are trying here to maximize the probability that the generator will generate images that don't get recognized by the discriminator as fake having calculated the loss we apply the backward to calculate the gradients and use generator_optimizer.step to update the weights. Here is also commented the learning rate decay but if we have applied the learning rate decay as defined above then we would need to apply discriminator_scheduler.step and generator_scheduler.step so that this will count as a step in the learning rate.

The K counter after performing this step the learning rate will be reduced by the factor that we have assigned above. Then we increment the counter for the generator and we increment the generator loss by using that item the next snippet is responsible for doing some bookkeeping and logging so every ten epochs we print the line with the number of the epoch and the generator

loss and the discriminator loss. We divide here by the counter to get the average and then if we have used the scheduler like above we can use scheduler.get_lr so that we can visualize the learning rate that we have obtained after applying the decay method.

Finally every five epochs we visualize the results of the generator by passing the fixed noise and the fixed conditions to the generator and visualizing the output we visualize the output once again at the end of the training. Since we are not sure that the visualization method got invoked every five epochs and that included the last epoch so we perform that as a safeguard. At the end of the training now we visualize the loss function as reported in the loss function arrays for the discriminator and the generator using matplotlib the blue curve represents discriminator loss which is almost zero for most of the time.

That means that the discriminator outperforms the generator the generator loss started to be very low while the discriminator was still learning and then it increased after the discriminator got stronger by epoch number 50 and number seventy five the generator loss has decreased and the generator started to learn some useful information and fine-tune its outputs. Here we visualized the outputs we have obtained after 512 epochs using the PyTorch utility for visualization make grid you can see here that the first cell is almost 1.

The second is 2, the third is around 3. The first two are not really four but then we have got 7 9 7 9 and then 7 again and 8 so I have allowed the model to train further for 4000 epochs on Google Colab and you can see here the results which is much better. You can see the first is 0 1 2 3 4 5 6 7 8 9 which is very clear very vivid. Excellent results. You can actually obtain much better results if you are allowed to train further and fine-tune the hyperparameters change the learning rate apply the learning rate decay apply momentum dampening find out which tricks you can apply to obtain even better results as a hint you can even apply a median filter to remove the noise after the generated images have been output so at the end of the notebook you find the link to the Google Colab notebook for reference so this is a bonus section in which I dedicated to learning rate decay.

So the learning rate decay formula as can be seen is alpha at time t will equal one divided by one plus alpha or the initial alpha, alpha at time zero multiplied by a decay factor the decay factor I used here 0.01 which is actually a hyperparameter that you have to apply grid search or cross validation to find the optimal value for this decay factor so I design a simple PyTorch model with two neurons as input and one output neuron and sigmoid activation.

This will learn the XOR function, which is a non-linear function. Remember the X or problem 0 x 0 0 0 0 x or 1 is 1 1 x or 1 is 0. And so on so I defined the dataset as X for the training data and y for the labels then I create an object of the model and create the optimizer Then I create an object of the model and create the stochastic gradient descent optimizer so I defined here a lambda learning rate which applies the formula above 1 over 1 plus e which is the epoch number multiplied by 0 point or 1 then I use binary cross entropy for the loss.

Finally I train the model for 10000 epochs. You can see here how the idea is very clear. We reset the gradients, we calculate the loss. I calculate the gradients and then update the weights and decay the learning rate using scheduler step. We use an array to log the learning rate value using

scheduler.get_lr so finally we visualize the learning rate as the number of epochs increases so we can see here the blue curve represents the learning rate as the number of epochs progresses towards the end.

You can see here it takes this sort of inverse exponential form as it decays sort of rapidly in the beginning and then slowly as the training progresses and that model is about to converge to its final output in the coding assignment at the end of this section I would ask you to apply the same conditional models on Fashion-MNIST. And here is a hint so you can use Fashion-MNIST from torchvision which is available as a data set and you can save it in another directory dot slash data slash Fashion-MNIST and you can apply the same flat and transform in the coding challenge or the coding assignment as well.

I would ask you to go very wild and apply the decay learning rate at your own imagination and see what works and what doesn't work. Here is an idea which actually didn't work very much. Here's an idea which actually didn't work very much as I have expected. Even though the results are really clear and really vivid I will show you at the end of this notebook. But what I mean is that the decay rate was very slow which means it tends almost to a flat learning rate.

However it is interesting to see that you can control the epochs and you can control different decay rate for different range of epochs. For example here for the first two hundred epochs I don't change the learning rate for the second 200 I change it by a bigger factor. So note here that in the denominator, if the denominator is large then the overall decay factor is small. So if the epoch is less than 400 then I use a large denominator by applying one plus E multiplied by point 1 and that will decay the learning rate slowly and then I decay it further by decreasing the denominator.

And here again I use lambda learning rate and apply it on the discriminator and the generator optimizers and as a hint for you when using the fashion in this data set. Here are the categories or the conditions or the class labels for t shirt trouser pullover dress and so on fashion. This is a more challenging dataset compared to MNIST. It was designed to challenge models further and see how they can attain further accuracy. What I have found during the training of MNIST and Fashion-MNIST again that Fashion-MNIST was much faster to converge even faster than MNIST which is contrary to what is believed when applying the classification problem because in classification question MNIST is more challenging than MNIST but for GANs I found it less challenging and much easier to converge so you can see here again.

The loss function that orange curve is the loss for the generator and the blue curve is the loss for the discriminator. So we can hear see again the same pattern. The discriminator loss is almost zero and the generator loss starts to be very high and then drops down and and converges around 6 or so here to obtain a visualization of the learning rate. And as I've told you the method I have applied above did not really produce the actual learning rate decay that we have seen in the previous notebook.

But here it shows you the importance of visualization. Whenever you are working as a data scientist or a machine learning engineer or deep learning engineer it's important to visualize every step of your work so by visualizing the learning rate decay. Here we can see that it is almost a flat learning rate the decay was very insignificant. What we can see how it works and this tool can be

applied to other problems and even to other assignments so here we can see samples of the visualized results of the generator and you can see here you can easily recognize the t shirt to the trouser the pullover the dress and as we progress through the training you can see the results are getting much better and more recognizable and actually very similar to the ones available in the Fashion-MNIST dataset that was trained on.

So now before we conclude this lecture there is a question that I would like to discuss with you. For example in this image we can see that the label at the top. Let's zoom in to see it clearer the title at the top of every image of generated sample. It gives the class or the condition that was used to generate this image. For example this one it looks more like a shirt but it has the label ankle boot. Why is the model confusing the labels. This is something I would like you to think about clearly because it happened also with MNIST dataset and it happened even further with fashion.

This dataset where the conditions are mixed up with the generated images. So for example if I give the condition of a dress I get a trouser. If I give as a condition of a bag I get a shirt and so on. This is a question we are going to answer in Section 4. I would like you to think about it in this lecture and come up with answers and share it in the discussion forum and this lecture we have successfully implemented the conditional GANs and played around with learning rate decay and we have demonstrated the performance of conditional GANs on the MNIST dataset and fashion MNIST dataset in the next lecture.

We see the workings of Wasserstein loss and how we would design a WGAN. Thank you and see you in the next lecture.

LECTURE 13

Working of Wasserstein Loss Function

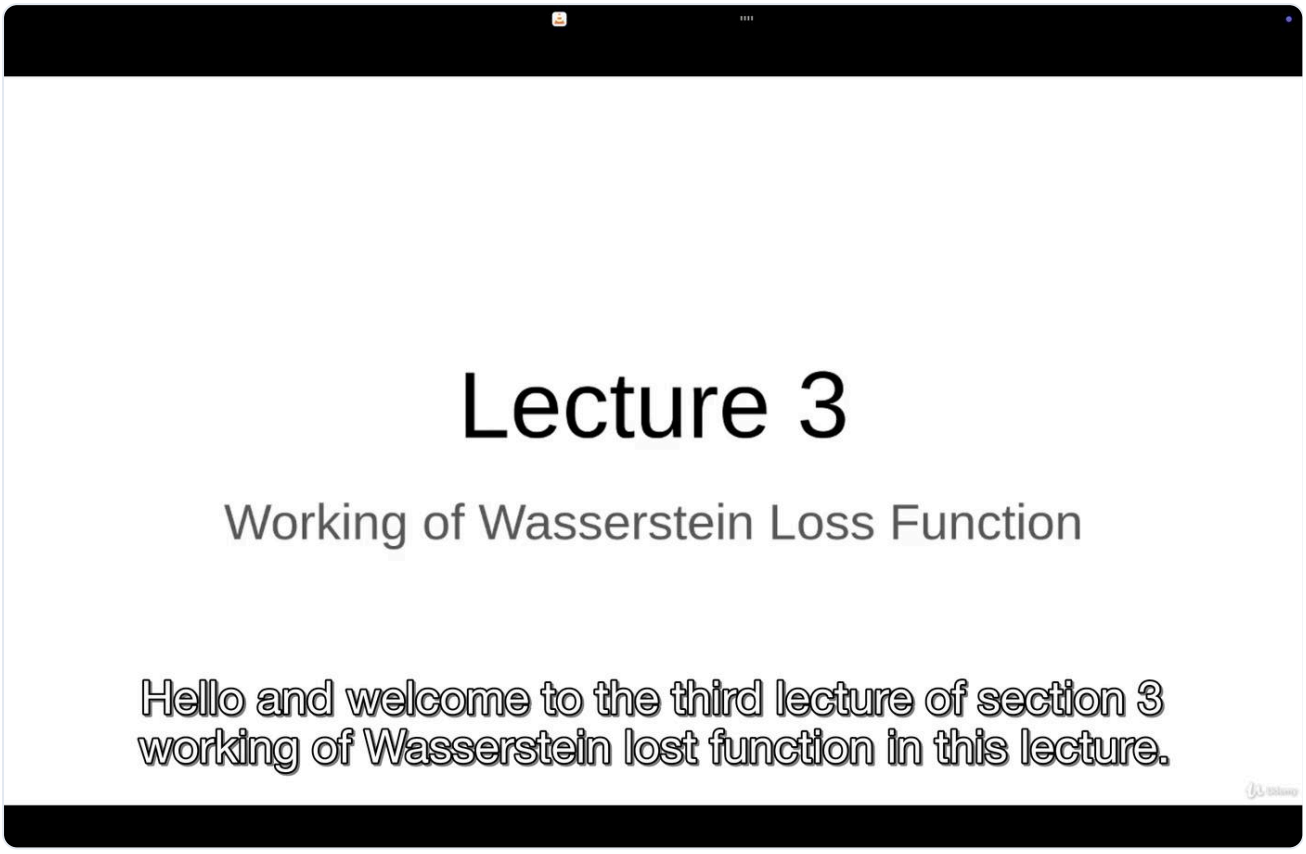


FIGURE 10.1

Hello and welcome to the third lecture of section 3 working of Wasserstein loss function in this lecture.

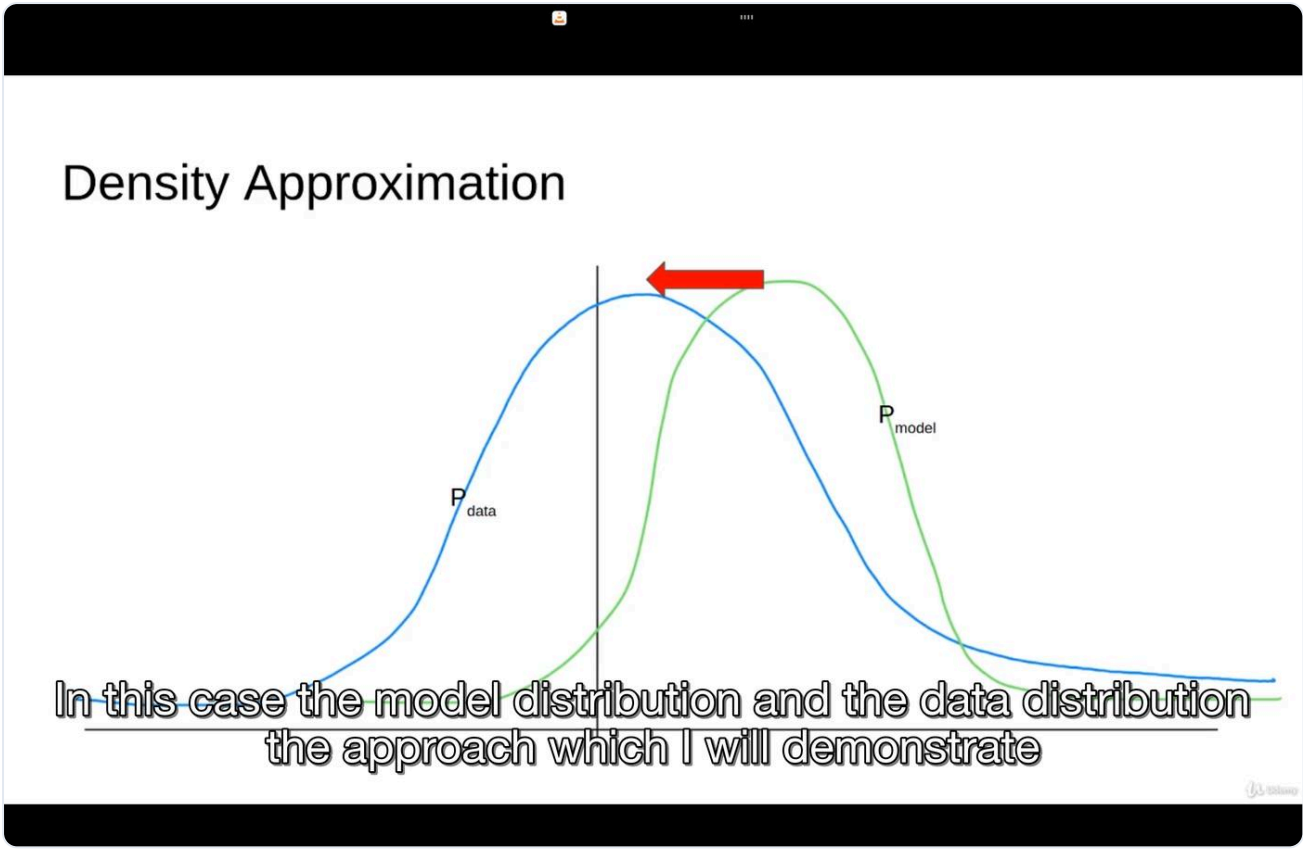


FIGURE 10.2

You will learn of the motivation and the inner workings of WGAN remember from Section 2 that the goal of generative models is to learn a mapping between two probability distributions. In this case the model distribution and the data distribution the approach which I will demonstrate in this lecture is inspired by this simple idea again from Section 2 coding exercise.

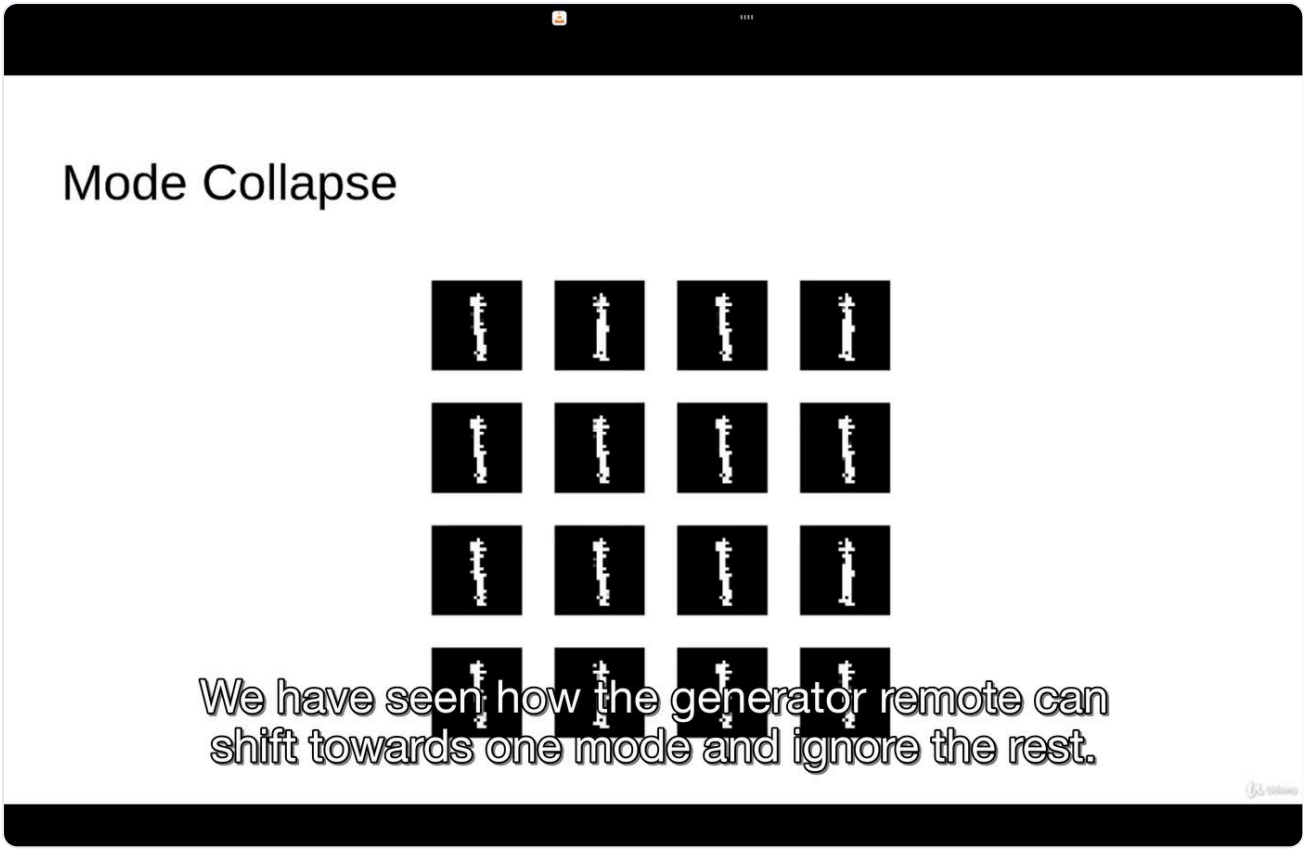


FIGURE 10.3 | Mode Collapse

We have seen how the generator mode can shift towards one mode and ignore the rest.

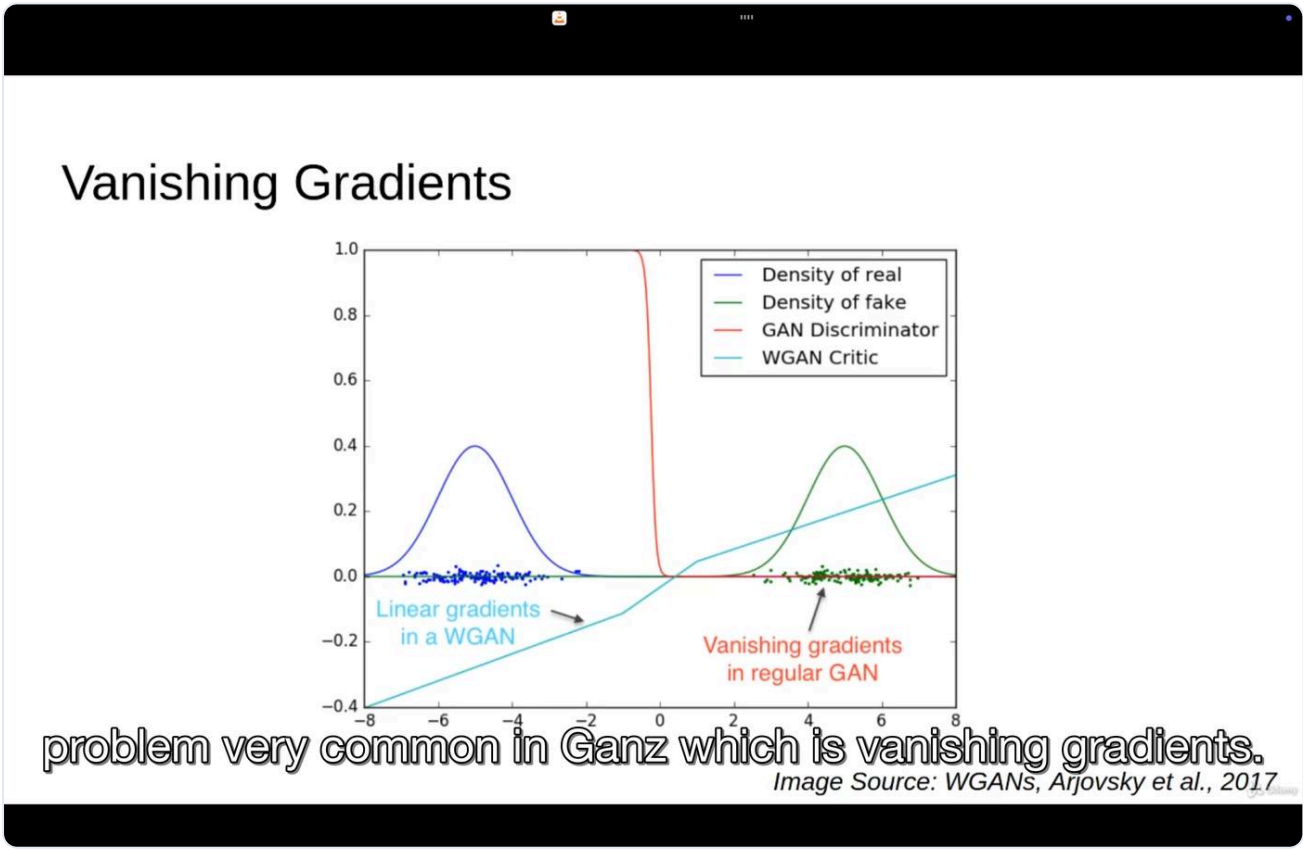


FIGURE 10.4

This is once again a motivation behind WGANs the graph on the screen right now demonstrates another problem very common in GANs which is vanishing gradients. The red curve is a perfect example of gradients disappearing or diluting after very few epochs. Very few to the point that it can even happen at the first epoch. Why is vanishing gradient a bad thing. Think about it for a second.

The gradient flow is essential for weight updates.

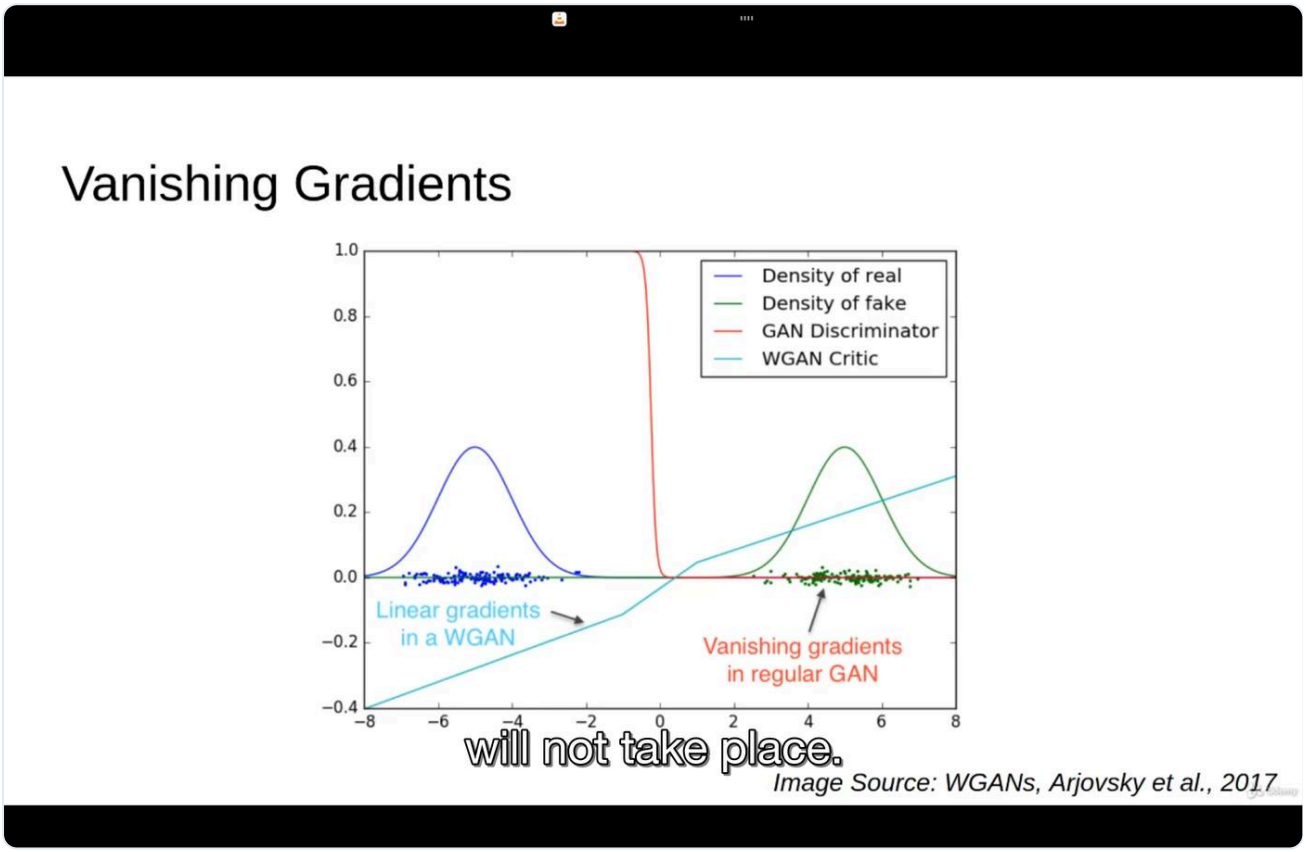


FIGURE 10.5

If the model is not receiving significant gradient values during the back propagation then the learning will not take place. Hence it is essential to visualize and understand the gradient flow as part of the model design. The greenish blue curve on the other hand shows how the gradient looks like when using WGAN approach in Section 1.

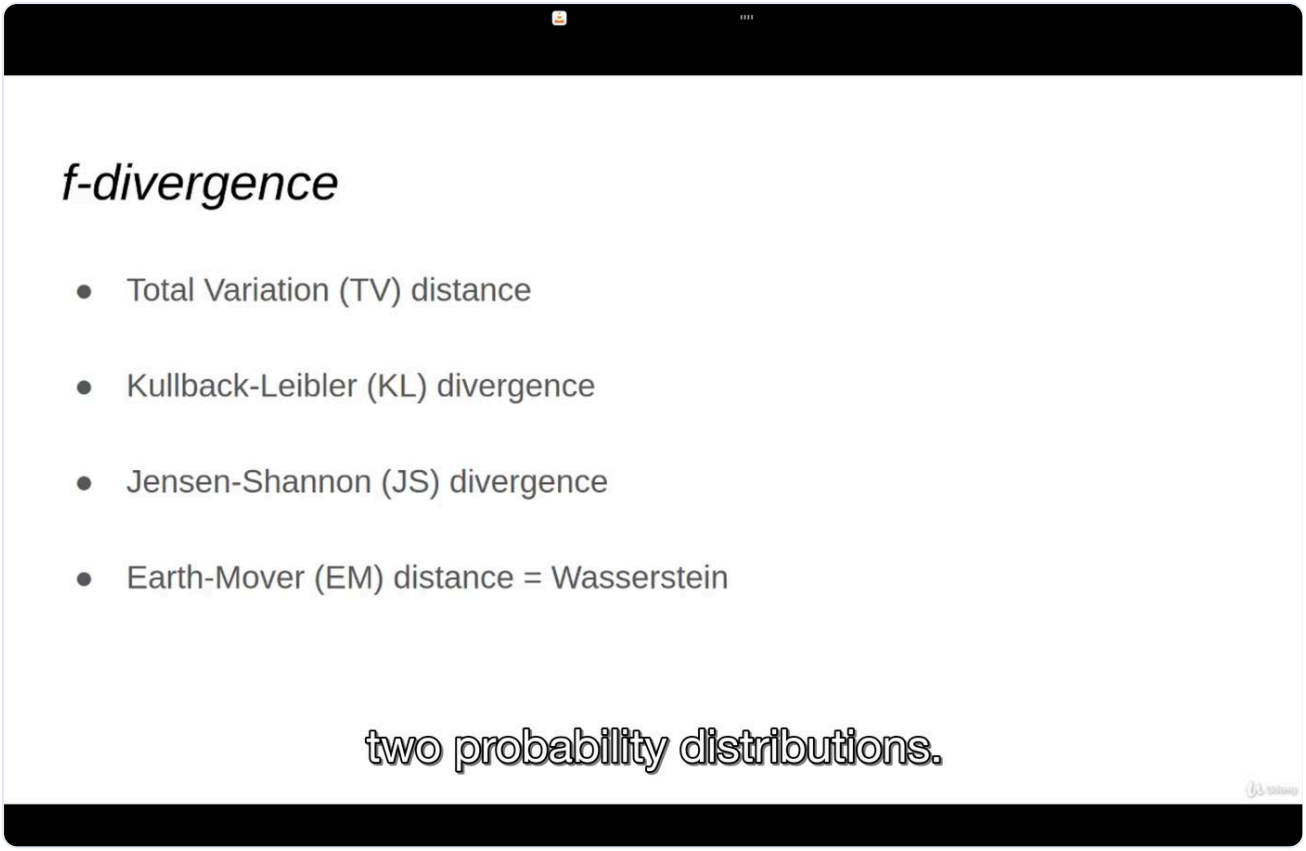


FIGURE 10.6

I have discussed Cauchy-Schwarz divergence and even implemented it in the coding exercise.

There is a complete family of metrics that are based on the idea of calculating the distance between two probability distributions. These metrics go by the name f-divergence in the next slides. I shall emphasize a few characteristics that make one of them better than the other for generative modelling in general and for GANs in particular I leave it however to the interested reader to look at the mathematical formulas and proofs in the original paper which is referenced at the end of this section and by the way it is in the reading assignment.

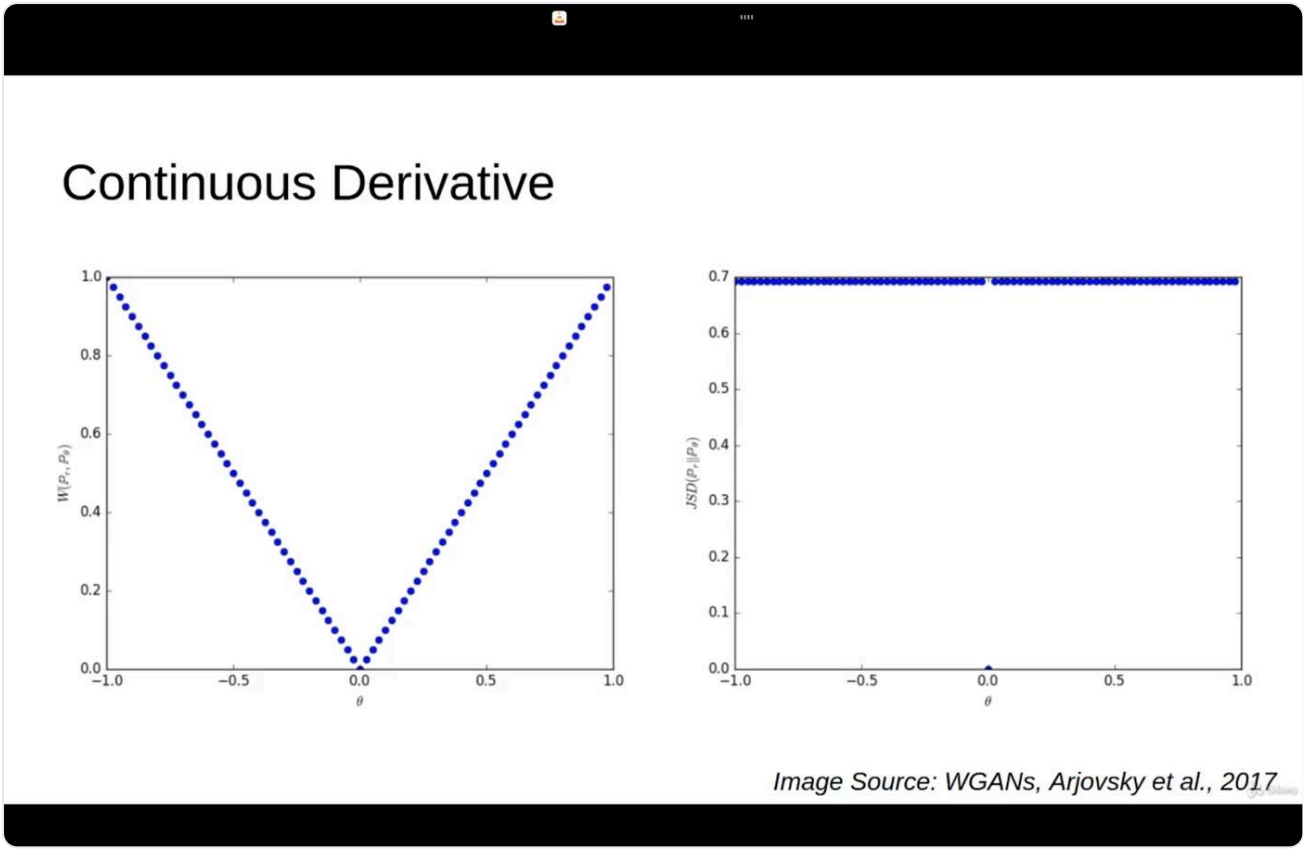


FIGURE 10.7 | Continuous Derivative

You know from elementary calculus classes that the first derivative corresponds to the slope of the tangent. However the derivative can be discontinuous at some points. This is the last thing we want when applying gradient descent on the left the gradient of Wasserstein distance is very steep and continuous on the right side.

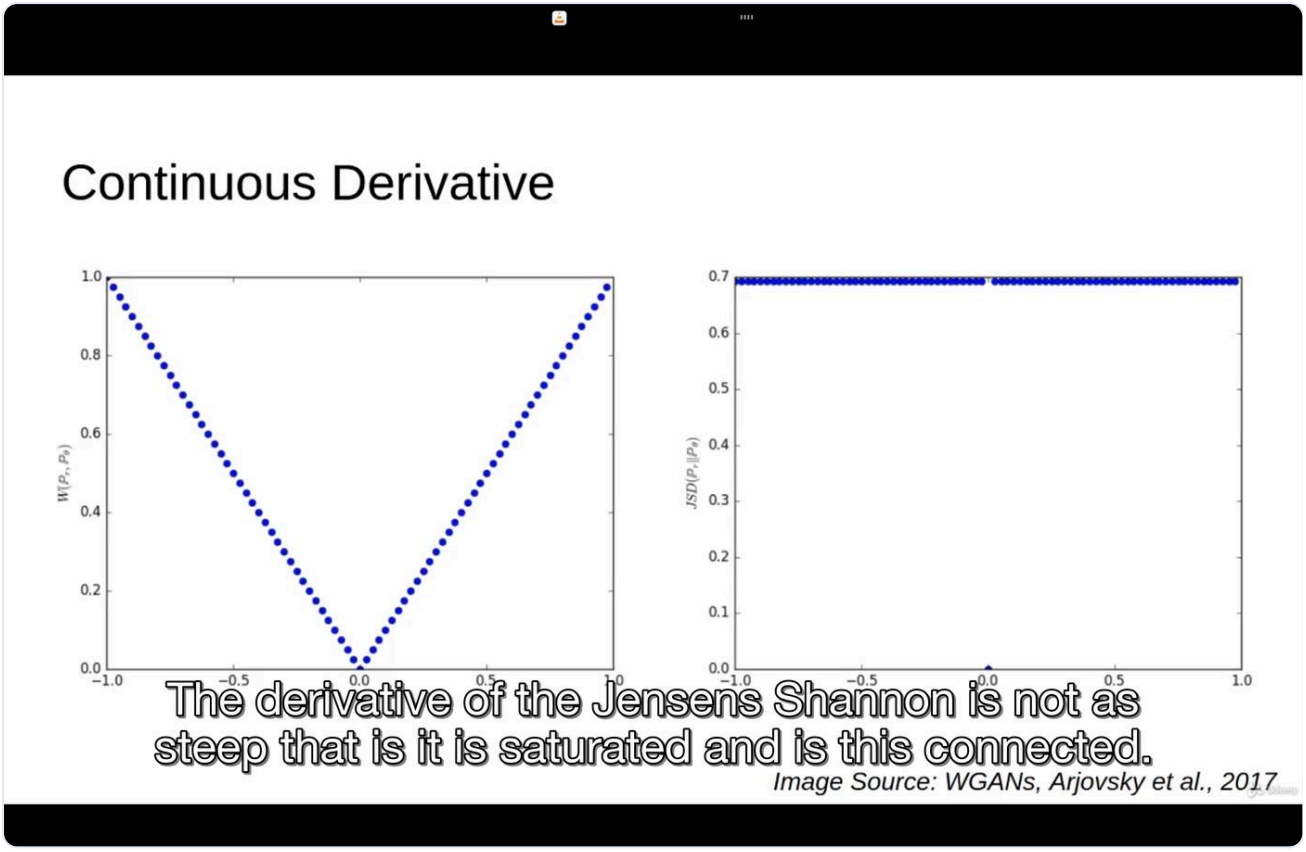


FIGURE 10.8 | Continuous Derivative

The derivative of the Jensen-Shannon is not as steep that is it is saturated and is disconnected.

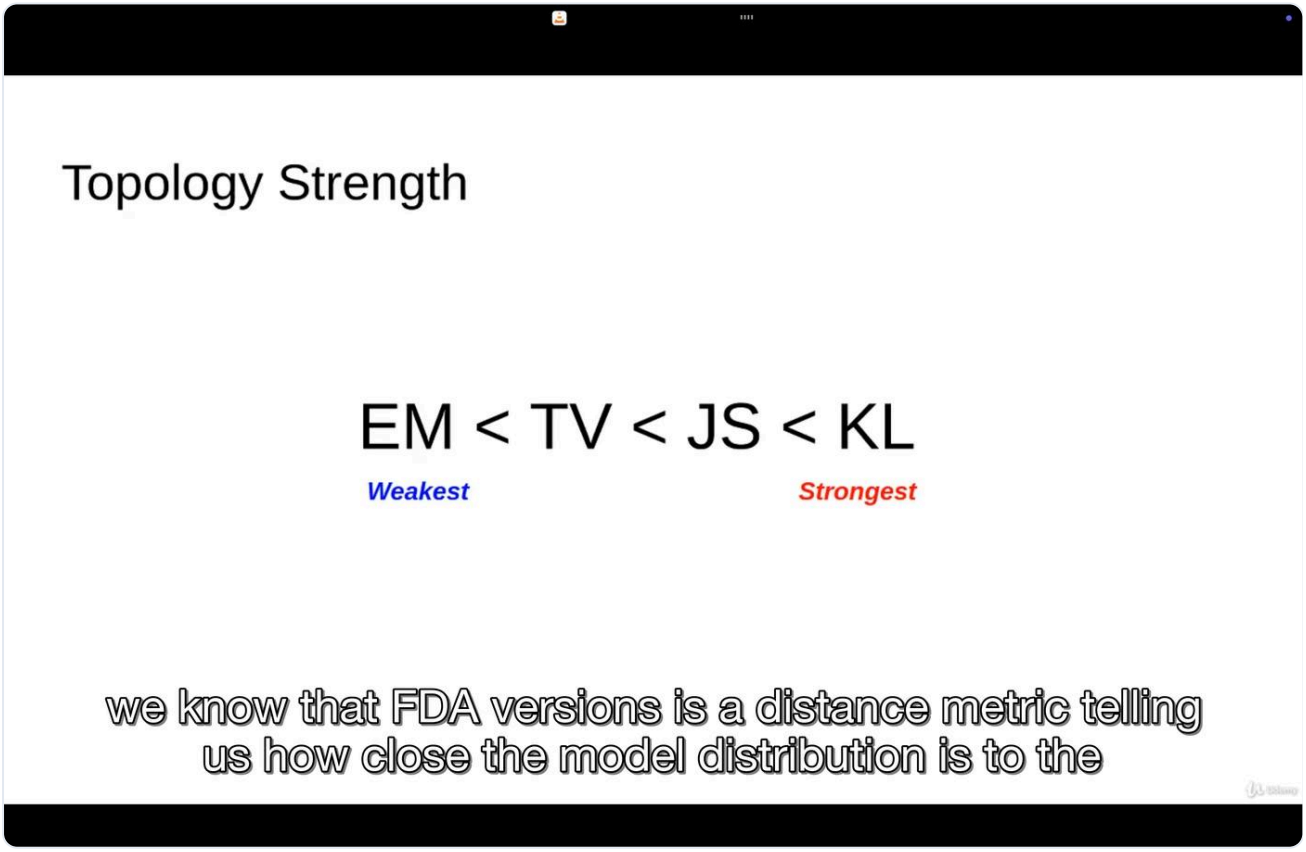


FIGURE 10.9

It must be clear by now that Wasserstein distance has a very desired derivative characteristics so we know that f-divergence is a distance metric telling us how close the model distribution is to the real data distribution.

So what is the concept of topology strengths informally a weaker topology makes it easier for distributions to converge since convergence is the ultimate goal of learning a weaker topology means that the number of sequences converging to be close to each other is more than the number of sequences in case of a stronger topology that is under a weaker distance metric.

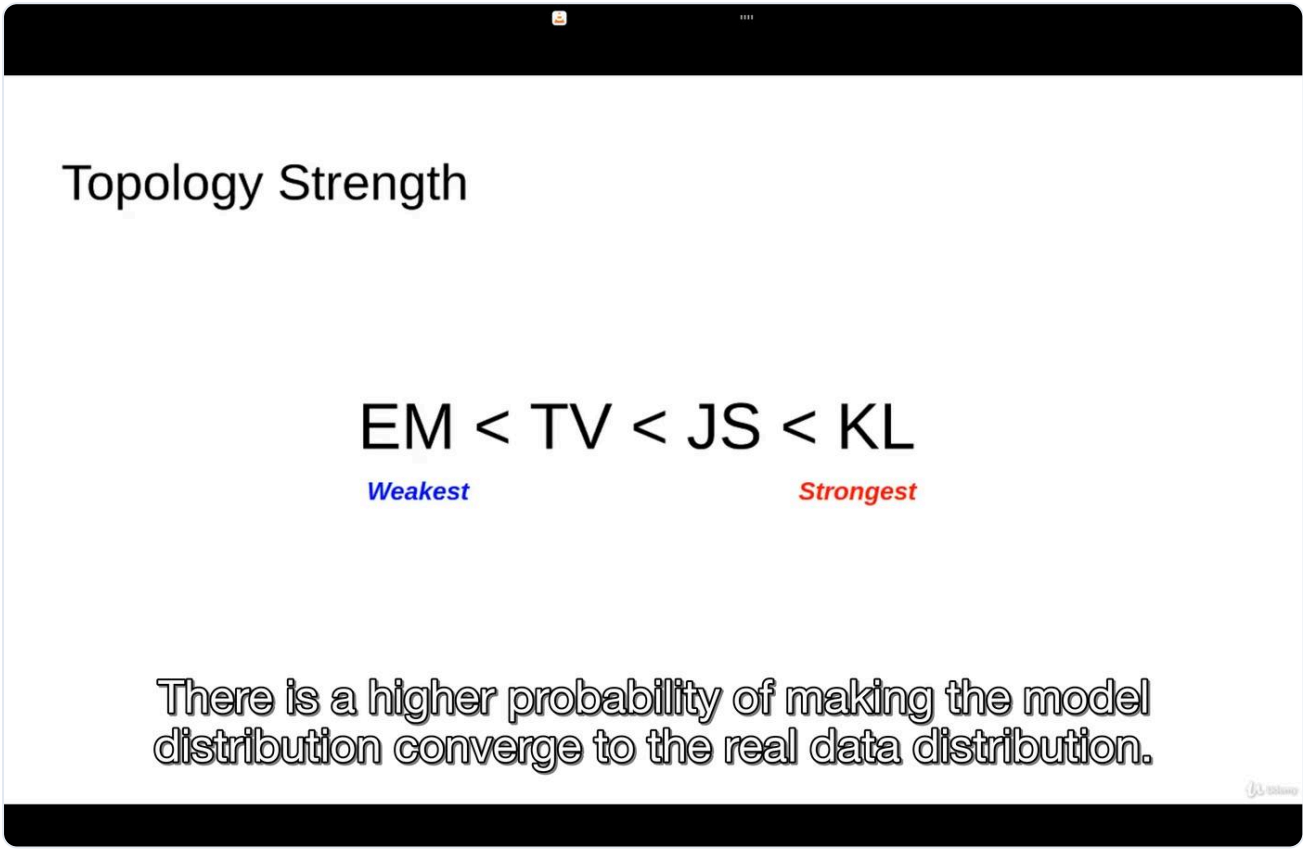


FIGURE 10.10

There is a higher probability of making the model distribution converge to the real data distribution.

The WGAN paper demonstrates a very concrete mathematical proof for such a concept. Moreover it compares to the different f-divergence metrics to each other in that aspect Wasserstein distance which is also called Earth Mover's distance has the weakest topology. That's why it is a perfect choice for GAN training. You may ask why it is called Earth Mover's.

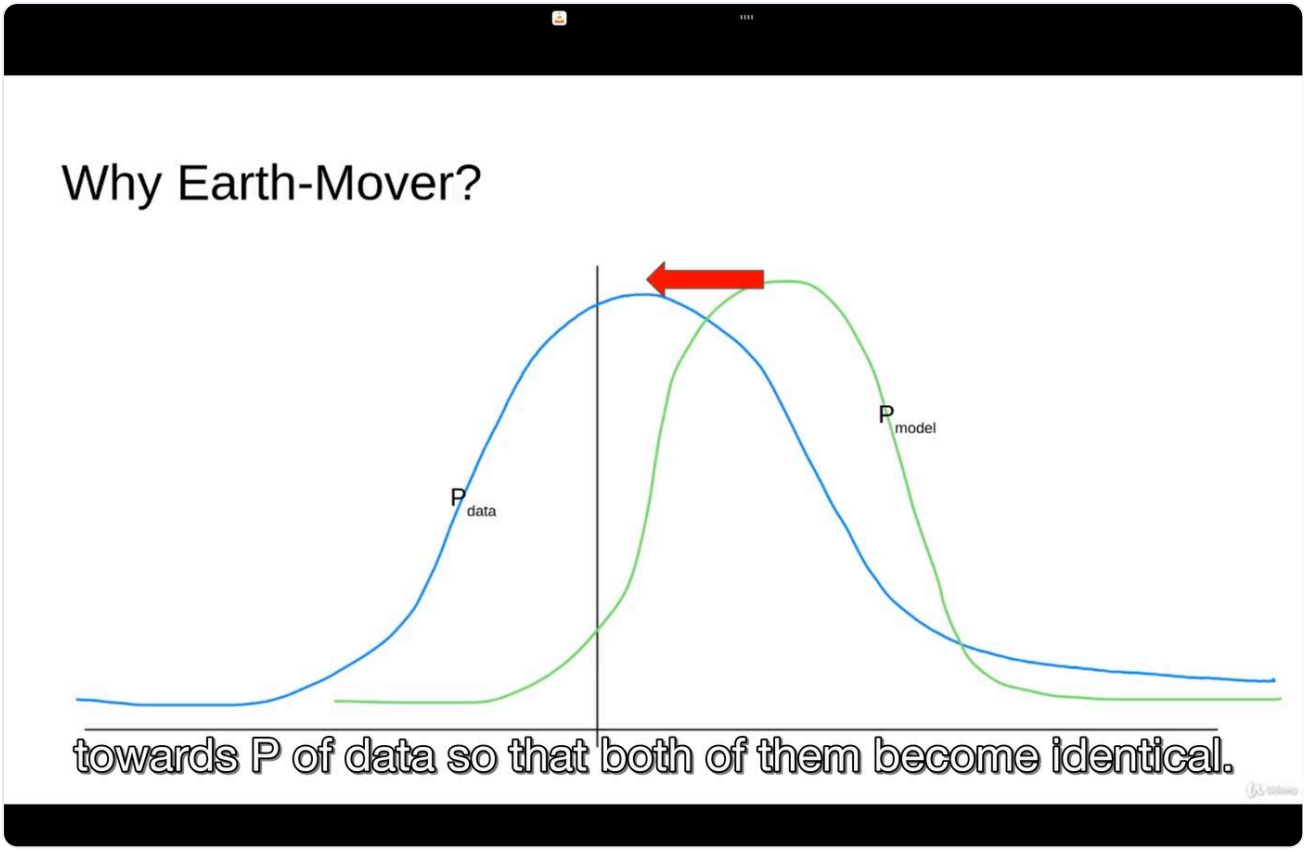


FIGURE 10.11 | P of Cata So That of Then Identical

It is an intuitive naming that signifies the minimum amount of earth that should be moved from P model towards P data so that both of them become identical.

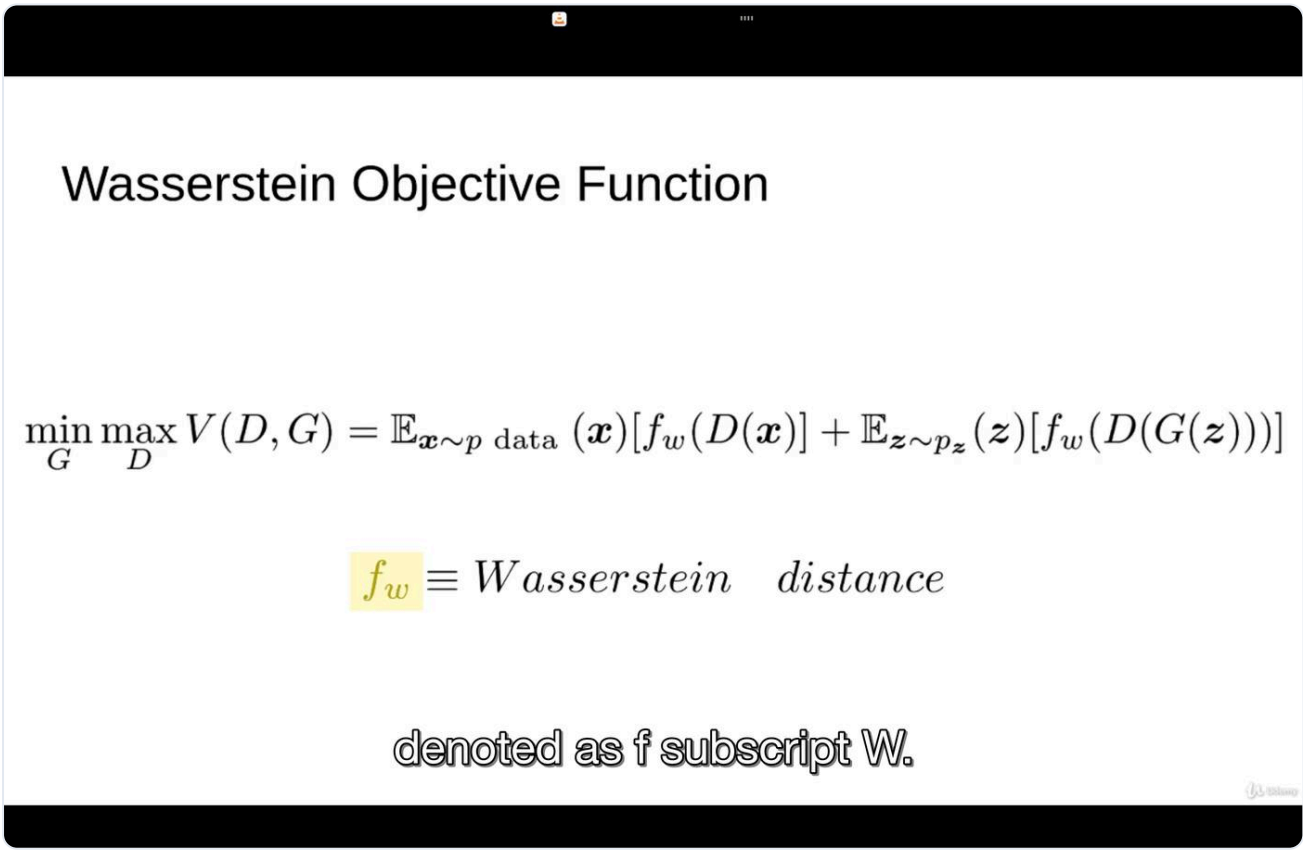


FIGURE 10.12 | Wasserstein Objective Function

Since it is an analogy of moving one region of earth towards the other region and that's why it is named Earth Mover's the objective function for WGAN can now be formulated using the Wasserstein distance denoted as f subscript W. So the loss for the discriminator now becomes f subscript W of D of X and the loss for the generator becomes f subscript w of D of G of Z.

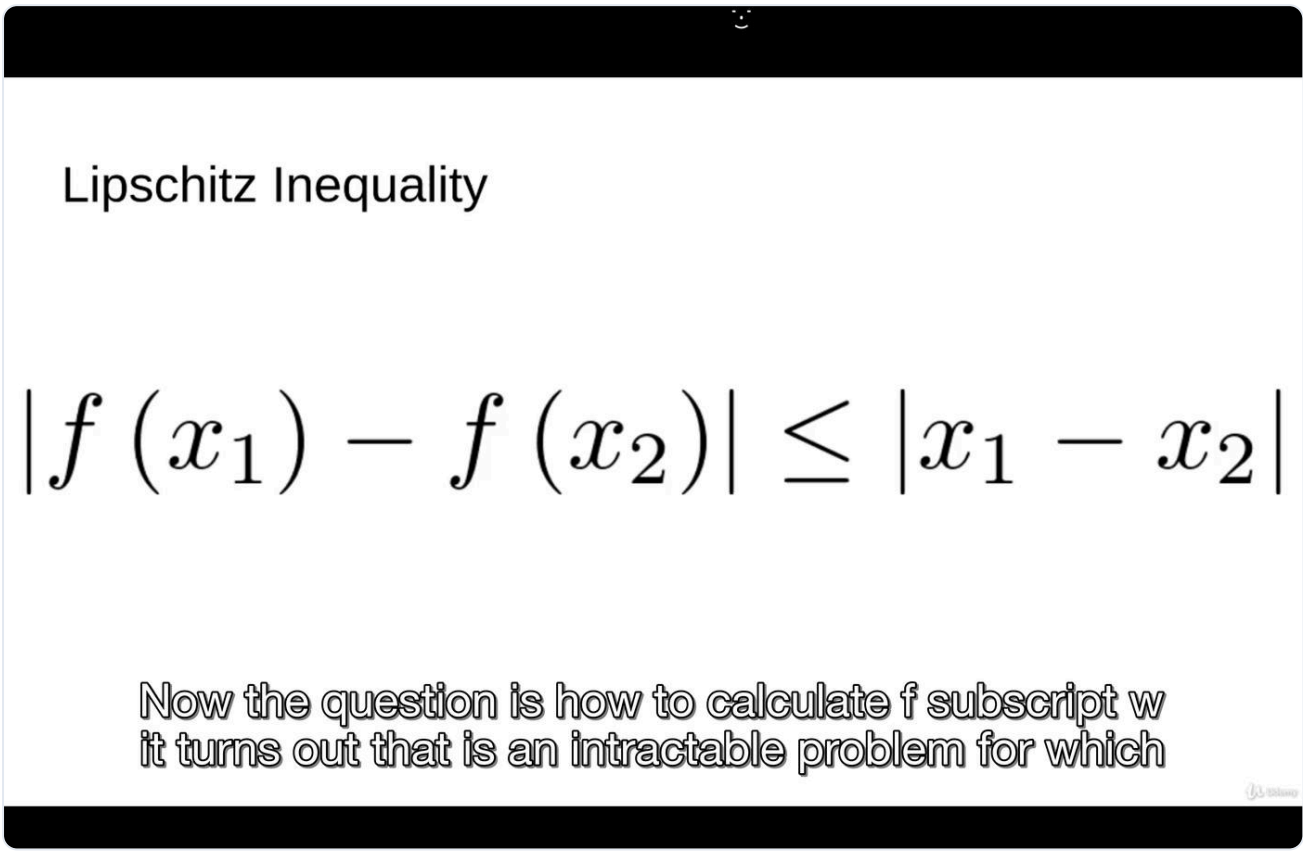


FIGURE 10.13 | Lipschitz Inequality

Now the question is how to calculate f subscript w it turns out that is an intractable problem for which there is no polynomial time solution in order to solve it.

Let's introduce the concept of Lipschitz inequality Lipschitz inequality states that under the assumption of continuity of a function the variable x can be replaced with the x value. The function of the variable x can be replaced with the x variable itself. So the idea here is that using the Lipschitz inequality we can replace f subscript w with the values passed to the function Lipschitz inequality is helpful to obtain an approximation for the loss function in WGAN.

So given that Wasserstein distance is in itself an optimization problem that can be solved using a minimization function it becomes an objective that is the model must learn along its training process. I shall materialize this idea in the coding exercise another question that you have part of its answer by now is why the gradients don't vanish anymore when using WGAN the part of the answer that you already know is that Wasserstein distance is continuous and its derivative is defined everywhere.

However another reason is because how the target for fake and real data is defined.

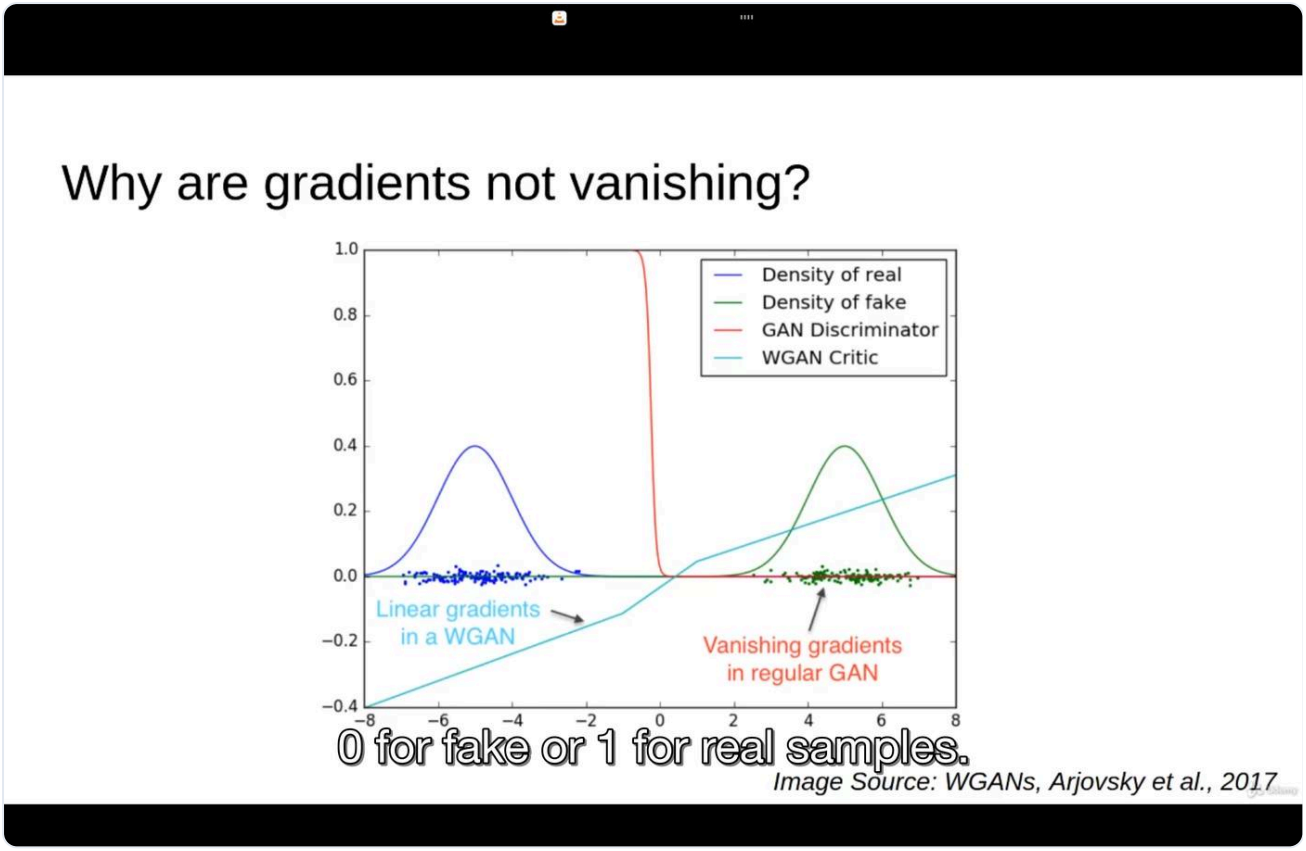


FIGURE 10.14 | Why Are Gradients Not Vanishing?

I'm talking here about the output of the discriminator and the basic GAN we had sharp outputs either 0 for fake or 1 for real samples. The output here in WGAN undergoes two main changes the first the change that the fake is assigned the value of negative 1 and the real is assigned the value of positive 1. The second the change is that the loss is not based on log loss anymore which doesn't give a useful signal when a sample is misclassified.

That is it only tells the model that a fake sample is classified as real or the opposite.

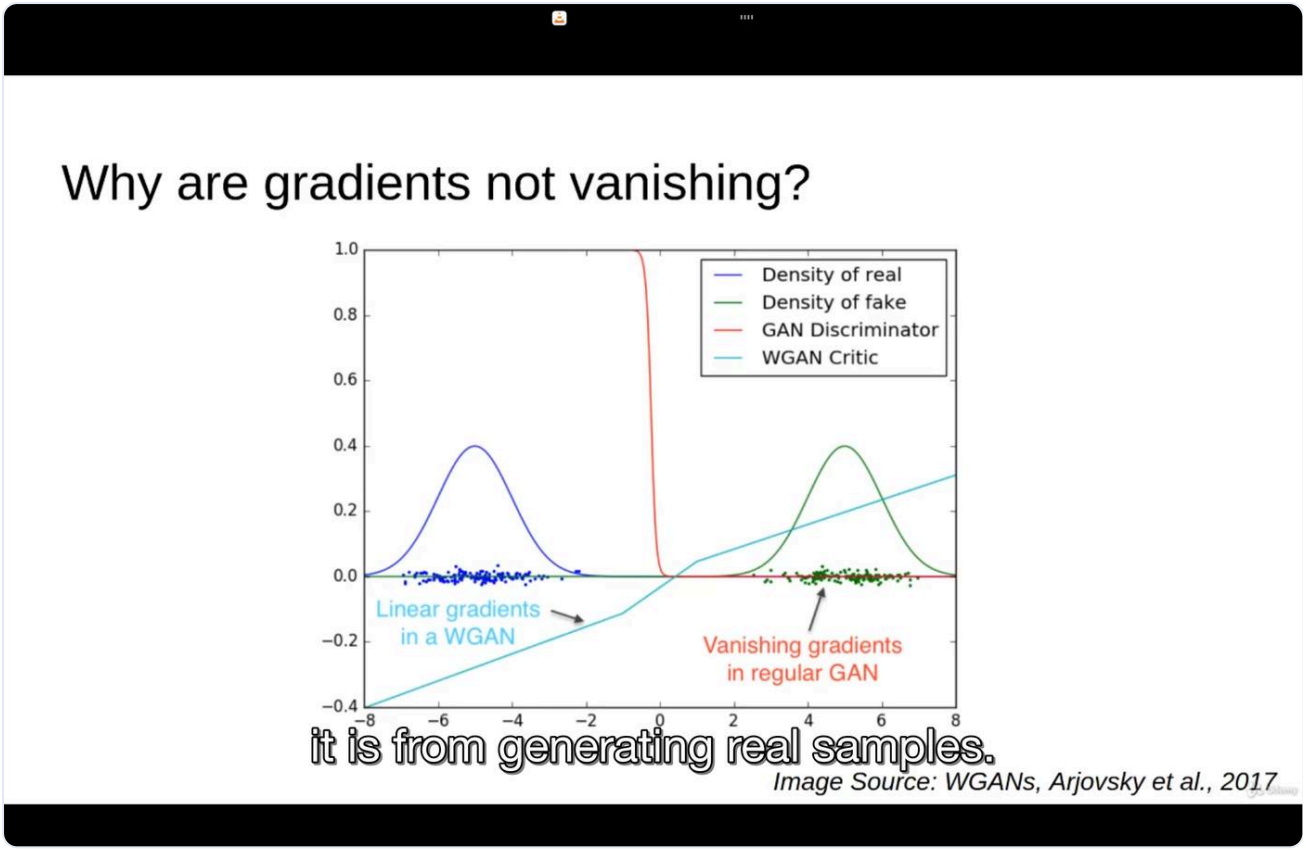


FIGURE 10.15 | Why Are Gradients Not Vanishing?

In other words only true or false output. However in WGAN the objective function gives the model an indicator of the distance or how far it is from generating real samples. This is exactly the reason why the gradients carry more information now that the model can use to update its weights here we've got the optimal hyperparameters that the authors of WGAN paper have used for the MNIST dataset.

The learning rate was set to 0 point 0 0 0 5 which guarantees a steady minimization without jumping back and forth.

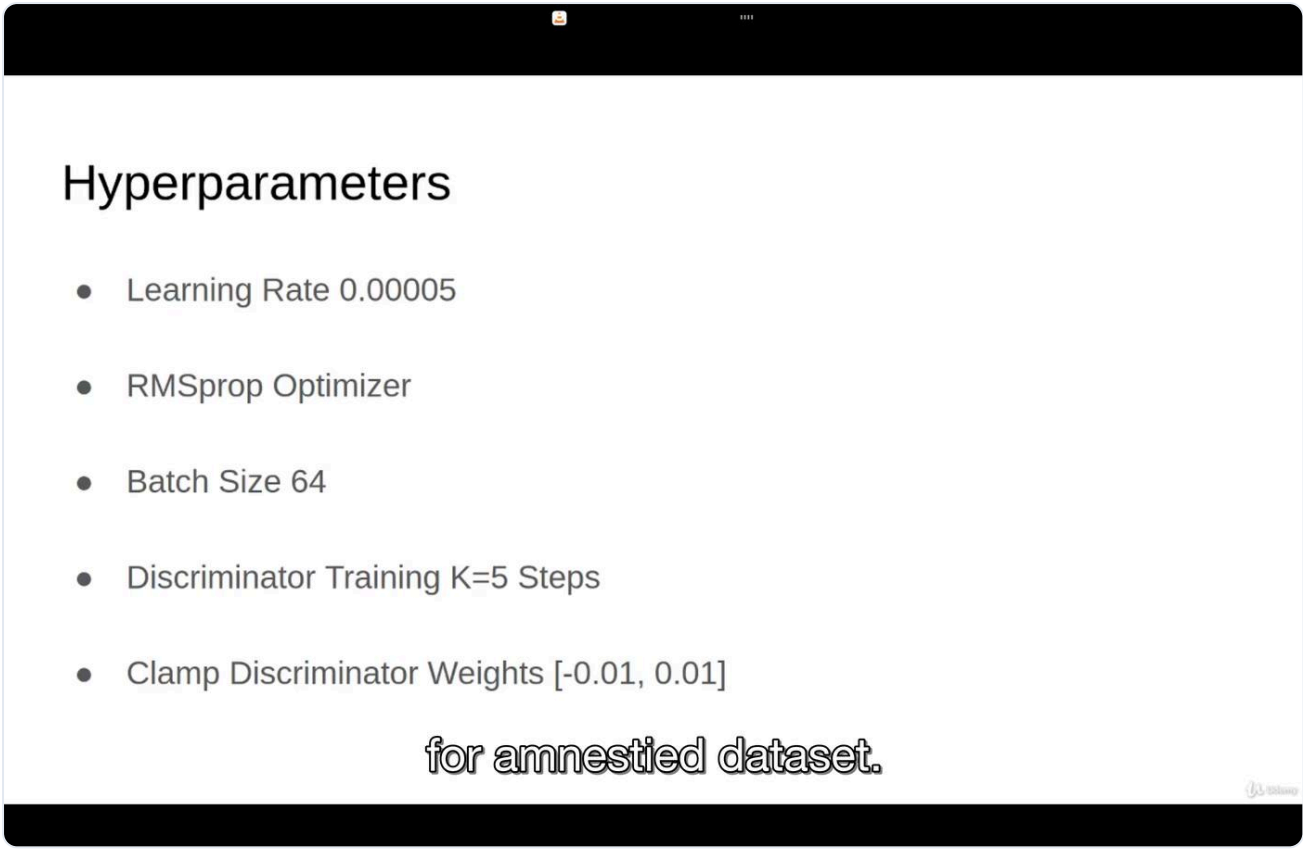


FIGURE 10.16 | RMSprop Optimizer

Also they have moved away from momentum-based optimizers and used the RMSProp optimizer. But why. Because they found out that a momentum-based optimizer causes WGAN to be highly unstable a problem that is solved by using RMSProp. Also the batch size is set to 64 samples per mini-batch and because the mode collapse, a problem has been properly addressed by the Wasserstein metric then the discriminator can be trained for five times or more till optimality without saturating the generator.

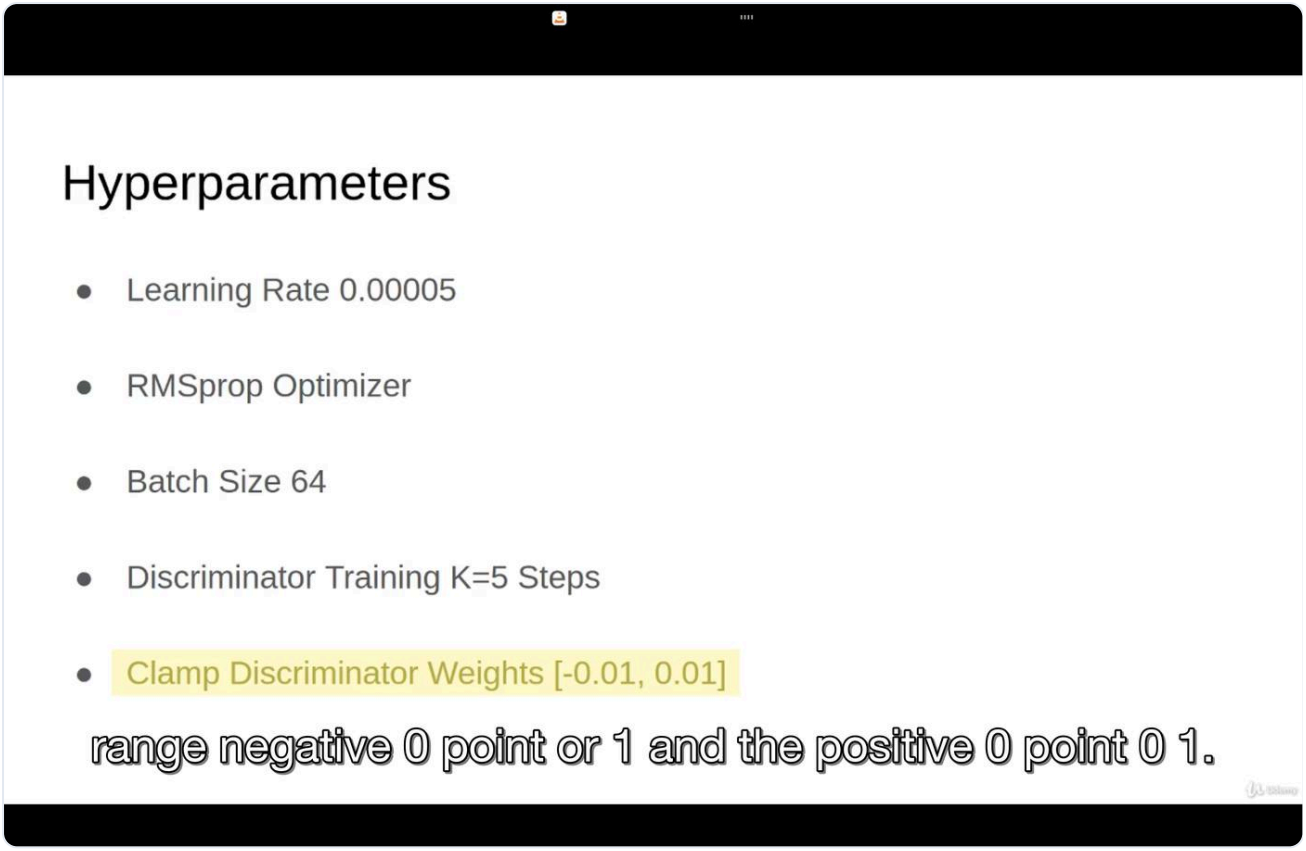


FIGURE 10.17

Another precaution that must be taken into consideration is that the weights must be clamped in the range negative 0.01 and the positive 0 point 0 1. But the question is why.

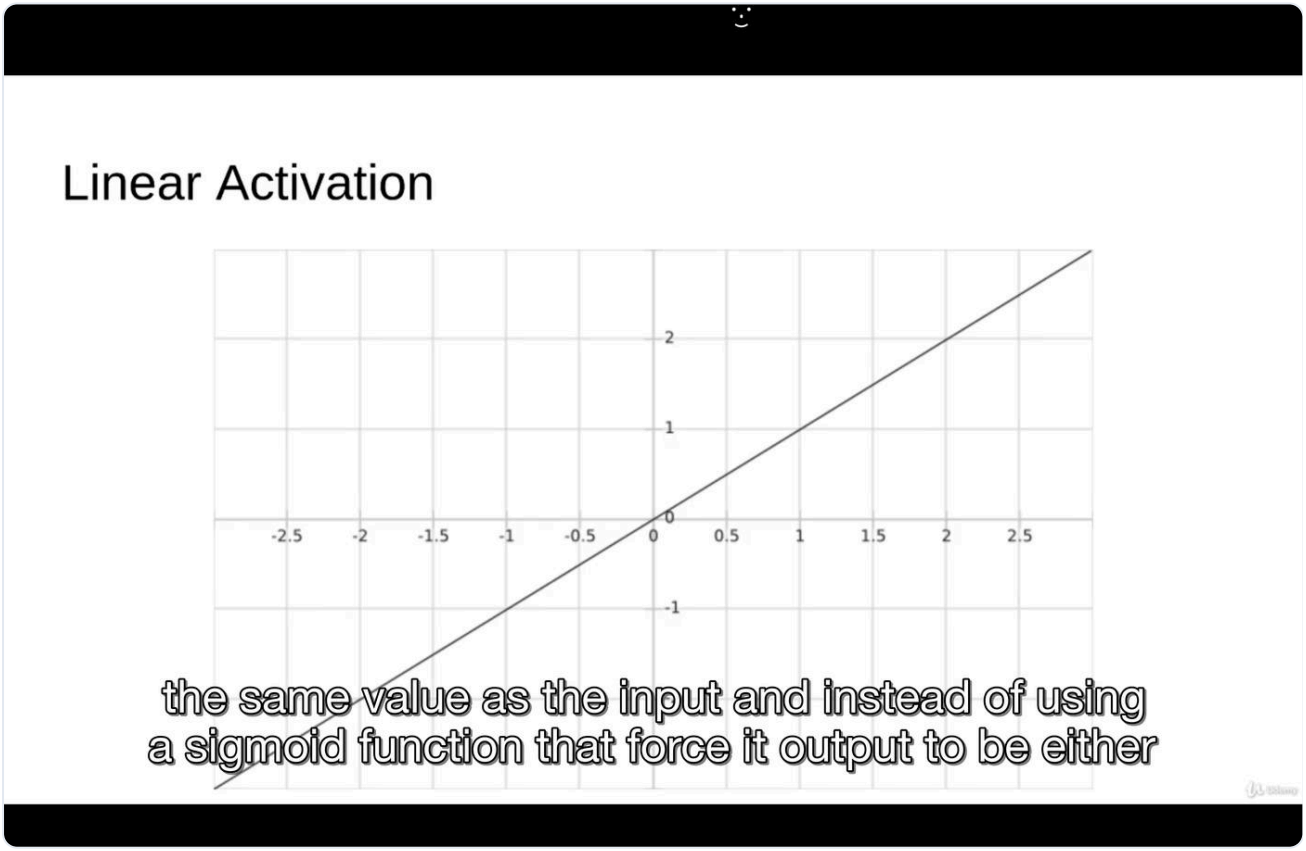


FIGURE 10.18 | Linear Activation

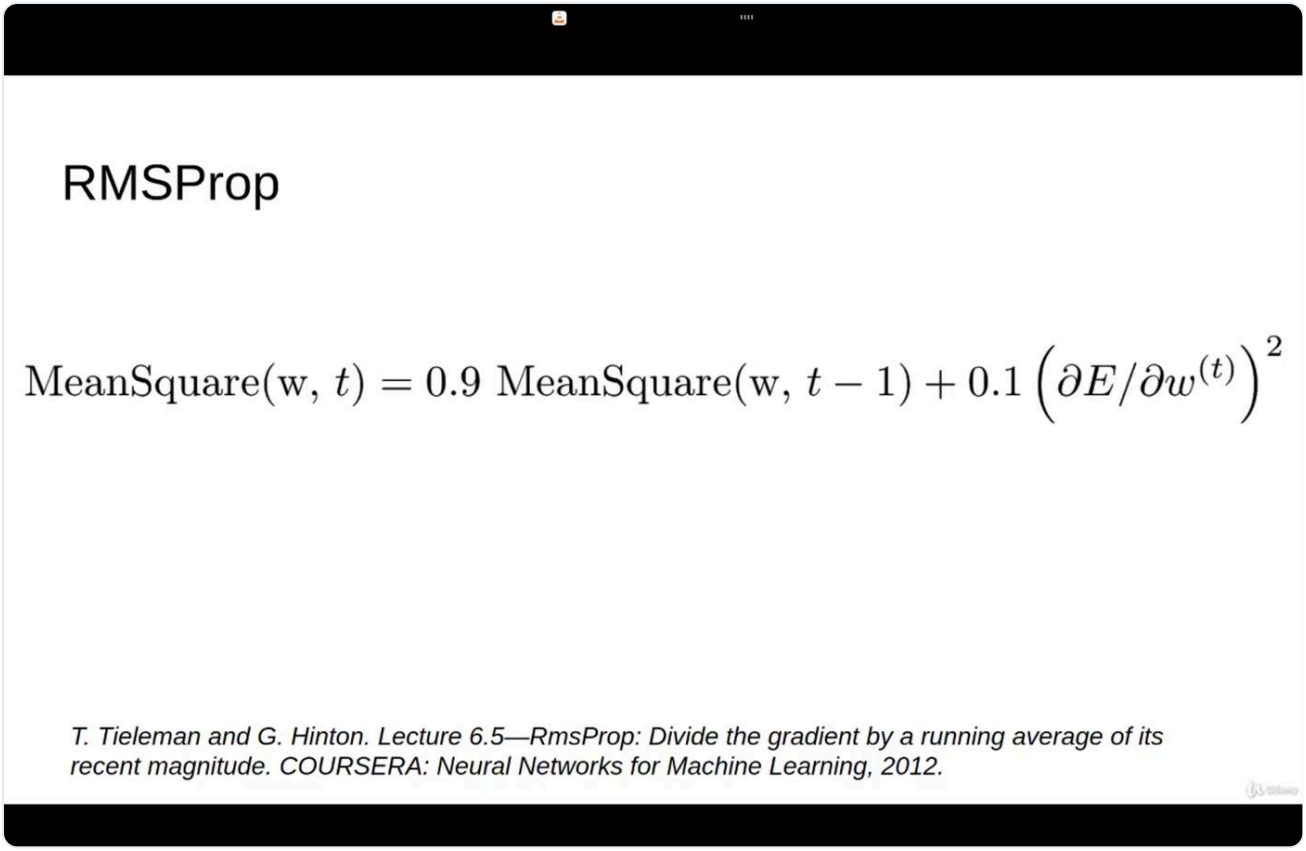


FIGURE 10.19

It turns out that clamping is the way to enforce the Lipschitz inequality constraint the actual values to use are again a hyperparameter that must be fine tuned the values here are the ones suggested in the WGAN paper one more major change to enforce the output of the discriminator to be a metric in the range between negative 1 and positive 1 is to use a linear activation function that simply outputs the same value as the input and instead of using a sigmoid function that forces its output to be either 0 or 1 finally for the sake of completeness I will give you a brief idea of how RMSProp works.

The main difference is that the gradient is calculated is a weighted average of previous mini-batches. For example if the gradient is calculated nine times as the value of two and only the tenth time is calculated as the value of twenty then we don't want the weights to be updated with a very high value. That's why we assign a higher weight to the average gradient which is point nine. In this case and then add the gradient for the current mini batch with lower weight which is 20 in this case it is one of the most popular optimizers and yet was not officially published.

Interested readers should look it up in Geoffrey Hinton's online lecture. In this lecture we have covered WGAN in a deep amount of detail. Next we shall implement it in the coming lecture. Thank you and see you in the next lecture.

LECTURE 14

Partial Derivative and Earth Mover's Distance

See the notebook `8.Implement-Wasserstein-Loss-Function.ipynb` for the code.

Implement Wasserstein Loss Function

Hello and welcome to a new lecture. In this lecture we are going to implement. the Wasserstein loss function. Let's get started we start by opening the Jupyter notebook file for lecture four available in the course material in the first block we install TensorBoard visualization tool which is part of the TensorFlow package however it's compatible with PyTorch and the latest release as a dependency you may need to install the Future package as well we import the regular libraries torch and matplotlib.

And don't forget the matplotlib inline directive. And then we import the tensorboard, we can import it as follows from torch.utils.tensorboard import SummaryWriter which is the object that we are going to use to write to the TensorBoard logging. We create a new instance of SummaryWriter and call it tb_writer and just to say something like hello world we do tb_writer.add_text and then the text for the text WGAN the actual text in it and then the epoch zero just like previous lectures we continue torch.cuda.is_available and then save the device, whether it is CPU or GPU.

And then the noise dimension which is hundred next we design our generator model the same way like we did in previous lectures using fully connected layers with ReLU activation and dropout afterwards we designed the discriminator model and this one will be quite different from the ones we have done in previous lectures. Let's see in detail what will change. So we defined the class Discriminator, inherits from torch.nn.Module inside the constructor.

def **init** of self and then we invoke the superclass of the parent class with super(Discriminator, self).**init**(). That will trigger the constructor on the parent class and then we define the forward network which is torch.nn.Sequential was started by fully connected layer linear layer and then the maxout activation another linear layer, maxout, linear and then note a difference here. We don't have the sigmoid function at the output layer anymore so the last layer of the sequential is the linear layer which stands for linear activation with one neuron or one output neuron inside the forward method.

We change the flow a little we reshape the batch input to be the batch with size and negative one for remaining dimensions and then we pass the inputs through the forward network. We take the outputs and notice here the difference is that we take outputs the mean along the first dimension dimension zero. So here the forward method will return a single value. Remember that when we do the training for the discriminator we don't mix the batches the mini batches of the real and the fake data.

That means whenever we have a mini batch it is either real data or fake data. It is never a mix. So when we take the means it gives us an indicator of the general output that is given for real or fake data. So for example if we annotate the real data as plus 1 and the fake as negative 1. So we get the mean on the mini-batch of real data. If it is somewhere between 0 and plus 1. So we have got a reasonable loss. And on the other hand if it is between zero and negative one that is a reasonable loss for fake data.

Otherwise then the loss would be very high. And the model will be instructed through the backward pass to update its weights to change this behavior and then in the forward method of the discriminator after getting the mean of the output we change the output to be a scalar value of one dimension and return it using output.view of one. Next we load the MNIST dataset like before with the flatten transform and define a data loader with shuffle equal True and the batch size equals sixty four so as we have discussed the in the slides we use RMSProp as an optimizer for the discriminator and the generator.

So we create an instance of the generator and discriminator classes and named them generator and discriminator. Then we create optimizer an instance for each one of them using torch.optim.RMSprop and remember that we set the momentum to zero as instructed in the paper and then we set the learning rate to 0.00005 which is also hyper parameter that is advised by the authors of the paper so here we come to the fun part of these WGAN off the air.

Move or distance. So we set a boolean variable is_EMD, or Earth Mover's distance so that we can switch between the f-divergence of Kullback-Leibler divergence and the Earth Mover's distance and we'll see how I use this boolean value during the training to fine tune the flow of the program. So the idea here is that we can run two experiments. The first experiment is_EMD equals True so that we use Earth Mover's distance and we'll see how the flow changes with that boolean change and the other experiment is where we use is_EMD equal False.

And in this case the criterion method or the loss function will be the Kullback-Leibler divergence. And again see how the flow changes and accuracy is affected in this lecture. I will show you the results for the first experiment in which is_EMD equal True. And I encourage you to set the variable to false and run the notebook and see the results yourself on your machine. Next we move to the GAN training. We defined the real labels to be plus one and the fake labels to be negative one and then we create a test set of random values which is drawn from normal distribution.

We have the batch size to be 25 so that we can draw 25 samples and see how the generator has synthesized the output for this experiment I trained for 1024 epochs. I encourage you to increase the number and try something like 10000 epochs to get a much better and fine grained accuracy. However I find 1024 a reasonable value to display and illustrate the concepts behind WGANs in experimental results. I have seen people getting much more fine-tuned accuracy and much better synthesized images using much higher number of epochs close to 10000 20000 and so on.

Then we define a variable the number of steps which is the length of the train loader divided by batch size. And here we use integer division so we loop for the number of epochs and then we loop for the number of steps that is. We loop for all the mini batches available in the dataset and then we loop for the number of steps for training the discriminator which is a hyperparameter

and instructed by the authors of the paper to be the value 5. Again you can feel free to play around with these numbers so like previous lecture we have the real images and convert them to the device.

Either CPU or GPU, and then we have the fake images which are the result of passing the noise through the generator and obtaining the output. Let's highlight the next step which is the one of the characteristics of WGANs. We said that we clamp the parameters or the weights to be in the range negative 0.01 and 0.01. We said that this is very important for the Lipschitz inequality to hold true. And so that the discriminator model will learn the loss function on its own and update its weights.

So the weight clipping. You can think of it as a condition to ensure the continuity of the function so that the Lipschitz inequality is valid and the rest of the WGAN proof is valid. In this case. So how do we clip the parameters we loop for p in discriminator parameters. That's for every learnable parameter in the network. Apparently the weights and then we say p.data.clamp that is, p.data would be the tensor of the parameter. So for example if we have a weight then that tensor of that weight would be the inner product of two hidden layers and then we clamp that that will bring it into the range negative point 0 1 and positive 0.01 using min max scalar.

Next we reset the gradients of the discriminator model so that we can compute new gradients for this mini batch. We best the real images through the discriminator and get the real outputs and we pass the fake images through the discriminator and we get the fake outputs so here, if is_EMD. That is, if using Earth Mover's distance. So how do we actually perform that Wasserstein GAN. We do that by saying real_outputs.backward of real labels and fake_outputs.backward of fake labels I elaborate more on how this works but let's continue and see the rest of the code and then I will elaborate in a bonus section how this is different from what we have done in the previous lectures and why it actually works this way.

So for logging purposes the discriminator underscore loss is a variable to hold. The loss of the discriminator which is the real outputs negative or minus the fake outputs. Again think about it this way if the real outputs are always plus 1 and the fake outputs are always negative 1 so that will return a plus 1 negative 1 which is zero in the ideal case and will return another value otherwise in the second experiment which I encourage you to try out when you use the Kullback-Leibler divergence.

So in this case is underscore M.D. will be equal to False and so the flow of the program will be directed to the else condition in which we apply the criterion just like we did in the previous lectures and we backward. The loss of both the discriminator on real data and the discriminator on synthesized or generated data and the loss again will be D of X plus D of G of Z. Here it gets more interesting as we use the TensorBoard for summary writing so I'm trying to demonstrate here different methods on the tensor boards writer.

So the first one is add_scalar, the first parameter discriminator slash gradients is a tagging that will hold the gradients at the final layer of the discriminator. We say that discriminator.fcn which is the forward network we have defined as sequential of index negative 1. That would be the last layer of that the sequential layer that with that would be the parameter we are interested to update or the learnable parameter. dot grad to be the actual value of the gradient.

dot grad to be the actual value of the gradient but then the gradient can be either positive or negative. We are only interested to see whether there is a useful gradient flow in the network. So we get the absolute value of the gradient using .abs and then we get the mean value since this is the tensor connecting the last neuron to the previous hidden layer which is 48 neurons we get the average of these values. I would encourage you to also look at the norm, either L1 norm or L2 norm and see whether it can be interpreted or observed in a different way.

The next method I would like to highlight is the TensorBoard writer add_graph that will actually report the model design so they can visualize the different layers of your model. In this case I call add_graph discriminator and pass the input parameters so that TensorBoard can actually use this data and give some nice annotations on the model. I've done it twice. This is redundant just for demonstration that you can use any one of them.

The method add_graph takes two parameters the model to be visualized and then the input data. In this case we use the first parameter discriminator the second parameter real_images and for the second case I use the fake_images. Just a redundancy to demonstrate the different functionality of this method. Finally we apply the backward pass and update the weights using discriminator_optimizer.step. So we train the discriminator for five epochs or five steps.

That's five steps more than the generator. And then we get the loss function of the discriminator for these different five steps and divide it by five so that it is averaged and log it using the method tb_writer.add_scalar with the tag discriminator slash loss the last parameter is epoch so that TensorBoard will keep track of the number and display the progression of the loss throughout the training epochs. Next we train the generator for only one step by using the vector of random noise we reset the gradients of the generator using generator.zero_grad and then we pass the noise through the generator and the synthesized output through the discriminator and we get the variable outputs.

Again if we are using Earth Mover's distance or WGAN we say outputs.backward. But here we flip the label and instead of passing fake on the score labels we pass real_labels. Next we report the scalar value of the generator loss and since we have applied label flipping we multiply the outputs .item by minus one so that we can locate the additional rate of loss last parameter is the epoch number. Otherwise if we are applying Kullback-Leibler divergence we apply the loss backward as usual in the previous lectures and again report the loss, we report the mean value of the gradient at the last layer.

Just like what we did in the discriminator but note here that the last layer is the sigmoid. So we take one step backward and visualize or log the layer before the sigmoid which is a linear layer and then we add the graph for the generator and pass the input as a sample input for annotation, the Z vector next we apply generator_optimizer.step so that the learning rate will be multiplied by the gradients and we get the updated weights so and instead of manually visualizing the output of the generator I will use TensorBoard the Visualize method every 10 epochs.

So if epoch modulus ten equals zero then generated equals generator of test_set .detach().view negative 1 so generated equals generator of the set .detach().cpu().view negative one comma one comma 28 by twenty eight which simply passes the noise to the generator and then converts it

from the GPU to the CPU and reshapes the output to be an image of twenty eight by twenty eight and one grayscale channel.

We passed that generated output to torchvision.utils.make_grid to visualize it in five rows. Since we are using 25 samples in this random set and with the padding of 10 pixels and pad value of one that is white color and we get a grid tensor object and instead of saving that file or displaying it we use tb_writer.add_image which actually logs the image to the dashboard and we give it the tag generator slash outputs and we pass the grid as the value to be logged.

And finally the epochs so that we can keep track of the progress of the generator output that will train the model and that after training you can use the last epoch or the last model that was trained after the last epoch to visualize the results once again like what we have done in the previous lectures using torchvision.utils.make_grid and then transpose the outputs so that we can get the channels at the last dimension and the width and height at the first and second dimensions.

We use matplotlib to show the image and here we have got the results. Of course we can do much better if we increase the number of epochs. If we fine tune the learning rate as per the design of our model if we increase the capacity of either their generator or the discriminator. These are all hyperparameters you can play with. I'm basically focusing in this lecture on displaying the mechanism of the WGANs and also give you some idea of the visualization that you can do using TensorBoard and PyTorch together since it is actually misconception that TensorBoard is only for TensorFlow.

Well you can see here that it also works mostly with PyTorch. The results here display digits without the problem we have talked about earlier which is mode collapse. We can see variations of images from different classes. The first image can be interpreted as two or seven and then we have got the five or six somewhere between 4 and 1 8 and 0. And as we go through the rows and columns we can see the values can be interpreted as digits the 3 8 1 9 and down there we can see some of them are 5 or 9 6 3 2 and so on.

So that's much better than the mode collapse a problem that we have encountered in previous sections and in the next section when we used deep convolutional GANs I'll show you how we can get a much more better results that are very fine tuned and very well synthesized in a very realistic way. However this is still a success especially that we are using only fully connected layers dense layers. We are not yet using convolution or transpose convolution at all.

And remember you can still improve that by increasing the number of epochs to ten thousand or so on again. If you don't have a GPU you will find a copy of this notebook available on Google Colab so you can play with CPU and even GPU and you can also reference the original source code of this WGAN. Paper published by the authors on GitHub now as a final step. I owe you an explanation why we use the output backward instead of loss.backward when we applied the WGAN training methodology so in this discussion I'll be talking more about the back propagation algorithm.

The derivatives and the chain rule to make this idea much clearer so we know that the weight update is performed as follows. We have the weight Omega or W at time T plus 1 which is actually the weight at the previous time step or the previous epoch but with a fraction of the gradient of their loss with respect to the weight this is written formally as alpha multiplied by the partial derivative of the loss with respect to the weight D of L of theta divided by D of W.

So the question now how do we calculate the partial derivative of the loss with respect to the weight. It turns out we use the chain rule. So D of L of theta. with respect to D of W equals D of L of theta with respect to D of output from the network and then this is again multiplied by D of output partial derivative of output divided by D of W. So the derivative of the loss function can be different based on the type of loss function we are using whether using root mean squared error or using Euclidean distance or using log loss and so on in case of linear function which is the case in this WGAN model we have a product w x plus B where W is the weight X is the input and B is the bias.

So the derivative of the linear function partial derivative of output divided with respect to derivative of weight will be equal to the derivative of W X plus B. With respect to D of w you can see here from elementary calculus that the derivative of W X plus B with respect to W will become x. So this is something we know. Still it doesn't really answer the question why we didn't use loss.backward and used output.backward. This would become very clear when we talk about the PyTorch autograd mechanism which we have talked about in Section 1 in decent amount of details but I will be recalling that section here.

We can observe here that in previous sections we used loss.backward. But in this time we used output that backwards by remember that the signature of the backward takes as a parameter the gradient. So it is essentially the same as saying loss.backward but loss.backward will be done twice: take the gradient of the loss, take the gradient and pass it to the backward and this will give us the chain rule when we say the partial derivative of the loss with respect to the weight.

So from Lipschitz inequality we have discussed in the slides we say that F of X minus f x2 is less than or equal to X1 minus x2. That is in the case of continuity. The value passed to the function can replace the actual function. So here and instead of calculating the partial derivative of the loss with respect to the output we simply passed minus one in case of fake samples and plus 1 in case of real samples. And this according to the mathematical proof in the WGAN paper and the Lipschitz inequality is equivalent and encourages the model to change its behavior towards outputting values in the positive range 0 to plus 1 for real samples and in the negative range 0 to negative 1 for a fake samples and it encourages it along with the weight clipping we applied on the model parameters or the model weights to learn the weights that will actually do that calculation and the approximation of the loss function which is in this case not a boolean value telling us whether a sample is only real or fake but it is actually a distance telling us.

How far is the sample from the real sample or from the fake samples and so on. So now before we conclude this section let's see the kind of visualization we have got from using the TensorBoard dashboard. I will open a new terminal and type the command tensorboard --logdir and then the log directory is dot slash runs which is a default logging directory that will give us a link, a URL,

that we can open in the browser and we should see something similar to this or different depending on how many times you run the algorithm and training the model and different trials you have tried.

So at the top of TensorBoard we have got four tabs. One for scalars since we use the method add_scalar one for images since we use the method add_image and one for graphs from add_graph and one for text from add_text. You have got four curves or four graphs one for the loss and the other for the gradient and one for the discriminator and the other for the generator so we can observe here that the gradients are not saturated as we are getting quite high values for the gradient and they are changing with time and they can be actually consistent with the curve that we have sketched in a previous lecture when we explained the vanishing gradient problem and that we should use log D of G of Z instead of log 1 minus D of G of Z.

So that's consistent over here we can see here that the loss of the discriminator is rather increasing which is a sign that the discriminator is getting fooled by the generator and it cannot really tell the real from the fake samples with time on the other hand the loss of the generator is kind of oscillating. This is not a very good sign and I encourage you to play around with the hyperparameters to make a generator loss more stable and instead of oscillating in that way.

And as a observation from different GAN papers and different GAN practitioners the generator loss cannot be stabilized by a known method since a stable generator loss will mean mode collapse and an oscillating generator loss will be very desirable property since it means that it is improving itself and shifting from mode to the other in order to find ways to trick the discriminator to believe that its output is actually real output rather than a synthesized or generated image.

However I mean here by trying to fine tune the hyperparameters to find the weights so that the overall direction of the generator loss is in the downward direction and instead of the upward direction so if we go to the images tab we can find here we've got a scroll bar and it displays the actual output of the generator along each epoch and if we scroll along the scroll bar we can see the different epochs and the different outputs. So for example at this time at epoch 1020 here we've got something at epoch 530 and we can visualize that progression of the generator as it moves throughout the training and we can see and observe whether a divergence from the overall direction has occurred to our generator and return and fine tune the parameters accordingly.

If we move to graphs then we find the generator graph if we double click on that for forward network then we can see a very detailed model design of the linear dropout of the model. Finally you can inspect the text tab to see whether the sentence we have printed has happened here. Usually it should be printed here but for some reason that cell was not executed or still loading. But now you have an idea how TensorBoard can be very useful in visualizing the performance of your network so that was it for the WGAN and the TensorBoard visualization.

The Earth Mover's distance and so on. So let me give you some ideas of some things and experiments that you can try on your own to test your understanding of the concept. So I would recommend that you try to merge the conditional GAN model along with the Wasserstein GAN model and see how you can direct the output of the generator to be of a specific label. As we have discussed in the first part of this section the conditional GAN models in the second bar the WGAN models.

So you should be able to do that with smoothness using the sample code we have got in the tutorials in the notebook. You can also try to train the model on Fashion-MNIST and it should be much easier than the MNIST dataset. Actually one of the things that I would like you to play around with so that your mind is prepared for the next section is to use convolutional layers and see how much you have to change your code in order to obtain fine results and we will discuss in Section 4 convolutional and transpose convolutional layers and how to do deep convolutional GANs another idea that you might find appealing is to play around with learning rate decay.

You can try exponential learning rate decay you can try multi-step learning rate you can try step learning rate there are a lot of variations in the PyTorch API for learning rate decay that you can experiment with. Finally try to visualize the gradients and instead of using the mean try the L1 norm, the L2 norm and try to visualize. the histogram of the gradients using add_histogram of the TensorBoard API s for the reading assignment. You can read the paper of the conditional GANs and the paper for WGANs for more inspiration and for reference to the mathematical proofs that we may have overlooked in order to simplify the ideas and illustrate the concept without too much mathematical jargon.

Congratulations.

You have made it through Section 3. That was a lot of things to learn and a lot of concepts in the next section. We get more interesting ideas about deep convolutional GANs and try with RGB color images and see much better results. Thank you and see you in the next section.

LECTURE 15

Gradient Penalty

See the notebook `9.Implementation-of-GP-WGAN.ipynb` for the code.

[Coding Exercise] Gradient Penalty Wasserstein GAN - GP-WGAN

Hello and welcome to a new lecture and this coding exercise. We are going to learn and implement the gradient penalty Wasserstein GAN or GP-WGAN. Let's get started. So we remember from the Wasserstein GAN the Earth Mover's distance.

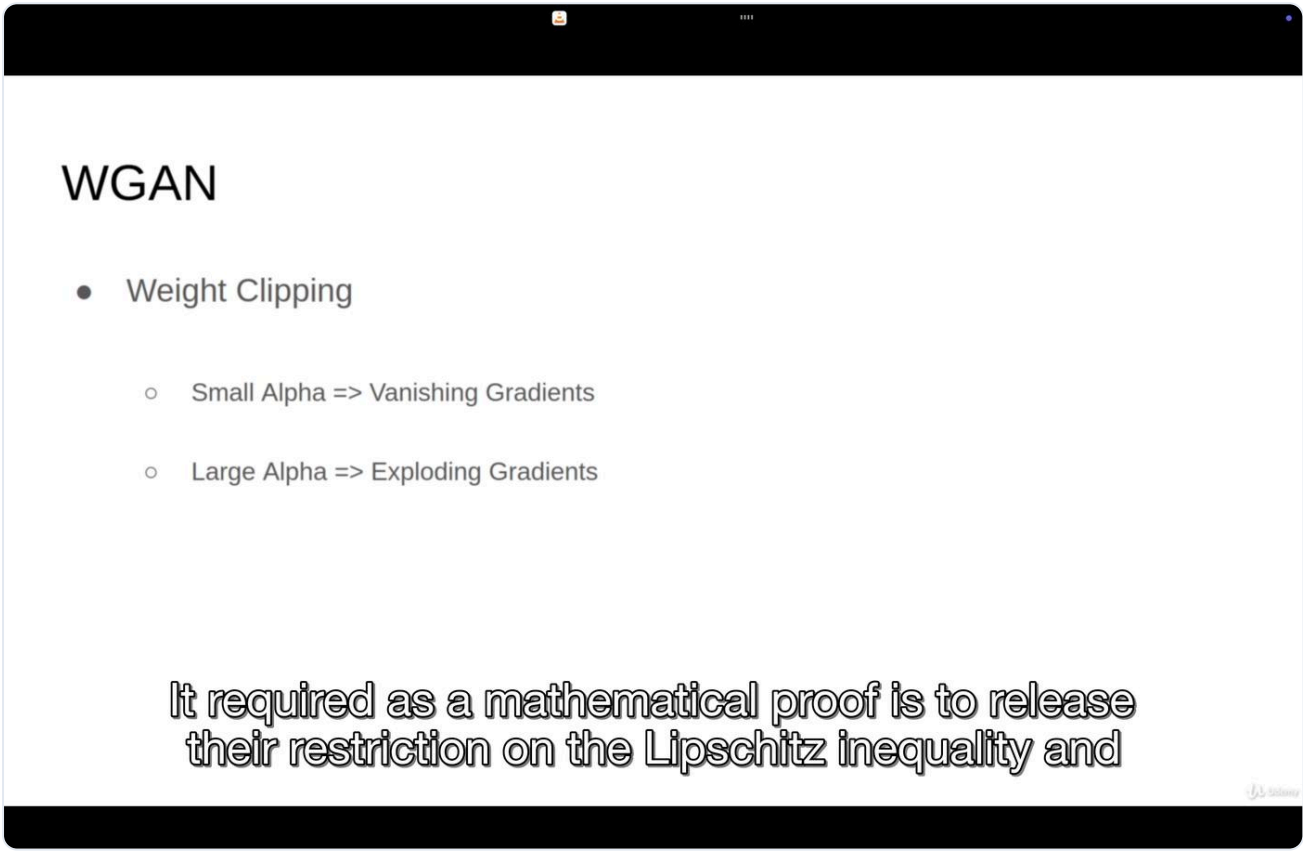


FIGURE 12.1

It required as a mathematical prerequisite to relax the restriction on the Lipschitz inequality and as a simplification and approximation of this Lipschitz inequality constraint we had to use weight clipping when we had the weights clipped in the range Alpha and negative alpha or the opposite negative Alpha and Alpha.

It turns out that the optimization of the hyperparameter alpha is very critical and it determines actually the shape and the smoothness of the gradients in the network. So if we choose a very small Alpha we are still prone to vanishing gradients problem and if we choose a larger value for Alpha we are prone to exploding gradients. Moreover if we don't use batch norm these problems tend to be more magnified or appear usually in the case when batch norm regularization is not used.

Also at times enforcing this inequality affected the performance of the model and it couldn't learn the non-linear manifolds of the data set and it couldn't express the relations and complex relations between the inputs and outputs. Remember that the weight clipping is also known as a type of regularization and remember that when you study regularization in machine learning and deep learning regularization is a form of reducing the capacity of the model.

That means the model will not overfit on the data set but if we go very far with regularization we tend to reduce the capacity of the model in a way that prevents it from learning non-linear and complex manifolds. That means it would not be able to generate realistic images and it will not be able in the case of the discriminator to tell the real and the fake images from each other. This is undesirable effect even though Wasserstein GAN is still a breakthrough in the area of generative adversarial networks and it has produced superior results to other methods.

But remember the problem of stabilizing the training of generative adversarial networks is an open question an open area of research. That's why in this lecture we introduce the concept of gradient penalty for stabilizing the training of the generative adversarial networks so the benefits we are expecting from applying gradient penalty as an additive loss term in the loss function.

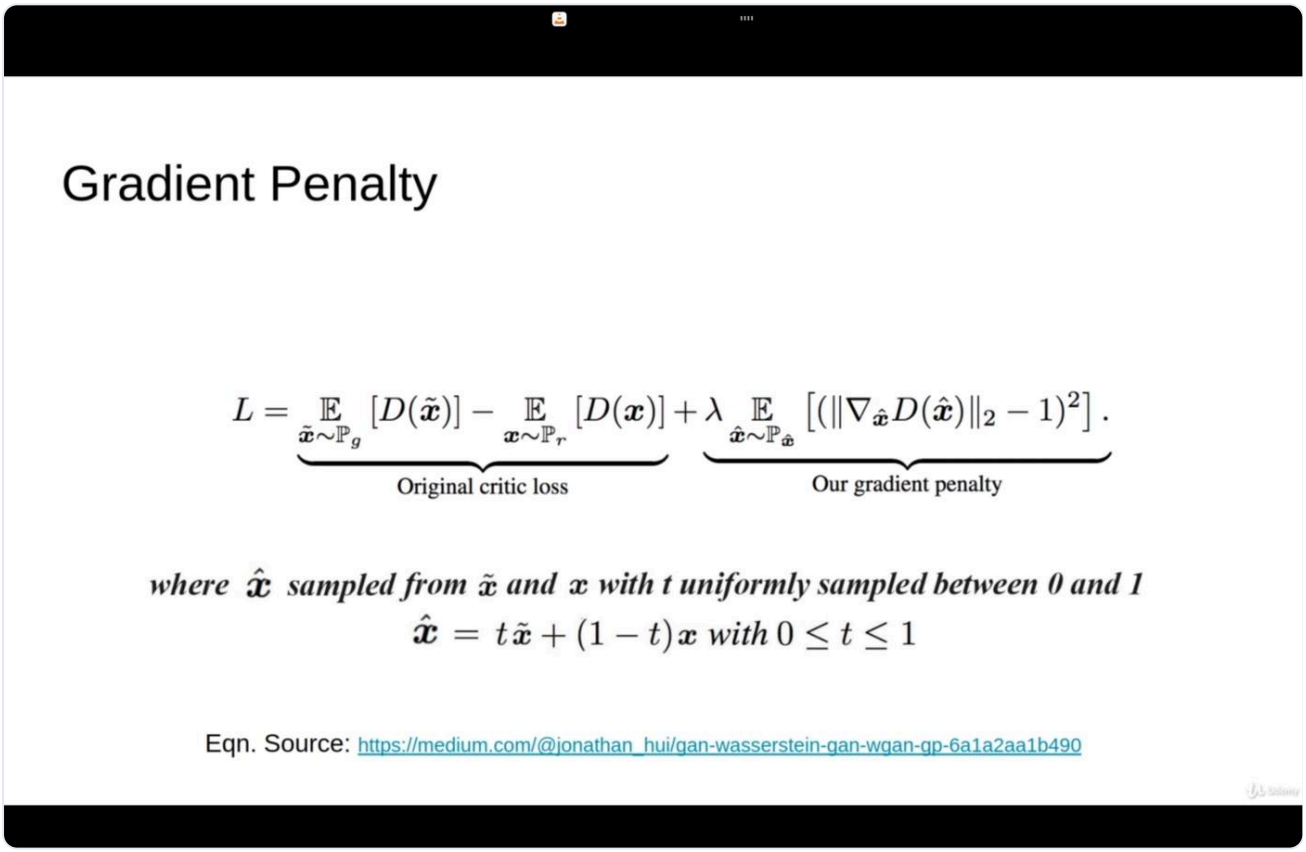


FIGURE 12.2 | Gradient Penalty

And notice how here we can combine multiple factors in the loss function a concept that we illustrate more on in the lecture where we work with multiple loss functions but for the sake of this exercise we can see you have got two terms the original GAN loss where we have the discriminator output on the real images and the discriminator output on the fake images and combining them together and that is they generated by the adversarial network loss as in the very first section of this lecture.

The second section and so on and then we have a lambda factor this lambda is a weighting factor usually is set to 10 as advised by the authors of the paper and then we have a new term this new term is a gradient penalty here we are trying to analyze the value of the gradients by restricting it from getting very far from the value one it can be proven mathematically that in order for the Lipschitz inequality to hold the norm of the gradient must not be very far from the value one hence we have here the value one and the L2 norm of the gradient of the discriminator output on the generated images and we subtract the L2 norm of the gradient minus the value one which is the ideal value for the norm of the gradient and then we square it so that it gives us a kind of squared error and then we add this as an additive term to the loss function as a direct result of this kind of modification to the loss function we expected that the training of the GAN model will be more stable and we will be sure to a certain point that the training will converge to a global minimum also the side effect will be able to use the gradient penalty WGAN on more complex models and deeper models such as ResNet recurrent neural networks and so on and finally the ultimate objective of this modification is that the network will have a higher capacity without overfitting and this will enable the network to learn non-linear complex manifold.

We apply the gradient penalty by having a real image and the fake image and combining them together by a factor alpha and one minus alpha or T and one minus T where T is a random number between 0 and 1. You can see here that the new X, X hat, equals t multiplied by X tilde or the generated image from the generator plus 1 minus t, which is the opposite value for if T is point three then one minus t will be 0.7. It is kind of random weighting to the real and generated images and then we multiply 1 minus t by the original image and this will give us an interpolated composition of the image and the generated image and this will be used in order to test the discriminator.

And also if you read the paper you will notice that sometimes the discriminator is referred to as the critic because this idea of gradient penalty and Wasserstein GAN in general is very similar to an idea and deep reinforcement learning of value function where the model is not actually outputting a binary value is a 0 or 1 but it is outputting a value from a function mapping. This function is a function that maps the inputs to a value in reinforcement learning.

So instead of having a binary classification for the output either real or fake we are having the output as a distance between the real and the fake images because the Wasserstein GAN is all about the Earth Mover's distance. We are trying to approximate two probability distributions by finding the minimum distance between them or by trying to approximate the generated image probability distribution to be as close as possible to the original dataset distribution.

So that simply that mathematical reformulation of the gradient penalty GAN, all we have to do is to have a new additive term by a lambda factor and combining the generated image and the real image with t and one minus t factors. This will become very clear as we implement it in the coding exercise.

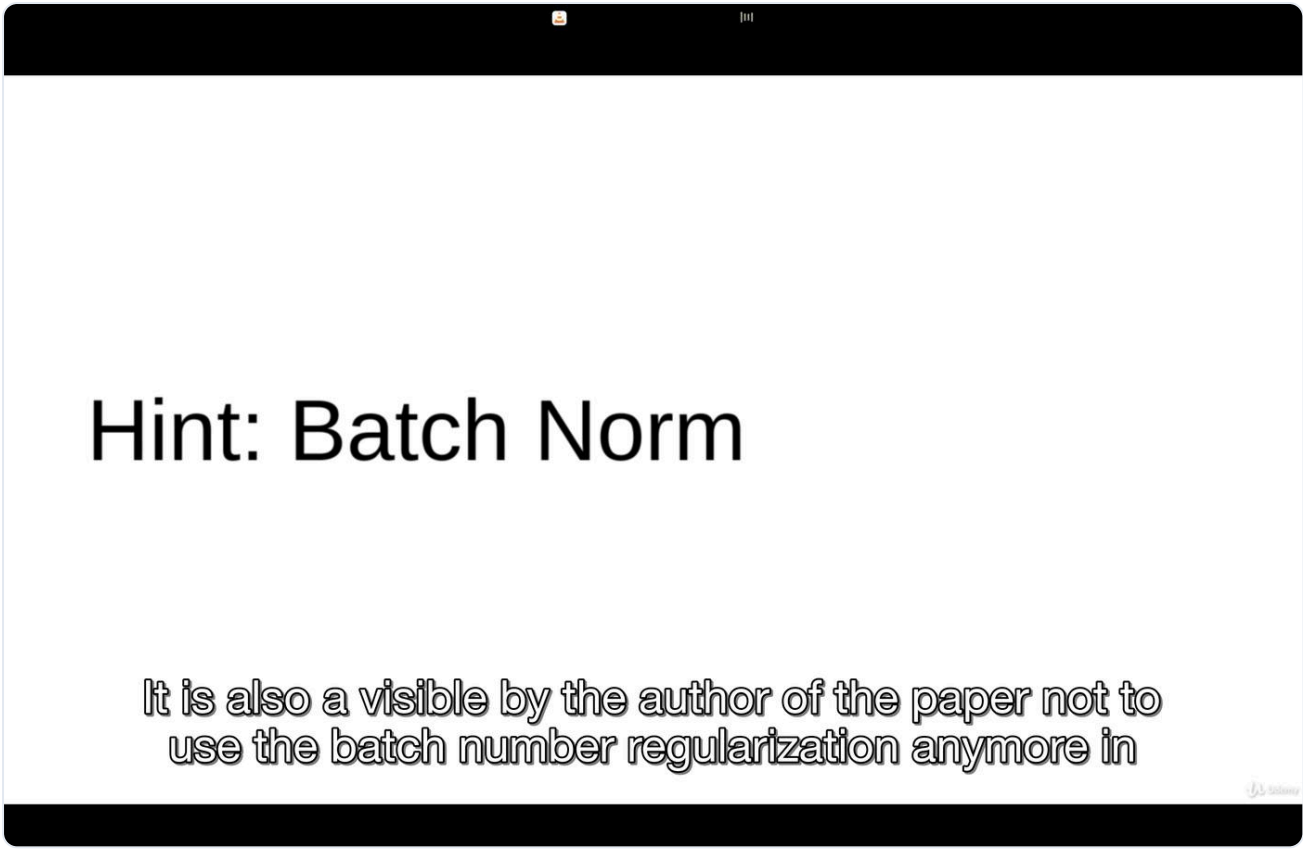


FIGURE 12.3 | Hint: Batch Norm

It is also advisable by the authors of the paper not to use the batch norm regularization anymore in the discriminator or the critic. In this case because it actually has a side effect of having a correlation between the samples in the mini batch because remember the batch norm is actually finding the mean and the standard deviation of the pixel values in the mini batch and then trying to learn a new mean and the new standard deviation so that the Mini batch along each hidden layer will have this kind of unified mean and standard deviation and because this is applied through gradient descent the mean and standard deviation are learnable parameters of the batch norm, beta and gamma in PyTorch and many other deep learning platforms.

This is not a desirable effect for Wasserstein GAN as it has the side effect of destabilizing the training. So be aware of this hint as you are implementing the code so you shall find that the coding exercise in the notebook file with the title. Section 3 lecture 5 GP-WGAN we start by checking for the GPU support then we define the global hyperparameters to be used during the training such at the learning rate the batch size, the discriminator number of steps.

And so on then we define the generator and the discriminator in a very similar way to the previous lectures. No more changes, however note here at the end of the discriminator class. We are using the sigmoid activation function since unlike the Wasserstein GAN here we are not only learning a function but we are actually applying a binary cross entropy to the output to the image and you can remove the binary cross entropy and let the network learn the function and see how the output changes.

Ideally I would vote for not using the activation sigmoid in here because as I have just said in the theory part the whole point of Wasserstein GAN is to learn a function mapping and not only a binary classification. So I would vote if you are implementing this exercise that you remove the sigmoid activation function from the output of the discriminator and see how it works with you. Next we load the data set the MNIST dataset with the flatten transform to transform the image to a 1D flat vector just like we did in previous lectures.

Here we use the Adam optimizer for both the generator and the discriminator with its suggested learning rate and beta 1 and beta 2 values. As hyperparameters suggested by the authors of the paper and they are defined above in the hyperparameters of global variables here the actual fun begins where we are using the gradient penalty loss as an input to the function. We have the real images and the generated images and along the way you find that I have a very well documented line by line code for this section so you can follow it along.

If you are following along on your screen and doing the exercise on your own let's go through the code and see what it does in the beginning of the function. I have X which is equal to torch.squeeze of X along dimension one that is simply to remove an extra dimension found in the original images because they are flattened in two shapes of one column and then under this column we have seven hundred eighty four or 28 by 28 pixels and in order to match the shape of the original images and the shape of the generated images.

I had to drop that dimension. It doesn't affect the values of the pixels. And then here I generate a random Alpha which it takes of the shape of the batch size and I give it excellent shape so that it can be edited and multiplied to the original images. X and generated images y without any change to the shape of these original vectors and then interpolate both of them X and Y using alpha and one minus alpha. Afterwards I use the autograd Variable so that I can have a variable which requires_grad so that I can have a variable which requires_grad.

That means it can have and calculate the gradients in the graph on that variable and I apply that to the interpolated variable and I use it. Var underscore interpolated next I pass this combination of the original and the generated images as var interpolated through the discriminator and I get the outputs and then I use another method from the autograd.grad which actually calculates the gradients. And here it has many parameters. I give it the outputs which is the output from the discriminator.

I give it the inputs which are the combination of generated and real images and then I give it a placeholder for the gradients which is tensor of ones and then I pass some more parameters to retain and create the graph. I set them as true and only inputs as false so that it calculates the gradient along the whole path. And then I use the square brackets of zero so that I can get the gradients of the inputs. And this way it will be accumulated along the full path of the network or the discriminator network.

Next I can easily apply gradients.norm I apply the L2 norm along the first dimension and I get the mean norm and at the next line I use gradient Norm minus one which is actually the target ideal value for the gradient. The value 1 and I square it as stress as the X2 that's a square and I return

the gradient penalty afterwards I will use the binary cross entropy criterion for the ordinary training of the network as as generated by the adversarial network and proceed with the training by having real and fake labels.

I have an iterator. I have number of epochs and I iterate for the number of epochs, I iterate for the K steps for the discriminator. I have some real images and I have some fake images. I passed them through the discriminator. Get the real outputs and the fake outputs and then calculate the binary cross entropy loss on the real outputs versus the real labels and the fake outputs versus the fake labels and then I combined them both gan_loss equals D of G of Z minus D of X which is actually the same formula in the slides before.

Note here that in the previous sections we use the addition D of G of Z plus D of X but in the gradient penalty WGAN we do a subtraction and then I calculate the gradient penalty using the real and the fake images. I combined them using the lambda value and as regularly I do, the loss backward, optimizer step, I log the loss and then divide by the number of steps to get the average loss and then I divide by the number of steps to get the average. Afterwards I'll do the same for the generator except that I will not have the real values as a parameter in the loss function of the GAN let's scroll through to it you can see here again this is merely the criterion the binary cross entropy will be applied on the output from the fake images which is supposed to be real labels and added by the lambda value again to the gradient penalty and green and fake images of light backwards step and log the loss and it goes on.

I have trained it only for 10 epochs because I was merely interested in demonstrating the mechanism by which the gradient penalty WGAN works. I'll suggest that you'll fine-tune the hyperparameters and train for longer epochs and see how fine the results are. After you go through the coding exercise and implement it on your own and fine tune it you might find that the performance is not really superior to WGAN or in future sections when we apply DC-GANs and progressive growing GANs.

However the promise that gradient penalty holds in its folds is that it stabilizes the training. It helps us to achieve a convergence to the global minima of the loss function. That means regardless of the value it scores on the benchmark in terms of the quality of the generated images. But it gives us a promise a potential that the training is more stable which is actually an inherent problem in GANs in general. So don't be disappointed by the results.

This technique can be and potential for an open question in the area of GANs how to stabilize the training I have added here the reference to the original paper so they can reference it and also don't forget to reference the blogs published by Jonathan here. I have a reference to these blogs in the future sections and it addresses these ideas in nice details and with a lot of illustrations. I'll be looking forward to seeing the results you have obtained using gradient penalty WGANs in the forums of this course.

Thank you and see you in the next section.

LECTURE 16

Introduction to DC-GANs

Hello and welcome to Section 4 where the fun begins in this section you are going to get your hands on DC-GANs and auxiliary GANs.

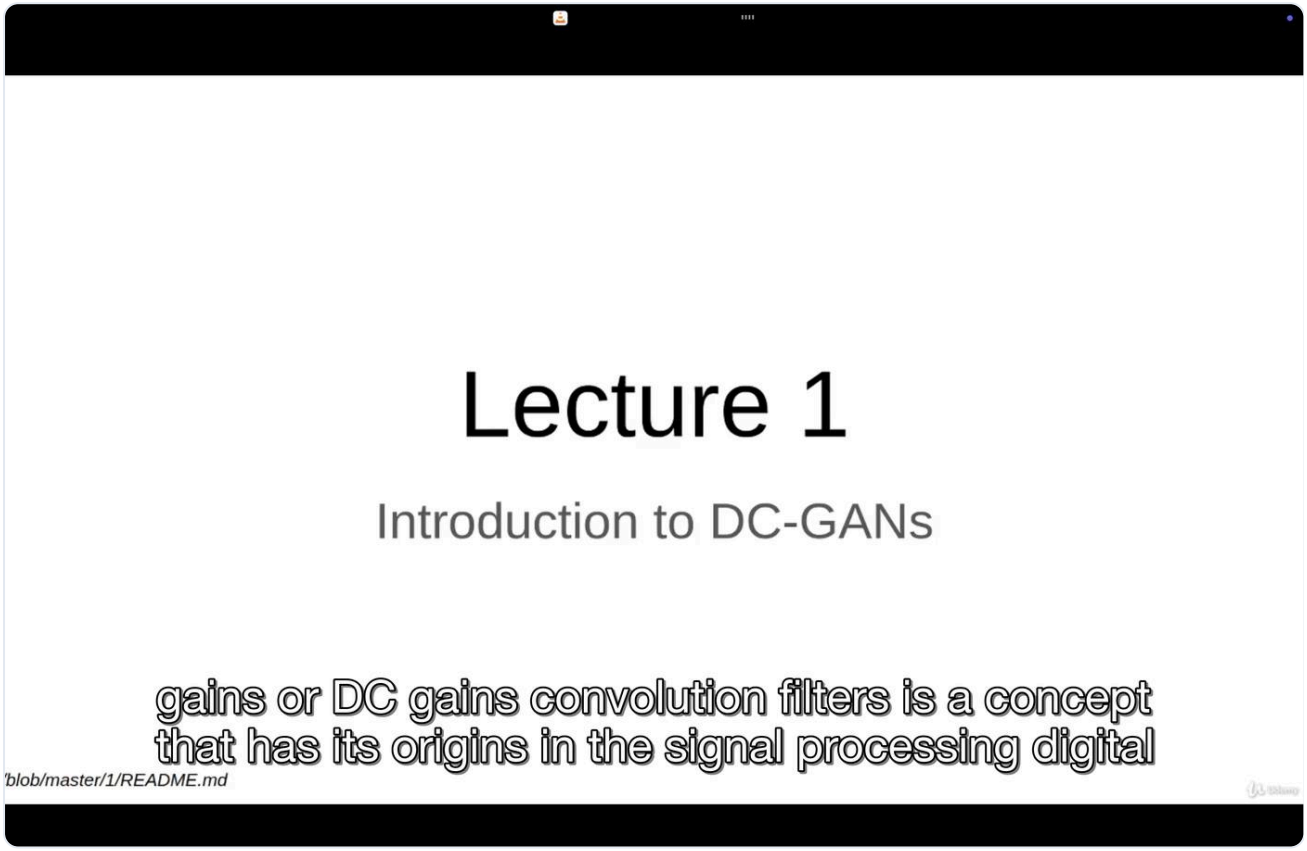


FIGURE 13.1

Let's get started in the first lecture I shall focus on understanding the concepts behind deep convolutional GANs or DC-GANs convolution filters is a concept that has its origins in the signal processing digital image processing and it carried on to be the cornerstone of the modern deep learning revolution. The convolution filter is slid over every pixel of the source image and product is calculated to find out the new pixel value in that location.

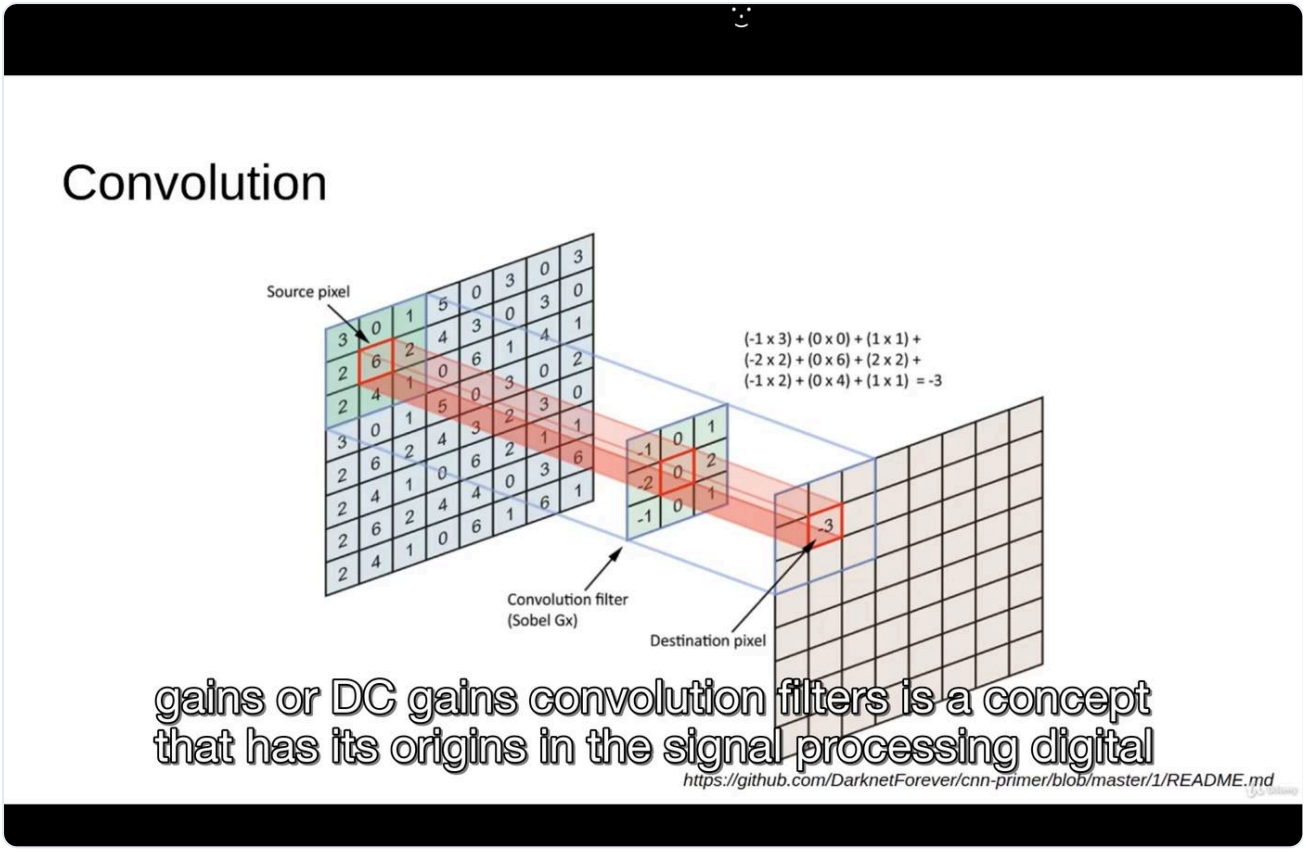


FIGURE 13.2

The values in the filter are initialized with random values and then the optimal values are learned along with the network parameters by back propagation.

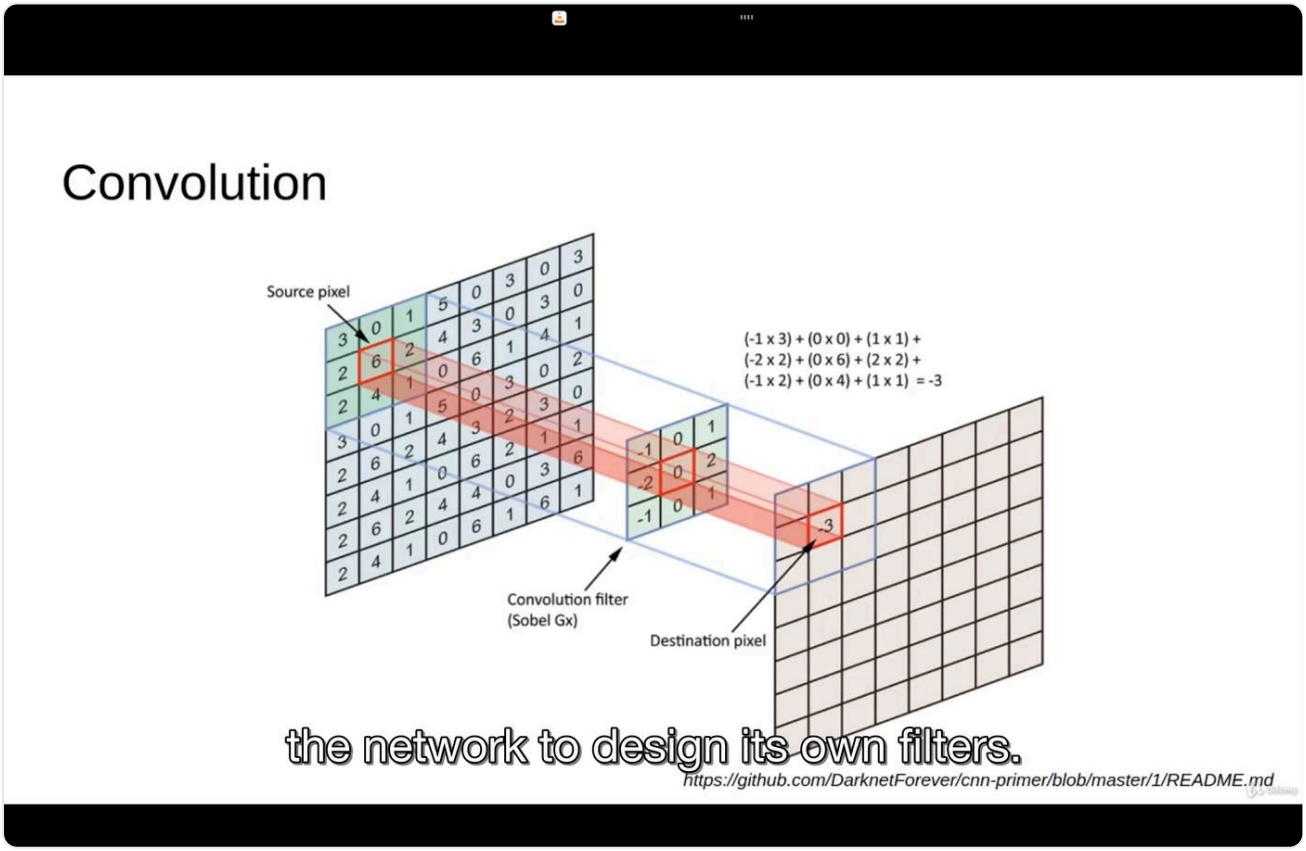


FIGURE 13.3

Unlike digital image processing where one designs the values of the filter on his own to perform a specific task such as blurring an image or extracting edges in convolutional networks we allow the network to design its own filters. It starts to learn useful features such as edges corners and shapes the opposite operation of convolution is the transpose convolution.

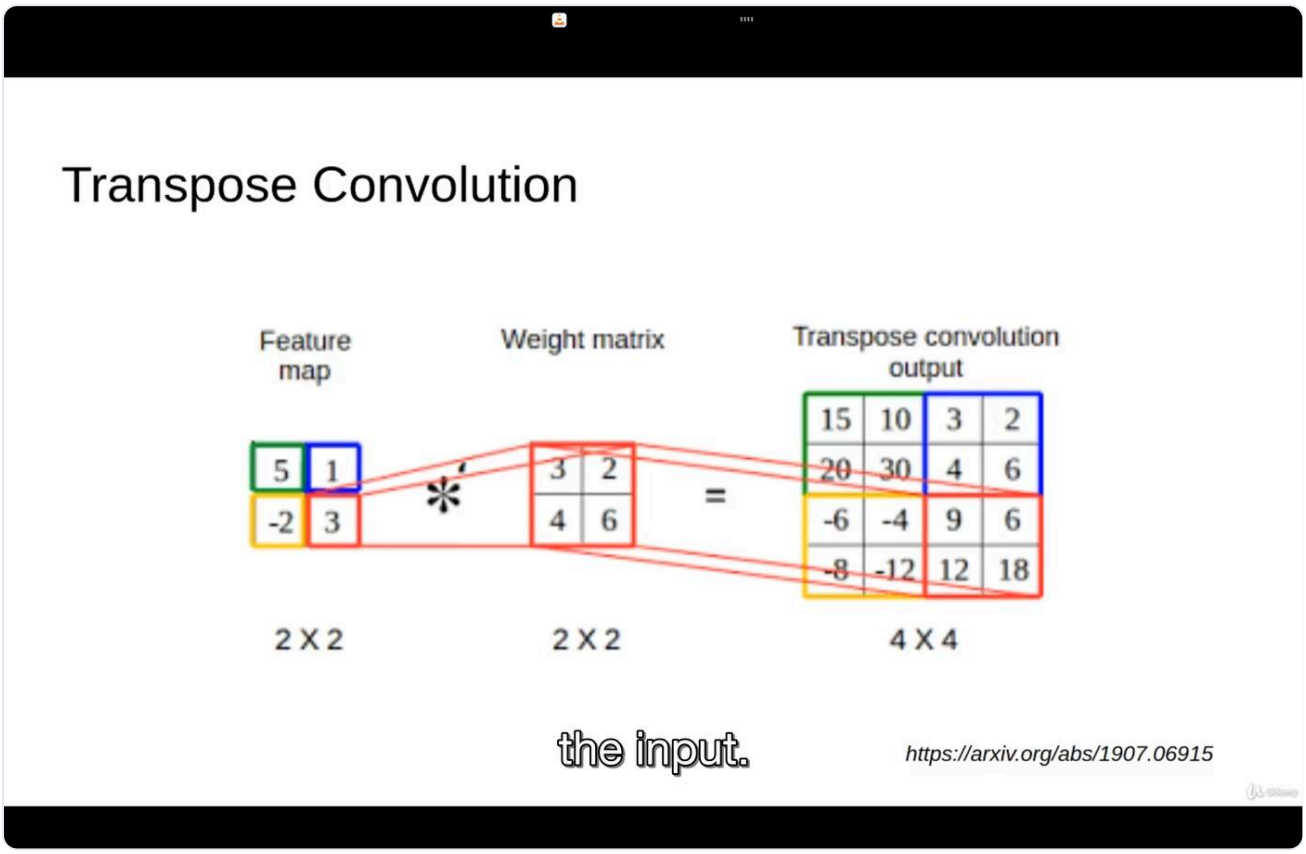


FIGURE 13.4 | Transpose Convolution

The process is reversed and the resulting matrix is larger than the original one for every pixel of the input. The filter is applied to produce a region equal to the size of the filter usually convolutions are followed by a max pooling layer to perform dimensionality reduction.

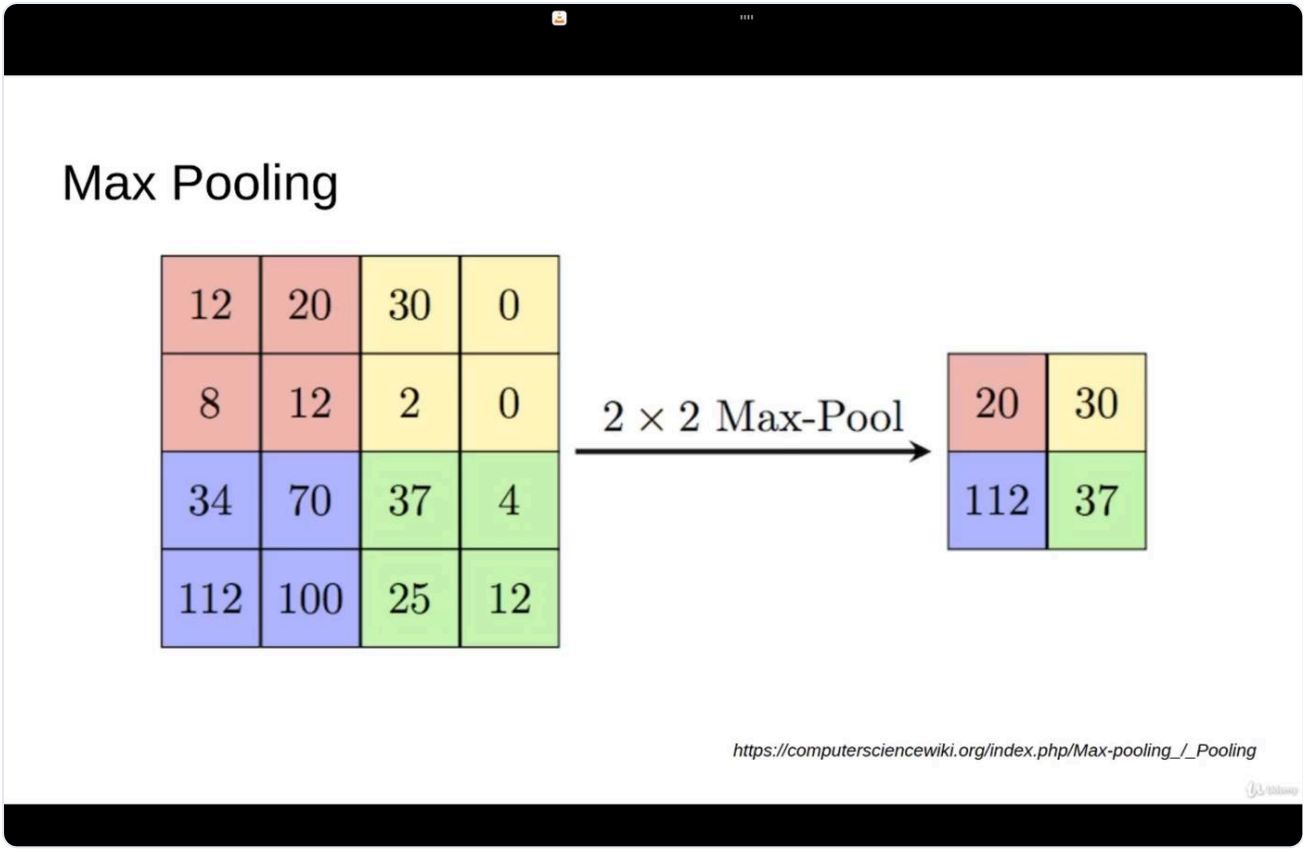


FIGURE 13.5

For example a max pooling of two by two results in shrinking every four pixels to only one pixel. The value of the new pixel is the maximum value in the original two by two region again.

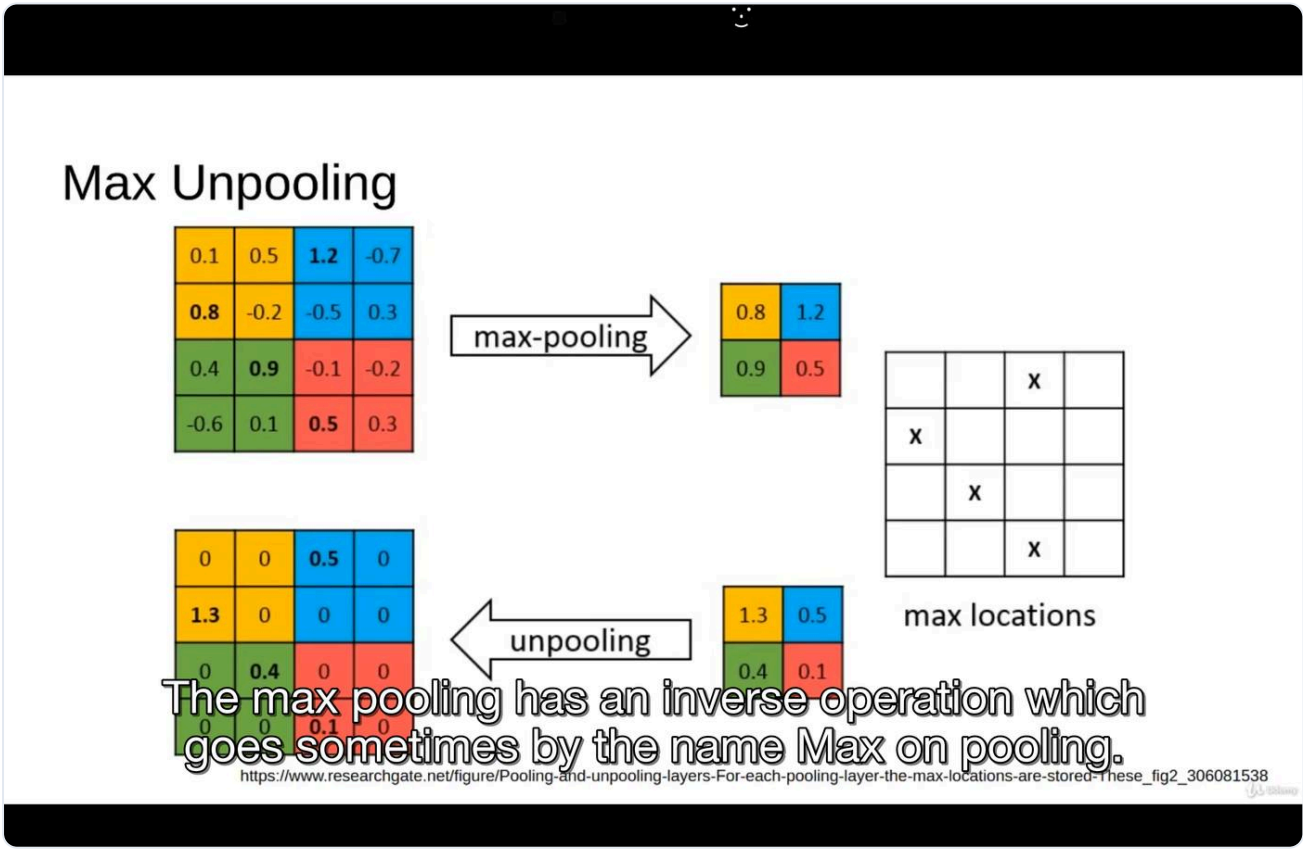


FIGURE 13.6 | Max Unpooling

The max pooling has an inverse operation which goes sometimes by the name max unpooling. The model has however to record the locations of the max values so that it can fill them back. That is while doing Max pooling the location of the max value is preserved. Then when applying max unpooling that location is filled and the others are set to zero. There is a variation of this approach which fills all the pixels with the same value.

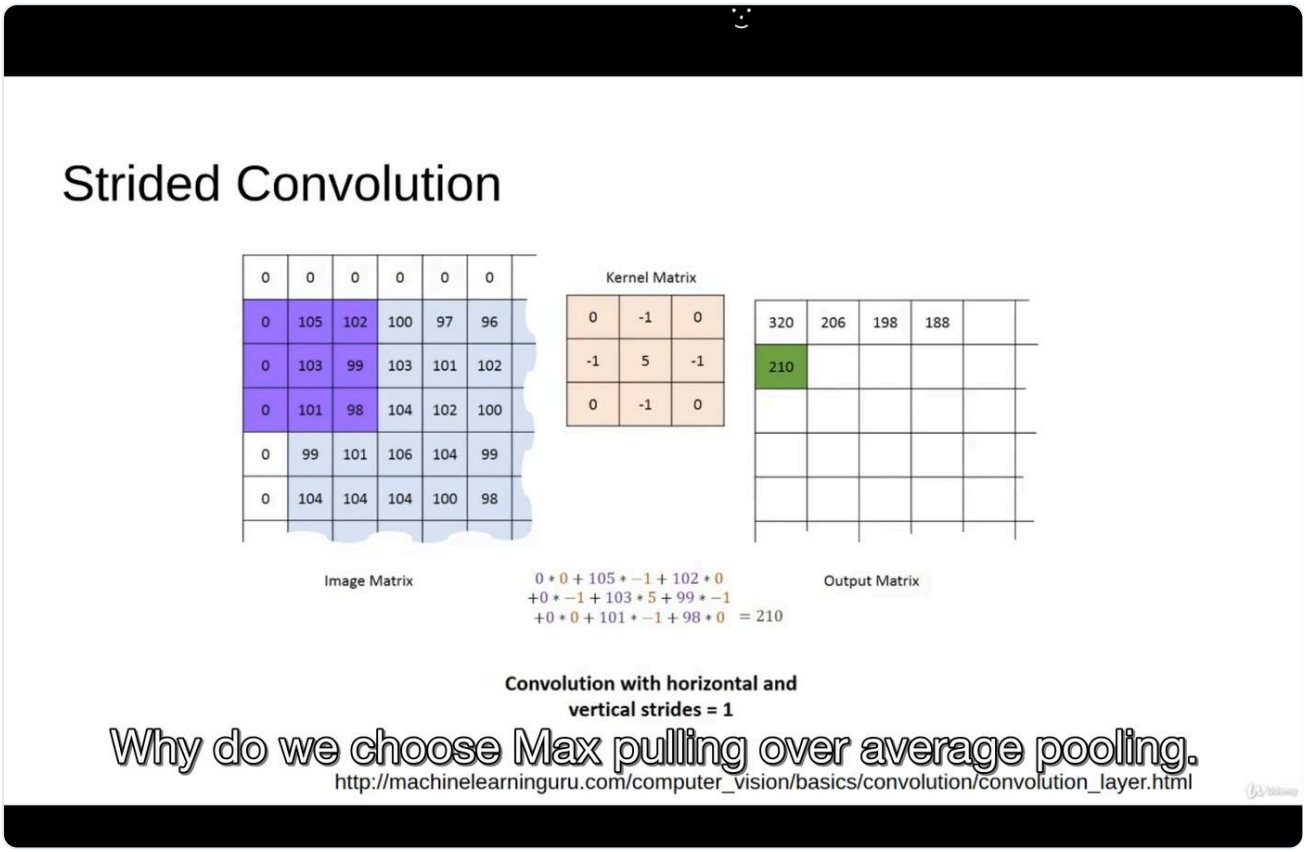


FIGURE 13.7 | Strided Convolution

Why do we choose max pooling over average pooling.

Why do we hard code this operation.

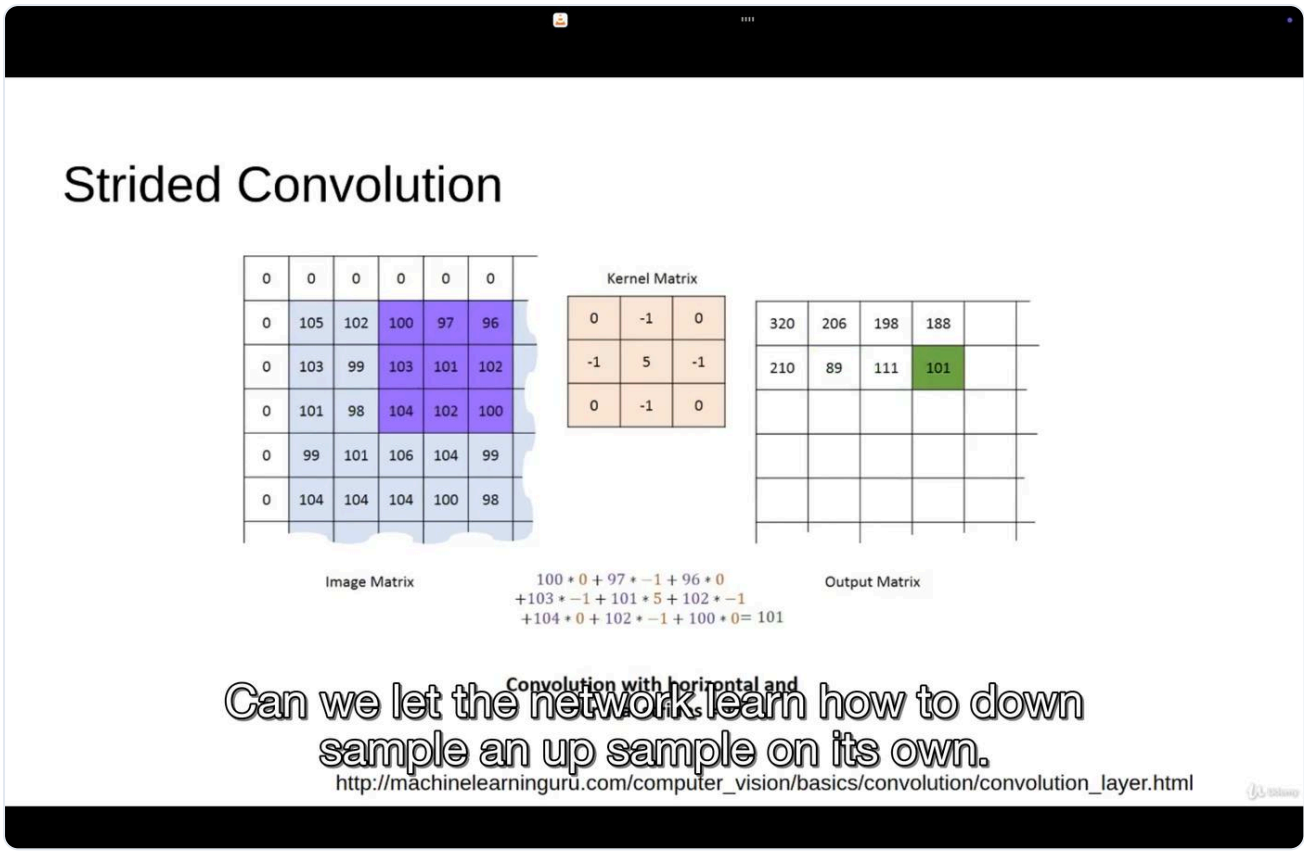


FIGURE 13.8 | Strided Convolution

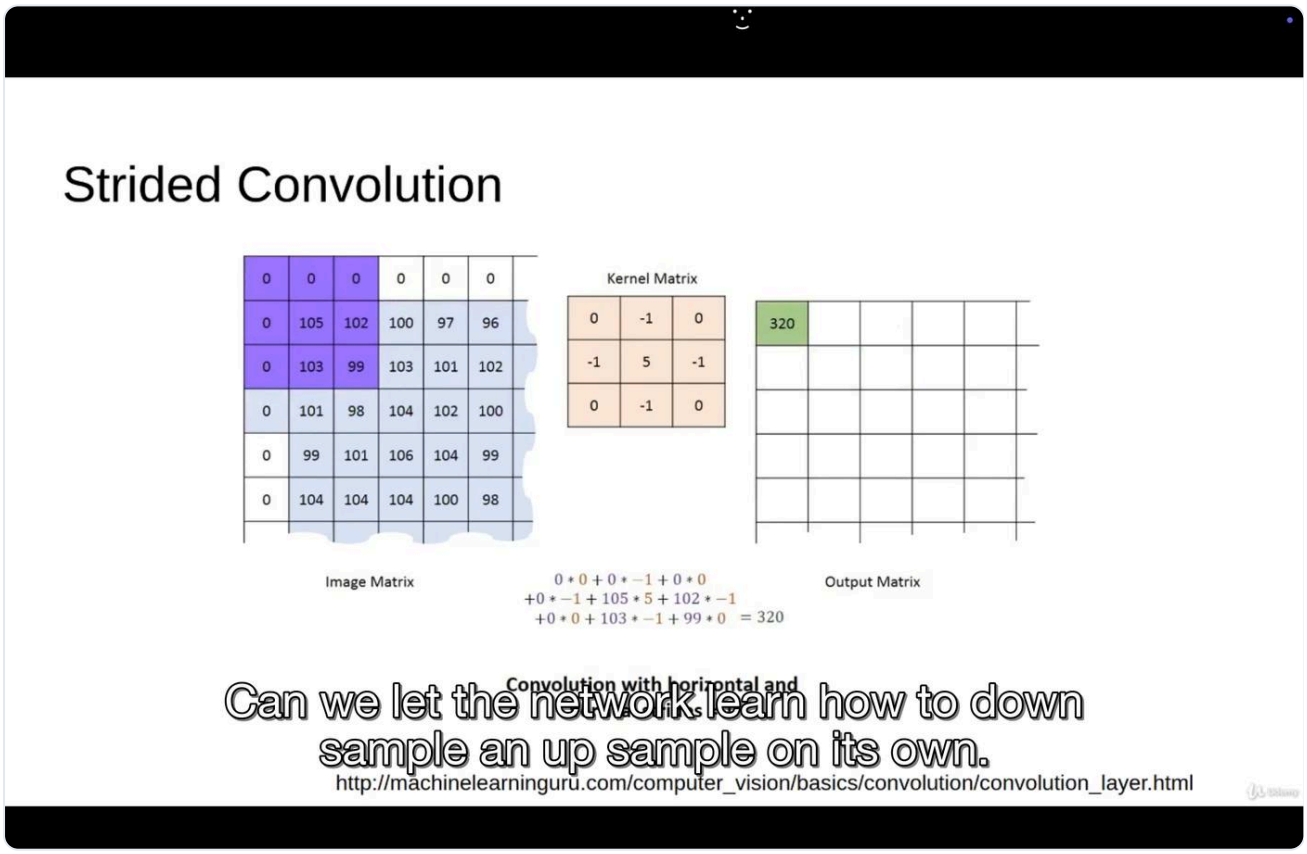


FIGURE 13.9 | Strided Convolution

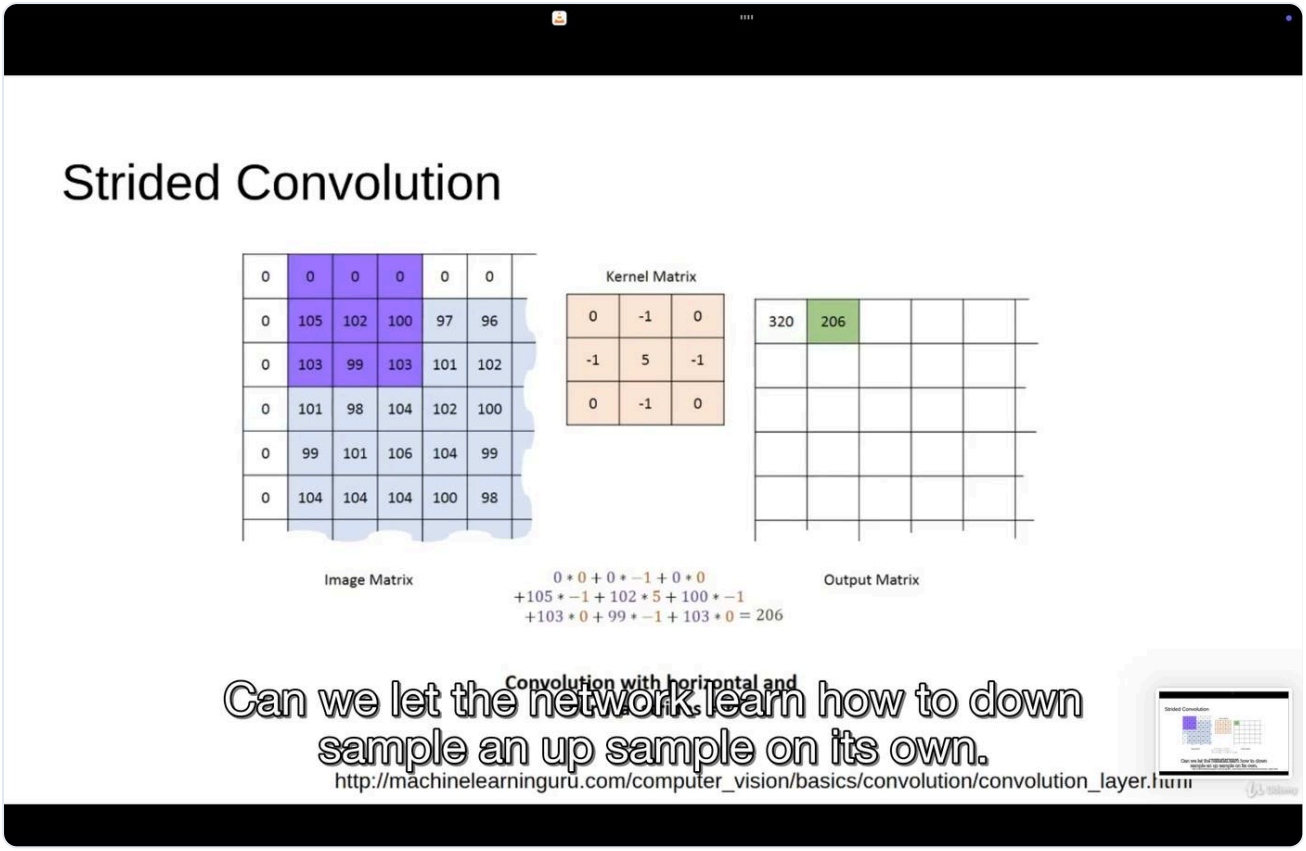


FIGURE 13.10 | Strided Convolution

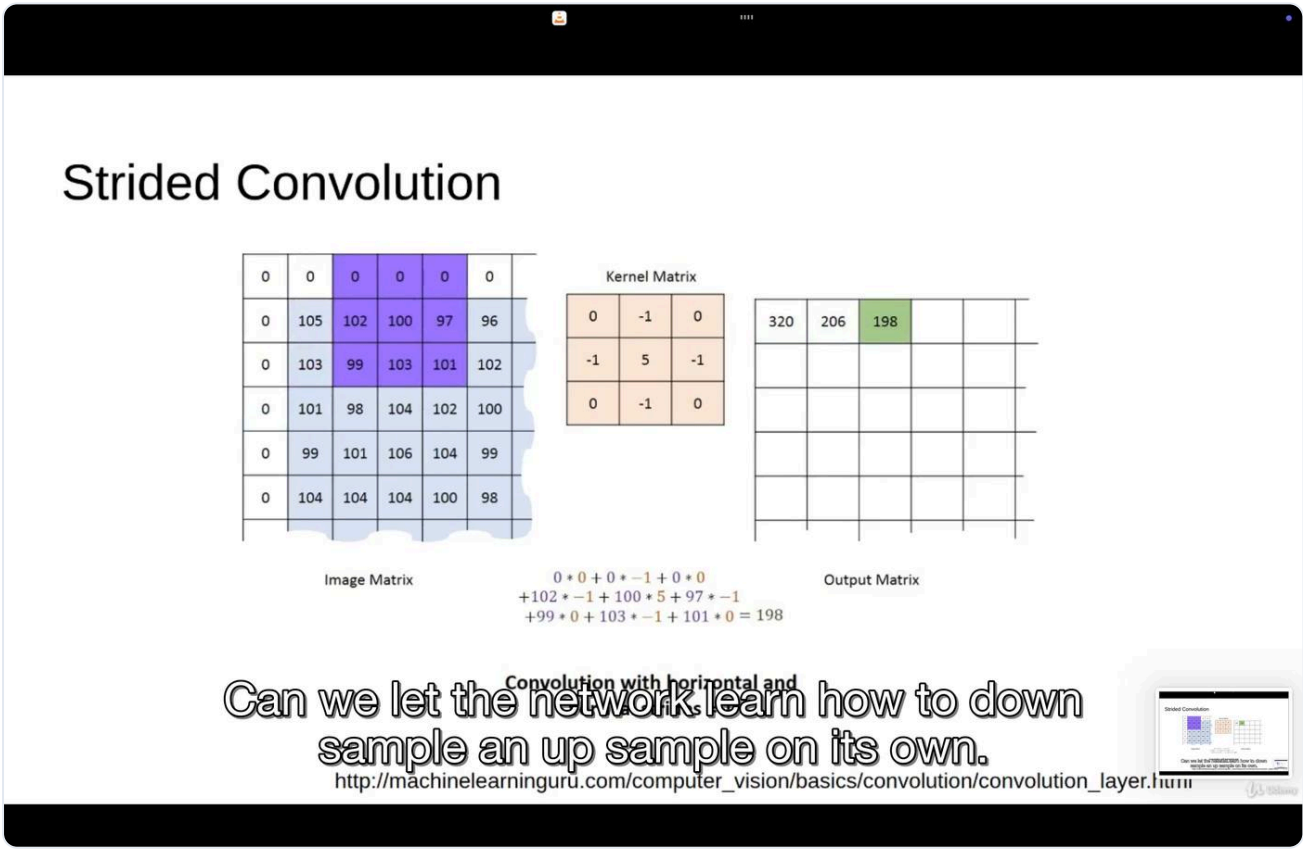


FIGURE 13.11 | Strided Convolution

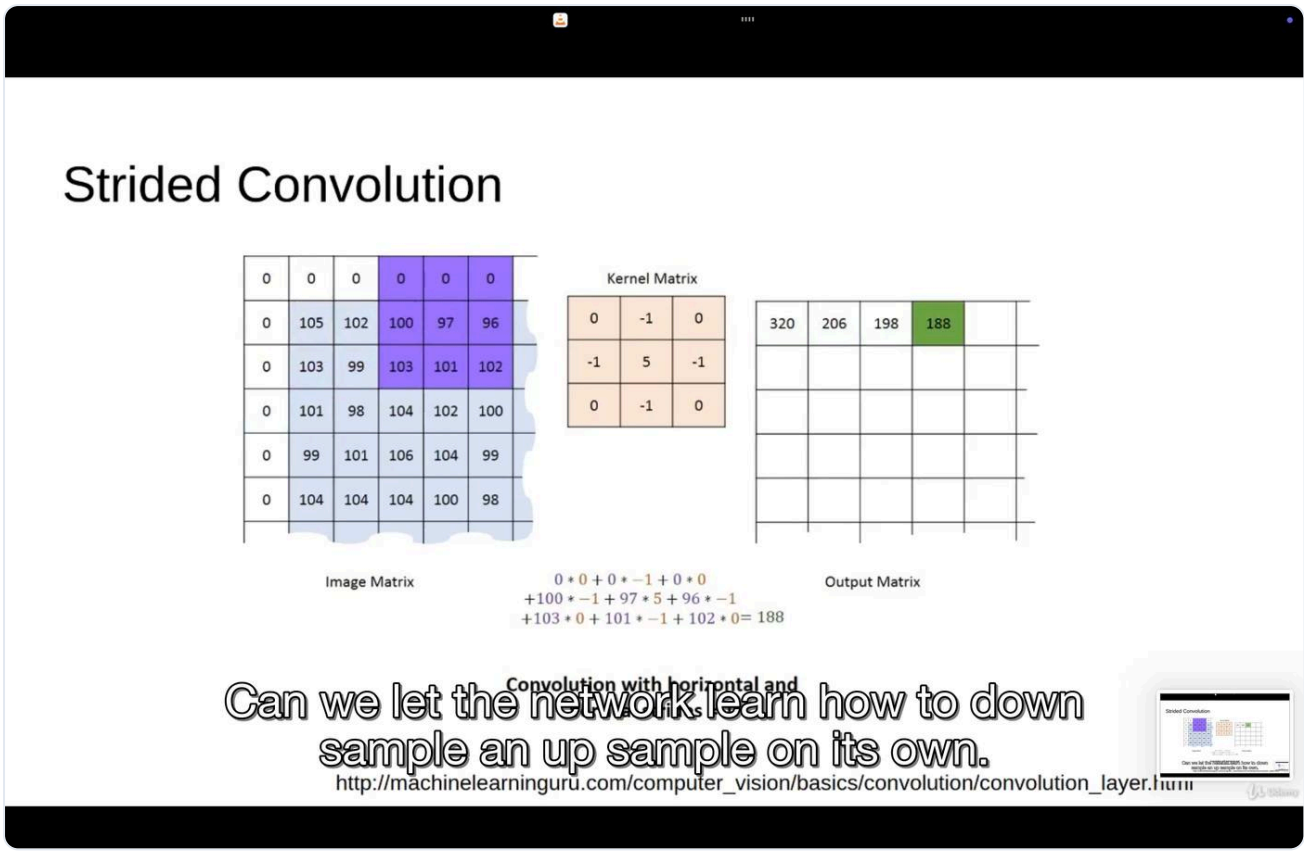


FIGURE 13.12 | Strided Convolution

Can we let the network learn how to downsample and upsample on its own.

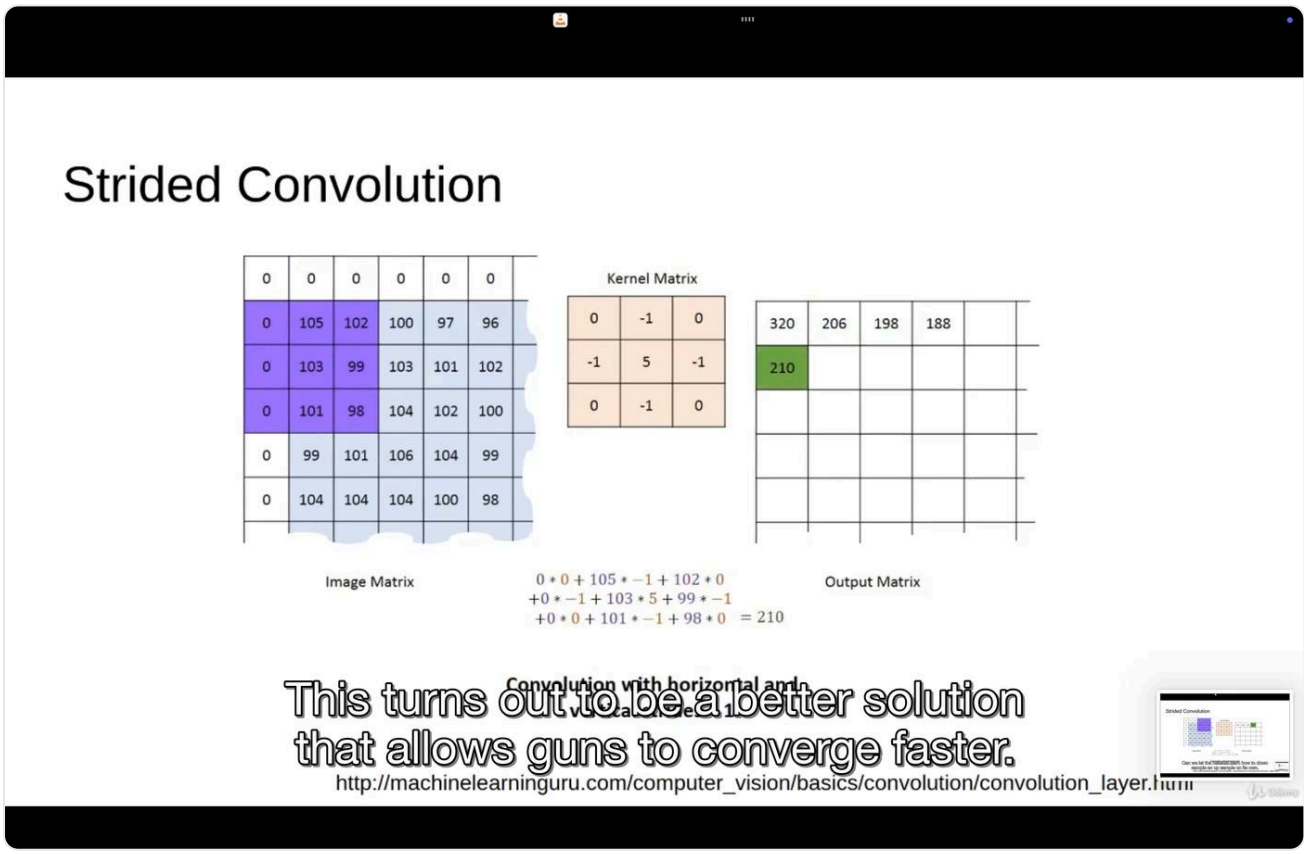


FIGURE 13.13 | Strided Convolution

This turns out to be a better solution that allows GANs to converge faster. One of the parameters of a convolutional layer is a stride or the step. If we slide the window one pixel at a time then this is called a convolution of stride equals one. On the other hand if the window is slid two pixels at a time along the horizontal and the vertical directions.

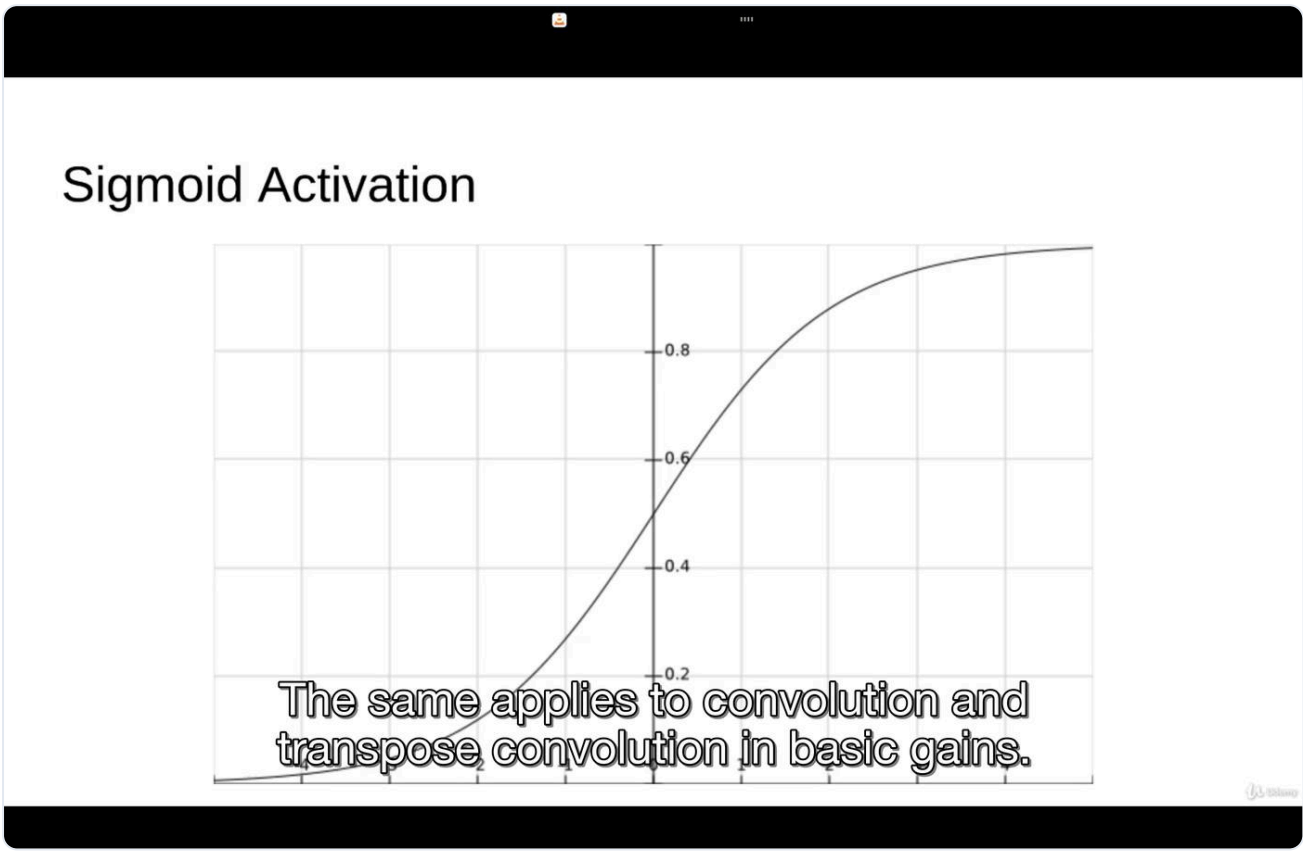


FIGURE 13.14 | Sigmoid Activation

This is called the convolution with a stride equals two or more commonly strided convolution. It turns out that this technique allows GANs to learn better downsampling and upsampling filters. The resulting images this way are going to have fewer artifacts. The same applies to convolution and transpose convolution in basic GANs.

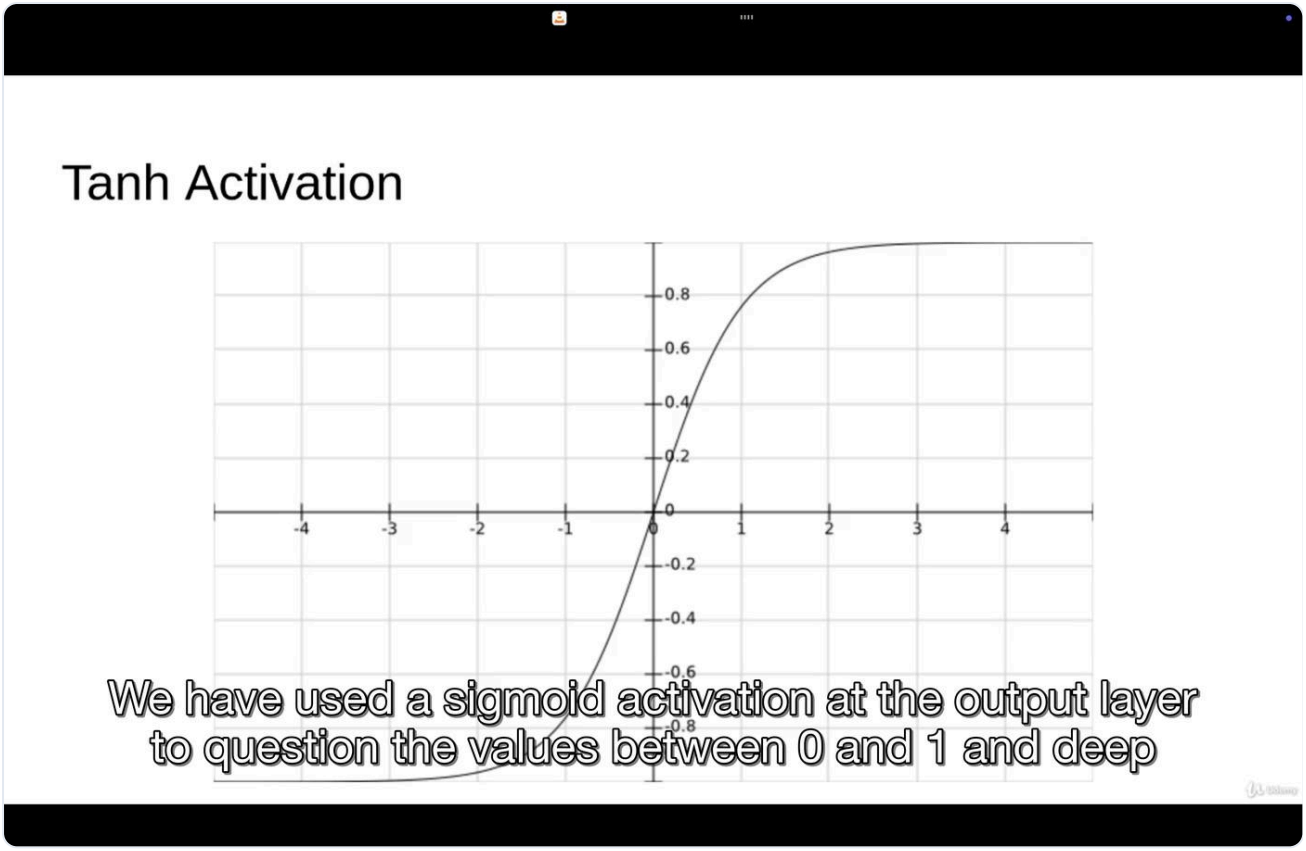


FIGURE 13.15 | Tanh Activation

We have used a sigmoid activation at the output layer to squash the values between 0 and 1 and deep convolutional GANs.

It is recommended to use tanh activation. This will result in values squashed between negative 1 and plus 1.

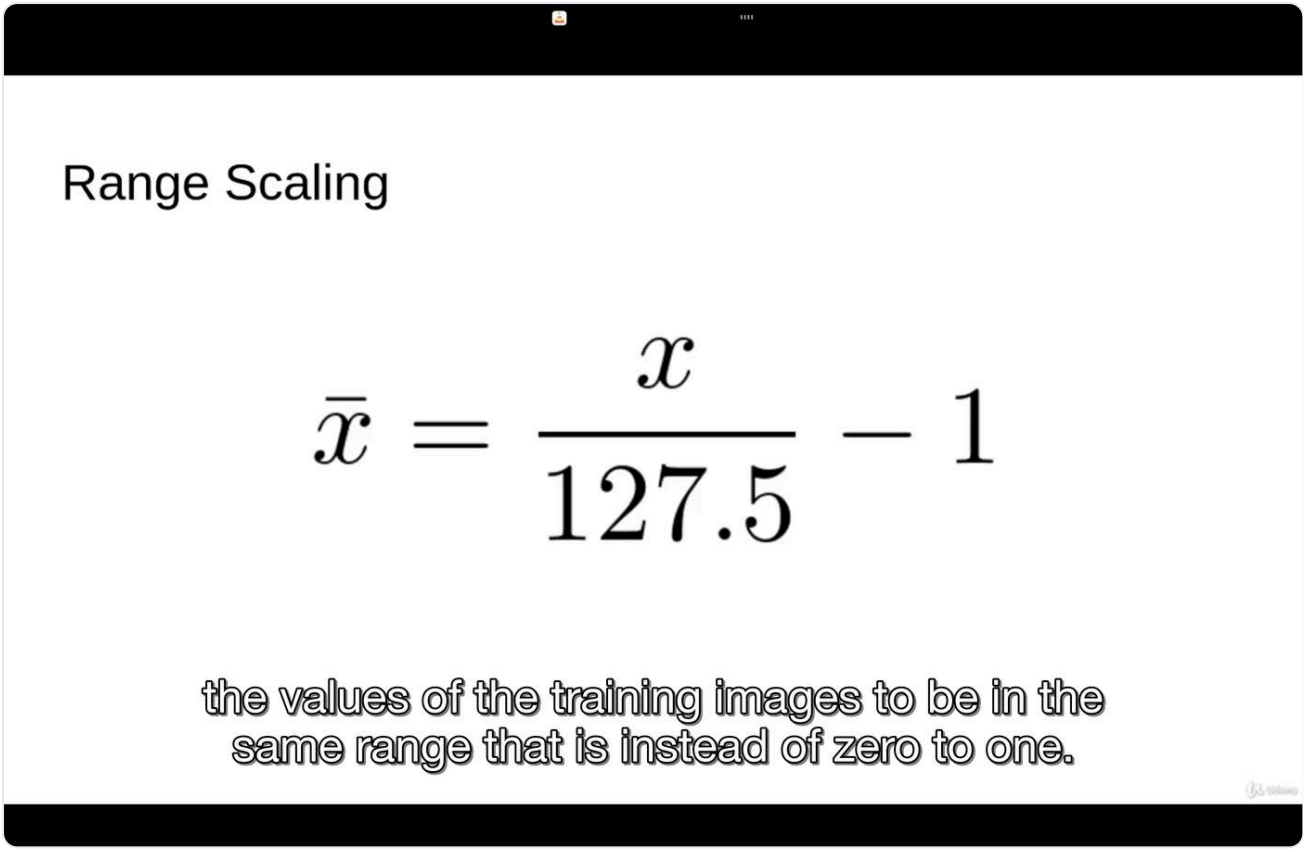


FIGURE 13.16

It was found empirically that this produces more appealing results however we should not forget to scale the values of the training images to be in the same range that is instead of zero to one. It should be also between negative 1 and plus 1. This equation does exactly that.

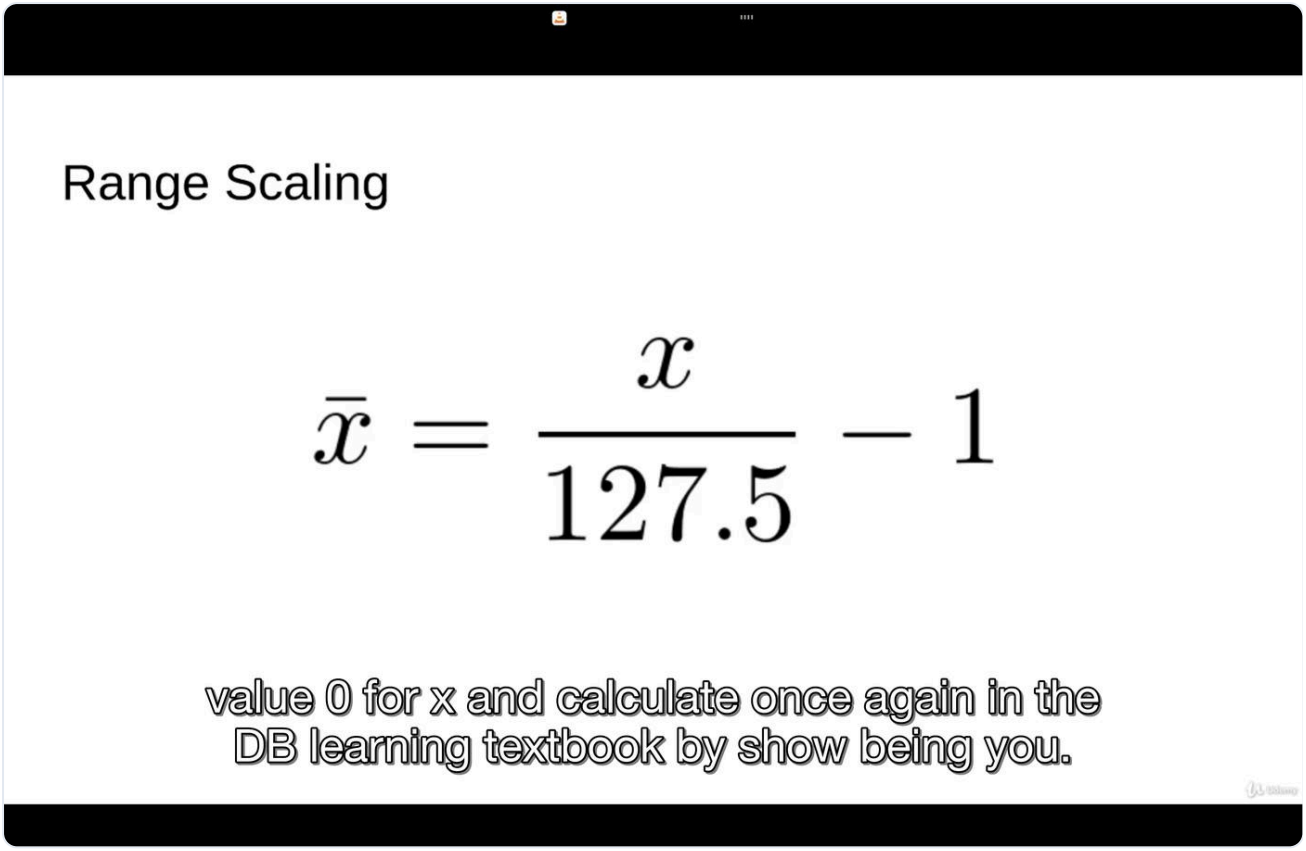


FIGURE 13.17

You can verify that by substituting 255 for X and calculate the new x value and again substitute the value 0 for x and calculate once again in the Deep Learning textbook by Yoshua Bengio.

There is a dedicated section to representation learning. It is one of the interesting concepts of deep learning. So let's take for example AlexNet.

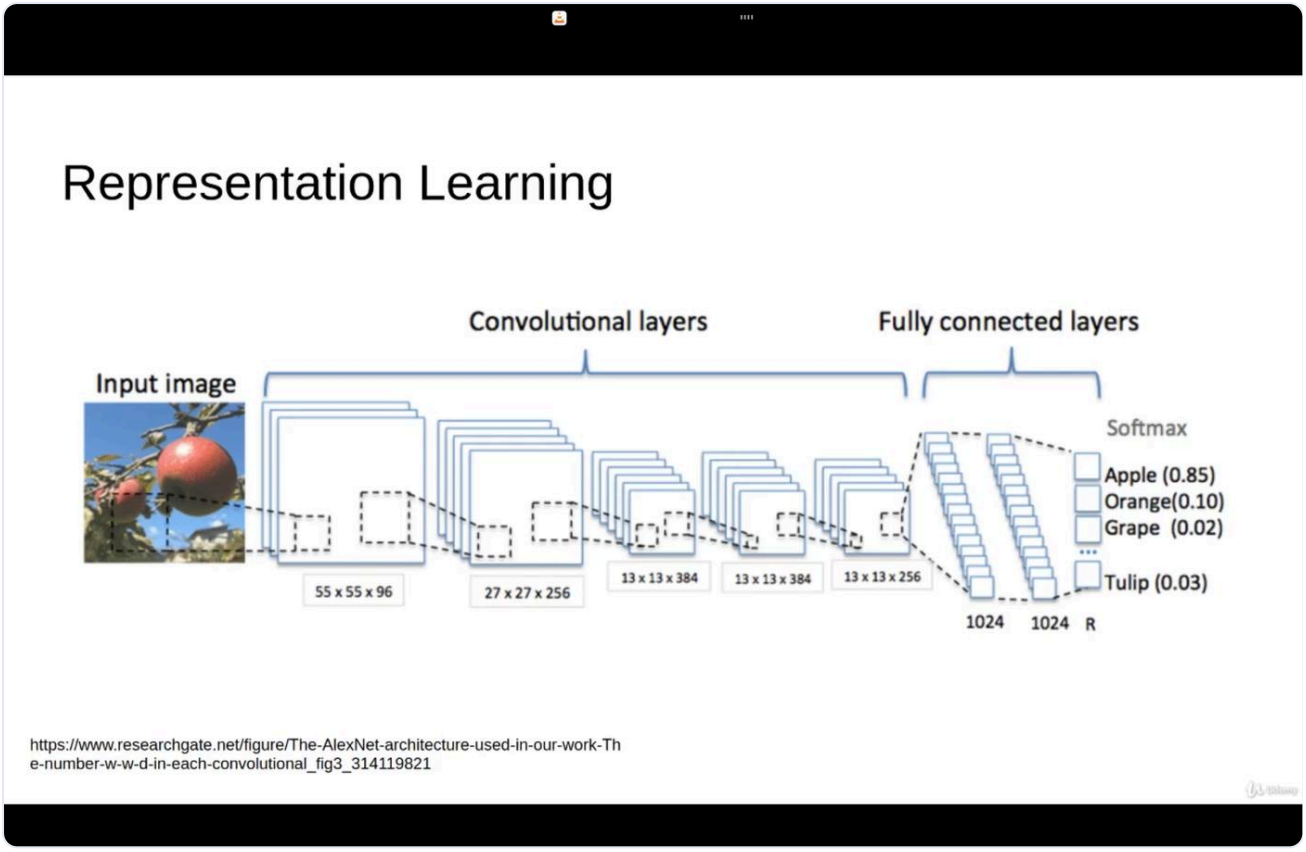


FIGURE 13.18 | Representation Learning

Architecture in which the network consists of convolutional blocks followed by fully connected layers. The convolutional blocks function as feature extractors learning interesting patterns such as shapes edges and corners The convolutional blocks also learn these patterns while preserving the hierarchical topology of features.

For example if the network is trained on cropped human faces then it will learn that the eyes and the nose will be located inside a face. And so on the output of the convolutional block is called a feature map or a vector representation of the input.

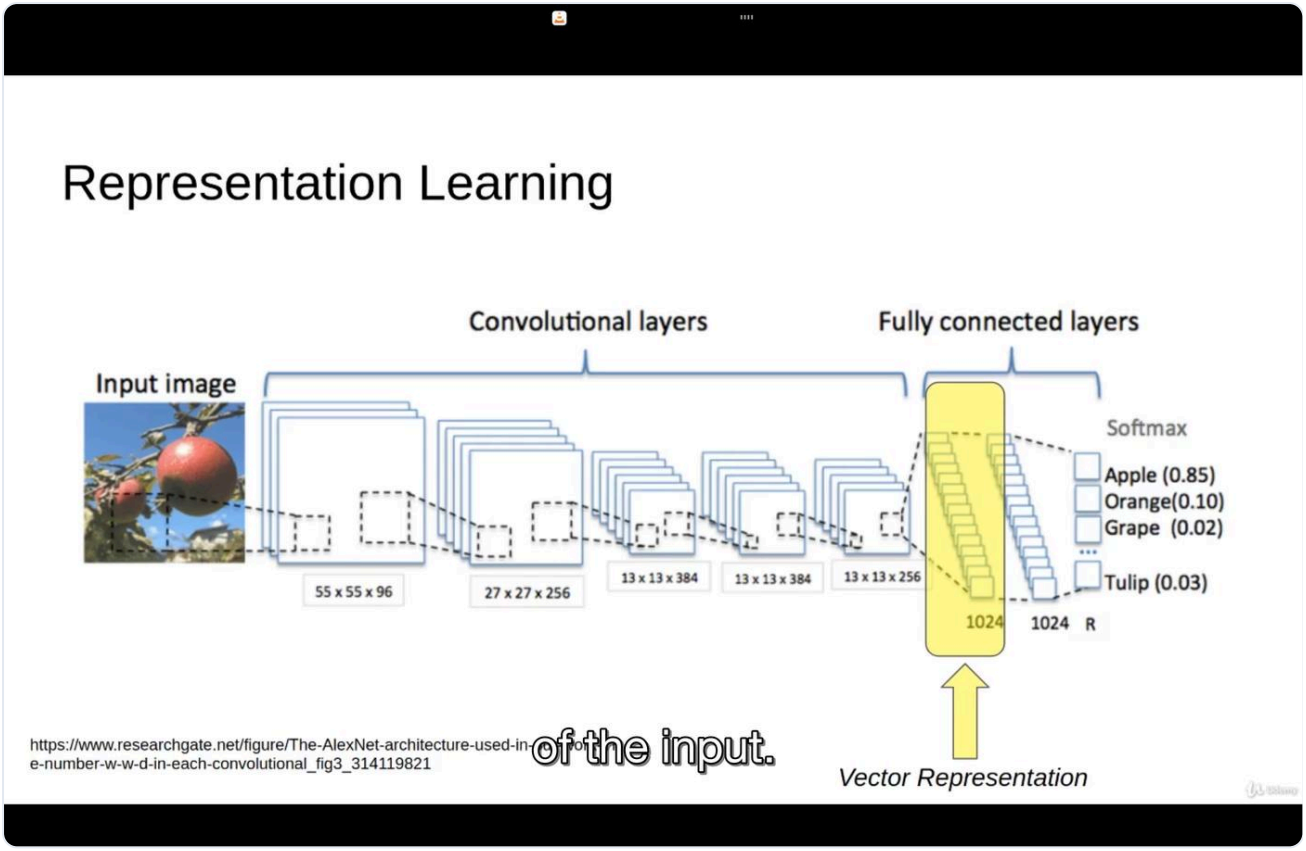


FIGURE 13.19 | Representation Learning

For example in AlexNet the first dense layer of 4096 neurons will be a valid representation for ImageNet dataset samples the same idea extends to autoencoders where the output of the encoder serves as a compressed version of the input.

It turns out that the output of the discriminator in the DC-GAN architecture serves the same purpose. That is the discriminator can learn useful features from unlabeled data sets in an unsupervised learning. This is quite useful in cases where there are not many labelled samples available for training in a paper published by Mikolov on learning feature vectors or word embeddings for natural language processing.

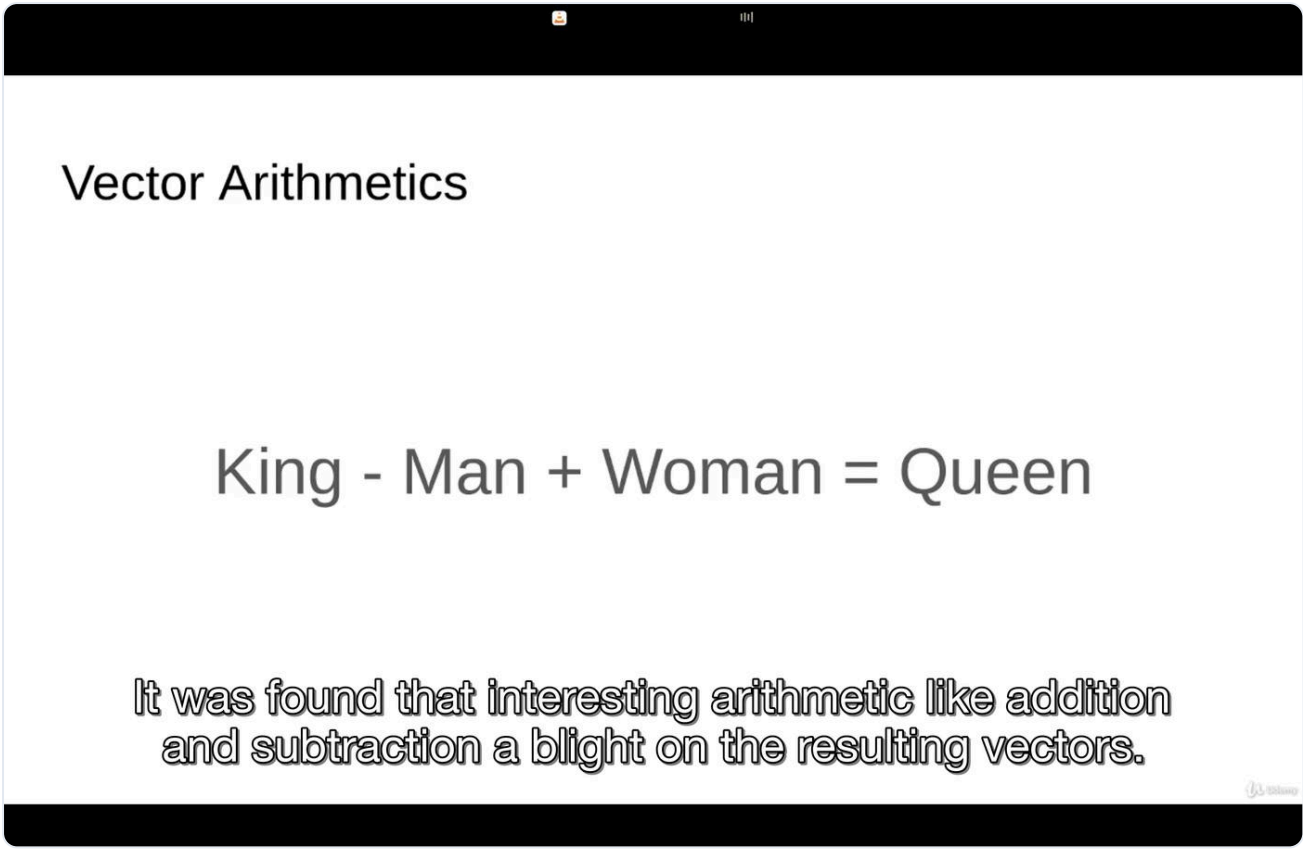


FIGURE 13.20

It was found that interesting arithmetic like addition and subtraction applied on the resulting vectors.

One example which is actually a very famous one. The word man can be subtracted from the word King and then added to the word woman to give the result of a queen that is of course a vector addition and subtraction on the learned representation. So the question now is this applicable to GANs. In fact it holds true for the representation learned by the discriminator of a DC-GAN. The authors have averaged the vector representations of 3 sample images in three categories.

The first category was a man with glasses. The second man without glasses.

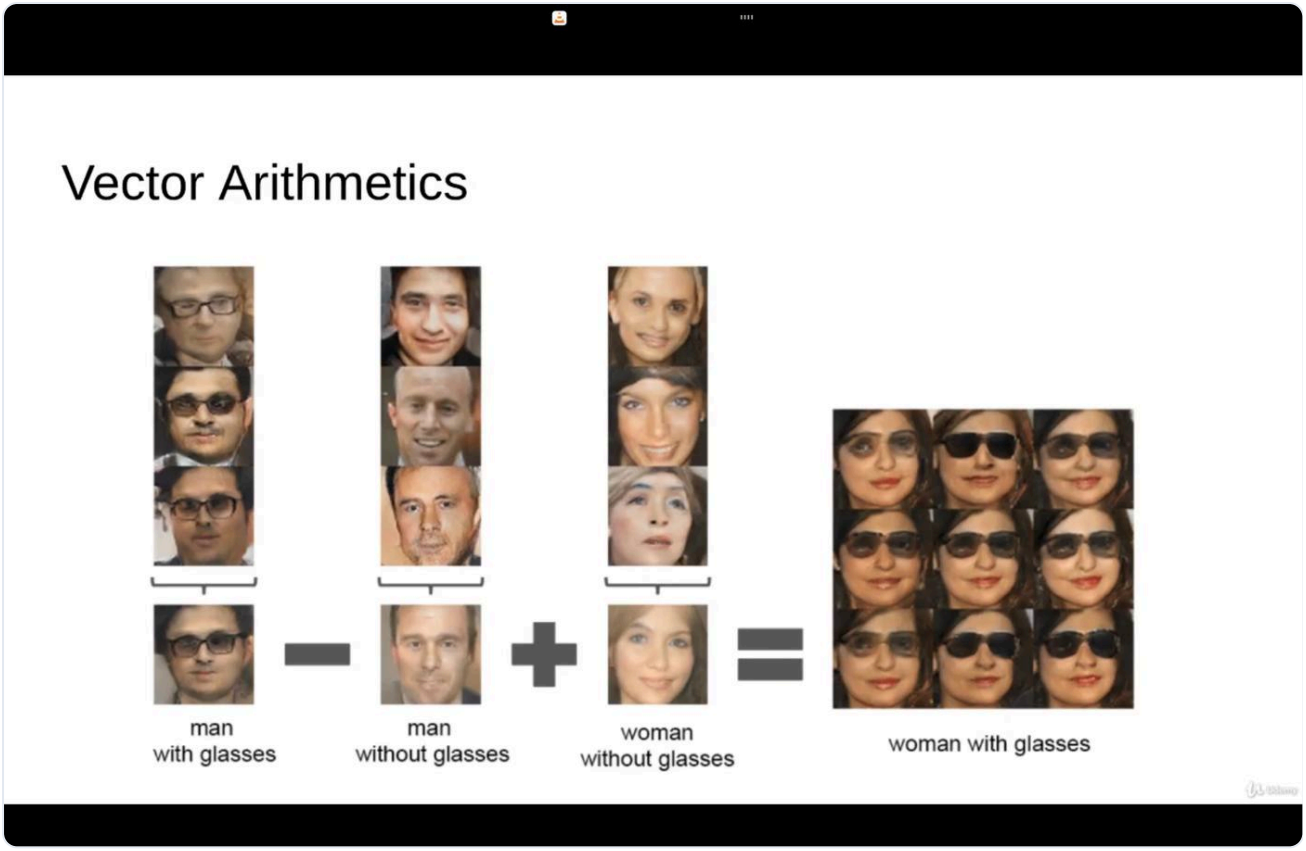


FIGURE 13.21

The third, a woman without glasses, then perform the vector maths and voila here you get a woman with glasses. You may ask why there are nine images of the lady with glasses. The center one is a direct result of the operation. The rest of them were produced after adding noise to the vector representations to test the robustness of the algorithm. This is a very common technique to test the model's robustness by simply adding Gaussian noise to the inputs and observe the results.

The authors have also performed other experiments to visualize the learned convolutional filters remove some neurons from the generators etc. It was also found that the vector representation can be used to change the pose or the emotion on the resulting image. For example the subtract smiling faces from angry faces and then add the difference to an angry face.

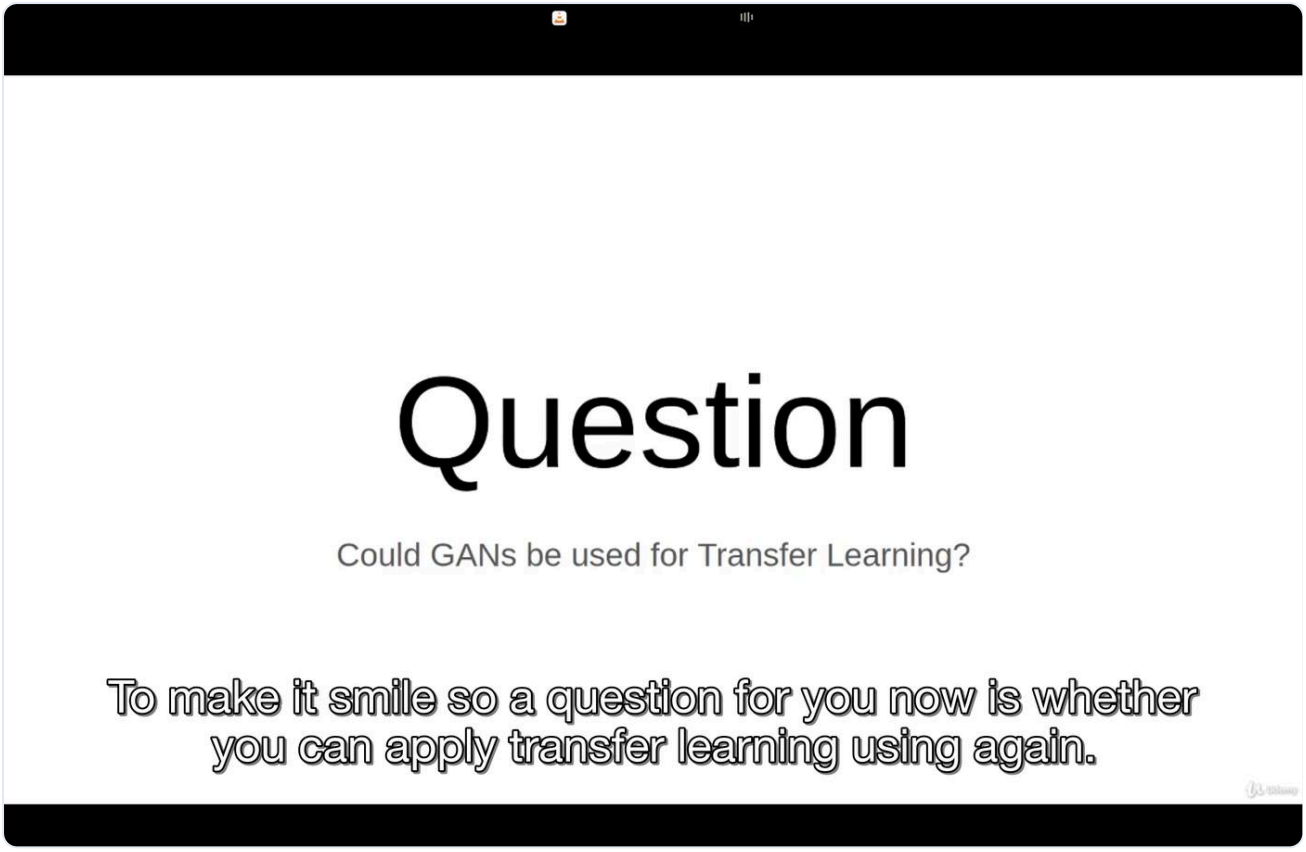


FIGURE 13.22 | Could GANs Be Used for Transfer Learning?

To make it smile so a question for you now is whether you can apply transfer learning using a GAN.

Think about it and share the answers together on the course forums and the next lecture you shall get your hands dirty and implement DC-GAN. See you in the next lecture.

LECTURE 17

Implement DC-GAN on UC Birds Dataset

See the notebook `10.Implement-DC-GAN-on-UC-Birds-Dataset.ipynb` for the code.

Implement DC-GAN on UC Birds Dataset

Hello and welcome.

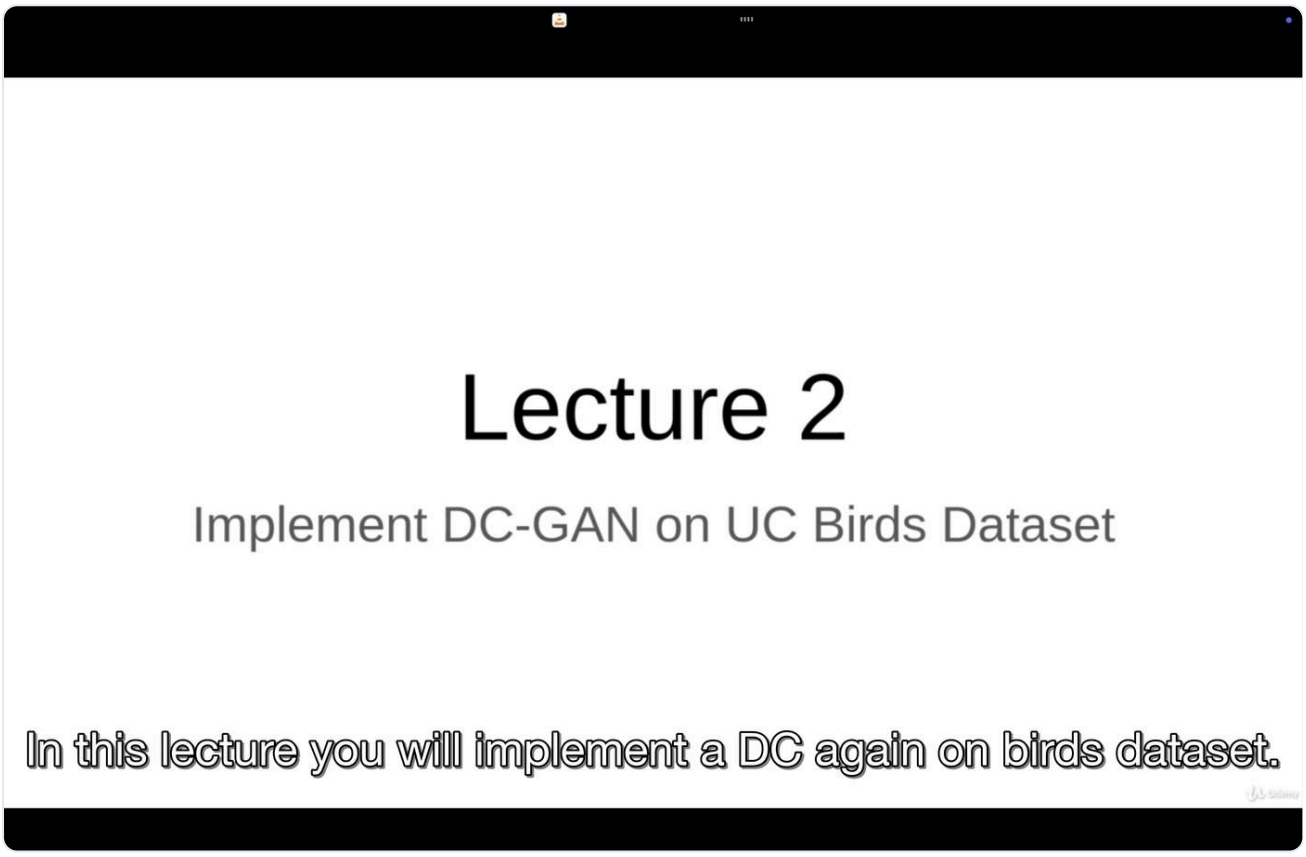


FIGURE 14.1 | Implement DC-GAN on UC Birds Dataset

In this lecture you will implement a DC-GAN on birds dataset. Let's get started. Let's first review the general guidelines which are recommended by the authors of the paper. These guidelines tend to be true for convolutional GANs. In general you'll find them written on every tech blog on the topic. The first recommendation and instead of using max pooling or average pooling you should instead allow the network to learn its optimal downsampling and upsampling weights.

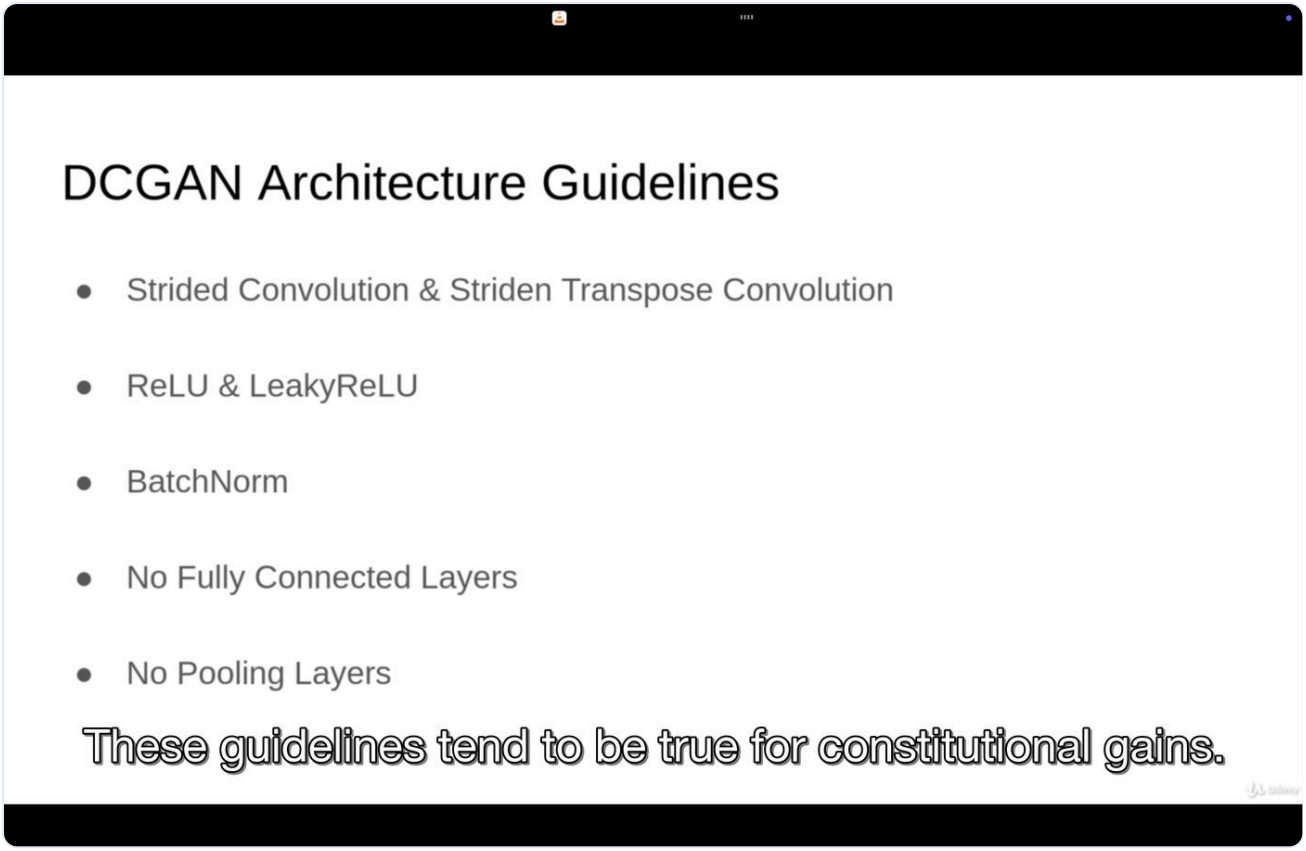


FIGURE 14.2

This is achieved using strided convolutions as we have discussed in the previous lecture. You should also use ReLU activation for the generator and Leaky ReLU for the discriminator. This is an experimental finding. It's also recommended to use batch normalization over mini-batches which does a good job on gradient flow. Finally you shouldn't use any fully connected layers in the process. Here you go.

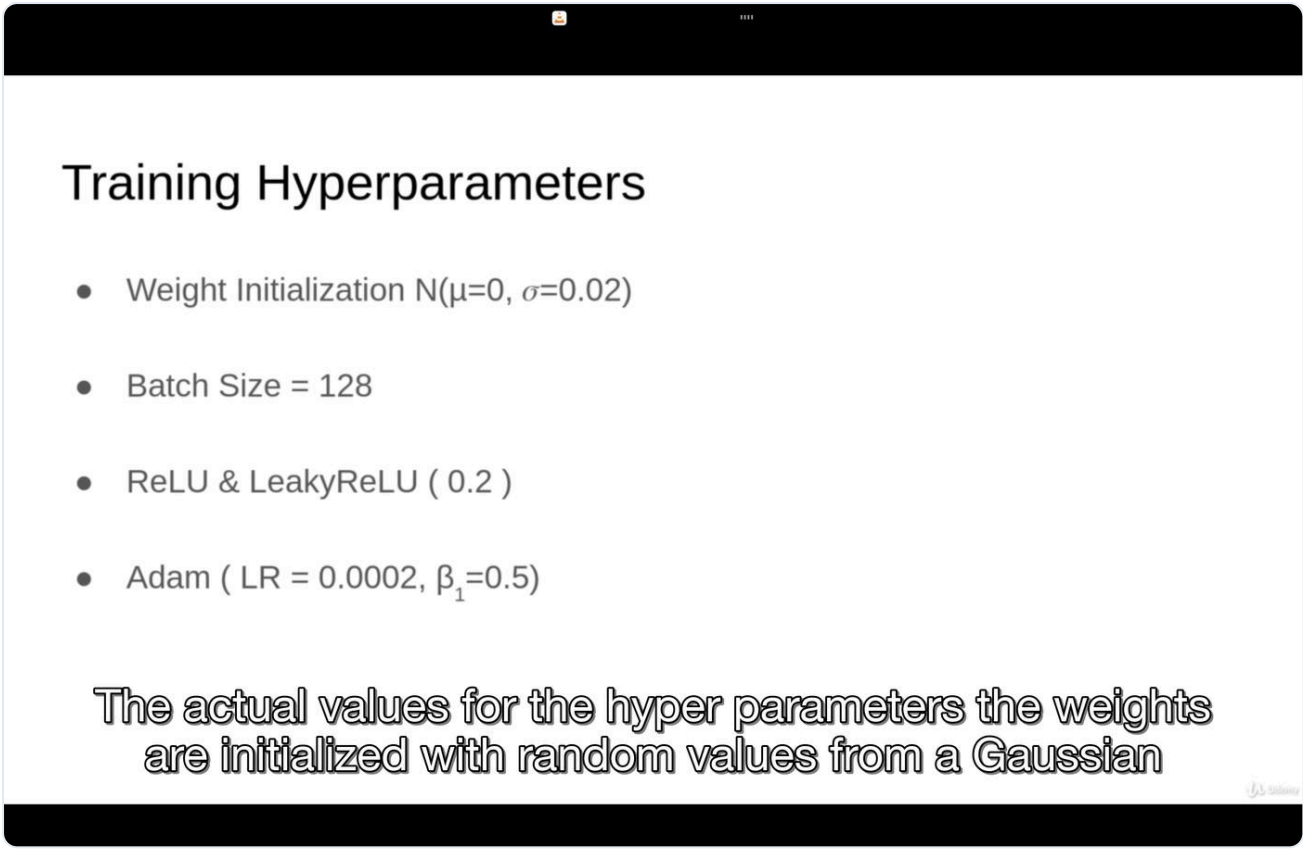


FIGURE 14.3 | Training Hyperparameters

The actual values for the hyperparameters the weights are initialized with random values from a Gaussian distribution with zero mean and 0 point or two standard deviation.

The batch size is set to 128. The slope of the Leaky ReLU in the negative region is set to 0.2 Adam optimizer is used with learning rate of 0 point 0 0 0 2 and the beta 1 coefficient of 0 point 5. Let's go straight to coding. We begin by opening the associated notebook from the course materials for Section 4 lecture 2. Then we import the torch and check for cuda. Otherwise we use the regular CPU in this exercise. I use the bird data set which is around 12000 images of birds falling in two hundred categories I shall use wget to download the dataset and shutil to unzip the tarball file.

It helps to automate the process for running the notebook and Google Colab. The dataset. Comes with meta files for class labels and bounding boxes. I shall use matplotlib to visualize a couple of images so here we go is one example. And here is another example and here you go with the third example and as you can notice the bounding boxes don't fully cover the bird body. I was going to use the bounding boxes as a baseline to crop the images but apparently this is not the best approach for GANs.

Instead let's first train the model on full images. I would encourage you to try cropping and share the results here I delete the file objects to free the memory for training. It is also a best practice to visualize the frequency of the labels. That is how many training samples are available for each category and this bar chart we can see that there is a category imbalance in the data set. That's why I'll stick only with the categories that have got the full or the maximum number of images and these ones will have 60 images per category.

I can simply use numpy.max to find the maximum number of images per category which is 60 so at this point we've got two options. The first is to use the data train loader to resize the image on the run as it is loading. But this has the downside that it will resize the image every time it is loaded for training. That is if we have got thousand epochs the image will be resized a thousand times. So instead I decided to resize the image on disk and store them on the hard disk and load them only once.

Now it's time to import the data and squash the range to minus 1 and plus 1. First we create a squash function to convert images from the range 0 to 1 to the range negative 1 and plus 1. The **call** method is triggered on the input batch then we compose the preprocessing pipeline. So here I commented out the line that is responsible for resizing the image to 64 by 64. As you can see I've already resized the images on the disk.

I would also encourage you to play around with different values for resizing and the preprocessing pipeline. I use the method to tensor which we have used in previous lectures. This method squashes the image from 0 to 255 down to 0 to 1. Then I apply this squash transform to squash it further to the range. Negative 1 up to plus 1 then I create the train loader with the batch size I set batch size to 128 and I set shuffle to true then the number of steps is the number of images divided by the mini batch size.

So now I visualize a mini batch to inspect the preprocessing I use torchvision.utils.make_grid. But notice here that the normalized flag is set to True. This will fix the range again for visual inspection but notice here that the normalized flag is set to true. This will fix the range again for visual inspection. Now here is how our data set looks like it is ready now to go through our training pipeline. Next we set a few global variables of 4 dimensions. The variable nc equals 3 is the number of RGB channels in our color image.

the variable nz is the noise vector length ngf is the number of feature maps for the generator. It is convenient to divide the convolutional features in terms of a base line value so that you can easily fine tune the convolutional network then you get the optimal value for this variable. The same applies for the discriminator. And here we use ndf to do exactly the same this coding exercise is benchmarked upon an open source tutorial on the PyTorch documentation. The generator class inherits from torch.nn.Module then it has transposed convolution blocks each of them has a transpose convolution layer a batch norm and a ReLU activation notice here how at the first layer the variable ngf is multiplied by eight.

Then at the next layer it is multiplied by four and the value decreases from layer to layer as follows At the output layer. I have tanh activation function. Here is the point where the output image reaches its final size of three by 64 by 64. The forward method passes the mini batch input through the network and output image is returned. Next we design the discriminator the discriminator inherits from torch.nn module and has convolutional blocks of convolutional layer followed by a batch norm and a Leaky ReLU activation the last layer is a sigmoid activation that squashes the output between 0 and 1 in the forward method I use the view method to reshape the output from 4 dimensions down to two dimensions.

That's because convolution results in a four dimension feature map of batch size by number of channels by width by height as per the authors directly applying the batch norm to all layers resulted in sample oscillation and model instability. This was avoided by not applying batch norm to the generator output layer and the discriminator input layer next the weights are initialized with random values drawn from Gaussian distribution with zero mean and one standard deviation for the batch norm layers.

The mean is set to be one these values are based on experimental trial and error and fine tuned to produce the best results. You can always try other values on your own next we create the models and apply the initialization method using the apply method available on the nn.Module API. Now I create the Adam optimizer for both the generator and discriminator. Here I. Initialize the objective function and named criterion and create a fixed noise for visualization.

Then I create a matrix of ones for real labels and another matrix of zeros for fake labels. Next I define a method for training the discriminator. I found it better to separate the module for training the discriminator into a method and the same for the generator so that you can better visualize the nested loops in the training pipeline. There are no major differences between the training of the discriminator in this exercise and the ones we have discussed in previous sections and here I do the same once again for training the generator over here I initialize TensorBoard for loss and results visualization it is always the best practice to use the check points.

So I load the checkpoint if it exists for the module for the generator for the discriminator and also for the optimizer for both of them. Since the learning rate is changed in the Adam optimizer so we need to keep track of these changes and load them as a check points as well. So here is the actual training I iterate for the number of epochs. And then I iterate for the number of steps and then I train the discriminator for K steps. In this case I set K equal to four.

And I train the generator for one step. Only the if condition if epoch modulus hundred equals zero allows us to log the lost values and visualize the results every hundred epochs. Note here how the loss is divided by four to obtain the mean loss over the K steps then the loss and the generated images using the fixed noise are logged to TensorBoard for visualization and down here I saved the checkpoint every hundred epochs so that I can resume the training I don't have to start over every time the machine is restarted or reset so I have trained the model for 4000 epochs and now we are ready to visualize the results here you can see that the model is trying to generate the wings the beaks, the body of the bird and it is also trying very hard to visualize the background and similar to the images in the data set the background it is a sky a tree or even the surface of the sea you can see here that some examples are better than the others, and if you allow the model to train further you will get even better results and in future videos you will learn a further technique to get even better results based on other techniques described in research papers.

So now it's time to inspect the results in TensorBoard. I run TensorBoard by running the command tensorboard --logdir ./runs after you open TensorBoard you'll find two tabs, one for scalars and the other for images by inspecting the discriminator loss we find that it started with a high value and then it started to decrease afterwards it stabilized at a certain value. On the contrary when we inspect the loss for the generator we find that it started low and then it started to increase.

This indicates that it was doing a good job at the beginning in fooling the discriminator but afterwards it struggled in a mode collapse, a problem. Again, mode collapse is inherent to all GAN models unless you solve the problem using a clever technique but it will stick along with your training and you will find it very annoying. Most of the time but you have to be able to diagnose it using the loss curves if we navigate to the images tab we find the visualization which is done every hundred epochs you can see here that some results are better in the intermediate epochs than the last ones and it is normal.

So usually what you do is that you save the model each hundred epochs and then subjectively decide which model is best. Or you can even use Inception score. That's a technique we have discussed in previous lectures to decide the best model and save that model checkpoint. You can resume training using different hyperparameters from that point. Or you can even use that model in production. In the next video I shall discuss auxiliary GANs thank you and see you in the next lecture.

LECTURE 18

Working of Multi-way Loss Function

Hello and welcome.

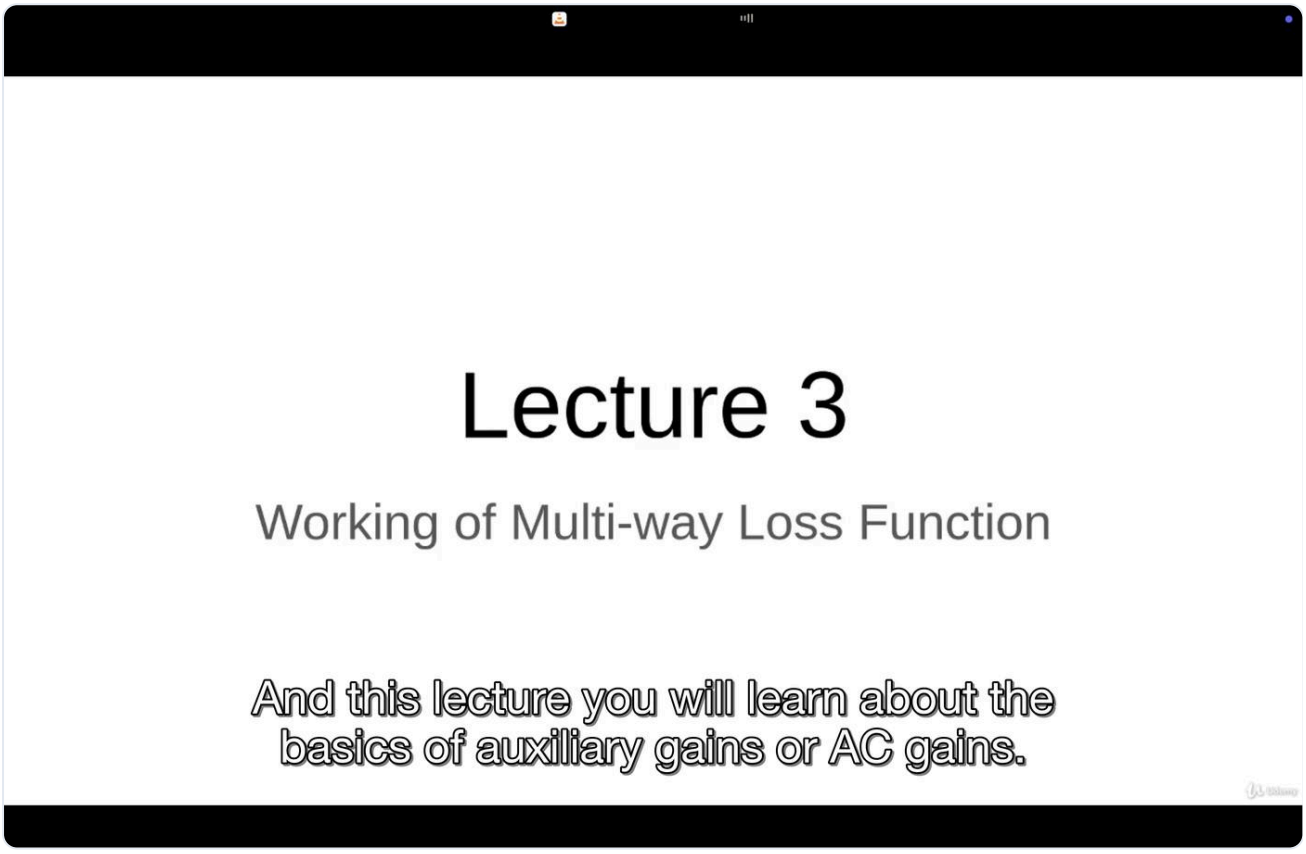


FIGURE 15.1 | Working of Multi-way Loss Function

And this lecture you will learn about the basics of auxiliary GANs or AC-GANs. Let's get started in section three.

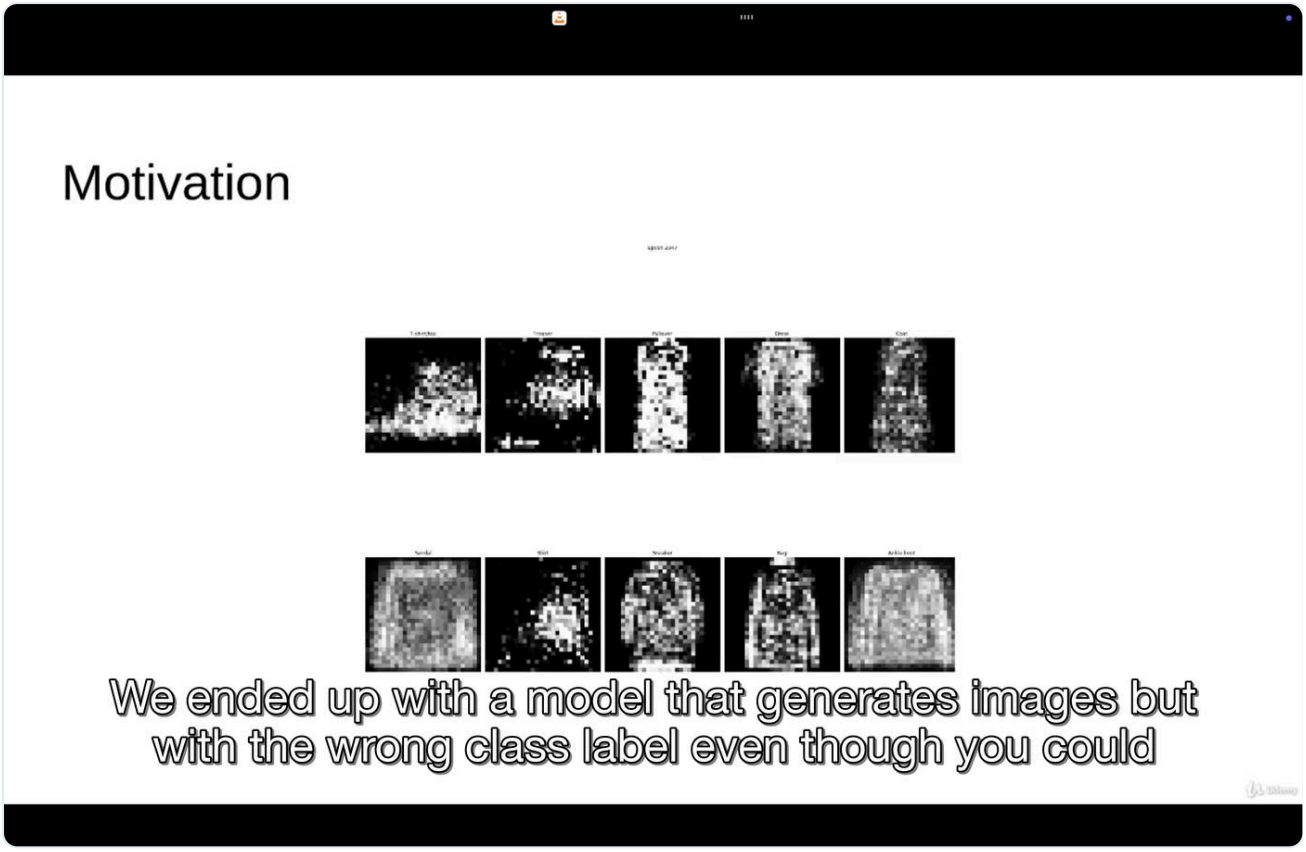


FIGURE 15.2

We ended up with a model that generates images but with the wrong class label even though you could solve this issue by fine tuning the hyperparameters of the model. I will use this example to demonstrate the workings of AC-GANs and multi-way loss functions AC-GANs addressed the following issues which are inherent in GAN models.

The first is the generation of appealing high resolution images.

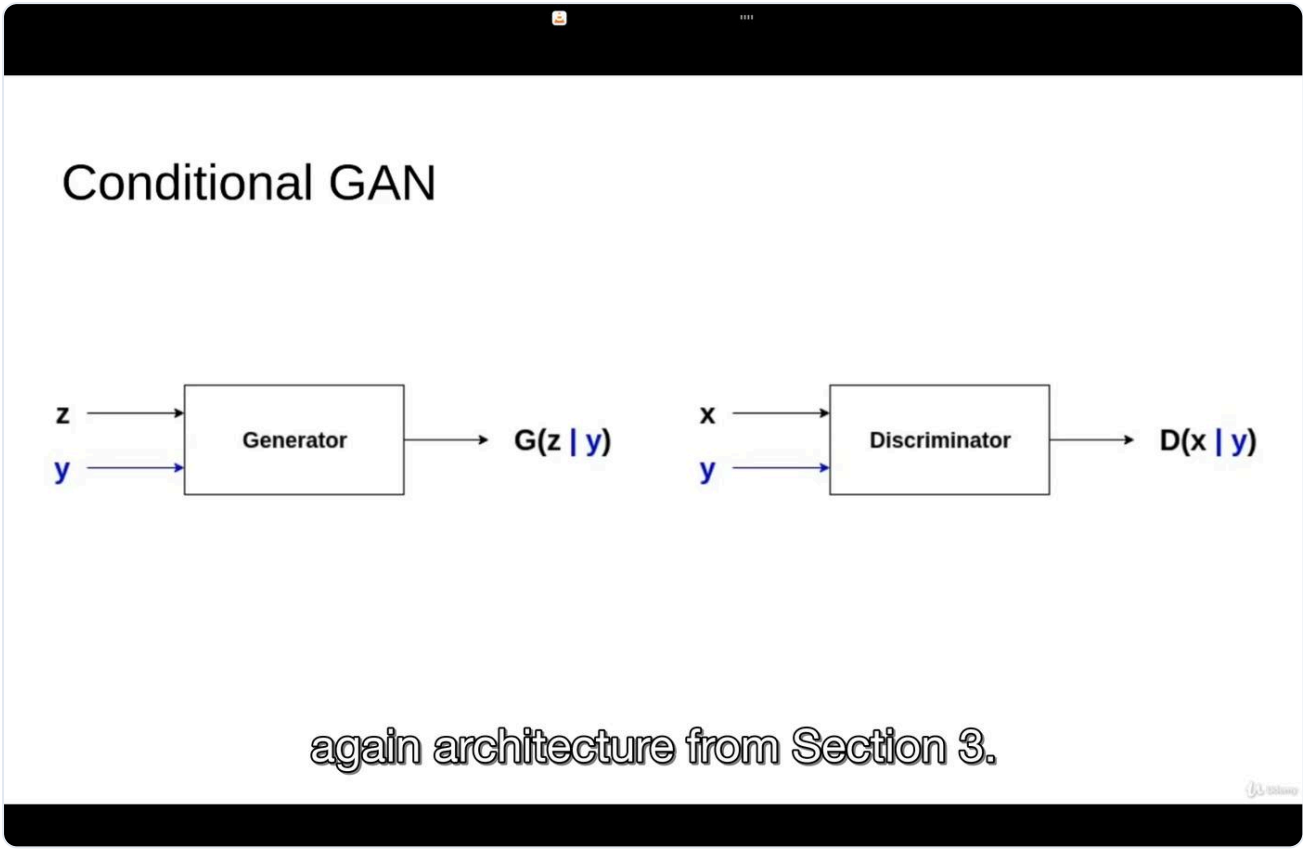


FIGURE 15.3

The second is the generation of different samples for the same label and instead of mode collapse to generate the same image for a specific class Let's dive deeper in the next slides recall the conditional GAN architecture from Section 3. We passed the condition label to both the generator and discriminator since we didn't use convolutional layers back then we simply concatenated the condition to the input auxiliary classifier GANs or AC-GAN.

On the contrary encourages the discriminator to reconstruct the label.

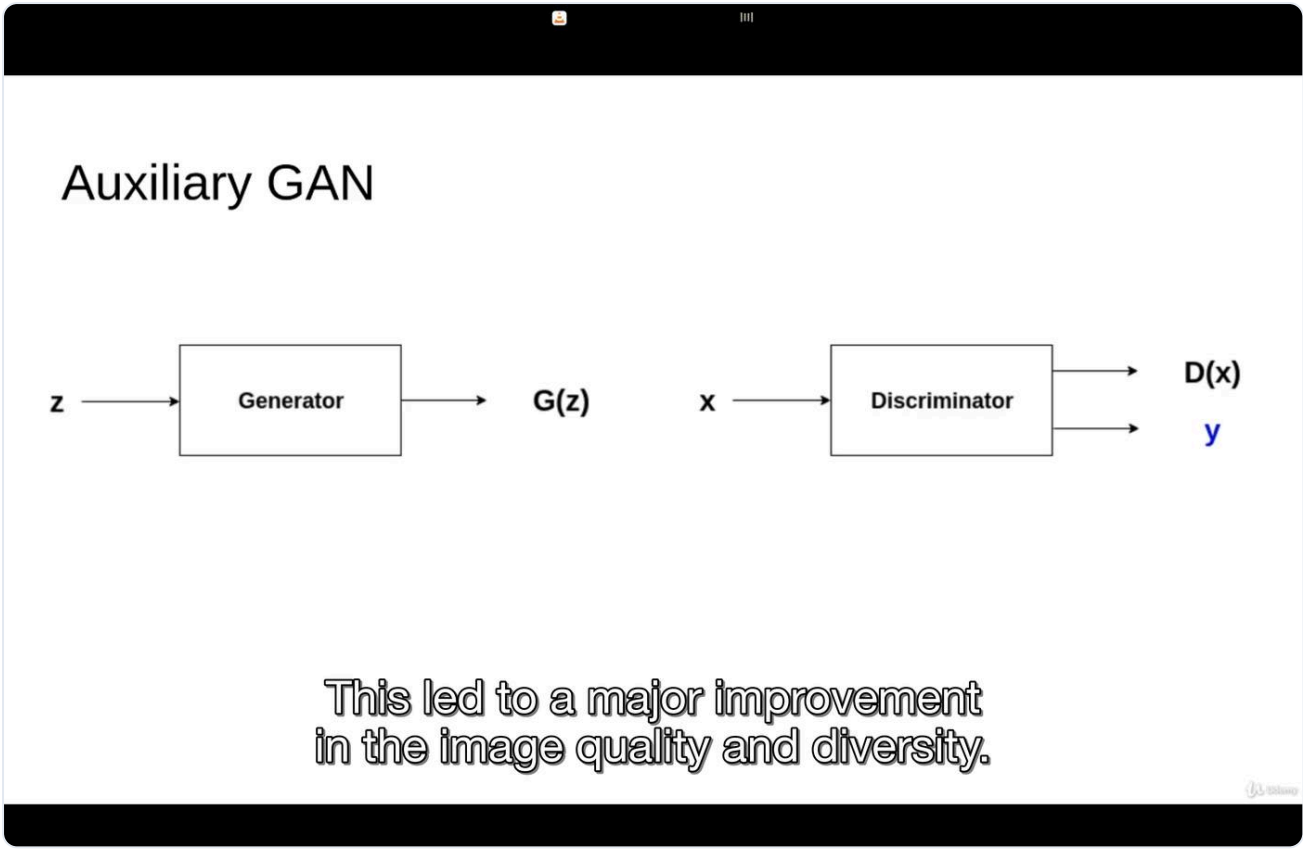


FIGURE 15.4

This led to a major improvement in the image quality and diversity. So now the discriminator should produce two outputs the first one is whether a sample is fake or real and the second one is the sample classification it is worth mentioning here that this reconstruction technique should apply to other metadata such as bounding boxes. It is still valid to pass the condition label to the generator in order to produce relevant samples.

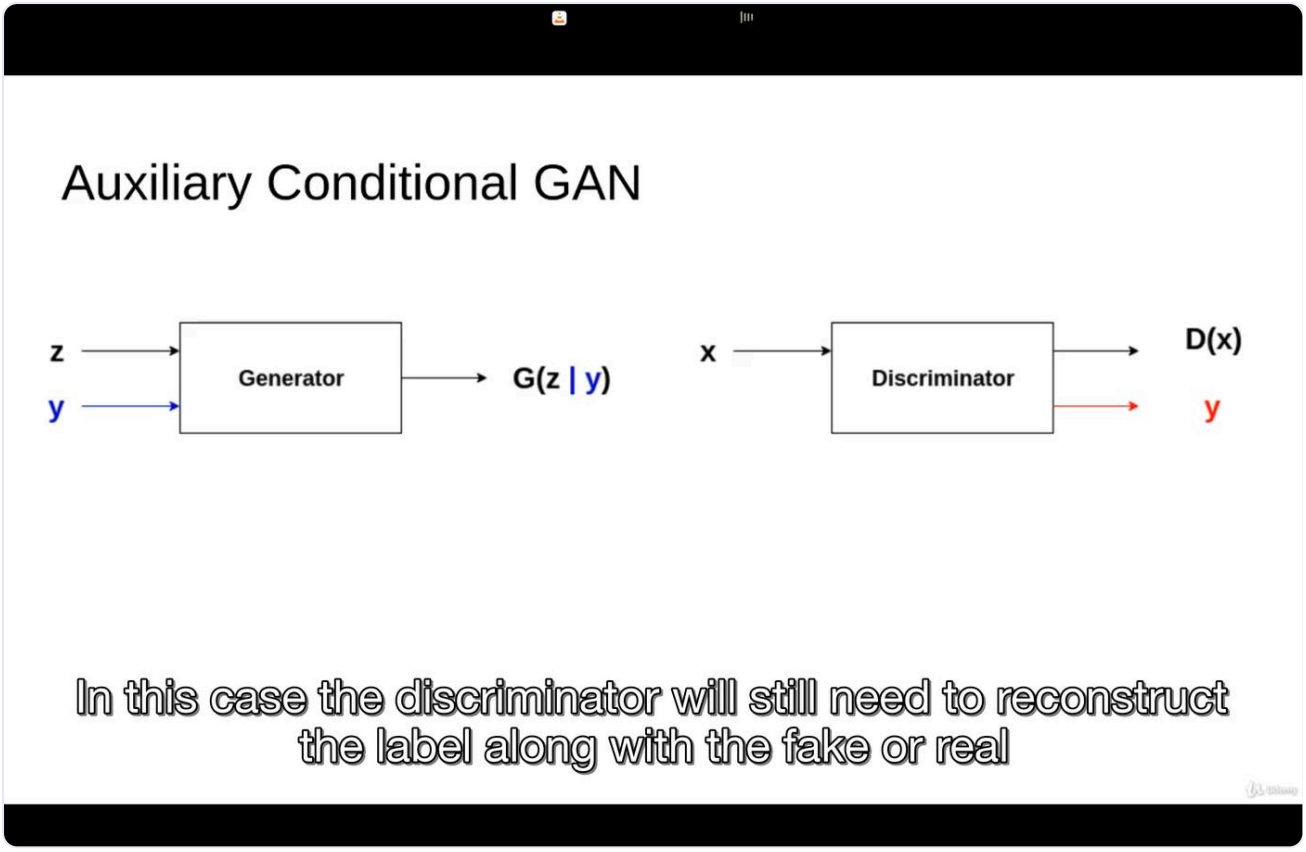


FIGURE 15.5 | Auxiliary Conditional GAN

In this case the discriminator will still need to reconstruct the label along with the fake or real result. I have doubted whether the information will leak between the generator and the discriminator. However the reason that the discriminator does not cheat and copy the label from the generator could be that that serial training makes one of them fixed at a time. I haven't found any clear answer to this question and it may still be a topic for research and investigation let's look inside the discriminator box and explore some nifty details.

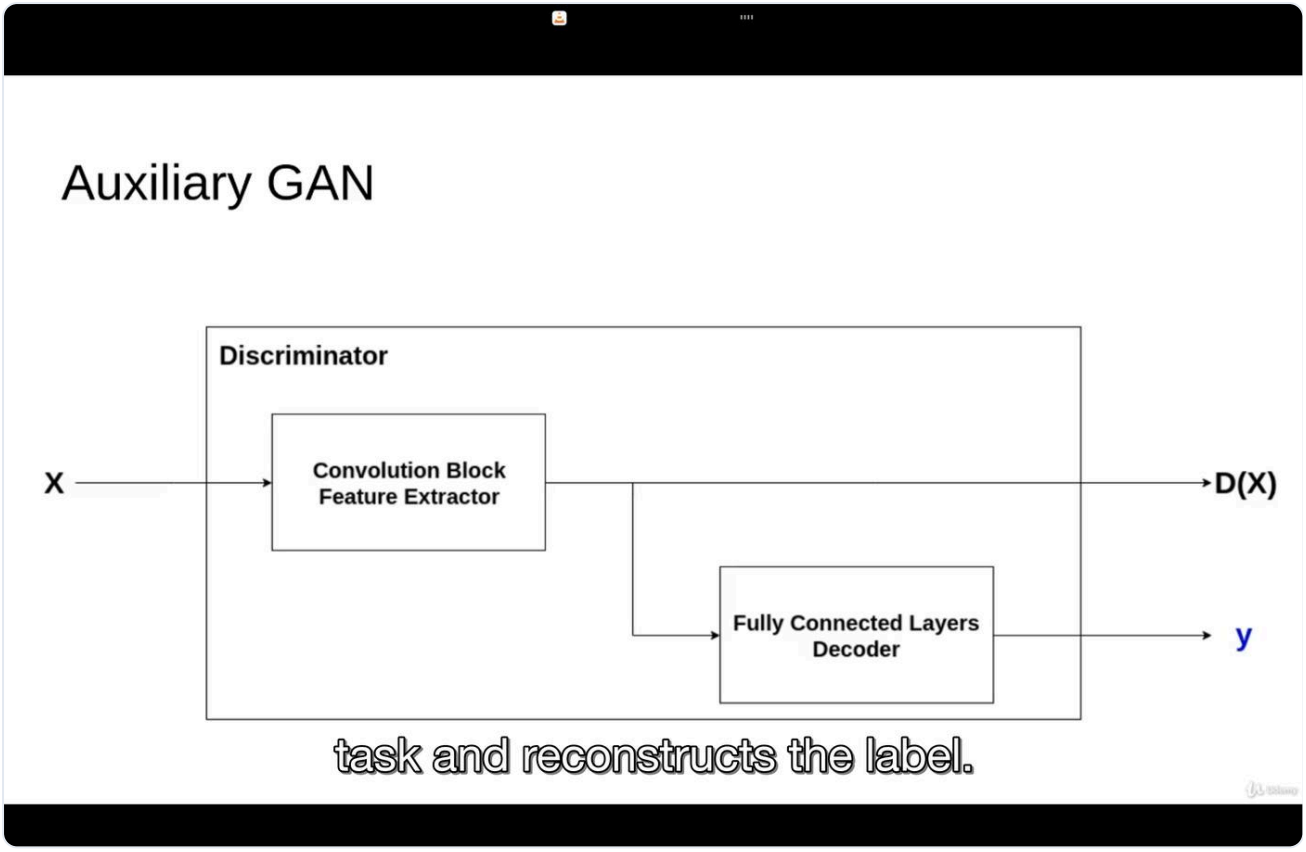


FIGURE 15.6 | Feature Extractor

The image is first passed through a convolutional block which functions as a feature extractor. Remember from lecture 1 when we talked about the latent space or in other words the vector representation of images then the result is passed through a fully connected layer that performs the classification task and reconstructs the label. It could also perform a regression task and reconstruct the bounding boxes or even both of them. This is called multitask learning.

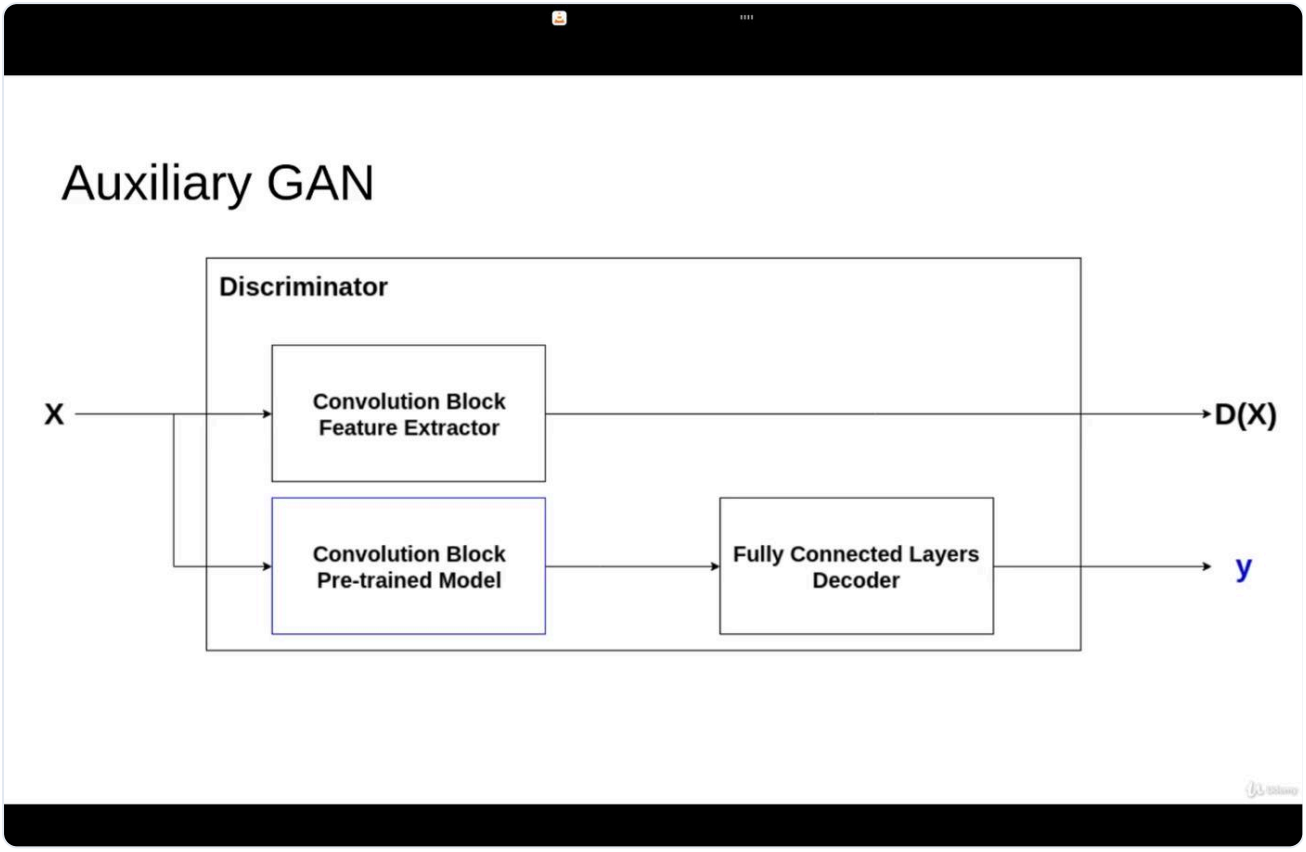


FIGURE 15.7

It is also possible to use a pre trained model. i.e. transfer learning to reconstruct the metadata. This is done by passing the input image through two different paths. The first one is responsible for determining whether the sample is real or fake. The second path reconstructs the label let's discuss the loss function for AC-GANs the basic GAN loss which we have discussed in previous sections is reformulated for convenience.

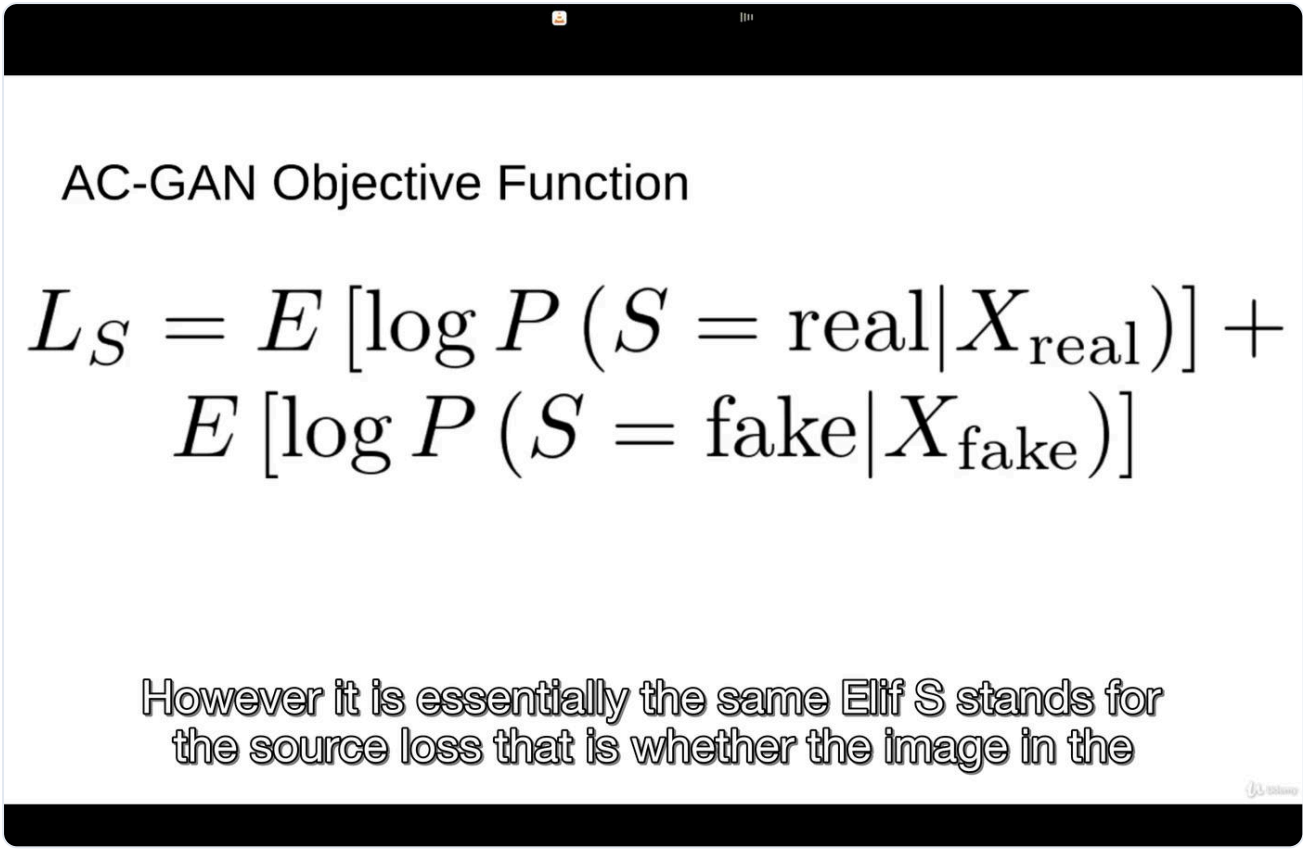


FIGURE 15.8

However it is essentially the same L of S stands for the source loss that is whether the image is real or fake L of C on the other hand stands for the class loss.

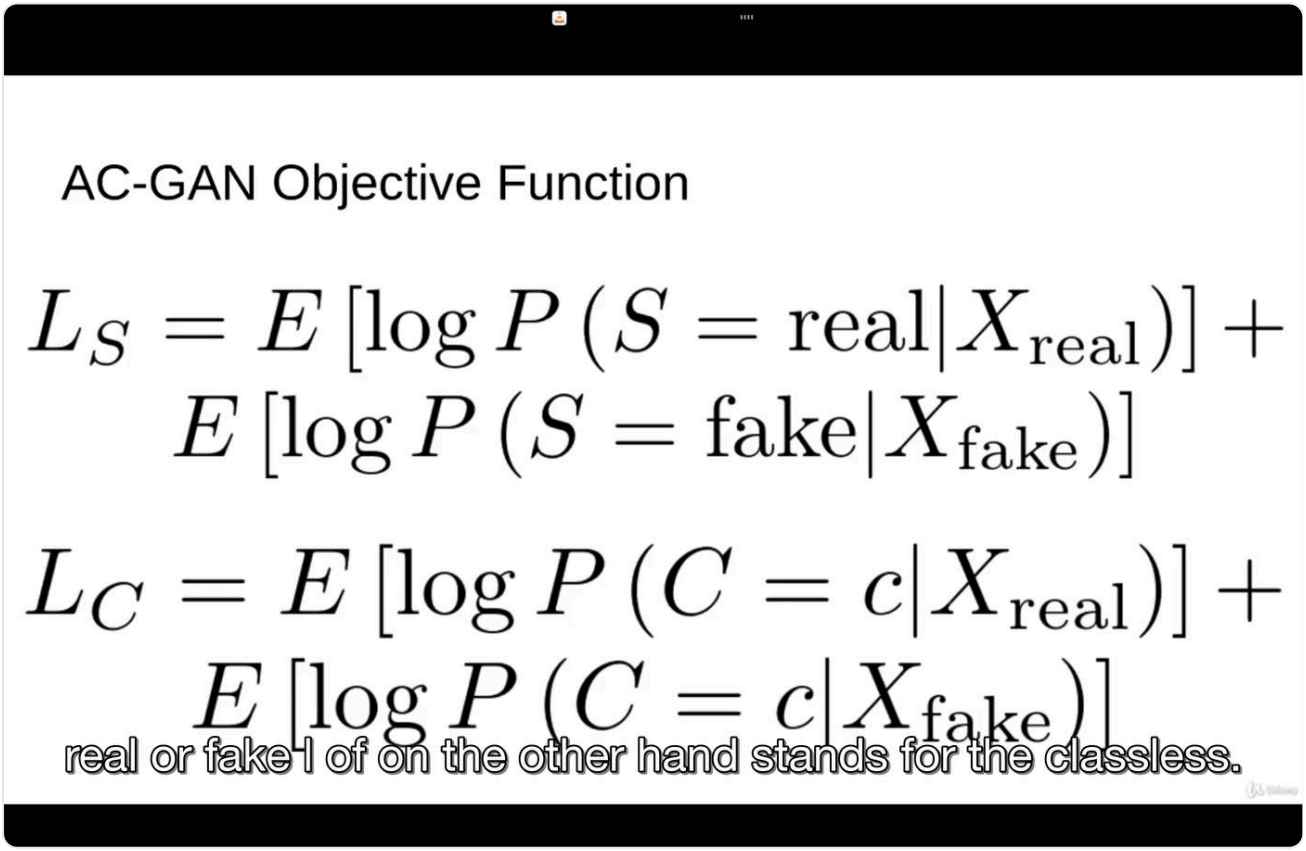


FIGURE 15.9

That is whether the label is reconstructed correctly so the discriminator now tries to maximize the sum of both losses.

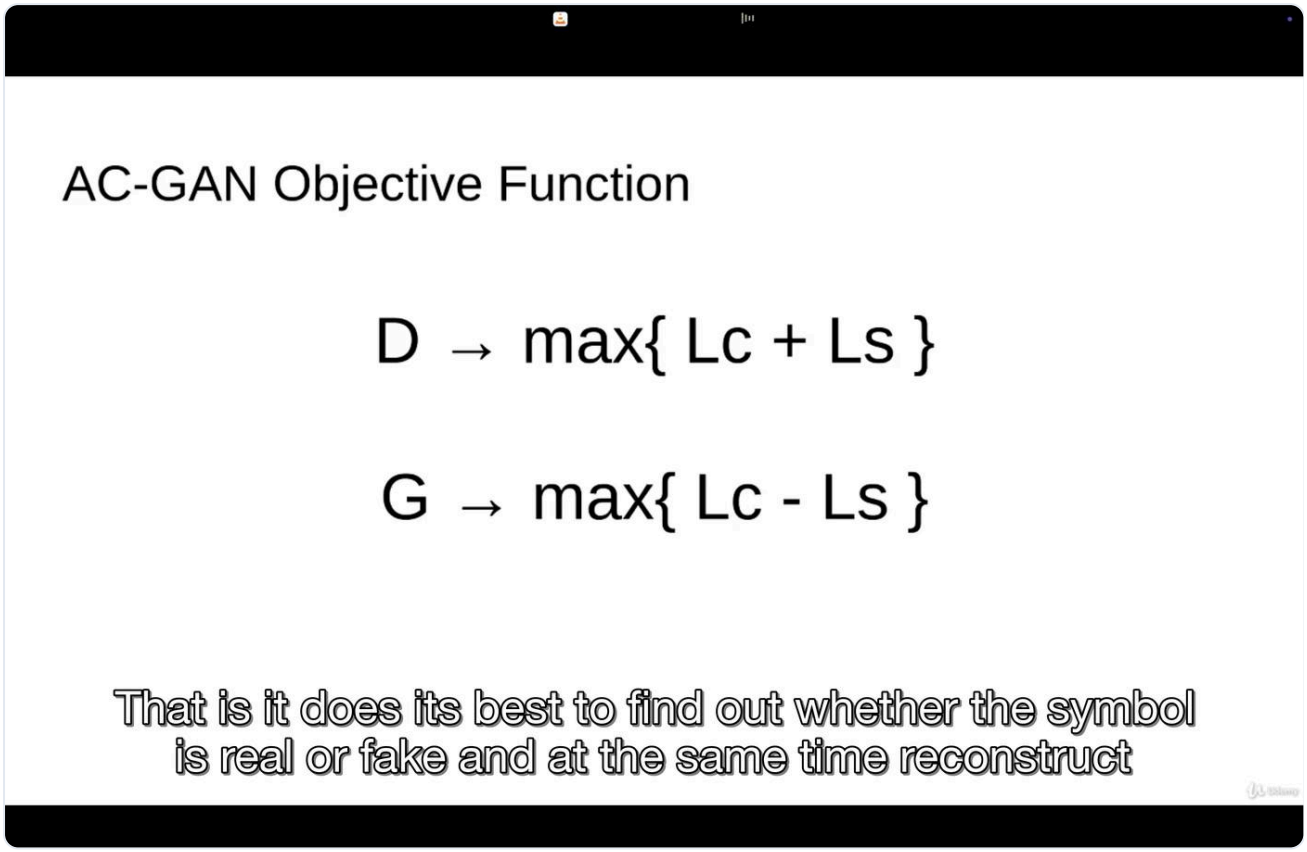


FIGURE 15.10

That is it does its best to find out whether the sample is real or fake and at the same time reconstruct the label on the other hand. The generator is trying to minimize the probability that it gets caught as fake.

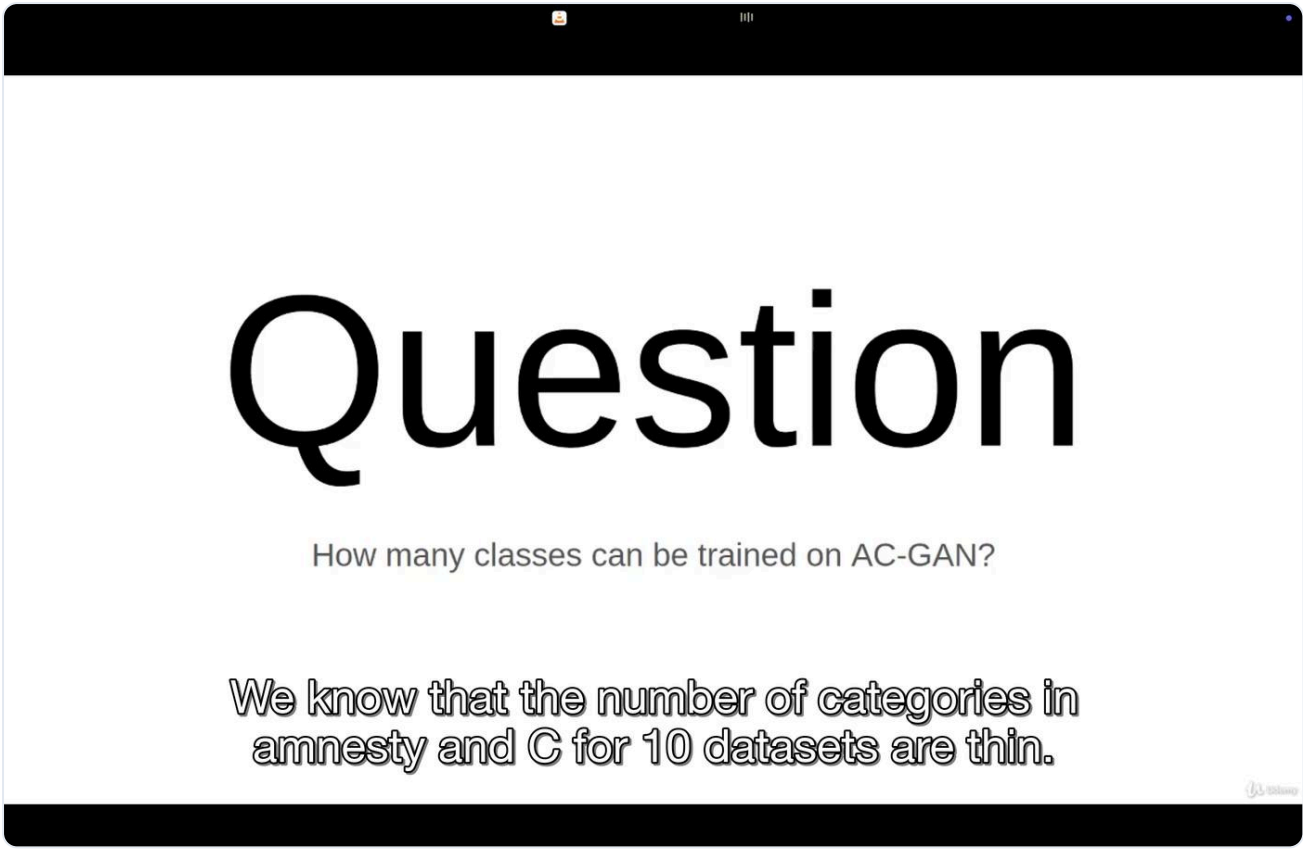


FIGURE 15.11 | How Many Classes Can Be Trained on AC-GAN?

We know that the number of categories in MNIST and CIFAR-10 datasets are ten. We also know that the ImageNet dataset has 1000 categories could one GAN model be able to learn all 1000 categories or fewer categories or more categories.

The authors of AC-GANs have trained 10 categories per model and trained 100 such models to learn on the 1000 categories of ImageNet dataset. Another question may arise whether there were any preferences which categories to train together on the same model.

Inception Score

$$\exp\left(E_x\left[D_{KL}(p(y|x)\|p(y))\right]\right)$$

open research question.

FIGURE 15.12

Measuring Diversity

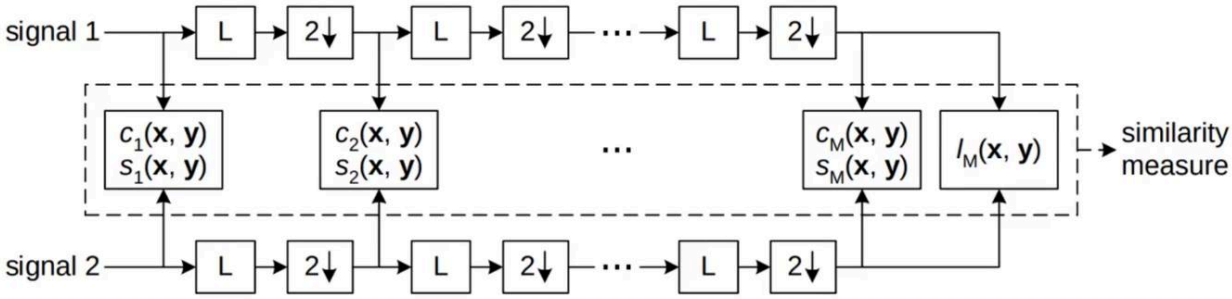


Fig. 1. Multi-scale structural similarity measurement. L: low-pass filtering; 2 ↓: downsampling by 2.

Wang, Zhou, Eero P. Simoncelli and Alan C. Bovik. "Multi-scale structural similarity for image quality assessment." (2003).

FIGURE 15.13

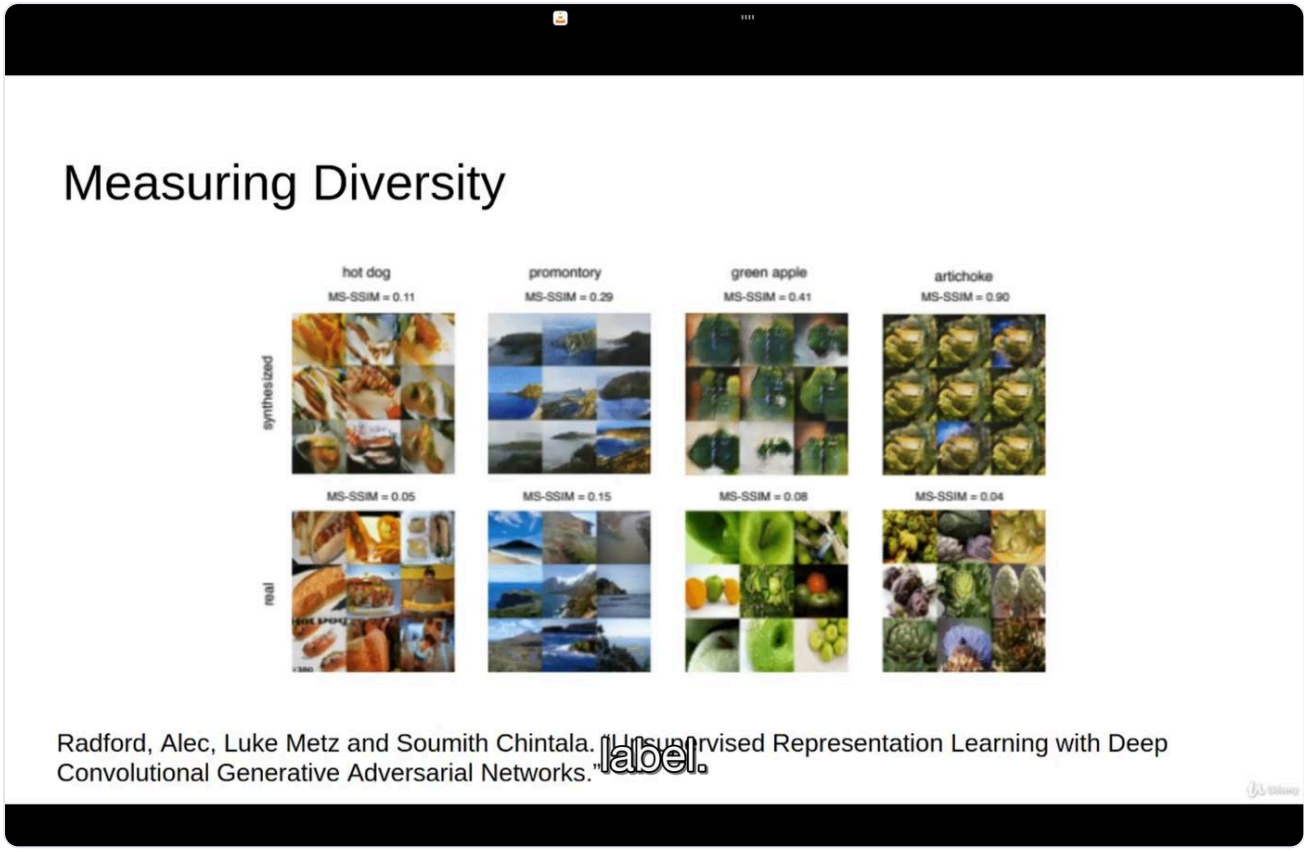


FIGURE 15.14

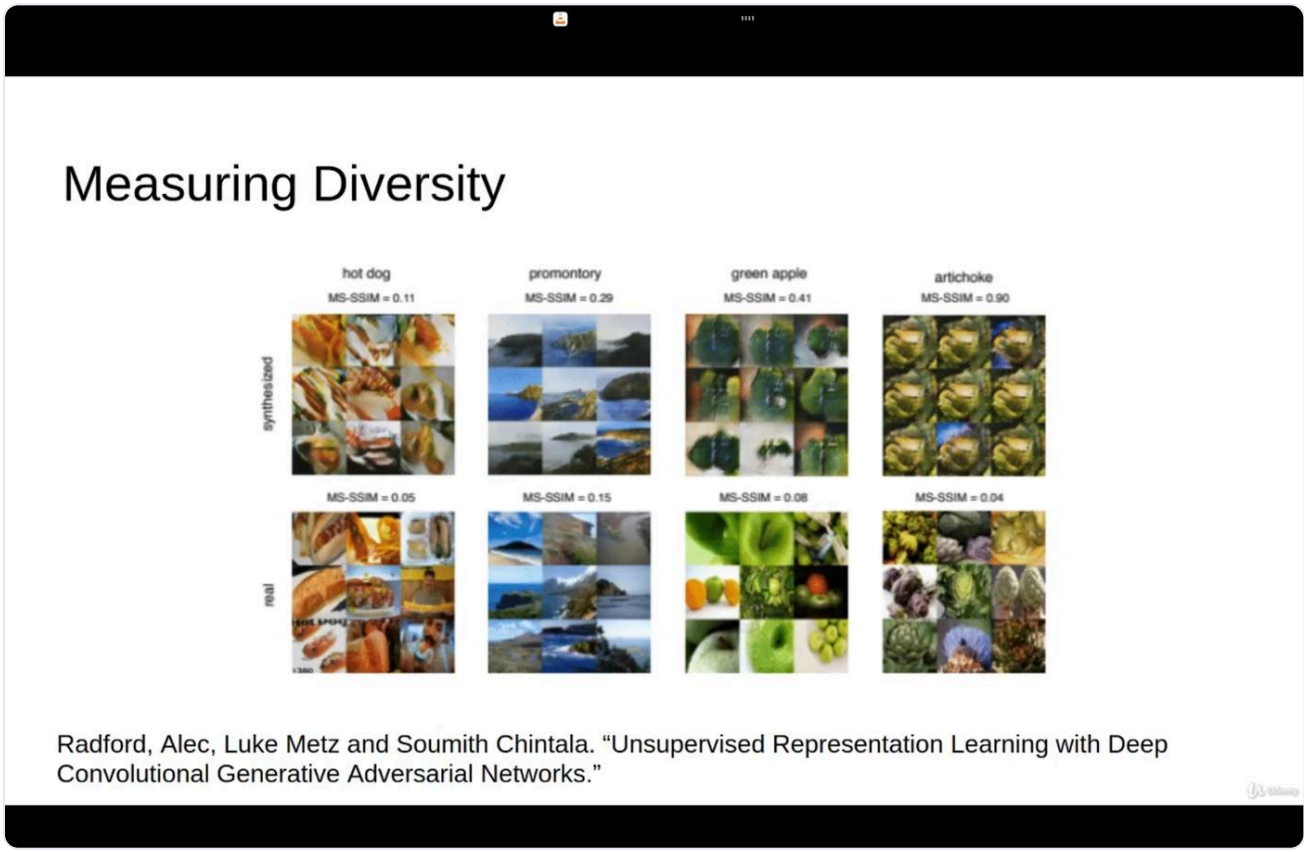


FIGURE 15.15

However it was found empirically from the experimental results that it didn't affect the result recall from section two our discussion of the evaluation techniques for generative models and how this is an open research question.

the Inception score was one of the evaluation techniques that works by passing the generated images through an inception network that is trained on ImageNet and then calculates the Kullback-Leibler divergence. This has two downsides. The first is that the network trained on ImageNet does not necessarily perform well on other datasets. The second is that it is not straightforward to interpret the Kullback-Leibler divergence for individual classes especially for conditional GANs.

The authors have proposed a variation of the Inception score that is easier to interpret for individual classes at a very high level of detail generated images are downsampled using bilinear interpolation and then fed through the inception network and the classification accuracy is reported. This method has two advantages. The first is that figures are easier to interpret and the second is that this metric serves as a measure of the quality of the produced high resolution images.

It was found that Downsampling removes the important details from the image. This means that the high resolution was not the result of a trivial bilinear interpolation of small size images but it means that the network has learned to produce details in the high resolution case on its own. Another issue that one often encounters when training GANs is that the model can learn only one sample for each category and then memorize it to fool the discriminator that is named mode collapse on the class level.

The authors contributed another method to evaluate the diversity of the generated images. The idea is built on a modified SSIM image similarity score the multi-scale SSIM is robust to variations in scale. It applies a series of denoising filters and downsampling. Image similarity scores such as SSIM, PSNR and image hashing are a very useful tool that I have used in some business cases in my career.

It could help you as well. The question is however how did the authors use MS-SSIM score. MS-SSIM score falls in the range from 0 to 1. Zero means minimum similarity while 1 means maximum similarity. So they calculated the mean MS-SSIM score for images which are generated using the same condition label. The majority of the categories had a low mean MS-SSIM score that means high diversity for generated outputs. This is a metric that you could use to evaluate the diversity of images generated by your model.

In the next lecture you shall implement the auxiliary GANs on the Birds dataset. Thank you and see you in the next lecture.

LECTURE 19

Coding Challenge

See the notebook `11.Implement-multi-way-loss-with-Auxiliary-GAN-on-UC-Birds-Dataset.ipynb` for the code.

Implement multi-way loss with Auxiliary-GAN on UC Birds Dataset

Hello and welcome. And this lecture you will implement the auxiliary GAN by passing the condition to the generator and modifying the loss function as per the last lecture. Let's get started let's open the notebook file for Section 4 lecture 4 which is available in the course material everything should go the same way as we did in lecture 2 but I will highlight the changes so in the snippet where we defined the global noise dimensions so in the snippet where we defined the global dimensions I add one more variable nl that holds the number of categories in the dataset which I set it to 10.

This value is advised by the authors of the auxiliary GAN paper since they have found that a certain model is better suited to be trained on 10 categories and instead of more and then they divided the thousand categories in ImageNet dataset among a hundred models and trained each one of them on only 10 categories and this exercise I will do exactly the same next. I will change the generator design so that we can allow to pass the condition the number of channels in the first convolution layer.

will be changed to be nz plus nl which is the noise vector plus the condition vector then everything should go the same way like before in the forward method. I allow it to accept both the noise vector and the condition as parameter then both vectors are concatenated together using the PyTorch unsqueeze method the concatenated vector is changed from two dimensions to four dimensions. This is done by adding more dimensions along axis two and axis three.

The input is passed to the network and the generated image is returned. Now I will change the model design for the discriminator to add two outputs, one for the classifier and the other for the discriminator. That is whether the sample is real or fake and to which category the sample belongs so I will split the sequential model just before the last output layer and add another sequential block for a classifier that consists of a linear layer and a softmax activation then I add another block which is exactly the same like previous lectures where I have the sigmoid activation which acts as a validation whether the image is real or fake and the forward method the input image is passed through the main network.

The resulting features are a four dimensional vector. I pass it through to the discriminator which is the last block for verification whether it is real or fake. Then I reshape the features into two dimensions using the view method and finally pass it to the classifier at the end of the method. I will return the two outputs everything in the coding exercise continues to be the same like Section 4 lecture 2 and here at the categorical cross entropy section I shall define two loss func-

tions one for binary fake or real output as before the second one is categorical cross entropy for the classifier the method is defined by the name C2, which takes both the model outputs and the target labels.

Since we have applied one-hot encoding to the labels I need to convert that back to one value. I will use the method max which is triggered to get the max index and instead of the one-hot encoded vector. Finally both the outputs and the true values are passed to the cross entropy loss. Here I define the one-hot encoding method as previous section and create fixed conditions for testing along with the fixed noise now when it comes to training the discriminator I make a few more changes.

First the method is passed both images and labels then both are converted to the cuda device then fake conditions and fake images are created. Then fake conditions and fake images are created I reset the optimizer by using zero grad method then both real images and fake images are passed to the discriminator and we get the results in four variables real_valid, real_class, fake_valid and fake_class then I shall apply the formula exactly the same way I explained in the slides I pass the real valid and real labels to the binary cross entropy loss which is named C1 and pass the fake valid and fake labels to the binary cross entropy loss which is named C1 together they are summed and the sum is named L_s which stands for loss of source either fake or real.

The second part of the loss formula is the loss of the classifier I trigger the categorical cross entropy loss which is named the C2 and pass it real_class, real conditions, fake_class and fake conditions and the return value is summed together to be L_c. Next I sum L_s and L_c together and this becomes the discriminator loss. I apply the backward to calculate the gradients and then apply the optimizer step to update the weights according to the learning rate.

Finally for logging purpose I return the loss. In case of the generator I pass only the labels to the train. G In case of the generator I pass only the labels to the train G method then I calculate the loss by summing L_s an L_c values but hold on, shouldn't we subtract instead of summation. Well it works since we have flipped the labels. This does exactly the same as the subtraction and I would like you to try the same exercise by replacing L_c plus L_s with L_c minus L_s and see what difference it makes again here.

I initialize TensorBoard for logging and visualization and over here I restore any check points if they exist so that I can resume the training from previous checkpoints and instead of starting it all over. Next I shall train the network by iterating over the epochs, i.e. iterating over the dataset steps and iterating over the discriminator training for K steps and over the generator for only one step at this exercise. I use the k equals 1 and I encourage you to play with other values like 2 3 4 and 5 then I use TensorBoard for logging as previous lecture every hundred epochs then I save the check point to resume the training from there so I left the model to be trained for 1400 epochs and now we are ready to visualize the results and here again we can see a variety of conditions in the generated images different backgrounds.

The generator is trying to depict the same scene in which the bird exists either on a tree on a sea surface in the clouds in the sky and then it depicts the body parts of the bird either the tail the body the head, the beak and so on as an interesting experiment. Let's pass the same noise

vector to the generator but with changing the condition hypothetically I should get the same images style and the same theme but for different birds. That is I'm using the same noise but different conditions.

And since I'm training on 10 categories I will be passing the values from zero to nine as the conditions to the generator for the same noise. However here. What we get is the same image repeated 10 times what we expect is to get the same background. Same theme same style but different birds. But this doesn't happen. This is again a reported issue reported by the authors of the auxiliary GAN papers. It is an open research question. In a nutshell the problem is that the generator learns to ignore the conditions it focuses only on the noise regardless of the conditions given to the generator.

The noise will dominate. I would encourage you to read more about this problem try to come up with solutions and if you find one go ahead and publish it in a paper and you will nail it for sure. Now let's go ahead to TensorBoard to see the visualizations reported so if we open TensorBoard we can see the same pattern for the discriminator and the generator loss. We can visualize the min-max game the adversarial game whenever the generator catches up with the discriminator the discriminator improves and finds out more about the fake and the synthesized images and the loss for the generator increases and they go like this in cycles.

Ideally what you should get is a discriminator loss decreasing with the epochs and the generator loss decreasing with the epochs but whenever you get these kind of spikes in the loss it is an indicator of mode collapse. It is an indicator that you are not getting the right equilibria. The Nash equilibria of the training. Finding the perfect graphs for the loss for the GANs is a very challenging task and that's why as we have discussed in Section 1 The methods for evaluating the GANs training are still evolving but at least you know there is a problem when you see such spikes again.

If we go to the tab for images we can visualize the images that were resulted by the generator in the intermediate epochs and you can scroll backward and forward to find the optimal ones and use the model. that epoch that produced the best result. Remember that the model tends to diverge after some epochs.

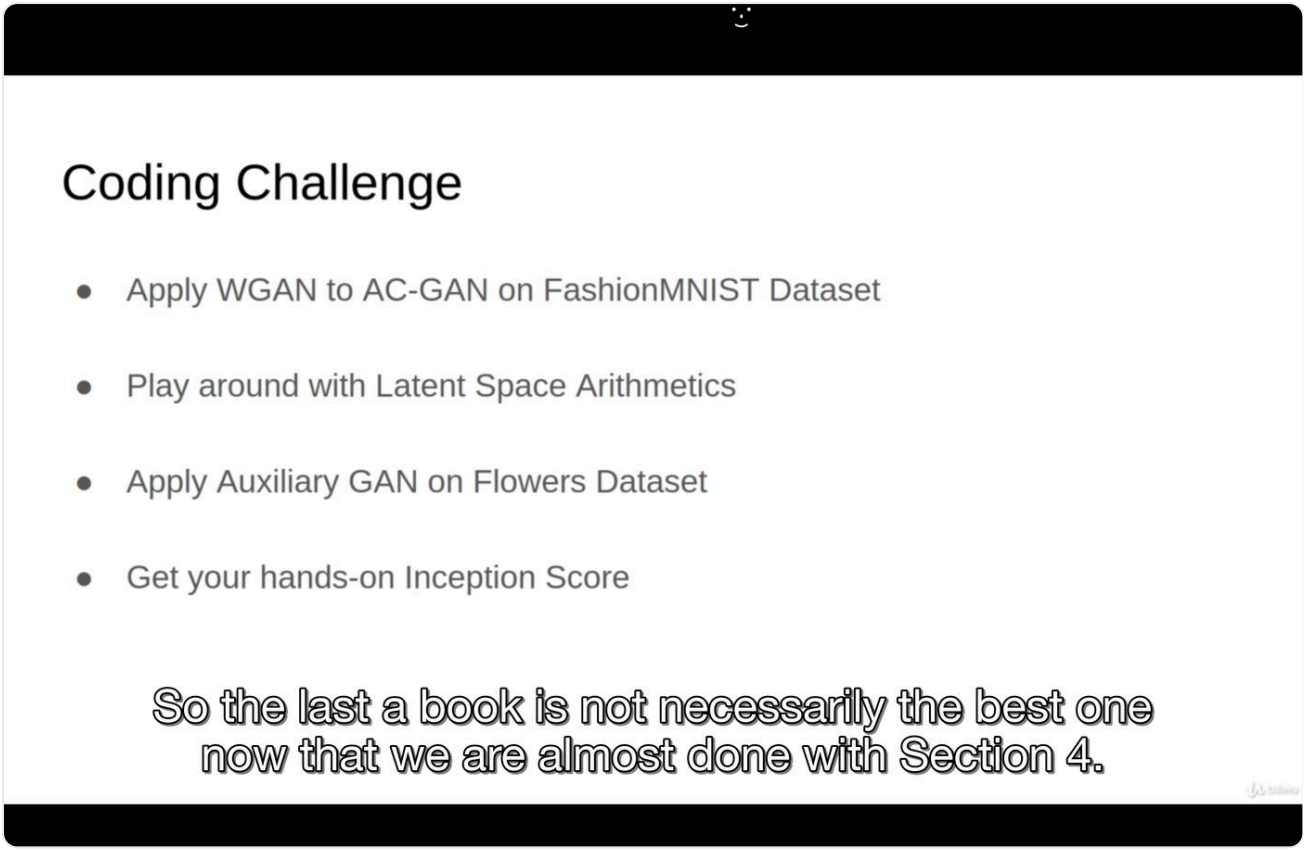


FIGURE 16.1 | Coding Challenge

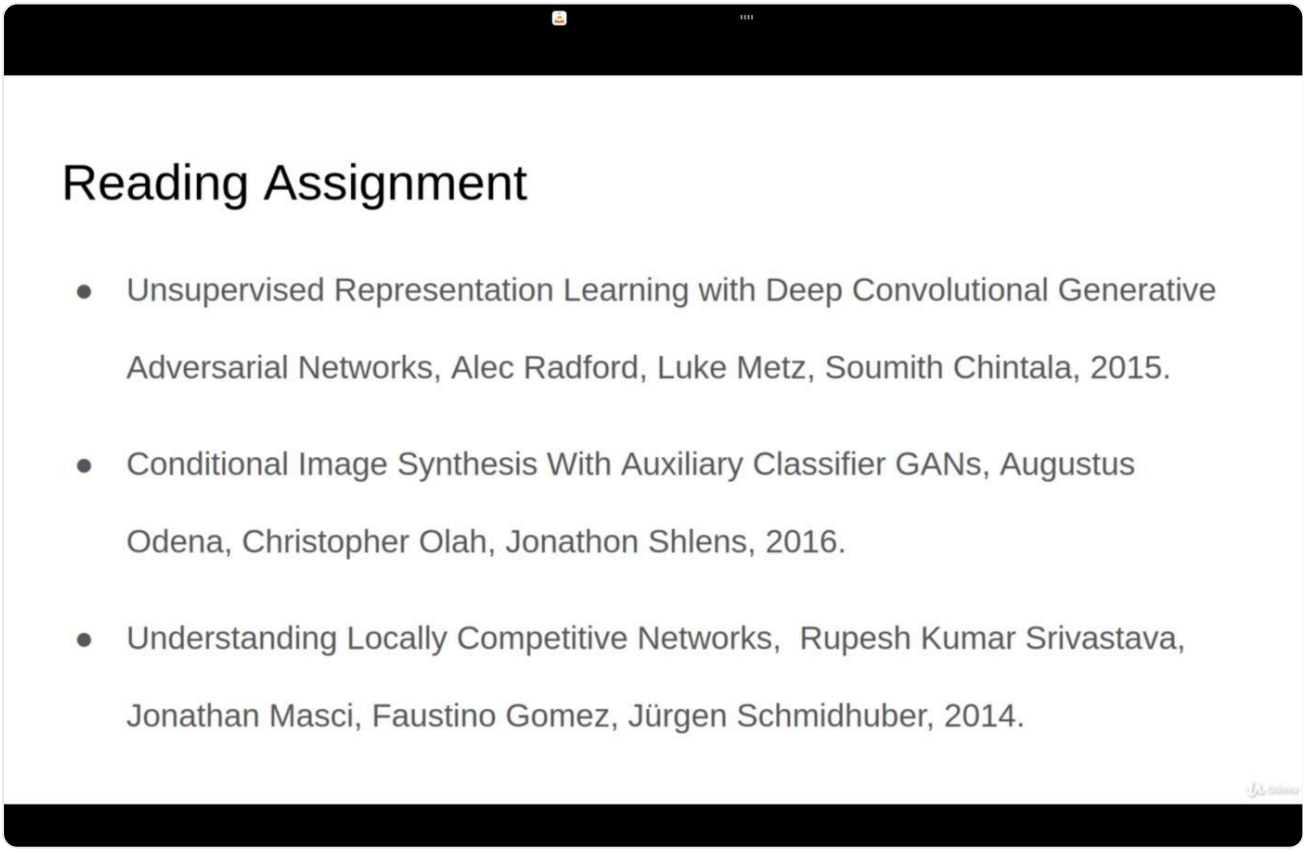


FIGURE 16.2

So the last epoch is not necessarily the best one now that we are almost done with Section 4. Let's review the coding challenge. I encourage you to implement the auxiliary GAN with Wasserstein loss.

Try to apply the GANs that you have learned in this section on the CIFAR-10 dataset or flowers data set. It would be great if you could get hints on by applying the inception score and play around with latent representations and here you are the reference to the original papers. For those who are interested in learning more and reading from the original sources congratulations on completing section 4 in Section 5 I shall introduce progressive GANs. Stay tuned and see you in the next section.

Thank you.

LECTURE 20

Introduction to Progressive GANs

Hello and welcome to a new section in this section we explore the workings of progressive GANs in the first lecture.

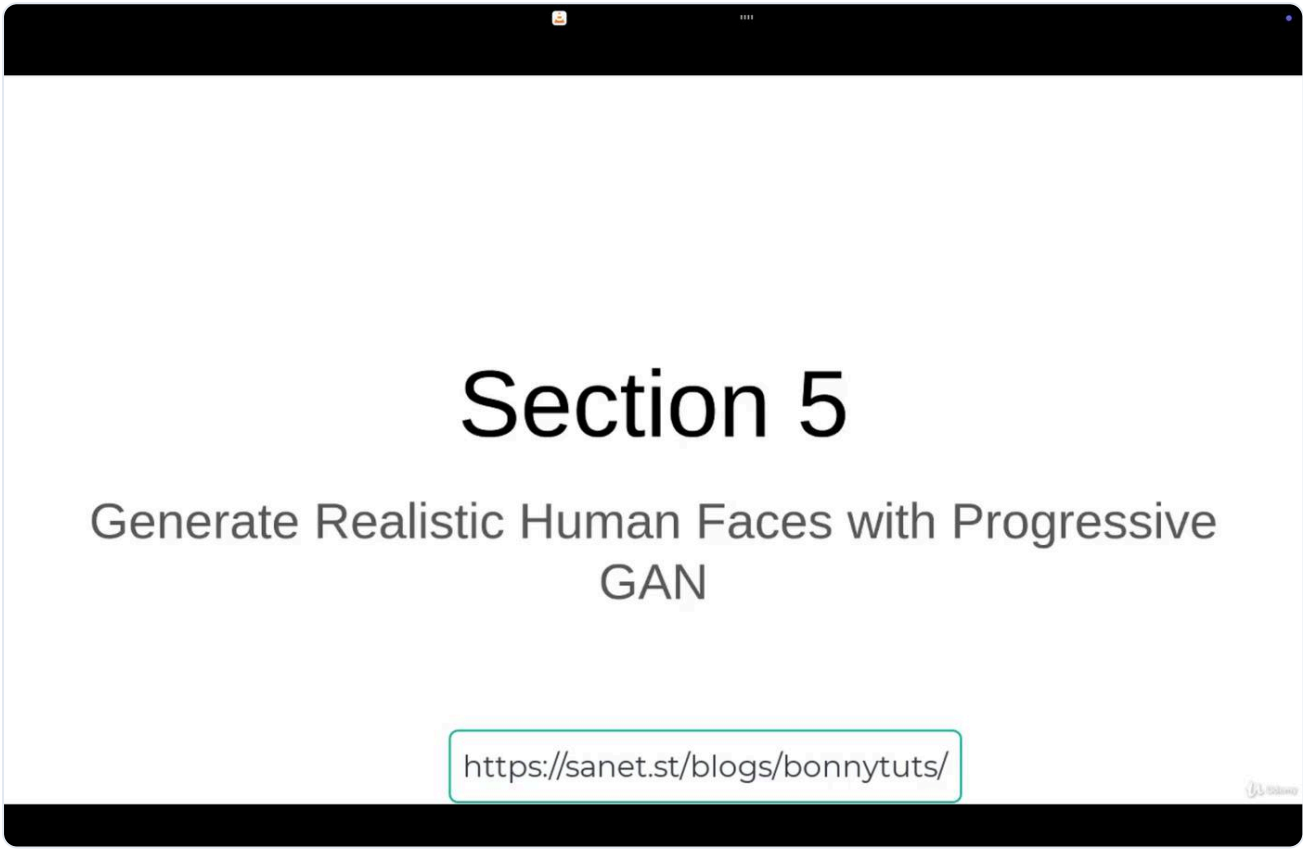


FIGURE 17.1

I shall lay the foundation of progressive GANs. Let's get started.

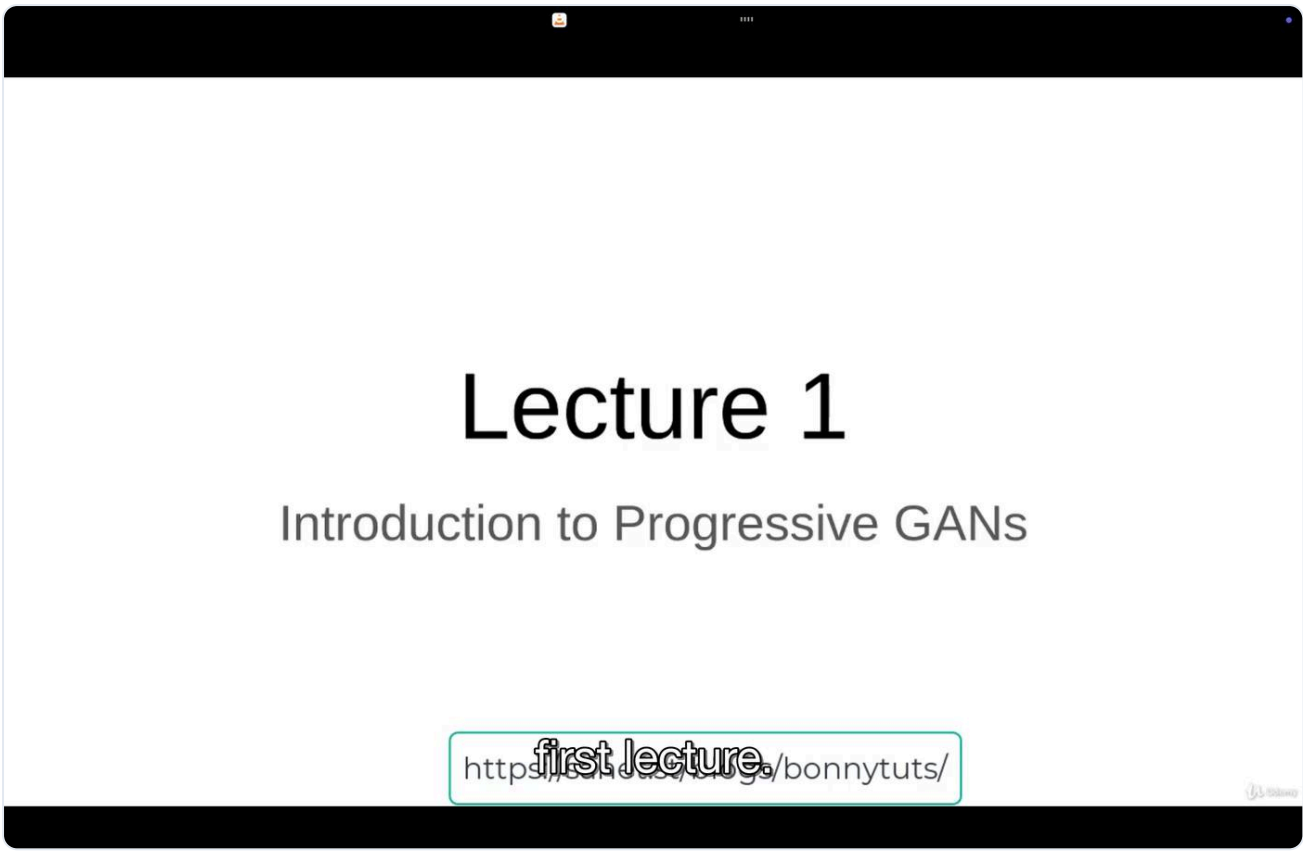


FIGURE 17.2 | Introduction to Progressive GANs

Similar to the different variations of GANs which we have explored in previous sections. The motivation behind the progressive growing of GANs is the same.

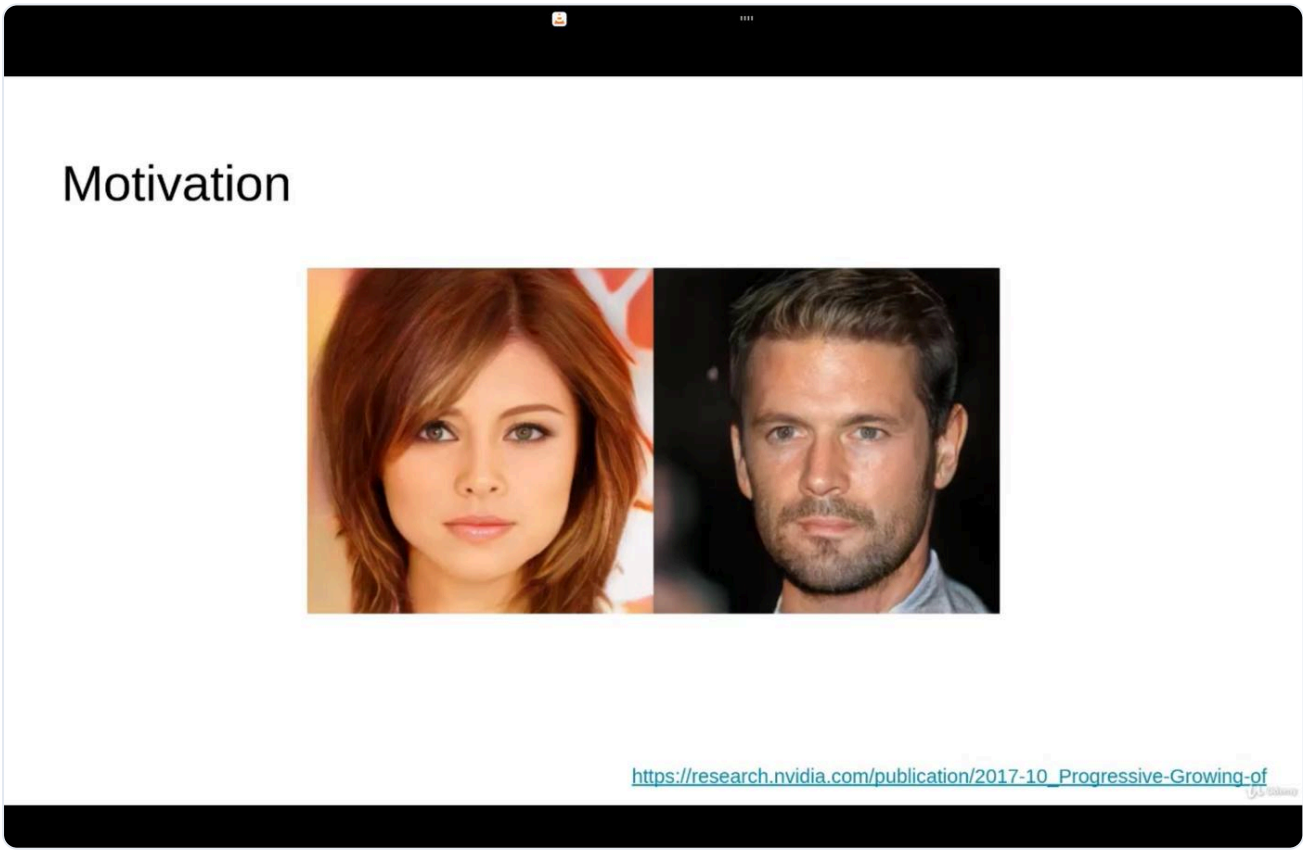


FIGURE 17.3

This method aims at solving mode collapse, a problem and increase the diversity of synthesized images. However generating high resolution images is the primary focus of this method.

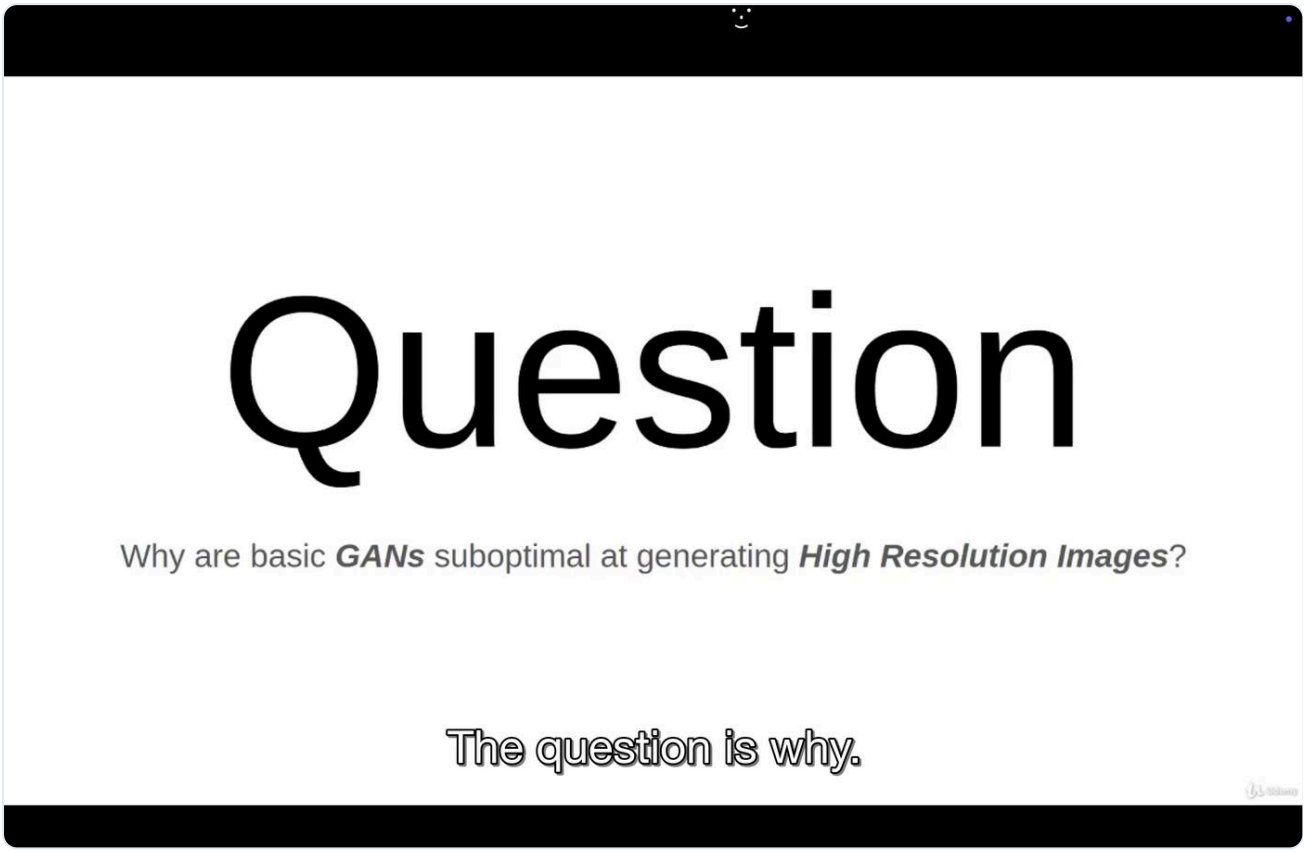


FIGURE 17.4 | The Question Fs Why

The question is why. GANs are not that good. When it comes to high resolution images it turns out the answer is fairly simple. At a high resolution the discriminator finds it very easy if not trivial to tell the fake from real samples. This behavior hurts the gradient flow in the network and the discriminator dominates the generator. And that's where the adversarial game is broken.

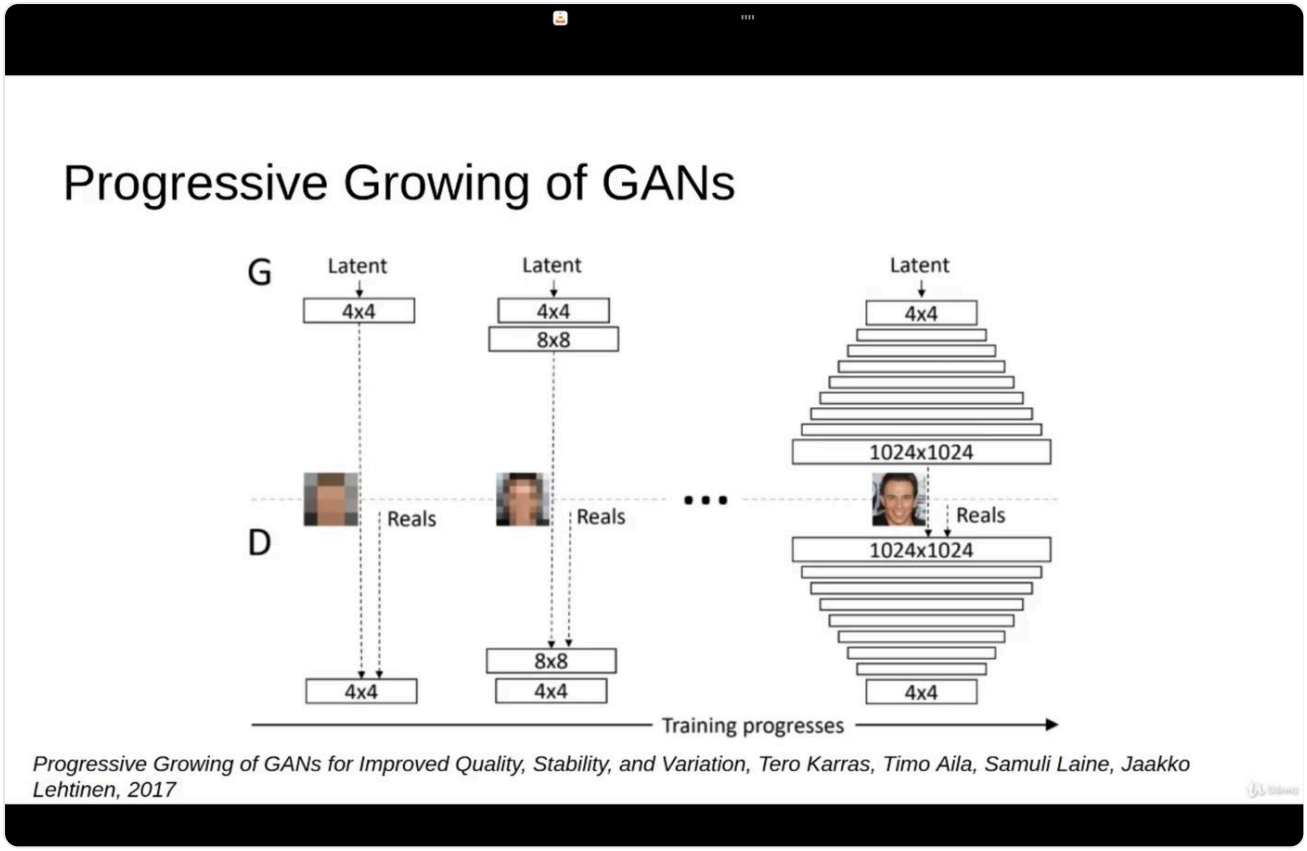


FIGURE 17.5

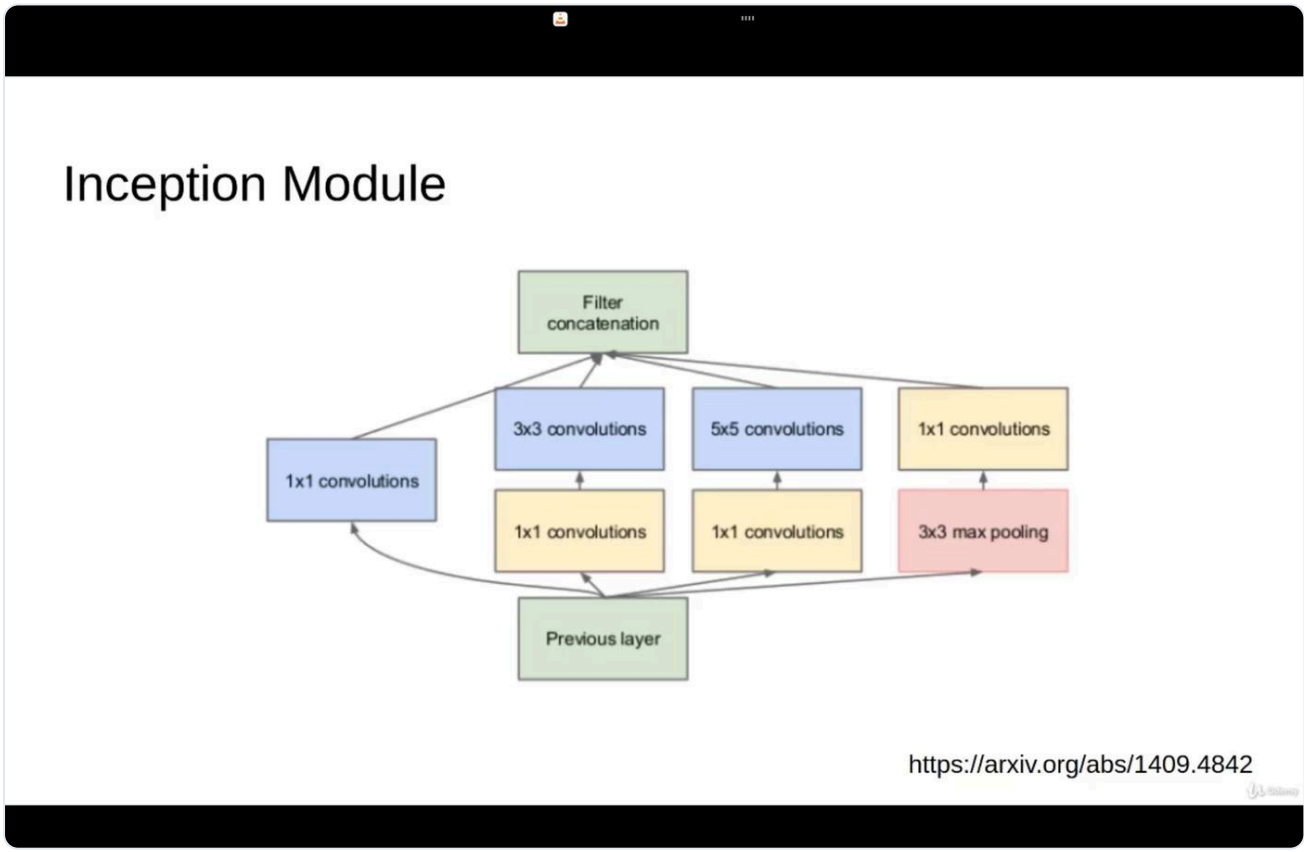


FIGURE 17.6 | Inception Module

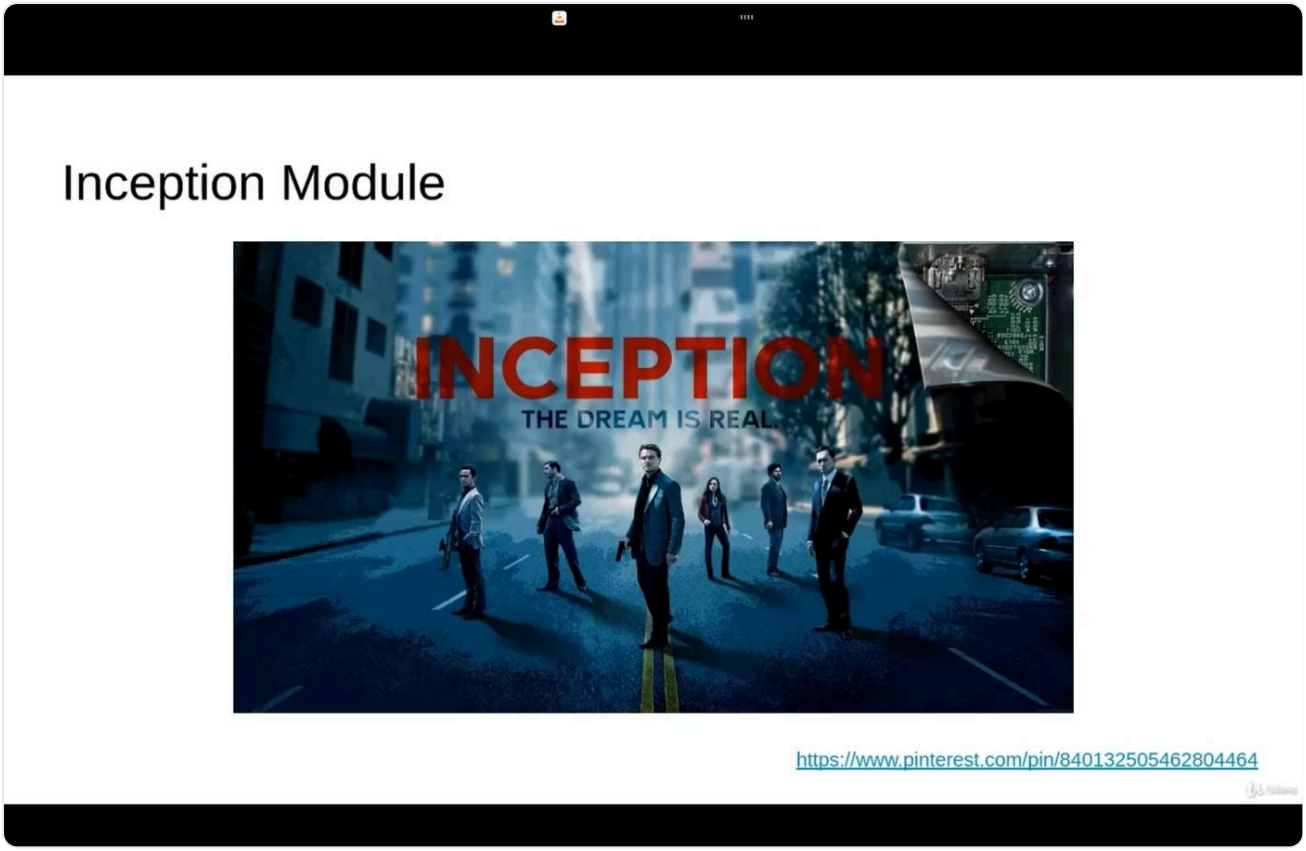


FIGURE 17.7 | Inception Module

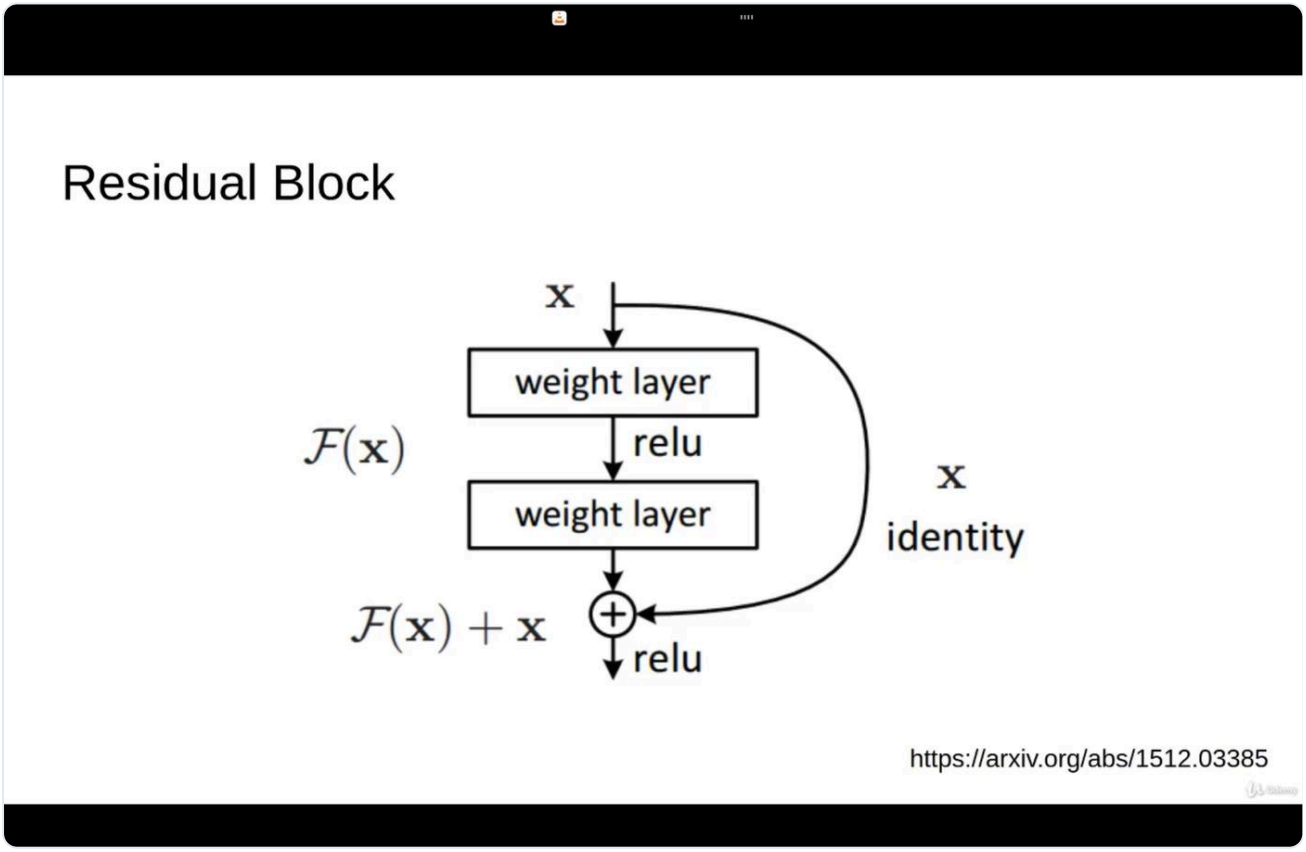


FIGURE 17.8 | Residual Block

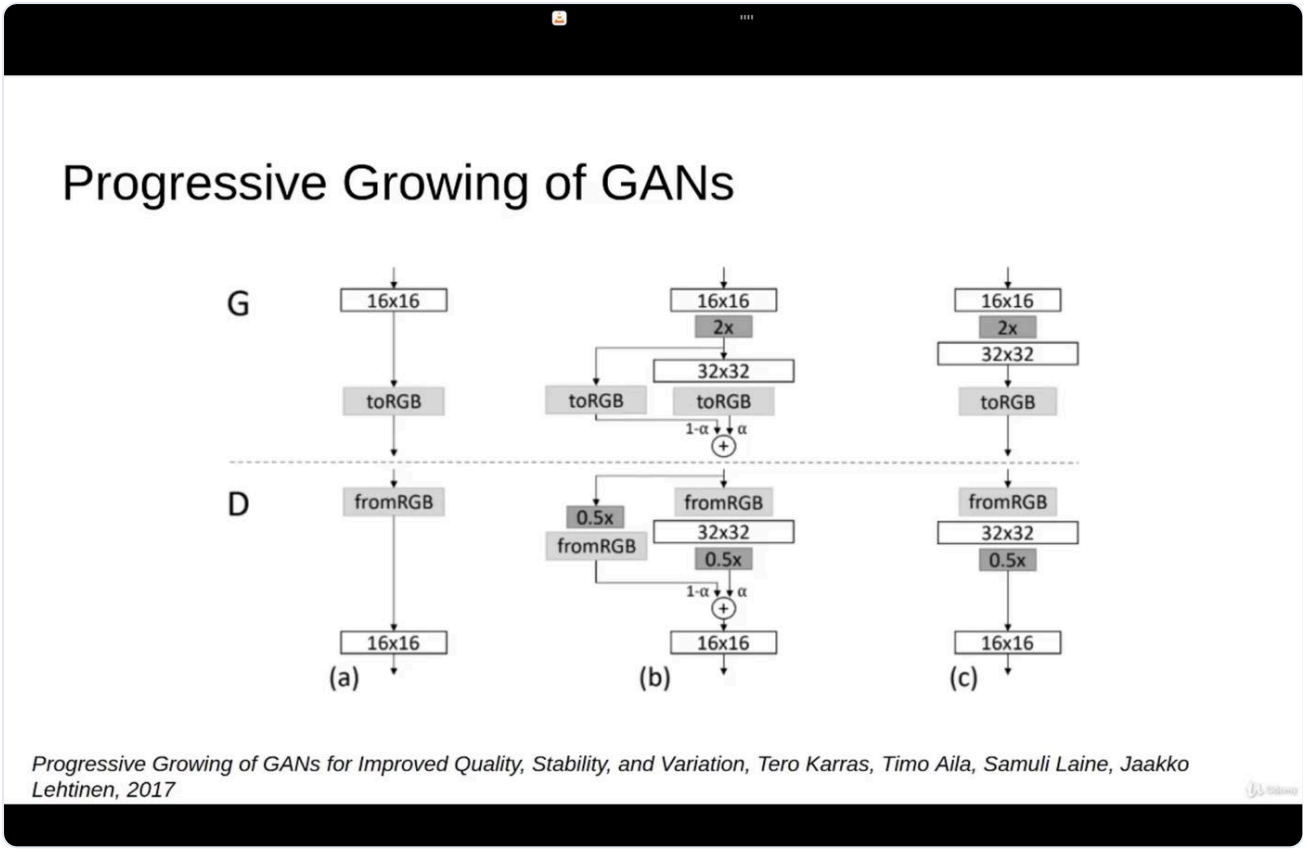


FIGURE 17.9

The progressive growing of GANs method is based on the idea of incrementally adding more layers to both the generator and the discriminator as the training progresses.

In the first epoch the generator and the discriminator both work at four by four resolution at the second epoch they both work on 8 by 8 images. It keeps on progressing till it arrives at 1024 by 1024 images. The rationale is that this method gives the generator a chance to learn the high level structure of the image and slowly learn to add fine details with time. This is a very intuitive idea and yet very powerful and as a side effect the training converges faster since most of the training time is spent dealing with lower resolutions which is relatively of lower computational cost in order to fully understand how the layers are grown.

Let's review a few building blocks first the Inception module is the basis for a complete family of deep learning models Inception modules excel in both image classification and object detection tasks. There are many variations of Inception networks at the core of an inception module is the usage of one by one convolution the one by one convolution functions as a feature reduction mechanism which reduces the number of channels of the input as a result.

This leads to reduction in number of operations and increases training speed. I'm not very sure whether the name Inception network was taken from the inception movie. It makes sense however because the movie involves the idea of a dream inside a dream which is similar to the inception network, a network inside the network. It's a good movie to watch by the way. The second building block is the residual block. One of the pioneering Deep Learning Networks for computer vision is the ResNet. ResNets use residual blocks to solve the problem of vanishing gradients in

very deep neural networks. Residual networks work by switching a layer on or off allowing the gradients to pass unchanged backwards ResNets are used in many research papers especially GAN papers that I have read.

So now let's answer the question of how the layers are added to the network in training time.

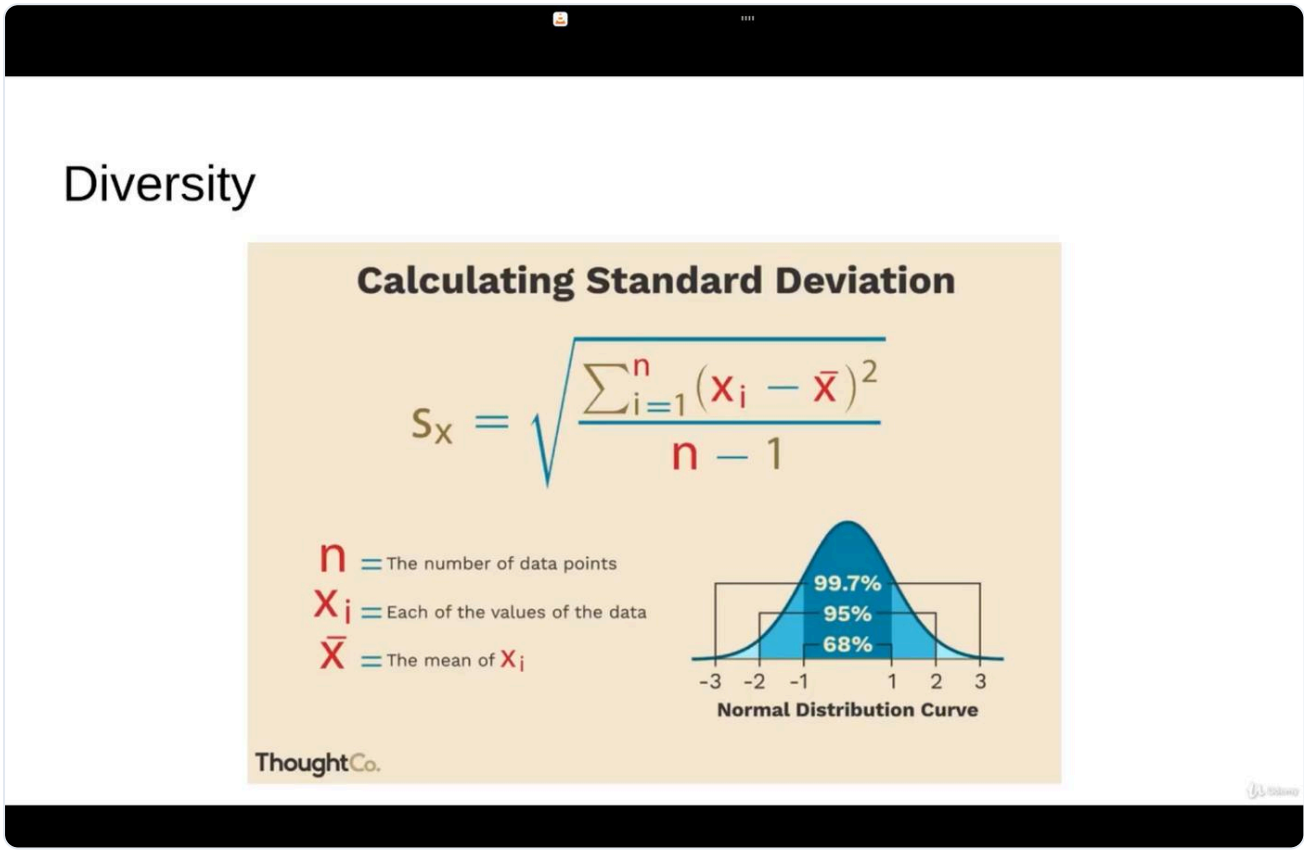


FIGURE 17.10

If the layers are added suddenly then the performance changes we need to add the layers slowly with a smooth transition.

In this graph on the screen right now the left side represents a model which handles 16 by 16 resolution in order to increase it to 32 by 32. Another convolution block is added. However it is treated as a residual block that is there are two paths, the network can skip this block the same way a residual block works in ResNet. There is a parameter Alpha that functions as a factor deciding how much the new layer should contribute to the network. The weight alpha falls in the range of zero to 1 and as a result the factor 1 minus alpha decides the weight of the original network.

Another consideration to take here is how do we feed the real images to the discriminator during the alpha transition. The answer is simple the same alpha value is used to mix the lower resolution and the high resolution versions of the real image. So in this example we downsample the real image twice, once to 16 by 16. And the second time to 32 by 32 then merge and add them together according to the alpha weight. Remember that original real images are available before training at the target resolution which is 1024 in this case the progressive GAN encourages diversity by appending a non-learnable layer at the end of the discriminator.

The diversity layer calculates the standard deviation for each pixel over every mini batch the standard deviation serves as an indicator to help the discriminator decide whether a sample is real or fake. For example if the image deviates too much from these statistics then it is fake. The authors of this paper have tested other statistical measures as well. They found that standard deviation was the only successful measure for such a task.

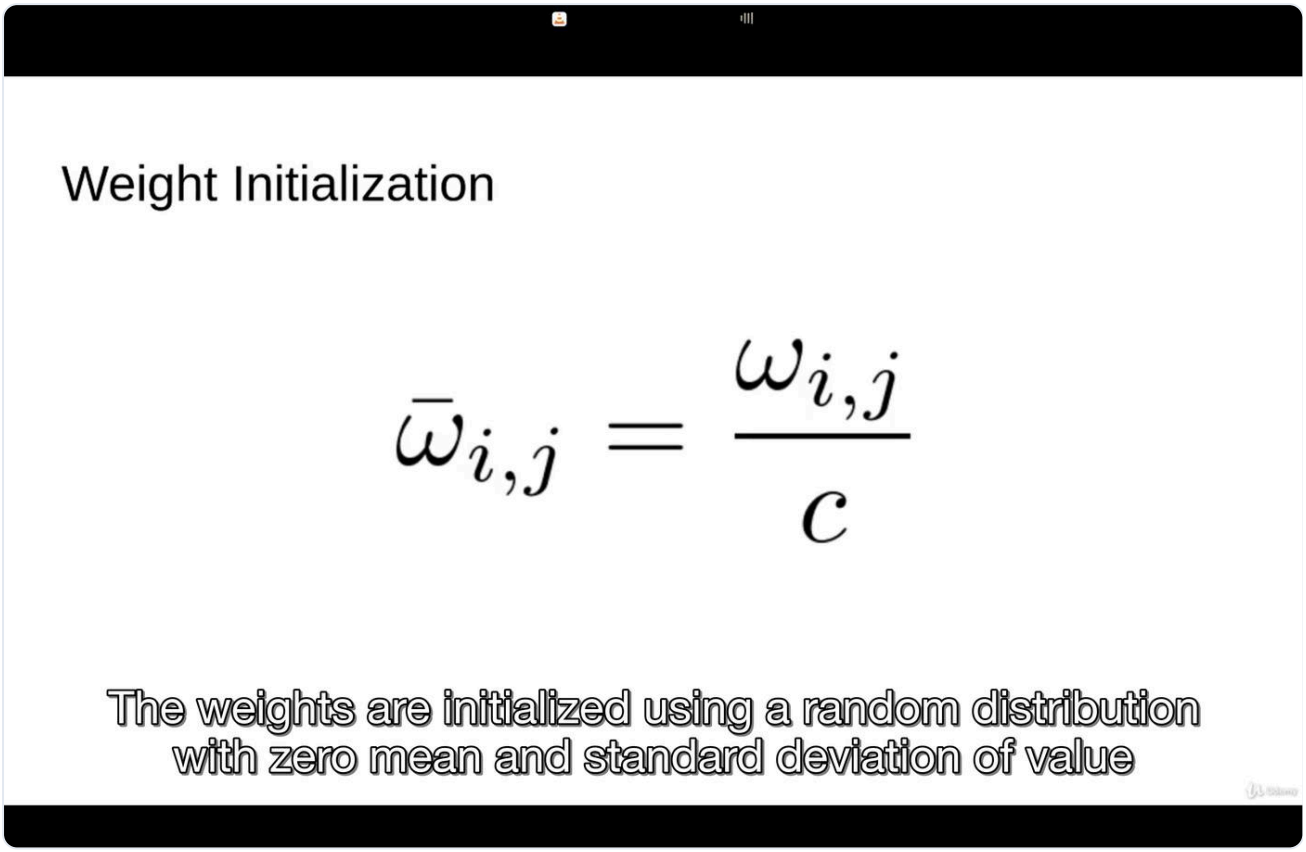


FIGURE 17.11 | Weight Initialization

The weights are initialized using a random distribution with zero mean and standard deviation of value 1.

The weights are normalized as training progresses by dividing over a factor C C is calculated based on the square root of the number of inputs to a layer. This ensures a dynamic learning rate adjustment based on the value of the gradient. That's why it prevents some weights from getting stuck and encourages them to adjust faster.

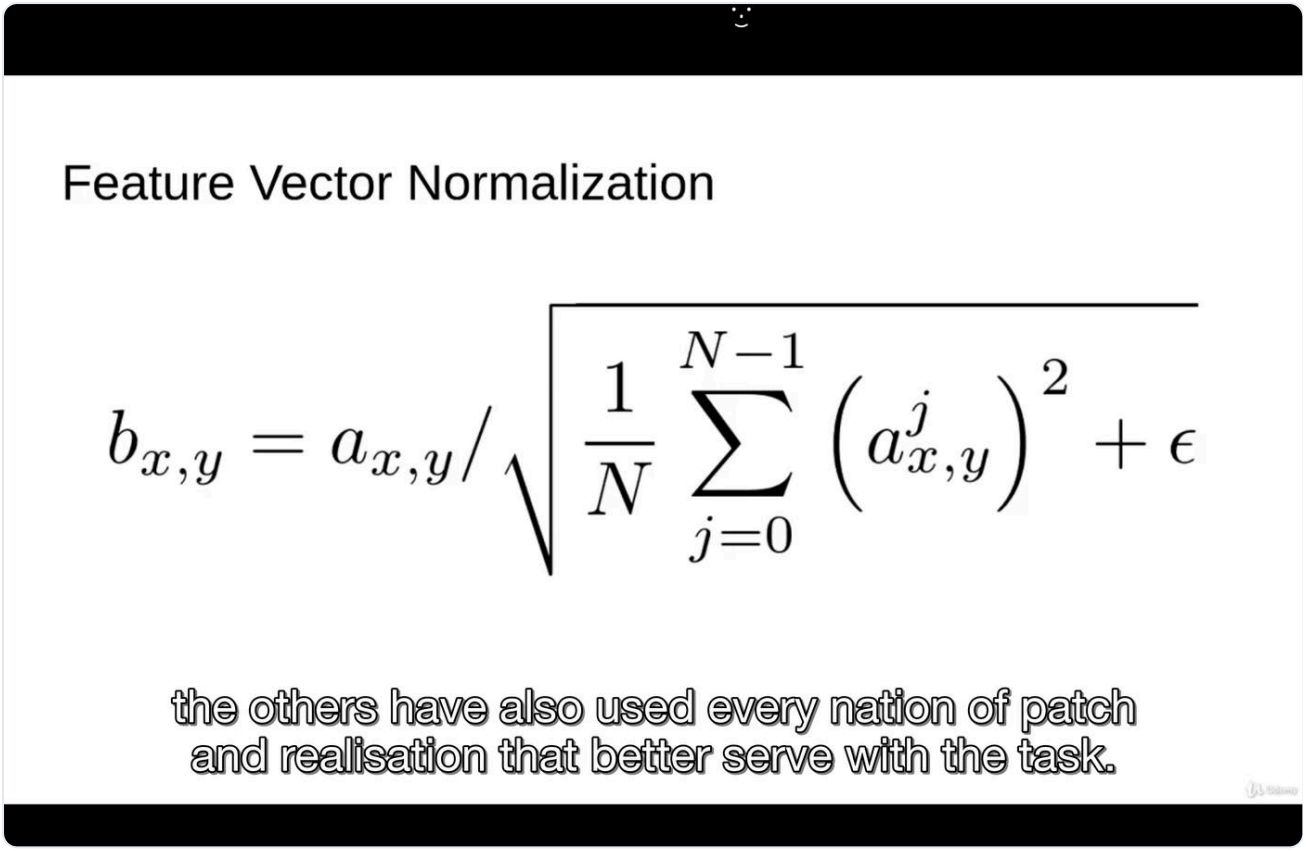


FIGURE 17.12

This is inspired by Xavier initialization and adaptive learning from RMSProp and the Adam optimizer the authors have also used a variation of batch normalization that better serves the task.

This normalization was used after each convolutional block of the generator in the formula, N stands for the number of feature maps the variable a stands for the old value and b stands for the normalized value. In simple words this divides the pixel value by the square root of the sum of the pixel value over all samples in the mini-batch X and Y are the pixel indices in this case and in the next lecture you will implement the progressive GAN on human faces.

Thank you and see you in the next lecture.

LECTURE 21

Conjoined Triangles of Artificial Intelligence

See the notebook `12.Implement-Progressive-GANs-on-Celebs-Dataset.ipynb` for the code.

Implement Progressive GANs on Celebs Dataset

Hello and welcome to a new lecture. In this lecture you will implement progressive growing of GANs. This is the most challenging lecture in our course. So grab a cup of coffee and let's get started based on our discussion of the theory behind the progressive growing of GANs in the previous lecture. I came up with this task breakdown the code below is organized under these main headlines. The first item is the pixel-wise normalization which I implemented as a custom PyTorch layer.

Next I address the weights normalization method which is proposed by the authors of the paper. Third on the list is how to actually implement a growing model. Then number four is another custom layer that is appended at the tail of the discriminator to calculate the pixel-wise standard deviation over a mini batch. Finally I implement a method to resize the images as the growing level of the model either four by four or eight by eight etc.. The code is based on the benchmarks referenced here but I implemented it in my own way and my own style so that I can highlight and visualize functionalities for illustrative purposes.

Finally I would recommend that you watch this lecture twice and spend some time playing with the notebook to wrap your head around it over here. I start by creating a global variable named Max levels which indicates how many times I grow the network in order to handle resolutions either four or eight or 16 or thirty two and the up to 256. Feel free to change this value and share the results with me. Let us begin by exploring the data set. Here you can find the reference to download the dataset.

Make sure that you have downloaded the code and unzip it inside the directory named celeba located at the root of the notebook files I use the image folder API to load the data and compose the processing transformations. The faces are already detected, cropped and aligned but I only need to apply resize and center crop to make all images 64 by 64 in width and height then I create a data loader and calculate the number of steps next. I use matplotlib to visualize a mini batch of images all the images contain human faces of celebrities.

Here you can view a mini batch of the dataset here the fun begins. We start by a custom layer for pixel wise normalization. The title of this section in the original paper is pixel wise feature vector normalization in generator. And here is the equation. So again we divide every pixel value by the square root of the mean of squared pixels in the same location across the mini-batch the class PixelNorm layer inherits from nn.Module and implements the constructor and the forward method you can notice here there is a term which is added at the end of the equation.

The term 1e-8 is a factor recommended by the authors of the paper to stabilize the value the next few blocks demonstrate the output shape of various operations which are used to calculate the pixel wise Norm. This way you can understand why we use dimension parameter equals 1 and set the keepdim parameter to True the real interpretation of pixel-wise norm is best understood by visualizing the original signal and then normalize the signal.

That's why I create a random tensor and then pass it to the pixel-wise norm layer then plot the output before and after normalization as you can see here the original signal is in the blue color and has many spikes which cause the model to exhibit instability. The orange curve on the other hand is a trimmed and regularized version of it. I can't stress enough how visualization is a key skill for you as a data scientist under the title equalized learning rate is again another regularization formula to help stabilize training further the weights of the network are multiplied by a factor C which is based on a variable n which is the number of inputs connected to a neuron it is implemented as a torch.nn.Module and uses the method numel which calculates the number of elements in any PyTorch operations.

You can see I highlight the method numel, you can reference it in the original documentation of PyTorch as a reminder. Here you go a visual form which we discussed in the previous lecture to help you build a visual map of the process of growing layers. I can break the task down to three steps in the first step I design a network that handles the image at Resolution for example 16 by 16. Then I add more layers to the model to handle resolution 32 by 32 which is double the value of 16 by 16.

But in order to make the transition smooth we need to implement the alpha transition between the two resolutions. The Alpha transition acts merely like a weighted average where we have one minus alpha of the previous resolution plus alpha of the new resolution and as alpha grows the weight of the previous resolution decreases and the weight of the new resolution increases. This is the basic idea upon which the lengthy code below is based to make our code more generic.

I create a method to return a sequential module of a basic convolutional block for the generator it uses transpose convolution and ReLU activation then I do the same for the discriminator but instead I use convolution layers with Leaky ReLU notice here how the batch norm is always between the convolution layer and activation function. Note here how the batch norm layer is always between the convolution layer and the activation function.

This is always the case for both the generator and the discriminator also notice how the pixel norm layer is after the activation. There is actually a tricky interview question that usually asks where to place the batch norm layer in your model. So be aware of such tricks I use the same global variables from Section 4 for DC-GAN. However notice here the recommended value for both ngf and ndf is much smaller. I have reduced it to quickly debug the code.

Now I create a generator class that inherits from torch.nn.Module I trigger the constructor of the parent using the method super. Then I create two module lists. The question now is what is torch.nn.ModuleList as it's a new data structure available in the PyTorch API. In previous lectures I have used torch.nn.Sequential and this one however I shall use torch.nn.ModuleList. It's a list of torch.nn modules, and it's a data structure that is friendly with modules operations.

That is we can initialize the object with list of modules and append the modules on the fly to the list. This is useful for the backward path of the back propagation. This will come in handy especially that I am interested in growing the network at later apex the blocks variable holds the internal convolutional blocks of the module to_rgb holds the output layer for every resolution to convert to the number of channels from whatever value resulting from the intermediate blocks to three channels for RGB. And by the way.

Have you ever thought about using other color spaces other than RGB for images. For example HSV. Think about it and share the answers together then I create a convolutional blocks to map the noise vector to ngf by forty eight channels. This one is always executed regardless of the resolution of the target. Image. So now we need to create convolutional blocks and add to_rgb output blocks and index them by the target resolution in our module list.

Here I implement a for loop over the number of resolutions supported and for every resolution I create. both a convolutional block and a to_rgb block the number of input and output channels can be formulated in a generic way by multiplying ngf by two power i for input and two power i minus one for output. However for simplicity and in order to visualize it you can print that resolution and trace it by replacing i with values 5 4 3 2 on one and track the outputs.

Think of it as a sketching exercise you can do it using a paper and pencil. The idea is quite simple. If the target resolution is four by four then we use fewer layers. Otherwise we use one more layer as the resolution increases. I will visualize this concept in a nice way below so in previous sections we have created a custom function to initialize weights. And this one I demonstrate that it is also possible to initialize weights inside the module structure using the self the module the method the method should put an end to it.

Timing in normal method is used to initialize weights. This is the PyTorch implementation of initializers which are a modification of the Xavier initializer the forward method now should accept two parameters one for the input mini batch and the other for the level of the growing of the model. Indices starting by 1 for 4 by 4, up to 5 for 64 by 64 and six for 128 by 128. Finally seven for 256 by 256 for a smooth transitions.

The level is a float decimal number that could take many values between integers such as two point one or three point four etc.. Let's use the decimal value as the value for the Alpha transition where alpha equals level minus the integer value of the level. For testing I set Alpha transition to false. However feel free to enable it by setting alpha to True and make the required modifications to the output function of the generator and instead of tanh don't use any activation function at the output.

Just use linear activation output instead of tanh at the end of the generator so the alpha equals level minus the integer value of the level. Then we round the level up and use the max levels as an upper bound. Then we pass the inputs through the first convolutional block using self.blocks of 0 and assign the return feature map to a variable named the feature map then implement a loop depending on the target resolution indicated by the level variable at each iteration pass the feature map to the convolutional blocks at that level so remember when I told you to break the task down to three steps.

Let's skip the alpha transition for now and assume that we are done. And next we pass the output feature map to self.to_rgb of level minus 1 since they are zero indexed but the level in our case it starts from 1. So this is the simple straightforward case. However assume that we are in the middle of an alpha transition and check for that by the if condition if the current level is greater than 1 and if the iterator is at the next to last convolutional block and if alpha is not zero then we need to resample the output to pass it to the to_rgb layer.

For example if the current level is 16 by 16 so I use the 8 by 8 image for the alpha blending. So if the alpha transition is true let's multiply the main output by Alpha and sum it to the previous output multiplied by one minus alpha as a final notice on this snippet I have implemented my own convolutional design and set my own parameters. However in the original paper it is recommended to use upsampling in order to double the size of the output from the previous layer.

I have commented it out since it wasn't very necessary in this case. I keep it as a guide for you if you want to change the parameters later. Now for the discriminator we use the same tricks but in the opposite direction for the sake of brevity let's head directly to the pixel-wise standard deviation layer at the end of the discriminator in order to implement the standard deviation of a mini batch layer. I use torch.std, a method which is used to calculate the standard deviation over the mini-batch of the feature map variable.

Notice here how dim equals zero. Ensure that the standard deviation is calculated for every pixel over the whole mini batch. Then I concatenate the standard deviation to the feature map. But notice here I use the method unsqueeze to insert that dimension and then expand_as to shape it similar to the feature map. Otherwise the torch.cat method will throw an exception. Finally the valid either fake or real output is returned as a single value.

Now let's test this code works before we make it even more complicated and try to create the model. I use another visualization library which is torchviz. My goal here is to show you as many visualization tools as possible so they can have many tools in your tool box. I need to create a discriminator object. Net D and then pass it a tensor filled with zeros just as a placeholder and set the dimensions of this tensor to be four by four which stands for a four by four image and set the level to one and then visualize it using the make_dot method.

Notice here that a graph that is printed in the backward path for calculating gradients of the back propagation and not the actual network itself. Well it's as I said a tool in order to visualize the size of the network. The main idea here is that at resolution 4 by 4 the network is at its smallest size. Now let's see is another tensor of zeros. But this time we set the size to 64 by 64 and we need to set the level to 5 and inspect the graph. And while our network has grown to its maximum size this is actually the main concept behind the growing layers of GANs the network is small when the resolution is small.

The network is large with more layers when the image and the target resolution is higher next I do the same for the generator and make the same observation and it becomes clear that our networks shrink and grow progressively as desired Is that enough. No it is not enough. Let's verify that the output of the discriminator is always a single binary value either real or fake and also verify that the output image resolution of the generator at each level is the required resolution.

For example if level equals 1 the output is almost four by four by three for RGB channels and so on. But more importantly the output at intermediate Alpha transitions it should be also set correctly. That's why I implement a for loop with the step point two between 1 and 6. Exclusive or the max levels. Then I create a mini-batch of random noise then verify the output shape of the generator again is the expected output shape at each level and let's see how I'm using a formula to make it generic as the level progresses the desired output shape is also calculated using that formula and I throw errors in case of mismatches between the required resolution and that output predicted resolution.

Here you can see how the output shape progresses at every level from four by four up to 256 by 256 and I am confident that my code is syntactically correct and I can train it without surprises. This is another concept that I'd like you to always do at work which is unit testing always test the small few lines of code separately and automate the test. Otherwise the code grows in complexity and you get lost later on when you try to debug it. Now the next blocks of code do exactly what we have been learning from Section 1 up to this point.

We convert models to GPU create optimizer and create the loss function and use the fixed noise for visual inspection. So I will head directly to the two main ideas remaining in our agenda. First how we decide the level value during the alpha transition and then how to down sample the images for lower resolutions. Now I define a method to update the level as the training proceeds between epochs and steps. The method update level takes in the number of current epoch The current step the total number of epochs and the total number of steps.

And also Max levels. It simply scales the value down between 0 and 1 and then it scales it up between 1 and max levels. And finally it returns the result by using a min-max scaler as you can see here the highlighted digit 4. I set it manually to start between 4 and Max resolution. And this way I avoid the very low resolutions 4 by 4, 8 by 8 and sixteen by 16. Since I found them very unnecessary during the training. This of course contradicts the original paper which suggests that you should start with four by four resolution but because of the power of the GPU device I'm having and of course to the one you would be having wouldn't be very powerful more than that.

So unlike the authors of the paper who work at NVIDIA, the house of GPUs in the whole world. We use a limited GPU power so I started directly by the resolution 32 by 32. And if you'd like to change that and experiment you can change the value 4 down to 1 and that would bring it back to the scale 1 to max levels but at the moment I set it between 4 and max levels. So again based on the concept of unit testing I'd like to test this snippet of code.

That's why I print the level at different random values that cover the range. Assuming total epochs 50 and total steps 20 it is easy to see that we get nice distribution of values and it's most alpha transitions and decimal points as one. It is easy to see that we get nice distribution of values and most alpha transitions as decimal points as well. Now I define another method downsample_minibatch to resize the original real image according to the current level.

Here I use adaptive average pooling which is a PyTorch method that applies average pooling which is exactly the same effect as down sampling the image. Feel free to use the non-adaptive version as well while you're trying to fine tune the hyperparameters. Now let's visualize the

downsample method to make sure it works. So I started by resizing the image to resolution four by four and here we can see the results at the resolution four by four. I do the same again but for a resolution 16 by 16.

And here we can see the results at that very same resolution. Now it is very easy and straightforward to train the network and expect the results. In reality the authors of the paper used a very powerful GPU for many days to train the model. So this leads us to the practices I share with you next.

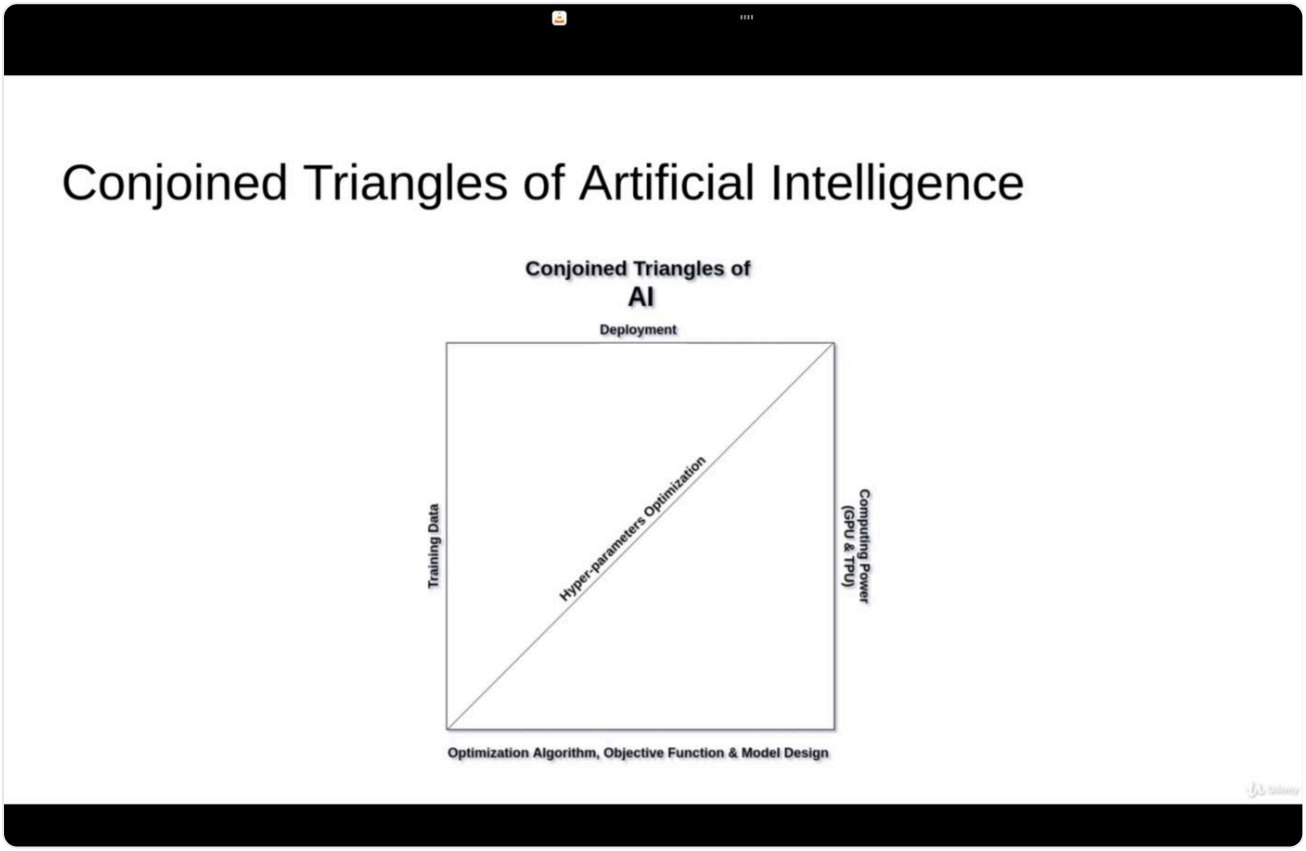


FIGURE 18.1 | Conjoined Triangles of Artificial Intelligence

I have come up with the conjoined triangles of artificial intelligence the conjoined triangles of AI explain the foundations of applying Deep Learning in real life do not accept a project if one of these elements is missing.

Otherwise you may run into troubles fulfilling your commitments. The first pillar is the availability of enough annotated data. The second is the availability of powerful computing hardware either in the cloud or on the premises then if there is a public model or research paper addressing a similar problem you are ready to work. Otherwise make sure to explain to the stakeholders that you will have to invent your own design which is a trial and error process.

They should be aware of that and either accept or reject it. Finally you need to make sure that you can successfully deploy your model to target environment either web, mobile or even an embedded system. This is tricky since some custom layers and even built in layers are not interoperable in different environments. Be aware of this trap early on and do not ignore it while you fine-tune the hyperparameters of your model and training. It is also tricky to decide the size of

the optimal dataset but you have to empirically measure the loss or whatever metric you are using against the dataset size in order to determine whether you have enough data or not and now let's head back to train our network.

We use the TensorBoard to visualize the intermediate results by initializing an object of the class SummaryWriter exactly the same way we did in previous sections. And then we use the check points to load the state dictionary of the torch module for both the optimizer and the network itself so that we don't have to start the training over and over from the beginning. We can start from the best model we have got so far. Then I loop over the number of epochs, to loop over the dataset and then we train the discriminator for key steps.

Here I set K to one which I found to be optimal and then train the generator. And finally every ten epochs we print the results in terms of the loss of generator and the loss of the discriminator and the current level.

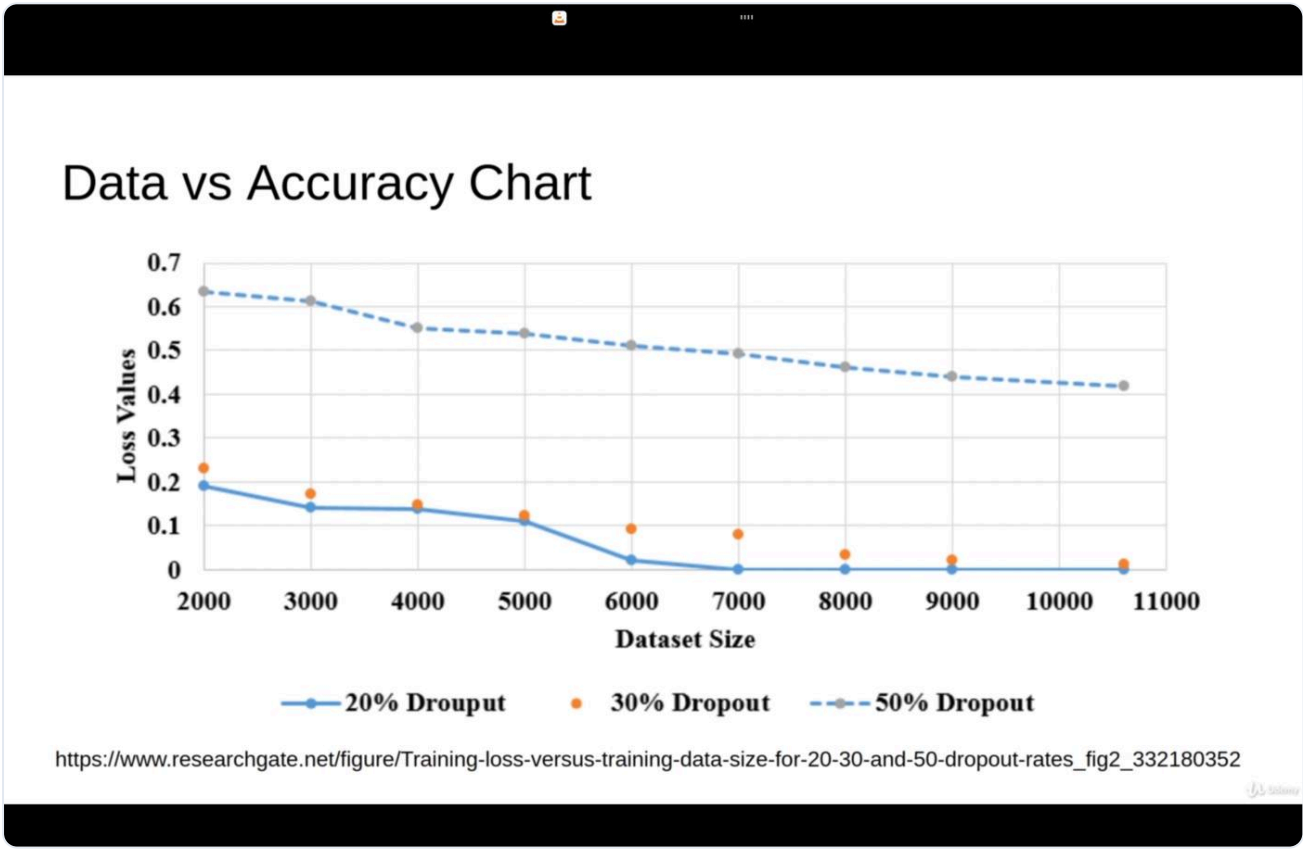


FIGURE 18.2

And we save the model, the state of the model and the State of the optimizer. And then I use the fixed noise to visualize an image of a mini-batch of the generated images by the generator at this step and use that to write it to TensorBoard so the idea I have used here is that I have trained starting from level 4 for 32 by 32 resolution and incrementally use the alpha transition up to 256 by 256 resolution.

Here is an example of the result I got at the end the resolution at the target resolution of 256 by 256. It is very clear that this model is undertrained and we need to train it further while we can see here the shadows of a human face. And it was not very far and we will inspect now the

intermediate results using the TensorBoard image gallery and see how far the model has got whether it has converged or diverged with the training and use that to fine tune the learning rate hyperparameters.

So now in order to use TensorBoard, type the command tensorboard --logdir=./runs and hit enter then I use the link to open in your browser window for the TensorBoard dashboard so hear from a previous run since I have trained this model using checkpoints over three different runs. That's why the images are logged under three different tabs so I start here at the first steps. It wasn't clear what is happening then I'll click the arrow button on the keyboard to see the next steps I can see here it is converging to something very close to a human face quite realistic by the low resolution 32 by 32.

And further it diverges and converges back again. You can see here that the results are much better and quite realistic. But then when it shifts to the next resolution 64 by 64 the results has diverged again. That's why I trained this for another epoch and let's inspect that in the next run. So here is the next run that started by 32 by 32. The one afterwards which is more about 64 by 64. You can see that at a higher resolution Max resolution. It has diverged again and that's why I need to be trained further as the transition function, the update level function which shifts the scale between 1 and Max resolution or the max resolution that gives less weight to the higher resolution.

And that's why we can play around with these values to let it train more on higher resolutions or the last resolution bar again as we have seen in the notebook.

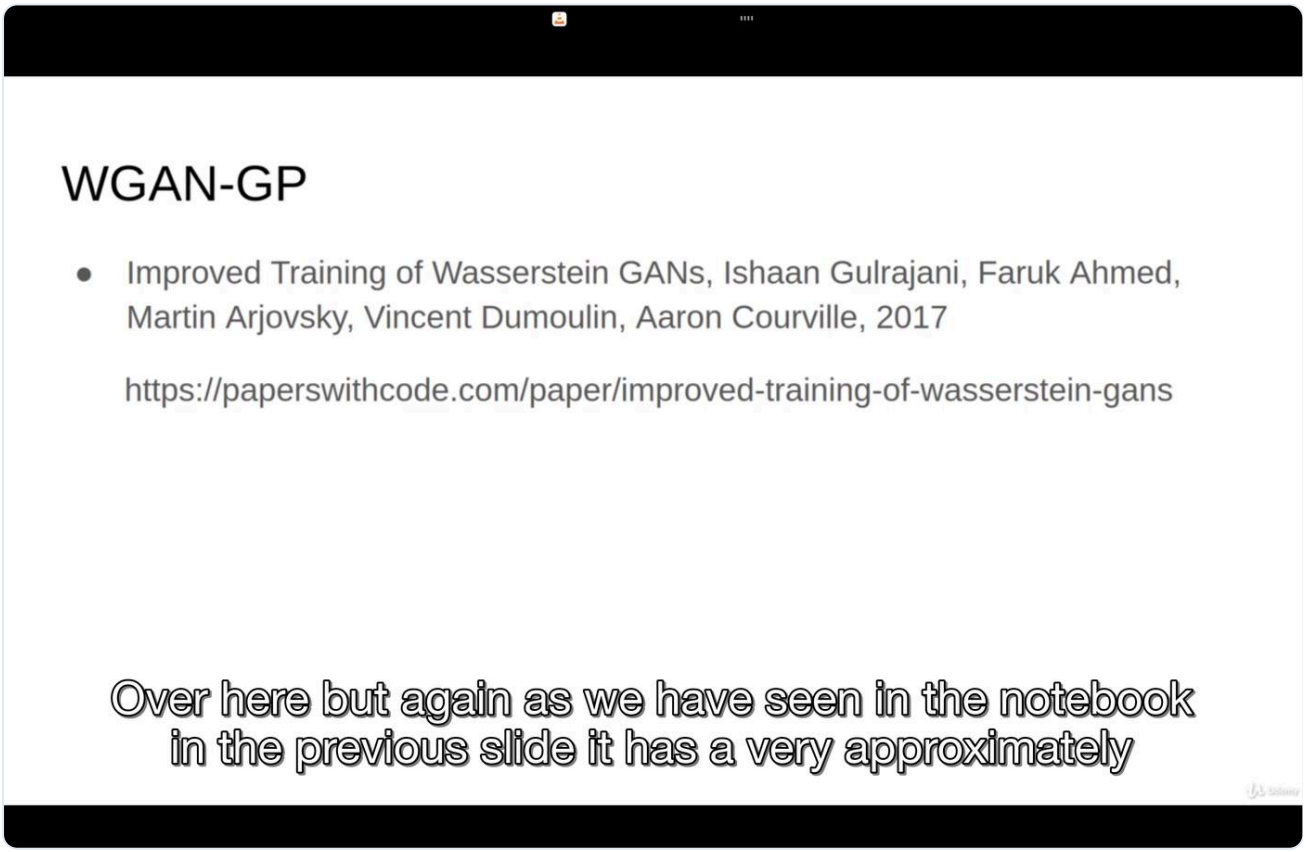


FIGURE 18.3

Over here but again as we have seen in the notebook in the previous slide it has a very approximately got very close to the desired output of a human face from the CelebA dataset for illustrative purposes. This is enough and now let's review another concept with a WGAN-GP. This is an improved loss function for training the GANs.

It builds on the Wasserstein GANs which we have discussed in previous sections but uses that gradient penalty which allows us to stabilize the training even further so remember from the WGANs lecture that I have implemented weight clipping to stabilize the model training in the paper of progressive growing of GANs. Another technique was used which is called a gradient penalty. It penalises the norm of the gradient of the weights of the discriminator with respect to the inputs.

I encourage you to read more about this technique using the reference displayed on the screen right now. So let's recap what we have learned in this lecture and also highlight some of the points that we have skipped. First we have used transpose convolution in contradiction to using convolution or block with average pooling or average unpooling, max pooling and max unpooling. This concept we have discussed in a previous section. However the authors of the original progressive growing of GANs paper have used convolutional block without transpose convolution.

This is very unclear why they did that. Maybe they wanted the transitions between different resolutions to be done using static upsampling and downsampling techniques. Otherwise the transposed convolution will learn the weights for a certain resolution four by four for example and then when it shifts to eight by eight it has to unlearn these weights again. So this is quite an important point to be aware of when implementing growing of GANs.

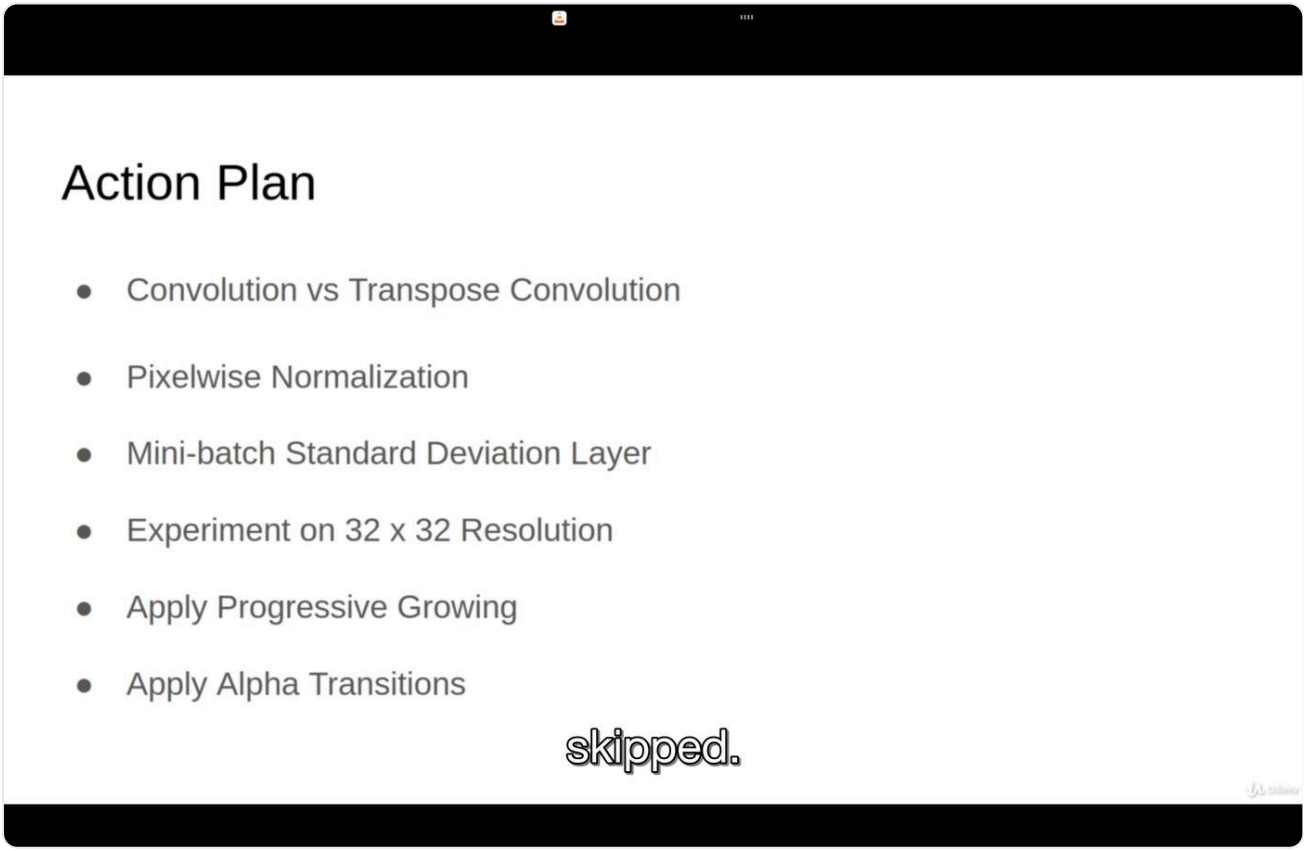


FIGURE 18.4 | Convolution vs Transpose Convolution

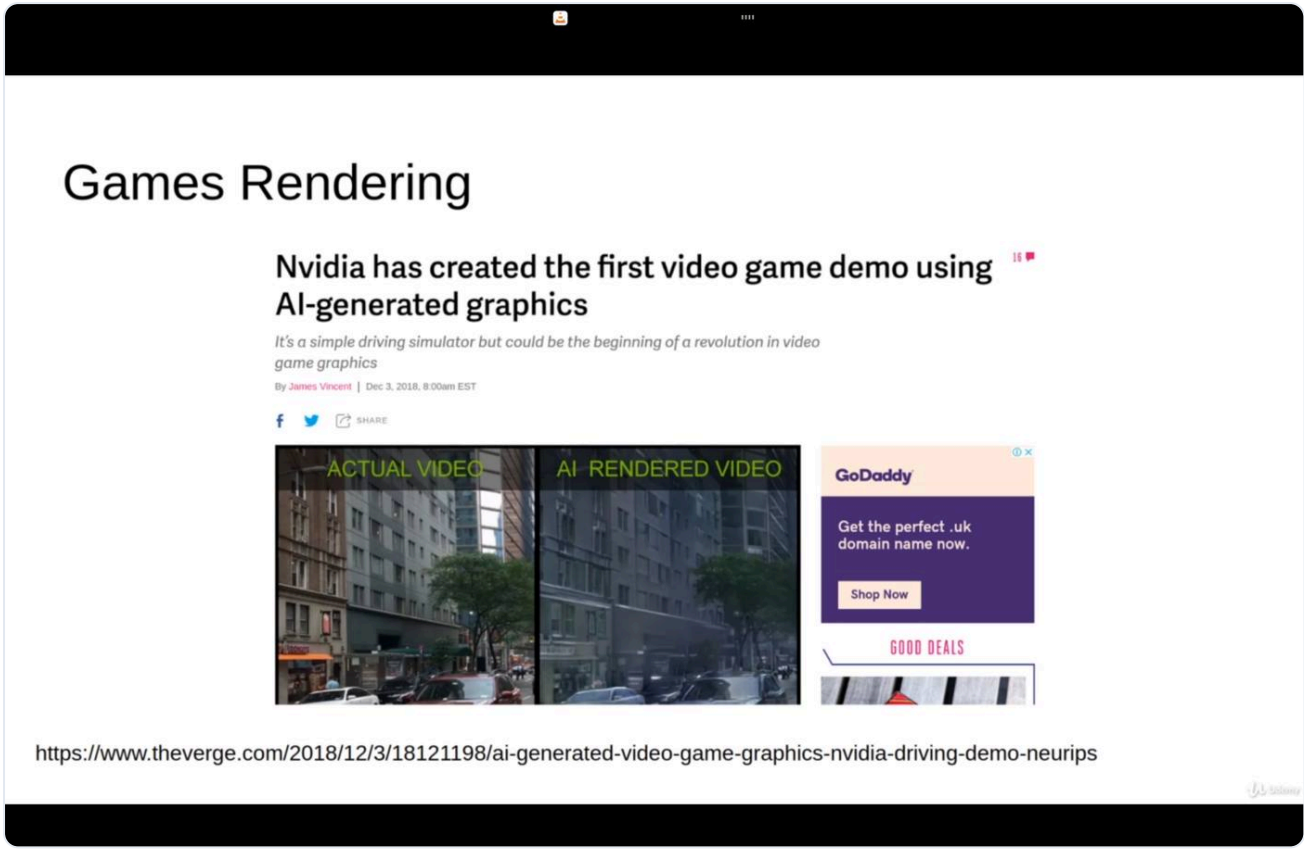


FIGURE 18.5 | Games Rendering

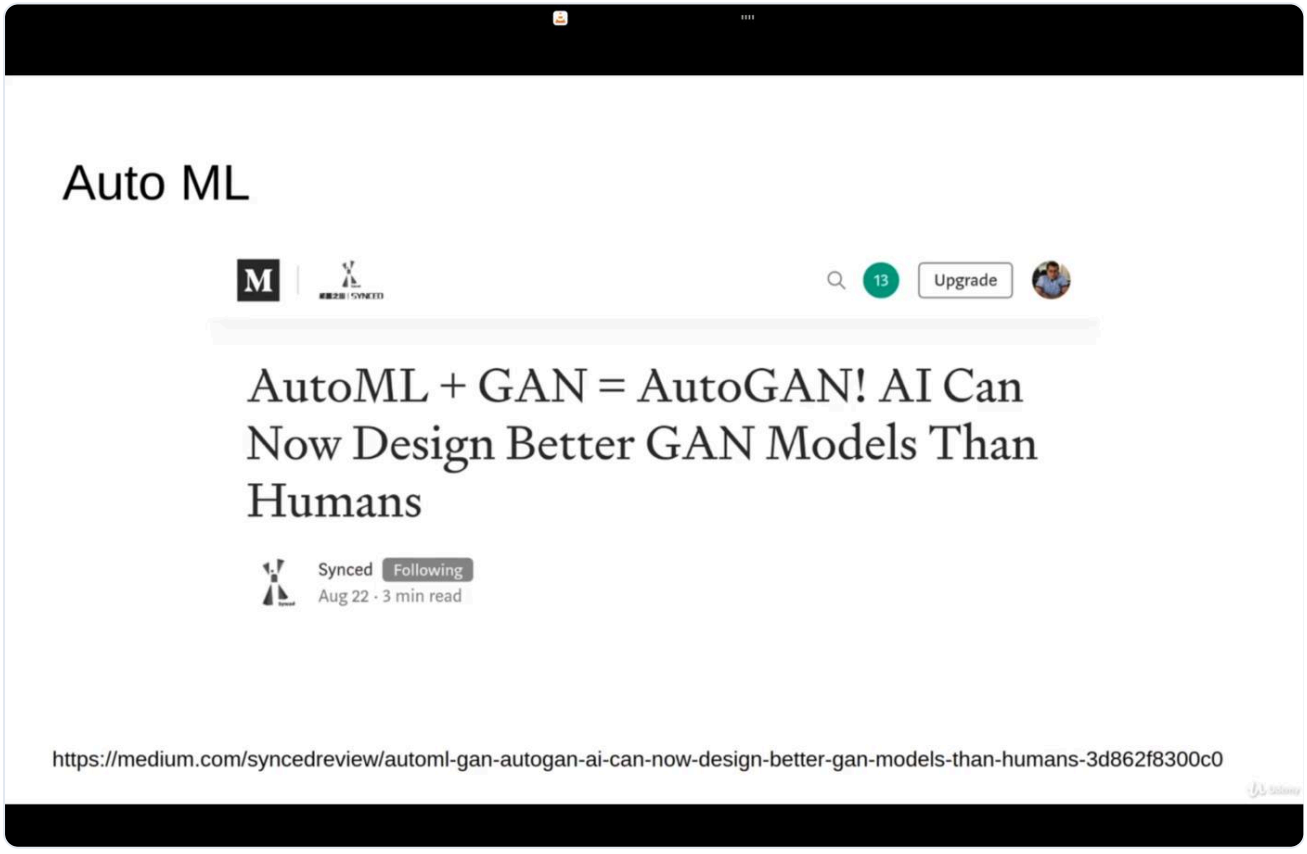


FIGURE 18.6

It is done using convolutional block but I have done it with transpose convolutions to see how different the results would be then we implemented the pixel wise normalization and we have implemented the mini batch standard deviation layer however we implemented on 32 by 32 resolution and apply the progressive growing of the network and finally applied Alpha transitions. So now that we are done with the progressive growing of GANs I would like to highlight some of the amazing applications of GANs in game scene rendering.

As you know games rely on powerful graphic cards to render textures lighting motion and camera. It is very challenging to maintain temporal and spatial relations in rendering videos but which is good news for you. In this article it announced how NVIDIA is tackling the problem using GANs so the future of videogames will change based on the techniques you have learned in this course of GANs. Another very promising area of research is AutoML where the computer decides the optimal model design the optimal hyperparameters and it can lead to amazing results when applied to GANs. So in the next lecture I will share with you more tips, tricks and hints for training GANs.

Thank you and see you in the next lecture.

LECTURE 22

Hints, Tips, and Tricks for GAN Training

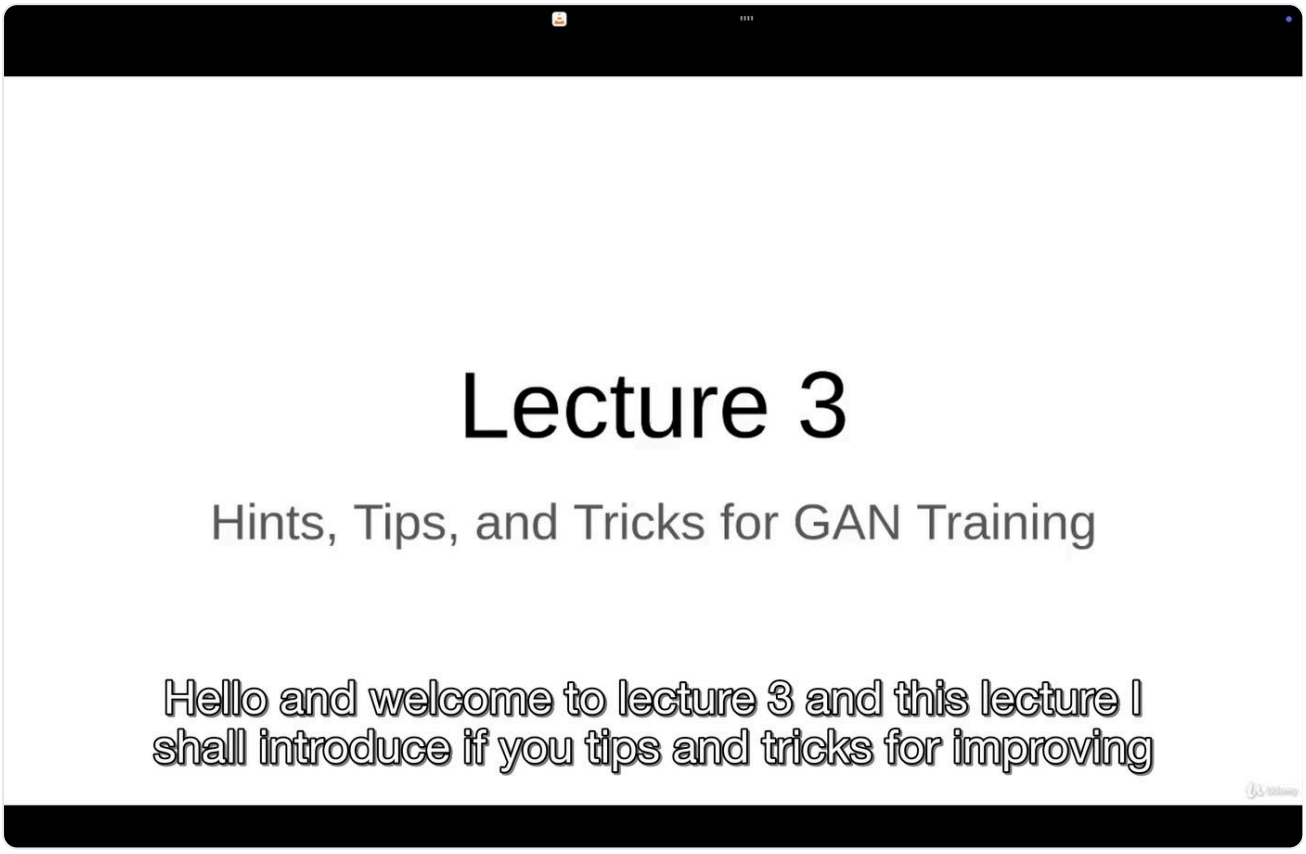


FIGURE 19.1 | Hints, Tips, and Tricks for GAN Training

Hello and welcome to lecture 3 and this lecture I shall introduce a few tips and tricks for improving GANs performance the first and the most obvious hyper parameter to fine tune is the learning rate.

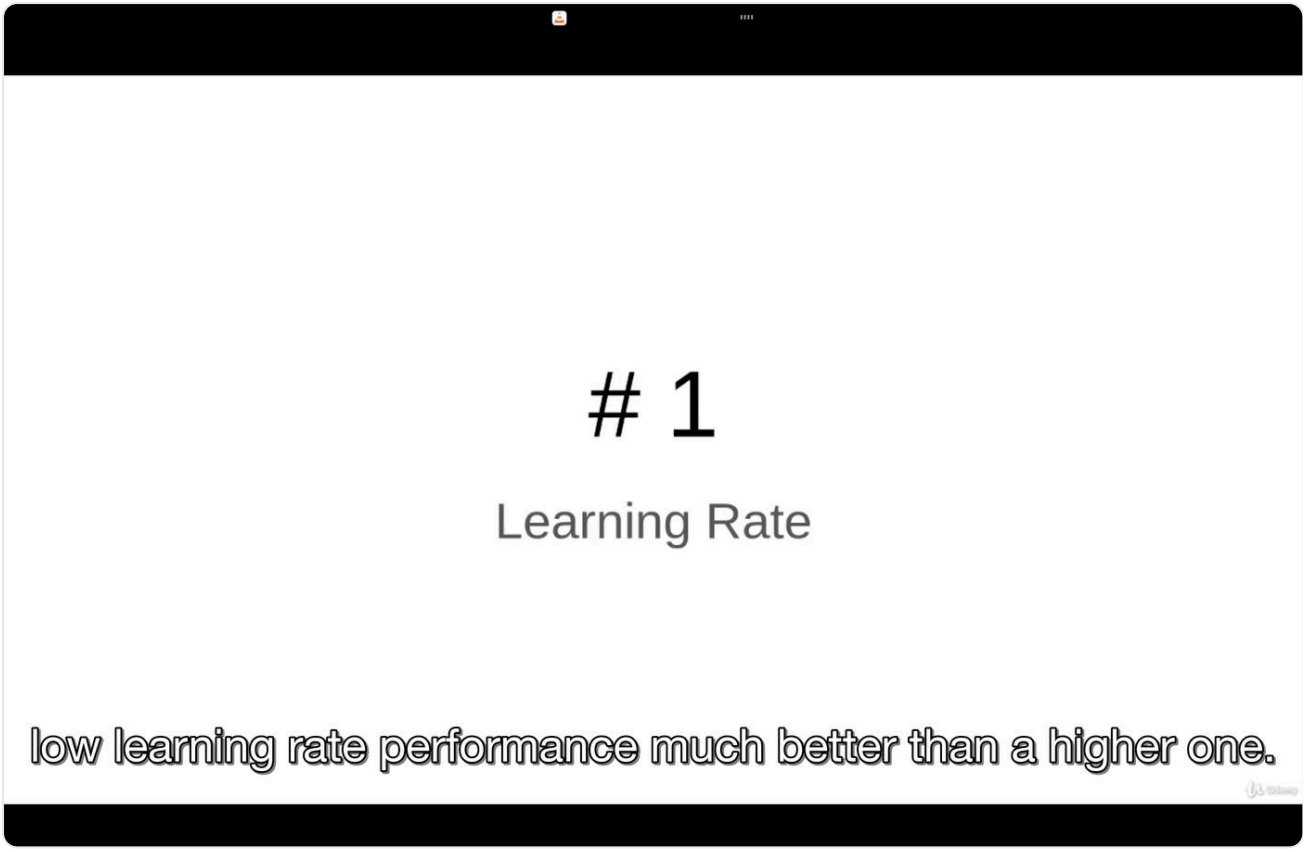


FIGURE 19.2 | Learning Rate

A low learning rate performs much better than a higher one. Even a decaying learning rate is helpful.

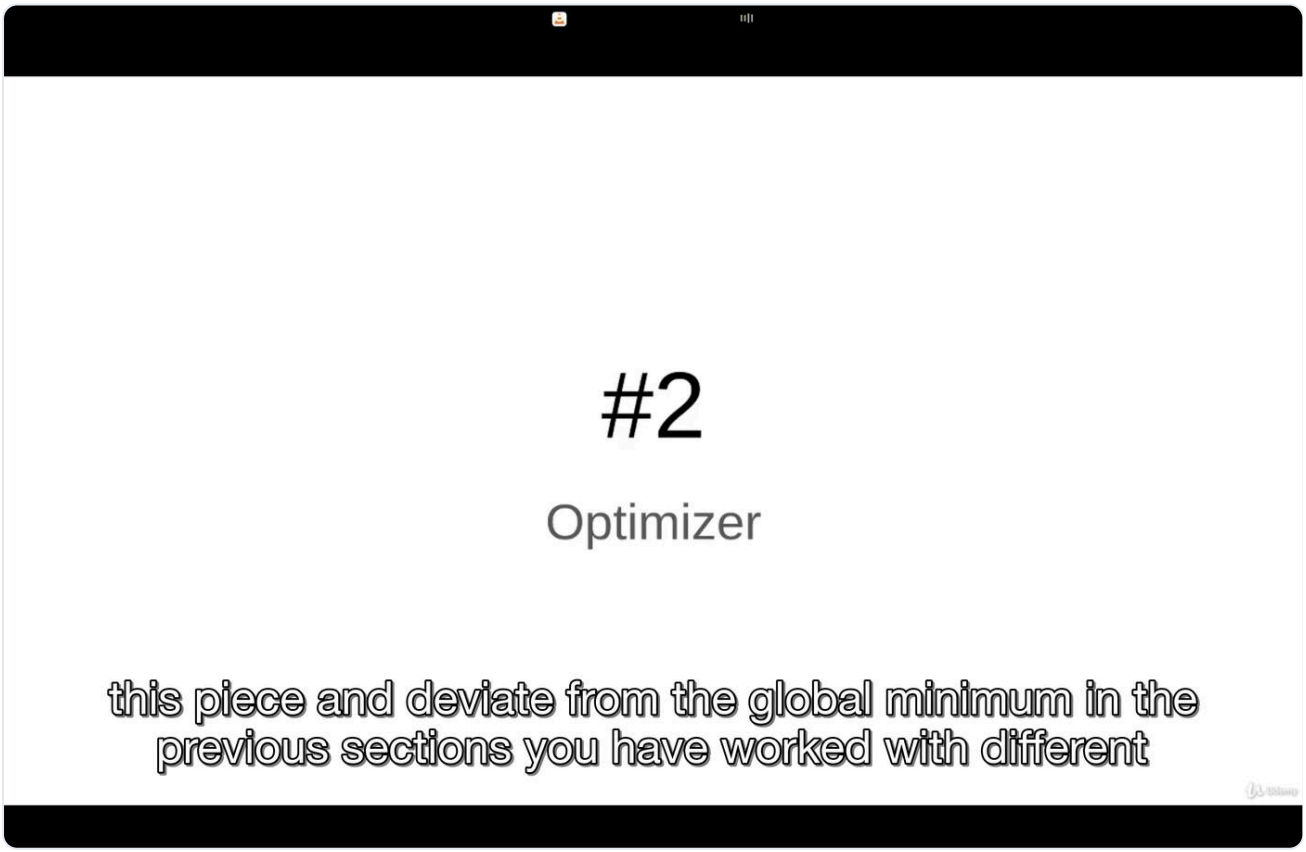


FIGURE 19.3

Most of the time setting a high value for learning rate can cause your model to jump in the parameter space and deviate from the global minimum in the previous sections you have worked with different optimizer algorithms.

I would always try stochastic gradient descent or SGD to begin with and follow a different path. If the results are not okay I would also begin with the hyperparameters recommended by a paper. Then I use the paper settings as a baseline and form my own configurations from there.

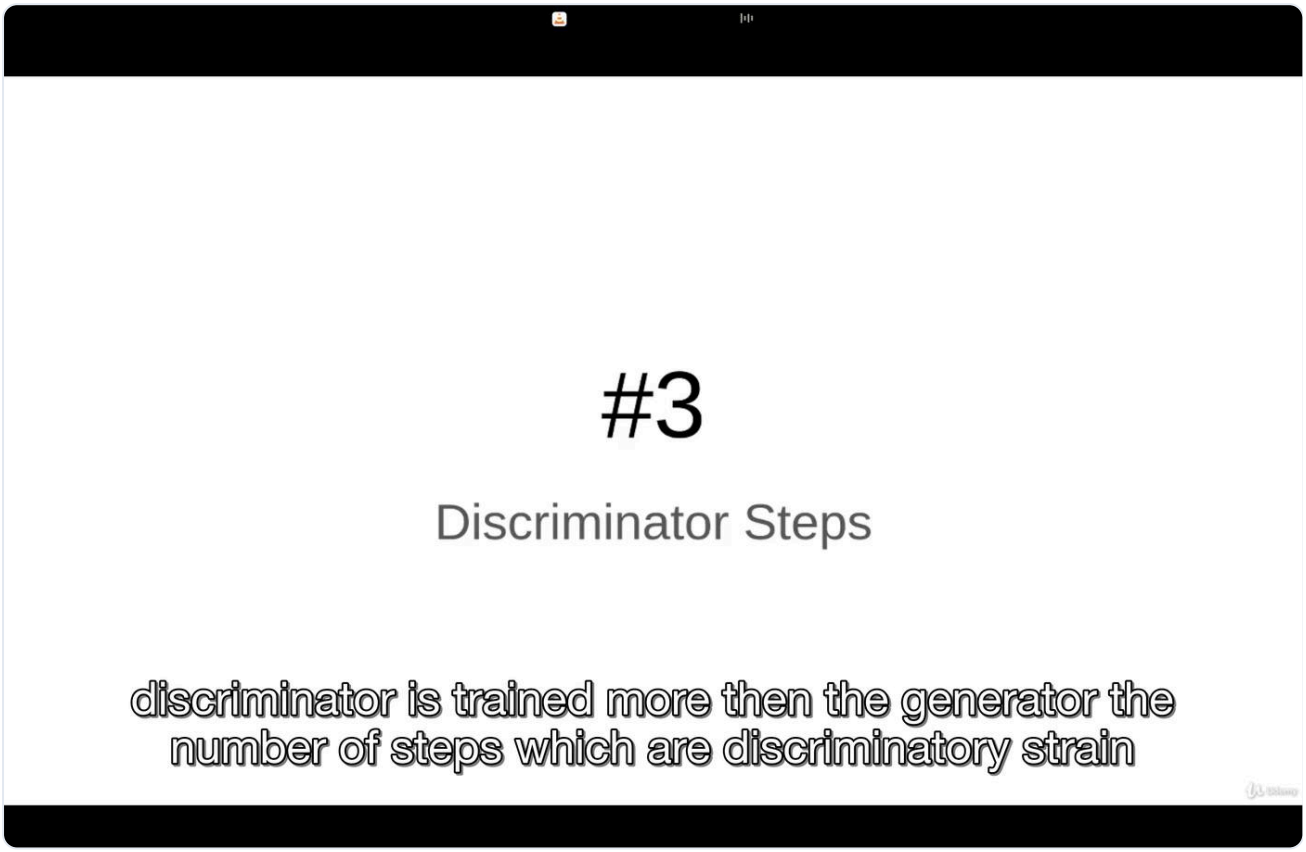


FIGURE 19.4 | Discriminator Steps

Most of the time a slight change in the settings can result in a model divergence the number of steps discriminator is trained more then the generator the number of steps the discriminator is trained more than the generator tends to make a difference in the results and of course in the Nash equilibria of the adversarial game I would start with one step and only if the discriminator is getting fooled then I would either increase the number of training steps or add more layers to the discriminator to increase its capacity.

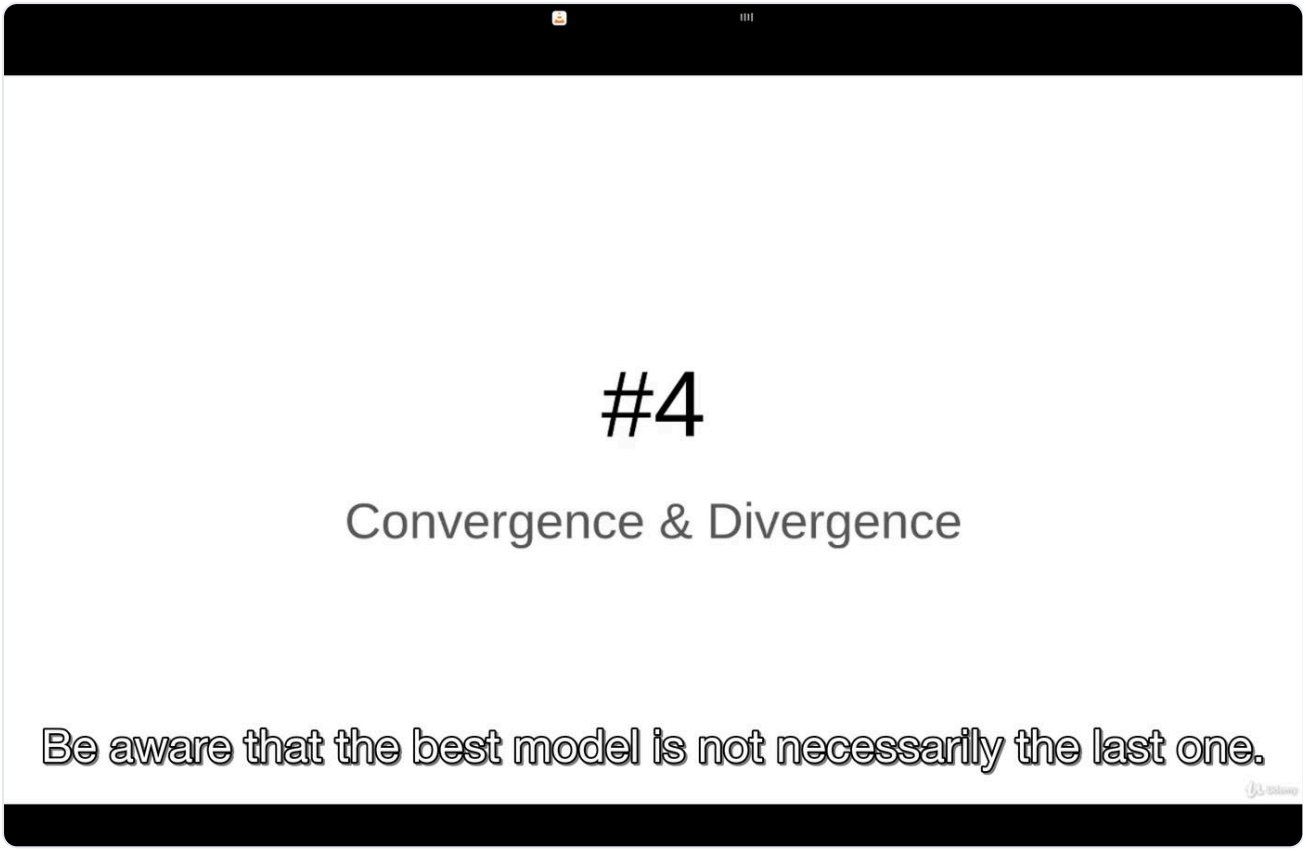


FIGURE 19.5

Be aware that the best model is not necessarily the last one. For example if you are training the network for 4K epochs it is not necessary that the best model is the one that you ended up with. That's why I use visualization and manual inspection of the results to determine the best subjective quality. It is always a good idea to use the checkpoints to save your work and intermediate models as well.

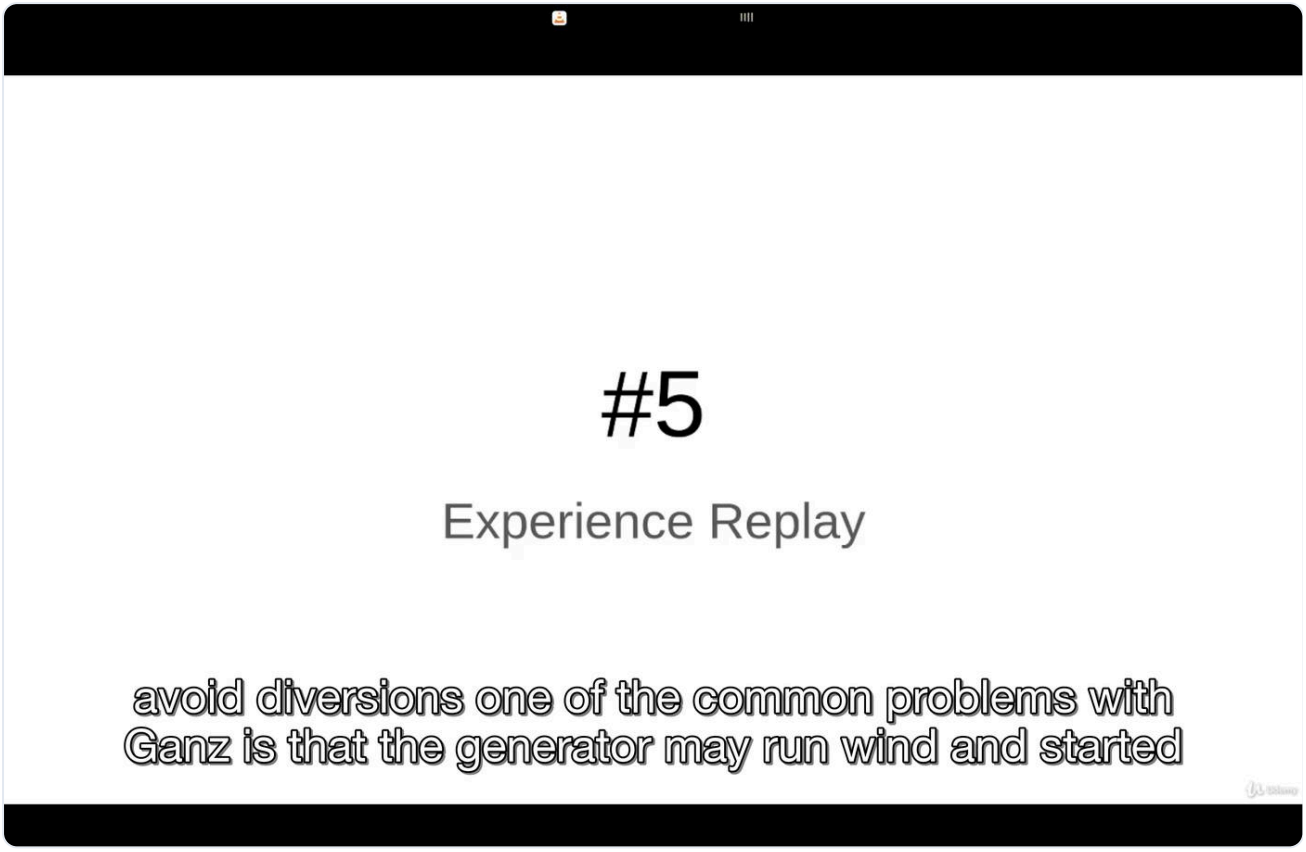


FIGURE 19.6

You can always get back to this model and change its parameters and retain it from the checkpoint to avoid divergence one of the common problems with GANs is that the generator may rewind and start generating bad samples that were rejected at earlier epochs.

As a result the training just springs back and forth between the same generated samples without real progress. One of the techniques used by GAN practitioners is to save a few random samples from earlier epochs then feed these samples to the discriminator at later epochs.

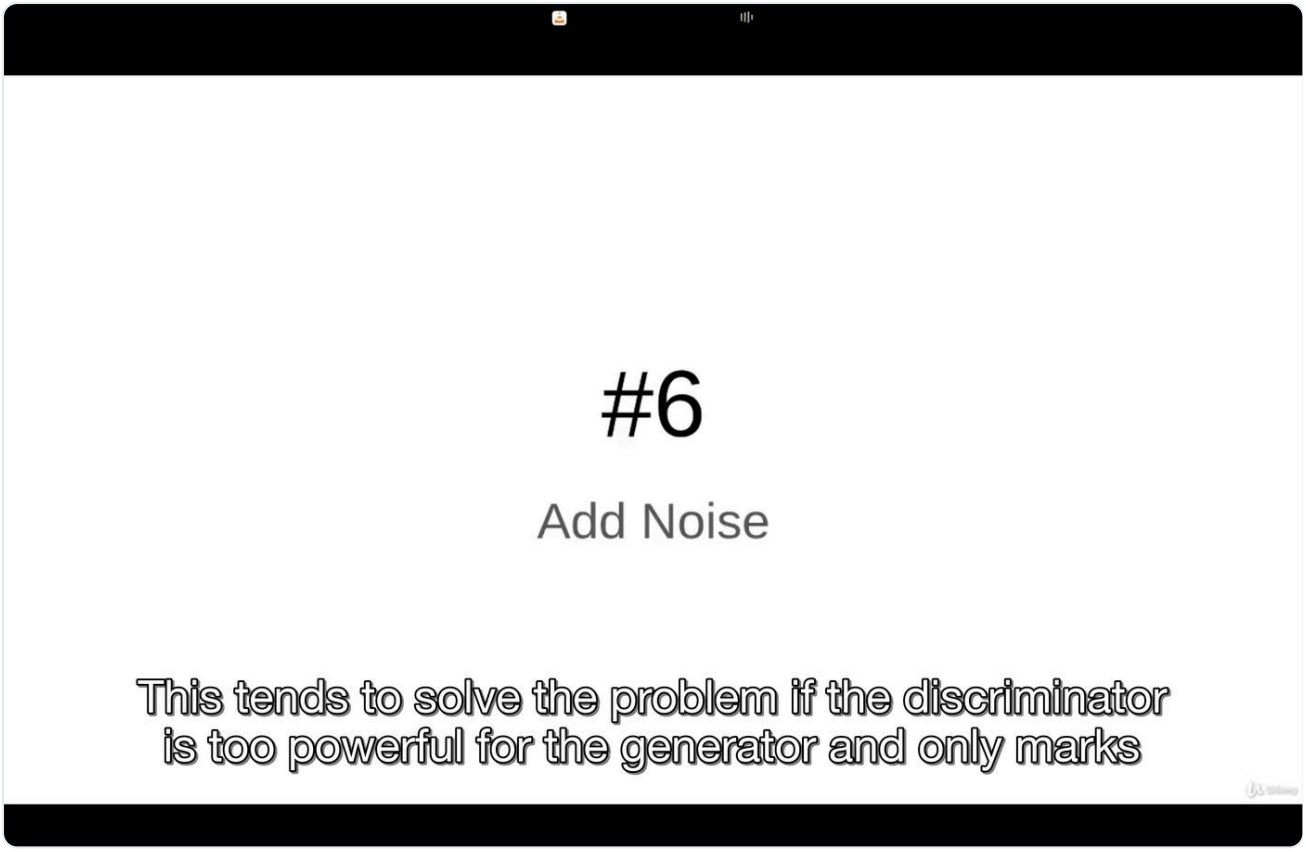


FIGURE 19.7 | Add Noise

This tends to solve the problem if the discriminator is too powerful for the generator and only marks its output as fake one of the ways to make the job a bit more challenging for the discriminator is to add a Gaussian random noise to the real images to confuse the discriminator.

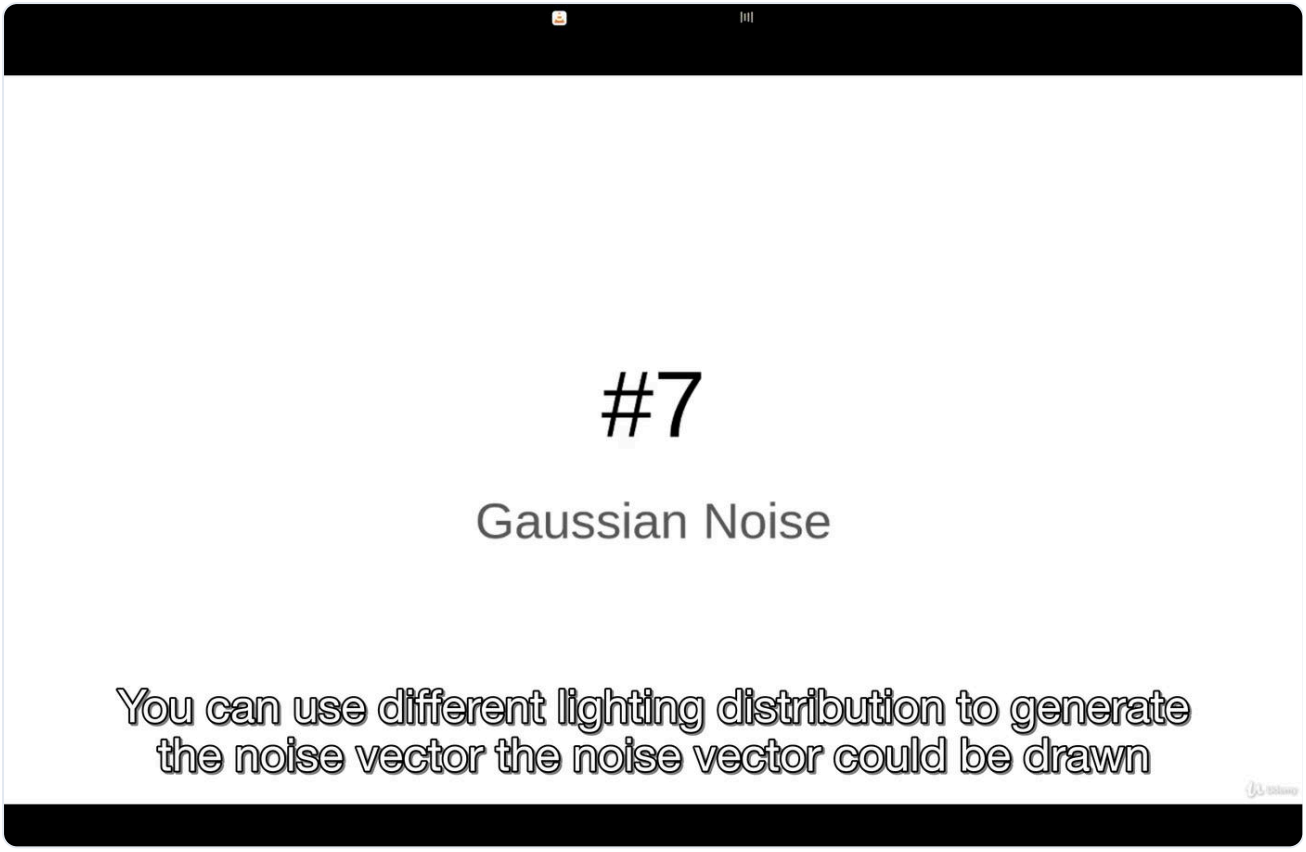


FIGURE 19.8 | Gaussian Noise

You can use a different latent distribution to generate the noise vector the noise vector could be drawn from any distribution and not necessarily a gaussian normal. It could be a uniform distribution for example. You could also try a different mean and variance for Gaussian normal just as I did in Section 3.

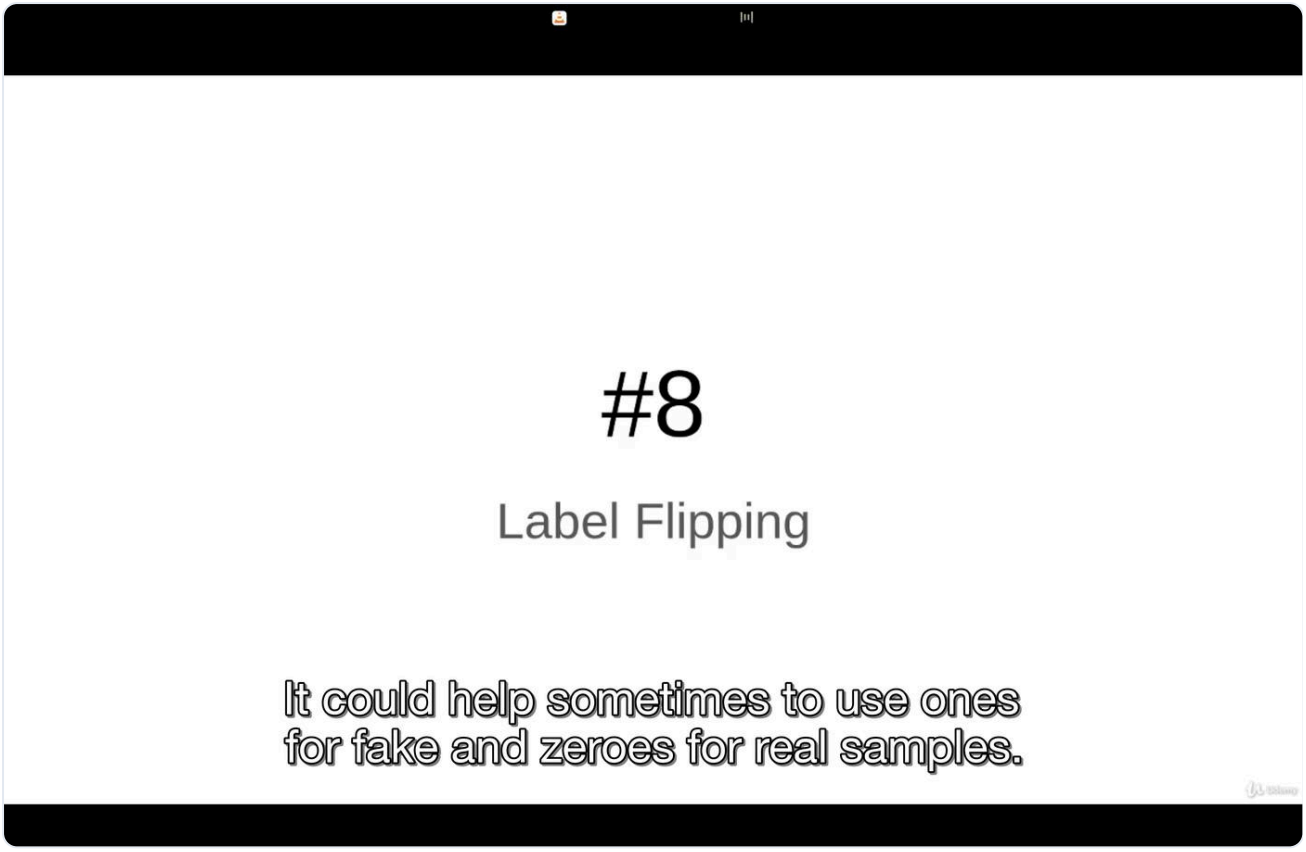


FIGURE 19.9

It could help sometimes to use ones for fake and zeroes for real samples. That is we are flipping the labels and instead of one for real and zero for fake I have demonstrated the gradient saturation problem in an earlier section.

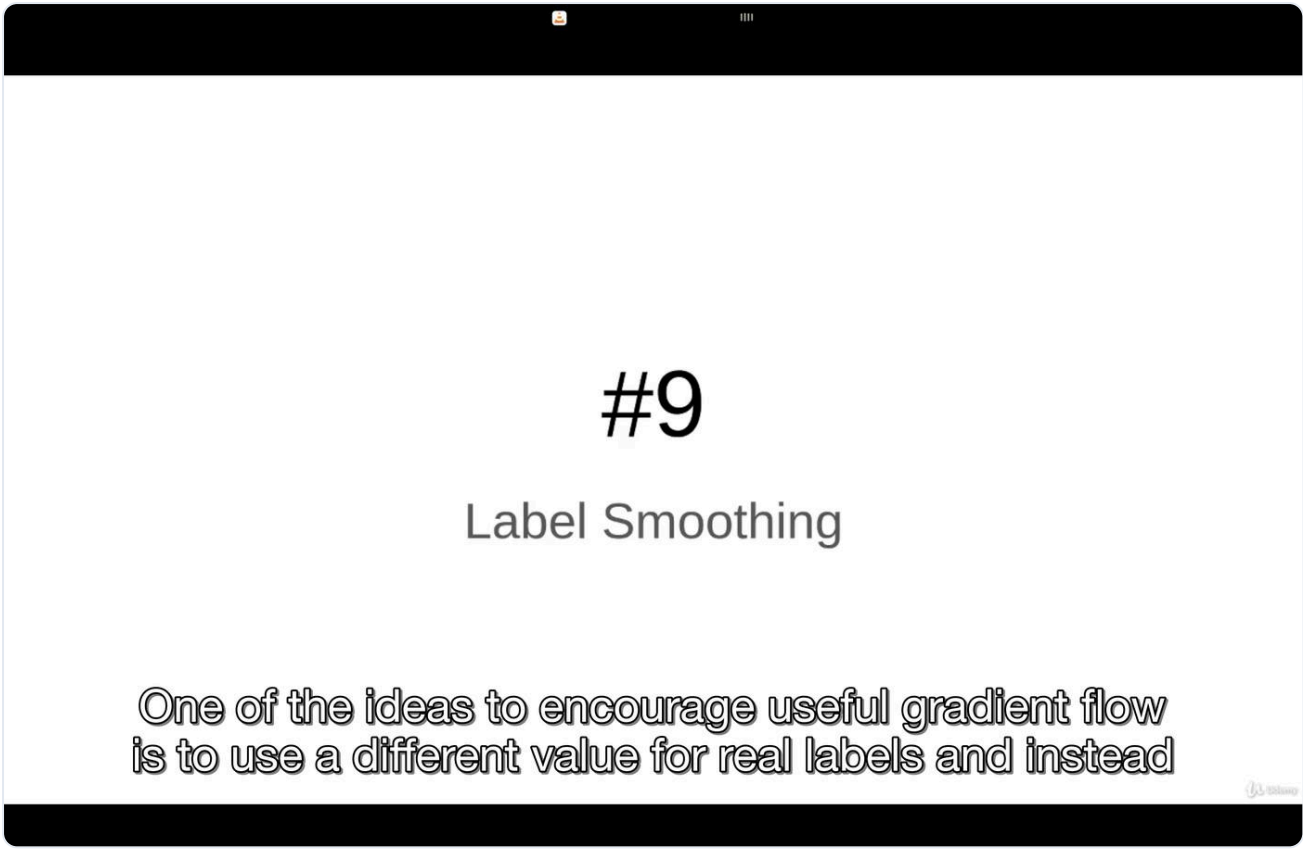


FIGURE 19.10

One of the ideas to encourage useful gradient flow is to use a different value for real labels and instead of using ones for real labels you could use point ninety nine. That's an example.

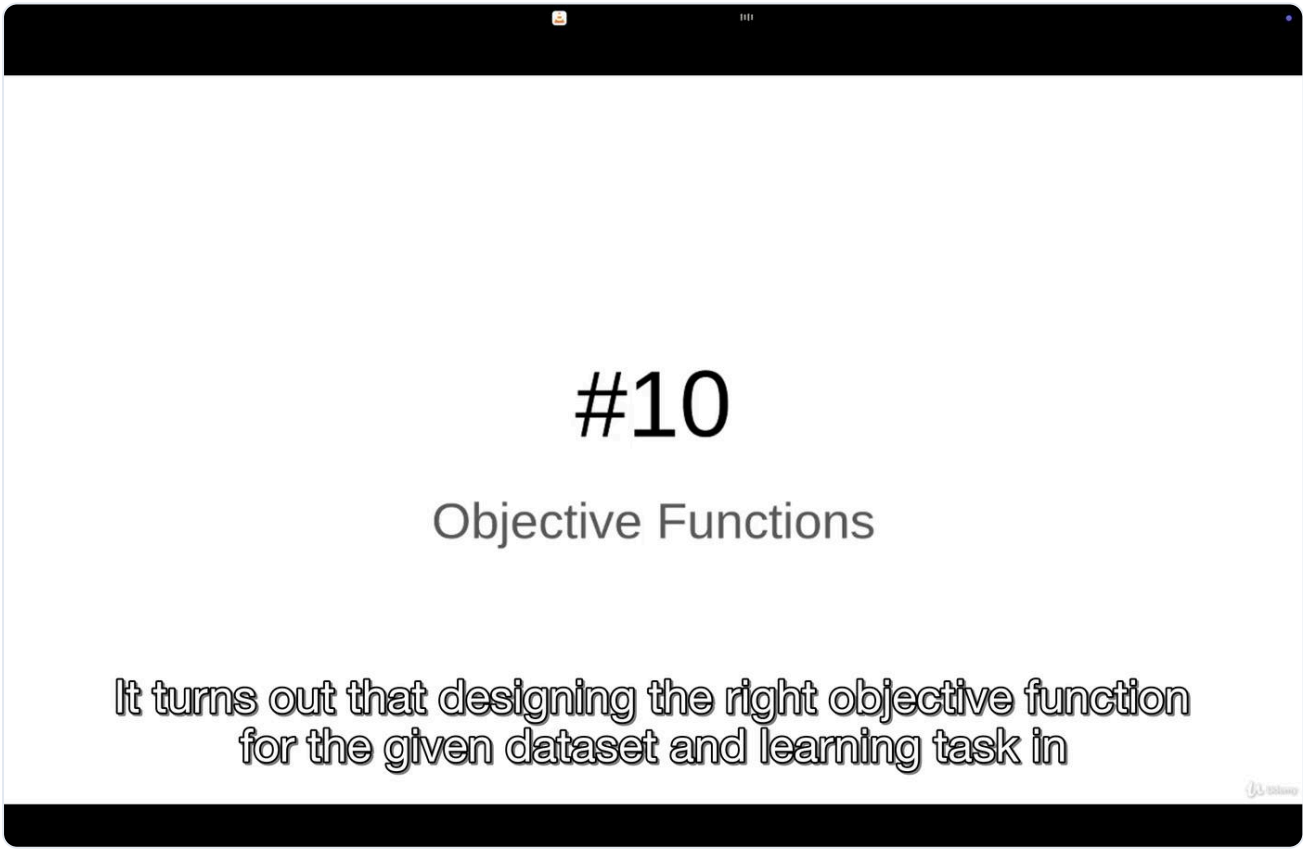


FIGURE 19.11 | Objective Functions

It turns out that designing the right objective function for the given dataset and learning task in one of the essential if not the most important steps a lot of innovation goes into designing a meaningful objective function speaking of objective functions.

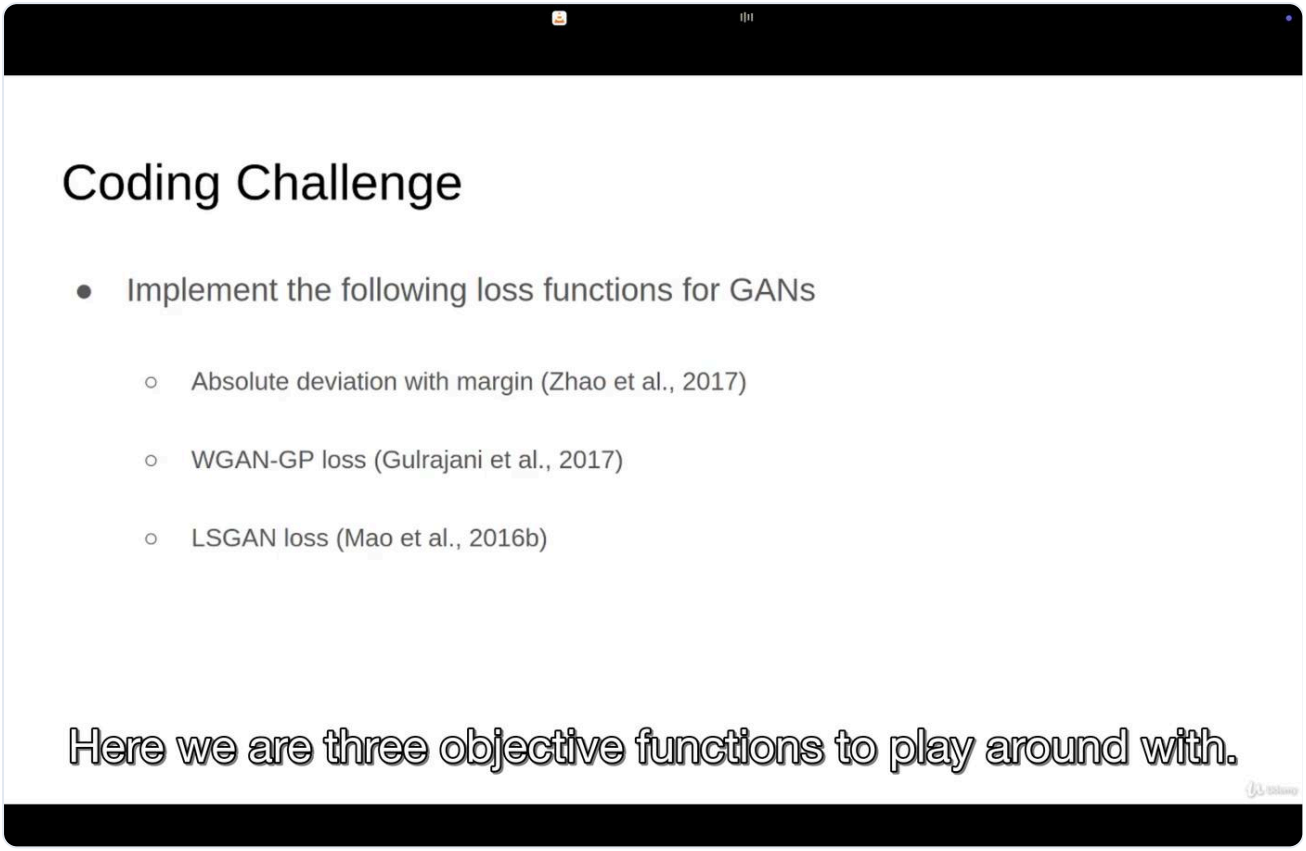


FIGURE 19.12

Here are three objective functions to play around with.

Remember that the more you read and implement the better you get.

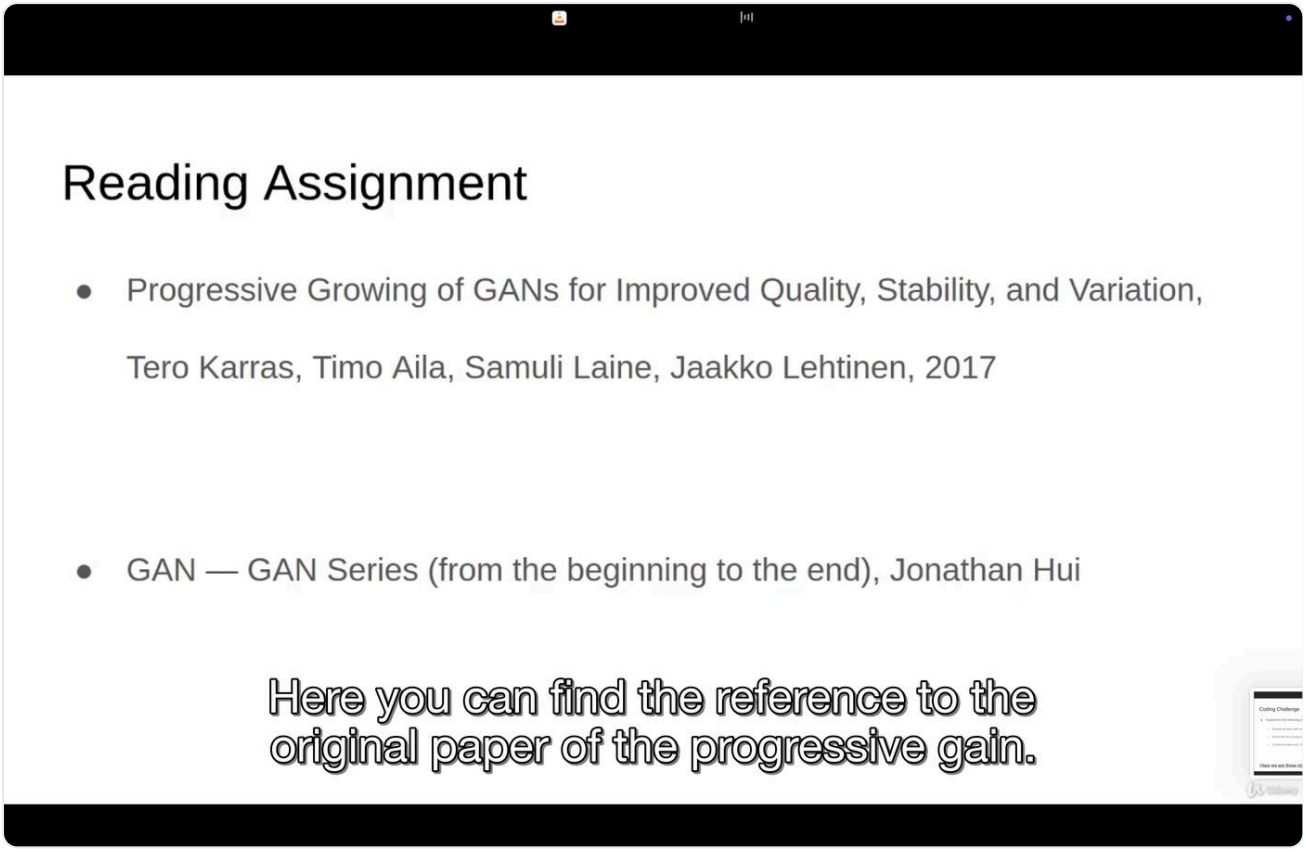


FIGURE 19.13

Here you can find the reference to the original paper of the progressive GAN. The second item is an online blog on medium by Jonathan who that I consider one of the most comprehensive blogs on GANs congratulations on completing Section 5. Thank you and see you in the next section.

LECTURE 23

Introduction to U-NET Architecture

Hello and welcome to the last section of our course.

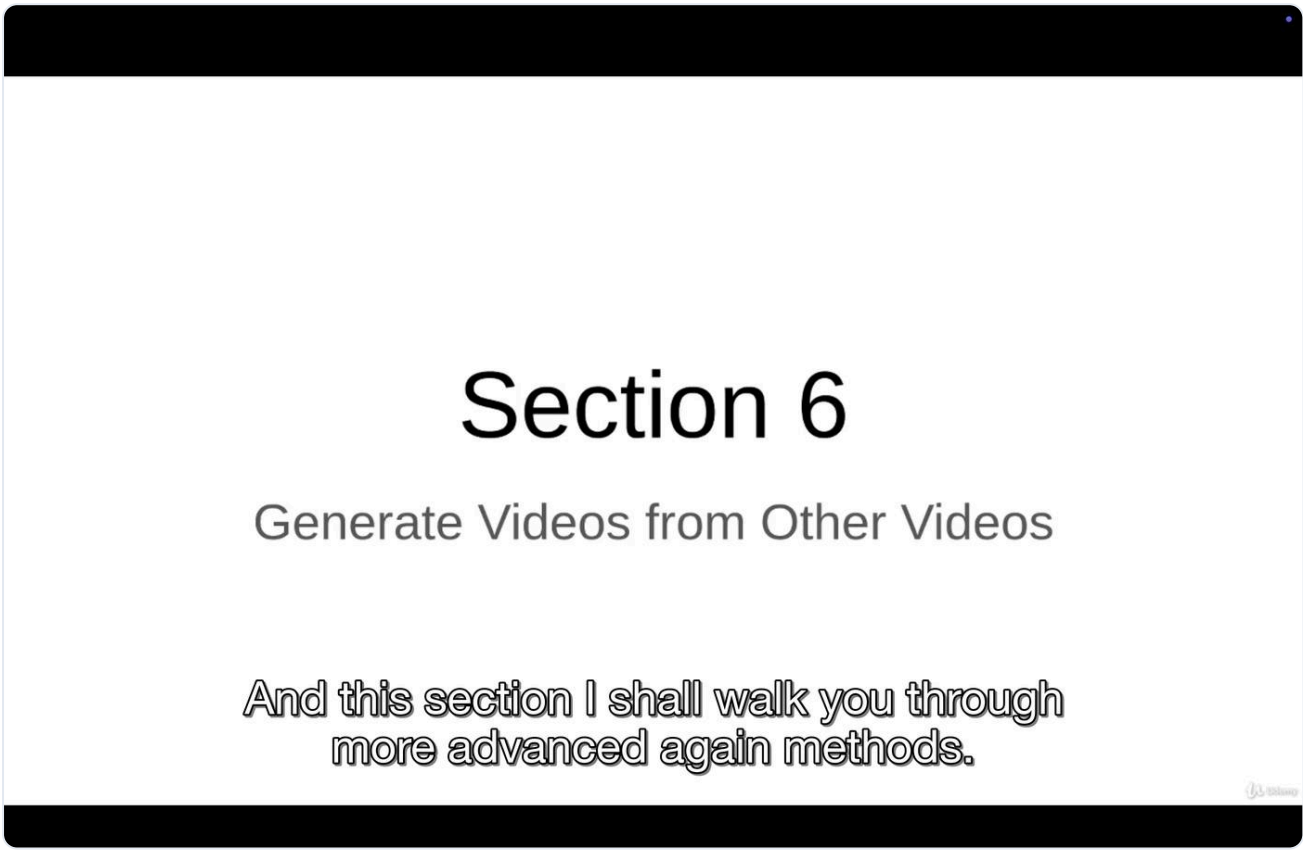


FIGURE 20.1

And this section I shall walk you through more advanced GAN methods.

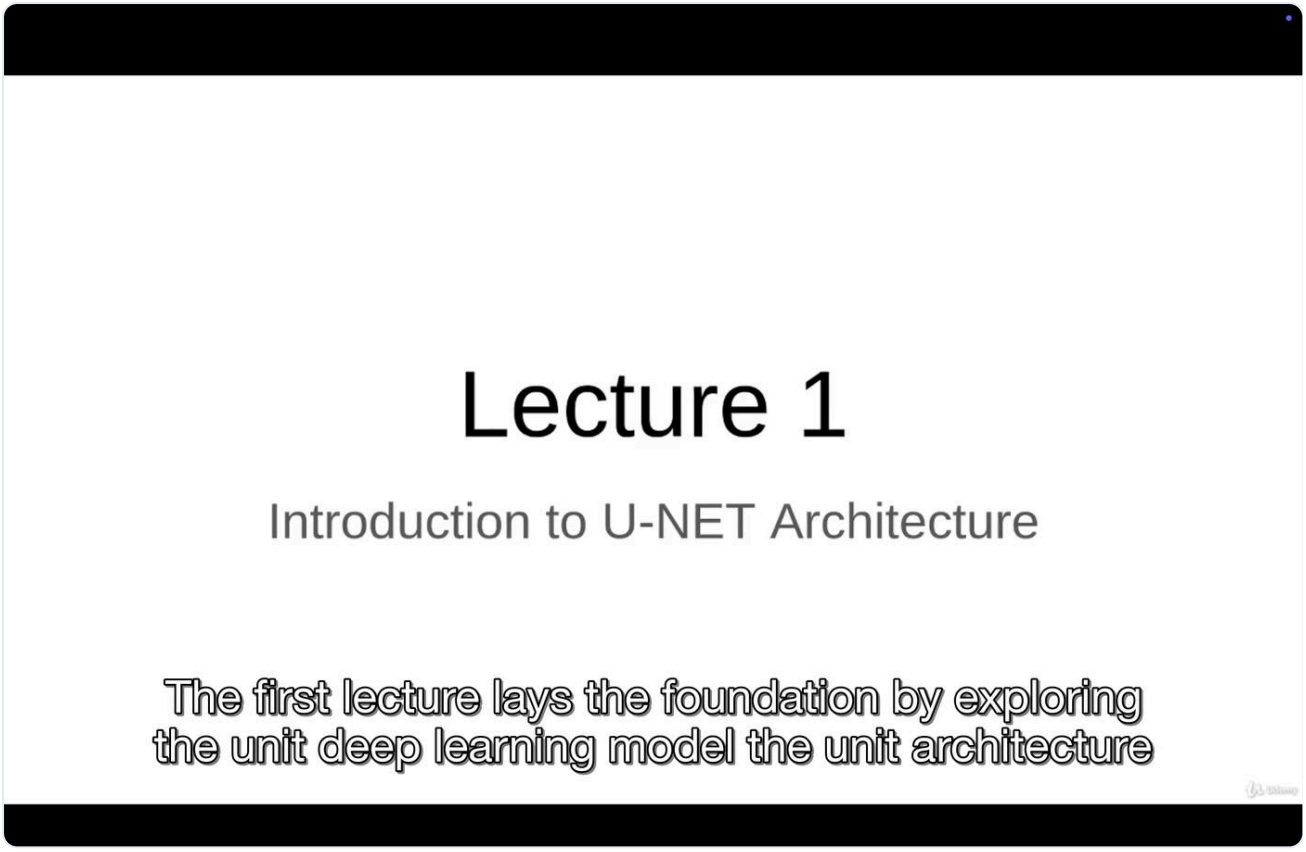


FIGURE 20.2 | Introduction to U-NET Architecture

The first lecture lays the foundation by exploring the U-Net deep learning model the U-Net architecture is designed to leverage supervised learning in situations where there are few annotated training samples. This is a common case in Biomedical Imaging applications and for many small and medium businesses as well.

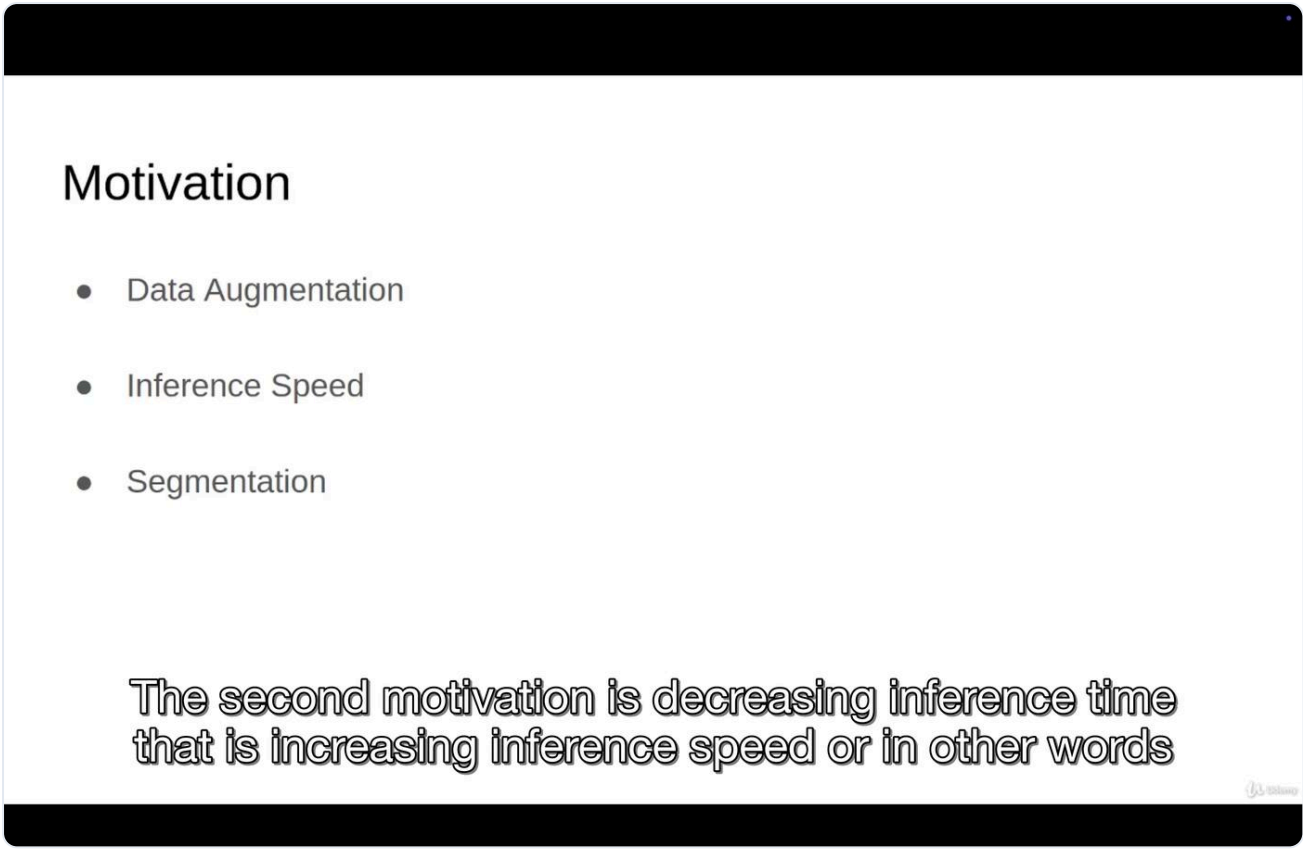


FIGURE 20.3

The second motivation is decreasing inference time that is increasing inference speed or in other words the time it takes to segment an image image segmentation is pixel wise classification.

Imagine that you have to label every pixel of an image previous attempts to use the sliding window segmentation. However this was very demanding in terms of computational complexity. There are many modern image segmentation object detection methods. However U-Net is the base for the GAN methods which we will discuss in the next lectures.

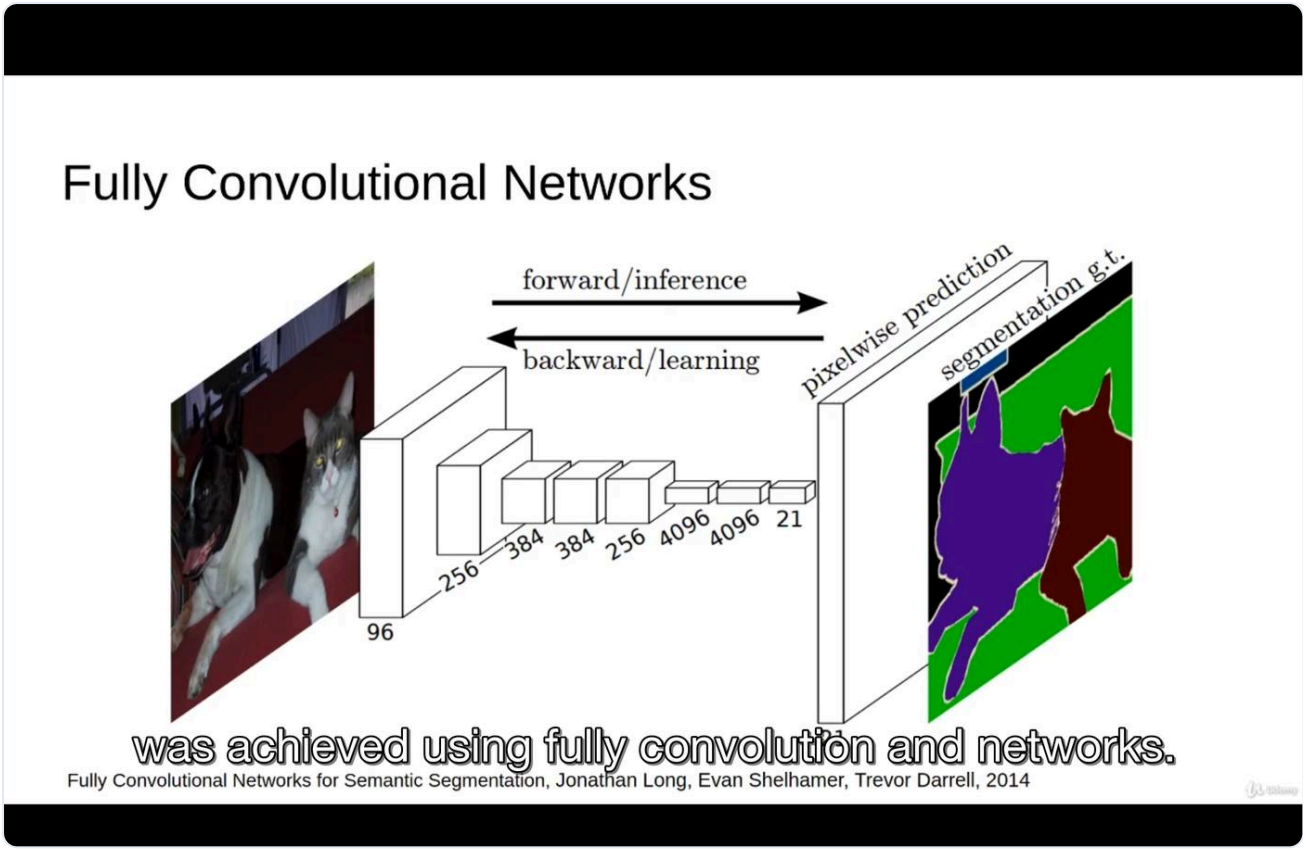


FIGURE 20.4

However U-Net is the base for the GAN methods which we will discuss in the next lectures semantic segmentation was achieved using fully convolutional networks.

Just one year before the publication of the U-Net paper. In this architecture only fully convolutional layers were used. The number of operations is reduced dramatically. This is achieved by reducing the number of filters at each layer then increasing it again towards the target resolution and then next slides.

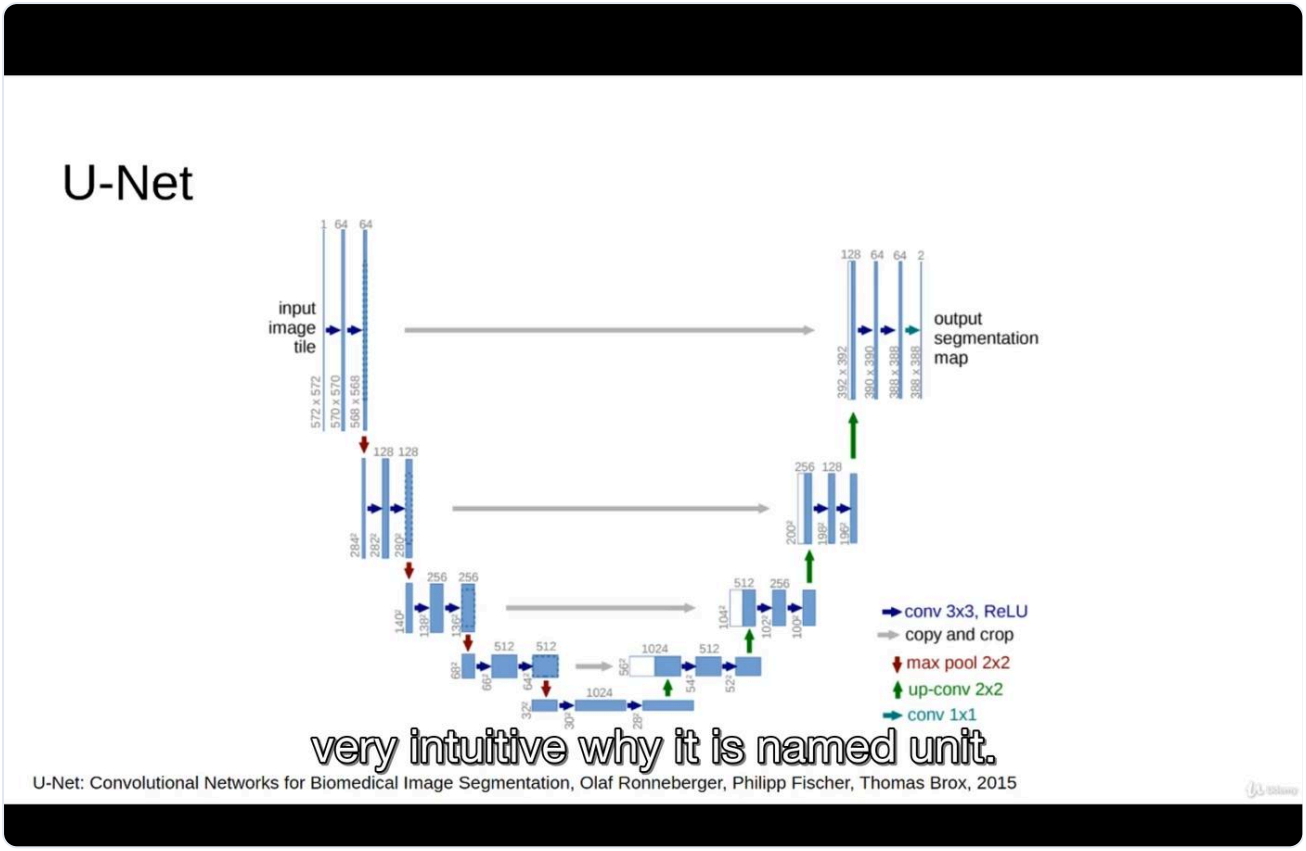


FIGURE 20.5

We shall discover the special contributions that make U-Net different from FCN this chart makes it very intuitive why it is named U-Net. The network resembles the Latin character U U-Net is an extension of FCN that is modified to be trained on fewer images and yield faster results and inference time as can be seen from the architecture diagram.

There are two parts of the network. The first part is named the contracting part. It is an intuitive naming since its job is to shrink the features towards a bottleneck. Then the second part is the expanding part.

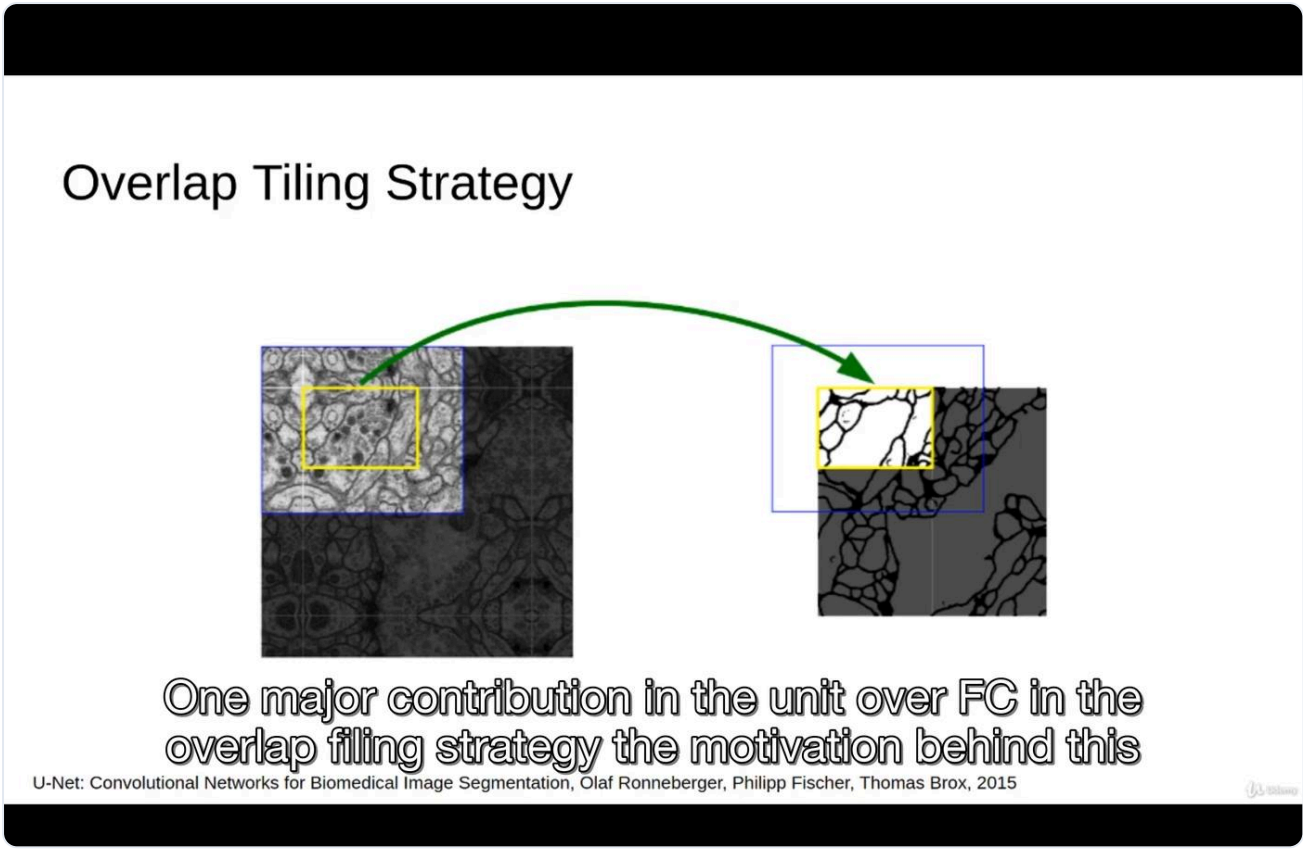


FIGURE 20.6 | Overlap Tiling Strategy

This is the part that expands the image towards a high resolution target. It is also the part that feeds contextual information to higher levels. the U-Net architecture is not symmetric. There are more filters in the expanding part than in the contracting part.

This is intentional to allow the network to learn more contextual details. There are three more distinct differences from FCN. The first is that the expanding path does not use max unpooling but it allows the network to learn its own up sampling.

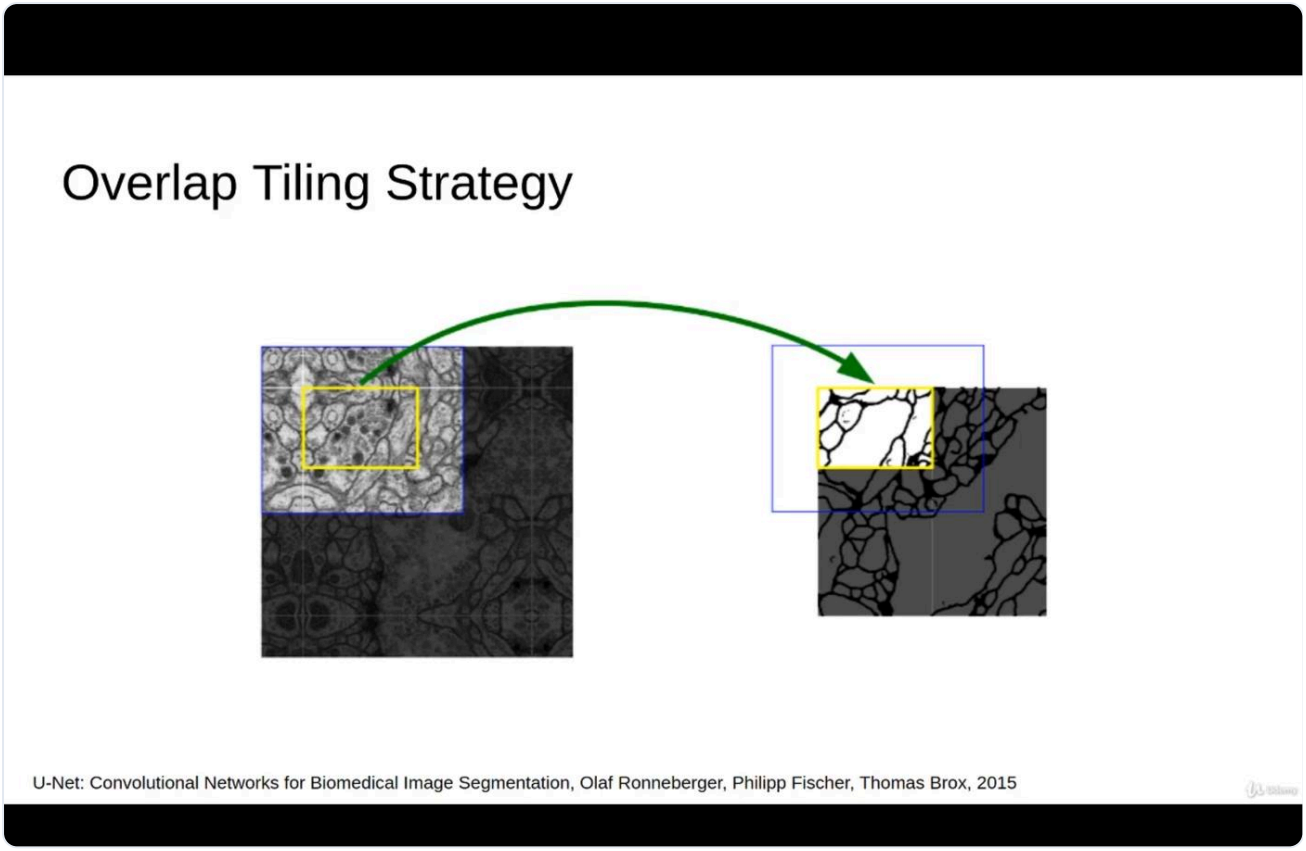


FIGURE 20.7 | Overlap Tiling Strategy

This should remind you of strided transpose convolution which I have discussed in section four the second. The difference is that the corresponding features maps from the contracting path is concatenated to the feature maps given to the convolutional block on the same level.

The second difference is that the corresponding feature maps from the contracting path are concatenated to the feature map which are given to the convolutional block on the same level. This provides more contextual details to the expanding path. It also functions as a skip connection similar to residual blocks. The third distinction is the use of cropping over the feature maps which assists the tiling strategy employed to handle border pixels. I elaborate more on the overlap tiling strategy in the next slide.

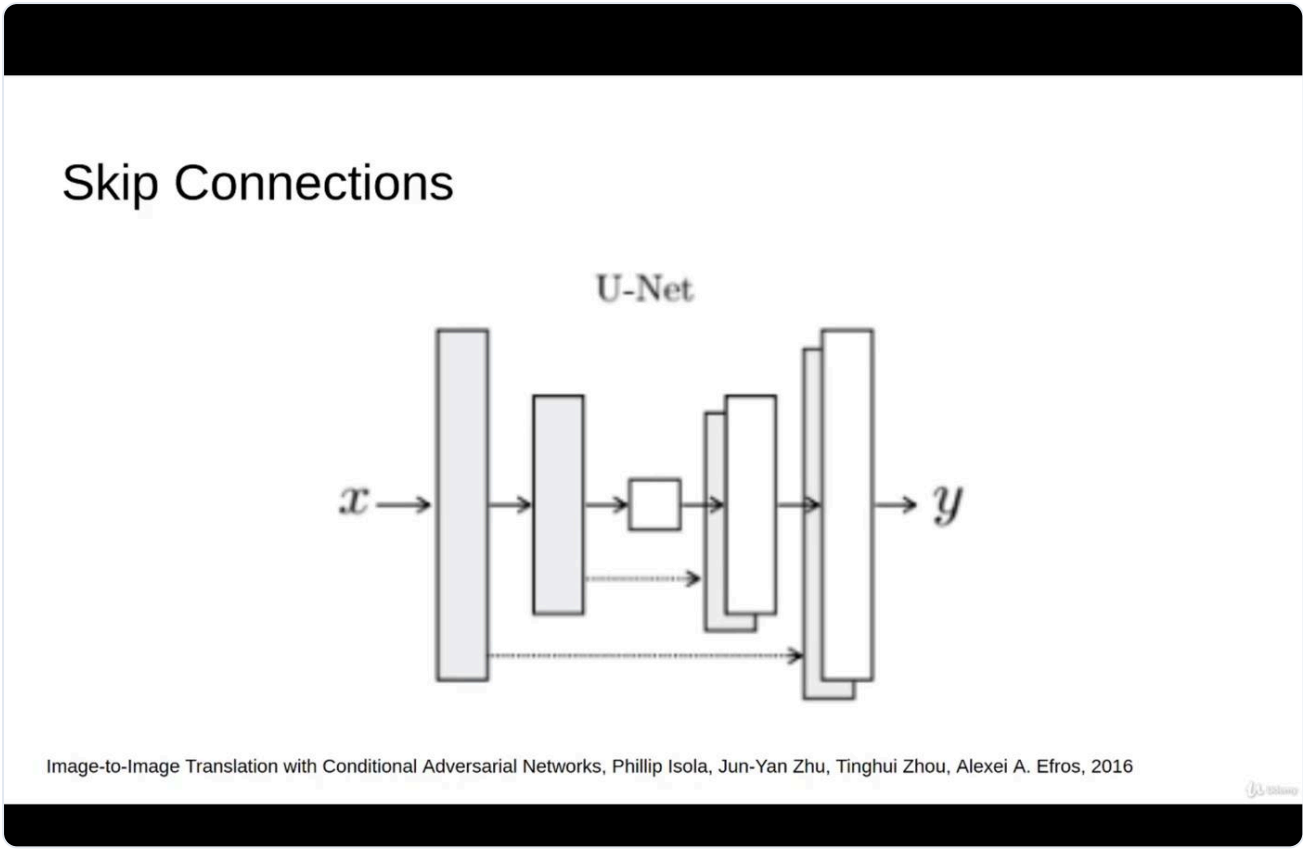


FIGURE 20.8 | Skip Connections

One major contribution in U-Net over FCN is the overlap tiling strategy the motivation behind this technique is twofold. The first reason is to increase precision on image boundaries. The borders of the image lack contextual information. This is solved by extrapolating the area around the border. Extrapolation is the opposite of interpolation in interpolation you infer a value having both previous and next values in extrapolation you infer a value having only previous values or vice versa.

The second reason why this technique is useful is that image segmentation is limited by the GPU memory by using overlapped tiling strategy arbitrary enlarged images can be fitted into GPU memory and get processed with high precision the concatenation between feature maps of layers on the same level is best described as a skip connection it is visualized here with right pointing arrows.

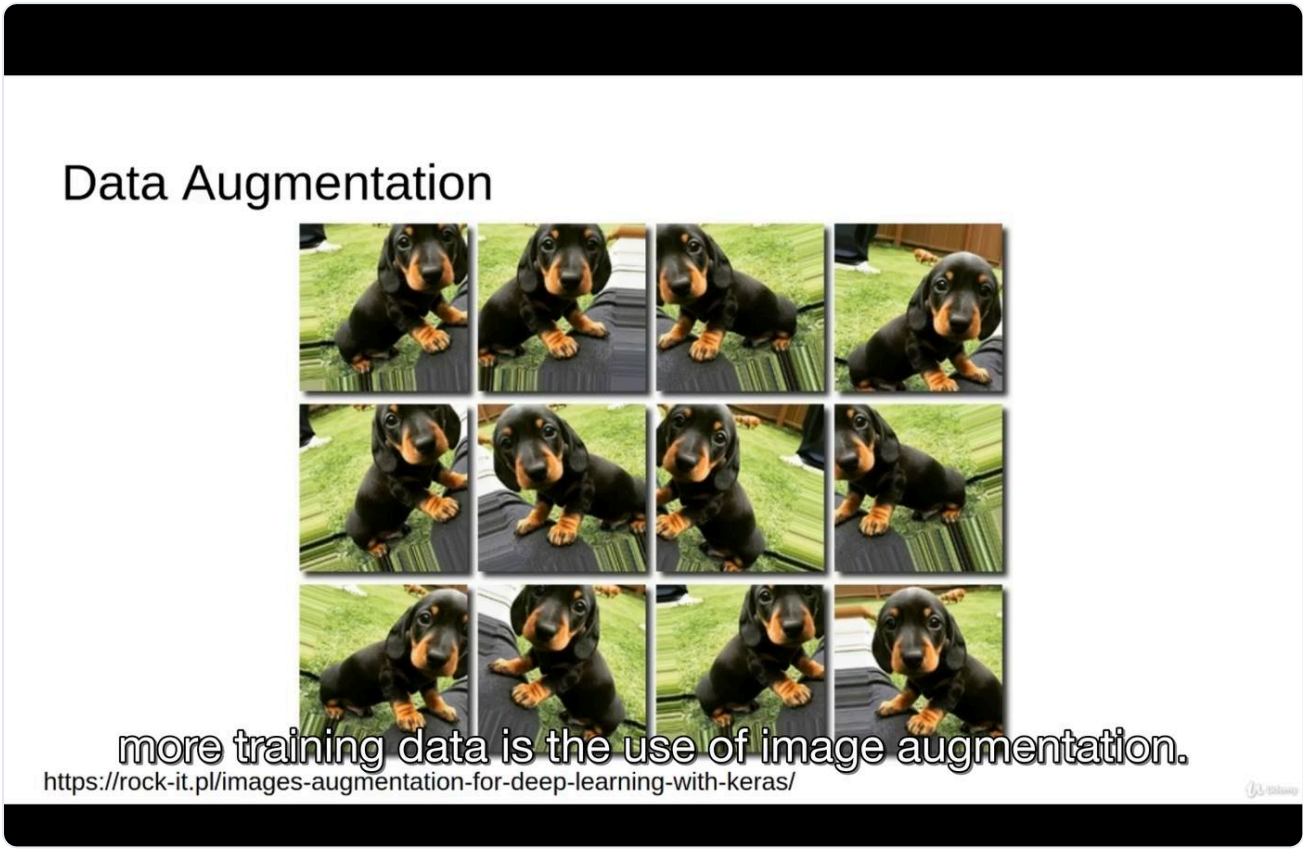


FIGURE 20.9

Skip connections are synonymous to residual blocks in a sense one of the Essential Techniques for generating more training data is the use of image augmentation.

It works by applying transformations to the original image. Example transformations are translation, flipping, rotation, skewing and normalization as a result the trained model becomes robust to scale.

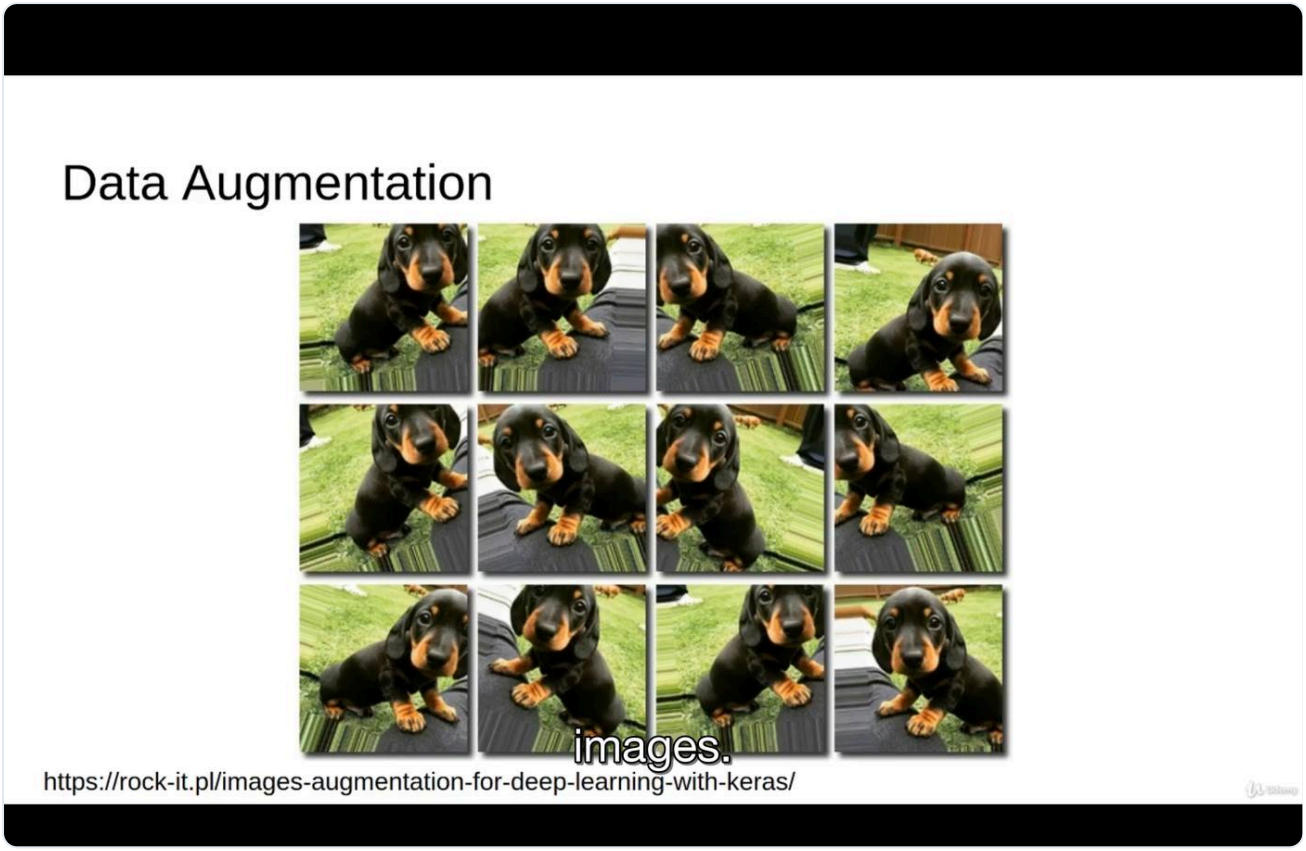


FIGURE 20.10 | Data Augmentation

Translation and orientation variations. However in image segmentation applying such transformations requires more labour to annotate the new images. This is not the case in U-Net.

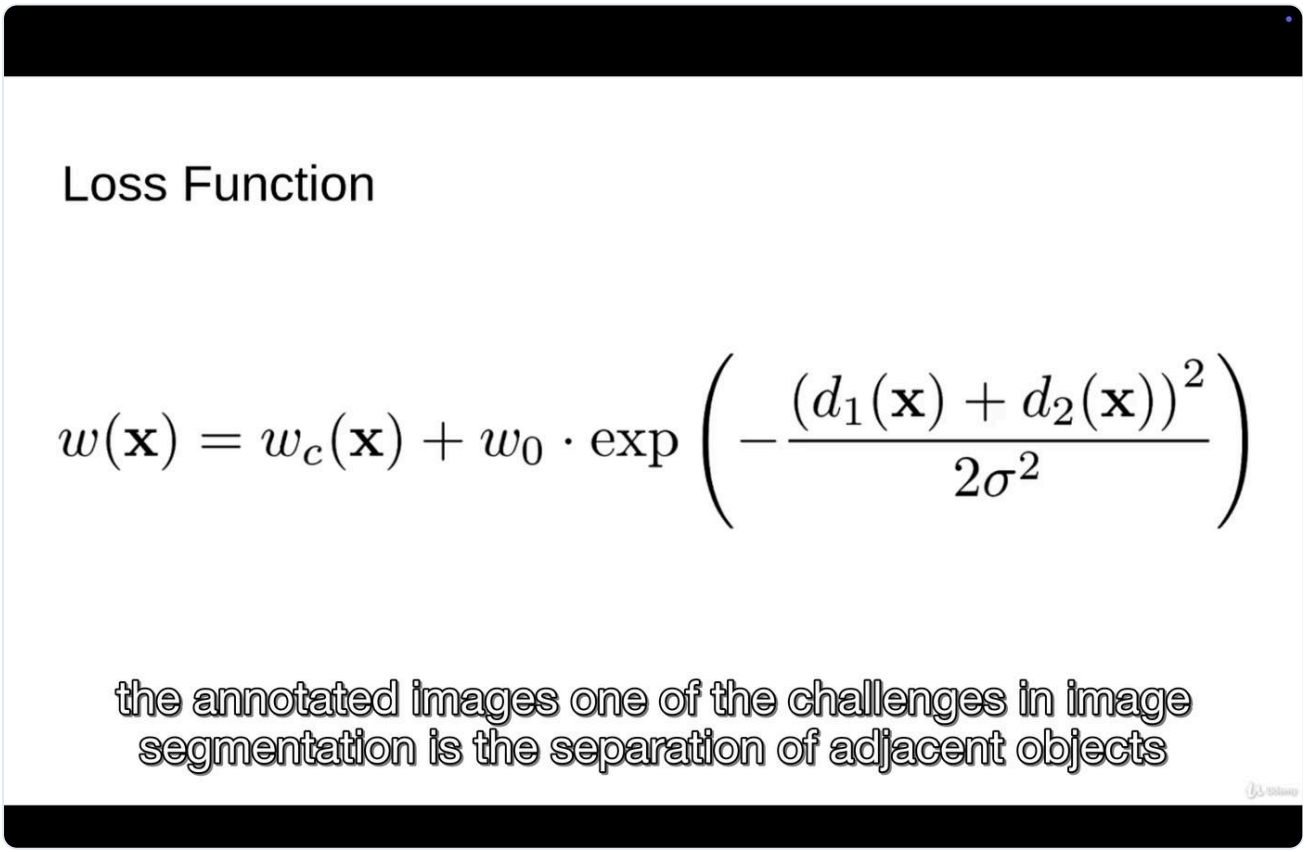


FIGURE 20.11

U-Net tends to learn the image topology without the need of changing the annotated images one of the challenges in image segmentation is the separation of adjacent objects of the same category.

That is when two objects are in physical contact and belonging to the same category. One of the concepts that I have emphasized throughout the course is the importance of loss functions. Again in this case it proves useful. The authors of U-Net have customized their own version of the segmentation loss function in order to encourage the model to separate adjacent same class objects. The equation on the screen is a weighting function for pixels that takes the distance between adjacent objects into consideration and also accounts for the pixel frequency.

The categorical cross entropy is then calculated as the weighted loss based on the weights assigned by this formula.

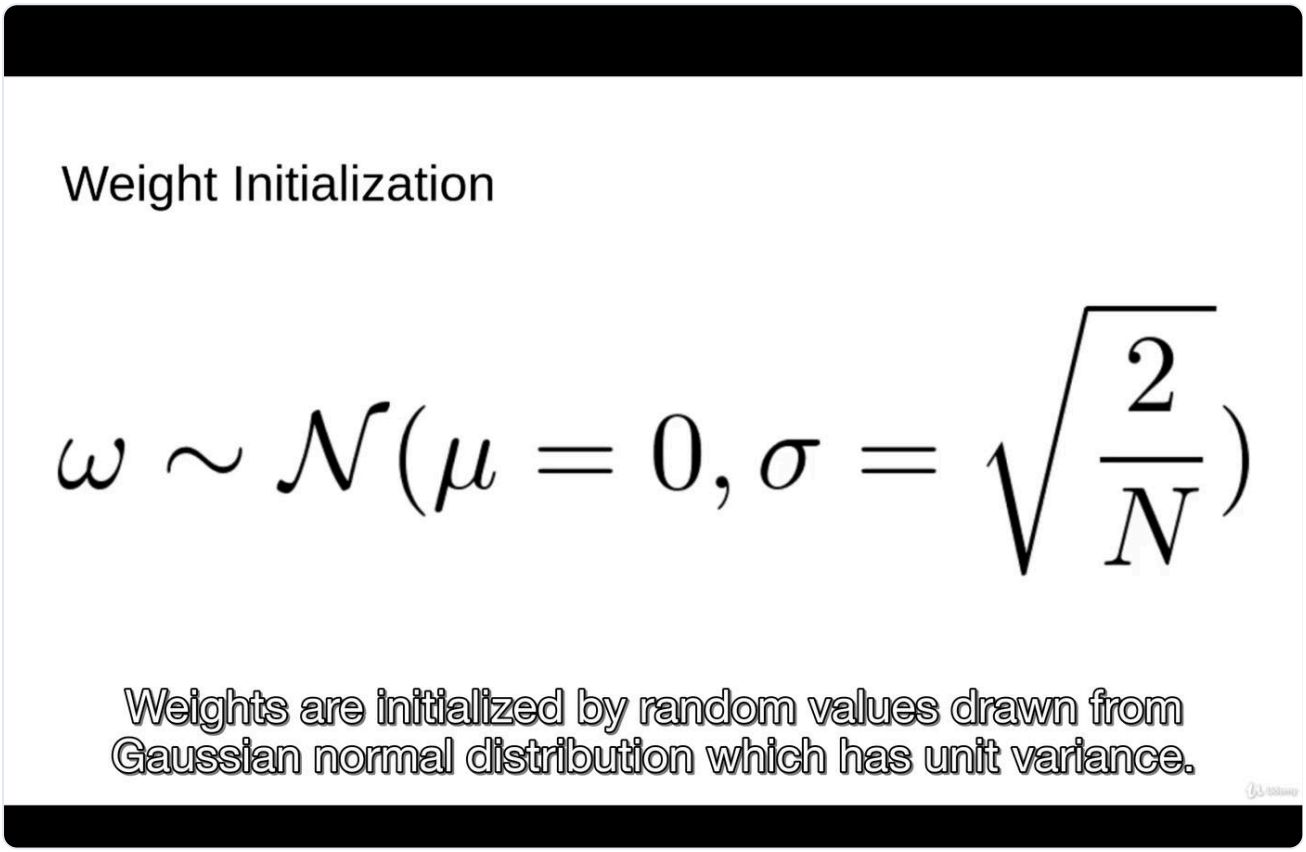


FIGURE 20.12 | Gaussian Normal Cistriloution Which Has Unit Variance

Weights are initialized by random values drawn from Gaussian normal distribution which has unit variance. The unit variance is encouraged by the square root of 2 over N, where N is the number of incoming connections to a node It is however tricky to decide the number of incoming connections when working with convolutional networks.

In this case it is the size of the previous convolutional kernel multiplied by the number of channels. For example a two by two kernel with 16 channels yields four by sixteen outgoing connections which are the multiplication of two by two by 16 now that we have laid down the foundations in the next lecture. I shall discuss the workings of two popular GAN architectures namely Pix2Pix and CycleGAN. Thank you and see you in the next lecture.

LECTURE 24

Working of Pix2Pix GAN and CycleGAN

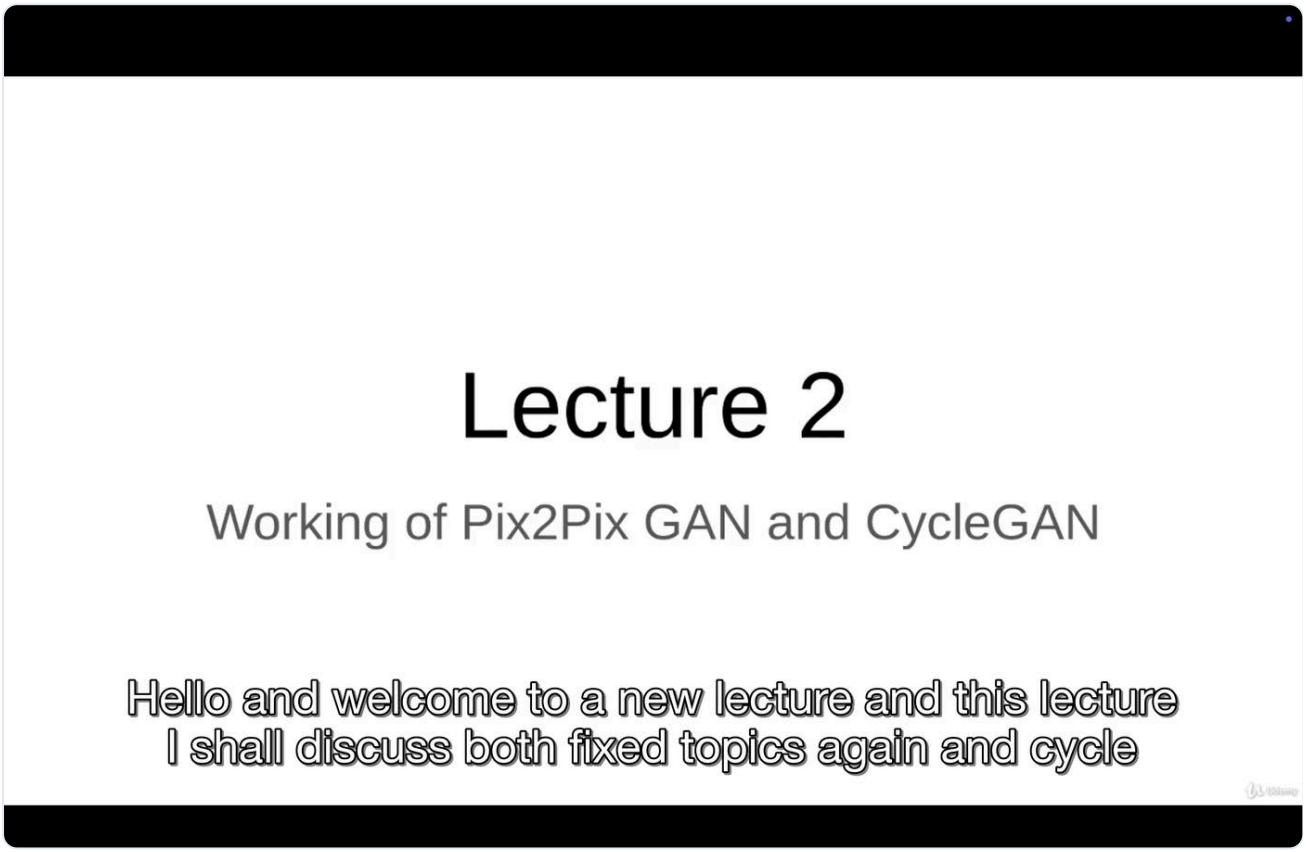


FIGURE 21.1 | Working of Pix2Pix GAN and CycleGAN

Hello and welcome to a new lecture. In this lecture I shall discuss both Pix2Pix GAN and CycleGANs.

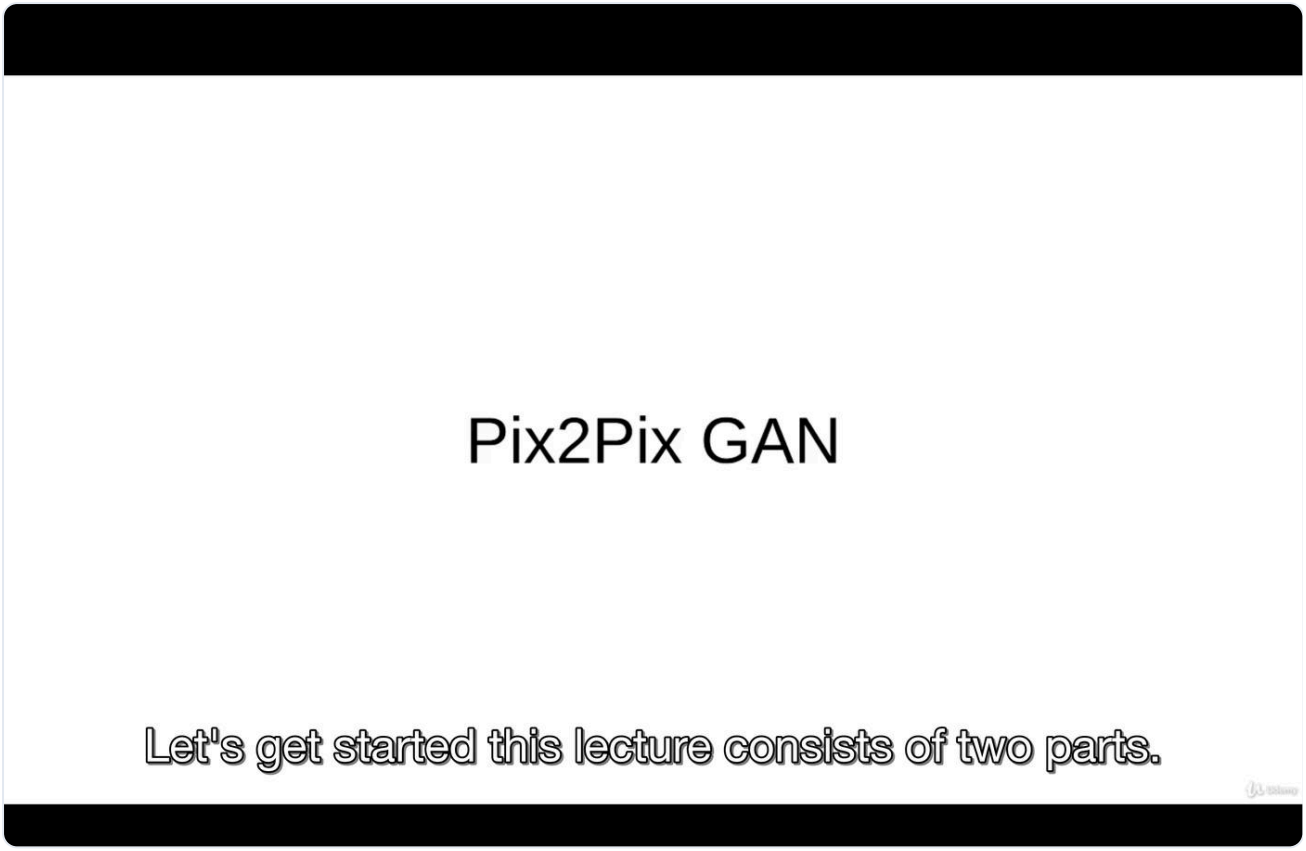


FIGURE 21.2

Let's get started this lecture consists of two parts. The first part is about Pix2Pix GANs. And in computer vision images translation represents a group of problems where one wants to transfer an image from one domain to the other. It could be a hand-drawn sketch that we want to convert to a professional painting.

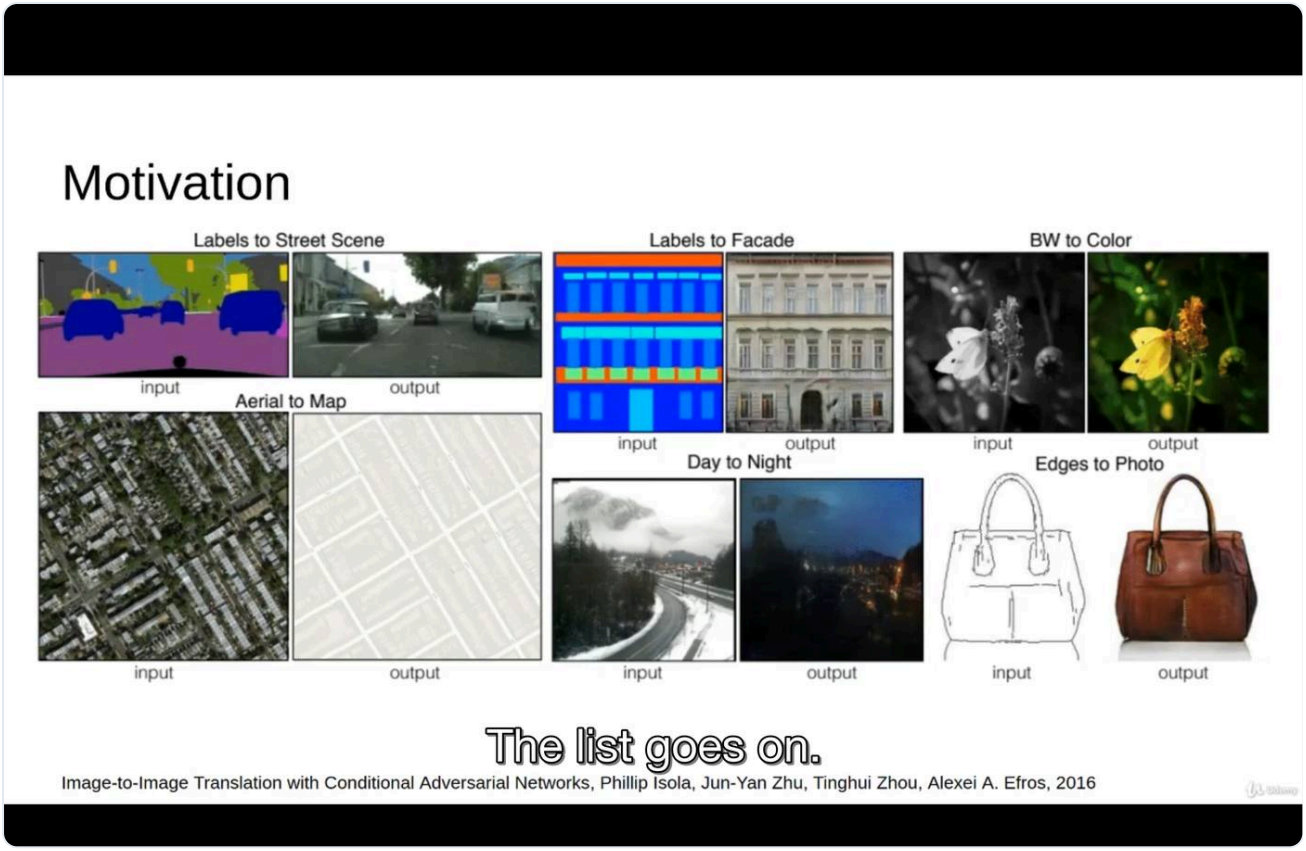


FIGURE 21.3

It could be an elevation view outline that we want to convert to a map.

It could be a black and white photo that we want to convert or colour. The list goes on. One of the main innovative contributions of Pix2Pix GANs is that the method can be used for any kind of such image translations. It depends on the image pairs that you are using in the training set Pix2Pix GANs are inspired by conditional GANs.

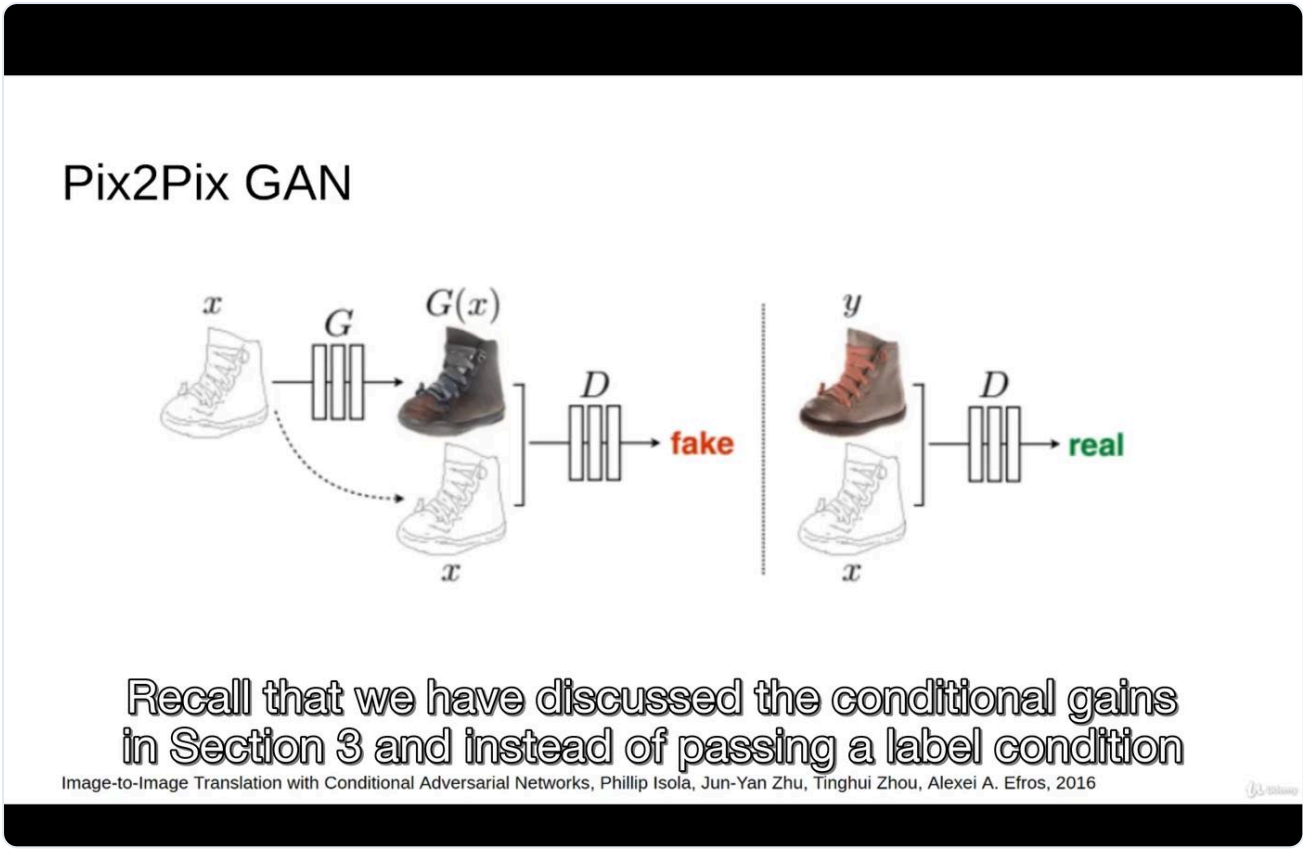


FIGURE 21.4

Recall that we have discussed the conditional GANs in Section 3 and instead of passing a label condition to both the generator and the discriminator in Pix2Pix GAN we pass an input image.

This input image is the source image for the translation. It could be a sketch a black white image or an outline The generator design is based on the U-Net architecture and the discriminator is based on PatchGAN architecture. One major distinction here is the absence of latent noise vector z. How does it work then? For a while this may seem unintuitive since of the basic idea of GANs is to use a latent noise vector. The authors of this paper and previous other papers as well have confirmed that the model simply ignores the noise in the presence of a condition.

That's one of the issues which conditional GANs suffer from.

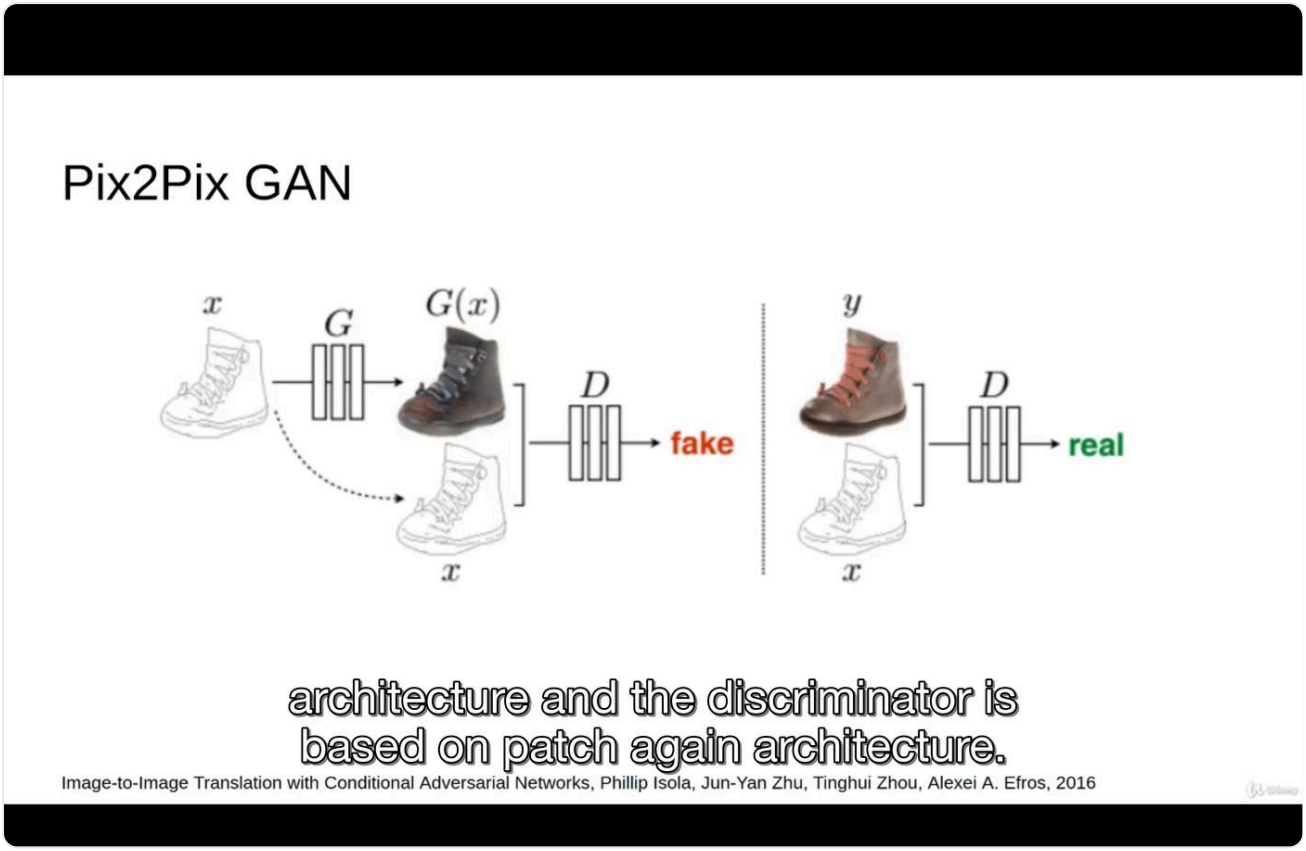


FIGURE 21.5

In the absence of noise. The model tends to produce deterministic outputs. That is, for every input it gives it the same output image. The noise helps the model randomize and diversify its outputs. That's why the authors have provided the noise in the form of dropout layers in the generator. By switching connections off at random. This trick works instead of feeding explicit random noise.

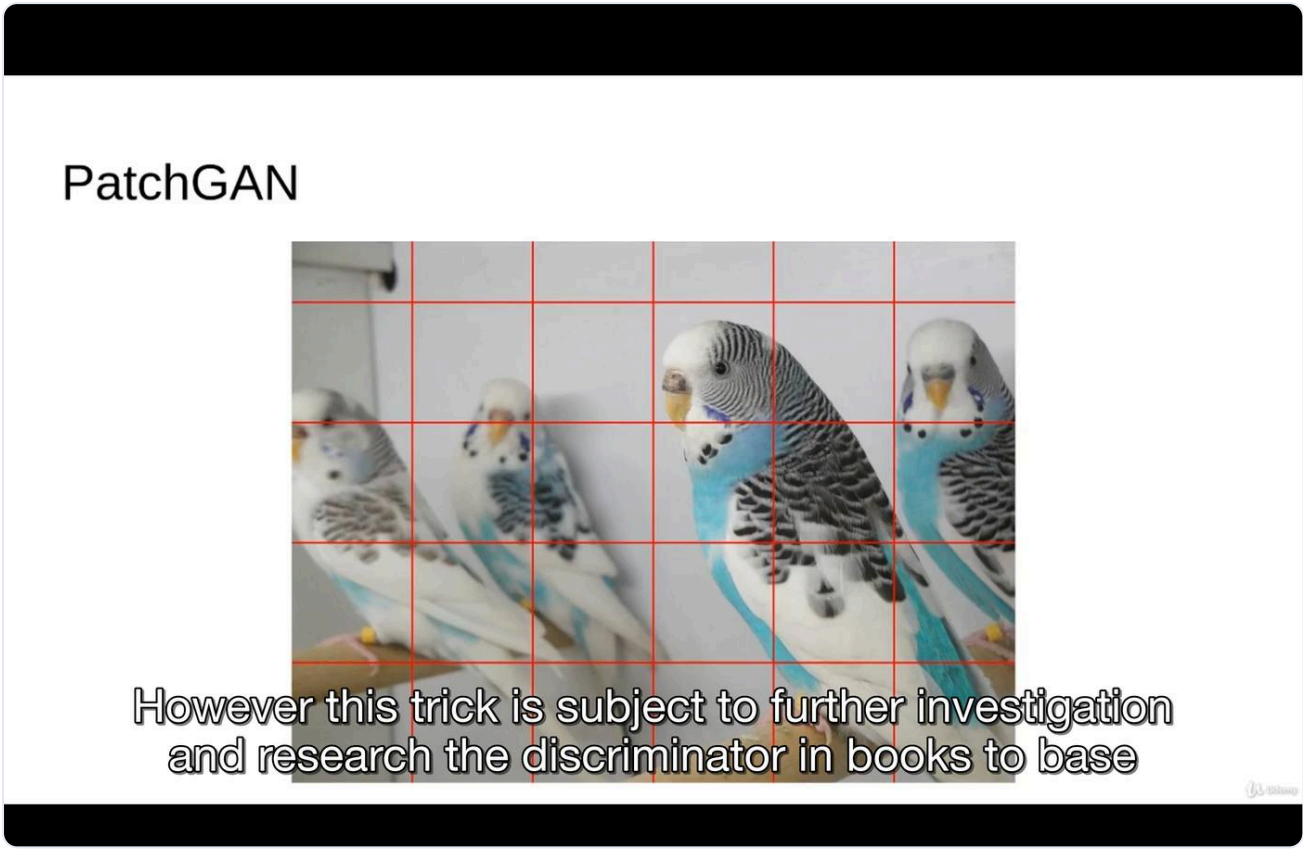


FIGURE 21.6

However this trick is subject to further investigation and research the discriminator in books to base again is based on a PatchGAN architecture.

However what does that mean. In previous sections the discriminator was given the whole image and decided whether it is real or fake In PatchGAN the image is divided into N by N patches and then the discriminator works on each patch and decides whether it is real or fake. Then the overall result is averaged for all patches. This has two advantages. The first is that the discriminator has fewer parameters and is lightweight. The second is that the pixels are treated as a Markov random field with independence assumption.

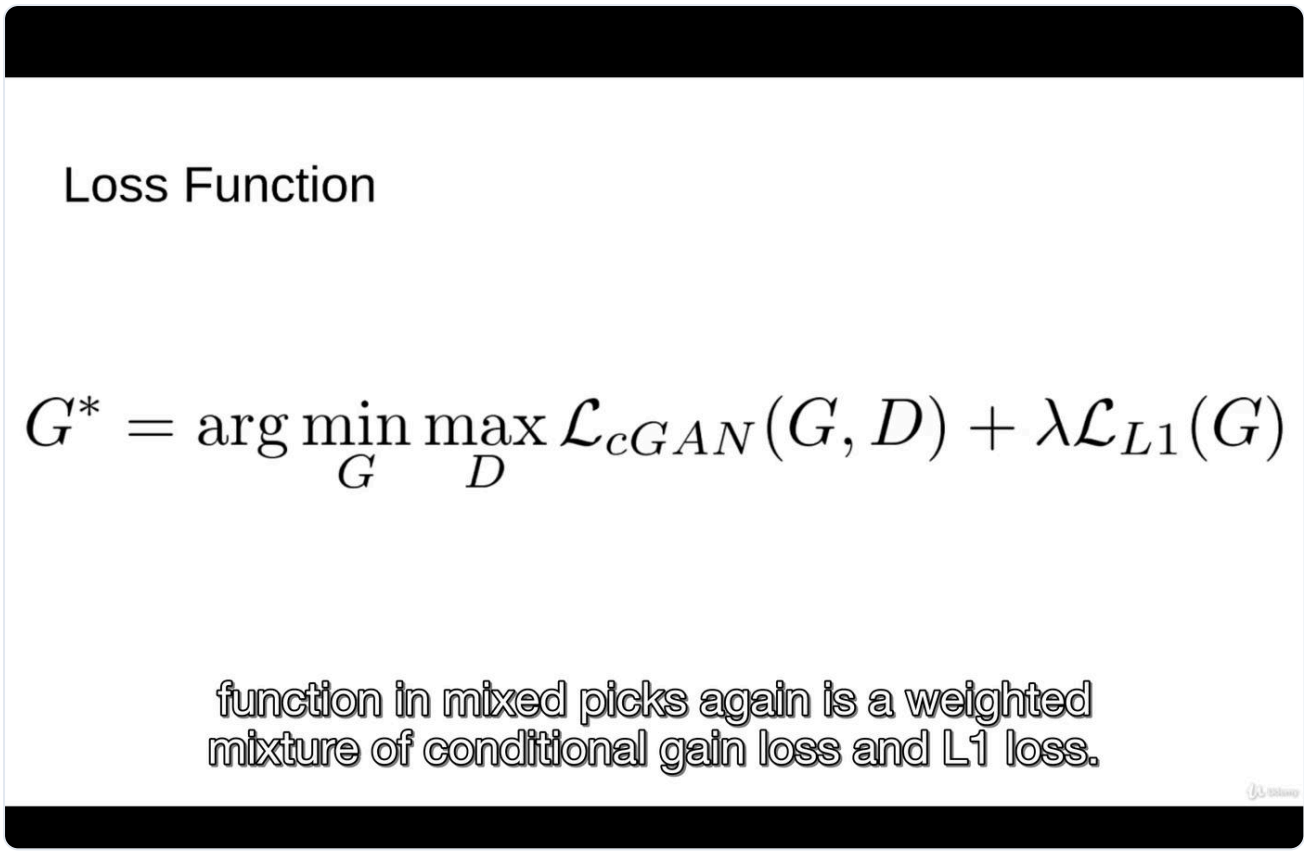


FIGURE 21.7

This formulation encourages the model to learn the finer details related to texture and style the loss function in Pix2Pix GAN is a weighted mixture of conditional GAN loss and L1 loss. The conditional GAN loss is based on the conditional probability as explained in Section 3. The authors have also favored the L1 loss over L2 loss since L2 loss is known for the blurriness in GANs. It is found that L1 loss makes images less blurry and to make these fancy terms less fancy L2 loss is simply the Euclidean distance L1 loss is simply the difference.

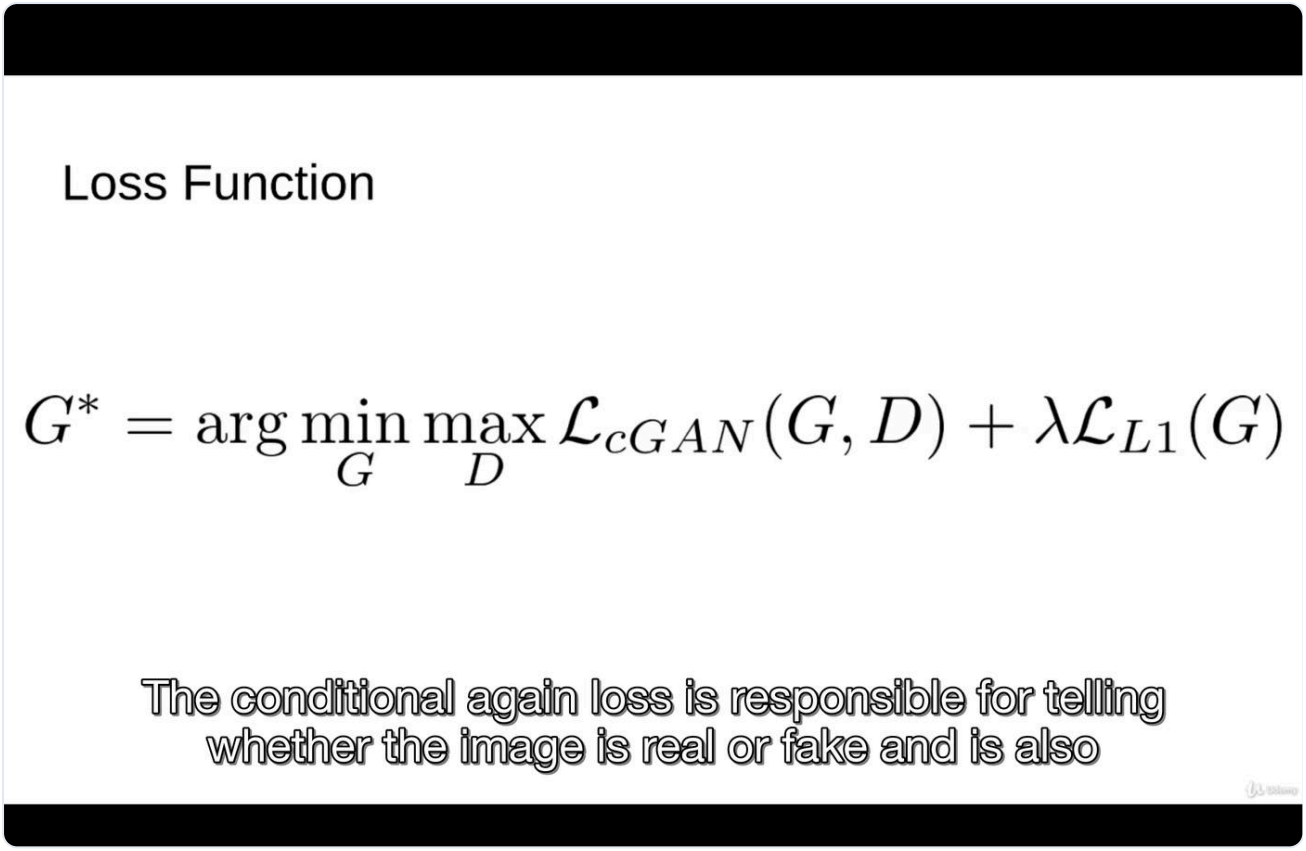


FIGURE 21.8

The conditional GAN loss is responsible for telling whether the image is real or fake and is also responsible for telling whether the image belongs to the correct category. So what role is L1 loss playing. It is a distance metric between generated images and original real images. This helps the model approximate its outputs to be as realistic as possible. The lambda factor is a weighting factor to decide the importance of the second term of the loss function.

FIGURE 21.9

I'm talking here about the Greek letter lambda which looks like an inverse y letter. So for example the Pix2Pix GAN has the capability to convert edges to objects. In this example the network learns to map human drawn sketches to photorealistic objects the potentials of Pix2Pix GANs are endless. In this sense again remember it is conditional GAN with a U-Net generator and a PatchGAN discriminator. Very intuitive and super powerful let's talk about one of the interesting open questions that that the Pix2Pix GAN authors left unanswered.

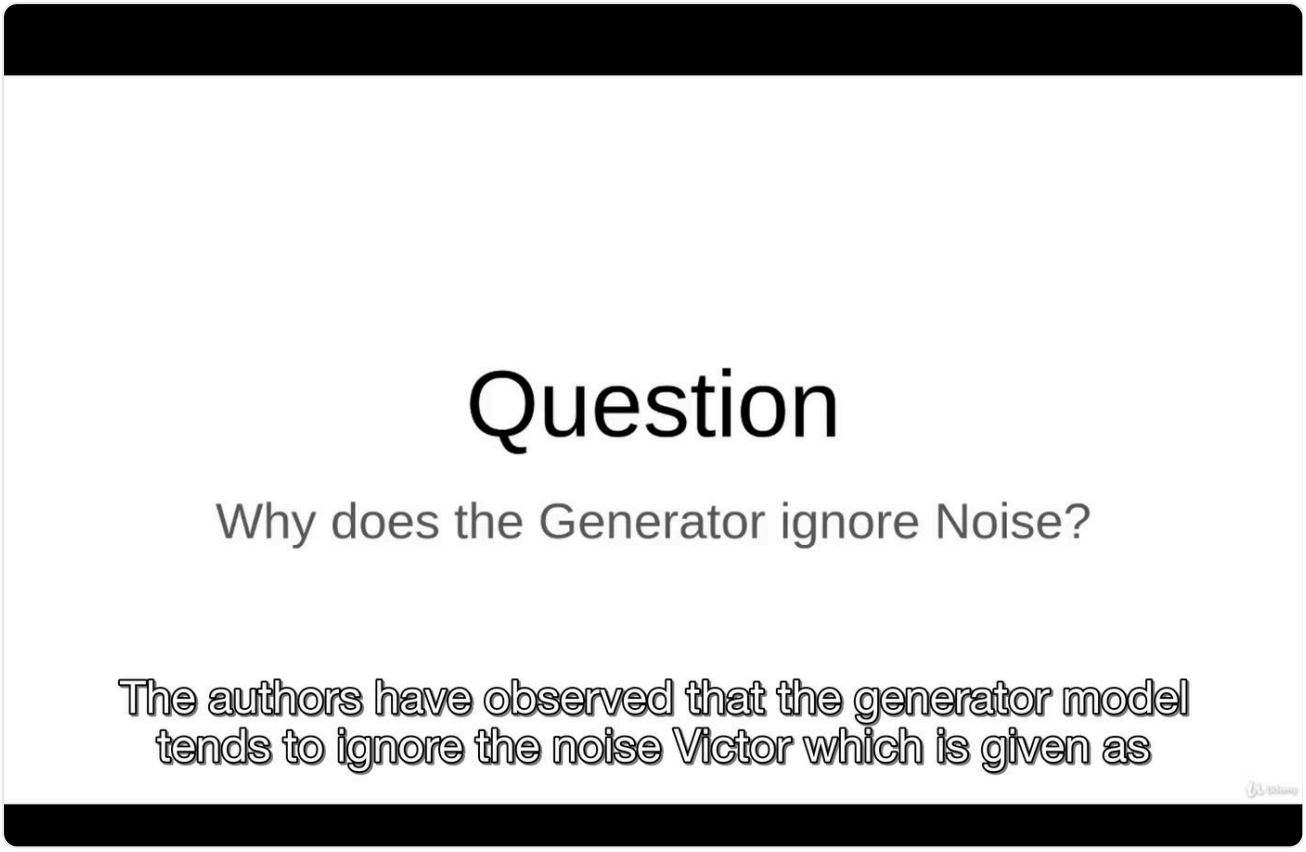


FIGURE 21.10 | Why Does the Generator Ignore Noise?

The authors have observed that the generator model tends to ignore the noise vector which is given as an input. The Z vector, remember, is responsible for encouraging randomness in the outputs. Even though they have used dropout layers as a substitute. Still haven't realized the output diversity desired.

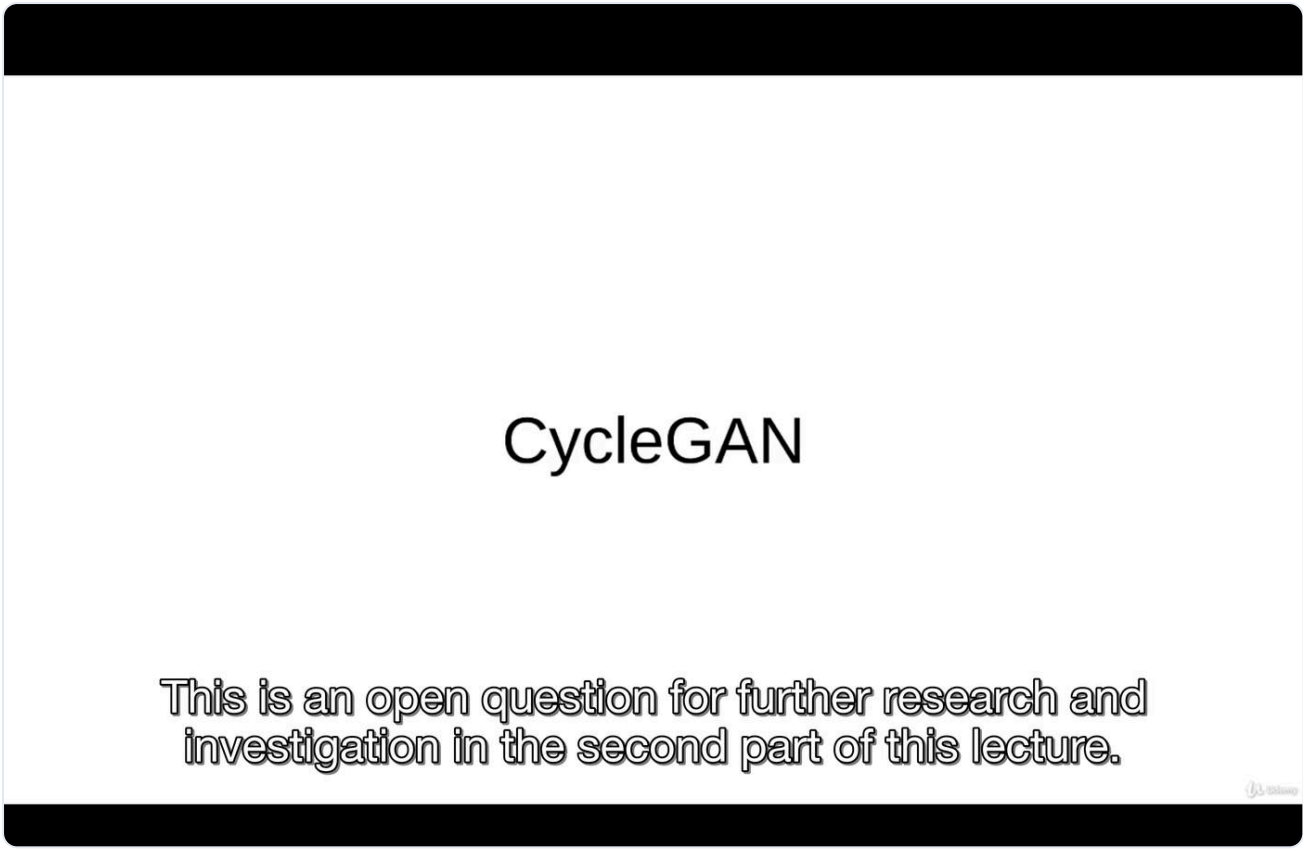


FIGURE 21.11

This is an open question for further research and investigation in the second part of this lecture. We shall explore another novel GAN architecture namely CycleGAN at the heart of CycleGANs lies the motivation of unsupervised learning.

What if we don't have image pairs for every situation. Can the networks still learn without using image pairs.

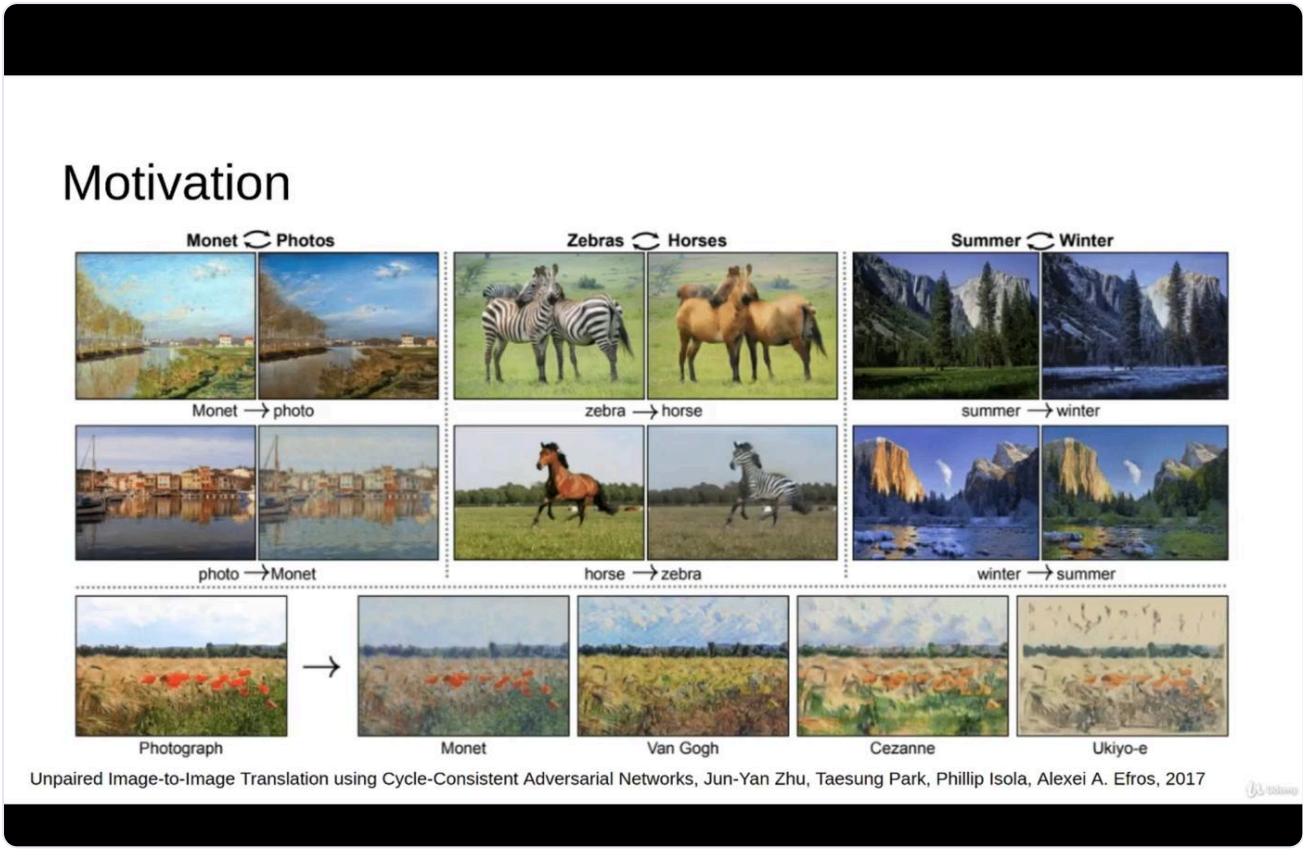


FIGURE 21.12

This is a serious issue. For example if you are learning to do age regression or age progression like the FaceApp does then you don't necessarily have image of the same person in every age group to train your model on CycleGANs address learning without having image pairs for training. For example if an artist paints a natural scene using his own imagination his own style and his own brush strokes.

Then if you as an observer see another natural scene then you can imagine how it looks like if painted by the same artist.

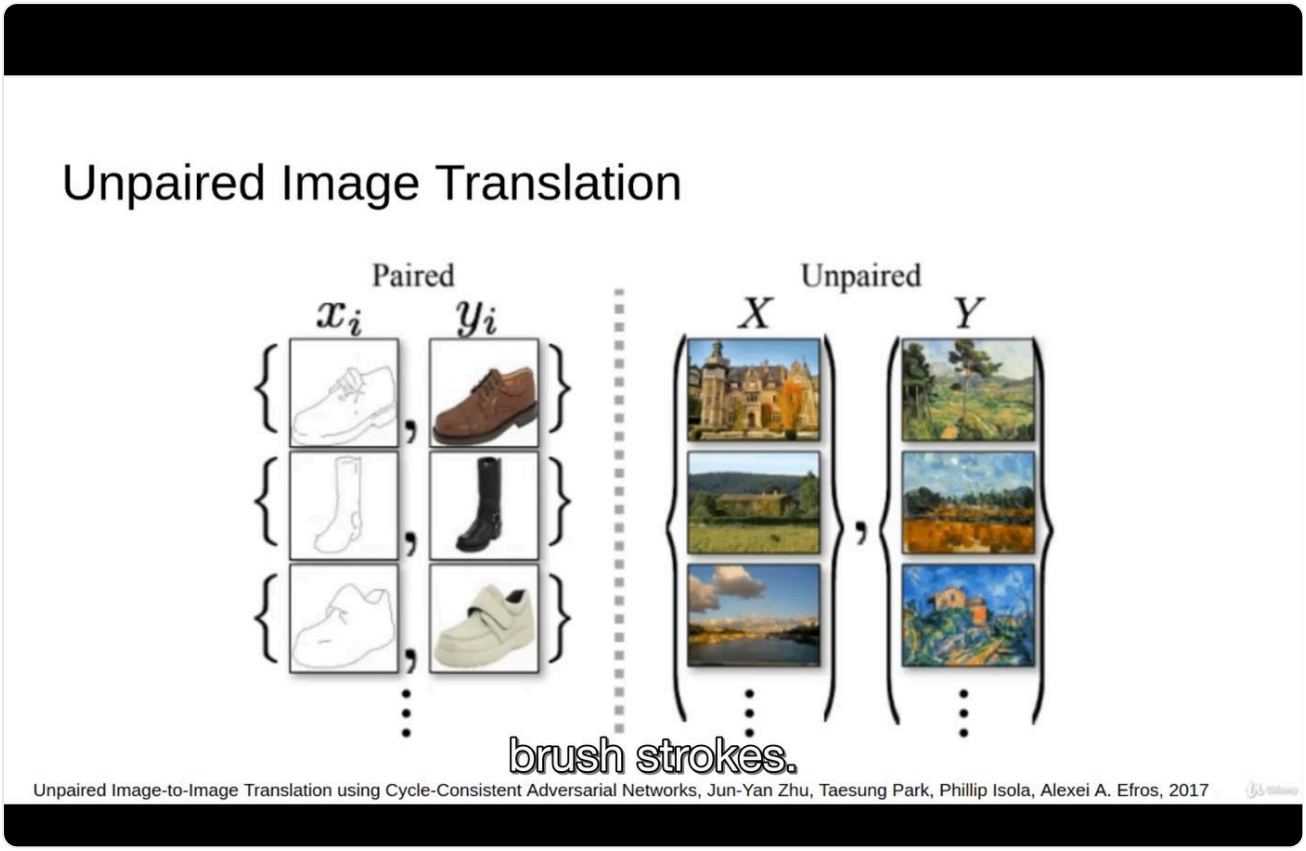


FIGURE 21.13

This is the intuition behind CycleGANs. if we have unpaired images from different domains without direct mapping between them then a CycleGAN model should still be able to translate images between these two domains Strict paired translation. In the ideal case where we have image pairs for every source and destination image however in unpaired image translation We have source images and we have a set of target images that don't necessarily correspond to any of the source images.

However we want to design a model that captures the inherent characteristics of the target images and projects them when given a source. Image from elementary school math.

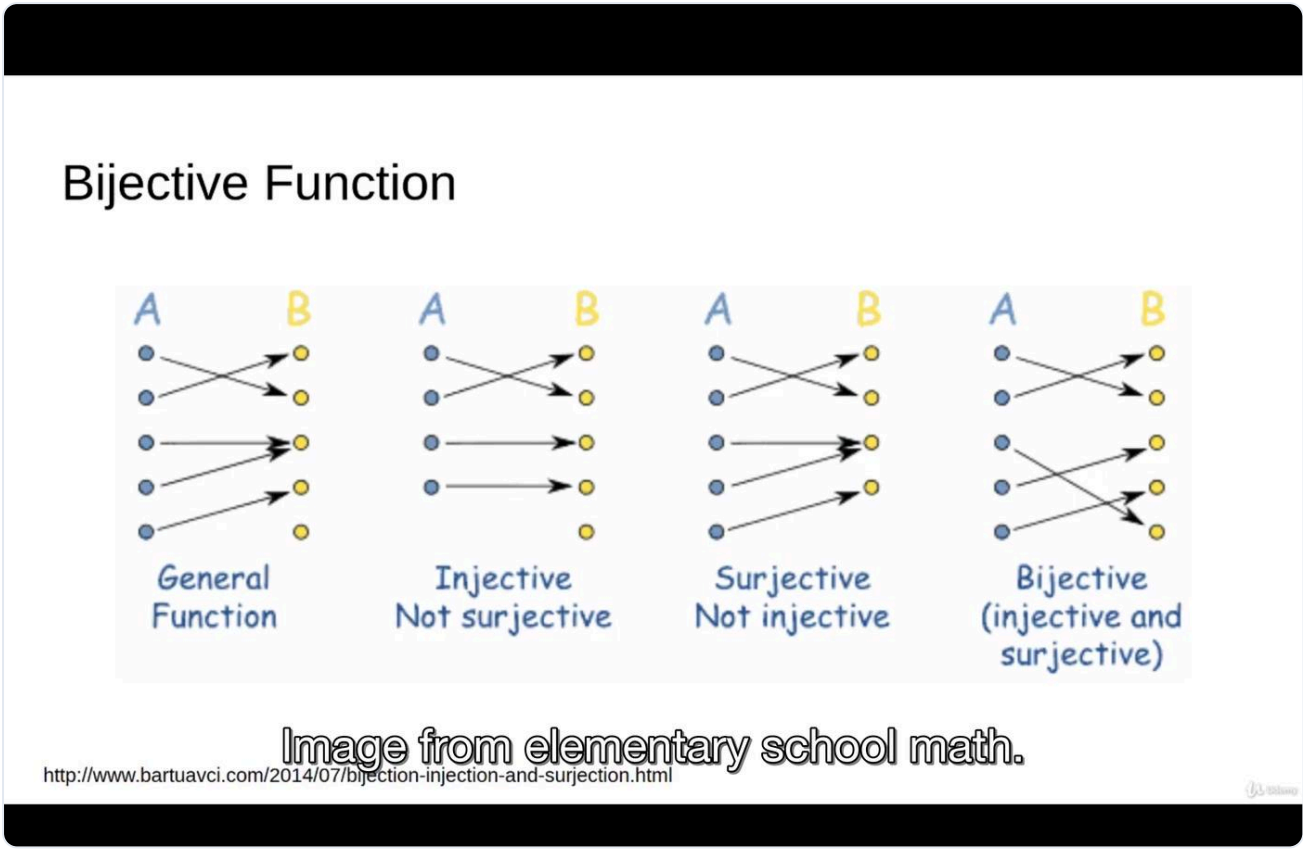


FIGURE 21.14 | Bijective Function

We know that a function is a mapping from the domain A to the co-domain B In an injective function we map every element from domain A to one and only one element from domain b a surjective function however ensures that for every source element there is a target element that is the co-domain elements are completely covered a bijective function is both injective and surjective.

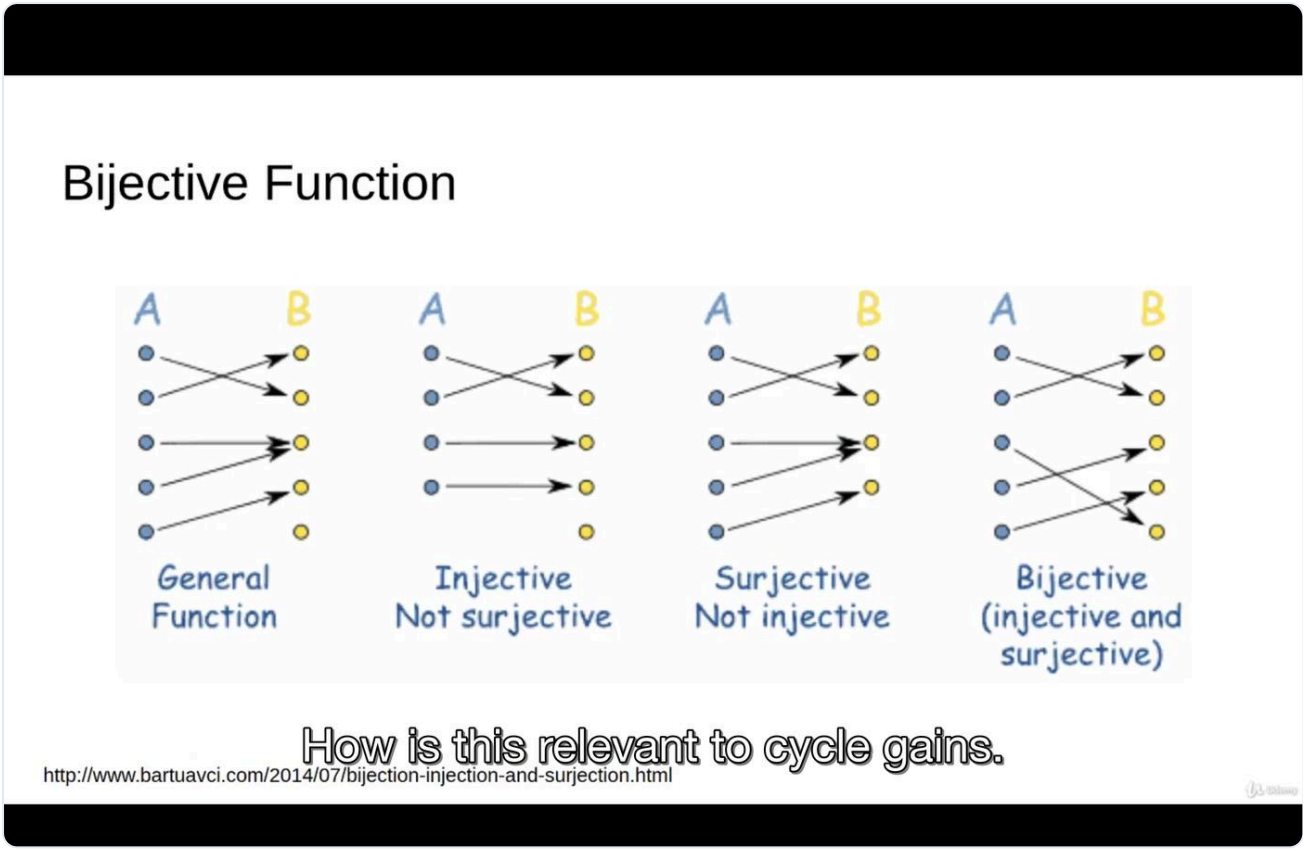


FIGURE 21.15 | Bijective Function

How is this relevant to CycleGANs. It turns out when training a GAN on unpaired images we have a mode collapse, a problem where all source images map to a single relevant target image. This is expected since there are no constraints on the mapping. So apparently we need an objective function that maps every source image to one and only one target image and makes maximum usage of available target images and covers all of them so given that X is the source image and y is the target image then the generator should learn a mapping from X to Y namely G of X.

So far we are still getting the most collapsed problem. So how about a reversal.

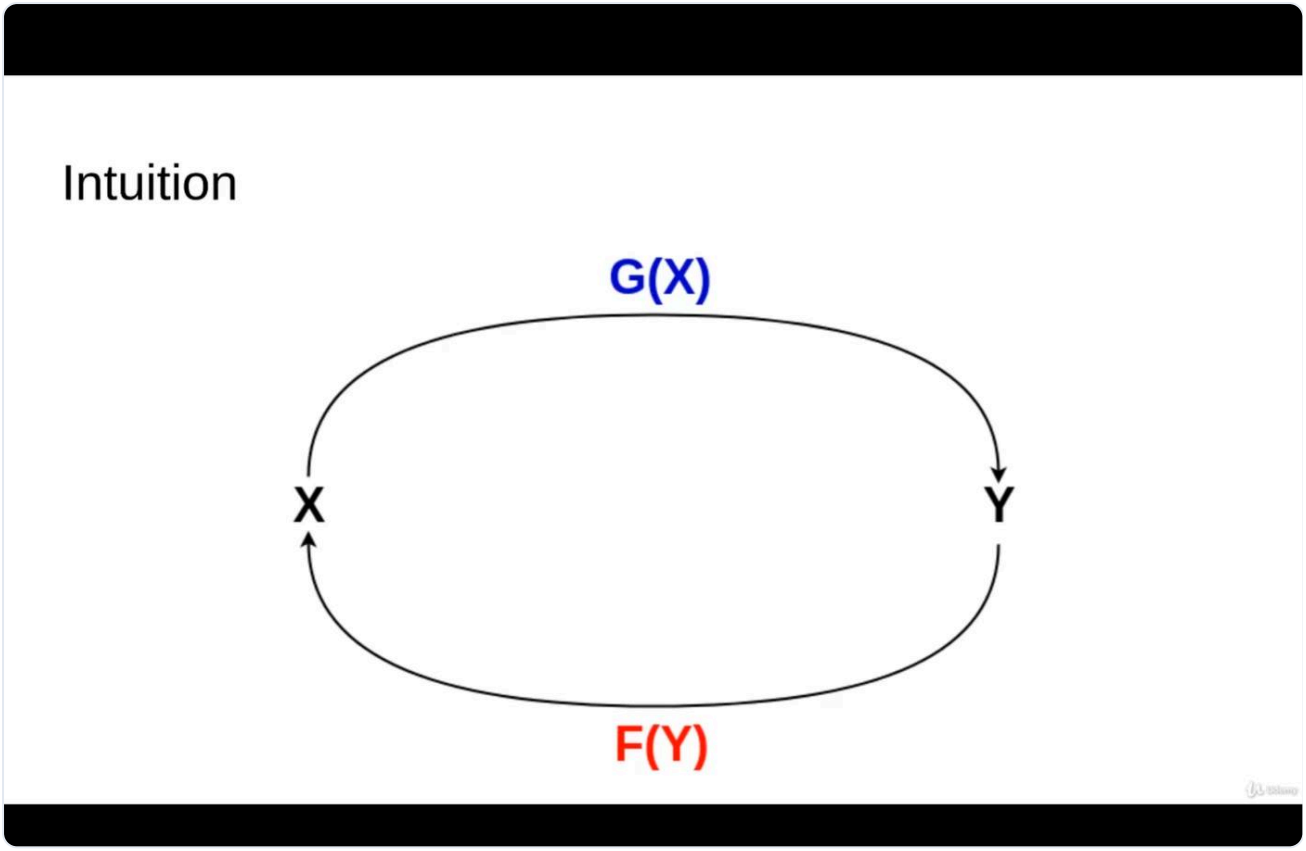


FIGURE 21.16

What if the model can learn a reverse mapping from y back to x. Let's name the reverse mapping f of Y this serves as the bijective function constraint. We need to translate the image forward and backward to reconstruct the original image. This also implies that we now have to train two generator models one for G and the other for F this is intuition. As you may have noticed throughout the course Deep Learning is more about intuitive ideas that means we have two generators and subsequently two discriminators. The first GAN. Again it's for mapping from source to target domain.

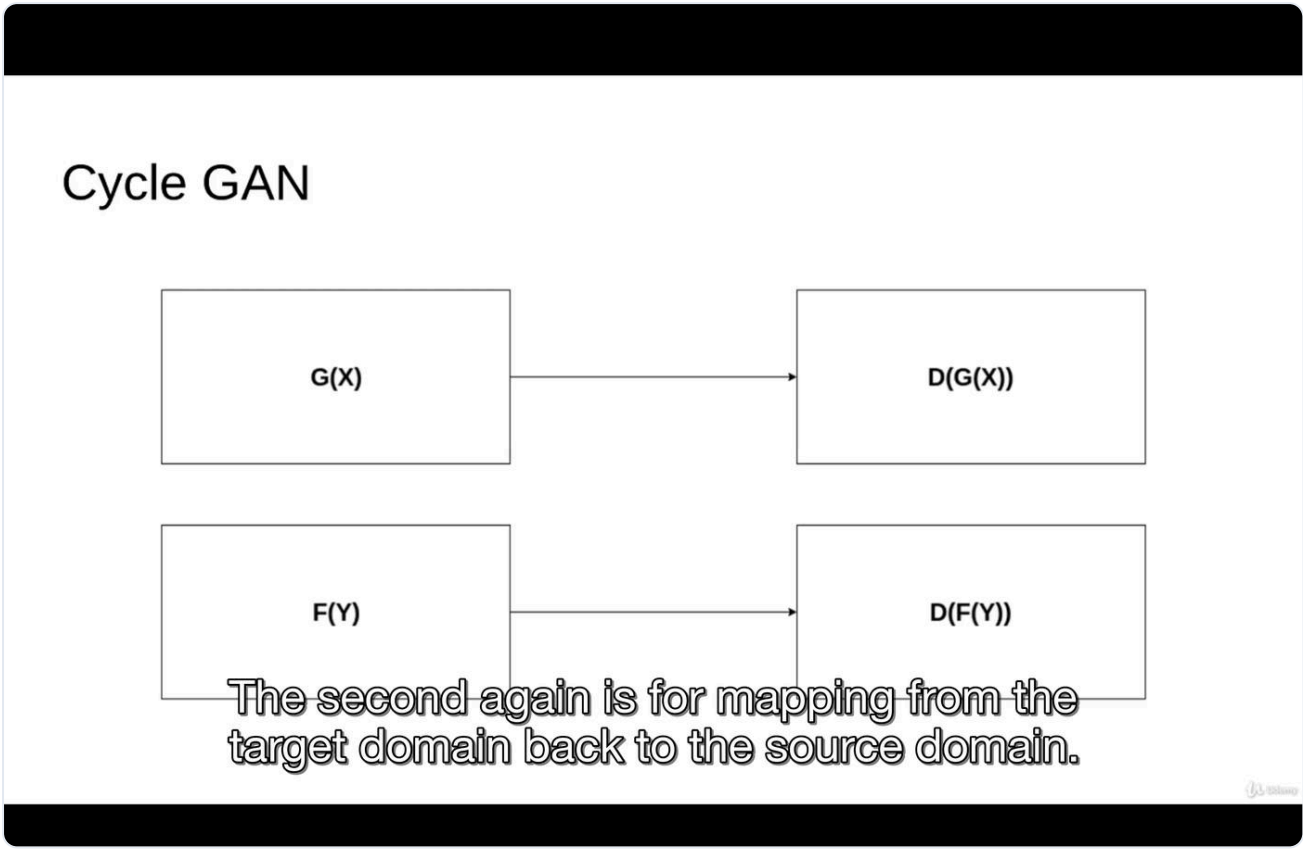


FIGURE 21.17

The second GAN is for mapping from the target domain back to the source domain. And this leads us to the recurring question what the loss function should be so in addition to the GAN min max objective function we need to enforce an objective function constraint.

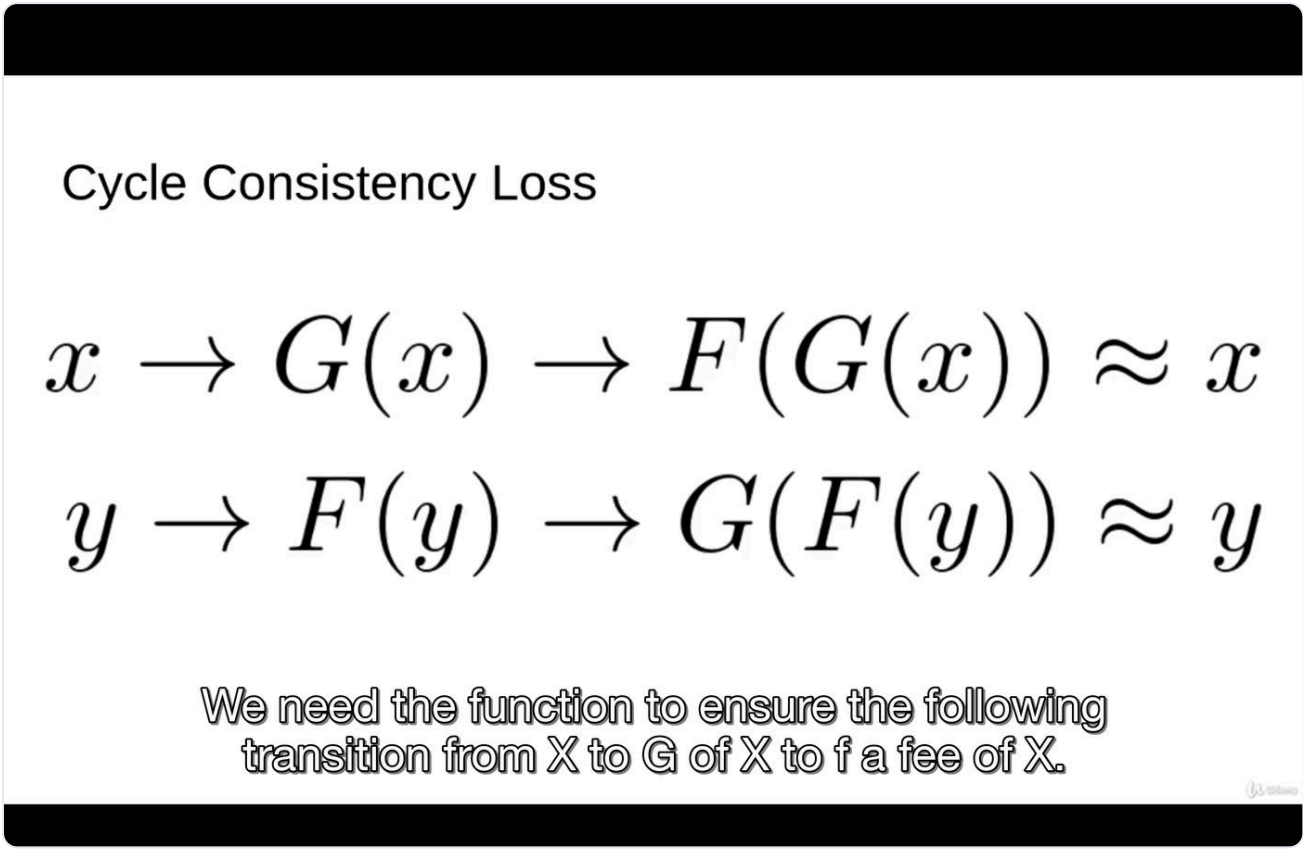


FIGURE 21.18

We need the function to ensure the following transition from X to G of X to F of G of X. And finally back to X. The same for y.

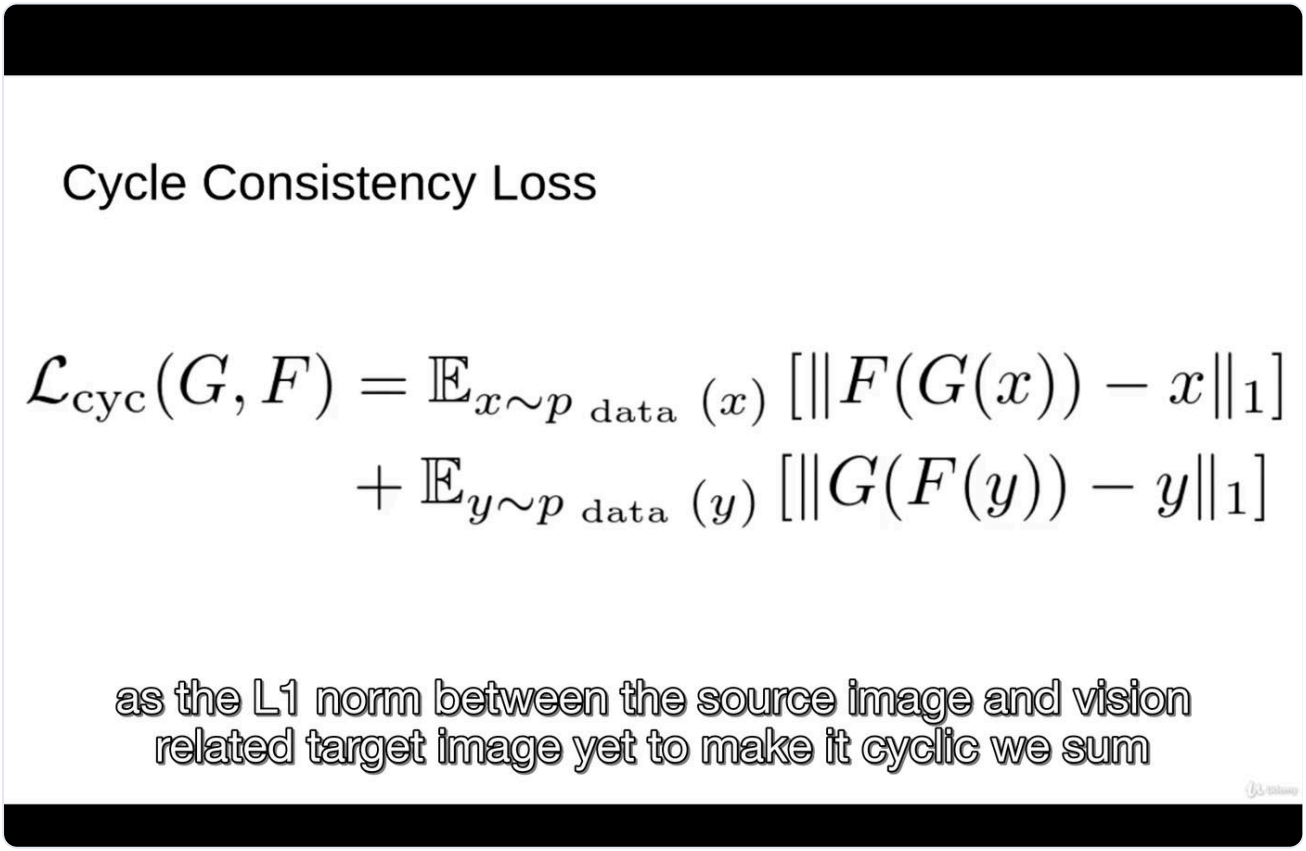


FIGURE 21.19 | Cycle Consistency Loss

This leads us to the formulation of the cycle consistency loss the cycle consistency loss can be written as the L1 norm between the source image and the generated target image yet to make it cyclic we sum to that the L1 norm between the target image and the reconstructed target image.

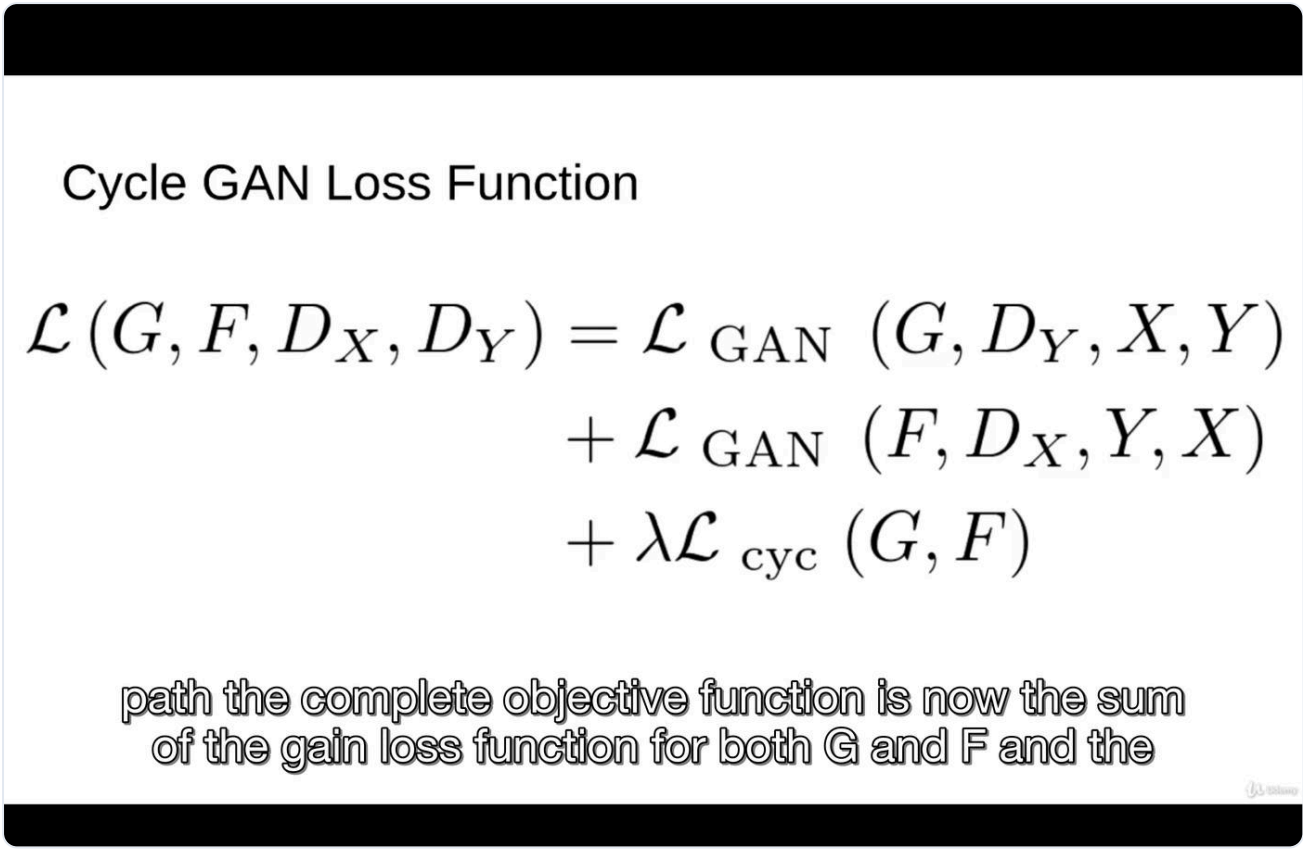


FIGURE 21.20 | Cycle GAN Loss Function

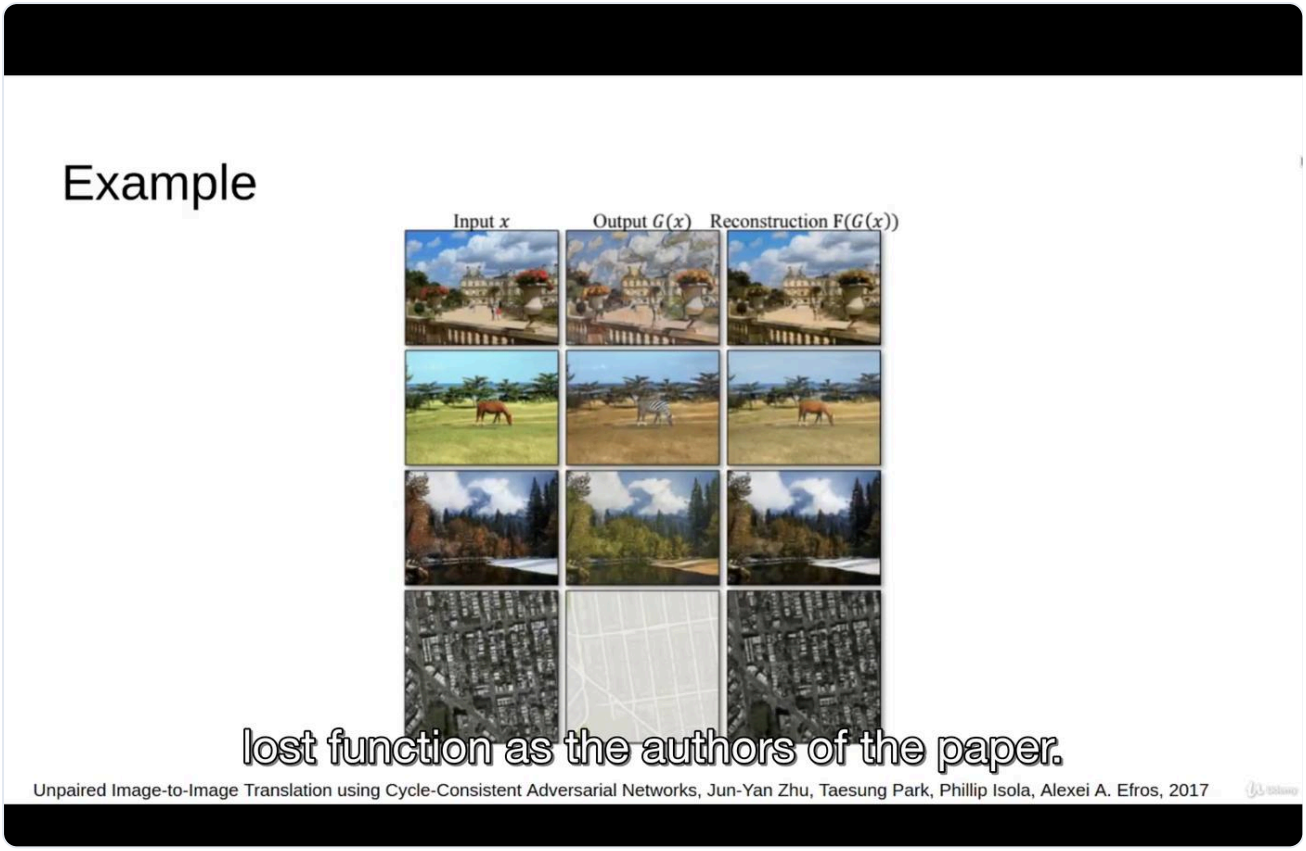


FIGURE 21.21

This form of cyclic loss is necessary and sufficient to ensure the bijective constraint and the reconstruction path the complete objective function is now the sum of the GAN loss function for both G and F and the cycle consistency loss as well. The Greek letter lambda is a weighting factor to decide the importance of the second term of the loss function as the authors of the paper. Here is an example of the working of the cycle loss function in the next lecture.

Let's cut to the chase and introduce the apex of our course by discussing the Vid2Vid GAN. Thank you and see you in the next lecture.

LECTURE 25

Upsampling Path and Downsampling Path

See the notebook `13.Hands-on-Pix2Pix-GAN.ipynb` for the code.

[Coding Exercise] Hands-on Pix2Pix GAN

Hello and welcome to a new coding exercise. And this coding exercise we are going to implement Pix2Pix GAN. Let's get started. We shall start by defining the device either to use the CPU or the GPU and then we keep our global hyperparameters in one place so that we can optimize them along the way such as the learning rate and the number of training steps for the discriminator. Next we download the dataset from Berkeley.edu. Afterwards.

We unzip the tar file on the disk. We define directories to be the directory name for each of the training validation and testing sets. Here it gets interesting. We have to define our own dataset. class, such as the way it is implemented by PyTorch for dataset classes like image folder class and in this way we need to implement two functions the getitem function and the len function. This is necessary because the data set has two images stitched together in one file.

We have to split them to two files and retain one as the source and the other as a target. Images you'll find that we are doing this here on line 24 where we split the width of the image over two and then we divide the 1 image into two images one for the real image and the other for the source image then with image. This will become clear as we visualize the images. Next we define the data loaders for both training validation testing sets then we visualize a couple of images from the data set after we have applied a custom data set loader here you can see the source image which is a segmentation that has one color for each object of the target image and then on the right side you have the target image which is the facade of the building.

This is the one that we are going to generate similar to this one and since both of them were stitched in one file we had to apply a custom dataset class that will load the images split them by the width into two one for input as the condition and the other for output and the target image. Next we design the model we build the convolutional blocks as reusable functions that define convolutional blocks. And here we use strided convolution and we use a condition if batch norm so that we can control sometimes we need a batch norm in the layer, some other times we don't we do the same for transpose convolution with the strides as well but this time we use dropout.

Batch norm is used all the way in both cases but sometimes we use dropout, sometimes we don't next we defined the generator class which is an implementation of the U-Net architecture.

Except that the original U-Net architecture had Max pooling layers. But in GANs we are not going to use max pooling because we know that it is better to use strided convolution from previous sections. And here you have some guidelines on the outputs dimensions of each layer so that you can follow along and be consistent with the design of this notebook. We do two blocks, one for

the downsampling path and the other for the upsampling path I remember that we have to apply skip connections we define the last layer which outputs the actual image we are using tanh activation, or hyperbolic tangent function, that would be between minus 1 and plus 1.

In the forward method we define a list named skips and we append each output from the downsampling path we reverse the order and remove the first one and then when we are applying the upsampling path we concatenate both the outputs from each layer of the upsampling path to the output of that downsampling path which is named the skip. So here you can see torch.cat which applies concatenation to both the outputs from an upsampling layer and the outputs from a downsampling layer together and concatenate them along the first dimension which is the channel dimension in PyTorch and then we pass the outputs through the last convolutional layer current transposed convolution and then each hyperbolic tangent (tanh) activation return the result which is an actual image next to redefine the discriminator which is not very peculiar except that it is a PatchGAN, that means that every portion of that input image has one element in the output tensor that corresponds to whether this patch is either fake or real.

The remaining of this notebook is consistent with everything that we have learned throughout the course. We defined the optimizer the models and the loss functions. We pay close attention to the loss function because the Pix2Pix has a multi-loss function that I will explain in the next slides so for the discriminator we use the same binary cross entropy loss for both the real images and the fake images. However for the generator for the generator we use mean absolute error loss together with the binary cross entropy loss to the binary cross entropy loss will determine whether the input image is real or fake and the mean absolute error loss will determine how far is the generated image from the target image.

And this way we can enforce the model to output images that are very close very realistic as possible to the target images and then we combine them both of them using the addition and we add a lambda factor that's a priority, a weight factor so that this factor will determine the importance of the mean absolute error loss in the equation. Finally we use TensorBoard and we use the checkpoints similar to the previous sections we loop over the number of epochs and over the dataset the number of steps and we do four K steps training the discriminator and only one step training the generator and every ten epochs we save the loss values for both the generator and the discriminator and checkpoint and save the optimizer and the network states we do also visualize a couple of images on fixed inputs given to the generator.

And here we can see because of the lack of GPUs I have trained it only for 10 epochs and usually you would need a GPU with more than 16 gigabytes to train this model because it has millions of parameters to be trained, millions of weights. So this was not possible at this stage I would suggest that you implement your own version of Pix2Pix from scratch and use any reference or material that you find useful along the way and share the results with us.

Thank you and see you in the next lecture.

LECTURE 26

Target Domain and Cyclic Loss

See the notebook `14.Hands-on-CycleGAN.ipynb` for the code.

[Coding Exercise] Hands-on CycleGAN

Hello and welcome to a coding exercise in this exercise. We're going to implement CycleGAN. Let's get started our benchmark on the code that we have developed in the Pix2Pix GAN exercise. And I would only change the loss function and I will make no changes to the dataset loader. Let's see how it works so I'd be using the same data set. For the dataset. I will add one more parameter, category, and this parameter will be either 0 or 1. If the parameter is 0 we return the input image and otherwise if the parameter is 1 we return the target image because CycleGAN is built on the idea of unpaired image translation.

That means we don't have pairs anymore of X and Y or source and target. But we have images lying in one domain and other images lying in the other domain that we want to transfer the images to. So here we have this concept of unpaired images. That's why I have changed in the dataset class to accommodate this concept so you can see here this condition inside get item method itself. If category equals zero we return the input image, otherwise we return the real image or the target image and hence each image is unpaired from the other image we can easily apply the random horizontal flip with probability 50 percent or point five.

Unlike the other exercise in Pix2Pix GAN when we had to forgo this part of the processing because it wasn't easy or straightforward to flip both the source and the target images at the same time then I will create a train set for the input image and another set for the target image. but with the category with value 1 subsequently I will try to implement two data loaders for both the input domain and the target domain. So here you go. A few samples from the data set from the target domain and you can see that this is only the facades dataset without the segmentation.

If we use the other data loader for the input domain we will get the segmentation images for simplicity. I will keep the model design the same exactly the same like we did in the Pix2Pix exercise. I will only change the fact that the discriminator takes two images as input. I will give it only one image of the input to the discriminator and hence I will reduce the number of channels from 6 down to 3. But everything will remain the same just for the sake of simplicity.

But in every situation I would suggest that you implement ResNets or residual blocks for both the generator and the discriminator as advised by the authors of the paper. In this exercise I am more interested in demonstrating how to implement the cyclic loss function and how to get your hands around it. That's why I'm focused in this exercise on implementing the cyclic loss for the CycleGAN so that we can appreciate the complexity of these models and the requirements of their RAM or memory and their GPU requirements.

Here you can see that we are creating 4 models and instead of only 2 that means double that number of parameters double the number of weights that need to be learned in the process. So we are creating a discriminator that discriminates in the input domain. Another discriminator named D_Y that discriminates the target domain. A generator that converts the image from the input domain to the target domain. Another generator that transforms the image from the target domain back to the input domain and subsequently will create four optimizers for each one of them, and now let's start with the focus of this exercise which is the objective or the loss function.

The cycle loss function. I will create the lambda variable assigning the value ten, unlike the value 100 in the previous exercise. Again this is a recommendation by the authors of the paper and then I'll create a variable for the binary cross entropy loss another one for the mean absolute error. I will see subsequently how I'm going to use both of them to create the real labels all ones fake labels all zeros. Remember that as a hint from a previous session you can switch the values ones and zeros for real and fake labels in order to improve the stability of the.

GAN training. Try it and see whether it improves it or actually degrades it because it depends on the situation at hand. Now when we moved to training the discriminator we have X the input domain, Y the target domain. Then we pass x and y to both net G and net F and then we get G of X and f y. And if you notice here I'm sticking to the annotation used in the mathematical formulation of the loss function so that it's easy for you to trace down the coding implementation and find out that it is a straightforward mapping from the mathematical equation to the coding and then zero_grad optimizer and I'll pass the X to the net D X and I'll pass the F of Y then net D of X again.

Remember that when we are doing net G of X that's equivalent to getting a Y back but this Y is a little bit shifted from the original target y. And again if we do net F of Y that's equivalent to getting an x back but again shifted from the original x value. So again we are trying to train the net D of X discriminator to recognize the X images or their equivalent by training it on X as the real value and F of Y as a fake value and encouraging the model to learn to discriminate.

Both cases whether valid or invalid. I would do the same again for discriminator Y and then I would have the binary cross entropy loss for the real outputs versus the real labels and the binary cross entropy loss for the fake outputs versus the fake labels. I will sum them together and since we have four terms I will divide by four or I will just do point two five of the whole value. That's equivalent to averaging the loss. I will apply it backward and then optimize.

I will apply an optimizer step for both optimizer of D_X and optimizer of D_Y. That's simply for the computer except of training on you on this computer. In this exercise we are training two discriminators, and aside from that nothing has changed except the formula where we sum all the values together and divide by four. It gets more interesting when we try to train the generator so we have X and Y as input and target domain images. We pass them to net G and net F and get the output G of X and F of Y and then we pay them again backwards through the net.

If an energy so that we can get the backward values. So for example if we have passed X through net G we get Y. If we pass G of X through net F we get X back again and the same for Y. So this is kind of cyclic forward and backward both in the networks because we have two discriminators,

two generators then we zero_grad the generators G and F and then we pass the values but only the generated ones through the discriminator and encourage the generator to maximize the probability that its output is determined as real labels.

Hence we are using binary cross entropy between the fake outputs generated outputs from the network and the real labels which is essentially a tensor of ones. We do that for the discriminator X we do that for the discriminator Y but then we apply two kinds of losses the identity loss and cyclic loss simply we are trying to make it so that the output of either generator is not differentiated or cannot be distinguished from the original images. So we are trying to advise the model that whatever images it is going to output or generate it should be very close as close as possible to the original images either in the input or the target domain.

We do that by simply applying the mean absolute error loss which calculates the difference the absolute difference between the input images and the generated images between the target images and the generated images. We call that identity loss and then we apply it again. the mean absolute error loss between the cycled images: we apply X through the generator we get Y we take Y applied through the backwards generator we get X again. So we tried to make sure that the X we have just obtained is very close very similar to the original x.

We have just passed into the input the same for y and that we call the cyclic loss. We divide by two or multiply by half so that we can get the mean of the sum of two values and that will constitute the identity loss and the cyclic loss of the general objective function of CycleGAN so next what we need to do is sum all the values we have got from the discriminator loss for discriminator X and the loss for discriminator Y and then we multiply lambda by the mean absolute error loss or the identity loss.

And again sum with lambda multiplied by the cyclic loss that would give us the total loss. We apply backward and we optimize for the generator G and F by applying optimizer_G.step, optimizer_F.step. That's pretty much it. You can find some tweaks for this kind of formula you can find different implementation of it. Just let me know if you have any minor tweaks or suggestions to modify this formula but in a general sense and a high level sense of implementation that's how it goes about CycleGAN.

Aside from the cyclic loss you need to apply residual blocks, ResNets and instead of using the U-Net architecture which I have borrowed from the Pix2Pix implementation but is still valid once you have a powerful enough GPU you can implement the CycleGAN in a reasonable way and get reasonable results. But with the limited resources of graphical processing units, GPUs, and limited resources of the memory RAM it is not possible to go beyond this implementation much and can.

I don't have much control to tweak it over but I suggest that you can use something like Paperspace or Google Cloud or Amazon Cloud to train the model, but that would be very costly otherwise it is just sufficient to follow the guidelines in this implementation and you will get a feeling of how the CycleGAN works. Next I will apply the same for loop iterations over the epochs over the number of steps and I will train that discriminator for key steps and then alternating the generator, report the loss for each of them as you can see from here.

I've run it only for 10 epochs. I don't have hardware powerful enough to accommodate training four models, large models, of millions of training weights and parameters but that's pretty much how it works. Thank you and see you in the next lecture.

LECTURE 27

Working of Vid2Vid GAN

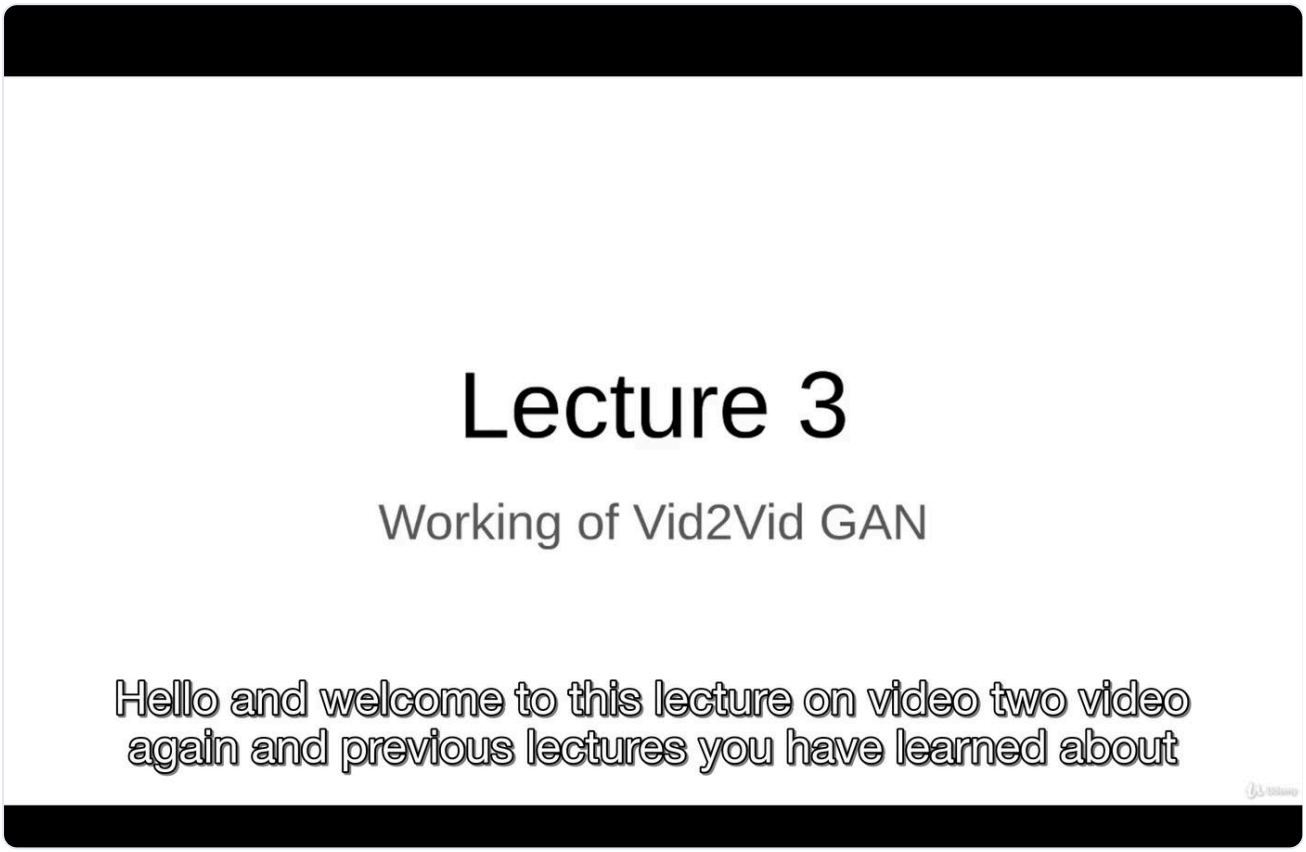


FIGURE 24.1 | Working of Vid2Vid GAN

Hello and welcome to this lecture on Vid2Vid GAN and previous lectures you have learned about image to image translation. with both Pix2Pix GANs and CycleGANs when we extend this model to video synthesis we start facing major issues which are inherent only to videos. The first is the spatio-temporal coherence problem. This describes how the videos maintain the locations of the objects at the sequential time frames models working with videos should be able to learn the dynamics of objects and their motion relative to each other.

The second is the resulting resolution the Vid2Vid GAN scored high on generating 2K resolution for 30 seconds long videos. Further we have to generate photorealistic videos in order to do that.

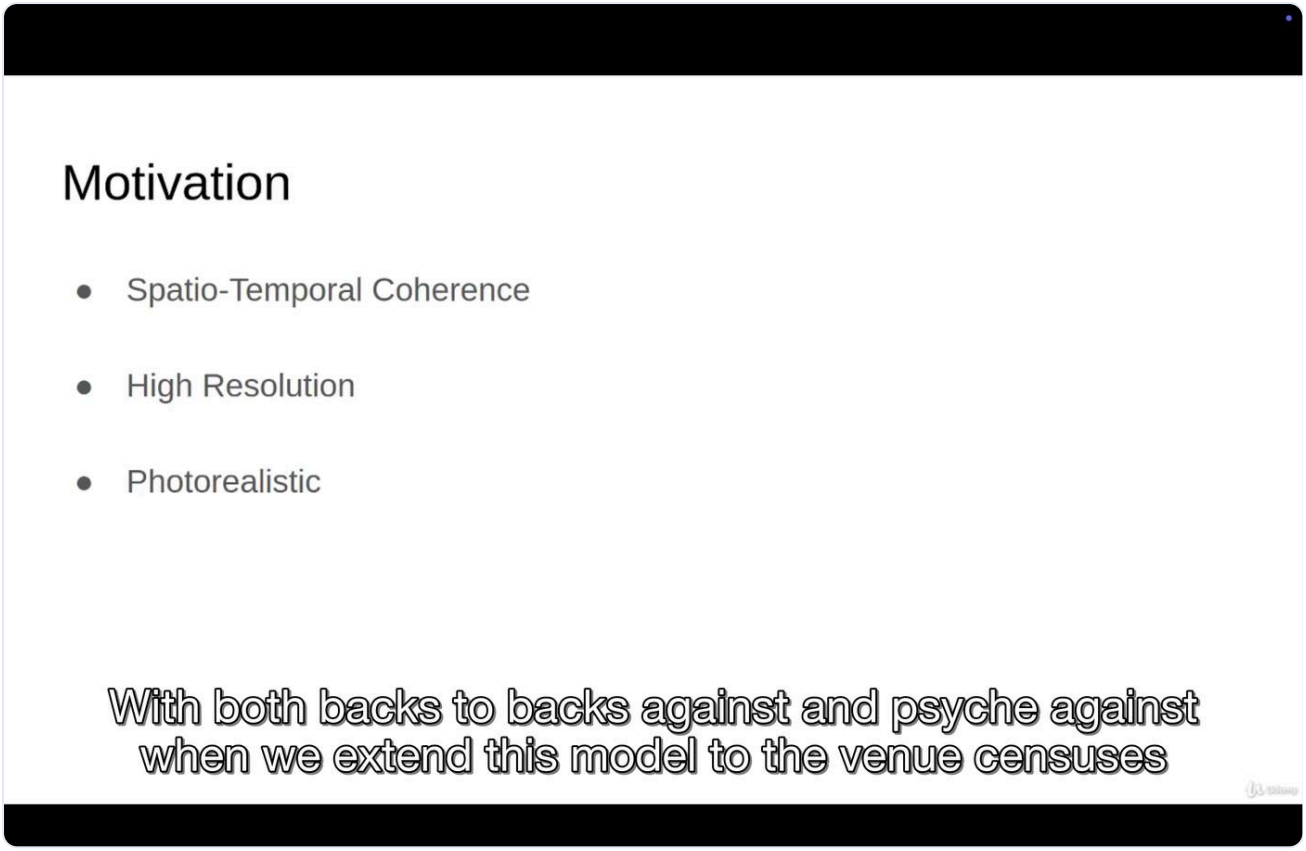


FIGURE 24.2

Teams should be working on texture lighting camera animations characters and scene geometry. Besides that we still need a powerful graphic card to do the scene rendering. This is an overkill and dilutes the value video synthesis brings to scientific and entertainment applications.

GANs address these issues with careful design of generators and discriminators and the choice of a meaningful objective function again objective functions demonstrate how important they are to model performance let us begin with the given input and the expected output from the Vid2Vid GAN. In this example the model is given a segmented sequence with labels.

FIGURE 24.3 | Input Labels

These labels could be trees road buildings or cars. Then the model generates realistic street views.

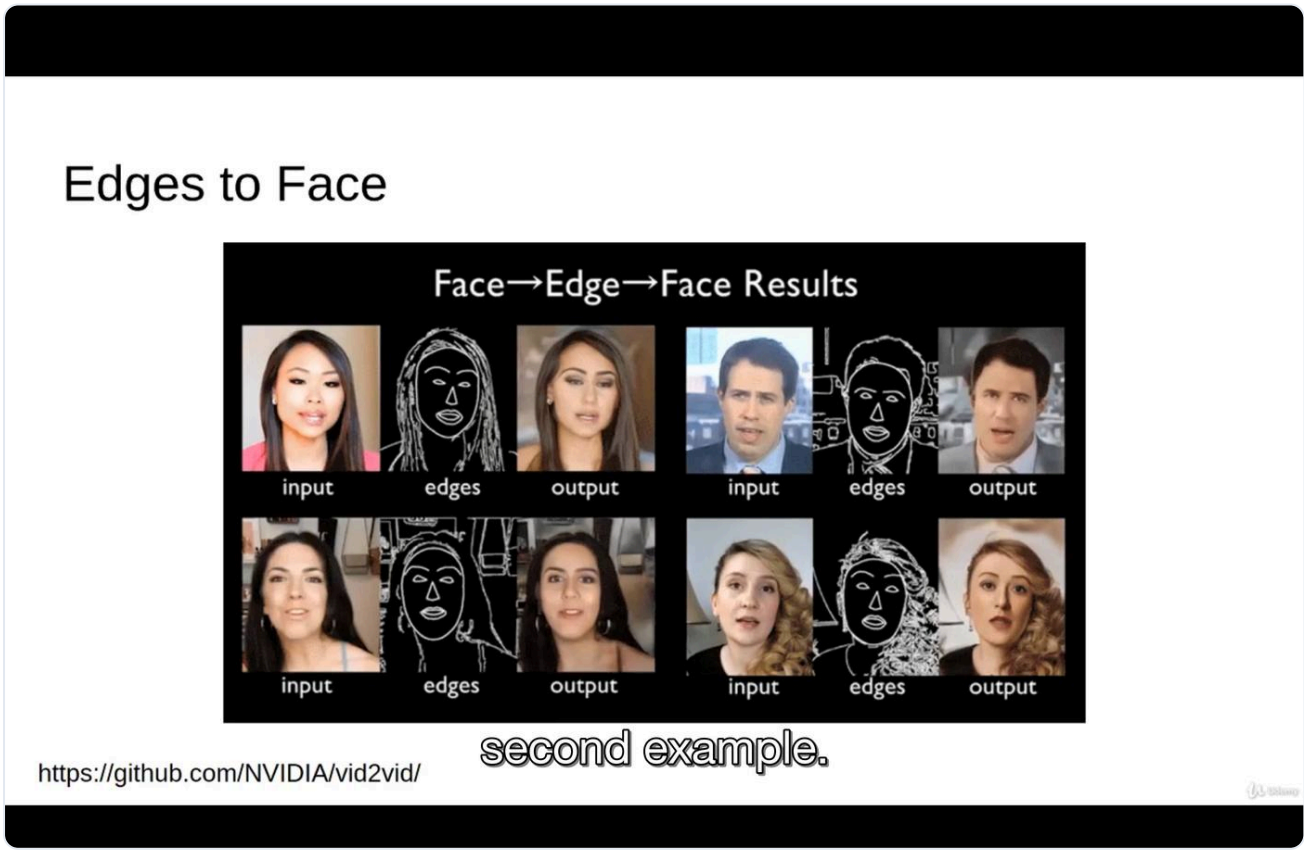


FIGURE 24.4 | Edges to Face

We can see that the model does quite well by generating various styles with a sense of randomness in the second example. The model is given a talking head the preprocessing extracts the edges and face landmarks.

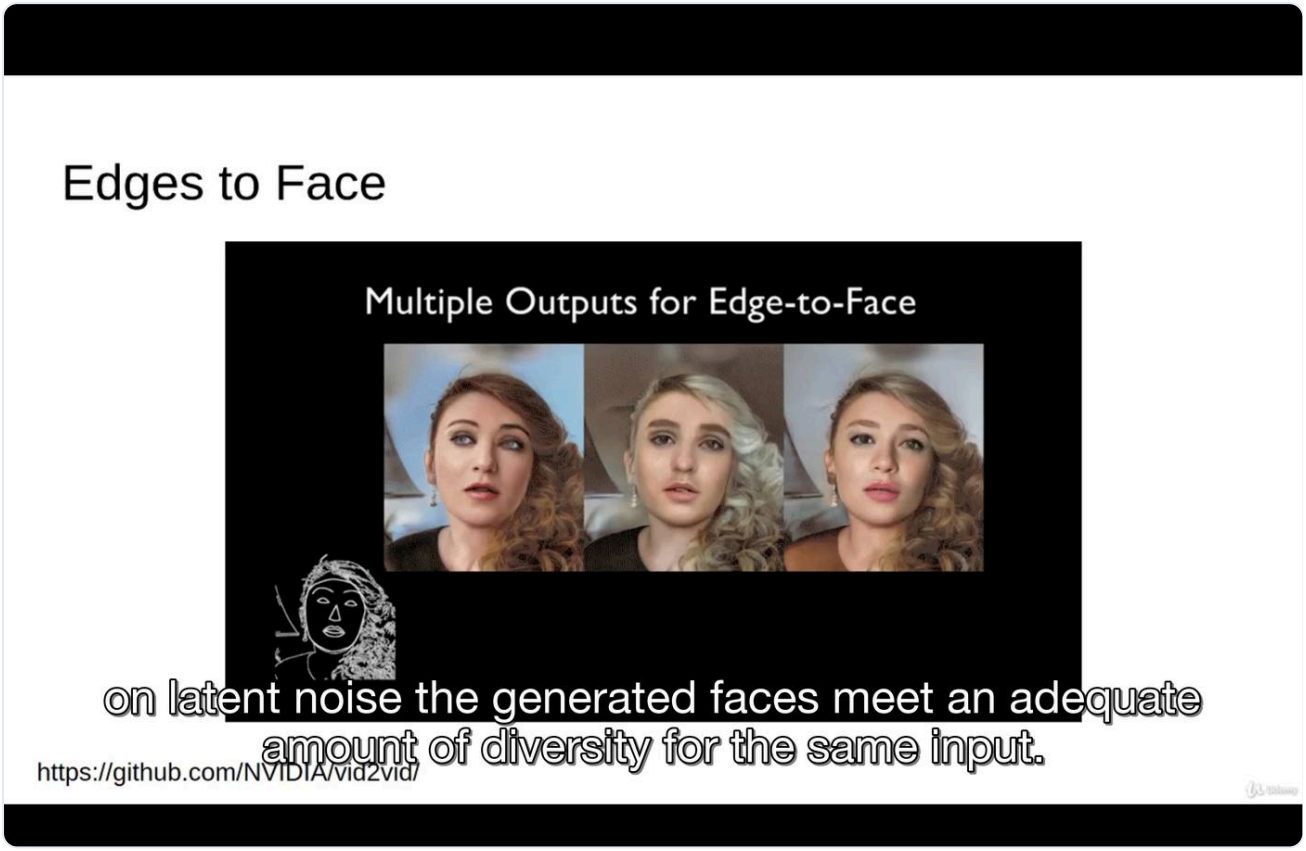


FIGURE 24.5 | Edges to Face

Then the model generates a different face maintaining the lips motion and facial emotions again based on latent noise the generated faces meet an adequate amount of diversity for the same input.

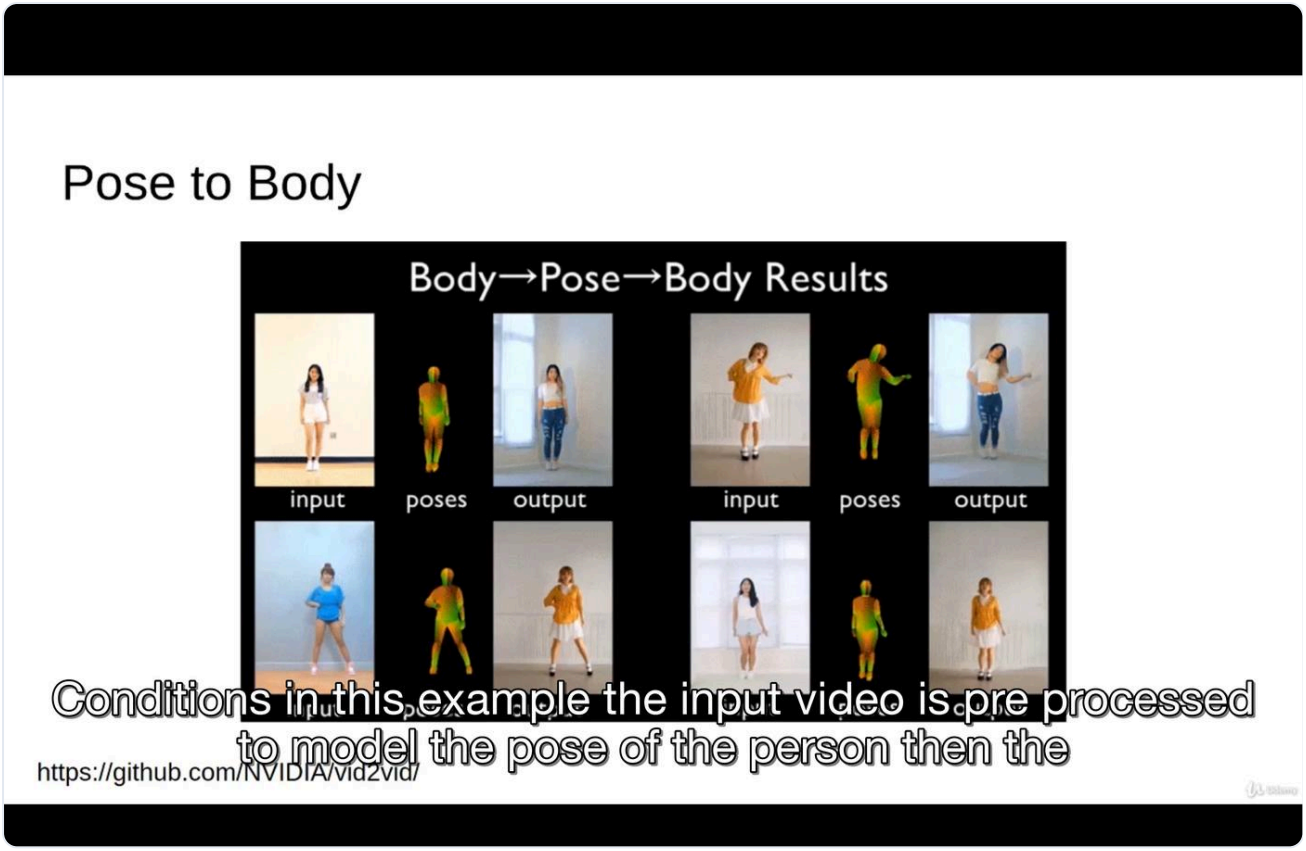


FIGURE 24.6

Conditions in this example the input video is pre processed to model the pose of the person then the generator produces different bodies with the same pose in the last example.

The model is given a frame sequence then its job is to predict the next frame. There could be many other applications for video to video synthesis and these applications could be in other areas related to science commerce and entertainment industry Vid2Vid GAN is based on an idea from a previous paper.

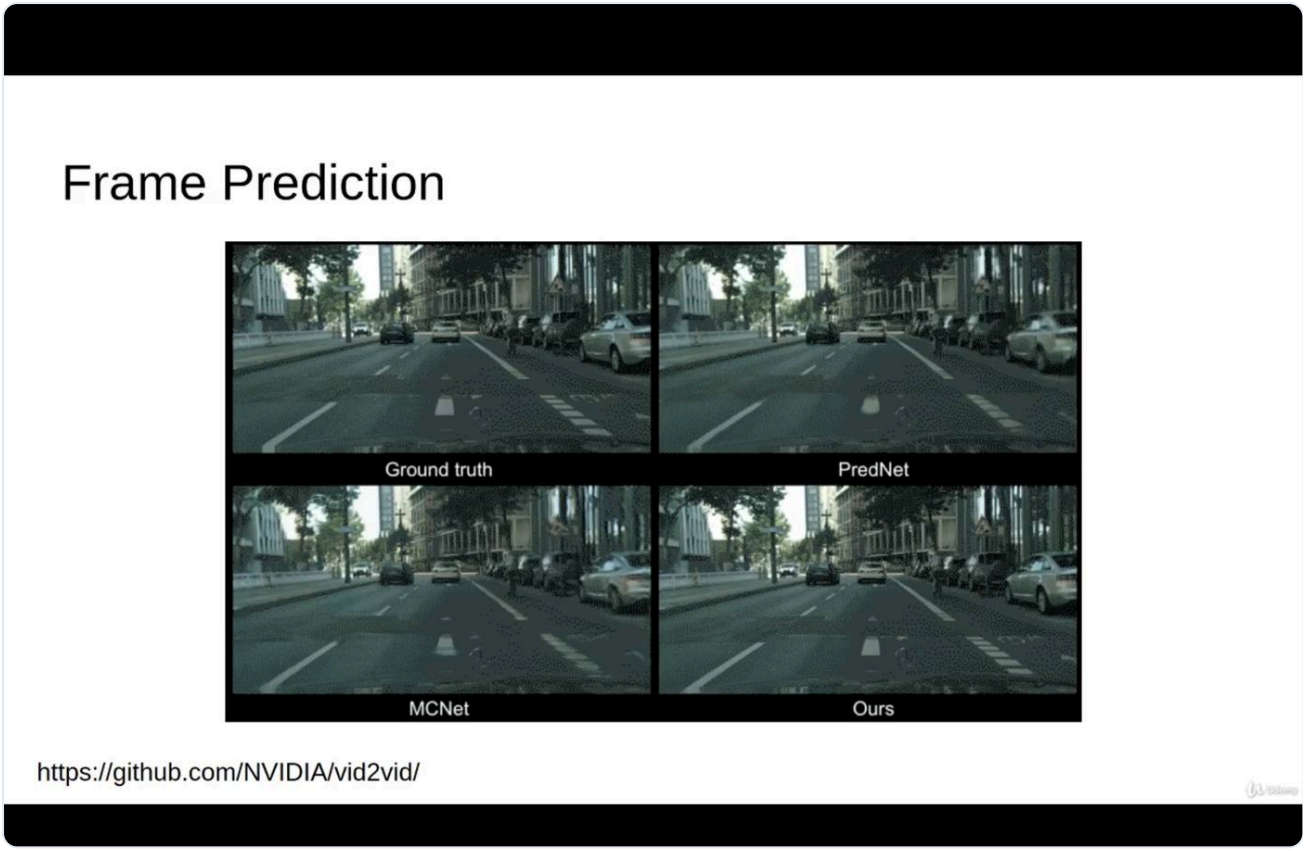


FIGURE 24.7 | Ground Truth

The coarse-to-fine GAN is a composite of two generator networks G1 and G2 G1 is trained to generate low resolution images. Having arrived at a satisfactory result then the G2 network is appended and is trained on high resolution images.

The last feature map from the G1 network and the first feature map from the G2 network are summed together and fed together to the rest of the G2 network.

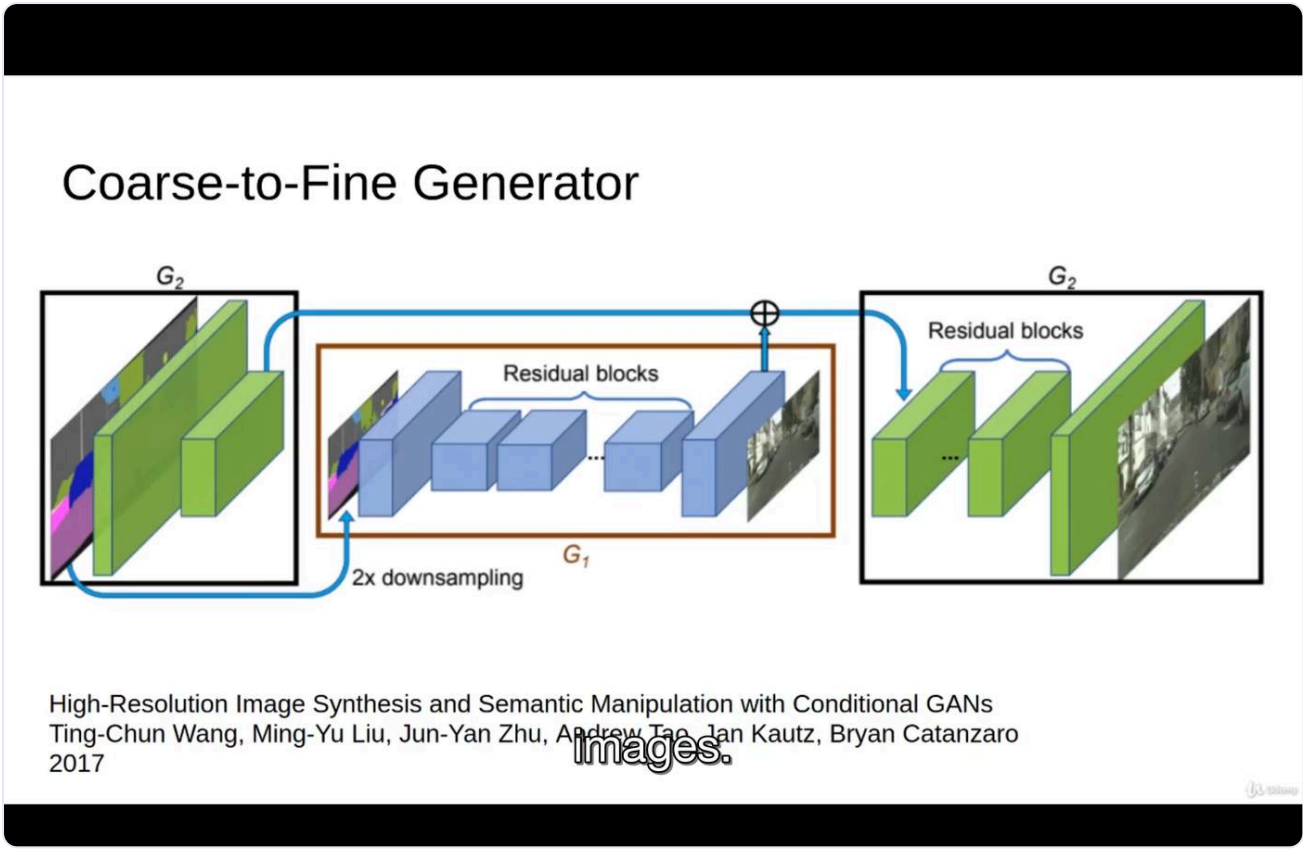


FIGURE 24.8

The Vid2Vid GAN makes use of two discriminators. The first discriminator is a regular image GAN discriminator. It outputs whether an image pair is real or fake.

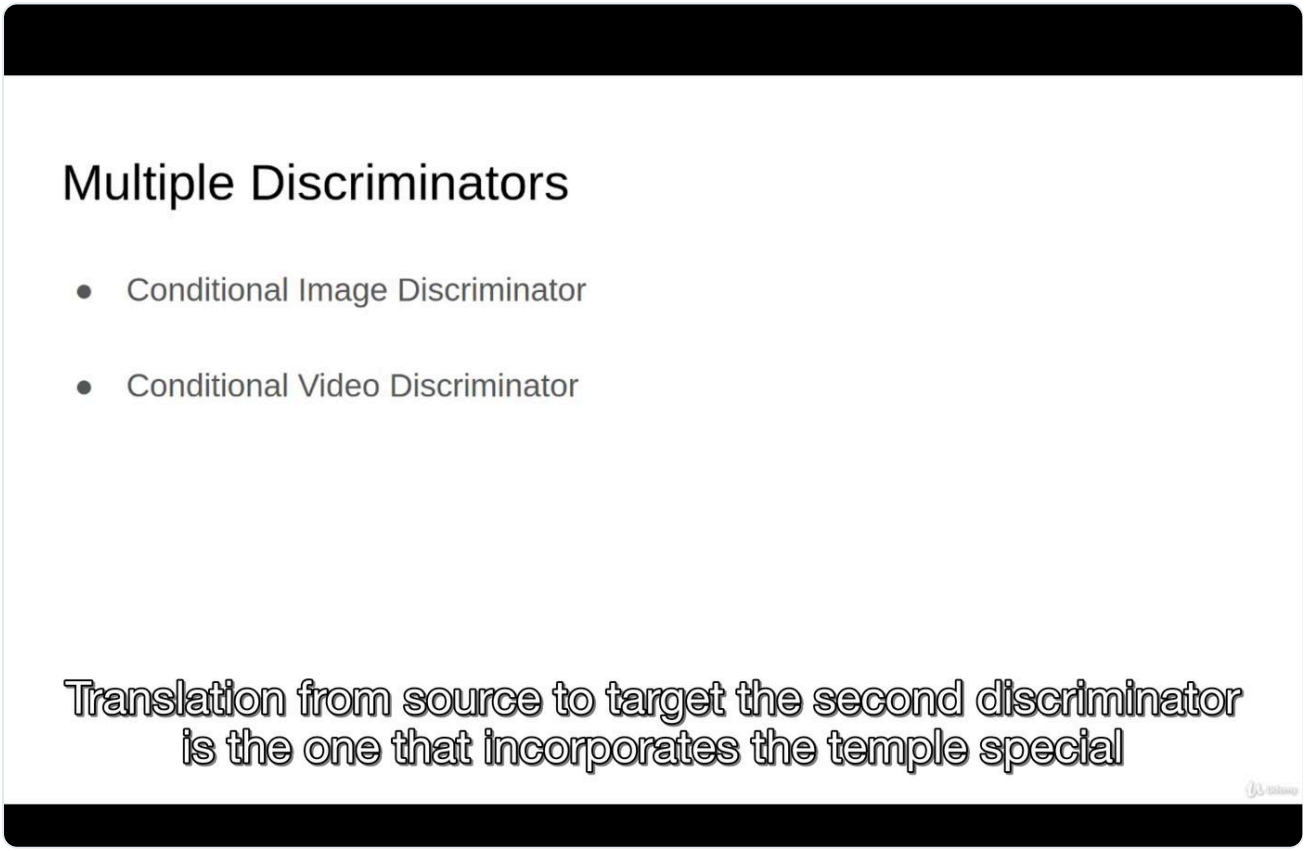


FIGURE 24.9

Remember we are breaking the video into a sequence of frames and then applying image translation from source to target the second discriminator is the one that incorporates temporal and spatial coherence.

It uses displacement information which is extracted with a method called optical flow. So what is optical flow. It is a well-established technique from digital image processing given a sequence of images a video.

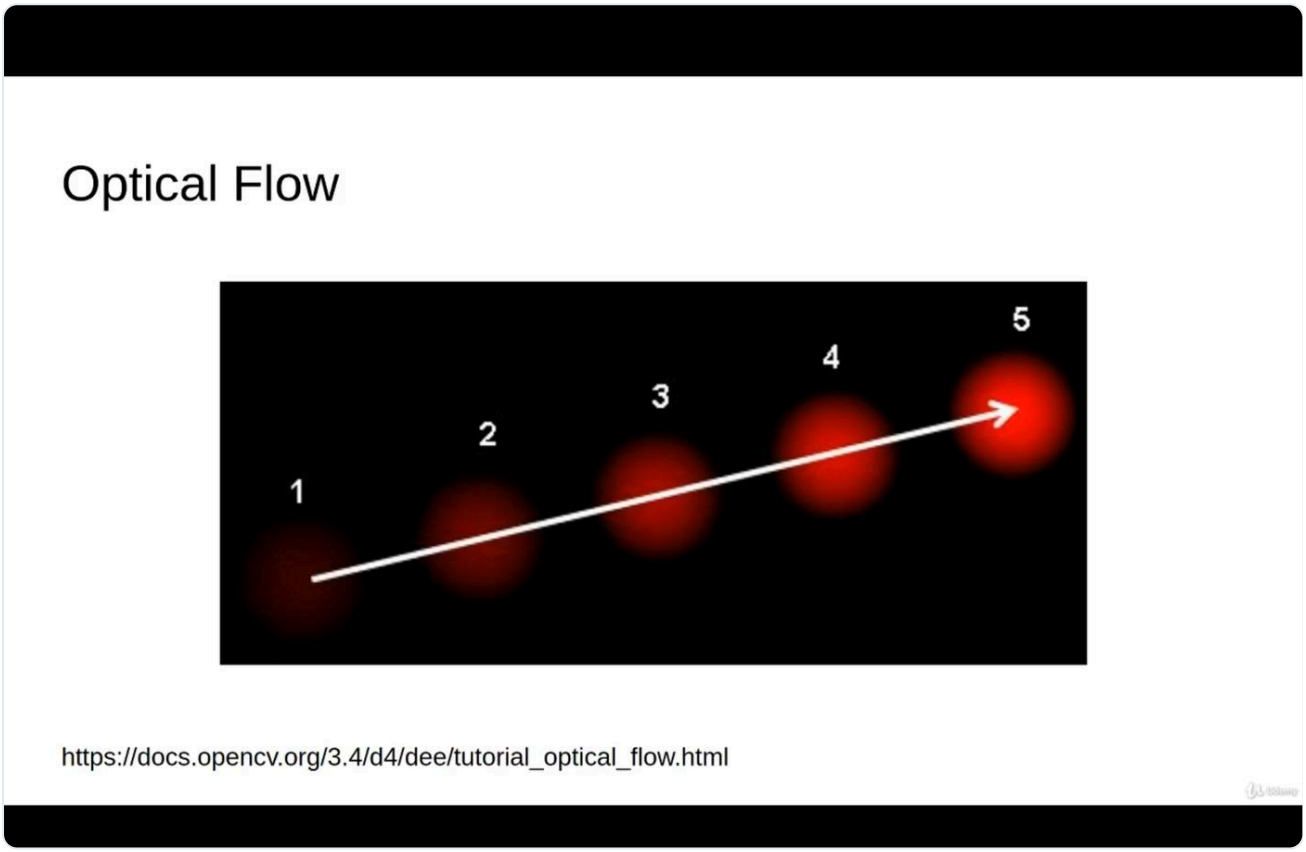


FIGURE 24.10

Then it can calculate both magnitude and direction for moving objects in order to achieve that. It solves the gradient equation over a local neighborhood of the pixel. It is also based on the assumption that moving objects do not tend to change intensity as in this example.

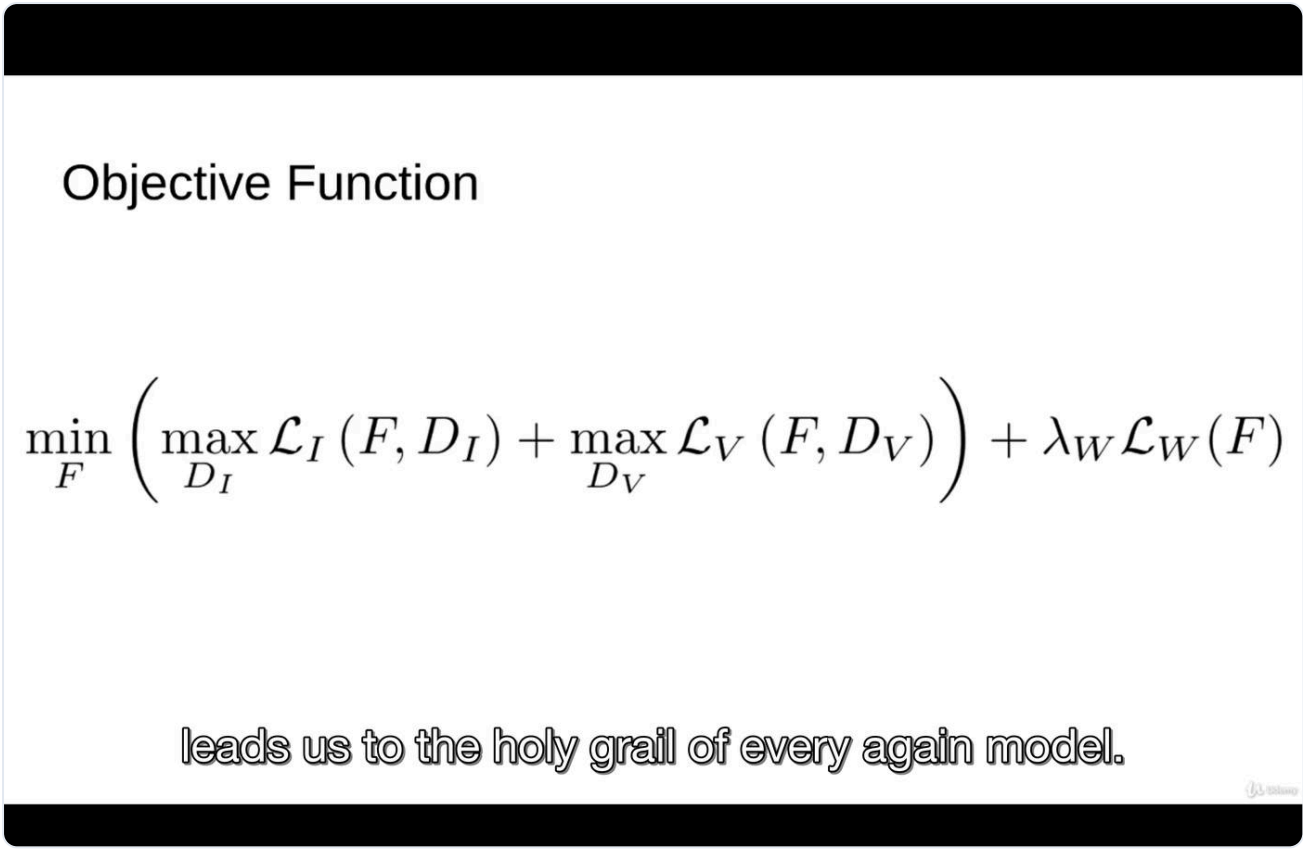


FIGURE 24.11 | Objective Function

We can observe the motion of the circular object at time t equals one up to time t equals five so this leads us to the holy grail of every GAN model.

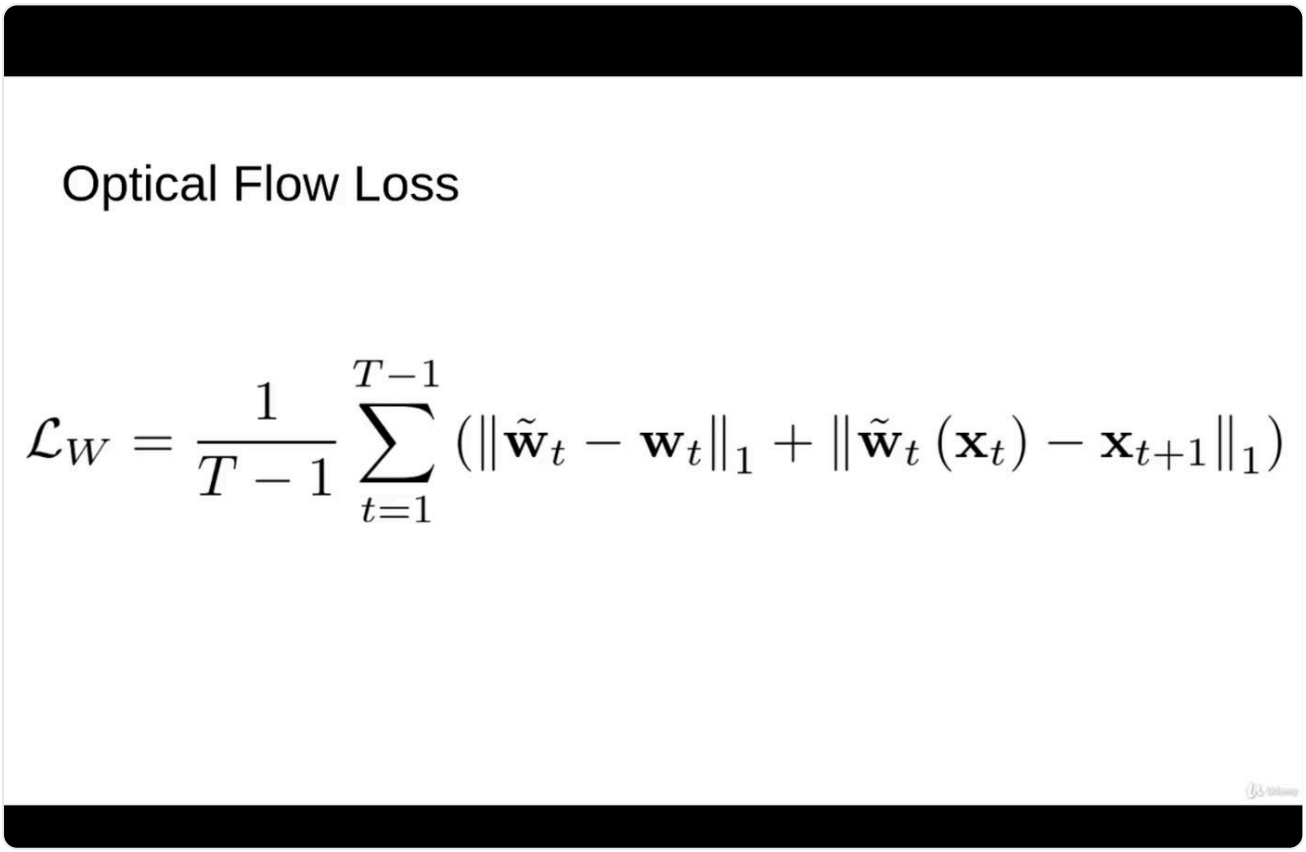


FIGURE 24.12 | Optical Flow Loss

The objective function the objective function here consists of three terms L1 stands for the image pair translation loss, it is a regular GAN loss L2 stands for the video sequence temporal and spatial coherence loss which is again a standard GAN loss of K images the loss term L of W is the optical flow loss note here again that the Greek letter lambda stands for a weight factor for these loss terms So how does the network learn to calculate the optical flow it is a two term sum over count i.e.

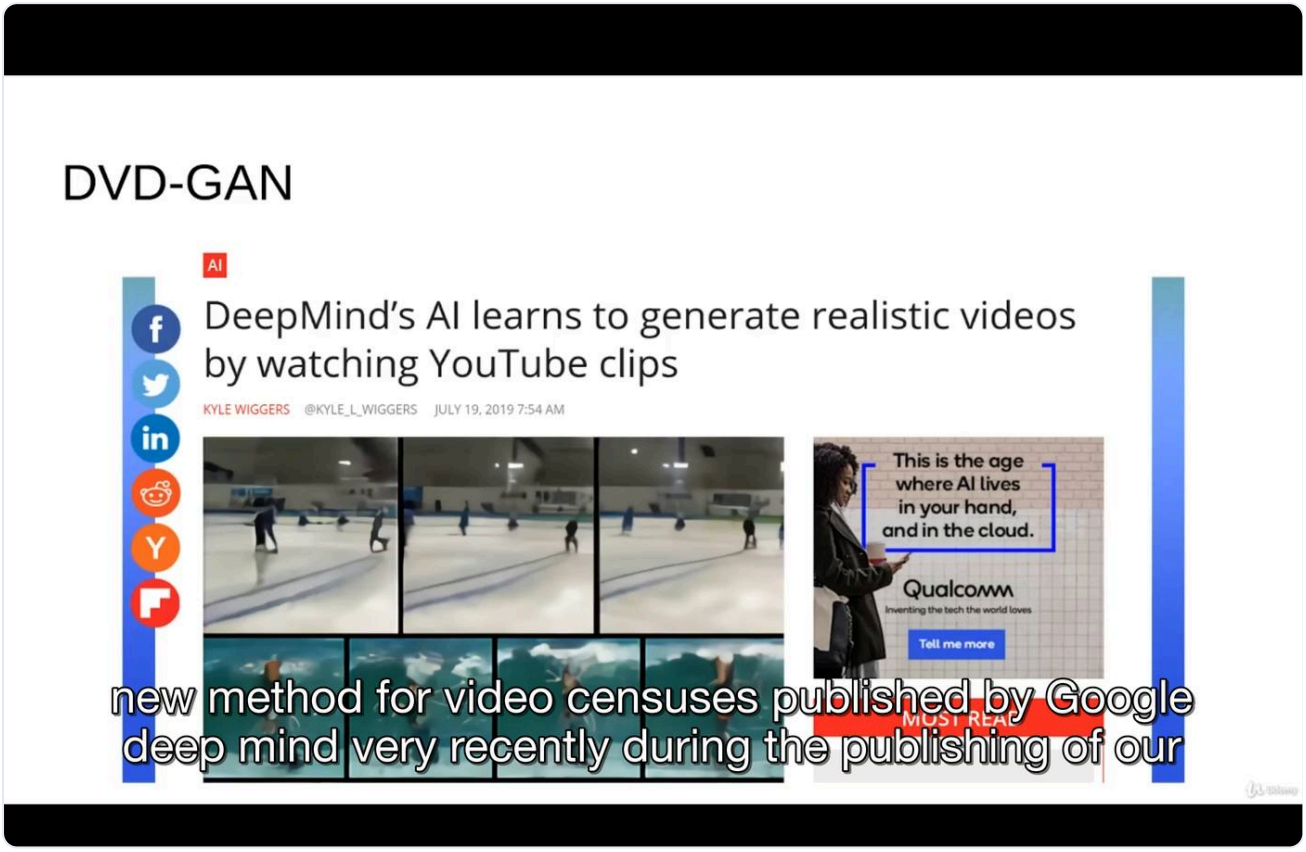


FIGURE 24.13

average of two terms for a given sequence of image frames the first term is the L1 norm of the ground truth and the estimated optical flow the second term is the displacement of a pixel between subsequent image frames as a bonus topic you may be interested in learning more about DVD-GAN which is a breakthrough new method for video synthesis published by Google DeepMind very recently during the publishing of our course in the next lecture I shall walk you through the source code for Vid2Vid GAN thank you and see you in the next lecture.

LECTURE 28

Reading Assignment

https://github.com/nvidia/vid2vid

Diving Deeper into Vid2Vid GAN using YouTube Dance Video Dataset

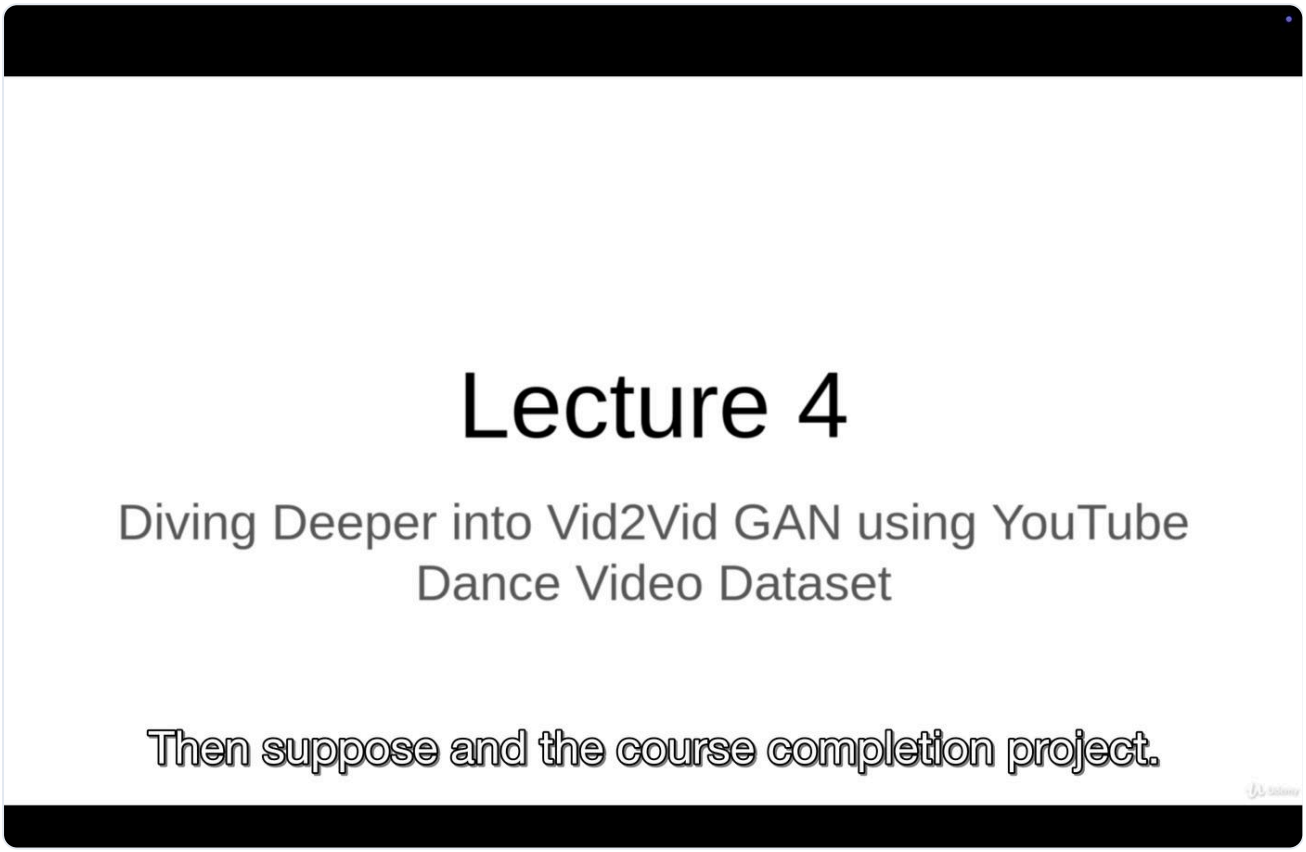


FIGURE 25.1 | Diving Deeper into Vid2Vid GAN Using YouTube

Hello and welcome to a new lecture. In this lecture I shall introduce the FlowNet2 architecture, OpenPose, DensePose and the course completion project. Let's get started.

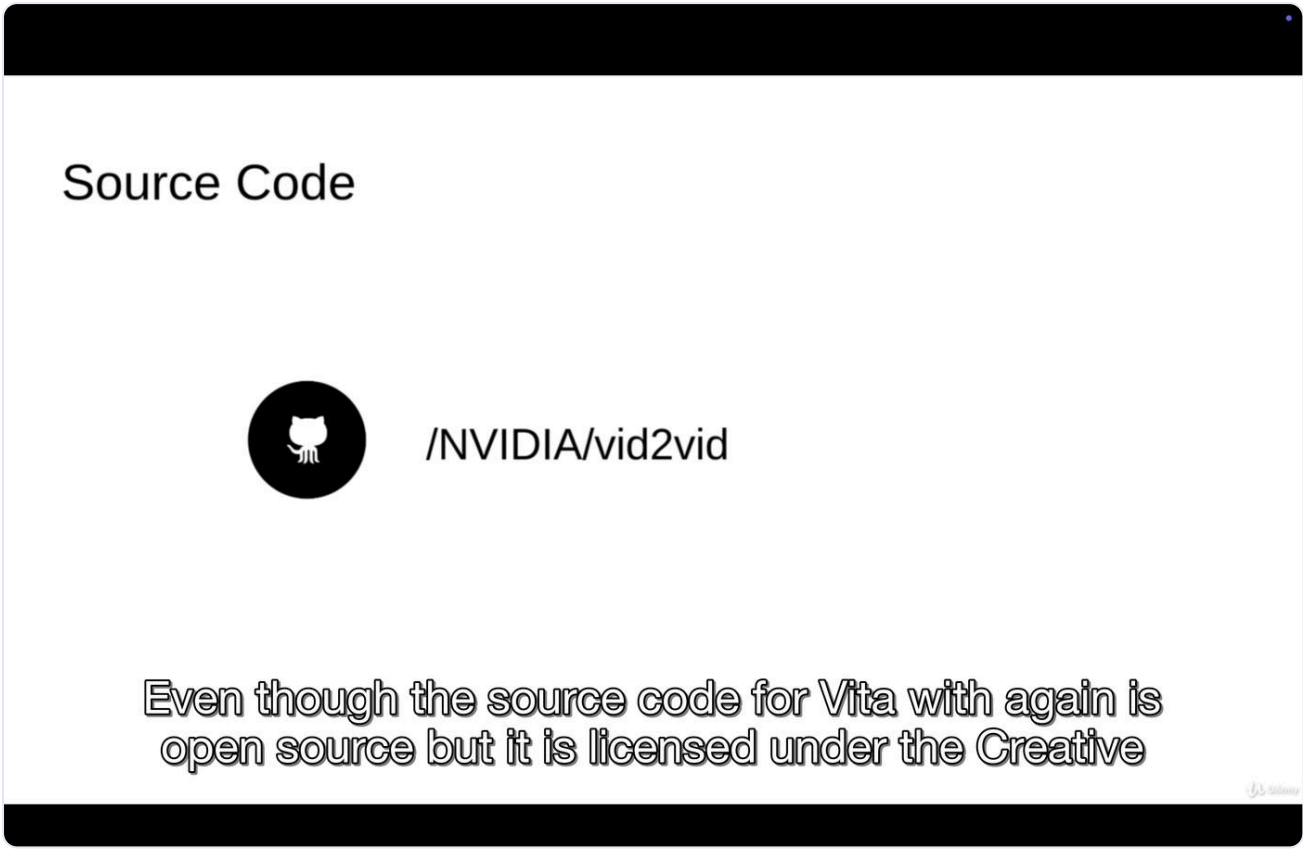


FIGURE 25.2 | Source Code

Even though the source code for Vid2Vid GAN is open source but it is licensed under the Creative Commons. BY-NC-SA license this doesn't allow us to use the original source code in this course material. I would recommend that you find the source code using the link displayed on the screen on github.

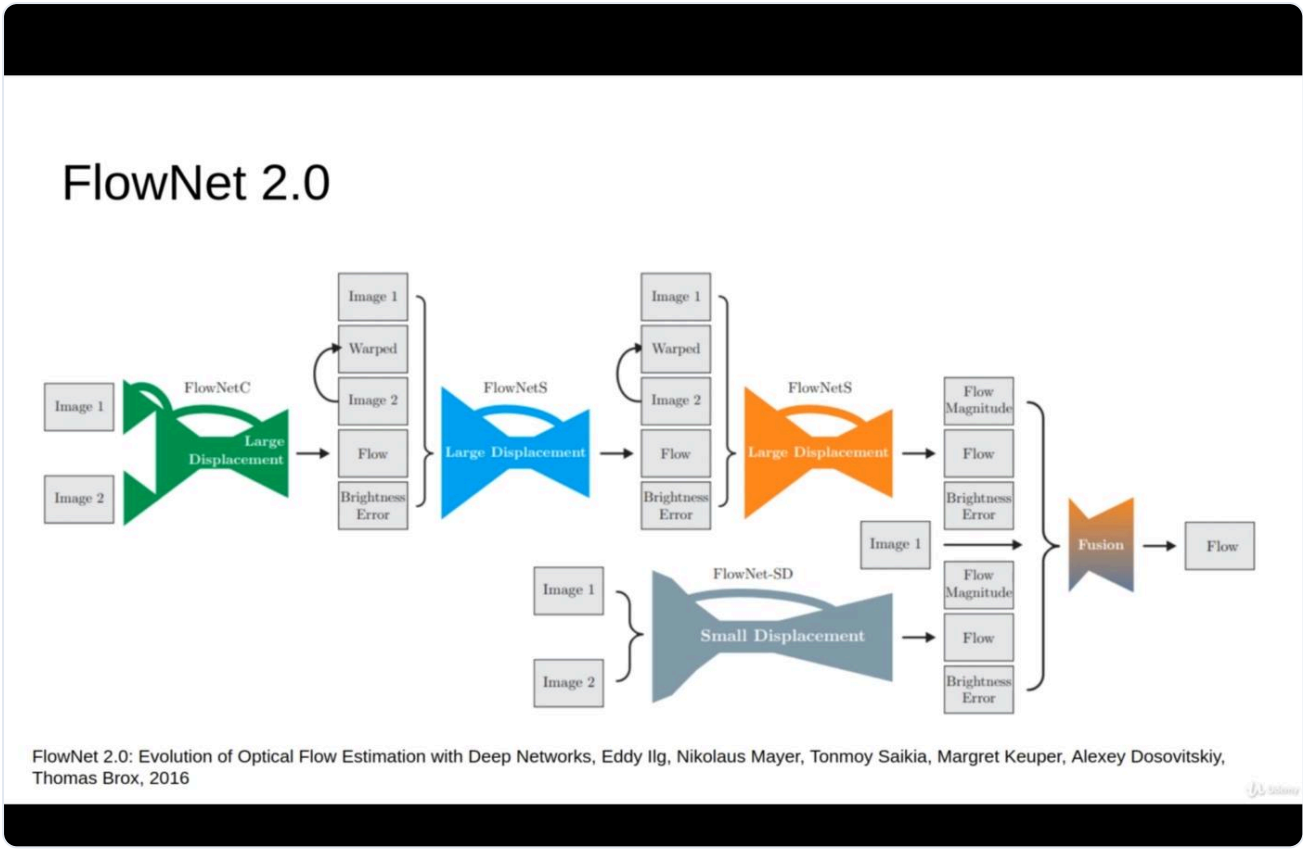


FIGURE 25.3

You have learned many basic building blocks that allow you to easily comprehend the method and its implementation in order to aid you in understanding the Vid2Vid GAN. I will introduce the FlowNet2 architecture. This is an essential component in Vid2Vid GAN which calculates the optical flow between video frames.

FIGURE 25.4

Remember how the optical flow is included in the objective function to ensure spatio-temporal coherence. The network is an improvement over earlier versions.

It takes a pair of images, then warps the images, then calculates the brightness difference and then passes them through multiple upsampling convolutions until it arrives to a fusion network that merges the results and amplifies the boundaries of moving objects the results obtained from FlowNet2 are heat maps which represent optical flow in terms of magnitude and direction for every pixel. FlowNet2 is apparently superior to previous architectures.

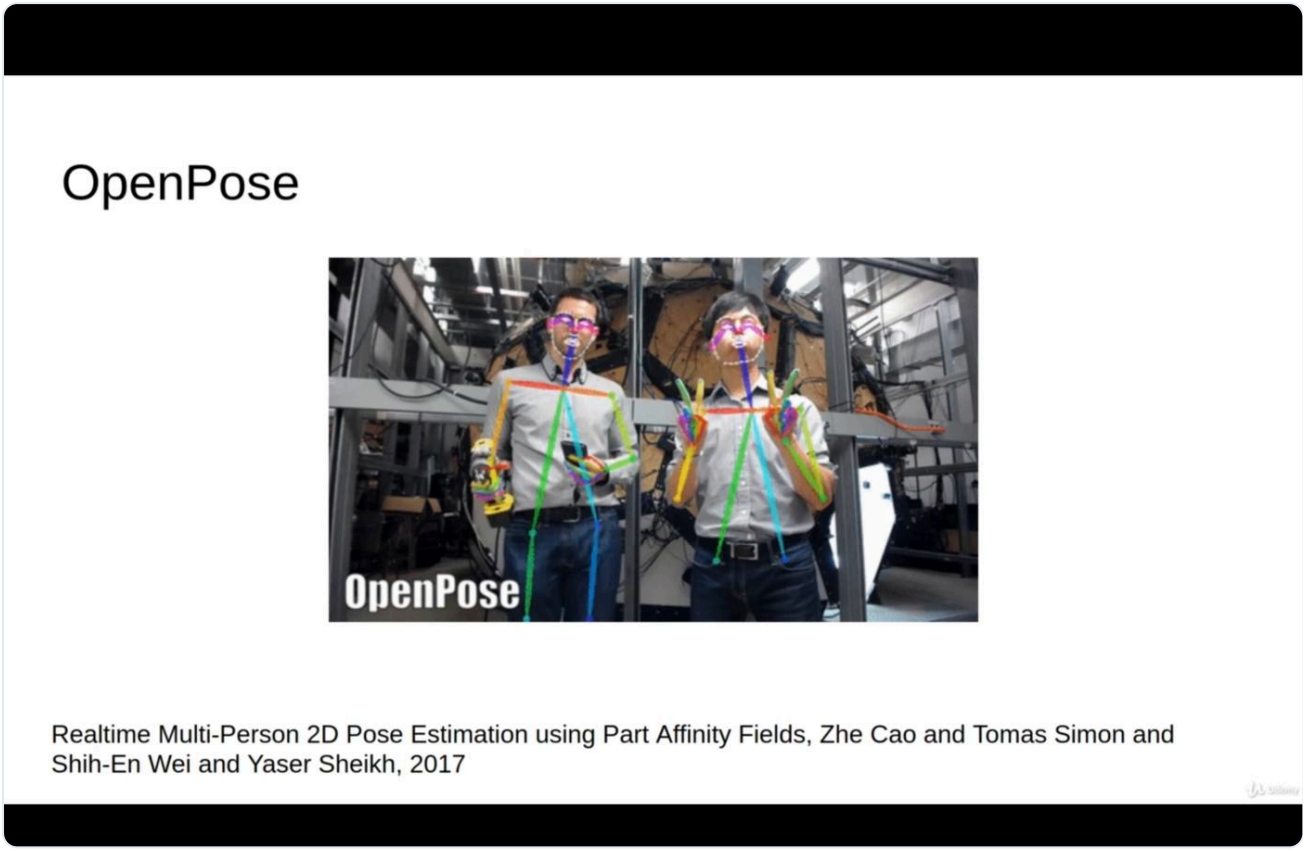


FIGURE 25.5

Now let's talk about another building block that you should encounter in the Vid2Vid GAN. Remember from previous lecture that Vid2Vid GAN is applicable to YouTube dance dataset dance videos are randomly sampled from YouTube and then downloaded then the videos are given to a specialized model to estimate the human pose in Vid2Vid GAN, or source code, it is mentioned that you can use either OpenPose or DensePose. OpenPose estimates in real time.

FIGURE 25.6

The body the hands the feet and the head pose then the resulting pose dataset is given to the Vid2Vid GAN for generation. On the other hand it is also possible to use DensePose which is published by Facebook Research. It goes even further to estimate the pose in 3D coordinates for every pixel of the human body.

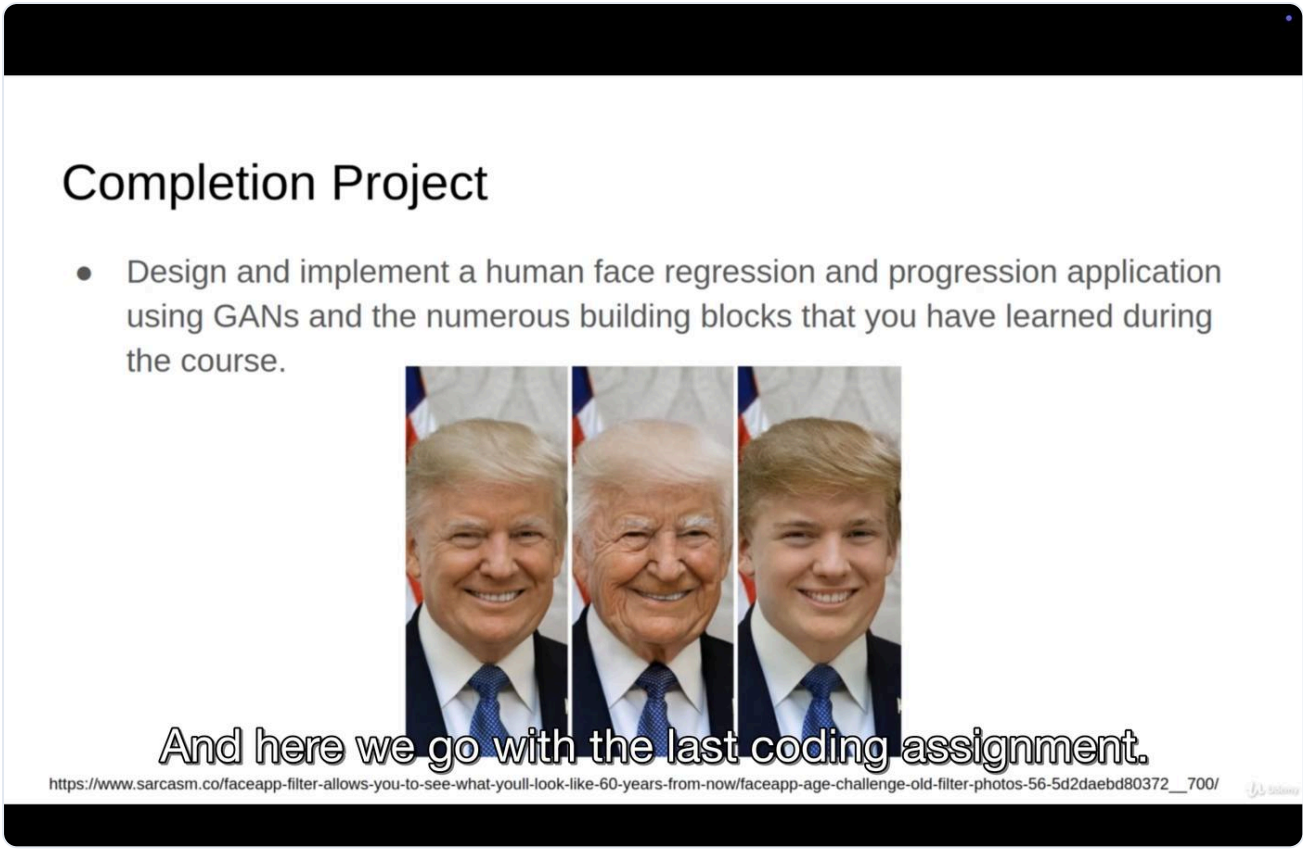


FIGURE 25.7 | Completion Project

And here we go with the last coding assignment. In this course the approach I have followed in this course is to organize the course around basic building blocks modular concepts that you can stack together in whatever order you think is right for your application.

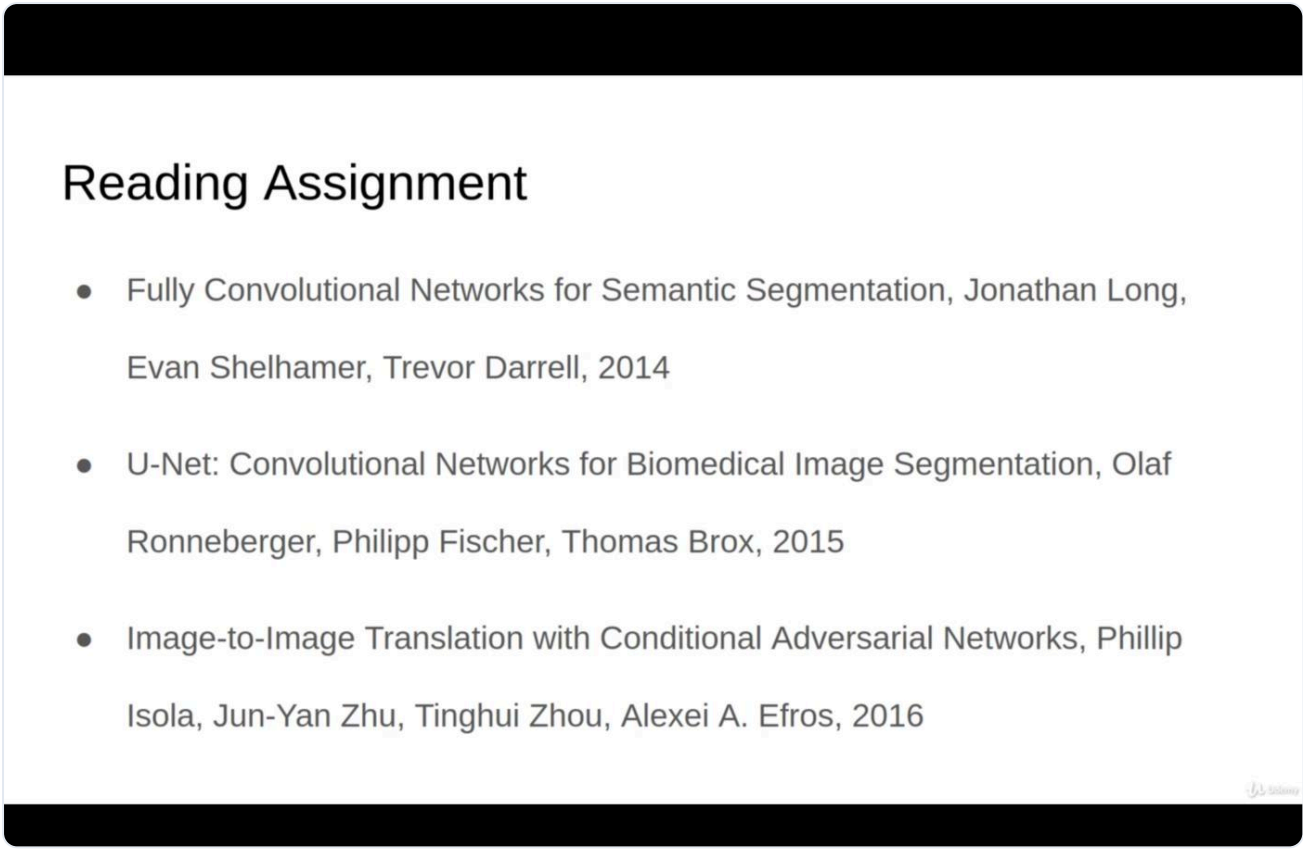


FIGURE 25.8 | Reading Assignment

So I am confident that you will find every GAN architecture in existence no matter how complex it is to be built around the same building blocks that you have mastered in this course in order to test your understanding of GANs. I propose the following completion project but first let's define age regression and age progression age regression is when you try to generate a human face.

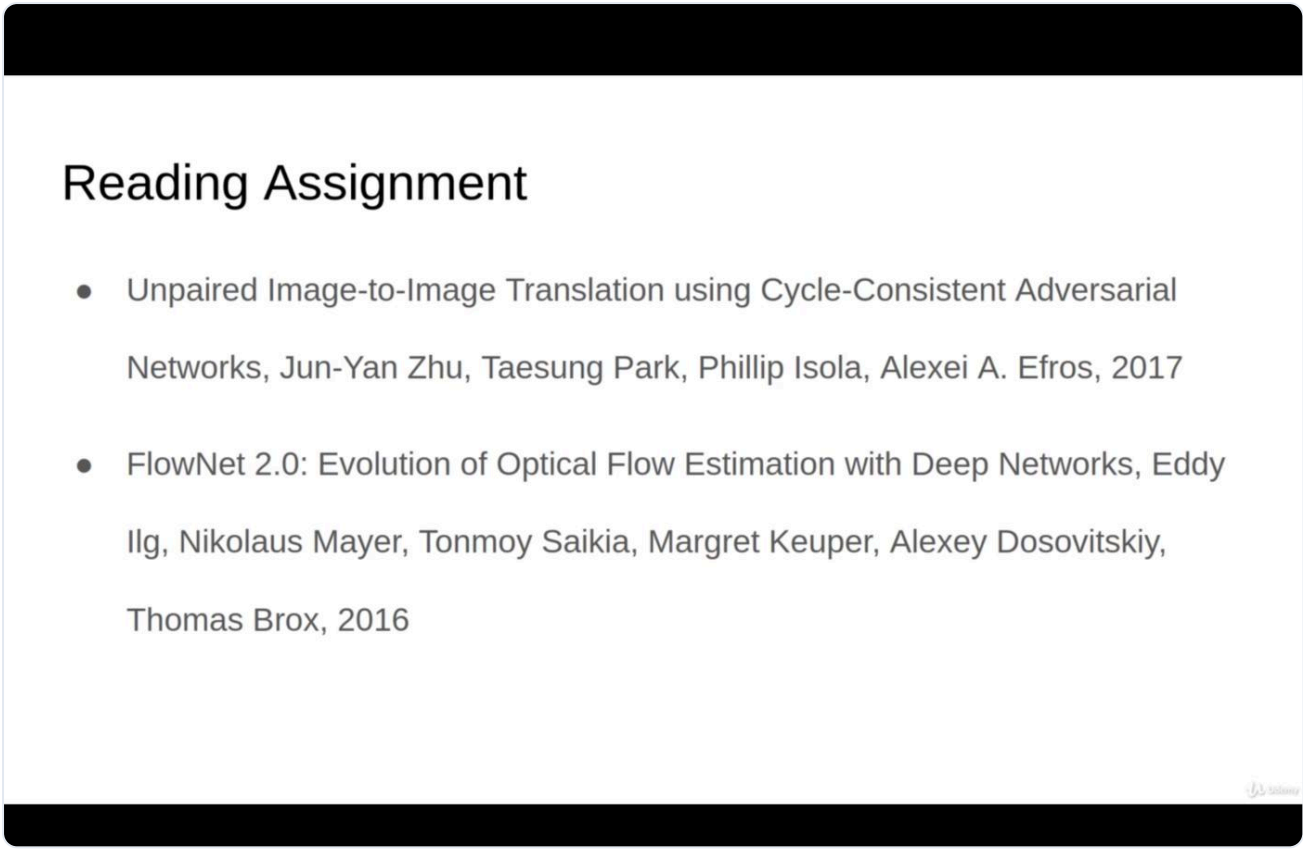


FIGURE 25.9 | Reading Assignment

Given an image of an older face on the other hand age progression is when you try to generate a human face given an image of a younger face so I would like you to implement an application either desktop web or mobile or even on a Raspberry Pi or a notebook.

That is when given a human face generates either a younger or an older image of the same person as a hint for you.

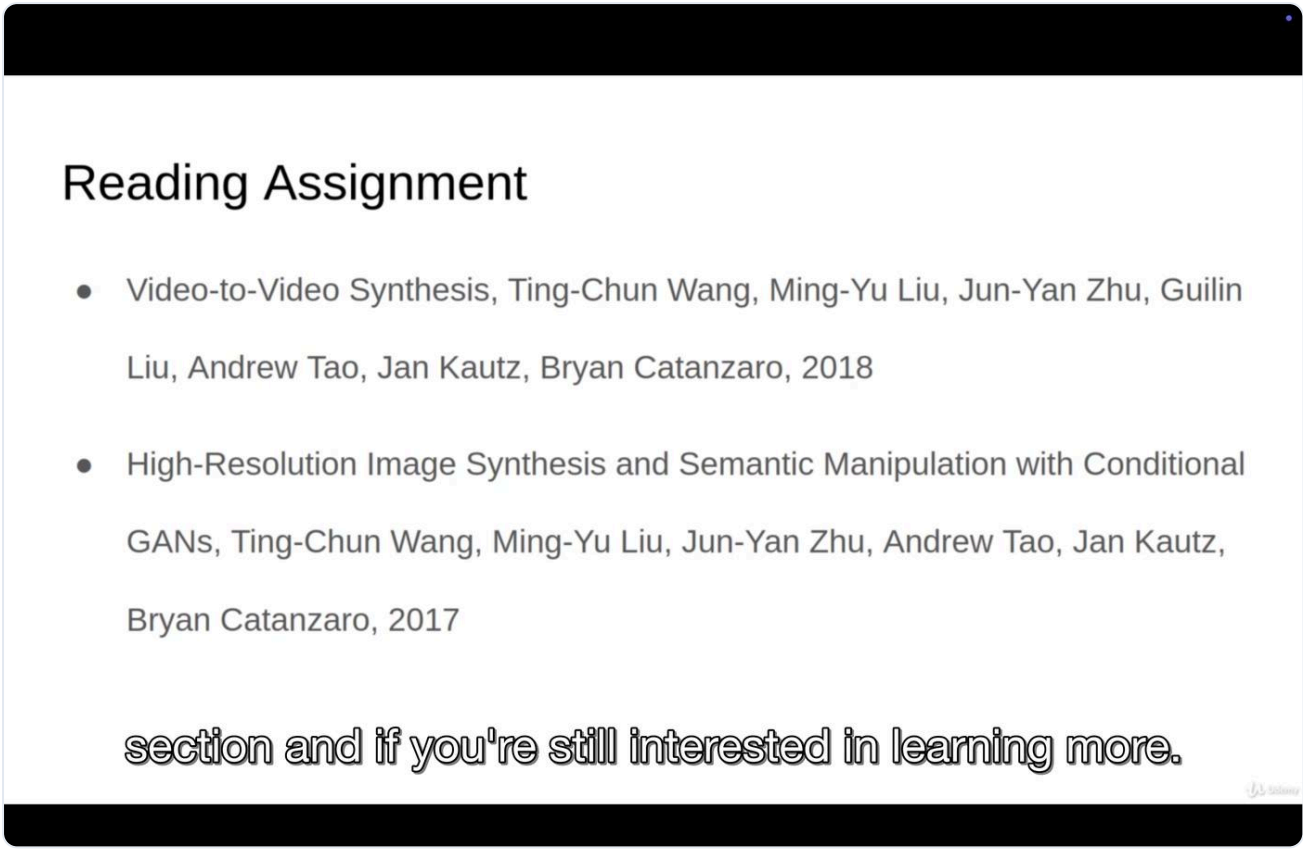


FIGURE 25.10 | Reading Assignment

You may use Age-GAN and read more about identity preservation in human face generation and here you go with the first page of the reading assignment for this section and here you are a few more resources which is the second page of the reading assignment for this section and if you're still interested in learning more. Here you go with the third page of the reading assignment for this section.

Thank you. See you in the next lecture.

LECTURE 29

Conclusion, Next Steps, and Future Directions

Hello and welcome to the last review of our course. In this video I will share with you a few tips hoping that it may help you in your future career path. Festina lente is a Latin expression which means to be both fast and slow at the same time. Learning is a long process. It takes patience sweat and long nights. It pays off when you reach your goals.

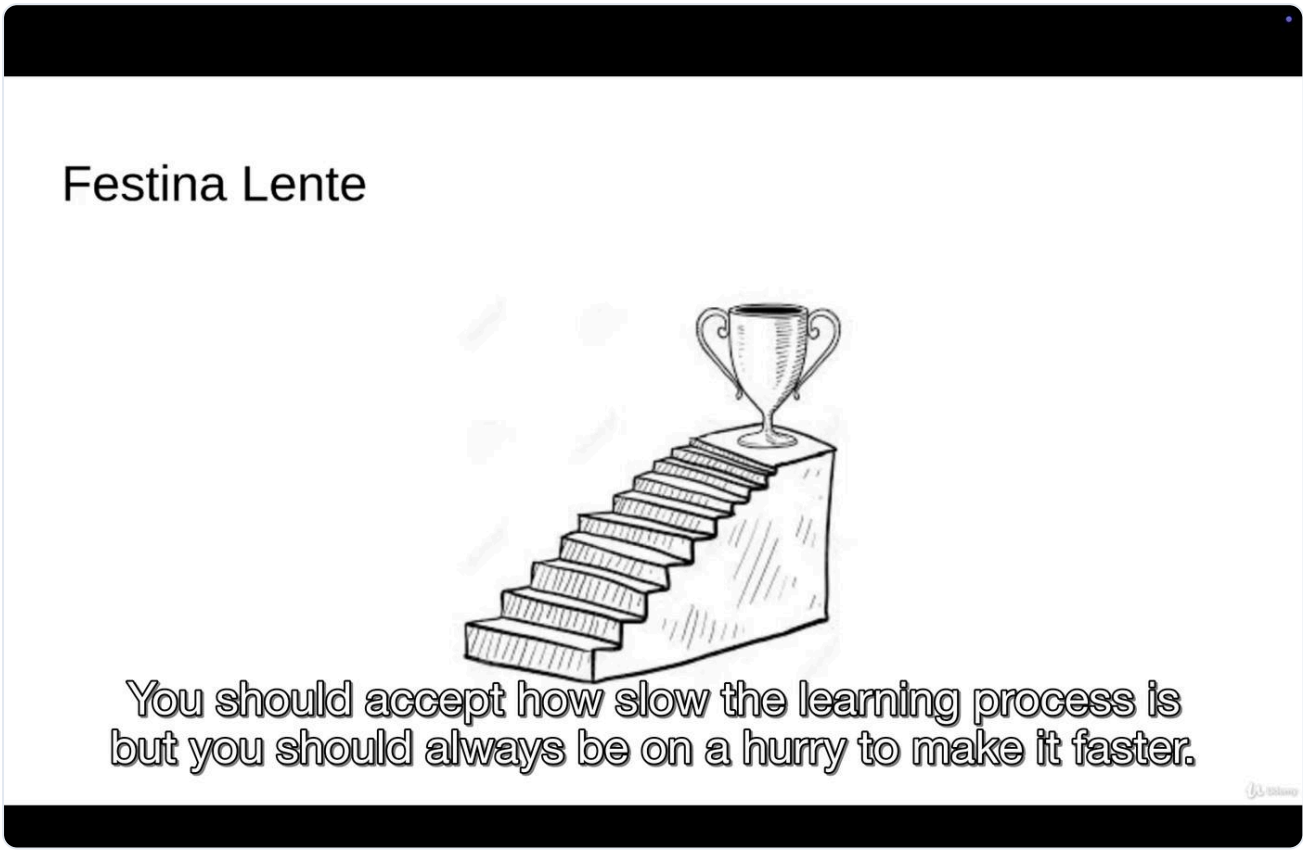


FIGURE 26.1 | Festina Lente

You should accept how slow the learning process is but you should always be in a hurry to make it faster.

This is a contradiction but in the paradox lies the value you learn by doing 70 percent of your knowledge comes from education courses training and so on. 20 percent comes from your hands on experience on projects hobby projects school projects and real life projects.

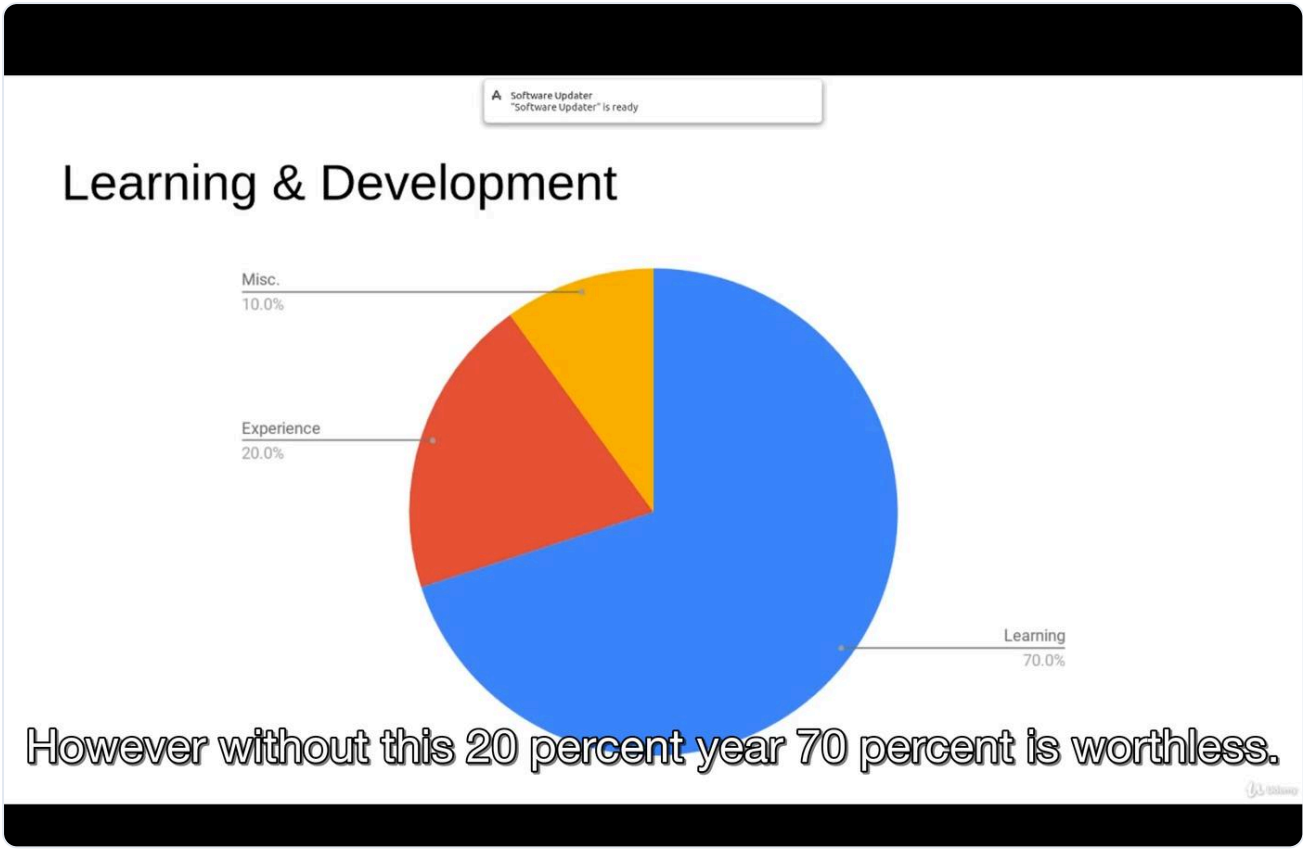


FIGURE 26.2

However without this 20 percent your 70 percent is worthless. So be willing to invest 80 percent of your time and effort into getting your hands dirty and learn by doing the other 10 percent comes from discussions.

Meetups, conferences and webinars so don't miss out on that and also don't over invest in such events. I see people who spend 100 percent of their time in such events be objective and to the point always set clear life goals. Think of the problem and work your way through backward to the solution be entrepreneurial solve problems build solutions and who knows maybe you are the next unicorn. It is not only about learning knowledge and experience we tend to miss out completely on a very essential factor of the success equation.

So be creative be innovative and invent things.

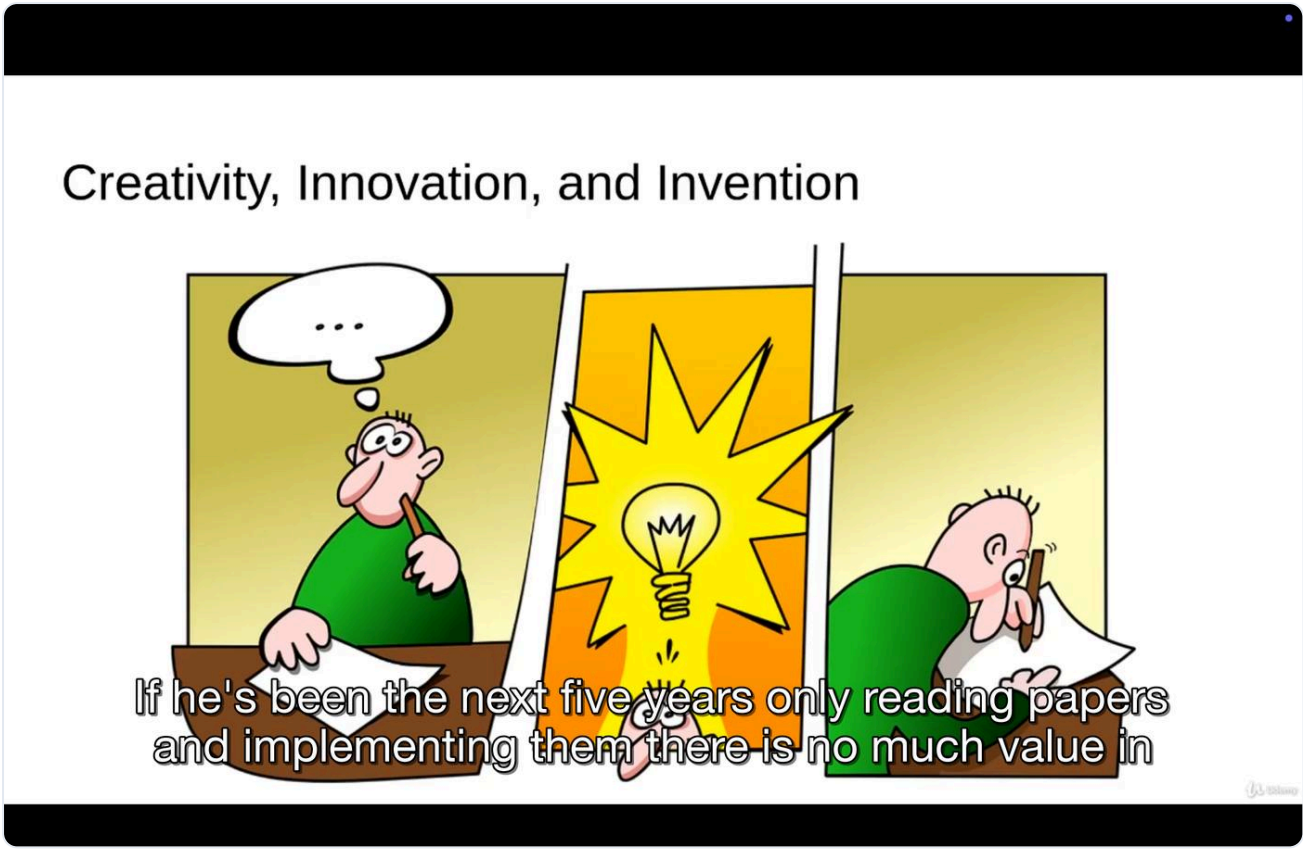


FIGURE 26.3

If you spend the next five years only reading papers and implementing them there is not much value in that. The papers always come with open source code but always ask yourself what is missing. How it could become better. How is it applicable to other problems. Be curious challenges competitions and contests tend to be the right way to go. Always Compete in online and offline challenges. Kaggle is one example but there are many more artificial intelligence challenges so go on and take part. At the end of this course.

I hope I have managed to help you understand GANs and give you some ideas to boost your future. I also hope that you have enjoyed the course. I wish you all the best and see you in future courses.